



面向 21 世纪 课程教材
Textbook Series for 21st Century

计算机组成原理

唐朔飞 编著



高等教育出版社
HIGHER EDUCATION PRESS

计算机组成原理
唐朔飞 编著

301
8

出版社

面向 21 世纪课程教材
Textbook Series for 21st Century

计算机组成原理

唐朔飞 编著



高等教育出版社
HIGHER EDUCATION PRESS

图书在版编目(CIP)数据

计算机组成原理/唐朔飞编著. —北京:高等教育出版社,2000.7

面向 21 世纪课程教材

ISBN 7-04-007927-5

I. 计... II. 唐... III. 电子计算机-理论-教材

IV. TP301

中国版本图书馆 CIP 数据核字 (2000) 第 62398 号

计算机组成原理

唐朔飞 编著

出版发行 高等教育出版社

社 址 北京市东城区沙滩后街 55 号

邮政编码 100009

电 话 010-64054588

传 真 010-64014048

网 址 <http://www.hep.edu.cn>

经 销 新华书店北京发行所

印 刷 中国科学院印刷厂

开 本 787×960 1/16

版 次 2000 年 7 月第 1 版

印 张 26.75

印 次 2000 年 12 月第 2 次印刷

字 数 490 000

定 价 22.50 元

本书如有缺页、倒页、脱页等质量问题,请到所购图书销售部门联系调换。

版权所有 侵权必究

内 容 提 要

本书是教育部提出的“面向 21 世纪计算机类专业教学内容和课程体系改革”课题的研究成果，是教育部高等学校计算机教学指导委员会组织编写的计算机科学与技术专业“九五”规划课程系列教材之一。

全书共分为四篇，第一篇（第一、二章）介绍计算机的基本组成、发展和应用；第二篇（第三、四、五章）介绍总线、存储器（包括主存、Cache 及辅存）和 I/O 系统；第三篇（第六、七、八章）介绍 CPU 的特性、结构和功能，包括计算机的基本算术逻辑运算单元、指令系统、指令流水、RISC 技术及中断系统；第四篇（第九、十章）介绍控制单元的功能和设计思想，包括时序系统以及采用组合逻辑和微程序设计控制单元的设计思想与实现措施。每章后均附有习题。

本书概念清楚，通俗易懂，书中举例力求与当代计算机结合，可作为高等院校计算机专业教材，也可作为其他科技人员的参考书。

作者简介



唐朔飞，哈尔滨工业大学教授，1964年毕业于哈尔滨工业大学计算机专业。长期从事计算机科学技术的教学和研究工作，研究方向以计算机系统结构、并行处理、人工智能为主。先后完成了“863”项目机器人规划知识库、航天基金项目规划库原理的研究。在并行处理方面主要从事阵列机、ME 芯片的研究等。

1985 年编著出版了《电子数字计算机原理》教材，并荣获黑龙江省普通高校优秀教材二等奖。1990 年对该教材作了补充修改又再版，总计发行了 34 000 册。1995 年又编著出版了《计算机组成原理习题集》，同时还与他人合作完成了计算机组成原理试题库的电子出版物，并于 1997 年先后获第二届全国高校优秀计算机辅助教学软件二等奖和全国高等学校工科优秀 CAI 软件一等奖。此外还主审了计算机各类教材 3 本。在国内外主要刊物上发表论文 40 余篇。多年来指导了计算机专业硕士生 20 名，博士生 2 名。

出版说明

计算机体系结构、计算机组成原理和微型计算机技术是计算机科学与技术专业的核心课程。长期以来,大家普遍感到这3门课程的教材内容陈旧,彼此交叉重复过多,不能适应我国培养面向21世纪人才的需要,迫切希望能对它们统一规划,全盘考虑,各有侧重,避免简单重复。为此,在教育部高等学校计算机科学与技术教学指导委员会主任孙钟秀院士的领导之下,在1996年9月经过反复认真讨论最后决定由教学指导委员会副主任陈国良教授负责统一策划,并根据“面向21世纪计算机专业内容和课程体系改革”的要求,以“体系结构——组成原理——微机技术”系列教材的形式,组织编写这3本图书。

按此决议精神,经过半年多的筹备,于1997年3月邀请国内著名大学中讲授该课程的一些资深教授,并参照了国际上的同类权威教材,对该系列教材的内容划分和所属重点进行了讨论,确定了统一的编写原则,即计算机体系结构应重点论述计算机系统的各种基本结构、设计技术和性能定量分析方法;计算机组成原理应侧重讨论计算机基本部件的构成和组成方式,基本运算的操作原理和单元的设计思想、操作方式及其实现;而微型计算机技术则应突出应用,详细讲述微处理器芯片、计算机主板、接口技术和应用编程方法。

根据上述确定的原则,经过专家推荐和多方面协商,在1997年10月逐一落实了系列教材的作者与审者:其中,计算机体系结构由国防科技大学张晨曦教授等主编,复旦大学朱传琪教授主审;计算机组成原理由哈尔滨工业大学唐朝飞教授主编,中国科学技术大学陈国良教授主审;微型计算机技术由上海交通大学孙德文教授主编,华东理工大学杨明福教授主审。

此后,在1997年11月对各书的三级提纲进行了最终统一审定,并约定计算机体系结构、计算机组成原理和微型计算机技术的书稿分别于1999年3月、8月和10月提交高等教育出版社。教育部于1999年批准将该3本书作为“面向21世纪课程教材”立项。同时,讨论了要为该系列教材配套CAI软件。

这套系列教材的出版,是全国高等学校计算机科学与技术教学指导委员会全体同志和参与编审系列教材的同志们的共同努力、辛勤劳动的结果。我们非常感谢高等教育出版社的支持与鼓励,感谢全国广大读者对此套书的厚望。希望此套教材能为培养我国面向21世纪的科技人才发挥应有的作用。

教育部高等学校计算机科学与技术教学指导委员会

1999年8月13日

前 言

目前,国内虽已出版了一些有关计算机原理的教材,但在使用过程中,普遍感到结构比较僵化,内容不够新颖,跟不上计算机科学与技术发展的形势。为了适应面向 21 世纪计算机类专业教学内容和课程体系改革的需要,全国高等学校计算机教学指导委员会统一组织编写了计算机科学与技术专业九五规划课程系列教材:《计算机体系结构》、《计算机组成原理》和《微型计算机技术》。本书是该系列教材之一。

按照系列教材总体规划的要求,本教材侧重于讲授计算机基本部件的构造和组织方式、基本运算的操作原理以及部件和单元的设计思想等。

本教材突出介绍计算机组成的一般原理,不结合任何具体机型,在体系结构上改变了过去自底向上的编写习惯,采用从外部大框架入手,层层细化的叙述方法,即采用自顶向下的分析方法,详述了计算机组成原理,这将使读者更容易形成计算机的整体概念。

如果把一台计算机看作一个由许多独立部件构成的机器,那么它的功能可由其各个独立部件的功能来描述,而每个独立部件又可进一步由其内部更细的结构和功能来描述。以此类述,计算机组成原理就可按四个层次或四个篇章来组织。

第一篇(第一、二章),介绍计算机的基本组成以及计算机的发展应用和展望,使读者对计算机的总体概貌有一初步轮廓。第二篇(第三、四、五章),详细介绍除 CPU 外的存储器、输入输出系统以及连接 CPU、存储器和 I/O 之间的通信总线。第三篇(第六、七、八章)详细介绍 CPU(除控制单元外)的特性、结构和功能,包括计算机的基本运算、指令系统和中断系统等。第四篇(第九、十章)专门介绍控制单元的功能,以及采用组合逻辑和微程序方法设计控制单元的设计思路 and 实现措施。

此外,为了适应计算机科学发展的需要,除了叙述基本原理外,本书还增加了不少新的内容,书中举例力求与当代计算机相结合。某些有关数制、码制的内容以及快速乘法器、除法器 and 相联存储器等放入附录中介绍。考虑到有些学校不设外部设备课程,故本书适当地增加了外存和外部设备的内容,以供读者自学或由老师酌情选讲。

本书在编写过程中力求语言通俗易懂,文字简洁明了,便于自学者阅读,

除可作为高等院校计算机专业的教材外,也可供从事计算机事业的工程技术人员及其他自学者学习参考。

本书在编写过程中得到了国家教育部高等学校计算机科学与技术教学指导委员会的同仁们的很多帮助和鼓励。中国科学技术大学陈国良教授主审了本书,提出了许多宝贵意见。哈尔滨工业大学计算机系胡铭曾教授和系统结构教研室的老师们对本书成稿也给予了很大支持。研究生孙鹏、裴玮、樊永友、陈阳、薛园、杜跃进、杨超峰、张丽杰、罗凤杰等为书稿的录入、排版、绘图做了大量工作。在此一并表示诚挚的谢意。

作者从事计算机原理教学已有几十年的经验,在本书编写过程中尽可能地作了一些有益的探索,力求反映计算机学科中的新技术,但考虑到与系列教材的内容分工和教学时数的限制,加之作者水平有限,成书仓促,错误和不足之处在所难免,谨请读者和同行专家批评指正。

唐朔飞

1999年10月

目 录

第一篇 概述

第一章 计算机系统概论	3
1.1 计算机系统简介	3
1.1.1 计算机的软硬件概念	3
1.1.2 计算机系统的层次结构	3
1.1.3 计算机组成和计算机体系结构	7
1.2 计算机的基本组成	9
1.2.1 冯·诺依曼计算机的特点	9
1.2.2 计算机的硬件框图	9
1.2.3 计算机的工作过程	11
1.3 计算机硬件的主要技术指标	18
1.3.1 机器字长	18
1.3.2 存储容量	19
1.3.3 运算速度	19
1.4 本书结构	20
思考题与习题	21
第二章 计算机的发展及应用	22
2.1 计算机的发展史	22
2.1.1 计算机的产生和发展	22
2.1.2 微型计算机的出现和发展	25
2.1.3 软件技术的兴起和发展	27
2.2 计算机的应用	30
2.2.1 科学计算和数据处理	30
2.2.2 工业控制和实时控制	31
2.2.3 网络技术的应用	32
2.2.4 虚拟现实	34
2.2.5 办公自动化和管理信息系统	35
2.2.6 CAD/CAM/CIMS	36
2.2.7 多媒体技术	37
2.2.8 人工智能	37
2.3 计算机的展望	39

第二篇 计算机系统硬件结构

第三章 系统总线	45
3.1 总线的基本概念	45
3.2 总线的分类	47
3.2.1 片内总线	47
3.2.2 系统总线	47

3.2.3 通信总线	49
3.3 总线特性及性能指标	49
3.3.1 总线特性	49
3.3.2 总线性能指标	50
3.3.3 总线标准	51
3.4 总线结构	53
3.4.1 单总线结构	53
3.4.2 多总线结构	53
3.4.3 总线结构举例	56
3.5 总线控制	58
3.5.1 总线判优控制	58
3.5.2 总线通信控制	60
思考题与习题	65
第四章 存储器	67
4.1 概述	67
4.1.1 存储器分类	67
4.1.2 存储器的层次结构	69
4.2 主存储器	71
4.2.1 概述	71
4.2.2 半导体存储芯片简介	74
4.2.3 随机存取存储器 (RAM)	76
4.2.4 只读存储器 (ROM)	87
4.2.5 存储器与 CPU 的连接	91
4.2.6 存储器的校验	98
4.2.7 提高访存速度的措施	101
4.3 高速缓冲存储器 (Cache)	105
4.3.1 概述	105
4.3.2 Cache—主存地址映像	111
4.3.3 替换算法	115
4.4 辅助存储器	115
4.4.1 概述	115
4.4.2 磁记录原理和记录方式	117
4.4.3 硬磁盘存储器	122
4.4.4 软磁盘存储器	130
4.4.5 磁带存储器	135
4.4.6 循环冗余校验码 (CRC 码)	138
4.4.7 光盘存储器	140
思考题与习题	144
附录 4A 相联存储器	148
第五章 输入输出系统	151
5.1 概述	151
5.1.1 输入输出系统的发展概况	151
5.1.2 输入输出系统的组成	153
5.1.3 I/O 设备与主机的联系方式	155
5.1.4 I/O 与主机信息传送的控制方式	158

5.2 外部设备	162
5.2.1 概述	162
5.2.2 输入设备	164
5.2.3 输出设备	168
5.2.4 其他外部设备	180
5.2.5 多媒体技术	182
5.3 I/O 接口	185
5.3.1 概述	185
5.3.2 接口的功能和组成	186
5.3.3 接口类型	189
5.4 程序查询方式	190
5.4.1 程序查询流程	190
5.4.2 程序查询方式的接口电路	192
5.5 程序中断方式	192
5.5.1 中断的概念	192
5.5.2 I/O 中断的产生	193
5.5.3 程序中断方式的接口电路	194
5.5.4 I/O 中断处理过程	196
5.5.5 中断服务程序的流程	198
5.6 DMA 方式	200
5.6.1 DMA 方式的特点	200
5.6.2 DMA 接口的功能和组成	203
5.6.3 DMA 的工作过程	205
5.6.4 DMA 接口的类型	208
思考题与习题	211
附录 5A.1 ASCII 码	213
附录 5A.2 BCD 码	215
附录 5A.3 奇偶校验码	216

第三篇 中央处理器 (CPU)

第六章 计算机的运算方法	219
6.1 无符号数和有符号数	219
6.1.1 无符号数	219
6.1.2 有符号数	219
6.2 数的定点表示和浮点表示	228
6.2.1 定点表示	228
6.2.2 浮点表示	229
6.2.3 定点数和浮点数的比较	231
6.2.4 举例	232
6.2.5 IEEE 754 标准	233
6.3 定点运算	234
6.3.1 移位运算	234
6.3.2 加法与减法运算	237
6.3.3 乘法运算	243
6.3.4 除法运算	259

6.4 浮点四则运算	270
6.4.1 浮点加减运算	270
6.4.2 浮点乘法除法运算	276
6.4.3 浮点运算所需的硬件配置	281
6.5 算术逻辑单元	281
6.5.1 ALU 电路	281
6.5.2 快速进位链	284
思考题与习题	291
附录 6A.1 各种进位制的对应关系	295
附录 6A.2 各种进位制的转换	296
附录 6B 阵列乘法器和阵列除法器	299
附录 6C 74181 逻辑电路	302
第七章 指令系统	303
7.1 机器指令	303
7.1.1 指令的一般格式	303
7.1.2 指令字长	307
7.2 操作数类型和操作类型	307
7.2.1 操作数类型	307
7.2.2 数据在存储器中的存放方式	308
7.2.3 操作类型	309
7.3 寻址方式	313
7.3.1 指令寻址	313
7.3.2 数据寻址	314
7.4 指令格式举例	323
7.4.1 设计指令格式应考虑的各种因素	323
7.4.2 指令格式举例	323
7.5 RISC 技术	326
7.5.1 RISC 的产生和发展	326
7.5.2 RISC 的主要特征	330
7.5.3 RISC 和 CISC 的比较	333
思考题与习题	335
第八章 CPU 的结构和功能	338
8.1 CPU 的结构	338
8.1.1 CPU 的功能	338
8.1.2 CPU 结构框图	339
8.1.3 CPU 的寄存器	340
8.1.4 控制单元 CU 和中断系统	343
8.2 指令周期	344
8.2.1 指令周期的基本概念	344
8.2.2 指令周期的数据流	346
8.3 指令流水	347
8.3.1 指令流水原理	349
8.3.2 影响流水线性能的因素	351
8.3.3 流水线中的多发技术	353
8.3.4 流水线结构	354

8.4 中断系统	356
8.4.1 概述	356
8.4.2 中断请求标记和中断判优逻辑	358
8.4.3 中断服务程序入口地址的寻找	360
8.4.4 中断响应	361
8.4.5 保护现场和恢复现场	363
8.4.6 中断屏蔽技术	363
思考题与习题	366

第四篇 控制单元 CU

第九章 控制单元的功能	371
9.1 微操作命令的分析	371
9.1.1 取指周期	371
9.1.2 间址周期	371
9.1.3 执行周期	372
9.1.4 中断周期	374
9.2 控制单元的功能	374
9.2.1 控制单元的外特性	374
9.2.2 控制信号举例	375
9.2.3 多级时序系统	379
9.2.4 控制方式	380
9.2.5 多级时序系统实例分析	382
思考题与习题	387
第十章 控制单元的设计	389
10.1 组合逻辑设计	389
10.1.1 组合逻辑控制单元框图	389
10.1.2 微操作的节拍安排	390
10.1.3 组合逻辑设计步骤	392
10.2 微程序设计	394
10.2.1 微程序设计思想的产生	394
10.2.2 微程序控制单元框图及工作原理	395
10.2.3 微指令的编码方式	399
10.2.4 微指令序列地址的形成	401
10.2.5 微指令格式	402
10.2.6 静态微程序设计和动态微程序设计	404
10.2.7 毫微程序设计	404
10.2.8 串行微程序控制和并行微程序控制	405
10.2.9 微程序设计举例	405
思考题与习题	411
参考文献	414

第一篇 概 论

本篇主要介绍计算机系统的基本组成、应用与发展，并通过对本书结构的介绍，给读者指出了学习本教材的基本思路。

第一章 计算机系统概论

本章主要介绍计算机的组成概貌及工作原理,旨在使读者对计算机总体结构有个概括的了解,为深入学习以后各章打下基础。

1.1 计算机系统简介

1.1.1 计算机的软硬件概念

计算机系统由“硬件”和“软件”两大部分组成。

所谓“硬件”即指计算机的实体部份,它由看得见摸得着的各种电子元件、各类光、电、机设备的实物组成,如主机、外设等等。

所谓“软件”,它是看不见摸不着的,由人们事先编制成具有各类特殊功能的信息组成。通常把这些信息,诸如各类程序寄寓于各类媒体中,如 RAM、ROM、磁带、磁盘、光盘、甚至纸带等。它们通常被作为计算机的主存或辅存的内容。由于“软件”的发展,它不仅可以充分发挥计算机的“硬件”功能,提高计算机的工作效率,而且已经发展到能局部模拟人类的思维活动,因此在整个计算机系统内,“软件”的地位和作用已经成为评价计算机系统性能好坏的重要标志。当然,“软件”性能的发挥,也必须依托“硬件”的支撑。因此,概括而言,计算机性能的好坏,取决于“软”、“硬”件功能的总和。

计算机的软件通常又可以分为两大类:系统软件和应用软件。

系统软件又称为系统程序,主要用来管理整个计算机系统,监视服务,使系统资源得到合理调度,确保高效运行。它包括:标准程序库、语言处理程序(如将汇编语言翻译成机器语言的汇编程序;将高级语言翻译成机器语言的编译程序)、操作系统(如批处理系统、分时系统、实时系统)、服务性程序(如诊断程序、调试程序、连接程序等)、数据库管理系统、网络软件等等。

应用软件又称为应用程序,它是用户根据任务需要所编制的各种程序。如科学计算程序,数据处理程序,过程控制程序,事务管理程序等等。

1.1.2 计算机系统的层次结构

现代计算机的解题过程通常是先由用户用高级语言编写程序(称作源程序),然后将它和数据一起送入计算机内,再由计算机将其翻译成机器能识别的机器语言程序(称作目标程序),机器自动运行该机器语言程序,并将计算

结果输出。其过程如图 1.1 所示。

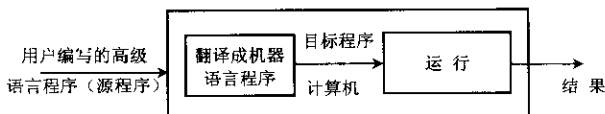


图 1.1 计算机的解题过程

实际上，早期的计算机只有机器语言（即用 0/1 代码表示的语言），用户必须用二进制 0/1 代码来编写程序（即机器语言程序）。这就要求程序员对他们所使用的计算机的硬件及指令系统十分熟悉，编写程序难度很大，操作过程也极易出错。但用户编写的机器语言程序，可以直接在机器上执行，我们把直接执行机器语言的实际机器称 M_1 ，如图 1.2 所示。

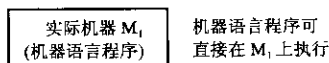


图 1.2 实际机器 M_1

20 世纪 50 年代开始出现了符号式的程序设计语言，即汇编语言。它用符号 ADD、SUB、MUL、DIV 等分别表示作加、减、乘、除等操作，并用符号表示指令或数据所在存储空间的地 址，这就使程序员摆脱了用繁杂而又易错的二进制代码编写程序。但是，实际上没有一种机器能直接识别这种汇编语言程序，必须先将汇编语言程序翻译成机器语言程序后，才能被机器接受并自动运行。这个翻译过程是由机器系统软件中的汇编程序来完成的。如果我们把具有翻译功能的汇编程序的计算机看作一台 M_2 机器，那么，可以认为 M_2 在 M_1 之上，用户可以利用 M_2 的翻译功能直接向 M_2 输入汇编语言程序，而 M_2 又会将翻译后的机器语言程序输入给 M_1 ， M_1 执行后将结果交付给人们。因此， M_2 并不是一台实际机器，它只是人们感到存在的一台具有翻译功能的机器，称这类机器为虚拟机。这样，整个计算机系统便具有两级层次结构，如图 1.3 所示。尽管有了虚拟机 M_2 使用户编程更为方便，但从本质上看，汇编语言仍是一种面向实际机器的语言，它的每一条语句都与机器语言的某一条语句（0/1 代码）一一对应。因此，使用汇编语言编写程序时，仍要求程序员对实际机器 M_1 的内部组成和指令系统非常熟悉，也就是说，程序员必须经过专门的训练，否则是无法操作计算机的。另一方面，由于汇编语言摆脱不了实际机器的指令系统，因此，汇编语言没有通用性，每台机器必须有一种与之相对应的汇编语言。这

就大大阻碍了计算机的广泛使用。

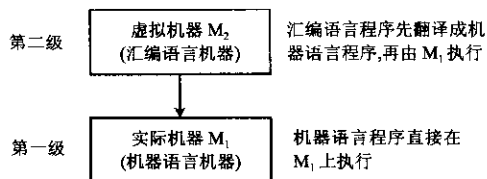


图 1.3 具有两级层次结构的计算机系统

20 世纪 60 年代开始出现了各种面向问题的高级语言, 如 FORTRAN、BASIC、Pascal、C 等等。这类高级语言对问题的描述十分接近人们的习惯, 并且还具有较强的通用性。程序员完全可以不必了解掌握实际机器 M_1 的机型、内部的具体组成及其自身的指令系统, 只要掌握这类高级语言本身所赋予的语法和语义, 便可直接用这种高级语言来编程, 这给程序员带来了极大的方便。当然, M_1 机器本身是不能识别高级语言的, 因此, 在进入 M_1 机器运行前, 必须先将高级语言程序翻译成汇编语言程序 (或其他中间语言程序), 然后再将其翻译成机器语言程序。也可以将高级语言程序直接翻译成机器语言程序。这些工作都是由虚拟机 M_3 来完成的, 对程序员而言, 他们并不知道这个翻译过程。由此我们又可得出具有三级层次结构的计算机系统, 如图 1.4 所示。

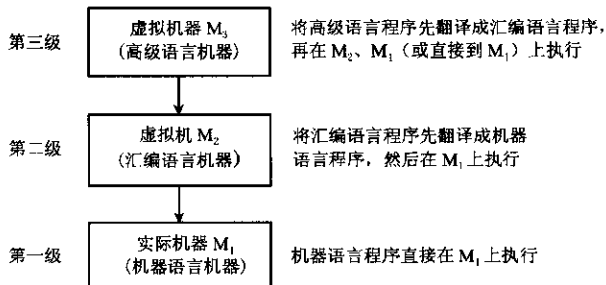


图 1.4 具有三级层次结构的计算机系统

通常, 我们把高级语言程序翻译成机器语言程序的软件叫做翻译程序。翻译程序有两种: 一种叫编译程序, 另一种叫解释程序。编译程序将用户编写的高级语言程序 (即源程序) 的全部语句一次全部翻译成机器语言程序, 而后再执

行机器语言程序。因此,只要源程序不变,就无需再次进行翻译。如 FORTRAN、PASCAL 等语言,就是用编译程序来完成翻译的。解释程序是将源程序的一条语句翻译成对应于机器语言的一条语句,并且立即执行这条语句,接着再翻译源程序的下一条语句,并执行这条语句,如此重复直至完成源程序的全部翻译任务。它的特点是翻译一次执行一次,即使下一次重复执行该语句时,也必须重新翻译。如 BASIC 语言的翻译就有解释程序和编译程序两种。

从上述介绍不难看出,由于软件的发展,使实际机器 M_1 向上延伸构成了各级虚拟机器。同理 M_1 机器内部也可向下延伸而形成下一级的微程序机器 M_0 。 M_0 机器是直接将 M_1 机器中的每一条机器指令翻译成一组微指令,即构成一个微程序。 M_0 机器每执行完对应于一条机器指令的一个微程序后,便由 M_1 机器中的下一条机器指令,使 M_0 机器自动进入与其相对应的另一个微程序的执行。由此可见,微程序机器 M_0 可看作是对实际机器 M_1 的分解,即用 M_0 的微程序解释并执行 M_1 的每一条机器指令(有关微程序机器的介绍,详见第十章)。由于 M_0 机器也是实际机器,因此,为了区别于 M_1 ,通常又将 M_1 叫做传统机器,将 M_1 叫做微程序机器。这样计算机系统又可认为具有四级层次结构,如图 1.5 所示。

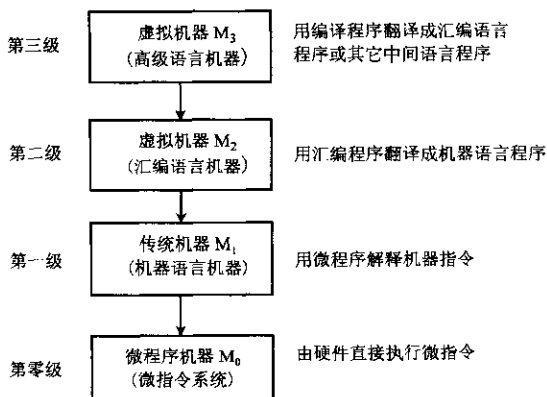


图 1.5 具有四级层次结构的计算机系统

在上述四级层次结构的系统中,实际上在实际机器 M_1 与虚拟机器 M_2 之间,还有一级虚拟机器,它是由操作系统软件构成的。操作系统提供了在汇编语言和高级语言的使用和实现过程中所需的某些基本操作,还起到控制并管理

系统硬件和软件全部资源的作用，为用户使用计算机系统提供极为方便的条件。操作系统的功能是通过其控制语言来实现的。图 1.6 描绘了一个常见的五级计算机系统的层次结构。

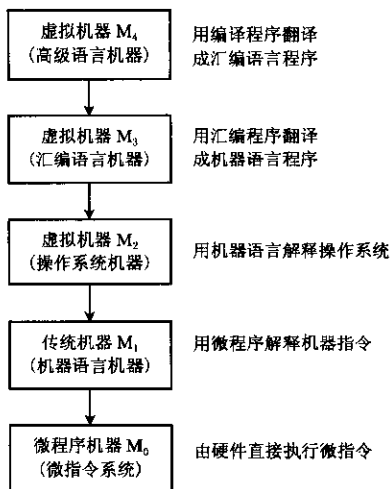


图 1.6 多级层次结构的计算机系统

从计算机系统的多级层次结构来看，可以将硬件研究的主要对象归结为传统机器 M_1 和微程序机器 M_0 。软件的研究对象主要是操作系统级以上的各级虚拟机。值得指出的是，软硬件交界面的划分并不是一成不变的。随着超大规模集成电路技术的不断发展，一部份软件功能将由硬件来实现，如目前操作系统已实现了部份固化等等。可见，软、硬件交界面变化的趋势正沿着如图 1.6 所示的愈来愈向上的方向发展。

本教材主要讨论传统机器 M_1 和微程序机器 M_0 的组成原理及设计思想，其他各级虚拟机的内容，均由相应的软件课程讲授。

1.1.3 计算机组成和计算机体系结构

在学习计算机组成时，应当注意如何区别计算机体系结构与计算机组成这两个基本概念。

计算机体系结构是指那些能够被程序员所见到的计算机系统的属性，即概念性的结构与功能特性，通常是指用机器语言编程的程序员（也包括汇编语言

程序设计者和汇编程序设计者)所看到的传统机器的属性,包括指令集、数据类型、存储器寻址技术、I/O 机理等等,大都属于抽象的属性。由于计算机系统具有多级层次结构,因此,站在不同层次上编程的程序员所看到的计算机属性也是各不相同的。例如,用高级语言编程的程序员,可以把 IBM PC 与 RS6000 两种机器看成是同一属性的机器。可是,对使用汇编语言编程的程序员来说,IBM PC 与 RS6000 是两种截然不同的机器。因为程序员所看到的这两种机器的属性,如指令集、数据类型、寻址技术等完全不同,因此,认为这两种机器的结构是各不相同的。

计算机组成是指如何实现计算机体系结构所体现的属性,它包含了许多对程序员来说是透明的(即程序员不知道的)硬件细节。例如,指令系统体现了机器的属性,这是属于计算机结构的问题。但指令的实现,即如何取指令、分析指令、取操作数、如何运算、如何送结果等等,这些都属于计算机组成问题。因此,当两台机器指令系统相同时,只能认为它们具有相同的结构。至于这两台机器如何实现其指令,完全可以不同,则我们认为它们的组成方式是不同的。例如,一台机器是否具备乘法指令的功能,这是一个结构的问题,可是,实现乘法指令采用什么方式的问题,则是一个组成问题。实现乘法指令可以采用一个专门的乘法电路,也可以采用连续相加的加法电路来实现,这两者的区别就是计算机组成的区别。究竟应该采用哪种方式来组成计算机,这要考虑到各种因素,如乘法指令使用的频度、两种方法的运行速度、两种电路的体积、价格、可靠性等等。

不论是过去还是现在,区分计算机结构与计算机组成这两个概念都是十分重要的。例如,许多计算机制造商向你提出一系列体系结构相同的计算机,而它们的组成却有相当大的差别,即使是同一系列不同型号的机器,其价格和性能也是有极大差异的。因此,只知其结构,不知其组成,就选不好性能价格比最合适的机器。此外,一种机器的体系结构可能维持许多年,但机器的组成却会随着计算机技术的发展而不断变化。例如,1970 年首次推出了 IBM System/370 结构,它包含了许多机型。一般需求的用户可以买价格便宜的低速机型;对需求高的用户,可以买一台升级的价格稍贵的机型,而不必抛弃原来已开发的软件。许多年来,不断推出性能更高、价格更低的机型,新机型总归保留着原来机器的结构,使用户的软件投资不致浪费。

本书主要研究计算机的组成,有关计算机体系结构的内容,将在《计算机体系结构》中讲述。

1.2 计算机的基本组成

1.2.1 冯·诺依曼计算机的特点

1945 年数学家冯·诺依曼 (von Neumann) 等人, 在研究 EDVAC 机时, 提出了“存储程序”的概念。以此概念为基础的各类计算机, 统称为冯·诺依曼机。它的特点可归结为:

- 计算机由运算器、存储器、控制器和输入设备、输出设备五大部件组成。
- 指令和数据以同等地位存放于存储器内, 并可按地址寻访。
- 指令和数据均用二进制码表示。
- 指令由操作码和地址码组成, 操作码用来表示操作的性质, 地址码用来表示操作数所在存储器中的位置。
- 指令在存储器内按顺序存放。通常, 指令是顺序执行的, 在特定条件下, 可根据运算结果或根据设定的条件改变执行顺序。
- 机器以运算器为中心, 输入输出设备与存储器的数据传送通过运算器。

1.2.2 计算机的硬件框图

典型的冯·诺依曼计算机是以运算器为中心的, 如图 1.7 所示。其中, 输入、输出设备与存储器之间的数据传送都需通过运算器。图中实线为数据线, 虚线为控制线和反馈线。

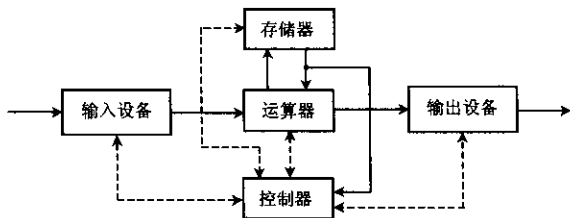


图 1.7 典型的冯·诺依曼计算机结构框图

现代的计算机已转化为以存储器为中心, 如图 1.8 所示。图中实线为控制线, 虚线为反馈线, 双线为数据线。

图中各部件的功能是:

- 运算器用来完成算术运算和逻辑运算, 并将运算的中间结果暂存在运算

器内；

- 存储器用来存放数据和程序；
- 控制器用来控制、指挥程序和数据的输入、运行以及处理运算结果；
- 输入设备用来将人们熟悉的信息形式转换为机器能识别的信息形式，常见的有键盘、鼠标等。
- 输出设备可将机器运算结果转换为人们熟悉的信息形式，如打印机输出、显示器输出等。

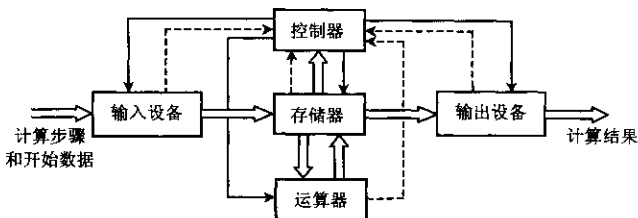


图 1.8 以存储器为中心的计算机结构框图

计算机的五大部件（又称五大子系统）在控制器的统一指挥下，实现有条不紊地自动工作。

由于运算器和控制器在逻辑关系和电路结构上联系十分紧密，尤其在大规模集成电路制作工艺出现后，这两大部件往往制作在同一芯片上，因此，通常将它们合起来统称为中央处理器（Central Processing Unit），简称 CPU。把输入设备与输出设备简称为 I/O 设备（Input/Output equipment）。

这样，现代计算机可认为由三大部分组成：CPU、I/O 设备及主存储器 M.M（Main Memory），如图 1.9 所示。CPU 与 M.M 合起来又可称为主机，I/O 设备可叫作外部设备。

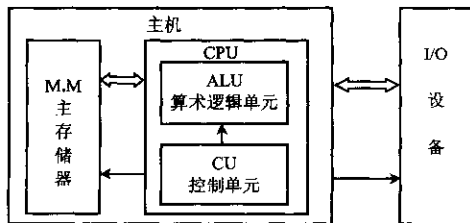


图 1.9 现代计算机的组成框图

图 1.9 中的主存储器 M.M 是存储器子系统中的一类, 用来存放程序和数
据, 它可以直接与 CPU 交换信息。另一类叫辅助存储器, 简称辅存, 又叫外存,
其功能参阅 4.4 节。

ALU (Arithmetic Logic Unit) 叫做算术逻辑运算单元 (简称算逻部件),
用来完成算术逻辑运算。CU (Control Unit) 叫做控制单元, 用来解释存储器
中的指令, 并发出各种操作命令来执行指令。ALU 和 CU 是 CPU 的核心部件。

I/O 设备也受 CU 控制, 用来完成相应的输入、输出操作。

可见, 计算机有条不紊地自动工作, 都是在控制器统一指挥下完成的。

1.2.3 计算机的工作过程

用计算机解决一个实际问题, 通常包含两大过程。一个是上机前的各种准
备, 再一个是上机运行。

1. 上机前的准备

在许多科学技术的实际问题中, 往往会遇到许多复杂的数学方程组, 而数
字计算机通常只能作加、减、乘、除四则运算, 这就要求在上机解题前, 先由
人工完成一些必要的准备工作。这些工作大致可归纳为: 建立数学模型、确定
计算方法、编制解题程序三个步骤。

(1) 建立数学模型

有许多科技问题很难直接用物理模型来模拟研究对象的变化规律, 如地球
大气环流、原子反应堆的核裂变过程、航天飞行速度对飞行器的影响等等。不
过, 通过大量的实验和分析, 总能找到一系列反映研究对象变化规律的数学方
程组。通常, 把这类方程组叫做被研究对象变化规律的数学模型。一旦建立了
数学模型, 研究对象的变化规律就变成了解一系列方程组的数学问题, 这便可
通过计算机来求解。因此, 建立数学模型是用计算机解题的第一步。

(2) 确定计算方法

由于数学模型中的数学方程式往往是很复杂的, 欲将它变成适合计算机运
算的加、减、乘、除四则运算, 还必须确定对应的计算方法。

例如, 欲求 $\sin x$ 的值, 只有采用近似计算方法, 用四则运算的式子来求
得 (因计算机内部没有直接完成三角函数运算的部件)。如:

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \frac{x^9}{9!} - \cdots$$

又如, 计算机不能直接解开方 x , 但可用迭代公式:

$$y_{n+1} = \sqrt{x} - \frac{1}{2} \left(y_n + \frac{x}{y_n} \right) \quad (n = 0, 1, 2, \cdots)$$

通过多次迭代, 便可求得相应精度的 $\sqrt{x}$ 值。

(3) 编制解题程序

程序是适合于机器运算的全部步骤，编制解题程序就是将运算步骤用一一对应的机器指令描述。

例如，计算 $ax^2 + bx + c$ 可分解为以下步骤：

- 将 x 取至运算器中；
- 乘以 x ，得 x^2 ，存于运算器中；
- 再乘以 a ，得 ax^2 ，存于运算器中；
- 将 ax^2 送至存储器中；
- 取 b 至运算器中；
- 乘以 x ，得 bx ，存于运算器中；
- 将 ax^2 从存储器中取出与 bx 相加，得 $ax^2 + bx$ ，存于运算器中；
- 再取 c 与 $ax^2 + bx$ 相加，得 $ax^2 + bx + c$ ，存于运算器中。

可见，不包括停机、输出打印共需八步。若将上式改写成： $(ax + b)x + c$ ，则其步骤可简化为五步：

- 取 x 至运算器中；
- 乘以 a ，得 ax ，存于运算器中；
- 加 b ，得 $ax + b$ ，存于运算器中；
- 乘以 x ，得 $(ax + b)x$ ，存于运算器中；
- 加 c ，得 $(ax + b)x + c$ ，存于运算器中。

将上述运算步骤写成某计算机的一一对应的机器指令，就完成了运算程序的编写。

若设某机的指令字长为 16 位，其中操作码占 6 位，地址码占 10 位，如图 1.10 所示。

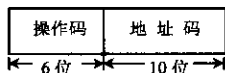


图 1.10 某机器指令格式

操作码表示机器所执行的各种操作，如取数、存数、加、减、乘、除、停机、打印等等。地址码表示参加运算的数在存储器内的位置。机器指令的操作码和地址码都采用 0/1 代码的组合来表示。表 1.1 列出了某机与上例有关的各条机器指令的操作码及其操作性质的对应关系。

表 1.1 操作码与操作性质的对应表

操作码	操作性质
000001	取数——将指令地址码指示的存储单元中的操作数取到运算器的累加器 ACC 中
000010	存数——将 ACC 中的数存至指令的地址码指示的存储单元中
000011	加——将 ACC 中的数与指令地址码指示的存储单元中的数相加, 结果存于 ACC 中
000100	乘——将 ACC 中的数与指令地址码指示的存储单元中的数相乘, 结果存于 ACC 中
000101	打印——将指令地址码指示的存储单元中的操作数打印输出
000110	停机

此例中所用到的数 a 、 b 、 c 、 x , 事先需存入存储器的相应单元内。

按 $ax^2 + bx + c$ 的运算分解, 可用上述机器指令编写出一份运算的程序清单, 如表 1.2 所列。

表 1.2 计算 $ax^2 + bx + c$ 程序清单

指令和数据存于主存单元的地址	指 令		注 释
	操作码	地址码	
0	000001	0000001000	取数 x 至 ACC
1	000100	0000001001	乘 a 得 ax 存于 ACC 中
2	000011	0000001010	加 b 得 $ax+b$, 存于 ACC 中
3	000100	0000001000	乘 x 得 $(ax+b)x$, 存于 ACC 中
4	000011	0000001011	加 c 得 ax^2+bx+c , 存于 ACC 中
5	000010	0000001100	存数, 将 ax^2+bx+c 存于主存单元
6	000101	0000001100	打印
7	000110		停机
8	x		原始数据 x
9	a		原始数据 a
10	b		原始数据 b
11	c		原始数据 c
12			存放结果

以上程序编完后，便可进入下一步上机。

2. 计算机的解题过程

为了比较形象地了解计算机的解题过程，首先分析一个比图 1.9 更细化的计算机组成框图，如图 1.11 所示。

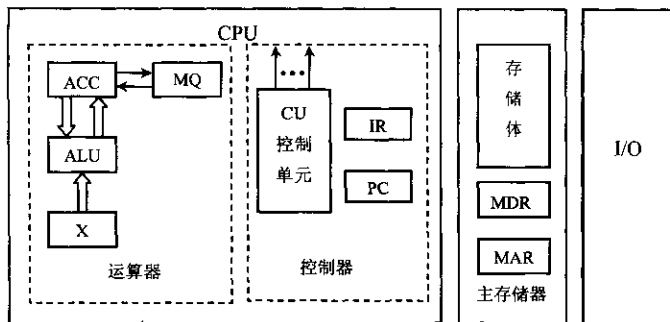


图 1.11 细化的计算机组成框图

(1) 运算器

运算器包括三个寄存器（现代计算机内部往往设有通用寄存器组）和一个算逻电路 ALU。其中 ACC（Accumulator）为累加器，MQ（Multiplier-Quotient Register）为乘商寄存器，X 为操作数寄存器。这三个寄存器在完成不同运算时，所存放的操作数类别也各不相同。表 1.3 列出了寄存器存放不同类别操作数的情况。

表 1.3 各寄存器所存放的各类操作数

运算 操作数 寄存器	加法	减法	乘法	除法
ACC	被加数及和	被减数及差	乘积高位	被除数及余数
MQ			乘数及乘积低位	商
X	加数	减数	被乘数	除数

不同机器的运算器结构是不同的。图 1.11 所示的运算器可将运算结果从 ACC 送至存储器中的 MDR；而存储器的操作数也可从它的 MDR 送至运算器中的 ACC、MQ 或 X。有的机器用 MDR 取代 X 寄存器。

下面简要地分析一下这种结构的运算器其加、减、乘、除四则运算的操

作过程。

设: M 表示存储器的任一地址号

[M] 表示对应 M 地址号单元中的内容

X 表示 X 寄存器

[X] 表示 X 寄存器中的内容

ACC 表示累加器

[ACC] 表示累加器中的内容

MQ 表示乘商寄存器

[MQ] 表示乘商寄存器中的内容

假设 ACC 中已存有前一时刻的运算结果, 并作为下述运算中的一个操作数。则:

- 加法操作过程:

$$[M] \rightarrow X$$

$$[ACC] + [X] \rightarrow ACC$$

即将 [ACC] 看作被加数, 先从内存中取一个存放在 M 地址号内的加数 [M], 送至运算器的 X 寄存器中, 然后将被加数 [ACC] 与加数 [X] 相加, 其结果和保留在累加器 ACC 中。

- 减法操作过程:

$$[M] \rightarrow X$$

$$[ACC] - [X] \rightarrow ACC$$

即将 [ACC] 看作被减数, 先取出减数 [M] 送入 X, 再作 $[ACC] - [X]$, 其结果差保留在 ACC 中。

- 乘法操作过程:

$$[M] \rightarrow MQ$$

$$[ACC] \rightarrow X$$

$$0 \rightarrow ACC$$

$$[X] \times [MQ] \rightarrow ACC // MQ^{①}$$

即把 [ACC] 看作被乘数, 先将在 M 号单元中的乘数 [M] 送入乘商寄存器 MQ, 再把被乘数送入 X 寄存器, 并将寄存器 A 清“0”, 然后 [X] 和 [MQ] 相乘, 其结果积的高位保留在 ACC 中, 积的低位保留在 MQ 中。

- 除法操作过程:

$$[M] \rightarrow X$$

① // 表示两个寄存器串接

$$[ACC] \div [X] \rightarrow MQ$$

余数 R 在 ACC 中

即将 $[ACC]$ 看作被除数, 先将 M 号单元内的除数 $[M]$ 送至 X 寄存器, 然后 $[ACC]$ 除以 $[X]$, 其结果商暂留于 MQ , $[ACC]$ 为余数 R 。若需要将商保留在 ACC 中, 只需做一步 $[MQ] \rightarrow ACC$ 即可。

(2) 主存储器

主存储器(简称主存或内存)包括存储体、各种逻辑部件及控制电路等。存储体由许多存储单元组成, 每个存储单元又包含若干个存储元件(或称存储基元、存储元), 每个存储元件能寄存一位二进制代码“0”或“1”。可见, 一个存储单元可存储一串二进制代码, 称这串二进制代码为一个存储字, 这串二进制代码的个数叫作存储字长。存储字长可以是 8 位、16 位或 32 位等。一个存储字可代表一个二进制的数, 也可代表一串字符, 如存储字为 0011011001111101, 既可表示为由十六进制字符组成的 367DH (有关十六进制数制详见附录 6A.1), 又可代表十六位的二进制数, 此值对应十进制数为 13949, 还可代表两个 ASCII 码: “3”和“}”(参见附录 5A.1 ASCII 编码表)。一个存储字还可代表一条指令(参阅表 1.2)。

如果我们把一个存储体比作一幢大楼, 那么每个存储单元可看作大楼中的每个房间, 每个存储元可看作每个房间中的一张床位。显然, 每个房间都需要有一个房间编号, 因此, 我们也赋予每个存储单元一个编号, 叫做存储单元的地址号。主存的工作方式就是按存储单元的地址号来实现对存储字各位的存(写入)、取(读出)。这种存取方式叫做按地址存取, 也即按地址访问存储器(简称访存)。存储器这种工作性质对计算机的组成和操作是十分有利的。例如, 人们只要事先将编好的程序按顺序存入主存各单元, 当运行程序时, 先给出该程序在主存的首地址, 然后采用程序计数器加 1 的办法, 自动形成下一条指令所在存储单元的地址, 机器便可自动完成整个程序的操作。又如, 由于数据和指令都存放在存储体内各自所占有的不同单元中, 因此, 当需要反复使用某个数据或某条指令时, 只要指出其相应的单元地址号即可, 而不必占用更多的存储单元重复存放同一个数据或同一条指令, 大大提高了存储空间利用率。此外, 由于指令和数据都由存储单元地址号来反映, 因此, 取一条指令和取一个数据的操作, 完全可视为是相同的, 这样就可使用一套控制线路来完成两种截然不同的操作内容。

为了能够实现按地址访问的方式, 主存中还必须配置两个寄存器 MAR 和 MDR。MAR (Memory Address Register) 是存储器地址寄存器, 用来存放欲访问的存储单元的地址, 其位数对应存储单元的个数(如 MAR 为 10 位, 则有

$2^{10}=1\,024$ 个存储单元, 记为 1K)。MDR (Memory Data Register) 是存储器数据寄存器, 用来存放从存储体某单元取出的代码或者准备往某存储单元存入的代码, 其位数与存储字长相等。当然, 要想完整地完一个取或存操作, CPU 还得给主存加以各种控制信号, 如读命令、写命令和地址译码驱动信号等。随着硬件技术的发展, 主存都制成大规模集成电路的芯片, 而将 MAR 和 MDR 制作在 CPU 芯片中 (参阅图 4.4)。

早期计算机的存储字长一般和机器的指令字长与数据字长相等, 故访问一次主存便可取一条指令或一个数据。随着计算机应用范围的不断扩大, 解题精度的不断提高, 往往要求指令字长是可变的, 数据字长也要求可变。为了适应指令和数据字长的可变性, 其长度不是由存储字长来确定, 而由字节的个数来表示。1 个字节 (Byte) 被定义为由 8 位 (bit) 二进制代码组成。例如 4 字节数据就是 32 位二进制代码; 2 字节构成的指令字长是 16 位二进制代码。当然, 此时存储字长、指令字长、数据字长三者可各不相同, 但它们必须是字节的整数倍。

(3) 控制器

控制器是计算机组成的神经中枢, 由它指挥全机各部件自动、协调地工作。具体而言, 它首先要命令存储器读出一条指令, 这叫做取指过程 (也称取指阶段)。接着, 它要对这条指令进行分析, 指出该指令要完成什么样的操作, 并按寻址特征指明操作数的地址, 这叫做分析过程 (也称分析阶段)。最后根据操作数所在的地址, 取出操作数并完成某种操作, 这叫作执行过程 (也称执行阶段)。以上就是通常所说的完成一条指令操作的取指、分析和执行三阶段。

控制器由程序计数器 PC (Program Counter), 指令寄存器 IR (Instruction Register) 以及控制单元 CU 几部分组成。PC 用来存放当前欲执行指令的地址, 它与主存的 MAR 之间有一条直接通路, 且具有自动加 1 的功能, 即可自动形成下一条指令的地址。IR 用来存放当前的指令, IR 的内容来自主存的 MDR。IR 中的操作码 (OP(IR)) 送至 CU (记作 $OP(IR) \rightarrow CU$), 用来分析指令; 其地址码 (Ad(IR)) 作为操作数的地址送至存储器的 MAR (记作 $Ad(IR) \rightarrow MAR$)。CU 用来分析当前指令所需完成的操作, 并发出各种微操作命令序列, 用以控制所有被控对象。

(4) I/O

I/O 子系统包括各种外部设备及相应的接口。每一种设备都是由 I/O 接口与主机联系的, 它接受 CU 发出的各种控制命令完成相应的操作。如键盘 (输入设备) 由键盘接口电路与主机联系; 打印机 (输出设备) 由打印机接口电路与主机联系。

下面结合图 1.11, 进一步深入领会计算机解题的全过程。

首先按表 1.2 所列的有序指令和数据, 通过键盘输入到主存第 0 号至第 12 号单元中, 并置 PC 的初值为 0 (即令程序的首地址为 0)。启动机器后, 计算机便自动按存储器中所存放的指令顺序, 有序地逐条完成取指令、分析指令和执行指令, 直至执行到程序的最后一条指令为止。

例如, 启动机器后, 控制器立即将程序计数器的内容送至主存的 MAR (记作 $PC \rightarrow MAR$) 并命令存储器做读操作, 此刻主存“0”号单元的内容“0000010000001000”(表 1.2 示为程序的第一条指令) 便被送入 MDR 内。然后由 MDR 送至控制器的 IR (记作 $MDR \rightarrow IR$), 完成了一条指令的取指过程。经 CU 分析操作码“000001”为取数指令, 于是 CU 又将 IR 中的地址码“0000001000”送至 MAR, 并命令存储器做读操作, 将该地址单元中的操作数 x 送至 MDR, 再由 MDR 送至运算器的 ACC (记作 $MDR \rightarrow ACC$), 完成了此指令的执行过程。此刻, 也即完成了第一条取数指令的全过程, 即将操作数 x 送至运算器 ACC 中。与此同时, PC 完成自动加 1 的操作, 形成了下一条指令的地址“1”号。同上所述, 由 PC 送至 MAR, 命令存储器做读操作, 将“0001000000001001”送入 MDR, 又由 $MDR \rightarrow IR$ 。接着 CU 分析操作码“000100”为乘法指令, 故 CU 又向存储器发出读命令, 取出对应地址为“0000001001”单元中的操作数 a , 经 MDR 送至运算器 MQ, CU 再向运算器发乘法操作命令, 完成 ax 的运算, 并把运算结果 ax 存放在 ACC 中。同时 PC 完成一次 $(PC) + 1 \rightarrow PC$, 形成下一条指令的地址“2”号。依次类推, 逐条取指、分析、执行, 直至打印出结果。最后执行完停机指令后, 机器便自动停机。

1.3 计算机硬件的主要技术指标

衡量一台计算机的性能是由多项技术指标综合确定的。既包含硬件的各类性能, 又包括软件的各种功能, 这里主要讨论硬件的技术指标。

1.3.1 机器字长

机器字长是指 CPU 一次能处理数据的位数, 通常与 CPU 的寄存器位数有关。字长越长, 数的表示范围也越大, 精度也越高。机器的字长也会影响机器的运算速度。倘若 CPU 字长较短, 又要运算位数较多的数据, 那么需要经过两次或多次的运算才能完成, 这样势必影响整机的运行速度。

机器字长对硬件的造价也有较大的影响。它将直接影响加法器 (或 ALU)、数据总线以及存储字长的位数。所以机器字长的确定不能单从精度和数的表示

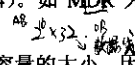
范围来考虑。

1.3.2 存储容量

存储器的容量应该包括主存储容量和辅存储容量。

主存储容量是指主存中存放二进制代码的总数。即

$$\text{存储容量} = \text{存储单元个数} \times \text{存储字长}$$

图 1.11 中的 MAR 的位数反映了存储单元的个数, MDR 的位数反映了存储字的长度。例如, MAR 为 16 位, 表示 $2^{16}=65\,536$, 即此存储体内有 65 536 个存储单元 (可称作 64K 内存, $1K=1\,024$)。如 MDR 为 32 位, 表示存储容量为 $2^{16} \times 32 = 2^{21} = 2\text{ M 位}$ ($1\text{ M} = 2^{20}$)。 

现代计算机中常以字节的个数来描述容量的大小, 因一个字节已被定义为 8 位二进制代码, 故用字节数便能反映主存容量。例如上述存储容量为 2M 位, 也可用 2^{18} 字节表示, 记作 2^{18}B 或 256KB (B 用来表示一个字节)。

同理, 辅存容量也可用字节数来表示, 例如, 某机辅存 (如硬盘) 容量为 4GB ($1\text{G}=1\text{KM}=2^{30}$)。

1.3.3 运算速度

计算机的运算速度与许多因素有关, 如机器的主频、执行什么样的操作、主存本身的速度 (主存速度快, 取指、取数就快) 等等都有关。早期用完成一次加法或乘法所需的时间来衡量运算速度, 即普通法, 显然是很不合理的。后来采用吉普森 (Gibson) 法, 它综合考虑每条指令的执行时间以及它们在全部分操作中所占的百分比。即

$$T_M = \sum_{i=1}^n f_i t_i$$

T_M 为机器运行速度

f_i 为第 i 种指令占全部操作的百分比数

t_i 为第 i 种指令的执行时间。

现在机器的运算速度, 普遍采用单位时间内执行指令的平均条数来衡量, 并用 MIPS (Million Instruction Per Second) 作为计量单位, 即每秒执行百万条指令。如某机每秒能执行 200 万条指令, 则记作 2 MIPS。也有用 CPI (Cycle Per Instruction) 即执行一条指令所需的时钟周期 (主频的倒数) 数, 或用 FLOPS (Floating Point Operation Per Second) 即每秒浮点运算次数来衡量运算速度。

1.4 本书结构

本书介绍计算机组成原理，其内容安排如下：

第一篇：概论，介绍计算机系统的基本组成、应用与发展。

第二篇：计算机系统的硬件结构，让读者站在顶层看计算机系统的硬件结构，包括中央处理器、存储器、I/O 等主要部件以及连接它们的系统总线。其中，除中央处理器比较复杂放在第三篇单独讲述外，其他各部件均在此篇介绍。

第三篇：中央处理器（CPU），CPU 主要由控制单元 CU、算术逻辑运算单元 ALU、寄存器组及它们之间的互连结构组成。本篇对影响 CPU 特性、结构和功能的算术逻辑运算单元及其运算方法、指令系统、中断系统、时序系统等，都将作详细分析。有关控制单元 CU 在第四篇单独介绍。

第四篇：控制单元（CU），本篇在详细分析微操作命令节拍安排的基础上，分别介绍如何用组合逻辑控制及微程序控制两种方法，设计和实现控制单元。

总之，全书按自顶向下，由表及里的层次结构，向读者展示计算机的组成及其工作原理，其目的是为了使读者能先从整体上对计算机有一个粗略的认识，然后逐步深入到机器内核，这将使读者更容易形成计算机的整机概念。图 1.12 形象地描述了上述各章节之间的联系。

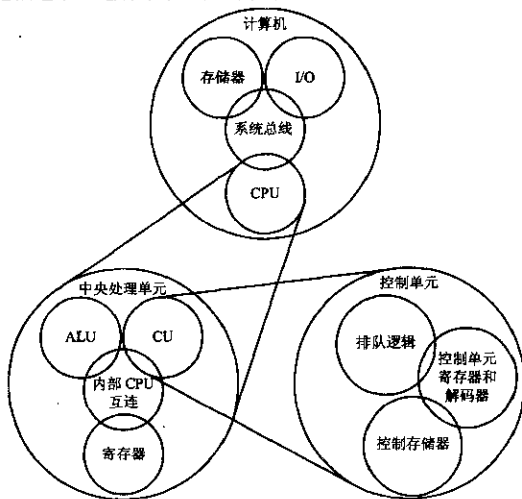


图 1.12 全书各章节之间的关系

思考题与习题

1. 什么是计算机系统、计算机硬件和计算机软件？硬件和软件哪个更重要？
2. 如何理解计算机系统的层次结构？
3. 说明高级语言、汇编语言和机器语言的差别和联系。
4. 如何理解计算机组成和计算机体系结构？
5. 冯·诺依曼计算机的特点是什么？
6. 画出计算机硬件组成框图，说明各部件的作用及计算机硬件的主要技术指标。
7. 解释下列概念：

主机、CPU、主存、存储单元、存储元件、存储基元、存储元、

存储字、存储字长、存储容量、机器字长、指令字长。

8. 解释下列英文缩写的含义：

CPU、PC、IR、CU、ALU、ACC、MQ、X、MAR、MDR、I/O、MIPS、CPI、FLOPS

9. 根据迭代公式 $\sqrt{x} = \frac{1}{2}(y_n + \frac{x}{y_n})$ ，设初态 $y_0=1$ ，要求精度为 ϵ ，试编制求 $\sqrt{x}$ 的解题程

序（指令系统自定），并结合所编程序简述计算机的解题过程。

10. 指令和数据都存于存储器中，计算机如何区分它们？

第二章 计算机的发展及应用

本章简要介绍计算机的发展史以及它的应用领域，旨在使读者对计算机有一感性的认识，最后，本章展望了计算机的未来。

2.1 计算机的发展史

谁也不曾想到，仅仅被当作军事计算工具应用的电子计算机，在半个世纪中竟然会成为改变社会结构、乃至促使人们的工作和生活方式发生惊人变化的不可抛弃的宠儿，真可谓 21 世纪下半世纪科技发展的最有影响的商品。并且它还将继续预示着未来世界的变化，使数千年人类文明史中曾有过的各种神话般的幻想逐渐变为现实。让我们一代一代地为把梦想变为现实而全身心地投入吧！让我们敞开双手去拥抱未来美好世界的到来吧！

2.1.1 计算机的产生和发展

1. 第一代电子管计算机

1943 年，正当第二次世界大战进入后期的阶段，因战争的需要，美国国防部批准了由 Pennsylvania 大学 John Mauchly 教授和 John Presper Eckert 工程师提出的建造一台用电子管组成的 ENIAC (Electronic Numerical Integrator And Computer) 电子数字积分机和计算机的计划，用它来解决当时国防弹道研究实验室(BRL)为开发新武器的射程和检测模拟运算表的任务。当时，由于运算能力不足，该实验室无法在规定的时间内拿出准确的运算表，严重影响了新武器的制作。

ENIAC 于 1946 年交付使用，其首要任务就是完成了一系列测定氢弹可靠性的复杂运算。ENIAC 采用十进制运算，电路结构十分复杂，使用 18 000 多个电子管，运行时耗电量达 150 千瓦，体积庞大，重量达 30 吨，占地面积为 1 500 平方英尺，而且需用手工搬动开关和拔、插电缆来编制程序，使用极不方便，但它却比任何机械计算机快得多，每秒可进行 5 000 多次加法运算。

ENIAC 的出现其意义不仅仅是实现了制造一台通用计算机的目标，而且标志了计算工具进入了一个崭新的时代，是人类文明发展史中的一个里程碑。仅仅半个世纪，计算机已经使人类社会从制造业社会发展到了信息化社会。虽然 ENIAC 于 1955 年正式退役，并陈列于美国国立博物馆供人们参观，但它的丰功伟绩将永远记载在人类的文明史中。

1945 年, ENIAC 的顾问、数学家 von Neumann 在为一个新的 EDVAC (电子离散变量计算机) 所制定的计划中首次提出了存储程序的概念, 即将程序和数据一起存放在存储器中, 使编程更加方便。这个思想几乎同时被 Turing 想到了。

1946 年, von Neumann 与他的同行们在 Princeton Institute 进行高级研究时, 设计了一台存储程序的计算机 IAS, 可惜因种种原因直到 1952 年 IAS 也未能问世。但 IAS 的总体结构从此得到了确认, 并成为后来通用计算机的原型, 图 2.1 就是 IAS 计算机的总体结构示意图。它由几部分组成: 一个同时存放指令和数据的主存储器; 一个二进制的算逻辑部件; 一个解释存储器中的指令并能控制指令执行的程序控制部件; 以及由控制部件操作的 I/O 设备。

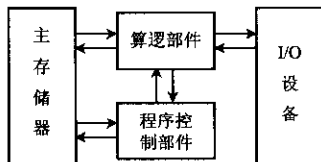


图 2.1 IAS 计算机结构

20 世纪 50 年代, 美国出现了 Sperry 和 IBM 两大制造计算机的公司, 后来又从 Sperry 公司分离出了 UNIVAC 子公司, 他们控制着计算机市场。

1947 年, Eckert 和 Mauchly 共同建立了生产商用计算机的计算机公司, 他们第一个成功的产品是 UNIVAC I (Universal Automatic Computer), 后来 Eckert-Mauchly 公司成为从 Sperry-Rand 公司分离出来的 UNIVAC 子公司, 并继续制造了一系列产品, 如 UNIVAC II 及 UNIVAC 1100 系列产品, 成为科学和商用计算机的主流产品。同时 IBM 公司在 1953 年推出了首台存储程序的计算机 701, 1955 年又推出了 702 机, 使之更适用于科学计算和商业应用, 后来形成了 700/7000 系列, 奠定了 IBM 成为计算机制造商的绝对权威。

自从 ENIAC 问世后, 人类为提高电子计算机性能的欲望从未减退过, 并在 20 世纪 50 年代初, 除美国外, 英、法、俄 (前苏联) 以及日本、意大利等国都相继研制出本国的第一台电子计算机, 我国也于 1958 年研制成自己的第一台电子计算机。可是在这十多年的时间里, 计算机的性能并未出现奇迹般的提高, 它的运算速度每秒仅在数千次至上万次左右, 其体积虽然不像 ENIAC 那样庞大, 但却也占了相当大的空间, 耗电量也很大。直到 20 世纪 50 年代末, 计算机遇上了第一次大飞跃的发展机遇, 其性能出现了数十倍以至几百倍的提高, 这就是用晶体管替代电子管的重大变革。

2. 第二代晶体管计算机

1947 年在 Bell 实验室成功地用半导体硅作基片，制成了第一个晶体管，它的小体积、低耗电以及载流子高速运行的特点，使真空管望尘莫及。进入 50 年代后，全球出现了一场以晶体管替代电子管的革命，计算机的性能有了很大的提高。以 IBM 700/7000 系列为例，晶体管机 7094（1964 年）与电子管机 701（1952 年）相比，其主存容量从 2K 增加到 32K 字；存储周期从 $30\mu\text{s}$ 下降到 $1.4\mu\text{s}$ ；指令操作码数从 24 增加到 185；运算速度从每秒上万次提高到每秒 50 万次。而且 7094 机还采用了数据通道和多路转换器等在当时看来是最新的技术。

尽管用晶体管代替电子管已经使电子计算机的面貌焕然一新，但是随着对计算机性能越来越高的追求，新的计算机所包含的晶体管个数已从一万个左右骤增到数十万个，人们需要把晶体管、电阻、电容等一个个元件都焊接到一块电路板上，再由一块块电路板通过导线连接成一台计算机。其复杂的工艺不仅严重影响制造计算机的生产效率，更严重的是很难避免由几十万个元件导致几百几千万个焊点所造成计算机工作的不可靠性。

随着 1958 年微电子学的深入研究，特别是新的光刻技术和设备的成熟，为计算机的发展又开辟了一个崭新时代——集成电路时代。

3. 第三代集成电路计算机

仔细分析就会发现，计算机的数据存储、数据处理、数据传送以及各类控制功能，基本上都是由具有布尔逻辑功能的各类门电路完成的。而大量的门电路又都是由晶体管、电阻、电容等搭接而成，因此，当集成电路制作技术出现后，可以利用光刻技术把晶体管、电阻、电容等构成的单个电路制作在一块极小（如几个平方微米）的硅片上。进一步发展，实现了将成百上千个这样的门电路全部制作在一块极小（如几个平方毫米）的硅片上，并引出与外部连接的引线，这样，一次便能制作成成百上千个相同的门电路，又一次大大地缩小了计算机的体积，大幅度下降了耗电量，极大地提高了机器的可靠性。这就是人们称作为小规模集成电路（SSI）和中等规模集成电路（MSI）的第三代计算机。其典型的代表为 IBM 的 System/360 和 DEC 的 PDP-8。

1964 年，IBM 推出了一个新的计算机家族 System/360，它打破了 7000 系列在体系结构方面的一些约束。为了推动集成电路技术，改进原来的结构，IBM 在经济和技术上作出了极大的付出，但最终使它占领了大约 70% 的市场，成为计算机领域内的绝对霸主。

System/360 家族中有不同的机型，但它们又都是互相兼容的，即在某台机型上运行的程序可以在这一家族中的另一台机型上运行。它们具有类似或相同的指令系统（家族中低档机的指令系统可以是高档机指令系统的一个子

集), 各机型有类似或相同的操作系统, 而且随着机器档次的提高, 机器的速度、存储器的容量、I/O 端口的数量以及价格都有所增长。

另一代表机器是 DEC 的 PDP-8, 它采用总线结构, 有迷你机之称。它以低价格、小体积吸引了不少用户, 售价仅 16 000 美元, 而当时 System/360 大型机的售价为数十万美元。PDP-8 使 DEC 暴发起来, 使其成为继 IBM 之后的第二人计算机制造商。

从 1946 年的 ENIAC 到 1964 年的 IBM System/360, 历时不到 20 年, 计算机的发展经历了电子管—晶体管—集成电路三个阶段, 通常把它称作计算机的三代。显然, 早期计算机的更新换代主要集中体现在组成计算机基本电路的元器件(电子管、晶体管、集成电路)上。

第三代计算机之后, 人们没有达成定义新一代计算机的一致意见, 表 2.1 列出了硬件技术对计算机更新换代的影响。

表 2.1 硬件技术对计算机更新换代的影响

代	时 间	硬件技术	速度(次/秒)
一	1946~1957	电子管	40 000
二	1958~1964	晶体管	200 000
三	1965~1971	中、小规模集成电路	1 000 000
四	1972~1977	大规模集成电路	10 000 000
	1978~现在	超大规模集成电路	100 000 000

进入到 20 世纪 70 年代后, 把计算机当作高级计算工具的狭隘观念已被人们逐渐摒弃, 计算机已成为一门独立的学科而迅猛发展着, 并且正在影响改变着人类的生活方式, 这是由于微处理器的出现(采用大规模和超大规模集成电路)、软件技术的完善及应用范围的不断拓宽所带来的必然结果。

2.1.2 微型计算机的出现和发展

集成电路技术把计算机的控制单元和算逻单元集成到一个芯片上, 制成了微处理器芯片。1971 年, 美国 Intel 公司 31 岁的工程师霍夫研制成世界上第一个 4 位的微处理器芯片 4004, 集成了 2 300 个晶体管。随后, 微处理器经历了 4 位、8 位、16 位、32 位和 64 位几个阶段的发展, 芯片的集成度和速度都有很大的提高。与此同时, 半导体存储器的研制也正在进行, 1970 年, Fairchild 制作了第一个存储芯片, 该芯片大约只有一个磁心这么大, 却能保存 256 位二进制信息, 但是每位的价格高于磁心。1974 年后, 随着半导体存储

器价格的迅速下降,位密度的不断提高,存储芯片的容量经历了1K、4K、16K、64K、256K、1M、4M、16M 和 64M 这几个阶段,每个新的阶段都比过去提高4倍的容量,而价格和访问时间都有所下降。

总之,芯片集成度不断提高,从在一个芯片上集成成百上千个晶体管的中、小规模集成电路,逐渐发展到了能集成成千上万个晶体管的大规模集成电路(LSI)和能容纳百万个以上晶体管的超大规模集成电路(VLSI)。微芯片集成晶体管的数目验证了Intel公司的缔造者之一Gordon Moore提出的“微芯片上集成的晶体管数目每三年翻两番”的规律,这就是人们常称的Moore(摩尔)定律。

由于微处理器芯片和存储器芯片的出现,微型计算机也随之问世。如1971年用4004微处理器制成了MCS-4微型计算机。70年代中期,8位微处理器8008、8080、R6502、M6800、Z80等相继出现,并用R6502制成了Apple II微型计算机,用Z80制成了CROMEMCO 80微型计算机等。

最值得一提的是世界上第一大微处理器的制造商Intel,其典型产品有:

- 8080: 世界上第一个8位通用的微处理器,1974年问世。
- 8088: 集成度达2.9万管,主频4.77MHz,字长16位(外部8位),又称准16位,地址20位,采用4个字节指令队列,被IBM首台微机(IBM PC)选用,1979年问世。
- 8086: 16位,2.9万管,地址20位,采用6个字节指令队列,指令系统与8088完全兼容,1978年问世。
- 80286: 16位,13.4万管,6MHz,地址24位,可用实际内存16KB和虚拟内存1GB,1982年问世。
- 80386: 32位,27.5万管,12.5MHz、33MHz、50MHz,地址32位,4GB实际内存,64TB($T=2^{40}$)虚拟内存。其性能可与几年前推出的小型机和大型机相比,1985年问世。
- 80486: 32位,120万管,25MHz、33MHz、50MHz,4GB实际内存,64TB虚拟内存,引用更加复杂的Cache技术和指令流水技术,速度比80386快一倍,性能指标高于80386 3~4倍,1989年问世。
- Pentium(80586): 64位,310万管,66MHz、100MHz,采用超标量技术,使多条指令可并行执行,速度比80486高出6~8倍,1993年问世。
- Pentium pro(P6): 64位,550万管,133MHz、150MHz、200MHz,采用动态执行RISC/CISC技术、分支预测、指令流分析、推理性执行和二级Cache等技术,1995年问世。
- Pentium II: 64位,550万管以上,233MHz、300MHz、400MHz、450MHz,

读性，但它们仍是面向机器的，即不同的机器各自有不同的汇编语言。为了使这种符号语言转变成机器能识别的语言，人们又创造了汇编程序，它能把汇编语言翻译成机器语言。

为了摆脱对具体机器的依赖，在汇编语言之后又出现了面向问题的高级语言。使用高级语言编程可以不了解机器的结构，高级语言的指令通常是一个英语词汇，词义本身反映出命令的功能，它比较接近人们习惯用的自然语言和数学语言，使程序具有很强的可读性。高级语言的发展经历了几个阶段。初级阶段的代表语言是 1954 年问世的 FORTRAN，它主要面向科学计算和工程计算。第二阶段可视为结构化程序设计阶段，其代表是 1968 年问世的 PASCAL 语言，它定义了一个真正的标准语言，按严谨的结构化程序编程，具有丰富的数据类型，写出的程序易读懂，易查错。第三阶段是面向对象程序设计阶段，其代表语言是 C++。近年来随着网络技术的不断发展，又出现了更适应网络环境的面向对象的 JAVA 语言，而且随着 Internet 技术的发展和应用，JAVA 语言越来越受到人们普遍欢迎。

为了使高级语言描述的算法在机器上执行，同样需有一个翻译系统，于是产生了编译程序和解释程序，它们能把高级语言翻译成机器语言。

可见，随着各种语言的出现，汇编程序、编译程序、解释程序的产生，逐渐形成了软件系统。

另一方面，随着计算机应用领域的不断扩大，外部设备的增多，为了使计算机资源让更多用户共享，又出现了操作系统。操作系统能协调管理计算机中各种软件、硬件及其他信息资源，并能调度用户的作业程序，使多个用户能有效地共用一套计算机系统。操作系统的出现使计算机的使用效率成倍地提高，并且为用户提供了方便的使用手段和令人满意的服务质量。如 DOS、UNIX 和 Windows 等等。

此外，一些服务性程序如装配程序、调试程序、诊断程序和排错程序等也逐渐形成。特别是随着计算机在信息处理、情报检索及各种管理系统中应用的发展，要求大量处理某些数据，建立和检索大量的表格。这些数据和表格按一定的规律组织起来，使用户使用更方便，于是出现了数据库。数据库和数据管理软件一起便组成了数据库管理系统。而且随着网络的发展，又产生了网络软件等等。

以上所述的各种软件均属系统软件，而软件发展的另一个主要内容就是应用软件。应用软件种类繁多，它是用户在各自的行业中开发和使用的各种程序。如管理财务的各种财务软件、办公用的文字处理和排版软件、帮助管理日常业务工作和图文报表的“电子表格”和“数据库”软件、帮助工程设计的 CAD 软件以及各种实用的网络通信软件等等。

软件发展有以下几个特点：

（1）开发周期长

研制一个软件往往因其规模庞大而需较长的开发周期。例如美国穿梭号宇宙飞船的软件包含 4 000 万行目标代码，倘若一个人一年开发一万行程序，则需集中 4 000 人花一年时间才能完成。而且要做到 4 000 人的默契配合，涉及种种技术问题的协调，如分析方法、设计方法、形式说明方法、版本标准等等都得有严格的规范，其难度远远超过自动化程度极高的硬件制造。

（2）制作成本昂贵

超大规模集成电路技术给硬件制造业带来巨大利益，使硬件的价格不断下降，使一台普通的微型计算机的价格与一台彩色电视机的价格相当，而且还在下降。可是软件的开发完全依赖于人工，致使软件开发成本费不断上涨，在美国软件成本约占计算机系统总成本的 90%，已成为司空见惯的现象。

（3）检测软件产品质量的特殊性

一种软件在刚开始推出时，主要实现其面向领域所需的核心功能，之后逐步集成大量的附加功能。也就是说，要完善一个软件产品，必须在应用过程中不断加以修改、补充。只有使用了一定时间后，才能对软件产品质量进行确定。

尽管软件技术兴起和发展比硬件晚，而且其发展速度比不上硬件快（如微处理器的性能以 Moore 定律所述的几何级数增长），但是仍可以说，如果没有当今的软件技术，计算机系统和应用的发展也不会有今天这样的成就。客观地说，软件的发展不断激励着微处理器和存储器性能的增长。

世界各国当前都十分重视软件人才的培养和软件产业的形成，但实际上它们都很难与当前计算机应用普及的广度和深度相适应。也正因为如此，有些软件开发商瞄准了特定的市场，一旦在性能、质量占到上风时，就会很快积聚财富，成为新的世界富商。例如美国微软公司十来年的发展就超过传统工业（如汽车制造业），同样微软的组建者也就很快成为现代世界最大富商之一。

在二三十年软件开发的实践中，人们对软件开发也逐渐有了较深刻的认识，逐渐体会到软件不是简单的编写程序，欲开发成一个优良的软件，和开发其他产品一样，必须明确开发要求，然后作可行性分析，确定基本方法，进行需求分析，再深入到用户核准需求，取得一致意见后才能进入软件设计阶段。因此，程序只是完成整个软件产品的一个组成部分，软件生存周期的各个阶段都要得出一个或几个组成部分，它们都是以文档资料形式存在。正如著名专家 Boehm 曾经指出：“软件是程序以及开发、使用和维护程序需要的

所有文档。”可见软件开发不是某种个体劳动的神秘技巧，它是个组织良好、管理严密、各类人员协同配合、共同完成软件工程的全过程。只有这样才能保证软件工程的顺利完成，并能节省大量开发费用。否则将会陷入事倍功半、长期无法正常运行的困境。

2.2 计算机的应用

自 ENIAC 问世后将近 30 余年的时间里，计算机一直被作为大学和研究机构的娇贵设备。20 世纪 70 年代中期后，大规模集成工艺日趋成熟，微芯片上集成的晶体管数一直按每三年翻两番的 Moore 定律增长，微处理器的性能也按此几何级数提高，而价格也以同样的几何级数下降。现在你用 1 000 美元就可以买到与 10 年前 IBM 大型机相当的个人计算机，这台机器包括微处理器、存储器以及其他芯片在内大约有 1 亿个晶体管。以至于以前需花数百万美元的机器（如 80M FLOPS 的 CRAY）变得价值仅为数千美元左右（而此类机器的性能可达 200M FLOPS），至于对性能不高的微处理器芯片而言，仅花数美元就可购到。因此，人们终于使计算机走出了实验室而渗透到各个领域乃至走进普通百姓的家中。当然，除了计算机价格猛跌外，计算机软件技术日趋完善也是计算机获得广泛应用的重要原因。尤其是近年来计算机技术和通信技术相互融合，出现了构造全球的 Internet 网，更使计算机的应用范围从科学计算、数据处理等传统领域扩展到办公自动化、多媒体、电子商务、虚拟工厂、远程教育等，遍及政治、经济、军事、科技、以及个人文化生活和家庭生活的各个角落，不久的将来，计算机将成为犹如人们离不开水和电一样的宠物。

2.2.1 科学计算和数据处理

1. 科学计算

科学计算一直是计算机的重要应用领域之一。其特点是计算量大和数值变化范围大。在天文学、量子化学、空气动力学和核物理学等领域都要依靠计算机进行复杂的运算。例如人们日常生活难以摆脱的天气预报，当你只要知道第二天的气候变化时，采用 1MIPS 的计算机顷刻时间便可获得。倘若想预报一个月，乃至一年的气候变化，使各地提前做好防汛、防旱、防爆等工作，则 100MIPS 或更高的计算机才能满足。现代的航空、航天技术，如超音速飞行器的设计、人造卫星和运载火箭轨道的计算，也都离不开高速运算的计算机。

此外，计算机在其他学科和工程设计方面，诸如数学、力学、晶体结构

分析、石油勘探、桥梁设计、建筑、土木工程设计等领域内，都得到了广泛的应用。

2. 数据处理

数据处理也是计算机的重要应用领域之一。早在 20 世纪五六十年代起，人们就把大批复杂的事务数据交给了计算机处理，例如政府机关公文、报表和档案；大银行、大公司、大企业的财务、人事、物料，包括市场预测、情报检索、经营决策、生产管理大量的数据信息，都由计算机收集、存储、整理、检索、统计、修改、增删等，并由此获得某种决策数据或趋势，供各级决策指挥者参考。

2.2.2 工业控制和实时控制

通过各种传感器获得的各种物理信号经转换为可测可控的数据信号后，再经计算机运算，根据偏差，驱动执行机构来调整，便可达到控制的目的。这种应用已被广泛用于冶金、机械、纺织、化工、电力、造纸等行业中。

目前的工业控制远比 20 世纪六七十年代先进得多。新型的工业自动控制系统以标准的工业计算机软、硬件平台构成集成系统，取代了传统的封闭式系统，具有更强的适应性，更好的开放性，更易于扩展，更经济、更短的开发周期等显著优点。通常将工控系统分为三层：控制层、监控层和管理层。控制层是最下层，它是通过各种传感器来获得各种有效信号的。监控层下连控制层，上连管理层，它不但实现对现场的实时监测与控制，而且常在自动控制系统中完成上传下达，组态开发的重要作用。特别是组态软件的出现，它使数据采集、过程控制变得十分简单，它为用户提供良好的开发界面和简便的使用方法，用各种软件模块可以非常容易地实现和完成监控层的各种功能。就目前发展趋势而言，工业控制的应用已经向控管一体化方向发展，利用网络技术，通过传感技术和多媒体技术，操作者可以在控制室内通过大屏幕显示，使你领受到各车间、各工位、各部门的生产运行情况，并可直接由控制室发出各种控制命令，指挥全厂正常工作。

在军事上，导弹的发射及飞行轨道的计算控制、先进的防空系统等现代化军事设施，通常也都是由计算机构成的控制系统，其中包括雷达、地面设施、海上装备等。例如将计算机嵌入到导弹的弹头内，利用卫星定位系统，将飞行目标和飞行轨迹事先存储在弹载计算机内，导弹在飞行中对实际飞行轨迹进行不断修正，直接袭击目标，其命中率几乎接近 100%。美国在海湾战争以及后来的军事冲突中，计算机实时控制技术发挥了极为突出的作用。

2.2.3 网络技术的应用

促使计算机网络诞生的最早动机在于实现硬件资源的共享。当时计算机十分昂贵,人们希望能远距离利用计算机,因此在 1954 年第一次实现了将穿孔卡上的数据从电话线发送到远方的计算机来完成运算,这可以说是计算机网络的雏形,可见网络技术的基礎是计算机技术与通信技术的结合。

1992 年美国政府提出了“国家信息基础设施计划”,1993 年西方七国提出“全球信息基础设施计划”,整个世界随着通信技术和计算机技术的结合,在新的世纪到来前,一个崭新的全球性的 Internet 网正在形成,并正以更新的姿态屹立在世界的顶端。由于全球网络化消除了人们之间因时间、距离和地理界限所形成的障碍,从而使各国人们在技术交流、商品交换、文化传递、感情沟通等方面变得十分迅捷,十分方便。如果再有性能良好的语言翻译机(实际上目前已经有翻译机了),那么原有的隔阂和障碍可能会全部消失。正因如此,Internet 的发展规模和速度达到了惊人的程度,人们称之为“新 Moore 定律”,全球上网量每六个月翻一番。至 1998 年止,全世界上网的计算机已达 5 000 万台,上网人数已达 1.5 亿。实际上从 Internet 开始至今,仅为 4 年。相比之下,全世界 5 000 万用户拥有电视机花了 13 年,拥有收音机经历了 38 年,拥有电话的时间就更长了。可以断言,全球网络化不仅改变着商业经济、工业生产、科技发展,还必将影响着人们的工作、娱乐和生活,它正在改变着整个世界。

网络应用涉及方方面面,在此仅举几个例子。

1. 电子商务

电子商务的含义是任何一个组织机构可利用 Internet 网络来改变他们与客户、供应商、业务伙伴和内部员工的交流,也可以认为是消费者、销售者和结算部门之间利用 Internet 完成商品采购和支付收款的过程。例如某企业可以通过在 Internet 上的网页向全球发布推出的商品,并向他的各地代理商发出各种指令;当某客户欲购此商品时,他可以通过网上直接与生产企业联络,也可与各地代理商联系,进一步了解该商品的性能,并将其姓名、地址、个人电子帐号及送货要求等告诉卖主。企业或推销者通过 Internet 与银行联络,查询核实该客户的资金状况,并通过协定的支付方式由银行实行电子交付,而商品则由企业推销者直接送到客户手中。这种简洁、快捷、可靠的商品销售方式,可从根本上改变传统的销售方式。它可以不要传统意义上的铺面,而直接用电子铺面来取代;它可以一夜之间将自己的品牌通告全世界;它可以

实现公平竞争,小企业不必惊怕大企业的广告效应,大企业也不必顾虑小企业的快速应变能力,各自都可以通过网上信息进行竞争;它可以取消纸币交易的各种弊端,完全实现电子钱币交换;它可以减少很多中间环节,以最高效率、最省人力、最广泛的市场实现商品的全球交换。目前世界各国都在蓬勃发展电子商务,我国的电子商务也在一些城市陆续展开。

2. 网络教育

传统的老师讲、学生听的课堂授教模式随着全球网络化的发展,将会在“知识爆炸”^①时代逐渐被淘汰或更新。旧的教学模式最大缺点是,作为受知主体的学生在教学过程中自始至终处于受灌输的被动地位,其主动性、积极性难以发挥,学生无法主动探索,主动发现社会上、国际上的信息资源,很难培养具有“信息能力”的劳动者。因此,不利于创新能力的形成和创新型人才的成长。此外,这种模式受场地、空间的限制,投资大,受教育面有限,不能适应各种学科的终身教育和全面教育。

基于 Internet 教育网络的建立,学生受教可以不受时间、空间和地域的限制,通过网络伸展到全球的每个角落,建立真正意义上的开放式的虚拟学校,每个学生可以在任意时间、任意地点通过网络自由地学习。不论学生的贫富贵贱都可以找到第一流老师的指导,都可以向世界最权威的专家请教,都可以从世界任何角落获取最新的信息和资料。到那时可以说,任何人都享有高等教育和终身教育的可能。这种基于网上的教育模式,不仅美、英、日等发达国家在积极实施,我国在有条件的地区和省市也正在加速启动建设 Internet 教育网络,实现由传统教育体制、教学模式向全新教育体制、教学模式的转变,实现教育的重大革新,满足 21 世纪人才培养的需求。

3. 敏捷制造

随着全球信息技术的发展,对工业制造业的制造模式和企业的组成和管理模式也产生了极大影响,新的被称为 21 世纪的制造模式——敏捷制造由此而生。敏捷制造由两部分组成:敏捷制造的基础结构和敏捷制造的虚拟企业。前者为形成虚拟企业提供环境和条件,后者对市场不可预期的变化作出迅速响应。

当出现某种市场机遇时,由敏捷制造基础结构所形成的虚拟企业,通过

^① 英国技术预测专家詹姆斯·马丁测算结果表明:人类知识到 19 世纪 50 年代增加了一倍;20 世纪初是每 10 年增加 1 倍;20 世纪 70 年代则是每 5 年增加 1 倍;近 10 年大约每 3 年增加 1 倍,故称作“知识爆炸”。

网上联络若干个具有核心资格的组织者，他们以各自的资金、技术、厂房、设备等优势，通过国家的法律和彼此的合同，组建成一个虚拟企业。该企业不必有集中的办公场地和固定的组织机构，完全通过网络实现产品的技术设计和制造，以及网上销售和网上服务，充分发挥各自的优势，以最优化的结合，在最低的成本核算下获取最大的利润。这种虚拟企业是在敏捷制造基础结构环境下形成的独立的、实体性的、社会性的团体，同时又是一个动态的联盟，他们可以根据市场的变化和要求，可以解散原来的虚拟企业，而与新的伙伴组成新的虚拟企业。可见网络技术的发展对社会原来的固定企业结构形式构成了严峻的挑战。

以上仅就几个方面列举了全球网络化对整个社会经济、文化、教育、工业制造等方面的影响。实际上由于网络技术的发展，现在已经形成了虚拟图书馆、虚拟医院、虚拟商场、虚拟娱乐场所等等。事实上 Internet 早已从对经济的干预，发展到对政治的干预。例如美国总统克林顿，从他的绯闻到国会弹劾，直至幸免弹劾，都与网民的直接参与分不开。又如非洲尼日利亚总统大选，两名主要的候选人都为选举分别建立了各自的网站。再如在英国戴安娜王妃和英国王室的众多网站上，充满政治性的窃窃私语已司空见惯。可以说全球的网络化必将进一步改变着整个世界。

2.2.4 虚拟现实

虚拟现实是利用计算机生成的一种模拟环境，通过多种传感设备使用户“投入”到该环境中，实现用户与环境直接进行交互的目的。这种模拟环境是用计算机构成的具有表面色彩的立体图形，它可以是某一特定现实世界的真实写照，也可以是纯粹构想出来的世界。这类技术虽然早在 20 世纪 60 年代初就开始研究，但只有在计算机技术迅速发展的 90 年代，各种传感设备以及计算机价格的不断降低，软件系统的日趋完善，如实时三维图形生成及显示、三维声音定位与合成、环境建模等等技术的发展，才有可能使虚拟现实技术获得迅速发展和广泛应用。虚拟现实在军事、教育、航天、航空以及娱乐、教育、生活中的应用，不仅会改变人们的思维方式和生活方式，而且必将导致一场重大的技术革命。

这里列举两个例子可看出虚拟现实的巨大魅力。

虚拟演播室近年来已成为影视制作的热点，它综合运用现代计算机图形和图像处理、计算机视觉和现代影视技术，将摄像机拍摄的图像实时地与计算机三维虚拟背景或另一地点实拍的背景，按统一的三维透视成像关系进行合成，从而形成一种新的影视节目，它的效果是传统影视制作无可比拟的。

在虚拟演播室里,演员可以在没有任何道具的舞台上演戏,然后根据剧情需要用计算机制作的画面进行合成。不仅如此,演员也可以是虚拟的,可以根据事先拍好的演员镜头,利用演技数据,用计算机图形学技术制作演员的特定动作,这对于一些特技的制作,显得尤为重要。这种在虚拟演播室制作的影视剧,大大降低了制作成本,缩短了制作时间,并且可以制作更有魅力的艺术作品。

飞行员的仿真虚拟现实系统与汽车驾驶员的虚拟现实系统也都广泛应用了虚拟现实技术。在飞行仿真系统中,要形成真实的飞行环境和飞行员的真实感觉。例如在环境图像生成中,以 50Hz 的频率生成彩色图像,而且具有纹理,还有亮点、透明、天气效果(如雾、雨、雪、晴、云等)、非线性图像映射、碰撞检测、高山地形、细节模拟等等。如飞机着陆时跑道灯应按飞机着陆角度不同而变换颜色,并能确认飞机与跑道上其他飞机甚至建筑物的相互距离。又如在虚拟现实仿真中,飞行员必须体验到真实飞行的感觉,尤如真正在一个真实飞机的机舱里,每个仪表都必须如在真实环境下工作,油表指示必须反映虚拟引擎对油的使用率,并且还必须精确地反映动力和温度。在飞机接触跑道时,还必须有真实的冲击感和震动感。显然对于价值数十万美元的飞机来说,让飞行员在虚拟现实仿真系统中训练是最合算的,它既不危及人的生命安全,又不损坏飞机,也不造成公害。所以各类仿真模拟训练器都已被广泛应用。

2.2.5 办公自动化和管理信息系统

顾名思义,办公自动化是利用计算机及自动化的办公设备来替代“笔、墨、纸、砚”“文房四宝”及办公人员的部分脑、体力劳动,从而提高了办公的质量和效率。例如,利用计算机来起草文件、登录文件;利用计算机来安排日常的各类公务活动,包括会议、会客、外出购票;利用计算机来收集各类信息,将各类信息以电子数字形式存于数据库内,并可随时进行查询、检索及修改。一个完整的办公自动化系统将包括文秘、财务、人事、资料、后勤等各项管理工作。近年来由于 Internet 的应用,将计算机、自动化办公设备与通信技术相结合,使办公自动化向更高层次发展。例如电子邮件的收发,远距离会议或电视会议、高密度的电子文件、多媒体的信息处理等还将会普遍获得应用。

与办公自动化相应的信息管理系统是企业管理信息系统。由于信息技术的飞速发展,造就了一个统一的全球市场,导致世界范围市场的激烈竞争。占领并主宰市场的关键在于如何不断开发独占性的产品,不断降低成本,以

质优价廉的产品投入市场。实现这个目标离不开信息管理，通过信息的获取、分析，开发独占性产品；通过优化的信息管理，实现信息资源的有效利用。

利用计算机参与人脑的辅助工作非常普遍，而且还在不断开拓新的领域，例如计算机辅助工艺规划 CAPP (Computer Aided Process Planning) 计算机辅助工程 CAE (Computer Aided Engineering)，乃至计算机的辅助教学 CAI (Computer Assisted Instruction) 等都越来越得到广泛的应用。

3. CIMS (Computer Integrated Manufacturing Systems)

计算机集成制造系统 CIMS 是信息技术和现代管理技术改造传统制造业、加强新兴制造业、提高企业市场竞争能力的一种生产模式。具体而言，以企业选定的产品为龙头，在产品设计过程、管理决策过程、加工制造过程、产品质量管理和控制等过程中，采用各种计算机辅助技术和先进的科学管理方法，在计算机网络和数据库的支持下，实现信息集成，进而使企业优化运行，达到产品上市快、质量好、成本低、服务好的目的，以此提高产品的市场占有率和企业的市场竞争能力。显然，要形成计算机集成制造系统的企业，必须广泛的采用 CAD/CAE/CAPP/CAM，并且已经建立了企业的 MIS (Management Information System) 系统，只有通过生产、经营各个环节的信息集成，支持了技术集成，并由技术集成进入技术、经营管理和人员组织的集成，最终达到物流、信息流、资金流的集成并优化运行，才能提高企业的市场竞争能力和应变能力。

2.2.7 多媒体技术

多媒体技术是计算机技术和视频、音频及通信等技术集成的产物。它是用来实现人和计算机交互地对各种媒体(如文字、图形、影像、音频、视频、动画等)的采集、传输、转换、编辑、存储、管理，并由计算机综合处理为文字、图形、动画、音响、影像等视听信息而有机合成的新媒体。因此它既可以将原来仅仅能体现或保存一种媒体的设备或手段，转换为由计算机集成。例如传统的音响设备只能录音、放音；档案库只能存档文件；图书馆只能收藏书籍；电视只能提供音频和视频信息；电话只能传递语音等等。而今用多媒体技术使声、图、文合成后全部集成到计算机中。同时，利用计算机还可以制作、创造新的媒体信息，例如合成音乐、电子动画等等。它不但使我们社会显得格外绚丽多彩，生活显得格外富有幻想，而且它还会对政治、经济、军事、工业、环境等都产生巨大的影响。如上所述飞行仿真模拟，虚拟演播室等，都离不开多媒体技术。它的深远意义还会影响未来计算机人工智能技术的发展。因此，有关多媒体技术的研究和应用也是当前计算机技术的热点之一。

2.2.8 人工智能

人工智能是专门研究如何使用计算机来模拟人的智能的技术。尽管经过了

近半个世纪的努力，被人们称之为“电脑”的计算机与“人脑”相比，仍无法相提并论。如集成度达 1 亿个晶体管的处理器芯片与人类的 $10^{11} \sim 10^{12}$ 个神经元相比，简直无可比拟，因每个神经元远不是一个晶体管，很可能相当于一台高速运行的处理器。可见“电脑”要真正模拟“人脑”，特别是要使电脑具有人的经验知识以及通过联想、比拟、推断来作出决策的功能，至少从目前来看还有相当距离。

尽管如此，人们还是想尽一切办法，赋予“电脑”一部分人的智力功能，并且还在不断扩大和增强这种智力。近年来在模式识别、语音识别、专家系统和机器人制作方面都取得了很大的成就。

模式识别是指对某些感兴趣的客体作定量的或结构的描述，研究一种自动生成技术，由计算机自动地把待识别的模式分配到各自的模式类中去。由此技术派生的图像处理技术和图像识别技术已被广泛应用。例如对人体细胞显微图像分析，可确定内脏是否发生病变；对动、植物细胞显微图像分析，可确定环境是否被污染；对地表植物经遥感图像分析，可判断作物长势的预测等等，诸如此类包括公安系统的指纹分辨及身份、证件、凭证鉴别等等。

文字/语音识别、语言翻译是人工智能的一重要应用领域。自计算机问世后，人们就企图让计算机来承担文字、语言的翻译工作，实际上让计算机正确认识文字和语音，正确理解自然语言，实现正确的语言翻译还是十分困难的。虽然经过几十年的努力，目前已有了很大的进展，如手写体的计算机输入系统已被广泛使用，语音录入计算机的软件也开始在市场上问世，当然它的正确识别率还有待进一步提高。此外，在自然语言理解的基础上研制成的文字/语言翻译机也在陆续问世，但离人们的实用要求还有一定距离，不过这些技术的突破看来是指日可待的，使计算机能听、会看、会说的时代已经不是很遥远了。

专家系统是人工智能的又一重要应用领域。它是利用计算机构成储存量极大的知识库，把各类专家丰富的知识和经验，以数据形式储存于知识库内，通过专用软件，根据用户输入查询的要求，向用户作出所要求的解答。这种系统早已被广泛应用在医学、工程、军事、法律等领域，尤其是近年来 Internet 的出现，更可以构成远程虚拟医疗、虚拟课堂、虚拟考试等。

机器人的出现也是人工智能领域的一项重要应用。通常人们让机器人作一些重复性的劳动，特别是在一些不适宜人们工作的劳动场所，机器人的应用显得格外重要。例如海底探测，人在海底的时间是非常有限的，如果让机器人进行海底探测就方便多了。可以让机器人配上摄像机，构成它的眼睛；配上双声道的声音接收器，变成它的耳朵；再配上合适的机械装置，使它可以活动、触摸、承受各种信息并直接送到计算机进行处理，这样它就可以模仿人完成海底

探测。现在还有一些更高级的“智能机器人”，具有一定的感知和识别能力，还能简单的说话和回答问题。总之，随着科学技术的不断发展，更高级的机器人将会不断出现。

2.3 计算机的展望

从 1946 年 ENIAC 问世至今，50 多年来计算机技术的进步推动了计算机的发展和广泛的应用，使计算机在人类的全部活动领域里占有极为重要的地位。从超级巨型机到心脏起搏器，从电话网络到汽车的汽化器无处不在，无所不及，几乎能填补甚至取代各类信息处理器，成为人类最得力的助手。

在即将进入 21 世纪的岁月里，世界上不少科学家预言，到了 2046 年人类社会几乎所有的知识和信息将全部融入于电脑空间，而任何人在任何地方任何时间都可以通过网络，对所有的知识和信息进行在线获取。这个预测是大家所希望的，也是必定会实现的。电脑空间将会为崭新的信息方式、娱乐方式和教育方式提供基础，并会提供新层次的个人服务和健康保健，最大的受益将是人们可以在远距离与他人进行全感知的交流。

实现上述目标在技术上还需有很大的突破，至少在电脑的容量、速度及网络的带宽上，目前的差距还是很大的。

最理想的信息处理、存储机构还是人脑，它具有每秒 10^{15} 的计算速度和 10^{13} 字节的存储容量，何况它还具有自组织、自适应、自联想、自修复的智能特性。因此，人们期待着下世纪有超级智能计算机的出现。

20 世纪七八十年代，人工智能的研究曾一度出现过高潮。日本首先提出研究具有高度智能的计算机。但后来人们发现，高度智能计算机的实现比早期创始者的想像要难得多，不过人们仍然还在孜孜不倦地探索着。

由于受物理极限的制约，VLSI 晶体管本身的线宽大约在 0.05 微米量级，因此 Moore 定律不可能无限期延续。另一方面，在芯片集成度不断提高的同时，其制造成本也在不断提高，即在微电子工业发展中还遵循另一规律：“每代芯片的成本大约为其前一代芯片成本的两倍”。目前，建造一个生产 0.25 微米工艺芯片的车间大约需 20~25 亿美元，而使用 0.18 微米工艺时，费用将跃到 30~40 亿美元。按这一成本几何比递增的情况发展，在未来 10 年内，该费用将超过 100 亿美元。因此，综合考虑技术和经济的限制，Intel 的技术负责人认为：“微电子工业以 Moore 定律几何比增长的态势最多可延续到 2017 年。”人们通过新的材料如镓砷化物以及正在开发的量子晶体管，仍然希望在高速低能耗方面开拓新的进展。此外，现代计算机采用高并行度的体系结构，通过大量高速处理器的高带宽局域网连接，使它具有类似人脑的高并行性的本

质。所以,大多数专家对于计算机能力增长的前景都持有乐观的态度。可以比较确切地说,实现人类级的智能所需的硬件可能在下世纪的前四分之一的时间具备,这与20世纪70年代计算机的硬件能力只够得上“昆虫级”的智能相比,显然应该更充满信心。

超级智能电脑不仅需要硬件支撑,而且还必须有软件支持。模拟大脑功能创建超级智能电脑,除了通过足够的硬件能力和适应电脑学习的软件外,还需有足够的初始体系结构和丰富的感官输入流。当前的技术对后者已经很容易满足,如采用视觉照相机、扬声器和各类触觉传感器,能保证特定的实时世界信息流入电脑。而前者则更难实现,因为大脑并非一开始就是一片空白。它有一个遗传可编码的初始结构,存在着神经皮层可塑性、大脑皮层的相似性及进化的论点。这些问题的解决必须随着神经科学的进一步发展,在对人脑的神经结构和它的学习算法了解得足够多的前提下,在具有很强计算能力的计算机上实现复制。科学家的估计大约在今后15年内,采用当前的设备支持输入输出渠道,对人脑继续研究,发现新的电脑学习方法和对新神经科学的深入研究,超级智能电脑的出现是势不可挡的必然趋势,只是时间问题。

进入新世纪除了人们继续追求超级智能计算机的问世外,更引起人们注目的是价格低廉的、使用方便的、体积更小的、外型多变的、具有人性化的电脑的研究和应用。

虽然电脑强大的功能促使它去处理相当多的事,但至今还存在着不尽人意的缺点。因此,普及面仍未达到应有的程度。其原因主要在于对绝大多数人而言,还不能非常方便地对它进行操作,而且很难适应各种场合的需要。因此,除了继续提高芯片主频外,在输入输出方式上应有更多的性能突破。输入输出方式将更多样化,更人性化。除了手写分辨率和速度进一步提高外,语音输入输出将随时可见,包括汽车、家电、电话、电视、玩具、手表等等。而且还可用人的手势、表情、眼睛瞳孔的位置、甚至利用人体的气味、体温来控制输入。三维图像输出将能实时地合成真实的视频图像,包括完整的戏剧电影,还允许计算机合成的图像和人面对面交谈。利用平面液晶显示器将如同眼镜一般放在眼睛前面,构成可移动的电脑。

电脑的外形及尺寸将随着不同的对象和环境而变化,甚至朝着个人量身定做的方向发展。特别是嵌入式的电脑,可以遍及汽车、房间、车站、机场及各种建筑场,使用者利用随身携带的信息操作器具,不需做任何连接方式,利用红外线传输方式,随时从公共场所服务器主机上接收所需的信息,包括个人的电子邮件等。尤其是个人身上穿戴的计算机连同身上网络,可以随时随地为你的健康、安全以及帮助你在复杂的物理空间环境中工作,如汽车、飞机驾驶等。

在普及型的电脑发展的同时，大型系统也将获得巨大发展，将由低价、通用的多处理机组成的群机系统来替代单一的大型系统。在这个群机系统中，每个计算机通过快速的系统级网络（SAN）和其他计算机通信。群机系统可以扩展到上千个结点，对于数据库和即时事务处理(OLTP)的应用，群机能像单机一样地运转。群机能开发隐含在处理并行多用户中或在处理包含在多个存储设备的大型查询中的并行性。一个具有几十个结点的 PC 群机系统，每天可执行十亿多次事务处理，比目前最大的大型机吞吐量还大。还可预知科学计算将在高度专用、类似 CRAY 的多向量结构的计算机上运行。

前面提到的网络带宽问题，到 2046 年，每光波长携带几个 G 的光纤将会很普遍地进入到广大家庭用户，到那时任意宽的带宽都可以传输。它们将为你的电话、可视电话、电视、网络访问、安全监控、家庭能源管理以及其他各种服务。

虽然我们不能对未来的电脑预知得那么清晰，那么准确，但是，仅就上述的描述，也就可以想像几十年后，电脑给人类带来的绚丽多彩的生活和人类社会的美好憧憬决不是幻想。

希望年轻的学者能在自身的数十年奋斗中，为电脑的发展和应用发挥自己的聪明才智，为人类社会的进步增添一砖一瓦。

第二篇 计算机系统硬件结构

计算机硬件系统由中央处理器、存储器、I/O 以及连接它们的系统总线组成。本篇介绍系统总线、存储器及 I/O 三部分，中央处理器将在第三篇单独讲述。

第三章 系统总线

本章着重介绍系统总线的基本概念及其分类、结构和总线控制逻辑。要求读者能对系统总线在计算机硬件结构中的地位和作用有所了解。

3.1 总线的基本概念

计算机系统的五大部件之间的互连方式有两种，一种是各部件之间通过单独的连线，叫做分散连接；另一种是将各部件连到一组公共信息传输线上，叫做总线连接。

早期的计算机大多数用分散连接方式，如图 1.7 所示。它是以运算器为中心的结构，其内部连线十分复杂，尤其是当 I/O 与存储器交换信息时，都需经过运算器，致使运算器停止运算，严重影响了 CPU 的工作效率。后来，虽然改进为以存储器为中心的如图 1.8 所示的分散连接结构，I/O 与主存交换信息可以不经过运算器，又采用了中断、DMA 等技术，使 CPU 工作效率得到很大的提高，但是仍无法解决 I/O 设备与主机之间连接的灵活性。随着计算机应用领域的不断扩大，I/O 设备的种类和数量也越来越多，人们希望随时增添或减撤设备，用分散连接方式简直是一愁莫展，由此出现了总线连接方式。

总线是连接多个部件的信息传输线，是各部件共享的传输介质。当多个部件与总线相连时，如果出现两个或两个以上部件同时向总线发送信息，势必导致信号冲突，传输无效。因此，在某一时刻，只允许有一个部件向总线发送信息，而多个部件可以同时从总线上接收相同的信息。

总线实际上是由许多传输线或通路组成，每条线可传输一位二进制代码，一串二进制代码可在一段时间内逐一传输完成。若干条传输线可以同时传输若干位二进制代码，如 16 条传输线组成的总线，可同时传输 16 位二进制代码。

采用总线连接的计算机结构，如图 3.1 所示，它是以 CPU 为中心的双总线结构。

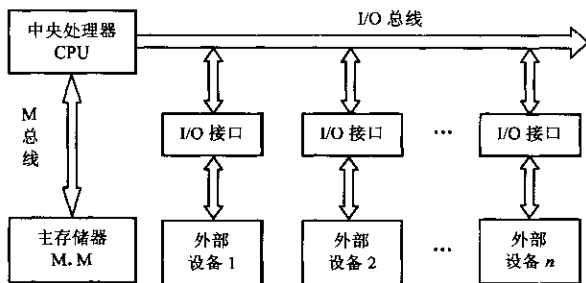


图 3.1 面向 CPU 的双总线结构框图

其中一组总线连接 CPU 和主存，叫存储总线(M 总线)，另一组用来建立 CPU 和各 I/O 之间交换信息的通道，叫输入/输出总线(I/O 总线)，各种 I/O 设备通过 I/O 接口挂到 I/O 总线上。这种结构在 I/O 设备与主存交换信息时仍然要占用 CPU，因此还会影响 CPU 的工作效率。

倘若将 CPU、主存和 I/O 设备（通过 I/O 接口）都挂到一组总线上，便形成单总线结构的计算机，如图 3.2 所示。

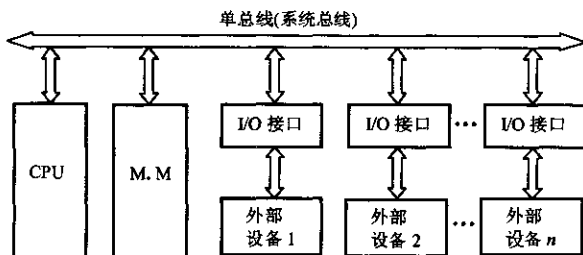


图 3.2 单总线结构框图

图 3.2 与图 3.1 相比，最明显的特点是当 I/O 与主存交换信息时，原则上不影响 CPU 的工作，CPU 仍可继续处理不访问主存或 I/O 的操作，这就使 CPU 工作效率有所提高。但是，因为只有一组总线，当某一时刻各部件都要占用时，就会出现争夺现象。为此，总线内部必须设判优机构，让各部件按优先级高低占用总线，这会影晌整机工作速度。PDP-11 和国产 DJS183 机采用这种结构。

还有一种以存储器为中心的双总线结构，如图 3.3 所示。

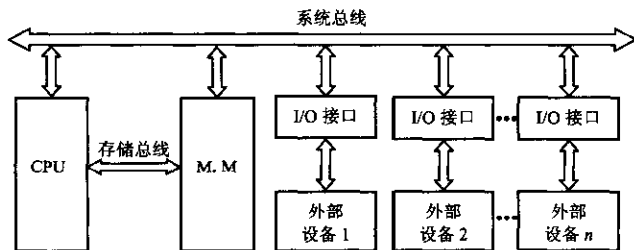


图 3.3 以存储器为中心的双总线结构框图

它是在单总线基础上，又单独开辟一条 CPU 与主存之间的通路，叫存储总线。这组总线速度高，只供主存与 CPU 之间传输信息。这样既提高了传输效率，又减轻了系统总线的负担，还保留了 I/O 与存储器交换信息时不经过 CPU 的特点。国产 DJS184 机采用这种结构。

现代计算机大多数采用各类总线结构。

3.2 总线的分类

总线的应用很广泛，从不同角度可以有不同的分类方法。按数据传送方式可分为并行传输总线和串行传输总线。在并行传输总线中，又可按传输数据宽度分 8 位、16 位、32 位、64 位等传输总线。若按总线的使用范围划分，则又有计算机（包括外设）总线、测控总线、网络通信总线等。下面按连接部件的不同，分几类介绍总线。

3.2.1 片内总线

片内总线是指芯片内部的总线，如在 CPU 芯片内部，寄存器与寄存器之间、寄存器与运算单元 ALU 之间都有总线连接。

3.2.2 系统总线

系统总线是指 CPU、主存、I/O（通过 I/O 接口）各大部件之间的信息传输线。由于这些部件通常都制作在各个插件板上，故又叫作板级总线（即在一块电路板上各芯片间的连线）和板间总线。

按系统传输信息的不同，又可分为三类：数据总线，地址总线和控制总线。

1. 数据总线

数据总线用来传输各功能部件之间的数据信息，它是双向传输总线，其位

数与机器字长、存储字长有关，一般为 8 位、16 位或 32 位。数据总线的条数称为数据总线宽度，它是衡量系统性能的一个重要参数。如果数据总线的宽度为 8 位，指令字长为 16 位，那么，CPU 在取指阶段，必须两次访问主存。

2. 地址总线

地址总线主要用来指出数据总线上的源数据或目的数据在主存单元的地址。例如，欲从存储器读出一个数据，则 CPU 要将此数据所在存储单元的地址送到地址线上。又如，欲将某数据经 I/O 设备输出，则 CPU 除了需将数据送到数据总线外，同时还需将该输出设备的地址（通常都经 I/O 接口）送到地址总线上。可见，地址总线上的代码是用来指明 CPU 欲访问的存储单元或 I/O 端口的地址，它是单向传输的。地址线的位数与存储单元的个数有关，如地址线为 20 根，则对应的存储单元个数为 2^{20} 。

3. 控制总线

由于数据总线、地址总线都是被挂在总线上的所有部件共享的，如何使各部件能在不同时刻占有总线使用权，需依靠控制总线来完成，因此控制总线是用来发出各种控制信号的传输线。对任一控制线而言，它的传输只能是单向的。例如，命令存储器读/写或命令 I/O 读/写都是由 CPU 发出的。但对于控制总线总体来说，又可认为是双向的。例如 I/O 设备也可以向 CPU 发出请求信号。如当某设备准备就绪时，便向 CPU 发中断请求；又如当某部件（如 DMA 接口）需获得总线使用权时，就得向 CPU 发出总线请求等等。此外，控制总线还起到监视各部件状态的作用。如查询该设备是处于“忙”还是“闲”，是否出错等等。因此总体而言，控制信号既有出，又有入。

常见的控制信号有：

- 时钟 用来同步各种操作；
- 复位 表示各模块恢复初始状态；
- 总线请求 表示某部件需获得总线使用权；
- 总线允许 表示需要获得总线使用权的部件已获得了控制权；
- 中断请求 表示某部件提出中断请求；
- 中断确认 表示中断请求已被接收；
- 存储器写 将数据总线上的数据写至存储器的指定地址单元内；
- 存储器读 将指定存储单元中的数据读到数据总线上；
- I/O 读 从指定的 I/O 端口将数据读到数据总线上；
- I/O 写 将数据总线上的数据输出到指定的 I/O 端口内；
- 数据确认 表示数据已被接收或已被读到总线上。

3.2.3 通信总线

这类总线用于计算机系统之间或计算机系统与其他系统（如控制仪表、移动通讯等）之间的通信。由于这类联系涉及到许多方面，如外部连接、距离远近、速度快慢、工作方式等等，差别极大，因此通信总线的类别很多。但按传输方式可分为两种：串行通信总线和并行通信总线。

3.3 总线特性及性能指标

3.3.1 总线特性

从物理角度来看，总线就是一组电导线，许多导线直接印制在电路板上，延伸的各个部件。图 3.4 形象地表示了各个部件与总线之间的物理摆放位置。

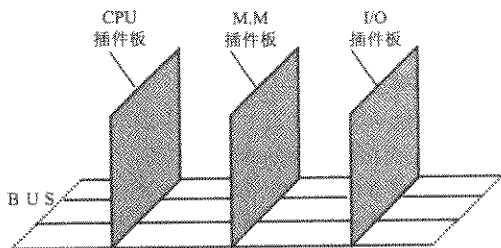


图 3.4 总线结构的物理实现

图中 CPU、M.M、I/O 都是部件插板，它们通过插头与水平方向总线插槽（按总线标准用印刷电路板或一束电缆连接而成的多头插座）连接。为了保证机械上的可靠连接，必须规定其机械特性；为了确保电气上正确连接，必须规定其电气特性；为保证正确地连接不同部件，还需规定其功能特性和时间特性。

(1) 机械特性

机械特性是指总线在机械连接方式上的一些性能，如插头与插座使用的标准，它们的几何尺寸、形状、引脚的个数以及排列的顺序，接头处的可靠接触等等。

(2) 电气特性

电气特性是指总线的每一根传输线上信号的传递方向和有效的电平范围。通常规定由 CPU 发出的信号叫输出信号，送入 CPU 的信号叫输入信号。如地

址总线属于单向输出线,数据总线属于双向传输线,它们都定义为高电平有效。控制总线的每一根都是单向的,但从整体看,有输入,也有输出。有的定义为高电平有效,也有的定义为低电平有效,必须注意不同的规格。不过,总线的电平定义与 TTL 是相符的。如 RS-232C (串行总线接口标准),其电气特性规定低电平表示逻辑“1”,并要求电平低于-3V;用高电平表示逻辑“0”,还要求高电平需高于+3V,额定信号电平为-10V 和+10V 左右。

(3) 功能特性

功能特性是指总线中每根传输线的功能,如地址总线用来指出地址号;数据总线传递数据;控制总线发出控制信号,既有 CPU 发出的,如存储器读/写、I/O 读/写;也有 I/O 向 CPU 发来的,如中断请求、DMA 请求等。可见各条线其功能不一。

(4) 时间特性

时间特性是指总线中的任一根线在什么时间内有效。每条总线上的各种信号,互相存在着一种有效时序的关系,因此,时间特性一般可用信号时序图来描述。

3.3.2 总线性能指标

总线性能指标包括:

① 总线宽度:它是指数据总线的根数,用 bit (位)表示,如 8 位、16 位、32 位、64 位 (也即 8 根、16 根、32 根、64 根)。

② 标准传输率:即在总线上每秒能传输的最大字节量,用 MB/s (每秒多少兆字节)表示。如总线工作频率为 33MHz,总线宽度为 32 位,则它最大的传输率为 132MB/s。

③ 时钟同步/异步:总线上的数据与时钟同步工作的总线称同步总线,与时钟不同步工作的总线称异步总线。

④ 总线复用:通常地址总线与数据总线在物理上是分开的两种总线。地址总线传输地址码,数据总线传输数据信息。为了提高总线的利用率,优化设计,特将地址总线 and 数据总线共用一组物理线路,只是某一时刻该总线传输地址信号,另一时刻传输数据信号或命令信号。这叫做总线的多路复用。

⑤ 信号线数:即地址总线、数据总线和控制总线三种总线数的总和。

⑥ 总线控制方式:包括并发工作、自动配置、仲裁方式、逻辑方式、计数方式等。

⑦ 其他指标:如负载能力问题。由于不同的电路对总线的负载是不同的,即使同一电路板在不同的工作频率下,总线的负载也是不同的,因此,总线负

载能力的指标不是太严格的。通常用可连接扩增电路板数来反映总线的负载能力。此外，还有如电源电压是 5V 还是 3.3V、总线能否扩展 64 位宽度等等，也十分重要。

表 3.1 列出了几种流行的微机总线性能，可供参考。

表 3.1 几种流行的微型计算机总线性能

名称	ISA (PC-AT)	EISA	STD	VESA (VL-BUS)	MCA	PCI
通用 机型	80286, 386, 486 系列机	386, 486, 586 IBM 系列机	Z-80, V20, V40 IBM- PC 系列机	i486, PC-AT 兼容机	IBM 个人 机与工作 站	P5 个人机, PowerPC, Alpha 工作 站
最大 传输率	15MB/s	33MB/s	2MB/s	266MB/s	40MB/s	133MB/s
总线宽度	16 位	32 位	8 位	32 位	32 位	32 位
总线工作 频率	8MHz	8.33MHz	2MHz	66MHz	10MHz	0~33MHz
同步方式	同步			异步	同步	
仲裁方式	集中	集中	集中	集中		
地址宽度	24	32	20			32/64
负载能力	8	6	无限制	6	无限制	3
信号线数		143		90	109	49
64 位扩展	不可	无规定	不可	可	可	可
并发工作				可		可
引脚使用	非多路复用	非多路复用	非多路复用	非多路复用		多路复用

(表中缺项待查)

3.3.3 总线标准

总线是在计算机系统模块化发展过程中产生的，随着计算机应用领域的不断扩大，计算机系统中各类模块（特别是 I/O 设备所带的各类接口模块），其品种极其繁杂，往往出现一种模块要配一种总线，很难在总线上更换、组合各类模块或设备。20 世纪 70 年代末，为了使系统设计简化，模块生产批量化，确保其性能稳定，质量可靠，实现可移化，便于维护等，人们开始研究如何使总线建立标准，在总线的统一标准下，完成系统设计，模块制作。这样，系统、模块、设备与总线之间不适应、不通用及不匹配的问题就迎刃而解了。

所谓总线标准，可视为系统与各模块、模块与模块之间的一个互连的标准界面。这个界面对它两端的模块都是透明的，即界面的任一方只需根据总线标准的要求完成自身一面接口的功能要求，而无需了解对方接口与总线的连接要

求。因此,按总线标准设计的接口可视为通用接口。

目前流行的总线标准有:

(1) ISA (Industrial Standard Architecture) 总线是 IBM 为了采用全 16 位的 CPU 而推出的, 又称 AT 总线, 它使用独立于 CPU 的总线时钟, 因此 CPU 可以采用比总线频率更高的时钟, 有利于 CPU 性能的提高。由于 ISA 总线没有支持总线仲裁的硬件逻辑, 因此它不能支持多台主设备(即不支持多台具有申请总线控制权的设备)系统, 而且 ISA 上的所有数据的传送必须通过 CPU 或 DMA (直接存储器存取) 接口来管理, 因此使 CPU 花费了大量时间来控制与外部设备交换数据。ISA 总线时钟频率为 8MHz, 最大传输率为 16MB/s, 数据线为 16 位, 地址线为 24 位。

(2) EISA (Extended Industrial Standard Architecture) 是一种在 ISA 基础上扩充开放的总线标准, 它与 ISA 可以完全兼容, 它从 CPU 中分离出了总线控制权, 是一种具有智能化的总线, 能支持多总线主控和突发方式的传输。EISA 总线的时钟频率为 8MHz, 最大传输率可达 33MB/s, 数据总线为 32 位, 地址总线为 32 位, 扩充 DMA 访问范围达 2^{32} 。

(3) VL-BUS 是由 VESA (Video Electronic Standard Association 视频电子标准协会) 提出的局部总线标准。所谓局部总线是指在系统外, 为两个以上模块提供的高速传输信息通道。VL-BUS 是由 CPU 总线演化而来的, 采用 CPU 的时钟频率达 33MHz、数据线为 32 位, 配有局部控制器。通过局部控制器的判断, 将高速 I/O 直接挂在 CPU 的总线上, 实现 CPU 与高速外设之间的高速数据交换(参见图 3.10)。

(4) PCI (Peripheral Component Interconnect 外部设备互连总线) 是由 Intel 公司提供的总线标准。它与 CPU 时钟频率无关, 自身采用 33MHz 总线时钟, 数据线为 32 位, 可扩充到 64 位, 数据传输率达 132MB/s~246MB/s。具有很好的兼容性, 与 ISA, EISA 总线均可兼容, 可以转换为标准的 ISA、EISA。它能支持无限读写突发方式, 速度比直接使用 CPU 总线的局部总线快。它可视为 CPU 与外设间的一个中间层, 通过 PCI 桥路 (PCI 控制器) 与 CPU 相连。

PCI 控制器有多级缓冲, 可把一批数据快速写入缓冲器中。在这些数据不断写入 PCI 设备过程中, CPU 可以执行其他操作, 即 PCI 总线上的外设与 CPU 可以并行工作。

PCI 总线支持两种电压标准: 5V 与 3.3V。3.3V 电压的 PCI 总线可用于便携式微机中。

EISA 和 PCI 都具有即插即用(plug and play)的功能, 即任何扩展卡只要插入系统便可工作, 尤其是 PCI 采用的技术非常完善, 它为用户提供了真正的

即插即用功能。

PCI 总线可扩充性好, 当总线驱动能力不足时, 可以采用多层结构 (见图 3.12)。每个 PCI 还配有一个延时器, 它规定系统中设备使用 PCI 总线的最长时间周期, CPU 通过 PCI 总线上的所有设备延时器来优化系统的性能。

有关各总线的实例将在 3.4.3 中介绍。

3.4 总线结构

总线结构通常可分为单总线和多总线两种。

3.4.1 单总线结构

图 3.2 是单总线结构的示意, 它是将 CPU、主存、I/O 设备 (通过 I/O 接口) 都挂在一组总线上, 允许 I/O 之间或 I/O 与主存之间直接交换信息。这种结构简单, 也便于扩充, 但所有的传送都通过这组共享总线, 因此极易形成计算机系统的瓶颈。它也不允许两个以上的部件在同一时刻向总线传输信息, 这就必然会影响系统工作效率的提高。这类总线多数为小型机或微型机所采用。计算机应用范围越扩大, 其外部设备的种类和数量就越多, 并且它们对数据传输的量和传输速度的要求也就越来越高。倘若仍然采用单总线结构, 那么, 当 I/O 设备量很大时, 总线发出的控制信号从一端逐个顺序传递到第 n 个设备, 其传播的延迟时间就会严重地影响系统的工作效率。在数据传输需求量和传输速度要求不太高的情况下, 为克服总线瓶颈问题, 尽可能采用增加总线宽度和提高传输速率来解决; 但当总线上的设备如高速视频显示器、网络传输接口等, 其数据量很大和传输速度要求相当高的时候, 单总线结构怎么也满足不了系统工作的需要。因此, 为了根本解决数据传输速率, 解决 CPU、主存与 I/O 设备之间传输速率的不匹配, 实现 CPU 于其他设备相对同步, 不得不采用多总线结构。

3.4.2 多总线结构

图 3.5 是双总线结构的示意图。

双总线结构的特点是将速度较低的 I/O 设备从单总线上分离出来, 形成主存总线与 I/O 总线分开结构。图中通道是一个具有特殊功能的处理器, CPU 将一部份功能下放给通道, 使其对 I/O 设备具有统一管理的功能, 以完成外部设备与主存之间的数据传送, 其系统的吞吐能力可以相当大。这种结构大多用于大、中型计算机系统。

如果将速率不同的 I/O 设备进行分类, 然后将它们连接在不同的通道上,

那么计算机系统的利用率将会更高，由此发展成多总线结构。

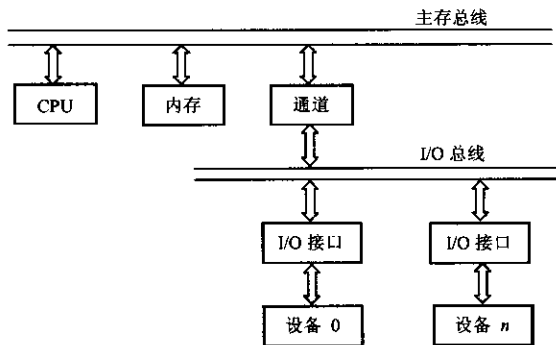


图 3.5 双总线结构

图 3.6 是三总线结构的示意。

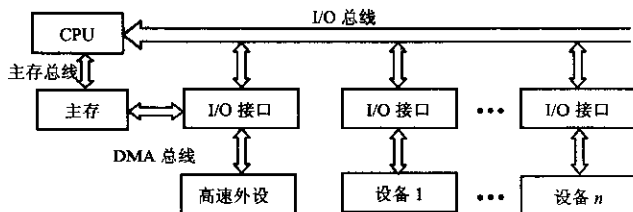


图 3.6 三总线结构

图中主存总线用于 CPU 与主存之间的传输；I/O 总线供 CPU 与各类 I/O 之间传递信息；DMA 总线用于高速外设（磁盘、磁带等）与主存之间直接交换信息。在三总线结构中，任一时刻只能使用一种总线。主存总线与 DMA 总线不能同时对主存进行存取，I/O 总线只有在 CPU 执行 I/O 指令时才用到。

图 3.7 是另一种三总线结构的示意。

由图可见，处理器与高速缓冲存储器 Cache 之间有一条局部总线，它将 CPU 与 Cache 或与更多的局部设备连接。Cache 的控制机构（详见 4.3 节）不仅将 Cache 连到局部总线上，而且还直接连到系统总线上，这样 Cache 就可通过系

统总线与主存传输信息。而且 I/O 与主存之间的传输也不必通过 CPU。还有一条扩展总线，它将局域网、小型计算机接口 (SCSI)、调制解调器 (Modem) 以及串行接口等都连接起来，并且通过这些接口又可与各类 I/O 设备相连，因此它可支持相当多的 I/O 设备。与此同时，扩展总线又通过扩展总线接口与系统总线相连，由此便可实现这两种总线之间的信息传递，可见其系统的工作效率明显地提高。

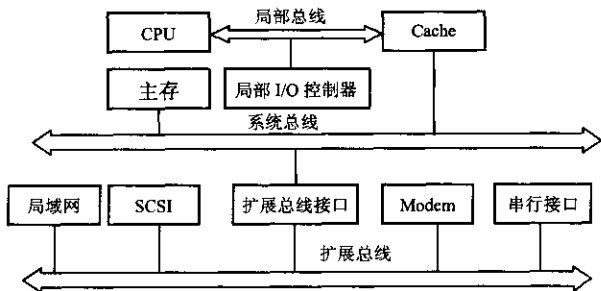


图 3.7 三总线结构的又一形式

为了进一步提高 I/O 的性能，使其更快地响应命令，又出现了四总线结构，如图 3.8 所示。

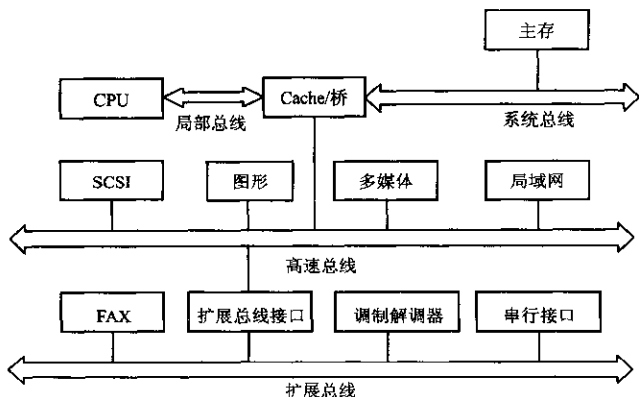


图 3.8 四总线结构

在这里又增加了一条与计算机系统紧密相连的高速总线。在高速总线上挂接了一些高速性能的外设，如高速局域网、图形工作站、多媒体、SCSI 等。它们通过 Cache 控制机构中的高速总线桥或高速缓冲器与系统总线和局部总线相连，使得这些高速设备与处理器更密切。而一些较低速的设备如图文传真 FAX、调制解调器及串行接口仍然挂在扩展总线上，并由扩展总线接口与高速总线相连。

这种结构对高速设备而言，其自身的工作可以很少依赖处理器，同时它们又比扩展总线上的设备更贴近处理器，可见对于高性能设备与处理器来说，各自的效率将获得更大的提高。在这种结构中，处理器、高速总线的速度以及各自信号线的定义完全可以不同，以至各自改变其结构也不会影响高速总线的正常工作，反之亦然。

3.4.3 总线结构举例

图 3.9 是传统微机总线的结构示意图。

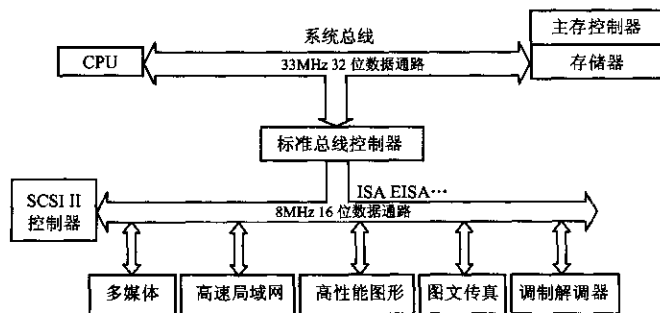


图 3.9 传统微型机总线结构

由图可见，不论高速局域网、高性能图形还是低速的 FAX、Modem 都挂在 ISA 或 EISA 总线上，并通过 ISA 或 EISA 总线控制器与系统总线相连，这样势必出现总线数据传输的瓶颈。只有将高速、高性能的外设，如高速局域网、高性能图形等尽量靠近 CPU 本身的总线，并与 CPU 同步或准同步，才可能消除瓶颈问题。这就要求改变总线结构，来提高数据传送速率，为此，出现了图 3.10 的 VL-BUS 局部总线结构。

由图可见，将原先挂在 ISA 总线上的高速局域网、多媒体卡、高性能图形板等从 ISA 总线卸下来，挂到局部总线 VL-BUS 上，再与系统总线相连。

而将打印机、FAX、Modem 等低速设备仍挂在 ISA 总线上。局部总线 VL-BUS 就相当于在处理器与高速外设之间架上了高速通道，使 CPU 与高性能外设得到充分发挥，满足了图形界面软件的要求。

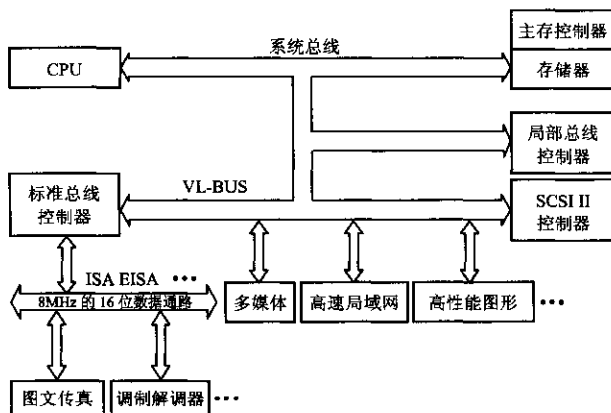


图 3.10 VL-BUS 局部总线结构

由于 VL-BUS 是从 CPU 总线演化来的，与 CPU 的关系太紧密（实际上这种总线与 486 配合最佳），以至很难支持功能强的 CPU，从而出现了 PCI 总线。图 3.11 是 PCI 总线结构示意图。

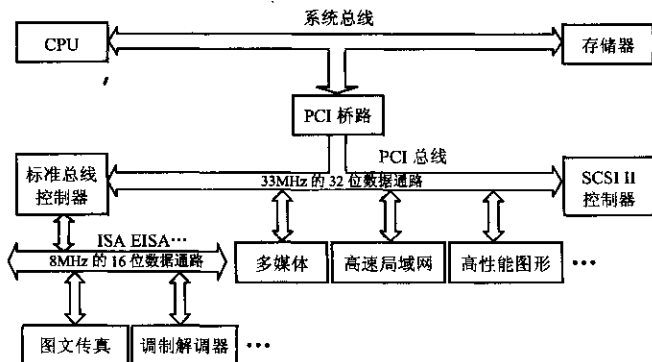


图 3.11 PCI 总线结构

从此图可见, PCI 总线是通过 PCI 桥路(包括 PCI 控制器和 PCI 加速器)与 CPU 总线相连。这种结构使 CPU 总线与 PCI 总线互相隔离, 具有更高的灵活性, 可以支持更多的高速运行设备, 而且具有即插即用的特性。当然, 挂在 PCI 总线上的设备都要求数据传输速率高的设备, 如多媒体卡、高速局域网适配器、高性能图形板等, 与高速 CPU 总线是相匹配的。至于低速的 FAX、Modem、打印机仍然挂在 ISA、EISA 总线上。

当 PCI 总线驱动能力不足时, 可采用多层结构, 如图 3.12 所示。

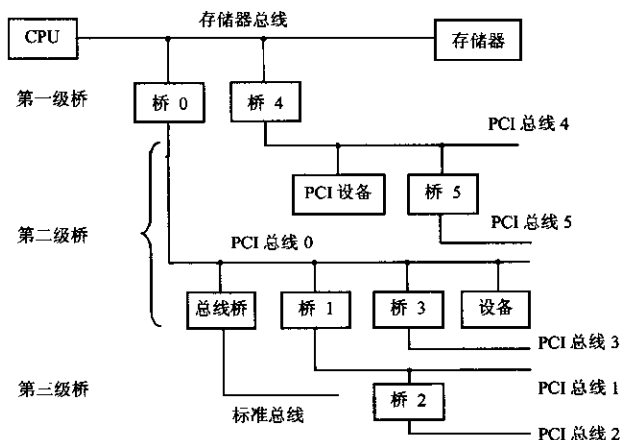


图 3.12 多层 PCI 总线结构

3.5 总线控制

由于总线上连接着许许多多部件, 什么时候由哪个部件发送信息; 如何给信息传送定时; 如何防止信息丢失; 如何避免多个部件同时发送; 如何规定接受信息的部件等等一系列问题, 都需要由总线控制器统一管理。它主要包括判优控制(或称仲裁逻辑)和通信控制。

3.5.1 总线判优控制

总线上所连接的设备, 按其对总线有无控制功能可分为主设备和从设

备两种。主设备对总线有控制权，从设备只能响应从主设备发来的总线命令。总线上信息的传送是由主设备启动的，如某个主设备欲与另一个设备（从设备）进行通信时，首先由主设备发出总线请求信号，若多个主设备同时要使用总线时，就由总线控制器的判优、仲裁逻辑按一定的优先等级顺序，确定哪个主设备能使用总线。只有获得总线使用权的主设备才能开始传送数据。

总线判优控制可分集中式和分布式两种，前者将控制逻辑集中在一处（如在 CPU 中），后者将控制逻辑分散在与总线连接的各个部件或设备上。

常见的集中控制有三种优先权仲裁方式：

（1）链式查询

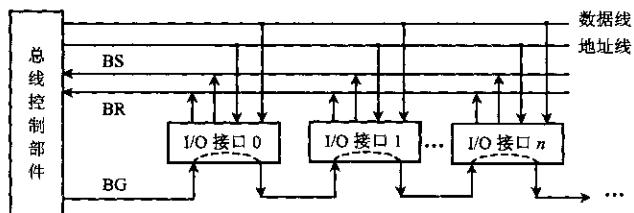
链式查询方式如图 3.13（a）所示。图中控制总线中有三根线用于总线控制（BS 总线忙、BR 总线请求、BG 总线同意），其中总线同意信号 BG 是串行地从一个 I/O 接口送到下一个 I/O 接口。如果 BG 到达的接口有总线请求，BG 信号就不再往下传，意味着该接口获得了总线使用权，并建立总线忙 BS 信号，表示它占用了总线。可见在查询链中，离总线控制部件最近的设备具有最高的优先级。这种方式的特点是：只需很少几根线就能按一定优先次序实现总线控制，并且很容易扩充设备，但对电路故障很敏感。

（2）计数器定时查询

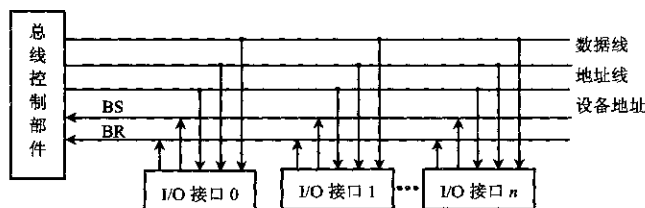
计数器定时查询方式如图 3.13（b）所示。与本图（a）相比，多了一组设备地址线，少了一根总线同意线 BG。总线控制部件接到由 BR 送来的总线请求信号后，在总线未被使用（BS=0）的情况下，由计数器开始计数，向各设备发出一组地址信号。当某个有总线请求的设备地址与计数值一致时，便获得总线使用权，此时终止计数查询。这种方式的特点是：计数可以从“0”开始，此时设备的优先次序是固定的；计数也可以从终止点开始，即是一种循环方法，此时设备使用总线的优先级相等；计数器的初始值还可由程序设置，故优先次序可以改变。此外，对电路故障不如链式查询方式敏感，但增加了主控制线（设备地址）数，控制也较复杂。

（3）独立请求方式

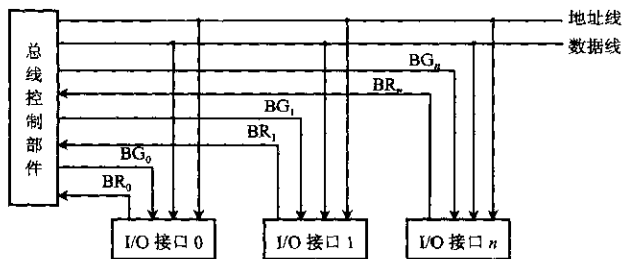
独立请求方式如图 3.13（c）所示。由图可见，每一设备均有一对总线请求线 BR_i 和总线同意线 BG_i 。当设备要求使用总线时，便发出该设备的请求信号。总线控制部件中有一排队电路，可根据优先次序确定响应哪一设备的请求。这种方式的特点是：响应速度快，优先次序控制灵活（通过程序改变），但控制线数量多，总线控制更复杂。链式查询中仅用两根线确定总线使用权属于哪个设备，在计数查询中大致用 $\log_2 n$ 根线，其中 n 是允许接纳的最大设备数，而独立请求方式需采用 $2n$ 根线。



(a) 链式查询方式



(b) 计数器定时查询方式



(c) 独立请求方式

图 3.13 集中控制的三种优先权仲裁方式

3.5.2 总线通信控制

众多部件共享总线，在争夺总线使用权时，只能按各部件的优先等级来解决。而在传送通信时间上，只能按分时方式来解决，即哪一个部件获得使用，此刻就由它传送，下一部件获得使用，接着下一时刻传送。这样一个接一个轮

流交替传送。

总线在完成一次传输周期时，可分为四个阶段：

- 申请分配阶段：由需要使用总线的主模块（或主设备）提出申请，经总线仲裁机构决定下一传输周期的总线使用权授予某一申请者；
- 寻址阶段：取得了使用权的主模块，通过总线发出本次打算访问的从模块（或从设备）的存储地址或设备地址及有关命令，启动参与本次传输的从模块；
- 传数阶段：主模块和从模块进行数据交换，数据由源模块发出经数据总线流入目的模块；
- 结束阶段：主模块的有关信息均从系统总线上撤除，让出总线使用权。

对于仅有一个主模块的简单系统，就无需申请、分配和撤除了，总线使用权始终归它占有。对于包含中断、DMA 控制或多处理器的系统，还得有某种分配管理机构来参与。

总线通信控制主要解决通信双方如何获知传输开始和传输结束，以及通信双方如何协调如何配合。一般常用四种方式：同步通信、异步通信、半同步通信和分离式通信。

1. 同步通信

通信双方由统一时标控制数据传送称为同步通信。时标通常由 CPU 的总线控制部件发出，送到总线上的所有部件；也可以由每个部件各自的时序发生器发出，但必须由总线控制部件发出的时钟信号对它们进行同步。

图 3.14 表示某个输入设备向 CPU 传输数据的同步通信过程。

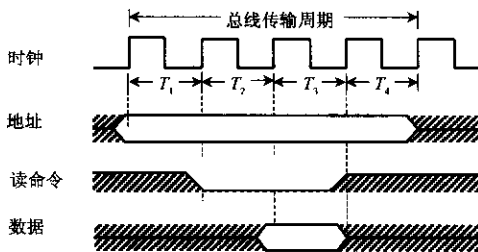


图 3.14 同步式数据输入传输

图中总线传输周期是总线上两个部件完成一次完整而可靠的传输时间，它包含 4 个时钟周期 T_1 、 T_2 、 T_3 、 T_4 。

主模块在 T_1 时刻发出地址信息； T_2 时刻发出读命令；从模块按照所指定的地址和命令进行一系列内部动作，必须在 T_3 时刻前找到 CPU 所需的数据，并送到数据总线上；CPU 在 T_3 时刻开始，一直维持到 T_4 时刻，可以从数据总线上获取信息并送到其内部寄存器中； T_4 时刻开始输入设备不再向数据总线上传送数据，撤消它对数据总线的驱动。如果总线采用三态驱动电路，则从 T_4 起，数据总线呈浮空状态。

同步通信在系统总线设计时， T_1 、 T_2 、 T_3 、 T_4 都有明确的、唯一的规定。

对于读命令，其传输周期为：

T_1 主模块发地址；

T_2 主模块发读命令；

T_3 从模块提供数据；

T_4 主模块撤消读命令。

对于写命令，其传输周期为：

T_1 主模块发地址；

$T_{1.5}$ 主模块提供数据；

T_2 主模块发出写命令，从模块接收到命令后，必须在规定时间内将数据总线上的数据写到地址总线所指明的单元中；

T_4 主模块撤消写命令和数据等信号。

上述时序如图 3.15 所示。

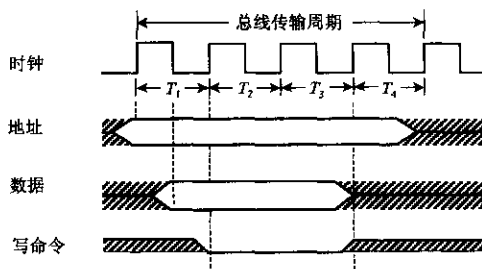


图 3.15 同步式数据输出传输

这种通信的优点是规定明确、统一，模块间的配合简单一致。其缺点是主从模块时间配合属强制性“同步”，必须在限定时间内完成规定的要求。并且对所有从模块都用同一限时，这就势必造成对各不相同速度的部件而言，必须

按最慢速度部件来设计公共时钟，严重影响总线的工作效率，也给设计带来了局限性，缺乏灵活性。

同步通信一般用于总线长度较短，各部件存取时间比较一致的场所。

2. 异步通信

异步通信克服了同步通信的缺点，允许各模块速度的不一致性，给设计者充分的灵活性和选择余地。它没有公共的时钟标准，不要求所有部件严格的统一动作时间，而是采用应答方式（又称握手方式），即当主模块发出请求（Request）信号时，一直等待从模块反馈回来“响应”（Acknowledge）信号后，才开始通信。当然，这就要求主从模块之间增加两条应答线（即握手交互信号线 Handshaking）。

异步通信方式可分为不互锁、半互锁和全互锁三种类型，如图 3.16 所示。

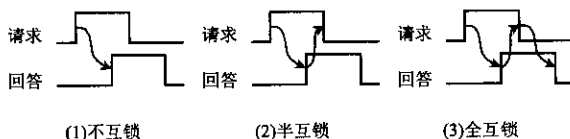


图 3.16 异步通信中请求与回答的互锁

(1) 不互锁方式

主模块发出请求信号后，不等待接到从模块的回答信号，而是经过一段时间，确认从模块已收到请求信号后，便撤消其请求信号；从设备接到请求信号后，在条件允许时发出回答信号，并且经过一段时间，确认主设备已收到回答信号后，自动撤消回答信号。可见通信双方并无互锁关系。

(2) 半互锁方式

主模块发出请求信号，待接到从模块的回答信号后再撤消其请求信号，存在着简单的互锁关系；而从模块发出回答信号后，不等待主模块回答，在一段时间后就撤消其回答信号，无互锁关系。故称半互锁方式。

(3) 全互锁方式

主模块发出请求信号，待从模块回答后再撤其请求信号；从模块发出回答信号，待主模块获知后，再撤消其回答信号。故称全互锁方式。

3. 半同步通信

半同步通信集同步与异步通信之优点，既保留了同步通信的基本特点，如所有的地址、命令、数据信号的发出时间，都严格参照系统时钟的某个前沿开

始，而接收方都采用系统时钟后沿时刻来进行判断识别。同时又像异步通信那样，允许不同速度的模块和谐地工作。为此增设了一条“等待”（ $\overline{\text{WAIT}}$ ）响应信号线。

以读命令为例，在同步通信中，主模块在 T_1 发出地址， T_2 发出命令， T_3 传输数据， T_4 结束传输。倘若从模块工作速度较慢，无法在 T_3 时刻提供数据，则必须在 T_3 之前通知主模块，使其进入等待状态，此刻，从模块置 $\overline{\text{WAIT}}$ 为低电平有效。主模块在 T_3 测得“等待”有效，则不立即从数据线上取数，这样一个时钟周期、一个时钟周期地等待，直到主模块测得 $\overline{\text{WAIT}}$ 为高电平即等待无效时，主模块即把此刻的下一周期当作正常周期 T_3 ，即时获取数据， T_4 结束传输。

半同步通信时序可为：

T_1 主模块发地址；

T_2 主模块发命令；

T_w 当 $\overline{\text{WAIT}}$ 为低电平有效时，进入等待，其间隔与 T 统一；

T_w 当 $\overline{\text{WAIT}}$ 为低电平有效时，进入等待，其间隔与 T 统一；

.....

T_3 从模块提供数据（若属读命令）；

T_4 从模块撤消数据。

半同步通信适用于系统工作速度不高、但又包含了许多工作速度差异较大的各类设备的简单系统。半同步通信控制方式比异步通信简单，在全系统内各模块又在统一的系统时钟控制下同步工作，可靠性较高，同步结构较方便。其缺点是对系统时钟频率不能要求太高，故从整体上来看，系统工作的速度还是不很高。

4. 分离式通信

以上三种通信方式都是从主模块发出地址和读写命令开始，直到数据传输结束。在整个传输周期中，系统总线的使用权，完全由占有使用权的主模块和由它选中的从模块占据。进一步分析读命令传输周期，发现除了申请总线这一阶段外，其余时间主要被花在如下三个方面：

- （1）主模块通过传输总线向从模块发送地址和命令；
- （2）从模块按照命令进行读数据的必要准备；
- （3）从模块经数据总线向主模块提供数据。

由（2）可见，对系统总线而言，从模块内部读出过程并无实质性的信息传输，纯属空闲等待。为了克服和利用这种消极等待，尤其在大型计算机系统中，总线的负载已处于饱和状态，充分挖掘系统总线每瞬间的潜力，对提高系

统性能起到极大作用。为此人们提出了“分离式”的通信方式。其基本思想是将一个传输周期（或总线周期）分解为两个子周期。在第一个子周期中，主模块 A 在获得总线使用权后将命令、地址以及其他的有关信息，包括该主模块编号（当有多个主模块时，此编号尤为重要）发到系统总线上，经总线传输后，由有关的从模块 B 接收下来。主模块 A 向系统总线发布这些信息只占用总线很短的时间，一旦发送完，立即放弃总线使用权，以便其他模块使用。在第二个子周期中，当 B 模块收到 A 模块发来的有关命令信号后，经选择、译码、读取等一系列内部操作，将 A 模块所需的数据准备好，便由 B 模块申请总线使用权，一旦获准，B 模块便将 A 模块的编号、B 模块的地址，A 模块所需的数据等一系列信息送到总线上，供 A 模块接收。很明显，上述两个传输子周期都只有单方向的信息流，每个模块都变成了主模块。

这种通信方式的特点是：①各模块欲占用总线使用权都必须提出申请；②在得到总线使用权后，主模块在限定的时间内向对方传送信息，采用同步方式传送，不再等待对方的回答信号；③各模块在准备数据传送的过程中都不占用总线，使总线可接受其他模块的请求；④总线被占用时都在作有效工作，或者通过它发命令，或者通过它传送数据，不存在空闲等待时间，最充分地发挥了总线的有效占用。从而实现了总线为多个主从模块间进行信息交叉重叠并行式传送，这对大型计算机系统是极为重要的。当然，这种方式控制比较复杂，一般在普通微机系统很少采用。

思考题与习题

1. 什么是总线？总线传输有何特点？为了减轻总线负载，总线上的部件应具备什么特点？
2. 总线如何分类？什么是系统总线？系统总线又分几类？它们各有何作用？是单向的还是双向的？它们与机器字长、存储字长、存储单元有何关系？
3. 常用的总线结构有几种？不同的总线结构对计算机的性能有什么影响？举例说明。
4. 为什么要设置总线判优控制？常见的集中式总线控制有几种？各有何特点？哪种方式响应时间最快？哪种方式对电路故障最敏感？
5. 解释下列概念：总线的主设备（或主模块）、总线的从设备（或从模块）、总线的传输周期和总线的通信控制。
6. 试比较同步通信和异步通信。
7. 画图说明异步通信中请求与回答有那几种互锁关系？
8. 为什么说半同步通信同时保留了同步通信和异步通信的特点？
9. 分离式通信有何特点？主要用于什么系统？

-
10. 为什么要设置总线标准？你知道目前流行的总线标准有哪些？什么叫 plug and play？哪些总线有这一特点？
11. 画一个具有双向传送功能的总线逻辑图。
12. 设数据总线上接有 A、B、C、D、四个寄存器，要求选用合适的 74 系列芯片，完成下列逻辑设计：
- (1) 设计一个电路，在同一时间实现 $D \rightarrow A$ 、 $D \rightarrow B$ 和 $D \rightarrow C$ 寄存器间的传送；
- (2) 设计一个电路，实现下列操作：
- T_0 时刻完成 $D \rightarrow$ 总线；
- T_1 时刻完成 总线 $\rightarrow A$ ；
- T_2 时刻完成 $A \rightarrow$ 总线；
- T_3 时刻完成 总线 $\rightarrow B$ 。

第四章 存 储 器

本章重点介绍主存储器的分类、工作原理、组成方式以及与其他部件（如 CPU）的联系。此外还介绍了高速缓冲存储器、磁表面存储器等的基本组成和工作原理。旨在使读者真正建立起如何用不同的存储器，组成具有层次结构的存储系统的概念。

4.1 概 述

4.1.1 存储器分类

存储器是计算机系统记忆设备，用来存放程序和数据。随着计算机发展，存储器在系统中的地位越来越重要。由于超大规模集成电路的制作技术，使 CPU 的速度变得惊人的高，而存储器的取数和存数的速度与它很难适配，这使计算机系统的运行速度在很大程度上受存储器速度的制约。此外，由于 I/O 设备的不断增多，如果它们与存储器打交道都通过 CPU 来实现，这将大大降低 CPU 的工作效率。为此，出现了 I/O 与存储器的直接存取方式（DMA），这也使存储器的地位更为突出。尤其在多处理机的系统中，各处理机本身都需与其主存交换信息，而且各处理机在互相通信中，也都需共享存放在存储器中的数据。因此，存储器的地位就更为显著。可见，从某种意义上讲，存储器的性能已成为计算机系统的核心。

当今，存储器的种类繁多，从不同的角度对存储器可作不同的分类。

1. 按存储介质分类

存储介质是指能寄存“0”、“1”两种代码并能区别两种状态的物质或元件。存储介质主要有半导体器件、磁性材料和光盘等。

（1）半导体存储器

存储元件由半导体器件组成的叫半导体存储器。现代半导体存储器都用超大规模集成电路工艺制成芯片，其优点是体积小、功耗低、存取时间短。其缺点是当电源消失时，所存信息也随即丢失，它是一种易失性存储器。近年来已研制出用非挥发性材料制成的半导体存储器，克服了信息易失的弊病。

半导体存储器又可按其材料的不同，分为双极型（TTL）半导体存储器和 MOS 半导体存储器两种。前者具有高速的特点，后者具有高集成度的特点，并且制造简单，成本低廉，功耗小，故 MOS 半导体存储器被广泛应用。

(2) 磁表面存储器

磁表面存储器是在金属或塑料基体的表面上涂一层磁性材料作为记录介质，工作时磁层随载磁体高速运转，用磁头在磁层上进行读写操作，故称为磁表面存储器。按载磁体形状的不同，可分为磁盘、磁带和磁鼓。现代计算机已很少采用磁鼓。由于用具有矩形磁滞回线特性的材料作磁表面物质，它们按其剩磁状态的不同而区分“0”或“1”，而且剩磁状态不会轻易丢失，故这类存储器具有非易失性的特点。

(3) 磁芯存储器

磁芯是由硬磁材料做成的环状元件，在磁心中穿有驱动线（通电流）和读出线，这样便可进行读写操作。磁芯属磁性材料，故它也是不易失的永久记忆存储器。不过，磁芯存储器的体积过大、工艺复杂、功耗太大，故七十年代后，逐渐被半导体存储器取代，目前几乎已不被采用。

(4) 光盘存储器

光盘存储器是应用激光在记录介质（磁光材料）上进行读写的存储器，具有非易失性的特点。由于光盘记录密度高、耐用性好、可靠性高和可互换性强等特点，光盘存储器越来越被用于计算机系统。

2. 按存取方式分类

按存取方式可把存储器分为随机存储器、只读存储器、顺序存储器和直接存取存储器四类。

(1) 随机存储器 RAM (Random Access Memory)

RAM 是一种可读写存储器，其特点是存储器的任何一个存储单元的内容都可以随机存取，而且存取时间与存储单元的物理位置无关。计算机系统中的主存都采用这种随机存储器。由于存储信息原理的不同，RAM 又分为静态 RAM（以触发器原理寄存信息）和动态 RAM（以电容充放电原理寄存信息）。

(2) 只读存储器 ROM (Read Only Memory)

只读存储器是能对其存储的内容读出，而不能对其重新写入的存储器。这种存储器一旦存入了原始信息后，在程序执行过程中，只能将内部信息读出，而不能随意重新写入新的信息去改变原始信息。因此，通常用它存放固定不变的程序、常数以及汉字字库，甚至用于操作系统的固化。它与随机存储器可共同作为主存的一部分，统一构成主存的地址域。

早期只读存储器的存储内容根据用户要求，厂家采用掩膜工艺，把原始信息记录在芯片中，一旦制成后无法更改，叫做掩膜型只读存储器 MROM (Masked ROM)。随着半导体技术的发展和用户需求的变化，只读存储器先后派生出可编程只读存储器 PROM (Programmable ROM)、可擦除可编程只读存储器 EPROM (Erasable Programmable ROM) 以及用电可擦除可编程的只读存储器

EEPROM (Electrically Erasable Programmable ROM)。近年来还出现了快擦型存储器 Flash Memory, 它具有 EEPROM 的特点, 而速度比 EEPROM 快得多。

(3) 串行访问存储器

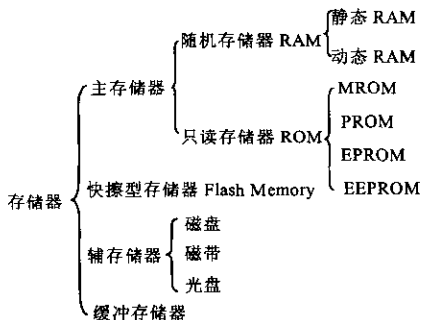
如果对存储单元进行读写操作时, 需按其物理位置的先后顺序寻找地址, 则这种存储器叫作串行访问存储器。显然这种存储器由于信息所在位置不同, 使得读写时间均不相同。如磁带存储器, 不论信息处在哪个位置, 读写时必须从其介质的始端开始按顺序寻找, 故这类串行访问的存储器又叫顺序存取存储器。还有一种属于部分串行访问的存储器, 如磁盘。在对磁盘读写时, 首先直接指出该存储器中的某个小区域(磁道), 然后再顺序寻访, 直至找到位置。故其前段是直接访问, 后段是串行访问, 称其为直接存取存储器。

3. 按在计算机中的作用分类

按在计算机系统中的作用不同, 存储器又可分为主存储器、辅助存储器、缓冲存储器。

主存储器的主要特点是它可以和 CPU 直接交换信息。辅助存储器是主存储器的后援存储器, 用来存放当前暂时不用的程序和数据, 它不能与 CPU 直接交换信息。两者相比, 主存速度快、容量小、每位价格高; 辅存速度慢、容量大、每位价格低。缓冲存储器用在两个速度不同的部件之中, 如 CPU 与主存之间可设置一个快速缓冲存储器(有关内容在 4.3 节讲述), 起到缓冲作用。

综上所述, 存储器分类如下所示:



4.1.2 存储器的层次结构

存储器有三个主要特性：速度、容量和价格/位(简称位价)。一般来说, 速度越高, 位价就越高; 容量越大, 位价就越低; 而且容量越大, 速度必越低。人们追求大容量、高速度、低位价的存储器, 可惜这是很难达到的。可以用一

个形象的存储器分层结构图，来反映上述的问题，如图 4.1 所示。

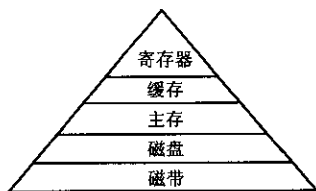


图 4.1 存储器分层结构图

图中由上至下，每位的价格越来越低，速度越来越慢，容量越来越大，CPU 访问的频率也越来越少。最上层的寄存器通常都制作在 CPU 芯片内。寄存器中的数直接在 CPU 内部参与运算，CPU 内可以有十几个、几十个寄存器，它们的速度最快、位价最高、容量最小。主存用来存放将要参与运行的程序和数据，其速度与

CPU 速度差距较大，为了使它们之间速度更好匹配，在主存与 CPU 之间，插入了一种比主存速度更快、容量更小的高速缓冲存储器 Cache，显然其位价要高于主存。主存与缓存之间的数据调动是由硬件自动完成的，对程序员是透明的。以上三层存储器都是由速度不同、位价不等的半导体存储材料制成，它们都设在主机内。第四、五层是辅助存储器，其容量比主存大得多，大都用来存放暂时未用到的程序和数据文件。CPU 不能直接访问辅存，辅存只能与主存交换信息，因此辅存的速度可以比主存慢得多。辅存与主存之间信息的调动，均由硬件和操作系统来实现。辅存的位价是最低廉的。

实际上，存储器的层次结构主要体现在缓存—主存和主存—辅存这两个存储层次上，如图 4.2 所示。

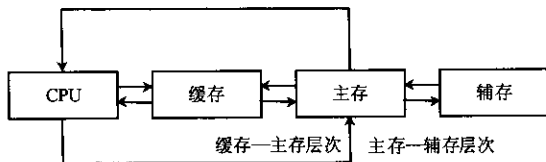


图 4.2 缓存—主存层次和主存—辅存层次

从 CPU 角度来看，缓存—主存这一层次的速度接近于缓存，高于主存；其容量和位价却接近于主存。这就从速度和成本的矛盾中获得了理想的解决办法。主存—辅存这一层次，从整体分析，其速度接近于主存，容量接近于辅存，平均位价也接近于低速、廉价的辅存位价，这又解决了速度、容量、成本这三者矛盾。现代的计算机系统几乎都具有这两个存储层次，构成了缓存、主存、辅存三级存储系统。

在主存—辅存这一层次的不断发展中，形成了虚拟存储系统。在这个系统

中, 程序员编程的地址范围与虚拟存储器的地址空间相对应。例如, 机器指令地址码为 24 位, 则虚拟存储器的存储单元可达 16M。可是这个数与主存的实际存储单元个数相比, 要大得多, 称这类指令地址码叫虚地址 (虚存地址、虚拟地址) 或叫逻辑地址, 而把主存的实际地址称作物理地址或实地址。物理地址是程序在执行过程中能够真正访问的地址, 也是真实存在于主存的存储地址。对具有虚拟存储器的计算机系统而言, 编程时可用的地址空间远远大于主存空间, 使程序员以为自己占有一个容量极大的主存, 其实这个主存并不存在, 这就是我们将其称之为虚拟存储器的原因。对虚拟存储器而言, 其逻辑地址变换为物理地址的工作, 是由计算机系统的硬设备和操作系统自动完成的, 对程序员是透明的。当虚地址的内容在主存时, 机器便可立即使用; 若虚地址的内容不在主存, 则必须先将此虚地址的内容传递到主存的合适单元后再为机器所用。有关这方面的内容, 读者可在《计算机体系结构》和《操作系统》中学到。

4.2 主存储器

4.2.1 概述

主存储器的基本结构已在第一章介绍过, 如图 1.11 所示。实际上, 根据 MAR 中的地址访问某个存储单元时, 还需经过地址译码、驱动等电路, 才能找到所需访问的单元。读出时, 需经过读出放大器, 才能将被选中单元的存储字送到 MDR。写入时, MDR 中的数据也必须经过写入电路才能真正写入到被选中的单元中。可见, 主存的实际结构如图 4.3 所示。

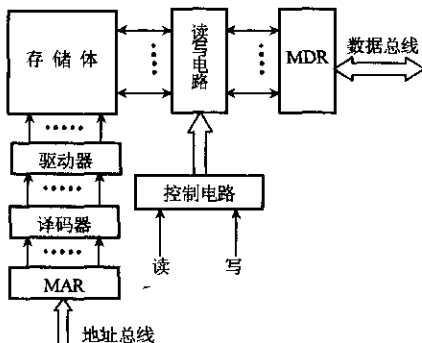


图 4.3 主存的基本组成

现代计算机的主存都由半导体集成电路构成, 图中的驱动器、译码器和读

写电路均制作在存储芯片中，而 MAR 和 MDR 制作在 CPU 芯片内。存储芯片和 CPU 芯片可通过总线连接，如图 4.4 所示。

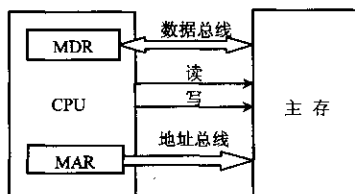


图 4.4 主存和 CPU 的联系

当要从存储器读出某一信息时，首先由 CPU 将该字的地址送到 MAR，经地址总线送至主存，然后发读命令。主存接到读命令后，得知需将该地址单元的内容读出，便完成读操作，将该单元的内容读至数据总线上，至于该信息由 MDR 送至什么地方，这已不是主存的任务，而是由 CPU 决定的。若要向主存存入一个信息字时，首先 CPU 将该字所在主存单元的地址经 MAR 送到地址总线，并将信息字送入 MDR，然后向主存发写命令，主存接到写命令后，便将数据线上的信息写入到对应地址线指出的主存单元中。

1. 主存中存储单元地址的分配

主存各存储单元的空间位置是由单元地址号来表示的，而地址总线是用来指出存储单元地址号的，根据该地址可读出一个存储字。不同的机器存储字长也不同，为了满足字符处理的需要，常用 8 位二进制数表示一个字节，因此存储字长都取 8 的倍数。通常计算机系统既可按字寻址，也可按字节寻址。例如 IBM370 机其字长为 32 位，它可按字节寻址，即它的每一个存储字包含 4 个可独立寻址的字节，其地址分配如图 4.5 (a) 所示。字地址是用该字高位字节的地址来表示，故其字地址是 4 的整数倍，正好用地址码的末两位来区分同一字的 4 个字节的位置。但对 PDP-11 机而言，其字地址是 2 的整数倍，它用低位字节的地址来表示字地址，如图 4.5 (b) 所示。

由图 4.5 (a) 所示，对 24 位地址线的主存而言，按字节寻址的范围是 16M，按字寻址的范围为 4M。由图 4.5 (b) 所示，对 24 位地址线而言，按字节寻址的范围仍为 16M，但按字寻址的范围为 8M。

2. 主存的技术指标

主存的主要技术指标是存储容量和存储速度。

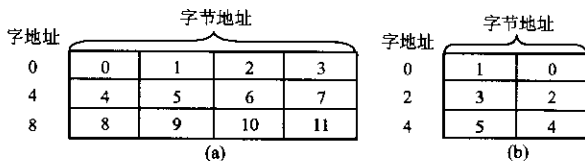


图 4.5 字节寻址的主存地址分配

(1) 存储容量

存储容量是指主存能存放二进制代码的总数，即

$$\text{存储容量} = \text{存储单元个数} \times \text{存储字长}$$

它的容量也可用字节总数来表示，即

$$\text{存储容量} = \text{存储单元个数} \times \text{存储字长} / 8$$

目前的计算机存储容量大多以字节数来表示，如某机存储容量为 4GB，则按字节寻址的地址线位数应对应 32 位。

(2) 存储速度

存储速度是由存取时间和存取周期来表示的。

存取时间又叫存储器的访问时间 (Memory Access Time)，它是指启动一次存储器操作 (读或写) 到完成该操作所需的全部时间。存取时间分读出时间和写入时间两种。读出时间是从存储器接收到有效地址开始，到产生有效输出所需的全部时间。写入时间是从存储器接收到有效地址开始，到数据写入被选中单元为止所需的全部时间。

存取周期 (Memory Cycle Time) 是指存储器进行连续两次独立的存储器操作 (如连续两次读操作) 所需的最小间隔时间，通常存取周期大于存取时间。现代 MOS 型存储器的存取周期可达 100ns；双极型 TTL 存储器的存取周期接近 10ns。

与存取周期密切相关的指标叫存储器的带宽，它表示每秒从存储器进出信息的最大数量，单位可用字/秒或字节/秒或位/秒表示。如存取周期为 500ns，每个存取周期可访问 16 位，则它的带宽为 32M 位/秒。

存储器的带宽决定了以存储器为中心的机器可以获得的信息传输速度，它是改善机器瓶颈的一个关键因素。为了提高存储器的带宽，可以采用以下措施：

- ① 缩短存取周期；
- ② 增加存储字长，使每个周期访问更多的二进制位；
- ③ 增加存储体 (详见 4.2.7 节)。

4.2.2 半导体存储芯片简介

1. 半导体存储芯片的基本结构

半导体存储芯片采用超大规模集成电路制造工艺，在一个芯片内集成具有记忆功能的存储矩阵、译码驱动电路和读写电路等，如图 4.6 所示。

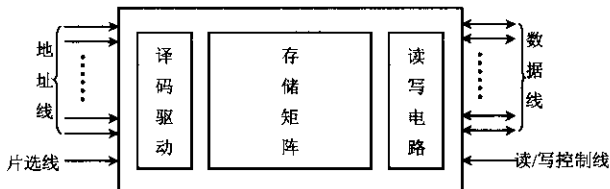


图 4.6 存储芯片的基本结构

译码驱动能把地址总线送来的地址信号翻译成对应存储单元的选择信号，该信号在读写电路的配合下完成对被选中单元的读写操作。

读写电路包括读出放大器和写入电路，用来完成读写操作。

存储芯片通过地址总线、数据总线和控制总线与外部连接。

地址线是单向输入的，其位数与芯片容量有关。

数据线是双向的（有的芯片可用成对出现的数据线分别作输入或输出），其位数与芯片可读出或写入的数据位数有关。

地址线和数据线的位数共同反映存储芯片的容量。如地址线为 10 根，数据线为 4 根，则芯片容量为 $2^{10} \times 4 = 4K$ 。

控制线主要有读/写控制线与片选线两种。读/写控制线决定芯片进行读/写操作，片选线用来选择存储芯片。由于存储器是由许多芯片组成，需用片选信号来确定哪个芯片被选中。例如，一个 $64K \times 8$ 位的存储器可用 32 片 $16K \times 1$ 位存储芯片组成，如图 4.7 所示。但每次读出一个存储字时，只需选中 8 片。

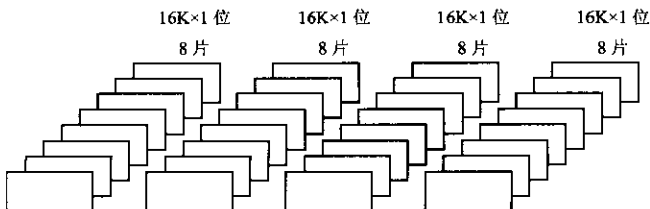


图 4.7 $64K \times 8$ 位的存储器

2. 半导体存储芯片的译码驱动方式

半导体存储芯片的译码驱动方式有两种：线选法和重合法。如图 4.8 和图 4.9 所示。

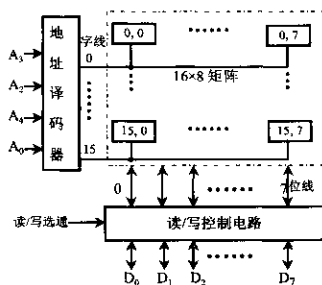


图 4.8 16×1 字节线选法结构示意图

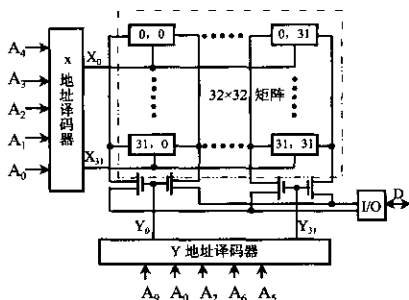


图 4.9 1K×1 位重合法结构示意图

图 4.8 是一个 16×1 字节线选法存储芯片的结构示意图。它的特点是用一根字选择线（字线），直接选中一个存储单元的各位（如一个字节）。这种方式结构较简单，但只适于容量不大的存储芯片。如当地址线 A₃A₂A₁A₀ 为 1111 时，则第 15 根字线被选中，对应图 4.8 中的最后一行八位代码便可直接读出或写入。

图 4.9 是一个 1K×1 位重合法结构示意图。显然，只要用 64 根选择线（X、Y 两个方向各 32 根），便可选择 32×32 矩阵中的任一位。例如当地址线为全 0 时，译码输出 X₀ 和 Y₀ 有效，选中矩阵中的第 0 行、第 0 列那位。由于被选单元是由 X、Y 两个方向地址决定的，故称重合法。当欲构成 1K×1 字节

的存储器时，只需用 8 片图 4.9 所示的芯片即可。

4.2.3 随机存取存储器 (RAM)

随机存取存储器按其存储信息的原理不同，可分为静态 RAM 和动态 RAM 两大类。

1. 静态 RAM (Static RAM 或记作 SRAM)

(1) 静态 RAM 基本单元电路

存储器中用于寄存“0”和“1”代码的电路叫做存储器的基本单元电路，图 4.10 示出一个六个 MOS 管组成的基本单元电路。

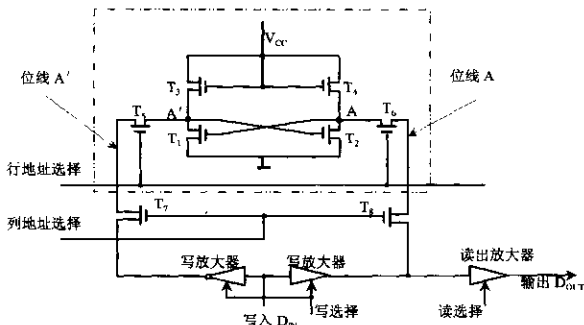


图 4.10 静态 RAM 的基本单元电路

图中 $T_1 \sim T_4$ 是一个由 MOS 管组成的触发器基本电路， T_5 、 T_6 尤如一个开关，受行地址选择信号控制。由 $T_1 \sim T_6$ 共同构成一个六管 MOS 基本单元电路。 T_7 、 T_8 受列地址选择控制，分别与位线 A 和 A' 相连，它们并不包含在基本单元电路内，而是芯片内同一列的各个基本单元电路所共有的。

假设触发器已存有“1”信号，即 A 点为高电平。当需读出时，只要使行、列地址选择信号均为有效，则使 T_5 、 T_6 、 T_7 、 T_8 均导通，A 点高电平通过 T_6 后，再由位线 A 通过 T_8 作为读出放大器的输入信号，在读选择有效时，将“1”信号读出。

由于静态 RAM 是触发器存储信息，因此即使信息读出后，它仍保持其原状态，不需要再生。但电源掉电时，原存信息丢失，故它属易失性半导体存储器。

写入时可以不管触发器原状态如何，只要将写入代码送至图 4.10 的 D_{IN} 端，在写选择有效时，经两个写放大器，使两端输出为相反电平。当行、列地址选择有效时，使 T_5 、 T_6 、 T_7 、 T_8 导通，并使 A 与 A' 点置成完全相反的电平。

这样,就把欲写入的信号写入到该单元电路中。如欲写入“1”,即 $D_N=1$,经两个写放大器使位线 A 为高电平,位线 A' 为低电平,结果使 A 点为高, A' 点为低,即写入了“1”信息。

(2) 静态 RAM 芯片举例

Intel 2114 芯片的基本单元电路由六个 MOS 管组成,图 4.11 是一个容量为 $1K \times 4$ 位的 2114 外特性示意图。

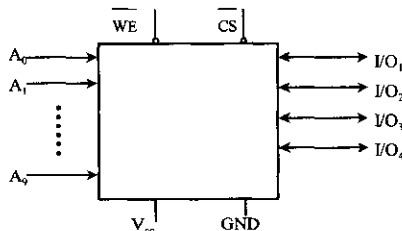


图 4.11 Intel 2114 外特性示意图

图中 $A_9 \sim A_0$ 为地址输入端;

$I/O_1 \sim I/O_4$ 为数据输入/输出端;

$\overline{CS}$ 为片选信号 (低电平有效);

$\overline{WE}$ 为写允许信号 (低电平为写);

V_{cc} 为电源端;

GND 为接地端。

图 4.12 为 2114 芯片内的结构示意图。其中每一个小方块均为一个六管 MOS 触发器基本单元电路,排列成 64×64 矩阵,64 列对应 64 对 T_7 、 T_8 管。又将 64 列分成 4 组,每组包含 16 列,并与一个读写电路相连,读写电路受 $\overline{WE}$ 和 $\overline{CS}$ 控制,4 个读写电路对应 4 根数据线 $I/O_1 \sim I/O_4$ 。由图可见,行地址经译码后可选中某一行;列地址经译码后可选中四组中的对应列,共四列。

当对某个基本单元电路进行读/写操作时,必须被行、列地址共同选中。例如,当 $A_9 \sim A_0$ 为全 0 时,对应行地址 $A_8 \sim A_3$ 为 000000,列地址 A_2, A_1, A_0 也为 0000,则第 0 行的第 0、16、32、48 这 4 个基本单元电路被选中。此刻,若作读操作,则 $\overline{CS}$ 为低电平, $\overline{WE}$ 为高电平,在读写电路的输出端 $I/O_1 \sim I/O_4$ 便输出第 0 行的第 0、16、32、48 这四个单元电路所存的信息。若作写操作,将写入信息送至 $I/O_1 \sim I/O_4$ 端口,并使 $\overline{CS}$ 为低电平、 $\overline{WE}$ 为低电平,同样这四个输入信息将分别写入到第 0 行的第 0、16、32、48 四个单元之中。

(3) 静态 RAM 读写时序

• 读周期时序

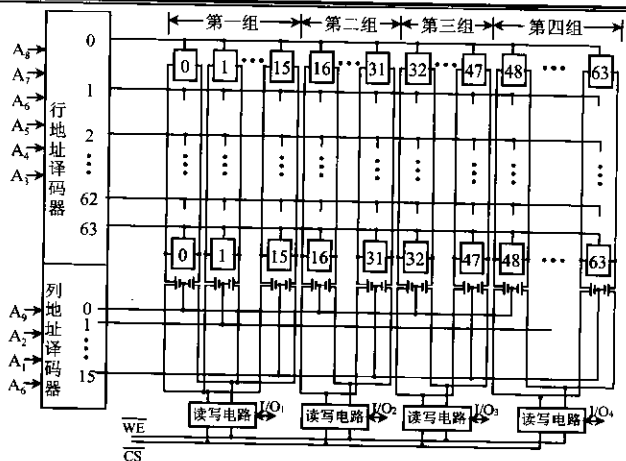


图 4.12 2114 RAM 矩阵结构示意图

图 4.13 是 2114 RAM 芯片读周期时序，在整个读周期中 $\overline{WE}$ 始终为高电平（故图中省略）。读周期 t_{RC} 是指对芯片进行两次连续读操作的最小间隔时间。读时间 t_A 表示从地址有效到数据稳定所需的时间，显然读时间小于读周期。图中 t_{CO} 是从片选有效到输出稳定的时间。可见只有当地址有效经 t_A 后，且当片选有效经 t_{CO} 后，数据才能稳定输出，这两者必须同时具备。根据 t_A 和 t_{CO} 的值，便可知当地址有效后，经 $t_A - t_{CO}$ 时间必须给出片选有效信号，否则信号不能出现在数据线上。

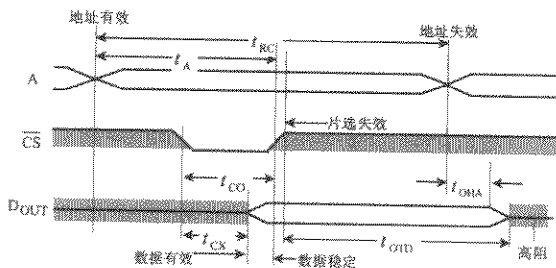


图 4.13 2114 RAM 的读周期时序

需注意一点,从片选失效到输出高阻需一段时间 t_{OTD} ,故地址失效后,数据线上的有效数据有一段维持时间 t_{OHA} ,以保证所读数据可靠。

• 写周期时序

图 4.14 是 2114 RAM 写周期时序。

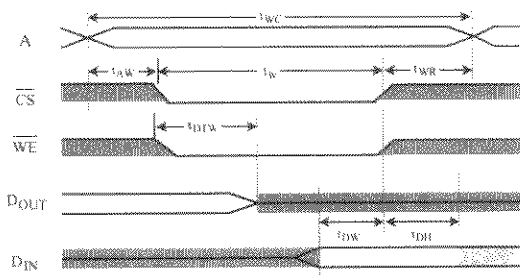


图 4.14 2114RAM 的写周期时序

写周期 t_{WC} 是对芯片进行连续两次写操作的最小间隔时间。写周期包括滞后时间 t_{AW} 、写入时间 t_{W} 和写恢复时间 t_{WR} 。在有效数据出现前, RAM 的数据线上存在着前一时刻的数据 D_{OUT} (如图 4.13 所示的维持时间), 故在地址线发生变化后, $\overline{\text{CS}}$ 、 $\overline{\text{WE}}$ 均需滞后 t_{AW} 再有效, 以避免将无效数据写入到 RAM 的错误。但写允许 $\overline{\text{WE}}$ 失效后, 地址必须保持一段时间, 叫做写恢复时间。此外, RAM 数据线上的有效数据 (即 CPU 送至 RAM 的写入数据 D_{IN}) 必须在 $\overline{\text{CS}}$ 、 $\overline{\text{WE}}$ 失效前的 t_{DW} 时刻出现, 并延续一段时间 t_{DH} (此刻地址线仍有效, $t_{\text{WR}} > t_{\text{DH}}$), 以保证数据可靠写入。

已制成的 RAM 芯片其读/写时序关系已被确定, 因此, 将它与 CPU 连接时, 必须注意它们相互间的时序匹配关系, 否则 RAM 将无法正常工作。

2. 动态 RAM (Dynamic RAM 或 DRAM)

(1) 动态 RAM 的基本单元电路

常见的动态 RAM 基本单元电路有三管式和单管式两种, 它们的共同特点都是靠电容存储电荷的原理来寄存信息的。若电容上存有足够多的电荷表示存“1”, 电容上无电荷表示存“0”。电容上的电荷一般只能维持 1~2ms, 因此即使电源不掉电, 信息也会自动消失。为此, 必须在 2ms 内对其所有存储单元恢复一次原状态, 这个过程叫再生或刷新。由于它与静态 RAM 相比, 具有集成度更高, 功耗更低等特点, 目前被各类计算机广泛应用。

图 4.15 示意了由 T_1 、 T_2 、 T_3 三个 MOS 管组成的三管 MOS 动态 RAM 基本单元电路。

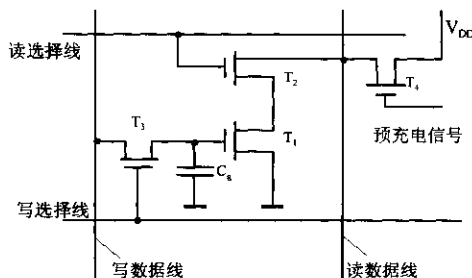


图 4.15 三管 MOS 动态 RAM 基本单元电路

读出时，先对预充电管 T_4 置一预充电信号（在存储矩阵中，每一列共用一个 T_4 管），使读数据线达高电平 V_{DD} 。然后由读选择线打开 T_2 ，若 T_1 的极间电荷 C_g 存有足够多的电荷（被认为原存“1”），使 T_1 导通，则因 T_2 、 T_1 导通接地，使读数据线降为零电平，读出“0”信息。若 C_g 没足够电荷（原存“0”），则 T_1 截止，读数据线为高电平不变，读出“1”信息。可见，由读出线的高低电平可区分其是读“1”，还是读“0”，只是它与原存信息反相。

写入时，将写入信号加到写数据线上，然后由写选择线打开 T_3 ，这样， C_g 便能随输入信息充电（写“1”）或放电（写“0”）。

为了提高集成度，将三管电路进一步简化，去掉 T_1 ，把信息存在电容 C_s 上，将 T_2 、 T_3 合并成一个管子 T ，便得单管 MOS 动态 RAM 基本单元电路，如图 4.16 所示。

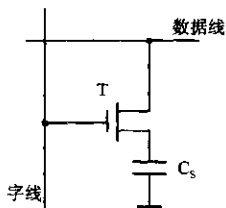


图 4.16 单管 MOS 动态 RAM 基本单元电路

读出时，字线上的高电平使 T 导通，若 C_s 有电荷，经 T 管在数据线上产生电流，可视为读出“1”。若 C_s 无电荷，则数据线上无电流，可视为读出“0”。读操作结束时， C_s 的电荷已泄放完毕，故是破坏性读出，必须再生。

写入时，字线为高电平使 T 导通，若数据线上为高电平，经 T 管对 C_s 充

电, 使其存“1”; 若数据线为低电平, 则 C_s 经 T 放电, 使其无电荷而存“0”。

(2) 动态 RAM 芯片举例

① 三管动态 RAM 芯片

三管动态 RAM 芯片的结构如图 4.17 所示。

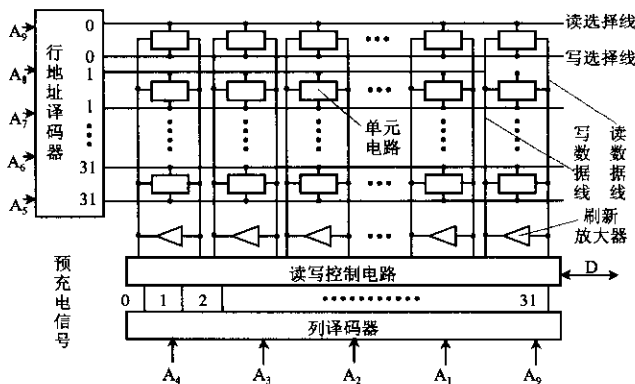


图 4.17 1K×1 位三管 MOS 动态 RAM 结构示意图

这是一个 1K×1 位的存储芯片，图中每一小方块代表由三个 MOS 管组成的动态 RAM 基本单元电路。它们排列成 32×32 的矩阵，每列都有一个刷新放大器（用来形成再生信息）和一个预充电管，芯片有 10 根地址线，采用重合法选择基本单元电路。

读出时，先置以预充电信号，接着按行地址 $A_5 \sim A_9$ 经行译码器给出读选择信号，同时由列地址 $A_0 \sim A_4$ 经列译码器给出列选择信号。只有在行、列选择信号共同作用下的基本单元电路，才能将其信息经读数据线送到读写控制电路，并从数据线 D 输出。

写入时，首先将写入信息由数据线 D 送入读写控制电路，并在列地址的作用下，由列译码器的输出控制输入信息只送到被选中列的写数据线上。然后在受行地址控制的行译码器给出的写选择信号的作用下，信息被写入到行列共同选中的基本单元电路内。

② 单管动态 RAM 芯片

单管动态 RAM 芯片结构的示意图如图 4.18 所示。这是一个 16K×1 位的存储芯片，按理应有 14 根地址线，但为了减少芯片封装的引脚数，地址线只

有 7 根。因此, 地址信息分两次传送, 先送 7 位行地址, 再送 7 位列地址。芯片内有时序电路, 它受行地址选通 $\overline{\text{RAS}}$ 、列地址选通 $\overline{\text{CAS}}$ 以及写允许信号 $\overline{\text{WE}}$ 控制。

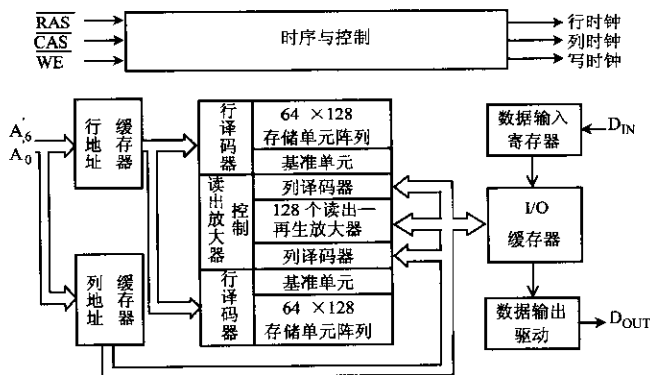


图 4.18 4116 动态 RAM (16K×1 位) 芯片结构

16K×1 位的存储芯片共有 16K 个单管 MOS 基本单元电路, 它们排列成 128×128 的矩阵, 如图 4.19 所示。图中的行线就是图 4.16 中的字线, 列线就是图 4.16 中的数据线。128 行分布在读放大器的左、右两侧 (左侧为 0~63 行, 右侧为 64~127 行)。每根行选择线与 128 个 MOS 管的栅极相连。128 列共有 128 个读放大器, 它的两侧又分别与 64 个 MOS 管相连, 每根列线上都有一个列地址选择管。128 个列地址选择管的输出又互相并接在一起与 I/O 缓冲器相连, I/O 缓冲器的一端接输出驱动器, 作为数据输出, 另一端接输入器, 作为数据输入。

读出时, 行、列地址受 $\overline{\text{RAS}}$ 和 $\overline{\text{CAS}}$ 控制, 分两次分别存入行、列地址缓冲器。行地址经行译码后选中一行, 使该行上所有的 MOS 管均导通, 并分别将其电容 C_s 上的电荷反映到 128 个读放大器的某一侧 (第 0~63 行反映到读放大器的左侧, 第 64~127 行反映到读放大器的右侧)。读放大器实质上是个触发器, 其左右两侧电平相反。此外列地址经列译码后选中某一列, 该列上的列地址选择管导通, 即可将读放大器右侧信号经读/写线、I/O 缓冲器输出至 D_{OUT} 端。例如, 选中第 63 行、第 0 列的单管 MOS 电路, 其 C_s 有电荷为 “1” 状态, 则反映到第 0 列读放的左侧为 “1”, 右侧为 “0”, 经列地址选择管输出至 D_{OUT} 为 0, 与原存信息反相。同理, 第 0~62 行经读放至输出线 D_{OUT} 的信息

与原存信息均反相。而读出第 64~127 行时, 因它们的电容 C_s 上的电荷均反映到读放的右侧, 故经列地址选择管输出至 D_{OUT} 的信息为同相。

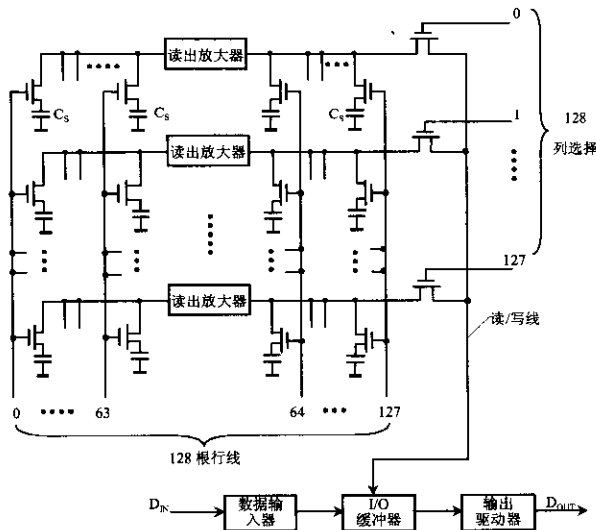


图 4.19 16K×1 位 4116 动态 RAM 存储矩阵示意图

写入时, 行、列地址也要分别送入芯片内的行、列地址缓存器, 经译码可选中某列。输入信息 D_{IN} 通过数据输入器, 经 I/O 缓冲器送至读/写线上, 但只有被选中的列地址选择管导通, 可将读/写线上的信息送至读放大器, 破坏了读放大器的平衡, 使读放大器的右侧与输入信息同相, 左侧与输入信息反相, 读放大器的信息便可写入到选中行的 C_s 中。例如, 选中第 64 行、第 127 列, 输入信息为“1”, 则第 127 列地址选择管导通, 将“1”信息送至第 127 列的读放右侧。虽然第 64 行上的 128 个 MOS 管均导通, 但唯有第 64 行、第 127 列的 MOS 管能将其读放右侧信息“1”对 C_s 充电, 使其写入“1”。值得注意的是写入读放左侧行的信息与输入信息都是反相的, 而由读出过程分析又知, 对读放左侧行进行读操作时, 读出的信息也是反相的, 故最终结果是正确的。

(3) 动态 RAM 时序

由图 4.18 可知, 动态 RAM 的行、列地址是分别传送的, 因此分析其时序时, 应特别注意 $\overline{RAS}$ 、 $\overline{CAS}$ 与地址的关系。即

- 先由 $\overline{RAS}$ 将行地址送入行地址缓存器, 再由 $\overline{CAS}$ 将列地址送入列地址

缓存器, 因此, $\overline{\text{CAS}}$ 滞后于 $\overline{\text{RAS}}$ 的时间必须要超过其规定值。

- RAS 和 $\overline{\text{CAS}}$ 正、负电平的宽度应大于规定值, 以保证芯片内部正常工作。
- 行、列地址对 $\overline{\text{RAS}}$ 和 $\overline{\text{CAS}}$ 的下沿 (负跳变) 应满足有足够的地址建立时间和地址保持时间, 以确定行、列地址均能准确写入芯片。

① 读时序

在读工作方式时 (写允许 $\overline{\text{WE}}=1$), 读工作周期是指动态 RAM 完成一次“读”所需的最短时间 t_{CRD} , 也是 $\overline{\text{RAS}}$ 的一个周期。由图 4.20 所示, 为了确保读出数据无误, 必须要求写允许 $\overline{\text{WE}}=1$ 在列地址送入前 (即 $\overline{\text{CAS}}$ 下沿到来前) 建立, 而 $\overline{\text{WE}}=1$ 的撤除应在 $\overline{\text{CAS}}$ 失效后 (即 $\overline{\text{CAS}}$ 上升沿后); 还要求读出数据应在 $\overline{\text{RAS}}$ 有效后一段时间 t_{aRAS} 且 $\overline{\text{CAS}}$ 有效后一段时间 t_{bCAS} 时出现, 而数据有效的撤除时间, 应在 $\overline{\text{CAS}}$ 失效后一段时间 $t_{\text{hCAS-OUT}}$ 。

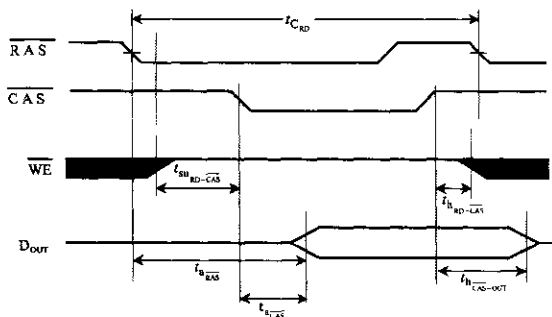


图 4.20 动态 RAM 读工作方式时序图

② 写时序

在写工作方式时 (写允许 $\overline{\text{WE}}=0$), $\overline{\text{RAS}}$ 的一个周期 t_{CWR} 即为写工作周期, 如图 4.21 所示。

为了确保写入数据准确无误, $\overline{\text{WE}}=0$ 应先于 $\overline{\text{CAS}}=0$, 而且数据的有效存在时间应与 $\overline{\text{CAS}}$ 及 $\overline{\text{WE}}$ 的有效相对应。即写入数据应在 $\overline{\text{CAS}}$ 有效前的一段时

间 $t_{\text{SU DIN-CAS}}$ 出现, 它的保持时间应为 $\overline{\text{CAS}}$ 有效后的一段时间 $t_{\text{H DIN-CAS}}$, 这是因为

数据的写入实际上是由 $\overline{\text{CAS}}$ 的下沿激发而成的。可见, 为了保证正常写入, $\overline{\text{WE}}$ 、 $\overline{\text{CAS}}$ 有效均要大于数据 D_{IN} 有效的时间。

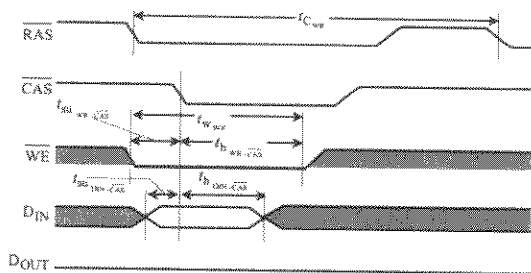


图 4.21 动态 RAM 写工作方式时序图

此外, 动态 RAM 还有读-改写工作方式和页面工作方式, 本书不再赘述。

(4) 动态 RAM 的刷新

刷新的过程实质上是先将原存信息读出, 再由刷新放大器形成原信息并重新写入的再生过程 (图 4.17 中的刷新放大器及图 4.19 中的读放大器均起此作用)。

由于存储单元被访问是随机的, 有可能某些存储单元长期得不到访问, 无读出也就无重写, 其原信息必然消失。为此, 必须采用定时刷新的方法, 它规定在一定的时间内, 对动态 RAM 的全部基本单元电路必作一次刷新, 一般取 2ms, 这个时间叫做刷新周期, 或叫再生周期。在刷新周期内, 由专用的刷新电路来完成对基本单元电路的逐行刷新。通常有两种方式刷新:

① 集中刷新

集中刷新是在规定的一个刷新周期内, 对全部存储单元集中一段时间逐行进行刷新, 此刻必须停止读/写操作。如 Intel 1103 动态 RAM 芯片内排列成 32×32 矩阵, 读/写周期为 $0.5\mu\text{s}$, 连续刷新 32 行需 $16\mu\text{s}$ (占 32 个读/写周期)。在刷新周期 2ms 内含 4000 个读/写周期, 实际分配是前 3968 个周期用于读/写操作或维持, 后 32 个周期用于刷新, 如图 4.22 所示。

这种刷新方式缺点在于出现了访存“死区”, 其占比例为 $32/4000 \times 100\% = 0.8\%$, 显然对高速高效的计算机系统工作是不利的。

② 分散刷新

分散刷新是指对每行存储单元的刷新分散到每个读/写周期内完成。把存

取周期分成两段，前半段用来读写或维持，后半段用来刷新，如图 4.23 所示。

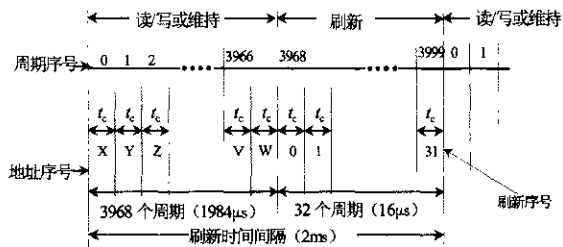


图 4.22 集中刷新时间分配示意图

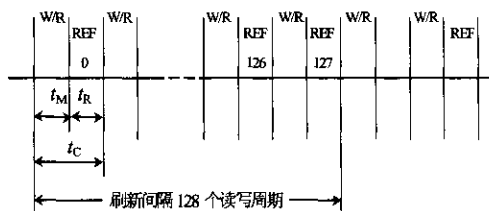


图 4.23 分散刷新时间分配示意图

显然，这种刷新克服了集中刷新出现“死区”的缺点，但它并不能提高整机的工作效率。因为尽管刷新分散在读/写周期之后，但刷新同样需要一个读/写周期时间，结果使机器存取周期由 $0.5\mu\text{s}$ 变成 $1\mu\text{s}$ ，使整机工作效率下降。

为了真正提高整机的工作效率，应该采用集中与分散相结合的方式，既克服出现“死区”，又充分利用最大刷新间隔为 2ms 的特点。例如，对于 128×128 的存储芯片，可采取在 2ms 内对 128 行各刷新一遍，即每隔 $15.6\mu\text{s}$ 刷新一行，而每行刷新的时间仍为读/写周期 $0.5\mu\text{s}$ 。这样，刷新一行只停止一个读/写周期，即对每行来说，刷新时间仍为 2ms ，而“死区”缩短为 $0.5\mu\text{s}$ 。然而，我们可以利用 CPU 对指令的译码阶段，即不访问主存的这段时间，安排动态 RAM 的刷新操作。这样，既不会出现集中刷新的“死区”问题，又解决了分散刷新独立占据 $0.5\mu\text{s}$ 的读/写周期问题，因此，从根本上提高了整机的工作效率。

3. 动态 RAM 与静态 RAM 的比较

目前，动态 RAM 的应用比静态 RAM 要广泛得多。其原因是：①在同样

大小的芯片中, 动态 RAM 的集成度远高于静态 RAM, 如动态 RAM 的基本单元电路为一个 MOS 管, 静态 RAM 的基本单元电路为六个 MOS 管; ②动态 RAM 行、列地址按先后顺序输送, 减少了芯片引脚, 封装尺寸也减少; ③动态 RAM 的功耗仅为静态 RAM 的 1/6; ④动态 RAM 的价格仅为静态 RAM 的 1/4。因此, 随着动态 RAM 容量不断扩大, 速度不断提高, 它被广泛应用于计算机的主存。

动态 RAM 也有缺点: ①由于使用动态元件 (电容), 因此它的速度比静态 RAM 低; ②动态 RAM 需要再生, 故需配置再生电路, 也需要消耗一部分功率。通常, 容量不大的高速存储器大多用静态 RAM 实现。

4.2.4 只读存储器 (ROM)

按 ROM 的原始定义, 一旦注入原始信息后是不能改变的, 但随着用户的需要, 总希望能任意修改 ROM 内的原始信息。这便出现了 PROM、EPROM 和 EEPROM 等多种。对于半导体 ROM, 基本器件为两种: MOS 型和 TTL 型。

1. 掩膜 ROM

图 4.24 所示为 MOS 型掩膜 ROM, 其容量为 $1K \times 1$ 位, 采用重合法驱动, 行、列地址线分别经行、列译码器, 各得 32 根行、列选择线。行选择线与列选择线交叉处既可有耦合元件 MOS 管, 也可没有。列选择线各控制一个列控

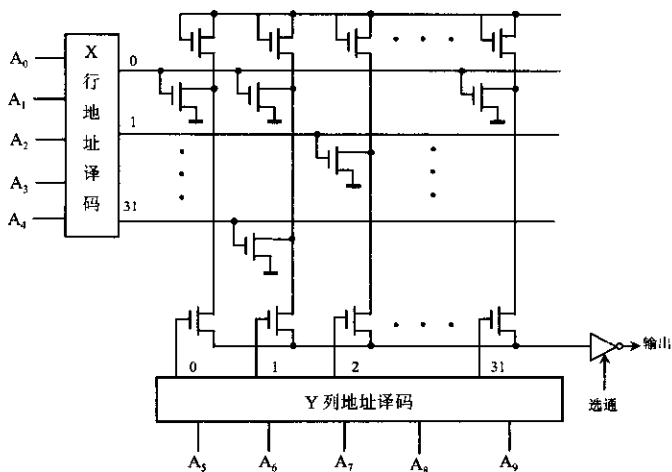


图 4.24 $1K \times 1$ 位的 MOS 管掩膜 ROM

选择线交叉处既可有耦合元件 MOS 管,也可没有。列选择线各控制一个列控制管,32 个列控制管的输出端共连一个读放大器。当地址为全“0”时,第 0 行、0 列被选中,若其交叉处有耦合元件 MOS 管,因其导通而使列线输出为低电平,经读放反相为高电平,输出“1”。当地址 $A_4 \sim A_9$ 为 11111, $A_0 \sim A_3$ 为 00000 时,即第 31 行、第 0 列被选中,但此刻行、列的交叉处无 MOS 管,故 0 列线输出为高电平,经读放反相为“0”输出。可见,用行、列交叉处是否有耦合元件 MOS 管,便可区分原存“1”还是存“0”。当然,此 ROM 制成后不可能改变原行、列交叉处的 MOS 管是否存在,所以,用户是无法改变原始状态的。

2. PROM

PROM 是可以实现一次性编程的只读存储器,图 4.25 示意一个由双极型电路和熔丝构成的基本单元电路。图 4.26 即是由该基本单元电路构成的 16×1 位双极型镍铬熔丝式 PROM 芯片。

在这个电路中,基极由行选择线控制,发射极与列线之间形成一条镍铬合金薄膜制成的熔丝(可用光刻技术实现),集电极接电源 V_{CC} 。用户在使用前,可按需要将信息存入行、列交叉的耦合元件内。若欲存“0”,则置耦合元件一大电流,将熔丝烧掉。若欲存“1”,则耦合处不置大电流,熔丝不断。当被选中时,熔丝断开处将读得“0”,熔丝未断处将读得“1”。当然,已断的熔丝是无法再恢复的,故这种 ROM 往往只能实现一次编程,不得再修改。

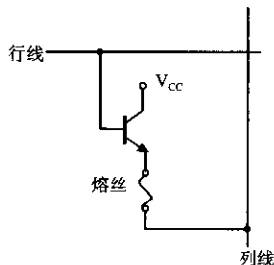


图 4.25 双极型镍铬熔丝式单元电路

3. EPROM

EPROM 是一种可擦洗可编程的只读存储器。它可以由用户对其所存信息作任意次的改写。目前用得较多的 EPROM 是用浮动栅雪崩注入型 MOS 管构成,又称 FAMOS 型 EPROM,如图 4.27 所示。

由图所示的 N 型沟道浮动栅 MOS 电路，在漏端 D 加上正电压（如 25V，50ms 宽的正脉冲），便会形成一个浮动栅，它阻止源 S 与漏 D 之间的导通，致使此 MOS 管处于“0”状态。若对 D 端不加正电压，则形成不了浮动栅，此 MOS 管便能正常导通，呈“1”状态。由此，用户可按需要对不同位置的 MOS 管 D 端施正电压或不施正电压，便制成了用户所需的 ROM。一旦用户需重新改变其状态时，可用紫外线照射，驱散浮动栅，再按需要对不同位置的

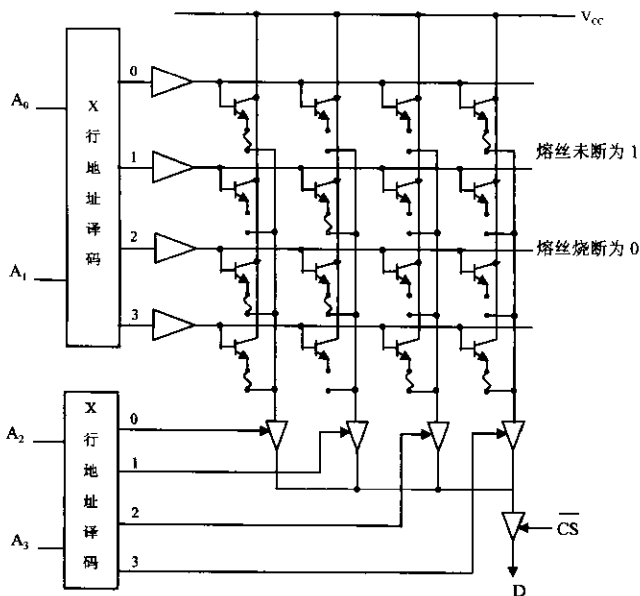


图 4.26 16×1 位双极型镍铬熔丝式 PROM

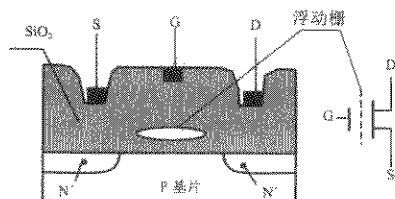


图 4.27 N 型沟道浮动栅 MOS 电路

MOS 管 D 端重新置于正电压，又得出新状态的 ROM。故称之为 EPROM。

图 4.28 为 2716 型 EPROM 的逻辑图和引脚图。

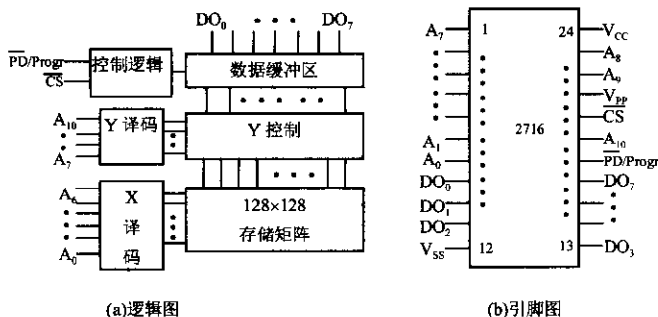


图 4.28 2716 EPROM 逻辑框图及引脚

这类芯片的外引脚除地址线、数据线外，还有两个电源引出头 V_{CC} 和 V_{PP} ，其中 V_{CC} 接 +5V， V_{PP} 平时接 +5V，当其接 +25V 时用来完成编程的需要， V_{SS} 为地。 $\overline{CS}$ 为片选端，读出时为低电平，编程写入时为高电平； $\overline{PD}/\text{progr}$ 是功率下降/编程输入端，在读出时为低电平。当此端为高电平时，可以使 EPROM 功耗由 525mW 降至 132mW。当需编程时，此端需加宽度为 50~55ms、+5V 的脉冲。

EPROM 的改写可用两种方法，一种用紫外线照射，但擦洗时间比较长，而且不能对个别需改写的单元进行单独擦洗或重写。另一种方法用电气方法将存储内容擦除，再重写。甚至在联机条件下，用字擦除方式或页擦除方式，既可局部擦写，又可全部擦写，这种 EPROM 叫 EEPROM。

进入到 20 世纪 80 年代，又出现了一种闪速存储器 (Flash Memory)，又叫快擦型存储器，它是在 EPROM 和 EEPROM 工艺基础上产生的一种新型的、具有性能价格比更好、可靠性更高的可擦写非易失性存储器。它既有 EPROM 的价格便宜、集成度高的优点，又有 EEPROM 电可擦洗重写的特性。它具有整片擦除的特点，其擦除、重写的速度快。一块 1M 位的闪速存储芯片的擦除、重写时间小于 5 μ s，比一般标准的 EEPROM 快得多，已具备了 RAM 的功能。它还具有高速编程的特点，例如采用快速脉冲编程算法对 28F256 快擦型存储器芯片每字节的编程时间仅需 100 μ s。此外，该器件具有高速的存储器访问周期，功耗很低及与计算机接口简单等优点。

在需要周期性地修改存储信息的应用场合, Flash Memory 是一个极为理想的器件,因为它至少可以擦写/编程 10000 次,这足以满足用户需要。它比较适合于作为一种高密度、非易失的数据采集和存储器件。在便携式计算机、工控系统及单片机系统中得到大量应用,近年来已将它用于微型计算机中存放输入输出驱动程序和参数等。

非易失性、长期反复使用的大容量 Flash Memory 还可替代软盘或硬盘,作为海量存储器。如在笔记本及手掌型袖珍电脑中都大量采用 Flash Memory 做成固态盘替代磁盘,使计算机平均无故障时间大大延长,功耗更低,体积更小,消除了机电式磁盘驱动器所造成的数据瓶颈。

4.2.5 存储器与 CPU 的连接

1. 存储容量的扩展

由于单片存储芯片的容量总是有限的,很难满足实际的需要,因此,必须将若干存储芯片连在一起才能组成足够容量的存储器,这就叫存储容量的扩展,通常有位扩展和字扩展。

(1) 位扩展

位扩展是指增加存储字长,如 2 片 $1\text{K} \times 4$ 位的芯片,可组成 $1\text{K} \times 8$ 位的存储器,如图 4.29 所示。图中两片 2114 的地址线 $A_9 \sim A_0$ 、 $\overline{\text{CS}}$ 、 $\overline{\text{WE}}$ 都分别连在一起,其中一片的数据线作为高 4 位 $D_7 \sim D_4$,另一片的数据线作为低 4 位 $D_3 \sim D_0$ 。这样,它便构成了一个 $1\text{K} \times 8$ 位的存储器。

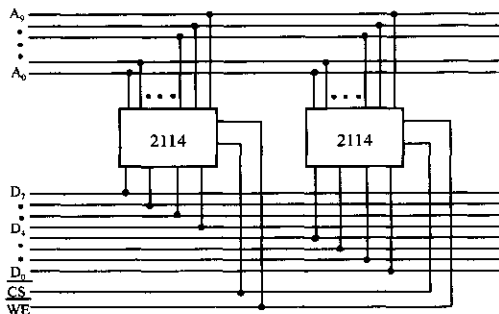
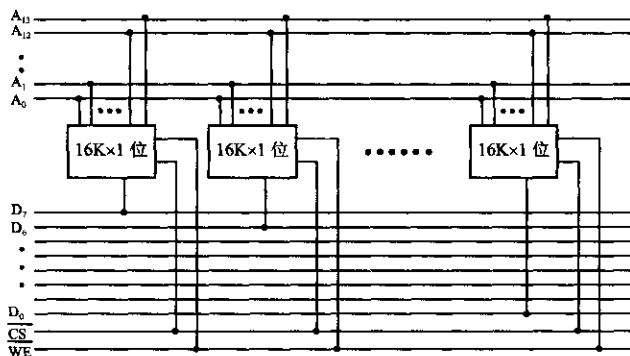


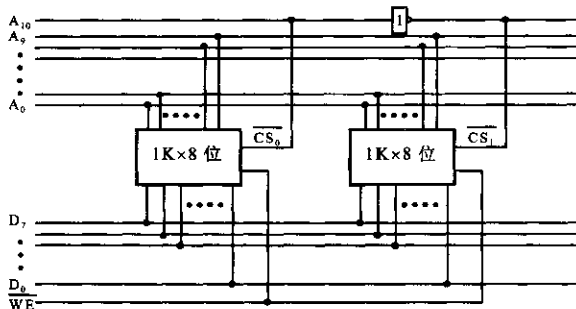
图 4.29 由两片 $1\text{K} \times 4$ 位的芯片组成 $1\text{K} \times 8$ 位的存储器

又如,将 8 片 $16\text{K} \times 1$ 位的存储芯片连接,可组成一个 $16\text{K} \times 8$ 位的存储器,如图 4.30 所示。

图 4.30 由八片 $16\text{K} \times 1$ 位的芯片组成 $16\text{K} \times 8$ 位的存储器

(2) 字扩展

字扩展是指增加存储器字的数量。如用 2 片 $1\text{K} \times 8$ 位的存储芯片，可组成一个 $2\text{K} \times 8$ 位的存储器，即存储字增加了一倍，如图 4.31 所示。

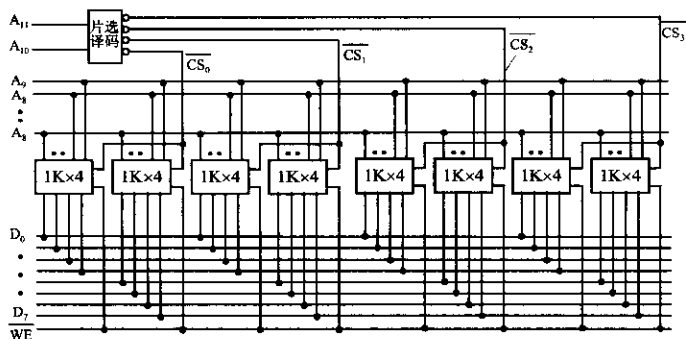
图 4.31 由两片 $1\text{K} \times 8$ 位的芯片组成 $2\text{K} \times 8$ 位的存储器

在此，将 A_{10} 用作片选信号。由于存储芯片的片选输入端要求低电平有效，故当 A_{10} 为低时， $\overline{CS_0}$ 有效，选中左边的 $1\text{K} \times 8$ 位芯片；当 A_{10} 为高时，反相后 $\overline{CS_1}$ 有效，选中右边的 $1\text{K} \times 8$ 位芯片。

(3) 字、位扩展

字、位扩展是指既增加存储字的数量，又增加存储字长。图 4.32 示意用 8

片 $1\text{K} \times 4$ 位的芯片组成 $4\text{K} \times 8$ 位的存储器。



图

4.32 由 8 片 $1\text{K} \times 4$ 位的芯片组成 $4\text{K} \times 8$ 位的存储器

由图可见，每两片构成 $1\text{K} \times 8$ 位的存储器，4 组两片便构成 $4\text{K} \times 8$ 位的存储器。地址线 A_{11} 、 A_{10} 经片选译码器得 4 个片选信号 $\overline{CS}_0$ 、 $\overline{CS}_1$ 、 $\overline{CS}_2$ 、 $\overline{CS}_3$ ，分别选择其中 $1\text{K} \times 8$ 位的存储芯片。 $\overline{WE}$ 为读/写控制信号。

2. 存储器与 CPU 的连接

存储芯片与 CPU 芯片相连时，特别要注意它们片与片之间的地址线、数据线和控制线的连接。

(1) 地址线的连接

存储芯片容量不同，其地址线数也不同，而 CPU 的地址线数往往比存储芯片的地址线数要多。通常总是将 CPU 地址线的低位与存储芯片的地址线相连。CPU 地址线的高位或作存储芯片扩充时用，或作其他用法，如作片选信号等。例如，设 CPU 地址线为 16 位 $A_{15} \sim A_0$ ， $1\text{K} \times 4$ 位的存储芯片仅有 10 根地址线 $A_9 \sim A_0$ ，此时，可将 CPU 的低位地址 $A_9 \sim A_0$ 与存储芯片地址线 $A_9 \sim A_0$ 相连。又如当用 $16\text{K} \times 1$ 位存储芯片时，则其地址线有 14 根 $A_{13} \sim A_0$ ，此时，可将 CPU 的低位地址 $A_{13} \sim A_0$ 与存储芯片地址线 $A_{13} \sim A_0$ 相连。

(2) 数据线的连接

同样，CPU 的数据线数与存储芯片的数据线数也不一定相等。此时，必须对存储芯片扩位，使其数据位数与 CPU 的数据线数相等。

(3) 读/写命令线的连接

CPU 读/写命令线一般可直接与存储芯片的读/写控制端相连，通常高电平为读，低电平为写。

(4) 片选线的连接

片选信号的连接是 CPU 与存储芯片正确工作的关键。由于存储器是由许多存储芯片叠加组成的，哪一片被选中完全取决于该存储芯片的片选控制端 $\overline{CS}$ 是否能接收到来自 CPU 的片选有效信号。

片选有效信号与 CPU 的访存控制信号 $\overline{MREQ}$ （低电平有效）有关，因为只有当 CPU 要求访存时，才要求选择存储芯片。若 CPU 访问 I/O，则 $\overline{MREQ}$ 为高，表示不要求存储器工作。此外，片选有效信号还和地址有关，因为 CPU 给出的存储单元地址的位数往往大于存储芯片的地址线数，故那些未与存储芯片连上的高位地址必须和访存控制信号共同作用，产生存储器的片选信号。通常需用到一些逻辑电路，如译码器及其他各种门电路。

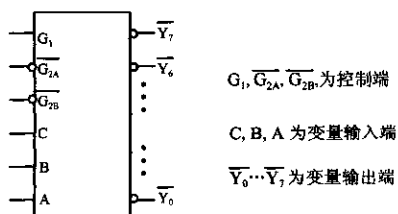
(5) 合理选择存储芯片

合理选择存储芯片主要是指存储芯片类型（RAM 或 ROM）和数量的选择。通常选用 ROM 存放系统程序、标准子程序和各类常数等。RAM 则是为用户编程而设置的。此外，在考虑芯片数量时，要尽量使连线简单方便。

读者在实际应用 CPU 与存储芯片时，将还会遇到两者时序的配合问题、速度问题、负载匹配问题等等，希望通过实验和实际工作进一步加深体会。

下面用一个实例来剖析 CPU 与存储芯片的连接方式。

例 4.1 设 CPU 有 16 根地址线，8 根数据线，并用 $\overline{MREQ}$ 作访存控制信号（低电平有效），用 $\overline{WR}$ 作读/写控制信号（高电平为读，低电平为写）。现有下列存储芯片：1K×4 位 RAM；4K×8 位 RAM；8K×8 位 RAM；2K×8 位 ROM；4K×8 位 ROM；8K×8 位 ROM 及 74LS138 译码器和各种门电路，如图 4.33 所示。画出 CPU 与存储器的连接图，要求：



74 138 译码器



图 4.33 译码器和门电路

① 主存地址空间分配：

6000H~67FFH 为系统程序区;

6800H~6BFFH 为用户程序区。

② 合理选用上述存储芯片, 说明各选几片?

③ 详细画出存储芯片的片选逻辑图。

此题按下述步骤完成:

第一步, 先将 16 进制地址范围写成二进制地址码, 并确定其总容量。

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	
0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	} 系统程序区 (S.P.R.) 2K×8 位
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
0	1	1	0	0	1	1	1	1	1	1	1	1	1	1	1	} 用户程序区 (U.P.R.) 1K×8 位
0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	0	
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	
0	1	1	0	1	0	1	1	1	1	1	1	1	1	1	1	

第二步, 根据地址范围的容量以及该范围在计算机系统中的作用, 选择存储芯片。

由 6000H~67FFH 系统程序区的范围, 应选 1 片 2K×8 位的 ROM, 无需选 4K×8 位和 8K×8 位的 ROM, 否则就浪费了。

由 6800H~6BFFH 用户程序区的范围, 应选 2 片 1K×4 位的 RAM 芯片, 选其他芯片也必然浪费。

第三步, 分配 CPU 的地址线。

将 CPU 的低 11 位地址 $A_{10} \sim A_0$ 与 2K×8 位的 ROM 地址线相连; 将 CPU 的低 10 位地址 $A_9 \sim A_0$ 与 2 片 1K×4 位的 RAM 地址线相连。剩下的高位地址与访存控制信号 $\overline{MREQ}$ 共同产生存储芯片的片选信号。

第四步, 片选信号的形成。

由题给出的 74138 译码器输入逻辑关系可知, 必须保证控制端 G_1 为高, $\overline{G_{2A}}$ 与 $\overline{G_{2B}}$ 为低, 得图 4.34 所示。

图中 A_{15} 为低, 接到 $\overline{G_{2A}}$, A_{14} 为高, 接到 G_1 , $\overline{MREQ}$ 为低, 接到 $\overline{G_{2B}}$, 保证了三个控制端的要求; A_{13} 、 A_{12} 、 A_{11} 接到译码器 C、B、A 输入端, 其输出 $\overline{Y_4}$ 有效时, 选中 1 片 ROM, $\overline{Y_5}$ 与 A_{10} 同时有效均为低电平时, 选 2 片 RAM。ROM 芯片接地端为 $\overline{PD/progr}$, 读出时低电平有效。RAM 芯片的读/写控制端与 CPU 的读/写命令端 $\overline{WR}$ 相连。ROM 的 8 根数据线是单向的, 与 CPU 数据

总线单向相连, 2 片 RAM 的数据线分别与数据总线高 4 位和低 4 位双向相连。

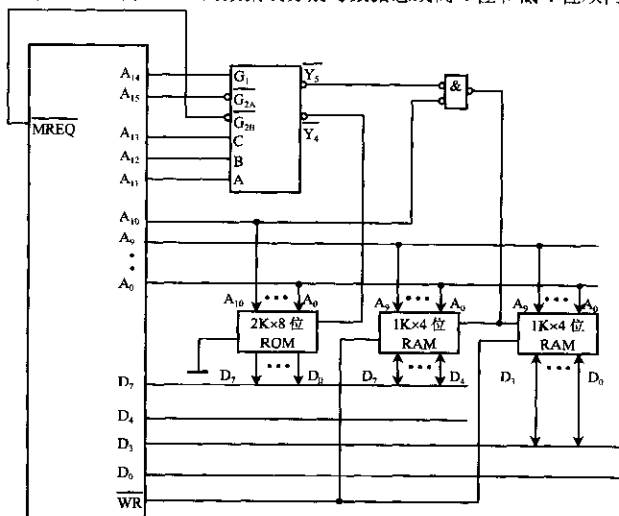


图 4.34 例 4.1 CPU 与存储芯片的连接图

例 4.2 CPU 及其他芯片假设同上题, 画出 CPU 与存储器的连接图。要求主存的地址空间满足下述条件: 最小 8K 地址为系统程序区, 与其相邻的 16K 地址为用户程序区, 最大 4K 地址空间为系统程序工作区。详细画出存储芯片的片选逻辑并指出存储芯片的种类及片数。

首先根据题目的地址范围写出相应的二进制地址码。

A_{15}	A_{14}	A_{13}	A_{12}	A_{11}	A_{10}	A_9	A_8	A_7	A_6	A_5	A_4	A_3	A_2	A_1	A_0	
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	最小 8K×8 位
0	0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	系统程序区
0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	0	1	1	1	1	1	1	1	1	1	1	1	1	1	1	相邻 16K×8 位
0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	
0	1	0	1	1	1	1	1	1	1	1	1	1	1	1	1	最大 4K×8 位
1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	系统程序工作区

第二步, 根据地址范围的容量及其在计算机系统中的作用, 确定最小 8K

系统程序区选一片 $8K \times 8$ 位 ROM；与其相邻的 16K 用户程序区选 2 片 $8K \times 8$ 位 RAM；最大 4K 系统程序工作区选 1 片 $4K \times 8$ 位 RAM。

第三步，分配 CPU 地址线。

将 CPU 的低 13 位地址线 $A_{12} \sim A_0$ 与 1 片 $8K \times 8$ 位 ROM 和两片 $8K \times 8$ 位 RAM 的地址线相连；将 CPU 的低 12 位地址线 $A_{11} \sim A_0$ 与 1 片 $4K \times 8$ 位 RAM 的地址线相连。

第四步，形成片选信号。

将 74LS138 译码器的控制端 G_1 接 +5V， $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$ 接 $\overline{MREQ}$ ，以保证译码器正常工作。CPU 的 A_{15} 、 A_{14} 、 A_{13} 分别接在译码器的 C、B、A 端，作为变量输入，则其输出 $\overline{Y_0}$ 、 $\overline{Y_1}$ 、 $\overline{Y_2}$ 分别作 ROM、 RAM_1 和 RAM_2 的片选信号。此外，根据题意，最大 4K 地址范围的 A_{12} 为高，故经反相后再与 $\overline{Y_7}$ 相“与”，其输出作为 $4K \times 8$ 位 RAM 的片选信号，如图 4.35 所示。

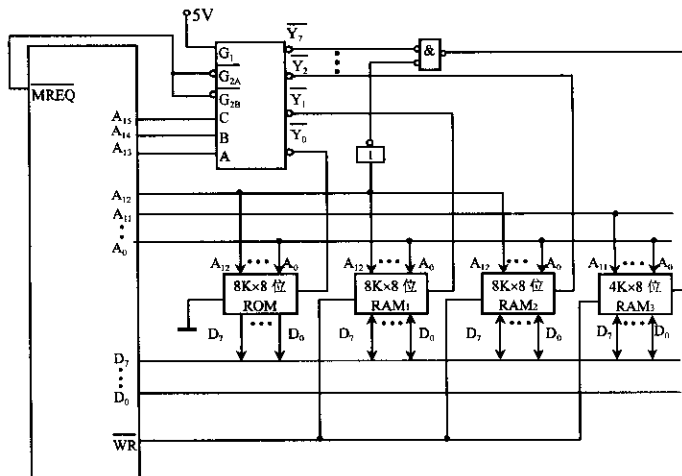


图 4.35 例 4.2CPU 与存储芯片的连接图

4.2.6 存储器的校验

在计算机运行过程中，由于种种原因致使数据在存储过程中可能出现差错。为了能及时发现错误并及时纠正错误，通常可将原数据配成海明编码。

1. 海明码的组成

海明码是由 Richard Hanming 于 1950 年提出的，它具有一位纠错能力。

由编码纠错理论得知，任何一种编码是否具有检测能力和纠错能力，都与编码的最小距离有关。所谓编码最小距离是指在一种编码系统中，任意两组合法代码之间的最少二进制位数的差异。根据纠错理论得：

$$L-1=D+C \quad \text{且 } D \geq C$$

即编码最小距离 L 越大，则其检测错误的位数 D 也越大，纠正错误的位数 C 也越大，且纠错能力恒小于或等于检错能力。如当编码最小距离 $L=3$ 时，这种编码可视为最多能检错二位，或能检错一位、纠错一位。可见，倘若能在信息编码中增加几位检测位，增大 L ，显然便能提高检错和纠错能力。海明码就是根据这一理论提出的具有一位纠错能力的编码。

设欲检测的二进制代码为 n 位，为使其具有纠错能力，需增添 k 位检测位，组成 $n+k$ 位的代码。为了能准确对错误定位以及指出代码没错，新增添的检测位数 k 应满足：

$$2^k \geq n+k+1$$

由此关系可求得不同代码长度 n 所需检测位的位数 k ，如表 4.1 所示。

表 4.1 代码长度与检测位位数的关系

n	k (最小)
1	2
2~4	3
5~11	4
12~26	5
27~57	6
58~120	7

k 的位数确定后，便可由它们所承担的检测任务，设定它们在被传送代码中的位置及它们的取值。

设 $n+k$ 位代码自左至右依次编为第 1, 2, 3, …… $n+k$ 位，而将 k 位检测位记作 C_i ($i=1, 2, 4, 8, \dots$)，分别安插在 $n+k$ 位代码编号的第 1, 2, 4, 8, …… 2^{k-1} 位上。这些检测位的位置设置，是为了保证它们能分别承担 $n+k$ 位

信息中, 不同数位所组成的“小组”的奇偶检查任务, 使检测位和它所负责检测的小组中 1 的个数为奇数或为偶数, 具体分配如下:

C_1 检测的 g_1 小组包含第 1, 3, 5, 7, 9, 11……位

C_2 检测的 g_2 小组包含第 2, 3, 6, 7, 10, 11, 14, 15……位

C_4 检测的 g_3 小组包含第 4, 5, 6, 7, 12, 13, 14, 15……位

C_8 检测的 g_4 小组包含第 8, 9, 10, 11, 12, 13, 14, 15, 24……位

⋮

其余检测位的小组所包含的位也可类推。这种小组的划分有如下特点:

① 每个小组 g_i 有一位且仅有一位为它所独占, 这一位是其他小组所没有的, 即 g_i 小组独占第 2^{i-1} 位 ($i=1, 2, 3, \dots$);

② 每两个小组 g_i 和 g_j 共同占有一位是其他小组没有的, 即每两小组 g_i 和 g_j 共同占有第 $2^{i-1}+2^{j-1}$ 位 ($i, j=1, 2, \dots$);

③ 每三个小组 g_i 、 g_j 和 g_k 共同占有第 $2^{i-1}+2^{j-1}+2^{k-1}$ 位, 是其他小组所没有的;

依次类推, 便可确定每组所包含的各位。

例如, 欲传递信息为 $b_4b_3b_2b_1$ ($n=4$), 根据 $2^k \geq n+k+1$, 可求出配置成海明码需增添检测位 $k=3$, 且它们位置的安排应如下:

二进制序号	1	2	3	4	5	6	7
名称	C_1	C_2	b_4	C_4	b_3	b_2	b_1

如果按配偶原则来配置海明码, 则

C_1 应使 1, 3, 5, 7 位中的“1”的个数为偶数;

C_2 应使 2, 3, 6, 7 位中的“1”的个数为偶数;

C_4 应使 4, 5, 6, 7 位中的“1”的个数为偶数;

故 C_1 应为 3 位 $\oplus$ 5 位 $\oplus$ 7 位, 即 $C_1=b_4 \oplus b_3 \oplus b_1$;

C_2 应为 3 位 $\oplus$ 6 位 $\oplus$ 7 位, 即 $C_2=b_4 \oplus b_2 \oplus b_1$;

C_4 应为 5 位 $\oplus$ 6 位 $\oplus$ 7 位, 即 $C_4=b_3 \oplus b_2 \oplus b_1$ 。

令 $b_4b_3b_2b_1=0101$, 则

$C_1=b_4 \oplus b_3 \oplus b_1=0 \oplus 1 \oplus 1=0$

$C_2=b_4 \oplus b_2 \oplus b_1=0 \oplus 0 \oplus 1=1$

$C_4=b_3 \oplus b_2 \oplus b_1=1 \oplus 0 \oplus 1=0$

故 0101 的海明码应为: $C_1C_2b_4C_4b_3b_2b_1$, 即 0100101。

2. 海明码的纠错过程

海明码的纠错过程, 实际上是对传送后的海明码形成新的检测位 P_i ($i=1, 2, 4, 8, \dots$), 根据 P_i 的状态, 便可直接指出错误的位置。 P_i 的状态是由原

检测位 C_i 及其所在小组内“1”的个数确定的。倘若按配偶原则配置的海明码，其传送后形成新的检测位 P_i 应为 0，否则说明传送有错，并且还可直接指出出错的位置。由于 P_i 与 C_i 有其对应关系，故 P_i 可由下式确定：

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7, \text{ 即 } P_1 = C_1 \oplus b_4 \oplus b_3 \oplus b_1$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7, \text{ 即 } P_2 = C_2 \oplus b_4 \oplus b_2 \oplus b_1$$

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7, \text{ 即 } P_4 = C_4 \oplus b_3 \oplus b_2 \oplus b_1$$

设已知传送的正确海明码（按配偶原则配置）为 0100101，若传送后接收到的海明码为 0100111，其出错位可按下述步骤进行：

令：

二进制序号	1	2	3	4	5	6	7
正确的海明码	0	1	0	0	1	0	1
接收到的海明码	0	1	0	0	1	1	1

则新的检测位为

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7, \text{ 即 } P_4 = 0 \oplus 1 \oplus 1 \oplus 1 = 1$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7, \text{ 即 } P_2 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7, \text{ 即 } P_1 = 0 \oplus 0 \oplus 1 \oplus 1 = 0$$

由此可见，传送结果 P_4 、 P_2 均不呈偶数，显然出了差错。那么，错位在 哪一位呢？极为有意思的是， P_4 、 P_2 、 P_1 所构成的二进制值恰恰是出错的位置，即 $P_4P_2P_1=110$ ，表示第六位出错。发现错误后，计算机便自动地将错误的第六位“1”纠正为“0”。

又如，若收到按偶配置的海明码为 1100101，则经检测得：

$$P_4 = 4 \oplus 5 \oplus 6 \oplus 7, \text{ 即 } P_4 = 0 \oplus 1 \oplus 0 \oplus 1 = 0$$

$$P_2 = 2 \oplus 3 \oplus 6 \oplus 7, \text{ 即 } P_2 = 1 \oplus 0 \oplus 0 \oplus 1 = 0$$

$$P_1 = 1 \oplus 3 \oplus 5 \oplus 7, \text{ 即 } P_1 = 1 \oplus 0 \oplus 1 \oplus 1 = 1$$

所以，出错位为 $P_4P_2P_1=001$ ，即第一位。可是第一位不是欲传送的信息位，而是检测位，在一般情况，可以不予纠正。

以上均以 $n=4$ 为例，其实对任意不同 n 位的信息，均可按上述步骤配置海明码，即先求出需增加的检测位位数 k ，再确定 C_i 的位置，然后，按奇或偶原则配置 C_i 各位的值即可。值得注意的是按奇配置与按偶配置所求得的 C_i 值正好相反，而新的检测位 P_i 的取值与奇偶配置原则是相对应的，读者可自行分析。

海明码常常被用在纠错一位的场合，若欲实现检错两位，实用时还得再添一位检测位。

4.2.7 提高访存速度的措施

随着计算机应用领域的不断扩大,处理的信息量也越来越多,对存储器的工作速度和容量要求越来越高。此外,因 CPU 的功能不断增强, I/O 设备的数量不断增多,致使主存的存取速度已成为计算机系统的瓶颈。可见,提高访存速度也成为迫不及待的任务。为了解决此问题,除了寻找高速元件和采用层次结构以外,调整主存的结构也可提高访存速度。

1. 单体多字系统

由于程序和数据在存储体内是连续存放的,因此 CPU 访存取出的信息也是连续的,如果可以在一个存取周期内,从同一地址取出 4 条指令,然后再逐条将指令送至 CPU 执行,也即每隔四分之一存取周期,主存向 CPU 送一条指令,这样显然增大了存储器的带宽,提高了单体存储器的工作速度,如图 4.36 所示。

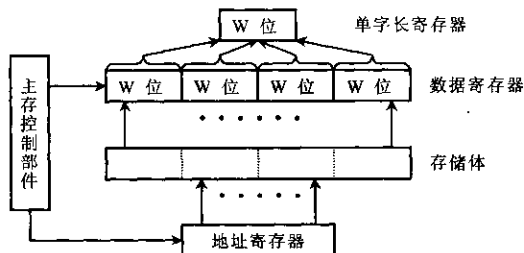


图 4.36 单体四字结构存储器

图中示意了一个单体四字结构的存储器,每字 W 位。按地址在一个存取周期内可读出 $4 \times W$ 位的指令或数据,使主存带宽提高到 4 倍。显然,采用这种办法的前提是:指令和数据在主存内必须是连续存放的,一旦遇到转移指令,或者操作数不能连续存放,这种方法的效果就不明显。

2. 多体并行系统

多体并行系统就是采用多体模块组成的存储器。每个模块有相同的容量和存取速度,各模块各自都有独立的地址寄存器、地址译码器、驱动电路和读写电路,它们能并行工作,又能交叉工作。

并行工作即同时访问 N 个模块,同时启动,同时读出,完全并行地工作(不过,同时读出的 N 个字在总线上需分时传送),图 4.37 是适合于并行工作

的高位交叉编址的多体存储器结构示意图，图中程序按体内地址存放，一个体存满后，再存入下一个体。显然，高位地址可表示体号。按这种编址方式，只要合理调动，便可提高存储器的宽带。例如，当第一个体用以执行程序时，另一个体可用来与外部设备进行直接存储器访问，实现两个体并行工作。

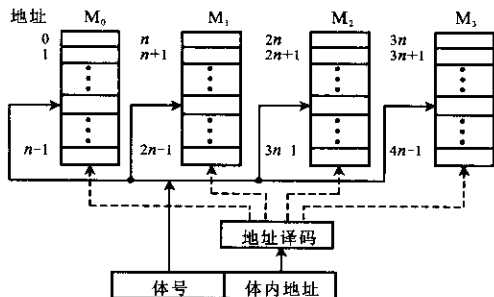


图 4.37 高位交叉编址的多体存储器

图 4.38 是按低位交叉编址的多体模块结构示意图。程序连续存放在相邻体中，显然低位地址用来表示体号，高位地址为体内地址。这种编址方法又叫作模 M 编址 (M 等于模块数)，表 4.2 列出了模 4 交叉编址的地址号。一般模块数 M 取 2 的方幂，使硬件电路比较简单。有的机器为了减少存储器冲突，采用质数个模块，如我国银河机的 M 为 31，其硬件实现比较复杂。

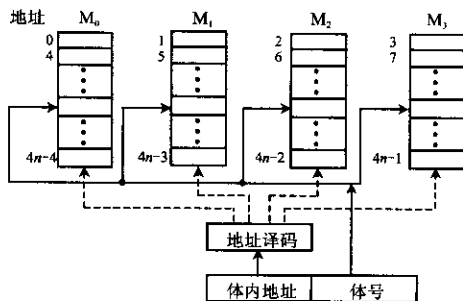


图 4.38 低位交叉编址的多体存储器

表 4.2 模 4 交叉编址地址表

体号	体内地址序号	最低两位地址
M_0	0, 4, 8, 12, $\dots, 4i+0$	00
M_1	1, 5, 9, 13, $\dots, 4i+1$	01
M_2	2, 6, 10, 14, $\dots, 4i+2$	10
M_3	3, 7, 11, 15, $\dots, 4i+3$	11

多体模块结构的存储器采用交叉编址后,可以在不改变每个模块存取周期的前提下,提高存储器的带宽。图 4.39 示意了四个存储体交叉访问的时间关系,负脉冲为启动每个体的工作信号。虽然对每个体而言,存取周期均未缩短,但由于 CPU 交叉访问各体,最终在一个存取周期的时间内,实际上向 CPU 提供了 4 个存储字。如果每模块存储字长为 32 位,则在一个存取周期内,存储器向 CPU 提供了 $32 \times 4 = 128$ 位二进制代码,大大加宽了存储器的带宽。

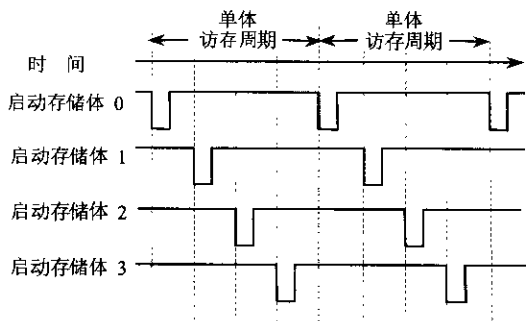


图 4.39 四个存储体交叉访问的时间关系

多体模块不仅要与 CPU 交换信息,还要与辅存、I/O 设备、乃至 I/O 处理机交换信息。因此,在某一时刻,决定主存究竟与哪个部件交换信息,必须由存储器控制部件(简称存控)来承担。存控具有合理安排各部件请求访问的顺序以及控制主存读写操作的功能。图 4.40 是一个存控基本结构框图,它由排队器、控制线路、节拍发生器及标记触发器等组成。

• 排队器

由于要求访存的请求源很多,而且访问都是随机的,这样有可能在同一

时刻出现多个请求源请求访问同一个存储体。为了防止发生两个以上的请求源同时占用同一存储体,并防止将代码错送到另一个请求源等各种错误的发生,在内存内需设置一个排队器,由它来确定请求源的优先级别。其确定原则为:

① 对易发生代码丢失的请求源,应列为最高优先级,如外设信息最易丢失,故它的级别最高;

② 对严重影响 CPU 工作的请求源,给予次高优先级,否则会导致 CPU 工作失常。

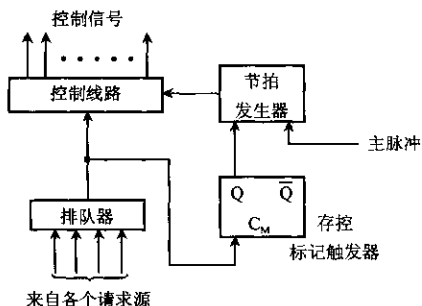


图 4.40 存控基本结构框图

例如写数请求高于读数,读数请求高于读指令。若运算部件不能尽快送走已算出的结果,会严重影响后续指令的执行,因此,发生这种情况时,写数的优先级比读数、读指令都高。而若没有操作数参与运算,取出更多的指令也无济于事,故读数的优先级又应比读指令高。

• 存控标记触发器 C_M

它用来接受排队器的输出信号,一旦响应某请求源请求, C_M 被置“1”,以便启动节拍发生器工作。

• 节拍发生器

它用来产生固定节拍,与机器主脉冲同步,使控制线路按一定时序发出信号。

• 控制线路

由它将排队器给出的信号与节拍发生器提供的节拍信号配合,向存储器各部件发出各种控制信号,用以实现对总线控制及完成存储器读写操作,并向请求源发出回答信号,表示存储器已响应了请求等等。

$B=2^b$ 反映了块的大小，称 B 为块长。

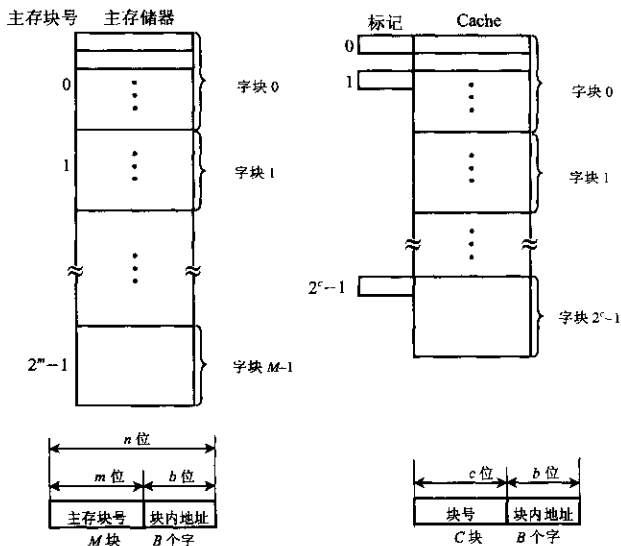


图 4.41 Cache/主存存储空间的基本结构

任何时刻都有一些主存块处在缓存块中。CPU 欲读取主存某字时，有两种可能：一种是所需要的数已在缓存中，即可直接访问 Cache（CPU 与 Cache 之间通常一次传送一个字）；另一种是所需的数不在 Cache 内，此时需将该数所在的主存整个字块一次调入 Cache 中。如果主存块已调入缓存块，则称该主存块与缓存块建立了对应关系。

上述第一种情况称为 CPU 访问 Cache 命中，第二种情况称 CPU 访问 Cache 不命中。由于缓存的块数 C 远小于主存的块数 M ，因此，一个缓存块不能唯一地、永久地只对应一个主存块，故每个缓存块需设一个标记（见图 4.41），用来表示当前存放的是哪一个主存块，该标记的内容相当于主存块的编号。CPU 读信息时，要将主存地址的高 m 位（或 m 位中的一部分）与缓存块的标记进行比较，以判断所读的信息是否已在缓存中（详见图 4.46）。

Cache 的容量与块长是影响 Cache 效率的重要因素，通常用“命中率”来衡量 Cache 的效率。命中率是指 CPU 要访问的信息已在 Cache 内的比率。一般而言，Cache 容量越大，其 CPU 的命中率就越高。当然也没必要太大，太大会增加成本，而且当 Cache 容量达到一定值时，命中率已不因容量的增大而

有明显的提高。因此, Cache 容量是总成本价与命中率的折衷值。如 80386 的主存最大容量为 4GB, 与其配套的 Cache 容量为 16KB 或 32KB, 其命中率可达 95% 以上。

块长与命中率之间的关系更为复杂, 它取决于各程序的局部特性。当块由小到大增长时, 起初会因局部性原理使命中率有所提高。由局部性原理指出, 在已被访问字的附近, 近期也可能被访问, 因此, 增大块长, 可将更多有用字存入在缓存, 提高其命中率。可是, 倘若继续增大块长, 很可能命中率反而下降, 这是因为所装入缓存的有用数据反而少于被替换掉的有用数据。由于块长的增大, 导致缓存中块数的减少, 而新装入的块要复盖旧块, 很可能出现少数块刚刚装入就又被复盖, 因此命中率反而下降。再之, 块增大后, 追加上的字, 距离所访问的字更远, 也更少会在近期用到。块长的最优值是很难确定的, 一般每块取 4 至 8 个可编址单位 (字或字节) 较好, 也可取一个主存周期所能调出主存的信息长度。例如 CRAY-1 的主存是 16 个体交叉, 每个体为单字宽, 其存放指令的 Cache 块长为 16 个字。又如 IBM 370/168 机主存是 4 体交叉, 每个体宽为 64 位 (8 个字节), 其 Cache 块长为 32 个字节。

3. Cache 的基本结构

Cache 的基本结构如图 4.42 所示。

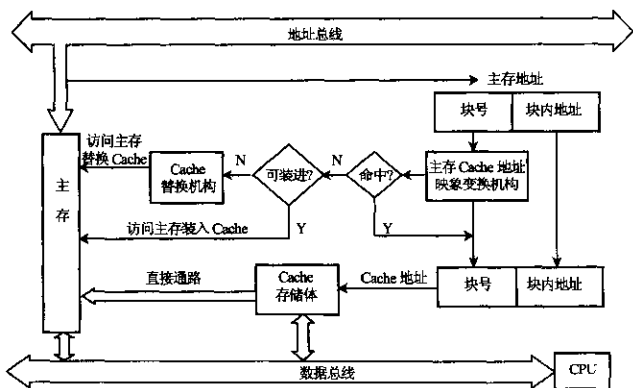


图 4.42 Cache 的基本结构

它由 Cache 存储体、地址映象变换机构、Cache 替换机构几大模块组成。

(1) Cache 存储体

Cache 存储体以块为单位与主存交换信息, 为加速 Cache 与主存之间的调

动,主存大多采用多体结构,且 Cache 访存的优先级最高。

(2) 地址映象变换机构

它是将 CPU 送来的主存地址转换为 Cache 地址。由于主存和 Cache 的块大小相同,块内地址都是相对于块的起始地址的偏移量(即低位地址相同),因此地址变换主要是主存的块号(高位地址)与 Cache 块号间的转换。而地址变换又与主存地址以什么样的函数关系映象到 Cache 中(称作地址映象)有关,这些内容可详见 4.3.2 节。

如果转换后的 Cache 块已与 CPU 欲访问的主存块建立了对应关系,即已命中,则 CPU 可直接访问 Cache 存储体。如果转换后的 Cache 块与 CPU 欲访问的主存块未建立对应关系,即不命中。此刻 CPU 在访问主存时,不仅将该字从主存取出,同时将它所在的主存块一并调入 Cache,供 CPU 使用。当然,此刻能将主存块调入 Cache 内,也是由于 Cache 原来处于未被装满的状态。反之,倘若 Cache 原来已被装满,即已无法将主存块调入 Cache 内时,就得采用替换策略。

(3) 替换机构

当 Cache 内容已满,无法接受来自主存块的信息时,就由 Cache 内的替换机构按一定的替换算法来确定应从 Cache 内移出哪个块返回主存,而把新的主存块调入 Cache。有关替换算法详见 4.3.3 节。

特别需指出的是,Cache 对用户是透明的,即用户编程时所用到的地址是主存地址,用户根本不知道这些主存块是否已调入 Cache 内。因为,将主存块调入 Cache 的任务全由机器硬件自动完成。

(4) Cache 的读/写操作

读操作的过程从前述 Cache 原理已经得知,可用流程图 4.43 来描述。

写操作比较复杂,因为对 Cache 块内写入的信息,必须与被映象的主存块内的信息完全一致。当程序运行过程中需对某个单元进行写操作时,会出现如何使 Cache 与主存内容保持一致的问题。目前主要采用以下几种方法。

- 写直达法,又叫通过式写(Write-through)或叫通过式存(Store-through),它能随时保证主存与 Cache 的数据始终一致。但有可能会增加访存次数,因每向 Cache 写入时,都需向主存写入。
- 写回法(Write-back),数据每次只是暂时写入 Cache,并用标志将该块加以注明,直至该块从 Cache 替换出时,才写入主存。这种方法又称标志交换式(Flang-Swap),其速度快,但因主存中的字块未经随时修改,可能失效。

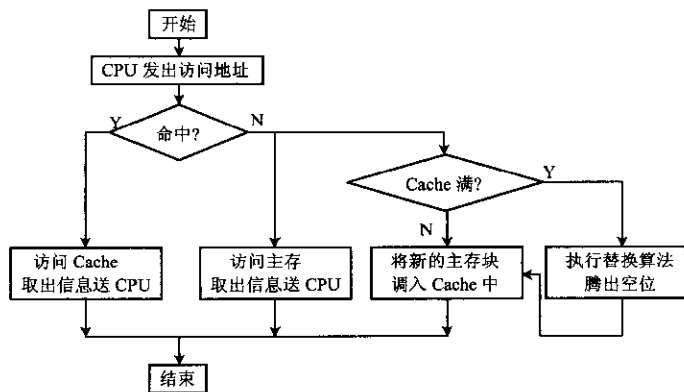


图 4.43 Cache 的读数操作流程

- 信息只写入主存, 同时将相应的 Cache 块有效位置“0”, 表明此 Cache 块已失效, 需要时从主存调入。还有一种可能, 被修改的单元根本不在 Cache 内, 因此写操作只对主存进行。

对于有多个处理器的系统, 各自都有独立的 Cache, 且都共享主存, 这样又出现了新问题。即当一个缓存中数据修改时, 不仅主存中相对应的字无效, 连同其他缓存中相对应的字也无效 (当然恰好其他缓存也有相应的字)。即使通过写直达法, 改变了主存的相应字, 而其他缓存中数据仍然无效。显然, 解决系统中 Cache 一致性的问题很重要。当今研究 Cache 一致性问题非常活跃, 想进一步了解可查阅有关资料。

4. Cache 的改进

Cache 刚出现时, 典型系统只有一个缓存, 近年来普遍采用多个 Cache。其含义有两方面: 一是增加 Cache 的级数; 二是将统一的 Cache 变成分开的 Cache。

(1) 单一缓存和两级缓存

所谓单一缓存, 顾名思义它是在 CPU 和主存之间只设一个缓存。随着集成电路逻辑密度的提高, 又把这个缓存直接与 CPU 制作在同一个芯片内, 故又叫片内缓存 (片载缓存)。片内缓存可以提高外部总线的利用率, 因为 Cache 做在芯片内, CPU 直接访问 Cache 不必占用芯片外的总线 (外部总线), 而且片内缓存与 CPU 之间的数据通路很短, 大大提高了存取速度, 外部总线又可更多地支持 I/O 设备与主存的信息传输, 增强了系统的整体效率。如 Intel 80486

CPU 芯片内就含 8K 字节的片内缓存。

可是, 由于片内缓存制在芯片内, 其容量不可能很大, 这就可能致使 CPU 欲访问的信息不在缓存内, 势必再通过外部总线访问主存, 访问次数多了, 整机速度就会下降。如果在主存与片内缓存之间, 再加一级缓存, 叫做片外缓存, 而且它是由比主存动态 RAM 和 ROM 存取速度更快的静态 RAM 组成, 那么, 从片外缓存调入片内缓存的速度就能提高, 而 CPU 占用外部总线的时间也就大大下降, 整机工作速度有明显改进。这种由片外缓存和片内缓存组成的 Cache, 叫做两级缓存, 并称片内缓存为第一级, 片外缓存为第二级。

(2) 统一缓存和分开缓存

统一缓存是指指令和数据都存放在同一缓存内的 Cache; 分开缓存是指指令和数据分别存放在两个缓存中, 一个叫指令 Cache, 一个叫数据 Cache。两种缓存的选用主要考虑如下两个因素。

其一, 它与主存结构有关, 如果计算机的主存是统一的 (指令、数据存在同一主存内), 则相应的 Cache 就采用统一缓存; 如果主存采用指令、数据分开存放的方案, 则相应的 Cache 就采用分开缓存。

其二, 它与机器对指令执行的控制方式有关。当采用超前控制或流水线控制方式时, 一般都采用分开缓存。

所谓超前控制是指在当前指令执行过程尚未结束时, 就提前将下一条准备执行的指令取出, 即超前取指或叫指令预取。所谓流水线控制实质上是多条指令同时执行 (详见第八章), 又可视作指令流水。当然, 要实现同时执行多条指令, 机器的指令译码电路和功能部件也需多个。超前控制和流水线控制特别强调指令的预取和指令的并行执行, 因此, 这类机器必须将指令 Cache 和数据 Cache 分开, 否则可能出现取指和执行过程对统一缓存的争用。如果此刻采用统一缓存, 则在执行部件向缓存发出取数请求时, 一旦指令预取机构也向缓存发出取指请求, 那么统一缓存只有先满足执行部件请求, 将数据送到执行部件, 让取指请求暂时等待, 显然达不到预取指令的目的, 从而影响指令流水的实现。可见, 这类机器将两种缓存分开尤为重要。

图 4.44 为简化的 Pentium 处理器框图。

图中有两个缓存。指令缓存是只读存储器, 从它中读出代码, 直接送至预取指令缓冲区。数据缓存有两个 32 位端口, 它们分别与两个可并行的整数 ALU 相连, 也可以合起来与 64 位浮点单元 (有独立的寄存器、乘法器、加法器、除法等) 相连, 因此, 数据 Cache 可直接支持整数 ALU 和浮点操作。

Pentium 处理器支持外部二级缓存, 其容量可达 256KB 或 512KB。

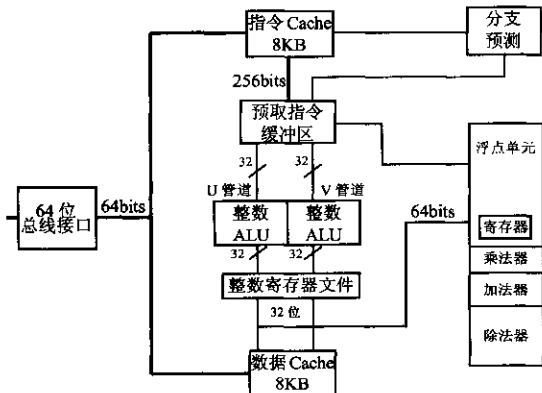


图 4.44 Pentium 处理器框图

图 4.45 是 PowerPC 620 处理器的示意。

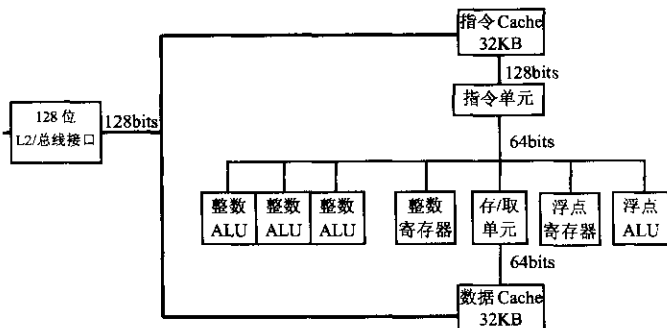


图 4.45 PowerPC 620 处理器框图

图中也有两个 Cache。数据 Cache 通过存/取单元支持整数和浮点操作；指令 Cache 为只读存储器，支持指令单元。执行部件是三个可并行操作的整数 ALU 和一个浮点运算部件（有独立的寄存器，乘、加、除部件）。

4.3.2 Cache—主存地址映象

由主存地址映象到 Cache 地址称为地址映象。地址映象方式很多，有直

接映象（固定的映象关系）、全相联映象（灵活性大的映象关系）、组相联映象（上述两种映象的折衷）和段相联映象（前两种的结合）。

1. 直接映象

图 4.46 示出了直接映象方式主存与缓存中字块的对应关系。

*假设 Cache 字块地址字段的内容为 00 …… 01

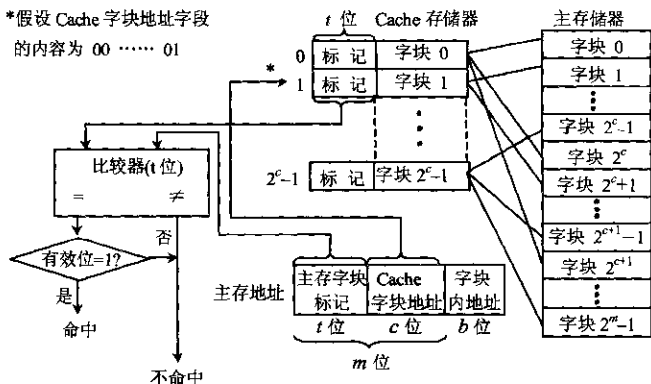


图 4.46 直接映象方式

图中每个主存块只与一个缓存块相对应，映射关系式：

$$i = j \bmod C \quad \text{或} \quad i = j \bmod 2^c$$

其中 i 为缓存块号， j 为主存块号， C 为缓存块数，映射结果表明每个缓存块对应若干个主存块，如表 4.3 所示。

表 4.3 直接映象方式主存块和缓存块的对应关系

缓存块	主 存 块
0	0, C , ..., $2^m - C$
1	1, $C + 1$, ..., $2^m - C + 1$
...	...
$C - 1$	$C - 1$, $2C - 1$, ..., $2^m - 1$

这种方式的优点是实现简单，只需利用主存地址的某些位直接判断，即可确定所需字块是否在缓存中。由图 4.46 可见，主存地址高 m 位被分成两部份：低 c 位指 Cache 的字块地址，高 t 位 ($t=m-c$) 是指主存字块标记，它被

记录在建立了对应关系的缓存块的“标记”位中。当缓存接到 CPU 送来的主存地址后, 只需根据中间 c 位字段找到 Cache 字块, 然后根据“标记”是否与主存地址的高 t 位相符来判断, 若符合且有效位为“1”, 表示该 Cache 块已与主存的某块建立了对应关系 (即已命中), 则可根据 b 位地址从 Cache 中取得信息; 若不符合, 或有效位为“0” (即不命中), 则就从主存读入新的字块来替代旧的字块, 同时将信息送往 CPU, 并修改 Cache “标记”。如果原来有效位为“0”, 还得将有效位置成“1”。

直接映象方式的缺点是不够灵活, 因每个主存块只能固定地对应某个缓存块, 即使缓存内还空着许多位置也不能占用, 使缓存的存储空间得不到充分的利用。此外, 如果程序恰好要重复访问对应同一缓存位置的不同主存块, 就要不停地替换, 从而降低了命中率。

2. 全相联映象

全相联映象允许主存中每一字块映象到 Cache 中的任何一块位置上, 如图 4.47 所示。这种映象方式可以从已被占满的 Cache 中替换出任一旧字块。显然, 这种方式灵活, 命中率也更高, 缩小了块冲突率。与直接映象相比, 它的主存字块标记从 t 位增加到 $t+c$ 位, 这就使 Cache “标记”的位数增多, 而且访问 Cache 时需要和 Cache 的全部“标记”位进行比较, 才能判断出所访问主存地址的内容是否已在 Cache 内。这种比较通常采用“按内容寻址”的相联存储器 (见附录 4A) 来完成。

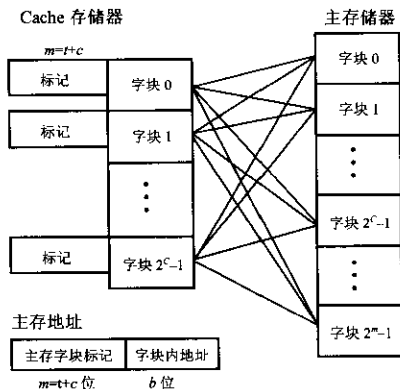


图 4.47 全相联映象

总之, 这种方式所需的逻辑电路甚多, 成本较高, 实际的 Cache 还要采

用各种措施来减少地址的比较次数。

3. 组相联映象

组相联映象是对直接映象和全相联映象的一种折衷。它把 Cache 分为 Q 组, 每组有 R 块, 并有以下关系:

$$i = j \bmod Q$$

其中, i 为缓存的组号, j 为主存的块号。即某一主存块按模 Q 将其映象到缓存的第 i 组内, 如图 4.48 所示。

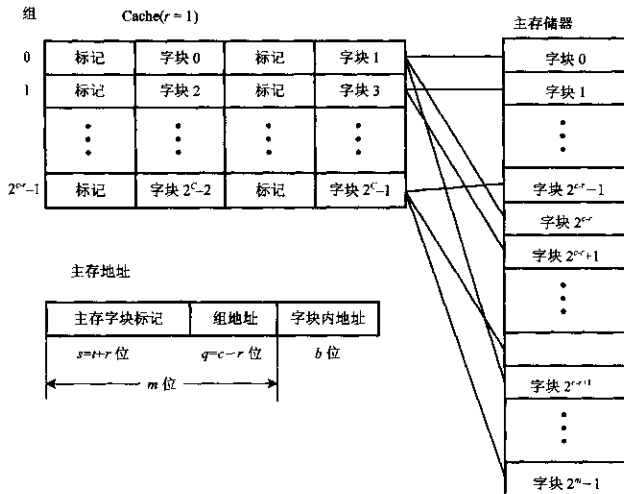


图 4.48 组相联映象

组相联映象的主存地址各段与直接映象 (见图 4.46) 相比, 还是有区别的。图 4.46 中 Cache 字块地址字段由 c 位变为 q 位, 且 $q = c - r$, 其中 2^c 表示 Cache 的总块数, 2^q 表示 Cache 的分组个数, 2^r 表示组内包含的块数。主存字块标记字段由 t 位变为 $s = t + r$ 位。为了便于理解, 假设 $c = 5$, $q = 4$, 则 $r = c - q = 1$ 。其实际含义为: Cache 共有 $2^c = 32$ 个字块, 共分为 $2^q = 16$ 个组, 每组内包含 $2^r = 2$ 个块。

根据上述假设条件, 组相联映象的含义是: 主存的某一字块可以按模 16 映象到 Cache 某组的任一字块中。即主存的第 0, 16, 32, ... 字块可以映象到 Cache 第 0 组 2 个字块中的任一字块。主存的第 15, 31, 47, ... 字块可以映象

到 Cache 第 i 组中的任一字块。显然, 主存的第 j 块会映象到 Cache 的第 i 组内。二者之间一一对应, 属直接映象关系; 另一方面, 主存的第 j 块可以映象到 Cache 的第 i 组内中的任一块, 这又体现出全相联映象关系。可见, 组相联映象的性能及其复杂性, 是介于直接映象和全相联映象两者之间, 当 $r=0$ 时, 它就是直接映象方式, 当 $r=c$ 时, 它就是全相联映象方式。

4. 段相联映象

段相联映象是直接映象和全相联映象两者结合的又一种方式。它是将主存和 Cache 都分成若干段, 且使它们每段包含的块数都相等, 段之间采用全相联映象, 段内块之间采用直接映象。当段数与 Cache 块数相等 (即每段只包含一块) 时, 便为全相联映象, 当段数为 1 时, 便为直接映象。

4.3.3 替换算法

当新的主存块需要调入 Cache 并且它的可用空间位置又被占满时, 就产生了一个替换算法 (策略) 问题。目前, 常用的两种算法是: 先进先出 (FIFO) 算法和近期最少使用 (LRU) 算法。

1. 先进先出 (FIFO) 算法

FIFO 算法的原则总是将最先调入 Cache 的字块替换出来, 它不需要随时记录各字块的使用情况, 所以容易实现、开锁小。但其缺点是可能把一些需要经常使用的程序 (如循环程序) 块也作为最早进入 Cache 的块而被替换出去。

2. 近期最少使用 (LRU) 算法

LRU 算法是将近期最少使用的块替换出来。它需要随时记录 Cache 中各个字块的使用情况, 以便确定哪个字块是近期最少使用的字块。LRU 算法的平均命中率比 FIFO 高, 尤其是当分组容量加大时 (组相联映象) 更能提高 LRU 算法的命中率。

4.4 辅助存储器

4.4.1 概述

1. 辅助存储器的特点

辅助存储器作为主存的后援设备, 又称作外部存储器, 简称外存, 它与主存一起组成了存储器系统的主存—辅存层次。与主存相比, 辅存具有容量大、速度慢、价格低、可脱机保存信息等特点, 属“非易失性”存储器。而主存具有速度快、成本高、容量小等特点, 而且大多由半导体芯片构成, 所存信息无法永久保存, 属“易失性”存储器。目前, 广泛用于计算机系统的辅助存储器

有硬磁盘、软磁盘、磁带、光盘等。前三种均属磁表面存储器。

磁表面存储器是在不同形状（如盘状、带状等）的载体上，涂有磁性材料层，工作时，靠载体高速运动，由磁头在磁层上进行读写操作，信息被记录在磁层上，这些信息的轨迹就是磁道。磁盘的磁道是一个个同心圆（见图 4.49（a）），磁带的磁道是沿磁带长度方向的直线（见图 4.49（b））。

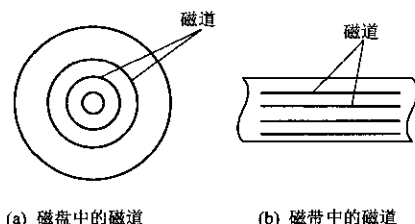


图 4.49 磁盘和磁带的磁道示意

下面结合磁表面存储器介绍它们的一些主要技术指标。

2. 磁表面存储器的主要技术指标

（1）记录密度

记录密度通常是指单位长度内所存储的二进制信息量。磁盘存储器用道密度和位密度表示；磁带存储器则用位密度表示。磁盘沿半径方向单位长度的磁道数为道密度，单位是道/英寸（Track Per Inch 缩写为 TPI）或道/毫米（TPM）。为了避免干扰，磁道与磁道之间需保持一定距离，相邻两条磁道中心线之间的距离叫道距，因此道密度 D_t 等于道距 P 的倒数。即

$$D_t = \frac{1}{P}$$

单位长度磁道能记录二进制信息的位数，称为位密度或线密度，单位是 bpi（bits per inch）或 bpm（位/毫米）。磁带存储器主要用位密度来衡量，常用的磁带带有 800bpi, 1600bpi, 6250bpi 等。对于磁盘，位密度 D_b 可按下式计算：

$$D_b = \frac{f_t}{\pi \cdot d_{\min}}$$

f_t 为每道总位数， $d_{\min}$ 为同心圆中最小直径，各磁道上所记录的信息量是相同的而位密度不同，一般的磁盘位密度是指最内圈磁道的位密度，即最大位密度。

（2）存储容量

存储容量是指外存所能存储的二进制信息总数量，一般以位或字节为单

位。以磁盘存储器为例，存储容量可按下式计算：

$$C = n \times k \times s$$

其中 C 为存储总容量， n 为存放信息的盘面数， k 为每个盘面的磁道数， s 为每条磁道上记录的二进制代码数。

磁盘有格式化容量和非格式化容量两个指标。非格式化容量是磁表面可以利用的磁化单元总数。格式化容量是指按某种特定的记录格式所能存储信息的总量，即用户可以使用的容量，它一般为非格式化容量的 60%~70%。

(3) 平均寻址时间

由存取方式分类可知，磁盘采取直接存取方式，寻址时间分为两个部分，其一是磁头寻找目标磁道的找道时间 t_s ，其二是找到磁道后，磁头等待欲读写的磁道区段旋转到磁头下方所需要的等待时间 t_w 。由于从最外圈磁道找到最里圈磁道和寻找相邻磁道所需时间是不等的，而且磁头等待不同区段所花的时间也不等，因此，取其平均值，称作平均寻址时间 T_a ，它是平均找道时间 t_{sa} 和平均等待时间 t_{wa} 之和：

$$T_a = t_{sa} + t_{wa} = \frac{t_{s \max} + t_{s \min}}{2} + \frac{t_{w \max} + t_{w \min}}{2}$$

平均寻址时间是磁盘存储器的一个重要指标。硬磁盘的平均寻址时间比较磁盘的平均寻址时间短，所以硬磁盘存储器比软磁盘存储器速度快。

磁带存储器采取顺序存取方式，磁头不动，磁带移动，不需要寻找磁道，但要考虑磁头寻找记录区段的等待时间，所以磁带寻址时间是指磁带空转到磁头应访问的记录区段所在位置的时间。

(4) 数据传输率

数据传输率 D_t 是指单位时间内磁表面存储器向主机传送数据的位数或字节数，它与记录密度 D 和记录介质的运动速度 V 有关：

$$D_t = D \times V$$

此外，辅存和主机的接口逻辑应有足够快的传送速度，用来完成接收/发送信息，以利主机与辅存之间的传送正确无误。

(5) 误码率

误码率是衡量磁表面存储器出错概率的参数，它等于从辅存读出时，出错信息位数和读出的总信息位数之比。为了减少出错率，磁表面存储器通常采用循环冗余码来发现并纠正错误。

4.4.2 磁记录原理和记录方式

1. 磁记录原理

磁表面存储器通过磁头和记录介质的相对运动完成读写操作。

写入过程如图 4.50 所示。

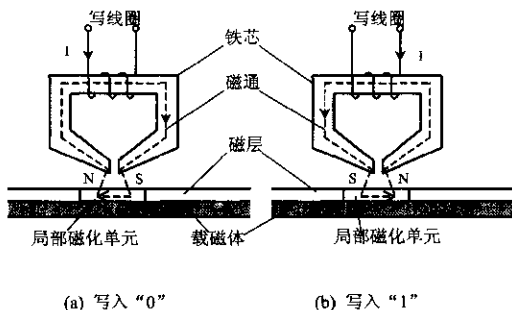


图 4.50 磁表面存储器写入原理

写入时，记录介质在磁头下方匀速通过，根据写入代码的要求，对写入线圈输入一定方向和大小的电流，使磁头导磁体磁化，产生一定方向和强度的磁场。由于磁头与磁层表面间距非常小，磁力线直接穿透到磁层表面，将对应磁头下方的微小区域磁化（叫作磁化单元）。可以根据写入驱动电流的不同方向，使磁层表面被磁化的极性方向不同，以区别记录“0”或“1”。

读出时，记录介质在磁头下方匀速通过，磁头相对于一个个被读出的磁化单元作切割磁力线的运动，从而在磁头读线圈中产生感应电势 e ，且
$$e = -n \frac{d\phi}{dt}$$
（ n 为读出线圈匝数），其方向正好和磁通的变化方向相反。由于原来磁化单元的剩磁通 ϕ 的方向不同，感应电势方向也不同，便可读出“1”或“0”两种不同信息。如图 4.51 所示。

2. 磁表面存储器的记录方式

磁记录方式又称为编码方式，它是按某种规律，将一串二进制数字信息转换成磁表面相应的磁化状态。磁记录方式对记录密度和可靠性都有很大影响，常用的记录方式有 6 种，如图 4.52 所示。

图中波形既代表了磁头线圈中的写入电流波形，也代表磁层上相应位置所记录的理想的磁通变化状态。

(1) 归零制 (RZ)

归零制记录“1”时，通以正向脉冲电流，记录“0”时，通以反向脉冲电流，使其在磁表面形成两个不同极性的磁饱和状态，分别表示“1”和“0”。

由于两位信息之间驱动电流归零，故叫归零制记录方式。这种方式在写入信息时很难覆盖原来的磁化区域，所以为了重新写入信息，在写入前，必须先抹去原存信息。这种记录方式原理简单，实施方便，但由于两个脉冲之间有一段间隔没有电流，相应的该段磁介质未被磁化，即该段空白，故记录密度不高，目前很少使用。

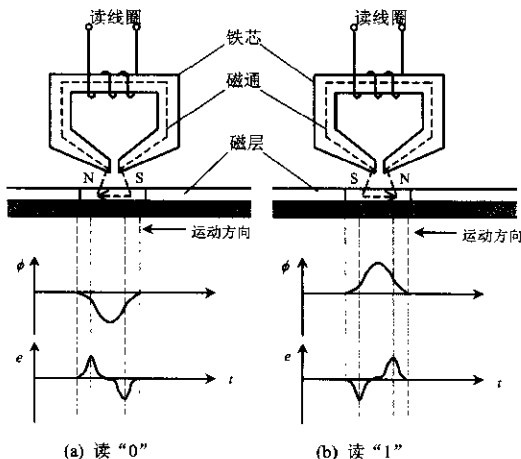


图 4.51 磁表面存储器读出原理

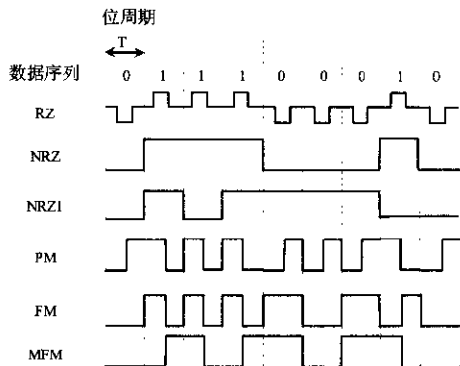


图 4.52 六种磁记录方式的写入电流波形

(2) 不归零制 (NRZ)

不归零制记录信息时,磁头线圈始终有驱动电流,不是正向,便是反向,不存在无电流状态。这样,磁表面层不是正向被磁化,就是反向被磁化。当连续记录“1”或“0”时,其写电流方向不变,只有当相邻两信息代码不同时,写电流才改变方向,故称为“见变就翻”的不归零制。

(3) 见“1”就翻的不归零制 (NRZI)

见“1”就翻的不归零制在记录信息时,磁头线圈也始终有电流。但只有在记录“0”时电流改变方向,使磁层磁化方向发生翻转;记录“1”时,电流方向保持不变,使磁层的磁化方向也维持原来状态,这就叫见“1”就翻的不归零制。

(4) 调相制 (PM)

调相制又称为相位编码 (PE),其记录规则是:记录“1”时,写电流由负变正;记录“0”时,写电流由正变负,而且电流变化出现在一位信息记录时间的中间时刻,它以相位差为 180° 的磁化翻转方向来表示“1”和“0”。因此,当连续记录相同信息时,在每两个相同信息的交界处,电流方向都要变化一次;若相邻信息不同,则两个信息位的交界处电流方向维持不变。调相制在磁带存储器中用得较多。

(5) 调频制 (FM)

调频制的记录规则是:以驱动电流变化的频率不同来区别记录“1”还是“0”。当记录“0”时,在一位信息的记录时间内电流保持不变;当记录“1”时,在一位信息记录时间的中间时刻,使电流改变一次方向。而且无论记录“0”还是“1”,在相邻信息的交界处,线圈电流均变化一次。因此,写“1”时,在位单元的起始和中间位置,都有磁通翻转;在写“0”时,仅在位单元起始位置有翻转。显然,记录“1”的磁翻转频率为记录“0”的两倍,故又称为倍频制。调频制记录方式被广泛应用在硬磁盘和软磁盘中。

(6) 改进调频制 (MFM)

这种记录方式基本上同调频制,即记录“0”时,在位记录时间内电流不变;记录“1”时,在位记录时间的中间时刻电流发生一次变化。两者不同之处在于,改进调频制只有当连续记录两个或两个以上的“0”时,才在每位的起始处电流改变一次,不必在每个位起始处都改变电流方向。由于这一特点,在写入同样数据序列时,MFM 比 FM 磁翻转次数少,在相同长度的磁层上可记录的信息量将会增加,从而提高了磁记录密度。FM 制记录一位二进制代码最多是两次磁翻转,MFM 制最多只要一次翻转,记录密度提高了一倍,故又称之为倍密度记录方式。倍密度软磁盘即采用 MFM 记录方式。

此外还有一种二次改进的调频制 (M^2FM),它是在 MFM 基础上改进的,

其记录规则是：当连续记录“0”时，仅在第一个位起始处电流方向改变，以后的位交界处电流方向不变。

3. 评价记录方式的主要指标

评价一种记录方式的优劣标准，主要反映在编码效率和自同步能力等方面。

(1) 编码效率

编码效率是指位密度与磁化翻转密度的比值，可用记录一位信息的最大磁化翻转次数来表示。如 FM、PM 记录方式中，记录一位信息最大磁化翻转次数为 2，因此编码效率为 50%；而 MFM、NRZ、NRZI 三种记录方式的编码效率为 100%，因为它们记录一位信息磁化翻转最多一次。

(2) 自同步能力

自同步能力是指从单个磁道读出的脉冲序列中所提取同步时钟脉冲的难易程度。从磁表面存储器的读出可知，为了将数据信息分离出来，必须有时间基准信号，称为同步信号。同步信号可以从专门设置用来记录同步信号的磁道中取得，这种方法叫做外同步，如 NRZI 制就是采用外同步的。图 4.53 画出了 NRZI 制驱动电流、记录磁通、感应电势、同步脉冲、读出代码等几种波形的理想对应关系（图中未反映磁通变化的滞后现象）。读出时将读线圈获得的感应信号放大（负波还要反相）、整形，这样，对于每个记录的“1”都会得到一个正脉冲，再将它们与同步脉冲相“与”，即可得读出代码波形。

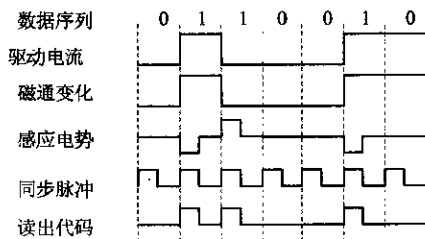


图 4.53 NRZI 的读出代码波形

对于高密度的记录系统，可直接从磁盘读出的信号中提取同步信号，这种方法称为自同步。

自同步能力可用最小磁化翻转间隔和最大磁化翻转间隔之比值 R 来衡量。 R 越大，自同步能力也越强。例如 NRZ 和 NRZI 方式在连续记录“0”时，磁

层都不发生磁化磁转,而 NRZ 方式在连续记录“1”时,磁层也不发生磁化翻转,因此,NRZ 和 NRZ1 都没有自同步能力。而 PM、FM、MFM 方式均有自同步能力。FM 记录方式的最大磁化翻转间隔是 T (T 为一位信息的记录时间),最小磁化翻转间隔是 $T/2$,所以 $R_{FM}=0.5$ 。

影响记录方式的优劣因素还有很多,如读分辨力、信息独立性(即某一位信息读出时出现误码而不影响后续其他信息位的正确性)、频带宽度、抗干扰能力以及实现电路的复杂性等等。

除上述所介绍的六种记录方式外,还有成组编码记录方式,如 GCR (5.4) 编码,它广泛用于磁带存储器。游程长度受限码(RLL 码)是近年来发展起来的用于高密度磁盘上的一种记录方式,在此均不详述。

4.4.3 硬磁盘存储器

硬磁盘存储器是计算机系统最主要的外存设备。第一个商品化的硬磁盘是由美国 IBM 公司于 1956 年研制而成的。四十多年来,无论在结构还是在性能方面,磁盘存储器有了很大的发展和改进。

1. 硬磁盘存储器类型

硬磁盘存储器的盘片由硬质铝合金材料制成,表面涂有一层可被磁化的硬磁特性材料。按磁头的工作方式分为固定磁头磁盘存储器和移动磁头磁盘存储器;按磁盘是否具有可换性又分为可换盘磁盘存储器和固定盘磁盘存储器。

固定磁头的磁盘存储器,其磁头位置固定不动,磁盘上的每一个磁道都对应一个磁头,如图 4.54 (a) 所示,盘片也不可更换。其特点是省去了磁头沿盘片径向运动所需寻找磁道的时间,存取速度快,只要磁头进入工作状态即可进行读写操作。

移动磁头的磁盘存储器在存取数据时,磁头在盘面上作径向运动,这类存储器可以由一个盘片组成,如图 4.54 (b) 所示。也可由多个盘片装在一个同心主轴上,每个记录面各有一个磁头,如图 4.54 (c) 所示。

图 4.54 (c) 中含有六个盘片,除上下两外侧为保护面外,共有 10 个盘面可作为记录面,并对应 10 个磁头。这类结构的硬磁盘存储器,目前应用最广泛。最典型的的就是温彻斯特磁盘。

可换盘磁盘存储器是指盘片可以脱机保存。这种磁盘可以在互为兼容的磁盘存储器之间交换数据,便于扩大存储容量。盘片可以只换单片,如在四片盒式磁盘存储器中,三片磁盘固定,只有一片可换。也可以将整个磁盘组(如六片、十一片、十二片等)换下。

固定盘磁盘存储器是指磁盘不能从驱动器中取下,更换时要把整个“头盘组合体”一起更换。

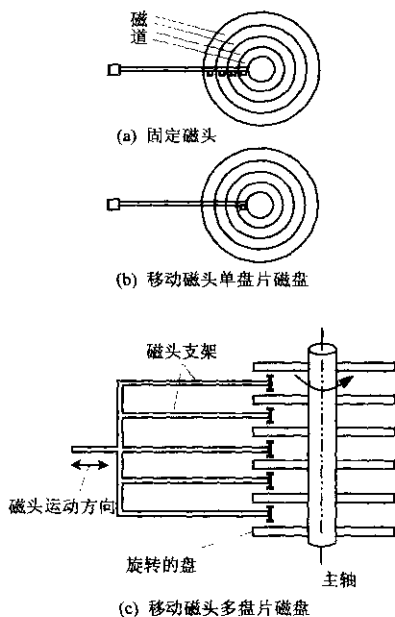


图 4.54 固定头和移动头磁盘

温彻斯特磁盘是一种可移动磁头固定盘片的磁盘存储器，简称温盘。它是目前用得最广，最有代表性的硬磁盘存储器。它于 1973 年首先应用在 IBM 3340 硬磁盘存储器中。其特点是采用密封组合方式，将磁头、盘片、驱动部件以及读写电路等制成一个不能随意拆卸的整体，叫作“头盘组合体”。因此，它的防尘性能好、可靠性高、对环境要求不高。过去有些普通的硬磁盘存储器要求在超净环境中应用，往往只能用在特殊条件的大、中型计算机系统中。

2. 硬磁盘存储器的结构

硬磁盘存储器是由磁盘驱动器、磁盘控制器和盘片组成，如图 4.55 所示。

(1) 磁盘驱动器

磁盘驱动器是主机外的一个独立装置，又称磁盘机。大型磁盘驱动器要占用一个或几个机柜，温盘只是一个比砖还小的小匣子。驱动器主要包括主轴、定位驱动及数据控制等。图 4.56 示意磁盘驱动器的主轴系统和定位驱动系统。

收选头选址信号，用以确定道地址和扇段地址。再根据写命令和写数据选定的磁记录方式，并将其转化为按一定变化规律的驱动电流注入磁头的写线圈中。按 4.4.2 所述的工作原理，便可将数据写入到指定磁道上。读操作时，首先也要接收选头选址信号，然后通过读放大器以及译码电路，将数据脉冲分离出来。

(2) 磁盘控制器

磁盘控制器通常制作成一块电路板，插在主机总线插槽中。其作用是接受由主机发来的命令，将它转换成磁盘驱动器的控制命令，实现主机和驱动器之间的数据格式转换和数据传送，并控制驱动器的读写。可见，磁盘控制器是主机与磁盘驱动器之间的接口。其内部又包含两个接口，一个是对主机的接口，称为系统级接口，它通过系统总线与主机交换信息；另一个是对硬盘（设备）的接口，称为设备级接口，又称为设备控制器，它接收主机的命令以控制设备的各种操作。一个磁盘控制器可以控制一台或几台驱动器。图 4.57 是磁盘控制器的接口示意。

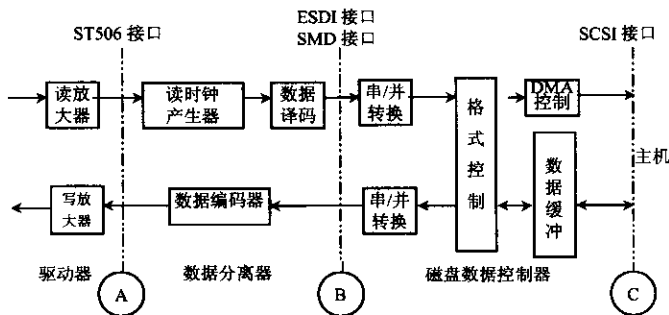


图 4.57 磁盘控制器接口示意

磁盘控制器与主机之间的界面比较清晰，只与主机的系统总线打交道，即数据的发送或接收，都是通过总线完成的。磁盘存储器属快速外部设备，它与主机交换信息通常采用直接存储器访问（DMA）的控制方式（详见 5.6），图中所示的 SCSI 标准接口即可与系统总线相连。

磁盘控制器与驱动器的界面可设在图 4.57 的 A 处，则驱动器只完成读/写和放大，如 ST506 接口就属这一类。如果将界面设在 B 处，则将数据分离电路和编码、解码电路划入驱动器内，磁盘控制器仅完成串/并（或并/串）转换、格式控制和 DMA 控制等逻辑功能，如 SMD 和 ESDI 等接口就属这种类型。如果界面设在 C 处，则磁盘控制器的功能全部转入到设备之中，主机与设备

之间便可采用标准通用接口,如 SCSI 接口。现在的发展趋势是后两类,增强了设备的功能,使设备相对独立,图 4.58 (a) 是采用了 SCSI 接口的系统结构示意图,其接口信号线如图 4.58 (b) 所示。

(3) 盘片

盘片是存储信息的载体,随着计算机系统的不断小型化,硬盘也在朝着小体积和大容量的方向发展。十年来商品化的硬盘盘面的记录密度已增长了 10 倍。表 4.4 列出了 1991 年以来正在研制或投产的各种硬盘某些主要指标所达到的水平(实际上这些指标都高于商品化硬盘指标)。

表 4.4 几种硬盘的某些指标

硬盘直径(英寸)	5.25	3.5	2.5	1.8
驱动器容量	3.7GB	1.4 GB	181.3MB	20MB
数据传输率 MB/s	20	14.5	6	2
平均存取时间 ms	11	8.5	14.5	20

3. 硬磁盘存储器的的发展动向

(1) 半导体盘

用半导体材料制成的“盘”,它既没有盘,也没有其他运动部件,它是以半导体芯片为核心,加上接口电路和其他控制电路,在功能上模拟硬盘,即按硬盘的工作方式存取数据。如 EEPROM,它可用电信号改写,断电时其原存信息也不被丢失,因此,它就可以作成半导体盘,其存取速度比硬盘要快得多,大约在 0.1ms 以下。

Flash Memory 是在 EPROM 和 EEPROM 基础上产生的一种新型的、具有性能价格比和可靠性更高的可擦写、非易失性的存储器。大容量的 Flash Memory 既能长期反复使用,又不丢失信息,因此它可以用来替代磁盘。Intel 生产的 27F0085A 的 Flash 存储器容量已达 8M 位。

(2) 提高磁盘记录密度

为提高磁盘记录密度,通常可采用以下技术:

- 采用高密度记录磁头;
- 采用先进的信息处理技术,克服由高密度带来的读出信号减弱和信号干扰比下降的缺点;
- 降低磁头浮动高度和采用高性能磁头浮动块;
- 改进磁头伺服跟踪技术;
- 采用高性能介质和基板的磁盘;
- 改进编码方式。

(3) 提高传输率和缩短存取时间

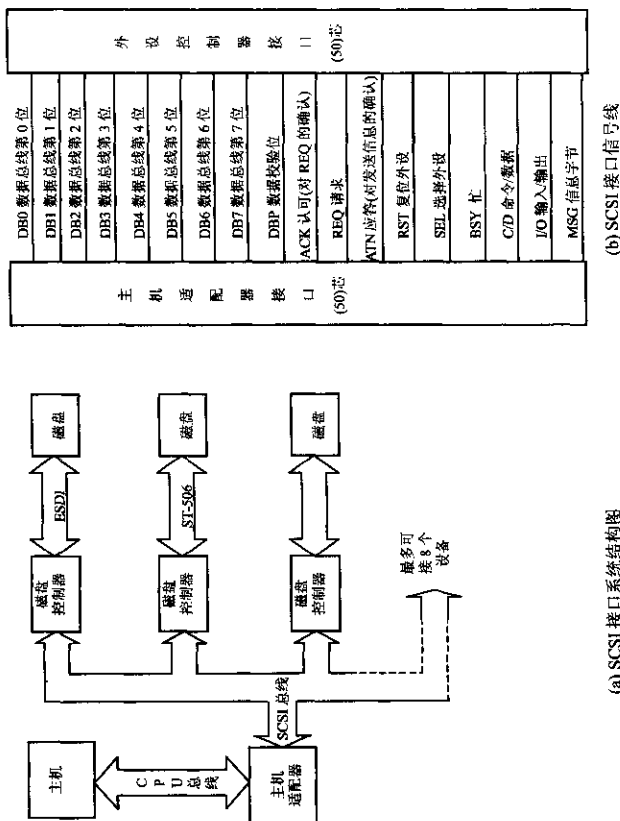


图 4.58 SCSI 接口系统结构和接口信号线

为实现磁盘高速化,可采用如下措施:

- 提高主轴转速,从过去的 2 400 转/分、3 600 转/分(记作 3 600rpm)提高到 4 400、4 500、5 400 和 6 300rpm。如美国 Maxtor Corp 开发的 MXT-1240S 型的 3.5 英寸硬盘,主轴转速为 6 300rpm,旋转等待时间为 4.76ms,平均存取时间为 8.5ms。
- 采用超高速缓冲存储器 Cache 芯片作读/写操作控制电路。如 IBM3990 型 14 英寸硬盘,以及 Quantum、Conner、日立制作所的 3.5 英寸硬盘的 Cache 容量已达 256KB。

(4) 采用磁盘阵列 RAID

尽管磁盘存储器的速度有了很大的提高,但与处理器相比,差距仍然很大。这种状态使磁盘存储器成了整个计算机系统功能提高的瓶颈。于是又出现了磁盘阵列 RAID (Redundant Array of Independent Disks)。它的基本原理是将并行处理技术引入到磁盘系统。使用多台小型温盘构成同步化的磁盘阵列,将数据展开分放在多台盘上,而这些盘又能像一台盘那样操作,使数据传输时间为单台盘的 $1/n$ (n 为并行驱动器个数)。有关 RAID 的内容,读者可在《计算机体系结构》中学习。

4. 硬磁盘的磁道记录格式

盘面的信息串行排列在磁道上,以字节为单位,若干相关的字节组成记录块,一系列的记录块又构成一个“记录”,一批相关的“记录”组成了文件。为了便于寻址,数据块在盘面上的分布遵循一定规律,称为磁道记录格式。常见的有定长记录格式和不定长记录格式两种。

(1) 定长记录格式

一个具有 n 个盘片的磁盘组,可将其 n 个面上同一半径的磁道看成一个圆柱面,这些磁道存储的信息叫做柱面信息。在移动磁头组合盘中,磁头定位机构一次定位的磁道集合正好是一个柱面。信息的交换通常在圆柱面上进行,柱面个数正好等于磁道数,故柱面号就是磁道号,而磁头号则是盘面号。

盘面又分若干扇区,每条磁道就被分割成若干个扇段,数据在盘片上的布局如图 4.59 所示。扇段是磁盘寻址的最小单位。在定长记录格式中,当台号决定后,磁盘寻址定位首先确定柱面,再选定磁头,最后找到扇段。因此寻址用的磁盘地址应由台号/柱面磁道号/盘面号/扇段号等字段组成。

CDC 6639 型、7637 型、ISOT-1370 型等磁盘都采用定长记录格式。ISOT-1370 盘磁道记录格式如图 4.60 所示。

ISOT 盘共有 12 个扇区,每个扇区内只记录一个数据块,每个扇段开始由扇区标志盘读出一个扇标脉冲,标志一个扇段的开始,0 扇区标志处再增加一个磁道标志,指明是起始扇区。

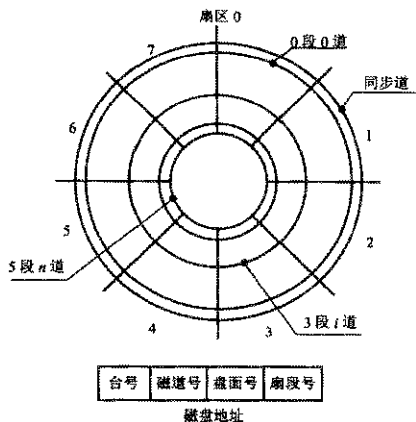


图 4.59 数据在盘片上的分布及磁盘地址定位

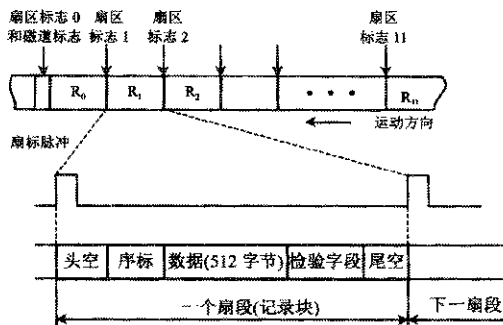


图 4.60 ISOT 盘磁道记录格式

每个扇段的头部是空白段，起到隧道清除作用。序标段以某种约定代码作为数据块的引导。数据段可写入 512 字节，若不满 512 字节，该扇段余下部分为空白；若超过 512 字节，则可占用几个扇段。检验字段写一个校验字，常用循环冗余码（CRC）检验，尾空白段为全 0 或空白区以，示数据结束。

这种记录格式结构简单，可按柱面号（磁道号）、盘面号、扇段号进行直接寻址，但记录区的利用率不高。

(2) 不定长记录格式

在实践应用中,信息常以文件形式存入磁盘。若文件长度不是定长记录块的整数倍时,往往造成记录块的浪费。不定长记录格式可根据需要来决定记录块的长度。如 IBM 2311、2314 等磁盘驱动器采用不定长记录格式,图 4.61 是 IBM 2311 盘不定长度磁道记录格式示意。

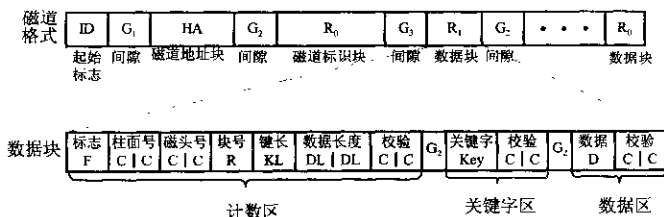


图 4.61 IBM 2311 盘的不定长度磁道记录格式

图中 ID 是起始标志,又叫索引标志,表示磁道的起点。间隙 G₁ 是一段空白区,占 36~72 个字节长度,其作用是使连续的磁道分成不同的区,以利于磁盘控制器与磁盘机之间的同步和定位。磁道地址块 HA 又叫标识地址或专用地址,它占有 7 个字节,用来表明四部分的状况,如磁道是否完好;柱面逻辑地址号;磁头逻辑地址号和校验码。间隙 G₂ 占 18~38 个字节长度。R₀ 是磁道标识块,用来说明本磁道的状况,不作为用户数据区。间隙 G₃ 包含一个以专用字符表示的地址标志,指明后面都是数据记录块。数据记录块 R₁ 由计数区、关键字区和数据区三段组成,这三段分别都有循环校验码。一般要求一个记录限于同一磁道内,若设有专门的磁道溢出手段,则允许继续记录到同一柱面的另一磁道内。数据区长度不定,实际长度由计数区的 DL 给定,通常为 1KB 至 64KB。从主存调出数据时,常常带有奇偶校验位,在写入磁盘时,则由磁盘控制器删去奇偶校验位,并在数据区结束时加上循环校验位。当从磁盘读出数据时,需进行一次校验操作,并恢复原来的奇偶校验位。可见,在磁盘数据区中,数据是串行的,字节之间没有间隙,字节后面没有校验位。

4.4.4 软磁盘存储器

1. 概述

软磁盘存储器的盘片是用类似塑料薄膜唱片的柔性材料制成的,简称软盘。

软磁盘存储器与硬磁盘存储器的存储原理和记录方式是相同的,但在结构上有较大差别:硬盘转速高,存取速度快;软盘转速低,存取速度慢。硬盘有

固定磁头, 固定盘、盘组等结构; 软盘都是活动头, 可换盘片结构。硬盘是靠浮动磁头读写, 磁头不接触盘片; 软盘磁头直接接触盘片进行读写。硬盘系统及硬盘片价格比较贵, 大部分盘片不能互换; 软盘价格便宜, 盘片保存方便、使用灵活、具有互换性。硬盘对环境要求苛刻, 要求采用超净措施; 软盘对环境的要求不苛刻。因此, 软盘在微型计算机系统中, 获得了广泛的应用, 甚至有的大中型计算机系统中也配有软盘。

软磁盘存储器的种类主要是按其盘片尺寸不同而区分的, 现有 8 英寸、5.25 英寸、3.5 英寸和 2.5 英寸几种。软盘尺寸越小, 记录密度就越高, 驱动器也越小。从内部结构来看, 若按使用的磁记录面 (磁头个数) 不同和记录密度不同, 又可分为单面单密度、单面双密度、双面双密度等多种软盘存储器。

世界上第一台软盘机是美国 IBM 公司于 1972 年制成的 IBM 3740 数据录入系统。它是 8 英寸单面单密度软盘, 容量只有 256KB。1976 年出现了 5.25 英寸软盘, 80 年代又出现了 3.5 英寸和 2.5 英寸的微型软盘, 其容量可达 1MB 以上。由于它价格便宜, 使用灵活, 盘片保管方便, 因此, 软盘已逐渐成为外存设备的主要部件。其销量已远远超过硬盘和磁带, 目前软盘还在向超小型化、薄形化和高密度化方向发展。

软盘存储器除主要用作外存设备外, 还可以和键盘一起构成脱机输入装置, 其作用是给程序员提供输入程序和数据, 然后再输入到主机上运行, 这样使输入操作不占用主机工作时间。

2. 软磁盘片

软磁盘片的盘基是由厚约为 $76\text{ }\mu\text{m}$ 的聚脂薄膜制成, 其两面涂有厚约为 $2.3\sim 3\text{ }\mu\text{m}$ 的磁层。盘片装在塑料封套内, 套内有一层无纺布, 用来防尘, 保护盘面不受碰撞, 还起到消除静电的作用。盘片连封套一起插入软盘机中, 盘片在塑料套内旋转, 无纺布消除因盘片转动而产生的静电, 保证信息可以正常读写。

塑料封套均为正方形, 其上有许多孔, 如用来装卡盘片的“中心孔”、用于定位的“索引孔”、用于磁头读写盘片的“读写孔”, 以及“写保护”缺口 (8 英寸盘) 或“允许写”缺口 (5.25 英寸盘) 等。图 4.62 示意了软磁盘盘片及其外形示意。

8 英寸软盘有 77 个磁道, 从外往里依次为 00 道到 76 道。5.25 英寸软盘有 40 个和 80 个磁道两种。

与硬盘相同, 软磁盘盘面也分为若干个扇区 (见图 4.59), 每条磁道上的扇区数相同, 记录同样多的信息。由于靠里的磁道圆周长小于外磁道的圆周长, 因此, 里圈磁道的位密度比外圈磁道高。至于一个盘面分成几个扇区, 则取决于它的记录方式。区段的划分采用软分段方式, 由软件写上的标志实现。

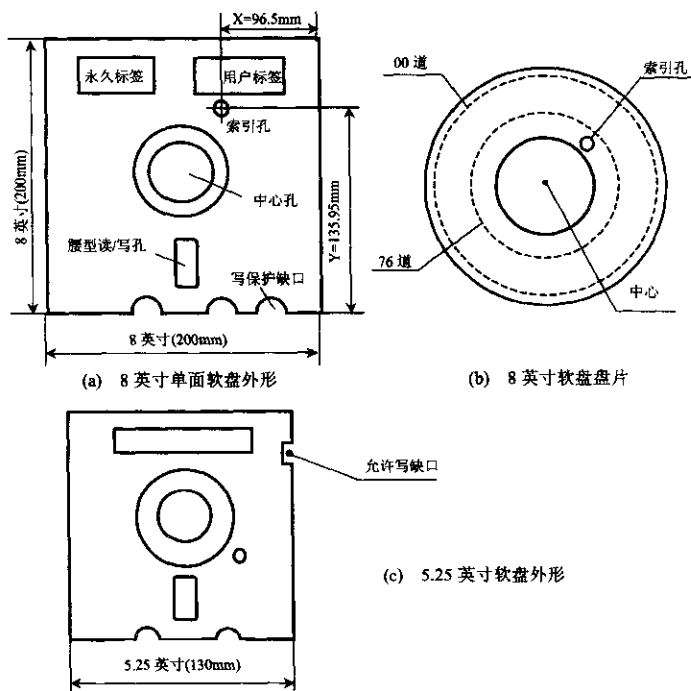


图 4.62 软磁盘盘片及外形

索引孔可作为旋转一圈开始或结束的标志，通常在盘片和保护套上各打有小孔。当盘片上的小孔转到与保护套上的小孔位置重合时，通过光电检测元件测出信号，即标志磁道已到起点或已为结束点。

近年来 8 英寸盘和 5.25 英寸盘已逐渐被 3.5 英寸盘取代，其盘片装在硬塑料封套内，它们的基本结构类似。

按软盘驱动器的性能区分，有单面盘和双面盘。前者驱动器只有一个磁头，盘片只有一个面可以记录信息。双面盘的驱动器有两个磁头，盘片有两个记录面。

按记录密度区分，有单密度和双密度两种。前者采用 FM 记录方式，后者采用 MFM 记录方式。

综上所述，软盘分为单面单密度 (SS, SD)、双面单密度 (DS, SD)、单

面双密度 (SS, DD)、双面双密度 (DS, DD) 四种。对于 5.25 英寸和 3.5 英寸的磁盘机而言, 均采用双面双密度及高密度 (四倍密度) 的记录方式。

3. 软磁盘的记录格式

软磁盘存储器采用软分段格式, 软分段格式有 IBM 格式和非 IBM 格式两种。IBM 格式被国际标准化组织 (ISO) 确定为国际标准。下面以 IBM 3740 的 8 英寸软盘为例, 介绍其软分段格式, 如图 4.63 所示。

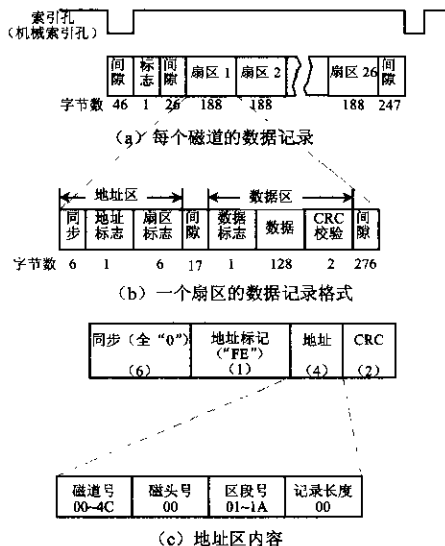


图 4.63 IBM 3740 软分段格式

软分段的磁道由首部、扇区部和尾部三部分组成。当磁盘驱动器检测到索引孔时, 便标志磁道的起始位已找到。首部是一段空隙, 它是为避免由于不同软盘驱动器的索引检测器和磁头机械尺寸误差引起读写错误而设置的。尾部是依次设置在首部和各扇区后所剩下的间隙, 起到转速变化的缓冲作用。首部和尾部之间的弧被划分成若干扇区, 又称为扇段。

图 4.63 (a) 中索引孔信号的前沿标志磁道开始, 经 46 个字节间隙后, 有一个字节的软索引标志, 后面再隔 26 个字节的间隙后, 便是 26 个扇区 (每个扇区 188 个字节), 最后还有 247 个字节的间隙, 表示一个磁道结束。

图 4.63 (b) 中标出了一个扇区的 188 个字节的分配。前 13 个字节是

地址区，详细内容可见图 4.63 (c)。其中地址信息占 4 个字节，分别指明磁道号、磁头号、区段号和记录长度。地址区字段的最后 2 个字节是 CRC 循环冗余校验码。此外，一个扇区内还有 131 个字节的数据区，它由数据标志、数据、CRC 校验码三部分组成。在地址区和数据区后各自都有一段间隙。

对图 4.63 所示的单面单密度软盘而言，其格式化容量为：

$$\text{磁道数/盘片} \times \text{扇区数/磁道} \times \text{数据字节数/扇区} = 77 \times 26 \times 128 \approx 256\text{KB}$$

不同规格的软盘，每磁道究竟分成多少区段，IBM 格式都有明确规定。例如，5.25 英寸软盘，每磁道区段数为 15、9 或 8 三种，每个区段字节数均为 512 个。

出厂后未使用过的盘片叫做白盘，需格式化后才能使用。采用统一的标准记录格式是为了达到盘片互换及简化系统设计的目的。但是软件生产厂家为了保护软件的产权，常用改变盘片上的数据格式来达到软件不被盗版的目的。因为通过对磁盘控制器编程，可以方便地指定每条磁道上的扇区数和所采用的记录格式，甚至可以调整间隙长度，改变磁盘地址的安排顺序等。经过这些处理，使用通用软件就不能正确拷贝磁盘文件了。

4. 软磁盘驱动器和控制器

软磁盘存储器也由软磁盘驱动器、软磁盘控制器和软磁盘片三部分组成。软磁盘驱动器是一个相对独立的装置，又称软磁盘机，主要由驱动机构、磁头及定位机构和读写电路组成。软磁盘控制器的功能是解释来自主机的命令，并向软盘驱动器发出各种控制信号，同时还要检测驱动器的状态，按规定的数据格式向驱动器发出读写数据命令等。具体操作如下：

(1) 寻道操作：将磁头定位在目标磁道上；

(2) 地址检测操作：主机将目标地址送往磁盘控制器，控制器从驱动器上按记录格式读取地址信息，并与目标地址进行比较，找到欲读（写）信息的磁盘地址；

(3) 读数据操作：首先检测数据标志是否正确，然后将数据字段的内容送入主存、最后用 CRC 校验；

(4) 写数据操作：写数据时，不仅要写原始信息经编码后写入磁盘，同时要写上数据区标志和 CRC 校验码以及间隙；

(5) 初始化：在盘片上写格式化信息，对每个磁道划分区段。

以上操作是由软盘控制器完成的，为此设计了软盘控制器芯片，将许多功能集成在一块芯片上，如 FD1771，FD1991， μ PD765 等。它们都是可编程的，将磁盘最基本的操作用这些芯片的指令编程实现便可实现对驱动器的控制。

软磁盘控制器发给驱动器的信号有：驱动器选择信号（表示某台驱动器与控制器接通）；马达允许信号（表示驱动器的主轴电机旋转或停止）；步进信号

(使所选驱动器的磁头按指定方向移动,一次移一道);步进方向(指磁头移动的方向);写数据与写允许信号;选头信号(选择“0”面还是“1”面的磁头)。

驱动器提供给控制器的信号有:读出数据信号;写保护信号(表示盘片套上是否贴有写保护标志,如果贴有标记,则发写保护信号);索引信号(表示盘片旋转到索引孔位置,表明一个磁道的开始);0 磁道信号(表示磁头正停在 0 号磁道上)。

图 4.64 是 IBM-PC 机上的软盘控制器框图。

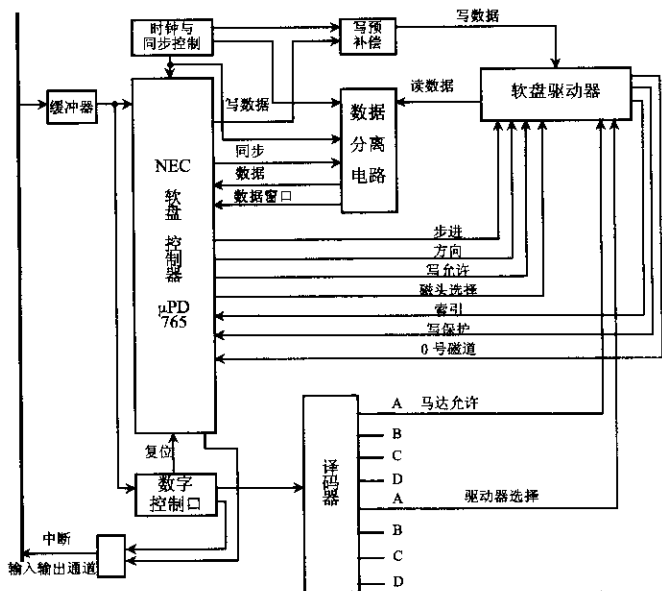


图 4.64 软盘控制器逻辑框图

4.4.5 磁带存储器

1. 概述

磁带存储器也属磁表面存储器,其记录原理和记录方式与磁盘存储器是相同的。但从存取方式来看,磁盘存储器属于直接存取设备,即只要知道信息所

在盘面、磁道和扇区的位置，磁头便可直接找到其位置并读写。磁带存储器必须按顺序进行存取，即磁带上的文件是按磁带头尾顺序存放的。如果某文件存在磁带尾部，而磁头当前位置在磁带首部，那么必须等待磁带走到尾部时才能读取该文件，因此磁带存取时间比磁盘长。但由于磁带容量比较大，位价格也比磁盘低，而且格式统一便于互换，因此，磁带存储器仍然是一种用于脱机存储的后备存储器。

磁带存储器是由磁带和磁带机两部分组成。磁带按长度分有 2 400 英尺、1 200 英尺、600 英尺几种；按宽度分有 1/4 英寸、1/2 英寸、1 英寸、3 英寸几种；按记录密度分有 800bpi、1 600bpi、6 250bpi 等几种；按磁带表面并行记录信息的道数分有 7 道、9 道、16 道等；按磁带外形分有开盘式磁带和盒式磁带两种。现在计算机系统较广泛使用的两种标准磁带为：1/2 英寸开盘式和 1/4 英寸盒式。

磁带机又有很多种类，按磁带机规模分有标准半英寸磁带机，海量宽带磁带机（Mass storage）和盒式磁带机三种。按磁带机走带速度分，有高速磁带机（4~5m/s）、中速磁带机（2~3m/s）和低速磁带机（2m/s 以下）。磁带机的数据传输率取决于记录密度和走带速度。在记录密度相同的情况下，带速越快，传输率就越高。按装卸磁带机构分，有手动装卸式和自动装卸式；按磁带传动缓冲机构分，有摆杆式和真空式；按磁带的记录格式分，有启停式和数据流式。数据流磁带机已成为现代计算机系统中主要的后备存储器，其位密度可达 8 000bpi，它用于资料保存，文件复制，作为脱机后备存储装置。特别是当温盘出现故障时，用以恢复系统。

磁带机正朝着提高传输率、提高记录密度、改善机械结构、提高可靠性等方向发展。

2. 数据流磁带机

数据流磁带机是将数据连续地写到磁带上，每个数据块后有一个记录间隙，使磁带机在数据块间不启停，简化了磁带机的结构，用电子控制替代了机械启停式控制，降低了成本，提高了可靠性。

数据流磁带机有 1/2 英寸开盘式和 1/4 英寸盒式两种。盒式磁带的结构类似录音带和录相带。盒带内装有供带盘和收带盘，磁带长度有 450 英尺和 600 英尺两种，容量分别为 45MB 和 60MB。近年来，容量高达 1GB 和 1.35GB 的 1/4 英寸盒式数据流磁带机也已问世。当采用数据压缩技术时，1/4 英寸盒式数据流磁带机容量可达 2GB 或 2.7GB。

数据流磁带机与传统的启停式磁带机的多位并行读写不同，它采用类似磁盘的串行读写方式，它的记录格式与软盘类似。

以 4 道数据流磁带机为例，四个磁道的排列次序如图 4.65 所示。在记录

信息时，先在第 0 道上从磁带首端 BOT 记到磁带末端 EOT，然后在第 1 道上反向记录，即从 EOT 到 BOT，第 2 道又从 BOT 到 EOT，第 3 道又从 EOT 到 BOT。读出信息时，也是这个顺序。这种方式叫做蛇形（Serpentine）记录。9 道 1/4 英寸数据流磁带记录格式也与此相同，偶数磁道从 BOT 到 EOT，奇数磁道从 EOT 到 BOT，依次首尾相接。

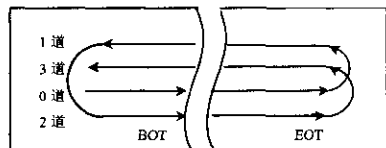


图 4.65 4 道 1/4 英寸磁带蛇形串行记录方式

盒式数据流磁带机与主机的接口是标准的通用接口，可用小型计算机系统接口 SCSI 与主机相连，也可以通过磁带控制器与主机相连。磁带控制器的作用类似于磁盘控制器，控制主机与磁带机之间进行信息交换。

3. 磁带的记录格式

磁带上的信息可以以文件形式存储，也可以按数据块存储。磁带可以在数据块之间启停，进行数据传输。按数据块存储的磁带互换性更好。

磁带机与主机之间进行信息传送的最小单位是数据块或叫做记录块（Block），记录块的长度可以是固定的，也可以是变化的，由操作系统决定。记录块之间有空白间隙，作为磁头停靠的地方，并保证磁带机停止或启动时有足够的惯性缓冲。记录块尾部有几行特殊的标记，表示数据块结束，接着便是校验区。图 4.66 示意了磁带上数据格式。

磁带信息的校验属于多重校验，由奇偶校验、循环冗余校验和纵向冗余校验共同完成。以 9 道磁带为例，横向可以并排记录 9 位二进制信息（称为一行），其中 8 位是数据磁道，存储一个字节，另一位是这一字节的奇偶校验位，叫横向奇偶校验码。在每一个数据块内，沿纵向（即走带方向）每一磁道还配有 CRC 校验码。此外对每一磁道上的信息（包括 CRC 在内），又有一个纵向奇偶校验码。纠错的原理是用循环冗余码的规律和专门线路，指出出错的磁道（CRC 可发现一个磁道上的多个错误码），然后用横向校验码检测每一行是否有错，纵横交错后就可指明哪行哪道处有错，如有错就立即纠正。

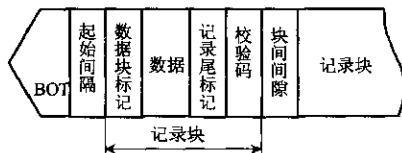


图 4.66 磁带的格式

4.4.6 循环冗余校验码 (CRC 码)

磁表面存储器由于磁介质表面的缺陷、尘埃等原因,致使出现多个错误码。CRC (Cyclic Redundancy Check) 码可以发现并纠正信息在存储或传送过程中连续出现的多位错误代码。因此 CRC 校验码在磁介质存储器和计算机之间通信方面得到广泛应用。

CRC 码是基于模 2 运算而建立编码规律的校验码。模 2 运算的特点是不考虑进位和借位的运算,其规律如下:

(1) 模 2 加和模 2 减其结果是相等的。

即 $0 \pm 1 = 1$, $0 \pm 0 = 0$, $1 \pm 0 = 1$, $1 \pm 1 = 0$ 。可见,两个相同数的模 2 和恒为 0。

(2) 模 2 乘是按模 2 和求部分积之和。

(3) 模 2 除是按模 2 减求部分余数。每求一位商应使部分余数减少一位。上商的原则是:当部分余数的首位为 1 时,上商 1;当部分余数的首位为 0 时,上商 0。当部分余数的位数小于除数的位数时,该余数即为最后余数。

(2) 和 (3) 的实例如下所示:

$$\begin{array}{r}
 \begin{array}{r}
 1010 \\
 \times 101 \\
 \hline
 1010 \\
 0000 \\
 1010 \\
 \hline
 100010
 \end{array}
 \qquad
 \begin{array}{r}
 101 \overline{) 10000} \quad \leftarrow \text{商} \\
 \underline{101} \\
 010 \\
 \underline{000} \\
 100 \\
 \underline{101} \\
 01 \quad \leftarrow \text{余数}
 \end{array}
 \end{array}$$

(2)

(3)

1. CRC 码的编码方法

设待编的信息码组为: $D_{n-1}D_{n-2}\cdots D_2D_1D_0$, 共 n 位, 它可用多项式 $M(x)$ 表示:

$$M(x)=D_{n-1}x^{n-1}+D_{n-2}x^{n-2}+\cdots\cdots+D_1x^1+D_0x^0$$

将信息码组左移 k 位, 得 $M(x) \cdot x^k$, 即成 $n+k$ 位信息码组:

$$D_{n-1-k}D_{n-2+k}\cdots\cdots D_{2-k}D_{1+k}D_{0+k}\underbrace{0000\cdots\cdots 0}_{k\text{位}}$$

空出的 k 位用来拼接 k 位校验位。

CRC 码就是用多项式 $M(x) \cdot x^k$ 除以生成多项式 $G(x)$ (即产生校验码的多项式), 所得余数作为校验位。为了得到 k 位余数 (校验位), $G(x)$ 必须是 $k+1$ 位。

设所得余数为 $R(x)$, 商为 $Q(x)$, 则有

$$M(x) \cdot x^k = Q(x) \cdot G(x) + R(x)$$

将余数拼接在左移了 k 位后的信息位后面, 就构成了这个有效信息的 CRC 码。这个 CRC 码用多项式表示为

$$\begin{aligned} M(x) \cdot x^k + R(x) &= [Q(x) \cdot G(x) + R(x)] + R(x) \\ &= [Q(x) \cdot G(x)] + [R(x) + R(x)] \\ &= Q(x) \cdot G(x) \quad (\text{模 } 2 \text{ 和}) \end{aligned}$$

因此, 所得 CRC 码是一个可被生成多项式 $G(x)$ 除尽的数码。如果 CRC 码在传输过程中不出错, 其余数必为 0; 如果传输过程中出错, 则余数不为 0, 再由该余数指出哪一位出错, 即可纠正之。

例 4.3 已知有效信息为 1100, 试用生成多项式 $G(x)=1011$ 将其编成 CRC 码。

解: 有效信息 $M(x)=1100=x^3+x^2 \quad (n=4)$

由 $G(x)=1011=x^3+x+1$

得 $k+1=4$

所以 $k=3$

将有效信息左移三位后再被 $G(x)$ 模 2 除, 即

$$\begin{aligned} M(x) \cdot x^3 &= 1100000 = x^6 + x^5 \\ \frac{M(x)x^3}{G(x)} &= \frac{1100000}{1011} = 1110 + \frac{010}{1011} \quad (\text{模 } 2 \text{ 除}) \end{aligned}$$

所以, $M(x) \cdot x^3 + R(x) = 1100000 + 010 = 1100010$ 为 CRC 码。

总的信息位为 7 位, 有效信息位为 4 位, 故上述 1100010 码又称 (7, 4) 码。这里的 (7, 4) 码, 即为码制, 还可以有 (7, 3) 码制和 (7, 6) 码制等。

2. CRC 码的译码和纠错

将收到的循环校验码用约定的生成多项式 $G(x)$ 去除, 如果无错, 则余数

应为 0, 如果某一位出错, 则余数不为 0。不同的出错位其余数也不同, 表 4.5 列出了对应 $G(x)=1011$ 的出错模式。

表 4.5 对应 $G(x)=1011$ 的 (7, 4) 循环的出错模式

序号	N_1	N_2	N_3	N_4	N_5	N_6	N_7	余数	出错位
正确	1	1	0	0	0	1	0	000	无
错 误	1	1	0	0	0	1	1	001	7
	1	1	0	0	0	0	0	010	6
	1	1	0	0	1	1	0	100	5
	1	1	0	1	0	1	0	011	4
	1	1	1	0	0	1	0	110	3
	1	0	0	0	0	1	0	111	2
	0	1	0	0	0	1	0	101	1

可以证明, 更换不同的待测码字, 余数和出错位的对应关系不变, 只与码制和生成多项式有关。表 4.5 给出的关系只对应 $G(x)=1011$ 的 (7, 4) 码, 对于其他码制或选用其他生成多项式, 出错模式将发生变化。

如果循环码有一位出错, 用 $G(x)$ 作模 2 除将得到一个不为 0 的余数。如果对余数补 0 继续除下去, 将发现各次所得余数将按表 4.5 顺序循环。例如第 7 位出错, 其余数为 001, 补 0 后再除, 第二次余数为 010, 以后依次为 100, 011……反复循环, 这就是“循环码”的名称由来。这个特点正好用来纠错, 即当出现不为零的余数后, 一方面对余数补 0 继续作模 2 除, 另一方面将被检测的校验码字循环左移。由表 4.5 可见, 当出现余数为 101 时, 出错位也移到了 N_1 位置。可通过异或门将它纠正后在下次移位时送回 N_7 。这样当移满一个循环 (对 7, 4 码共移七次) 后, 就得到一个纠正后的码字。

值得指出的是, 并不是任何一个 $(k+1)$ 位多项式都可以作为生成多项式。从检错和纠错的要求出发, 生成多项式应满足以下要求:

- (1) 任何一位发生错误, 都应该使余数不为零;
- (2) 不同位发生错误应使余数不同;
- (3) 对余数继续作模 2 除, 应使余数循环。

达到上述要求的数学关系比较复杂, 读者若有兴趣可查阅有关资料。

4.4.7 光盘存储器

1. 概述

光盘 (Optical Disk) 是利用光学方式进行读写信息的圆盘。光盘存储器是在激光视频唱片和数字音频唱片基础上发展起来的。应用激光在某种介质上

写入信息,然后再利用激光读出信息,这种技术叫光存储技术。如果光存储使用的介质是磁性材料,即利用激光在磁记录介质上存储信息,就叫做磁光存储。通常把采用非磁性介质进行光存储的技术,称为第一代光存储技术,它不能把内容抹掉重写新内容。磁光存储技术是在光存储技术基础上发展的,叫做第二代光存储技术,其主要特点是可擦洗重写。根据光存储性能和用途的不同,光盘存储器可分为三类。

(1) 只读型光盘 (CD-ROM)

这种光盘内的数据和程序是由厂家事先写入的,使用时用户只能读出,不能修改或写入新的内容。它主要用于电视唱片和数字音频唱片,可以获得高质量的图像和高保真的音乐。在计算机领域里,主要用于检索文献数据库或其他数据库,也可用于计算机的辅助教学等。因它具有 ROM 特性,故叫做 CD-ROM (Compact Disk-ROM)。

(2) 只写一次型光盘 (WORM)

这种光盘允许用户写入信息,写入后可多次读出,但只能写入一次,而且不能修改,故称它为“写一次型”WORM (Write Once Read Many)。主要用于计算机系统上的文件存档,或写入的信息不再需要修改的场合。

(3) 可擦写型光盘

这种光盘类似磁盘,可以重复读写。从原理上来看,目前仅有光磁记录(热磁反转)和相变记录(晶态-非晶态转变)两种。它是很有前途的辅助存储器。1989年下半年可擦写型5.25英寸的光盘,双面格式化的容量已达500~650MB。1991年上市的3.5英寸光盘单面格式化的容量已高达128MB。

2. 光盘的存取原理

光盘存储器利用激光束在记录表面上存储信息,根据激光束和反射光的强弱不同,可以实现信息的读写。由于光学读写头和介质保持较大的距离,因此,它是非接触型读写的存储器。

对于只读型和只写一次型光盘而言,写入时,将光束聚焦成直径为小于 $1\mu\text{m}$ 的微小光点,使其能量高度集中,在记录的介质上发生物理或化学变化,从而存储信息。例如,激光束以其热作用,融化盘表面的光存储介质薄膜,在薄膜上形成小凹坑,有坑的位置表示记录“1”,没坑的位置表示“0”。又比如,有些光存储介质在激光照射下,使照射点温度升高,冷却后晶体结构或晶粒大小会发生变化,从而导致介质膜光学性质发生变化(如折射率和反射率),利用这一现象便可记录信息。

读出时,在读出光束的照射下,在有凹处和无凹处反射的光强是不同的,利用这种差别,可以读出二进制信息。由于读出光束的功率只有写入光束的1/10,因此不会使盘面融出新的凹坑。

可擦写光盘是利用激光在磁性薄膜上产生热磁效应来记录信息（称作磁光存储）。其原理是：由磁记录原理可知，在一定温度下，对磁介质表面加一个强度高于该介质矫顽力的磁场，就会发生磁通翻转，这便可用于记录信息。矫顽力的大小是随温度而变的。倘若设法控制温度，降低介质的矫顽力，那么外加磁场强度便很容易高于此矫顽力，使介质表面磁通发生翻转。磁光存储就是根据这一原理来存储信息的。它利用激光照射磁性薄膜，使其被照处温度升高，矫顽力下降，在外磁场 HR 作用下，该处发生磁通翻转，并使其磁化方向与外磁场 HR 一致，这就可视作寄存“1”。不被照射处，或 HR 小于矫顽力处可视为寄存“0”。通常把这种磁记录材料因受热而发生磁性变化的现象，叫做热磁效应。

图 4.67 (a) 表示在记录方向外加一个小于矫顽力的磁场 HR，其介质表面不发生翻转；(b) 表示激光照射处温度上升，外加的磁场 HR 大于矫顽力，而使其发生磁通翻转；(c) 表示照射后，将磁通翻转保持下来，即写入了信息。

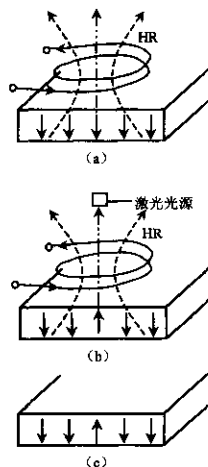


图 4.67 磁光记录原理

擦除信息和记录信息原理一样，擦除时外加一个和记录方向相反的磁场 HR，对已写入的信息用激光束照射，并使 HR 大于矫顽力，那么，被照射处又发生反方向磁化，使之恢复为记录前的状态。

这种利用激光的热作用改变磁化方向来记录信息的光盘，叫作“磁光盘”。

由于光盘是靠直径小于 $1\mu\text{m}$ 的激光束写入每位信息, 因此记录密度高, 可达 10^8 位/平方厘米, 约为磁盘的 $10\sim 100$ 倍。

光盘记录头份量重, 体积大, 使寻道时间长约 $30\sim 100\text{ms}$ 。写入速度低, 约为 0.2 秒, 平均存取时间为 $100\sim 500\text{ms}$, 与主机交换信息速度不匹配。因此, 它不能代替硬盘只能作为硬盘的后备存储器。

光盘的介质互换性好, 存储容量大, 可用于文献档案、图书管理、多媒体等方面的应用。但由于目前价格比较贵, 故尚不能替代磁带机。

硬盘存储器容量大, 数据传输率比光盘高 (采用磁盘阵列, 数据传输率可达 100Mb/s), 等待时间短, 它作为主存的后备存储器, 用以存放程序的中间和最后结果。

软盘存储器容量小, 数据传输率低, 平均寻道时间长, 而且是接触式存取, 盘片不固定在驱动器中, 运行时有大量的灰尘进入盘面, 易造成盘面磨损或出现误码, 不易提高位密度。但软盘盘片灵活装卸, 便于携带, 互换性好, 价格便宜。因此, 用它存储操作系统和应用软件极为方便。还可用于数据的输入、输出。特别是当今软件发展迅速, 种类繁多, 人们越来越愿意用软盘作为自己的小型数据库和软件库。

磁带存储器的历史比磁盘更久, 20 世纪 60 年代后期逐渐被磁盘取代。它的数据传输率更低, 采用接触式记录, 容量也很大, 每兆字节价格较低, 记录介质也容易装卸、互换和携带, 可用作硬盘的后备存储器。据统计, 80% 的磁带被用作磁盘的后备存储器, 20% 用作计算机的输入输出数据和文件的储存。

思考题与习题

1. 解释下列概念:

主存、辅存、Cache、RAM、SRAM、DRAM、ROM、
PROM、EPROM、EEPROM、CDROM、Flash Memory

2. 计算机中哪些部件可用于存储信息, 请按其速度、容量和价格/位排序说明。
3. 存储器的层次结构主要体现在什么地方? 为什么要分这些层次, 计算机如何管理这些层次?
4. 说明存取周期和存取时间的区别。
5. 什么是存储器的带宽? 若存储器的数据总线宽度为 32 位, 存取周期为 200ns , 则存储器的带宽是多少?
6. 某机字长为 32 位, 其存储容量是 64KB , 按字编址它的寻址范围是多少? 若主存以字节编址, 试画出主存字地址和字节地址的分配情况。
7. 一个容量为 $16\text{K}\times 32$ 位的存储器, 其地址线和数据线的总和是多少? 当选下列不同规

格的存储芯片时,各需要多少片?

1K×4 位, 2K×8 位, 4K×4 位, 16K×1 位, 4K×8 位, 8K×8 位

8. 试比较静态 RAM 和动态 RAM。
9. 什么叫刷新? 为什么要刷新? 说明刷新有几种方法。
10. 半导体存储器芯片的译码驱动方式有几种?
11. 画出用 1 024×4 位的存储芯片组成一个容量为 64K×8 位的存储器逻辑框图。要求将 64K 分成 4 个页面^①, 每个页面分 16 组, 指出共需多少片存储芯片。
12. 设有一个 64K×8 位的 RAM 芯片, 试问该芯片共有多少个基本单元电路(简称存储基元)? 欲设计一种具有上述同样多存储基元的芯片, 要求对芯片字长的选择应满足地址线和数据线的总和为最小, 试确定这种芯片的地址线 and 数据线, 并说明有几种解答。
13. 某 8 位微型机地址码为 18 位, 若使用 4K×4 位的 RAM 芯片组成模块板结构的存储器, 试问:
 - (1) 该机所允许的最大主存空间是多少?
 - (2) 若每个模块板为 32K×8 位, 共需几个模块板?
 - (3) 每个模块板内共有几片 RAM 芯片?
 - (4) 共有多少片 RAM?
 - (5) CPU 如何选择各模块板?
14. 设 CPU 共有 16 根地址线, 8 根数据线, 并用 $\overline{\text{MREQ}}$ (低电平有效) 作访存控制信号,

$\text{R}/\overline{\text{W}}$ 作读写命令信号(高电平为读, 低电平为写)。现有列存储芯片:

ROM (2K×8 位, 4K×4 位, 8K×8 位),

RAM (1K×4 位, 2K×8 位, 4K×8 位)

及 74138 译码器和其他门电路(门电路自定)。

试从上述规格中选用合适芯片, 画出 CPU 和存储芯片的连接图。要求:

- (1) 最小 4K 地址为系统程序区, 4096~16383 地址范围为用户程序区;
- (2) 指出选用的存储芯片类型及数量;
- (3) 详细画出片选逻辑。
15. CPU 假设同上题, 现有 8 片 8K×8 位的 RAM 芯片与 CPU 相连, 试回答:
 - (1) 用 74138 译码器画出 CPU 与存储芯片的连接图;
 - (2) 写出每片 RAM 的地址范围;
 - (3) 如果运行时发现不论往哪片 RAM 写入数据后, 以 A000H 为起始地址的存储芯片都有与其相同的数据, 分析故障原因。

① 将存储器分成若干个容量相等的区域, 每一个区域可看作一个页面。

- (4) 根据(1)的连接图,若出现地址线 A_{13} 与 CPU 断线,并搭接到高电平上,将出现什么后果?
16. 反映主存和外存的速度指标有何不同?
17. 某机字长 16 位,常规的存储空间为 64K 字,若想不改用其他高速的存储芯片,而使访存速度提高到 8 倍,可采取什么措施?画图说明。
18. 什么是“程序访问的局部性”?存储系统中哪一级采用了程序访问的局部性原理?
19. 计算机中设置 Cache 的作用是什么?能不能把 Cache 的容量扩大,最后取代主存,为什么?
20. Cache 做在 CPU 芯片内有什么好处?将指令 Cache 和数据 Cache 分开又有什么好处?
21. 设某机主存容量为 4MB,Cache 容量为 16KB,每字块有 8 个字,每字 32 位,设计一个四路组相联映象(即 Cache 每组内共有 4 个字块)的 Cache 组织,要求:
- (1) 画出主存地址字段中各段的位数;
- (2) 设 Cache 的初态为空,CPU 依次从主存第 0、1、2、……99 号单元读出 100 个字(主存一次读出一个字),并重复按此次序读 8 次,问命中率是多少?
- (3) 若 Cache 的速度是主存的 6 倍,试问有 Cache 和无 Cache 相比,速度提高多少倍?
22. 简要说明提高访存速度可采取那些措施?
23. 画出 NR、NRZ、NRZ1、PE、FM 写入数字串 1011001 的写入电流波形图。
24. 以写入 10010110 为例,比较调频制和改进调频制的写电流波形图。
25. 画出调制制记录 01100010 的驱动电流、记录磁通、感应电势、同步脉冲及读出代码等几种波形。
26. 磁盘组有六片磁盘,每片有两个记录面,存储区域内径 22 厘米,外径 33 厘米,道密度为 40 道/厘米,内层密度为 400 位/厘米,转速 2400 转/分,问:
- (1) 共有多少存储面可用?
- (2) 共有多少柱面?
- (3) 盘组总存储容量是多少?
- (4) 数据传输率是多少?
27. 某磁盘存储器转速为 3000 转/分,共有 4 个记录盘面,每毫米 5 道,每道记录信息 12288 字节,最小磁道直径为 230mm,共有 275 道,求:
- (1) 磁盘存储器的存储容量;
- (2) 最高位密度(最小磁道的位密度)和最低位密度;
- (3) 磁盘数据传输率;
- (4) 平均等待时间。
28. 磁表面存储器和光盘存储器记录信息的原理有何不同?
29. 试从存储容量、存取速度、使用寿命和应用场合方面比较磁盘、磁带和光盘存储器。
30. 写出 1100、1101、1110、1111 对应的海明码。

- (1) 按配偶原则配置;
- (2) 按配奇原则配置。
31. 已知接受到的海明码 (按配偶原则配置) 为 1100100, 1100111, 1100000, 1100001, 检查上述代码是否出错? 第几位出错?
32. 已知接收到下列海明码, 分别写出它们所对应的欲传送代码。
- 1100000 (按偶性配置);
- 1100010 (按偶性配置);
- 1101001 (按偶性配置);
- 0011001 (按奇性配置);
- 1000000 (按奇性配置);
- 1110001 (按奇性配置);
33. 欲传送的二进制代码为 1001101, 用奇校验来确定其对应的海明码, 若在第六位出错, 说明它的纠错过程。
34. 为什么海明码纠错过程中, 新的检测位 P_0, P_2, P_1 的状态即指出了编码中错误的信息位?
35. 设有效信息为 110, 试用生成多项式 $G(x) = 11011$, 将其编成循环冗余校验码。
36. 有一个 (7, 4) 码, 其生成多项式 $G(x) = x^3 + x + 1$, 写出代码 1001 的循环冗余校验码。

附录 4A 相联存储器

相联存储器既可按地址寻址，又可按内容（通常是某些字段）寻址，为与传统存储器区别，又叫按内容寻址的存储器。

相联存储器的每个字由若干字段组成，每个字段描述了一个对象的属性，也称一个内容。如在存储学生信息的相联存储器中，可分为学号、姓名、年龄、班号、成绩等字段（见图 4.70）

相联存储器的基本组成如图 4.69 所示。

图中检索寄存器 CR 用来存放检索字，其位数与相联存储器的字长相等。屏蔽寄存器 MR 用来存放屏蔽码，其位数与检索寄存器位数相同，其内容与需要检索的字段有关。如需检索 CR 的高 6 位字段（称为检索项），则 MR 的高 6 位为“1”，其余各位为“0”，即把 CR 中的第 7 到第 n 位屏蔽掉，也即这些位不参加比较。比较线路是把检索项和所有存储单元的相应位进行比较，如

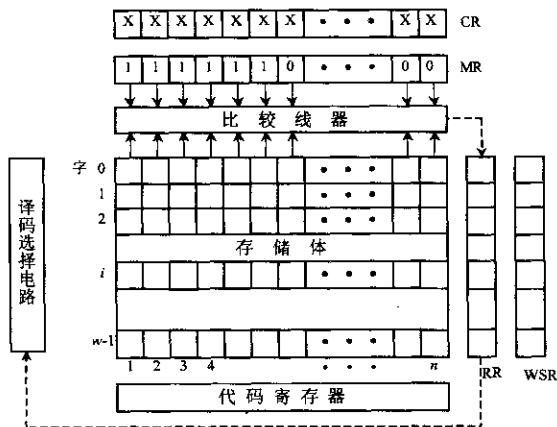


图 4.69 相联存储器基本组成框图

果比较结果相等，就将符合寄存器 RR 的相应位置“1”。RR 又叫查找结果寄存器，其位数等于相联存储器的字数。如果比较结果第 i 个字满足要求，则 RR 的第 i 位为“1”，其余各位为“0”；如果同时有 5 个字都满足要求，则 RR 中

就有 5 位为“1”。有的相联存储器还设有字选择寄存器 WSR，用来确定哪些存储字参与检索。若 WSR 某位为“1”，则表示对应的存储字参与检索，而对应 WSR 某位为“0”的存储字则不参与检索。可见 WSR 的位数与存储器字数相同。代码寄存器用来存放从存储体中读出的代码，或存放写至存储体中的代码。

相联存储器有三种基本操作：读、写、检索（比较）。读、写操作与传统存储器相同，检索只能按内容进行。例如，某系学生的考试成绩已存入相联存储器中，如图 4.70 所示。要求列出总分在 580 分到 600 分范围内的学生名单，可通过两次查找来完成。第一次找出总分大于 579 的学生名单，第二次找出总分小于 601 的学生名单。可见总分字段是关键字，故需要将 MR 中对应的位置成“1”，其他字段置成“0”。第一次查找时，CR 中的“总分”字段是 579（二进制表示），查找结果送入 RR。第二次查找时，将 CR 中“总分”字段改为“601”，并且将 RR 的内容送至 WSR，这样，第二次查找只需查 WSR 中对应“1”的各个存储字。最后将查找结果送入 RR，此时 RR 中为“1”的各位所对应的学生，其成绩便在 580~600 分之间。通过打印机将名单打印出来。

x	xxx	x	xx	xxxx	579	CR (第一次查找的内容)	
0...0	0...0	0...0	0...0	0...0	11...1	MR	
1	赵 xx	男	17	985101	586	1	1
2	钱 xx	女	18	985101	607	0	1
3	孙 xx	男	18	985102	582	1	1
4	李 xx	男	19	985103	570	0	0
					614	0	1
⋮						⋮	⋮
n	丁 xx	女	19	985105	590	1	1
学号	姓名	性别	年龄	班号	总分	RR	WSR

图 4.70 相联存储器检索举例

这里需要特别指出的是，相联存储器每次查找是将所有存储字的相关字段与检索项同时进行比较，这是由相联存储器的具体电路实现的。而如果是按地址访问的存储器，查找时则必须一次读出一个存储字，逐一与检索项进行比较。如果设存储器有 M 个单元，那么按地址访问的存储器检索出某一单元，平均需进行 $M/2$ 次操作，而相联存储器仅需进行一次检索操作。可见相联存储器大大提高了处理速度。

相联存储器还可以进行各种比较，如大于、小于、相等、不等、求最大值、

求最小值、相似、接近以及其他各种类型的逻辑检索。因此，相联存储器的每个单元不仅能存储，还要能进行逻辑运算，所以也称其为分布逻辑存储器。显然，其电路比一般存储器复杂得多，故相联存储芯片比一般存储芯片昂贵。随着大规模集成电路集成度的提高，相联存储芯片已由 4K 位、8K 位、发展到 20K 位，商品化容量已经达到 256×48 位。

相联存储器的原理在 Cache 中得到应用。例如在 Cache 中，将主存的字块地址同时与每个缓存字块的“标记”进行比较，就可迅速判断出该主存字块是否“命中”。若比较相等，表示命中，即可从缓存中读出信息；若比较不等，即不命中，则需将新的主存块调入缓存。

此外，相联存储器还广泛应用于虚拟存储器中，还常用于数据库和知识库中。近年来相联存储器在语音识别、图像处理、数据流计算机和 Prolog 机中也都有所应用。

第五章 输入输出系统

除了 CPU 和存储器两大模块外,计算机硬件系统的第三个关键部分即是输入输出模块,又称输入输出系统。随着计算机系统的不断发展,应用范围的不断扩大,输入输出设备的数量和种类也愈来愈多,它们与主机的联络方式及信息的交换方式也越来越各不相同。因此,输入输出系统涉及的内容极其繁杂,既包括具体的各类 I/O 设备,又包括各种不同的设备如何与主机交换信息。本章重点分析 I/O 与主机交换信息的三种控制方式(程序查询、中断和 DMA)及其相应的接口功能和组成,对几种常用的 I/O 设备也作了简单介绍,旨在使读者对 I/O 系统有一个较清晰的认识,进一步加深整机工作概念。

5.1 概 述

5.1.1 输入输出系统的发展概况

输入输出系统的发展大致可分为四个阶段。

1. 早期阶段

早期的 I/O 设备种类较少, I/O 设备与主机交换信息都必须通过 CPU, 如图 5.1 所示。

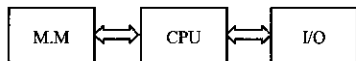


图 5.1 I/O 通过 CPU 与主机交换信息

这种交换方式延续了相当长的时间。当时的 I/O 设备具有以下几个特点:

- 每个 I/O 设备都必须配有一套独立的逻辑电路与 CPU 相连, 用来实现 I/O 与主机交换信息, 因此线路十分零散、庞杂。
- 输入输出过程是穿插在 CPU 执行程序之中进行的, 当 I/O 与主机交换信息时, CPU 不得不停止其各种运算, 因此, I/O 与 CPU 是按串行方式工作的, 极浪费时间。
- 每个 I/O 设备的逻辑控制电路与 CPU 的控制器紧密构成一个不可分割的整体, 它们彼此依赖, 相互牵连, 因此, 欲增添或撤减或更换 I/O 设备是非常困难的。

在这个阶段中, 计算机系统硬件价格十分昂贵, 机器速度不高, 配置的 I/O

设备不多, 主机与 I/O 交换的信息量也不大, 计算机应用极不普及。

2. 接口模块和 DMA 阶段

这个阶段 I/O 设备通过接口模块与主机连接, 计算机系统采用了总线结构, 如图 5.2 所示。

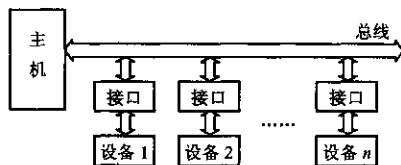


图 5.2 I/O 通过接口与主机交换信息

通常在接口中都设有数据通路和控制通路。数据经过接口既起到缓冲作用, 又可完成串—并变换或并—串变换。控制通路用以传送 CPU 向 I/O 设备发出的各种控制命令, 或使 CPU 接受来自 I/O 设备的反馈信号。许多接口还能满足中断请求处理的要求, 使 I/O 设备与 CPU 可按并行方式工作, 大大地提高了 CPU 的工作效率。采用接口技术还可以使多台 I/O 设备分时占用总线, 使多台 I/O 设备互相之间也可实现并行工作方式, 有利于整机工作效率提高。

虽然这个阶段实现了 CPU 和 I/O 并行工作, 但是在主机与 I/O 交换信息时, CPU 要中断现行程序, 也即 CPU 与 I/O 还不能做到绝对的并行工作。

为了进一步提高 CPU 的工作效率, 又出现了 DMA(Direct Memory Access)技术, 其特点是 I/O 与主存之间有一条直接数据通路, I/O 设备可以与主存直接交换信息, 使 CPU 在 I/O 与主存交换信息时, 能继续完成自身的工作, 故其资源利用率得到了进一步的提高。

3. 具有通道结构的阶段

在小型和微型计算机中, 采用 DMA 方式可实现高速外设与主机成组数据的交换, 但在大、中型计算机中, 外设配置繁多, 数据传送频繁, 若仍采用 DMA 方式会出现一系列问题:

(1) 如果每台外设都配置专用的 DMA 接口, 不仅增加了硬件成本, 而且为了解决众多 DMA 同时访问主存的冲突问题, 会使控制变得十分复杂。

(2) CPU 需要对众多的 DMA 进行管理, 同样会占用 CPU 的工作时间, 而且因频繁地进入周期挪用阶段, 也会直接影响 CPU 的整体工作效率(详见 5.6 节)。

因此在大、中型计算机系统中, 采用了 I/O 通道的方式来进行数据交换。图 5.3 示意了具有通道结构的计算机系统。

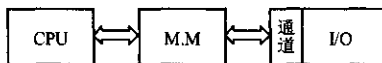


图 5.3 I/O 通过通道与主机交换信息

通道是用来负责管理 I/O 设备以及实现主存与 I/O 设备之间交换信息的部件，它可视为一种具有特殊功能的处理器。通道有专用的通道指令，它能独立地执行用通道指令所编写的输入输出程序，但它不是一个完全独立的处理器，它受 CPU 的 I/O 指令启动、停止或改变其工作状态，是从属于 CPU 的一个专用处理器。依赖通道管理的 I/O 设备在与主机交换信息时，CPU 不直接参与管理，故 CPU 的资源利用率更高。

4. 具有 I/O 处理机的阶段

输入输出系统发展到第四阶段是具有 I/O 处理机的阶段。I/O 处理机又叫做外围处理机 (Peripheral Processor Unit 或 PPU)，它基本独立于主机工作，既可完成 I/O 通道要完成的 I/O 控制，还可完成码制变换、格式处理、数据块检错、纠错等操作。具有 I/O 处理机的输入输出系统与 CPU 工作的并行性更高，这说明 I/O 系统对主机来说，具有更大的独立性。

本章主要介绍第二阶段的输入输出系统，有关通道及 I/O 处理机管理 I/O 系统的内容将在《计算机体系结构》中讲述。

5.1.2 输入输出系统的组成

输入输出系统应该由 I/O 软件和 I/O 硬件两部分组成。

1. I/O 软件

输入输出系统软件的主要任务是：①如何将用户编制的程序（或数据）输入至主机内；②如何将运算结果输送给用户；③如何实现 I/O 系统与主机工作的协调等。

不同结构的 I/O 系统所采用的软件技术差异很大。一般而言，当采用接口模块方式时，应用机器指令系统中的 I/O 指令及系统软件中的管理程序，便可使 I/O 与主机协调工作。当采用通道管理方式时，除 I/O 指令外，还必须要有通道指令及相应的操作系统。即使都采用操作系统，不同的机器其操作系统的复杂程度差异也是很大的。

(1) I/O 指令

I/O 指令是机器指令的一类，其指令格式与其他指令既有相似之处，又有不同点。I/O 指令可以和其他机器指令的字长相等，但它还应该能反映 CPU 与 I/O 设备交换信息的各种特点，如它必须反映出对多台 I/O 设备的选择，以

及在完成信息交换过程中,对不同设备应作哪些具体操作等。图 5.4 示意了 I/O 指令的一般格式。

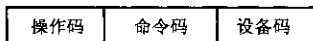


图 5.4 I/O 指令的一般格式

图中的操作码字段可作为 I/O 指令与其他类指令(如访存指令、算逻辑指令、控制指令等)的判别代码;命令码用来体现 I/O 的具体操作;设备码是作为对多台 I/O 设备的选择码。

I/O 指令的命令码,一般可表述如下几种情况:

- 将数据从 I/O 设备输入至主机。例如将某台设备接口电路中的数据缓冲寄存器中的数据读至 CPU 的某个寄存器(如累加器 ACC)中;
- 将数据从主机输出至 I/O 设备,例如将 CPU 中的某个寄存器(如 ACC)中的数据写入到某台设备接口电路中的数据缓冲寄存器内;
- 状态测试。利用命令码检测各个 I/O 设备所处的状态是“忙”(Busy)还是“准备就绪”(Ready),以便决定下一步是否可进入主机与 I/O 交换信息的阶段;
- 形成某些操作命令。不同 I/O 设备与主机交换信息时,需完成不同的操作。如磁带机需要正转、反转、读、写、写文件结束等等。又如对于磁盘驱动器,需要读扇区、写扇区、找磁道、扫描记录标识符等。这里值得注意的是:在第四章中,按磁盘机和磁带机的功能来看,它们都被视为存储系统的一部分。但从管理角度来看,调用这些设备与调用其他 I/O 设备又有共同之处,因此,本章又将它们视为 I/O 设备。

I/O 指令的设备码相当于设备的地址。只有对繁多的 I/O 设备赋以不同的编号,才能准确选择某台设备与主机交换信息。

(2) 通道指令

通道指令是对具有通道的 I/O 系统专门设置的指令,这类指令一般用以指明参与传送(写入或读出)的数据组在主存中的首地址;指明需要传送的字数或所传送数据组的末地址;指明所选设备的设备码及完成某种操作的命令码。这类指令的位数一般较长,如 IBM/370 机的通道指令为 64 位。

通道指令又叫通道控制字(CCW),它是通道用于执行 I/O 操作的指令,它可以由管理程序存放在主存的任何地方,由通道从主存中取出并执行。通道程序即由通道指令组成,它完成某种外围设备与主存传送信息的操作。如将磁带记录区的部分内容送到指定的主存缓冲区内。

通道指令是通道自身的指令,用来执行 I/O 操作,如读、写、反读、磁带走带及磁盘找道等。而 I/O 指令是 CPU 指令系统的一部分,是 CPU 用来控制

输入输出操作的指令，由 CPU 译码后执行。在具有通道结构的机器中，I/O 指令不实现 I/O 数据传送，主要完成启、停 I/O 设备，查询通道和 I/O 设备的状态及控制通道所作的其他一些操作。具有通道指令的计算机，一旦 CPU 执行了启动 I/O 的指令后，就由通道来代替 CPU 对 I/O 的管理。

2. I/O 硬件

输入输出系统的硬件组成是多种多样的，在带有接口的 I/O 系统中，一般包括接口模块及 I/O 设备两大部分。图 5.2 中的接口电路，实际上包含许多数据传送线和有关数据，还包含控制信号通路及其相应的逻辑电路（详见 5.3 节）。

图 5.5 是具有通道的 I/O 系统示意。

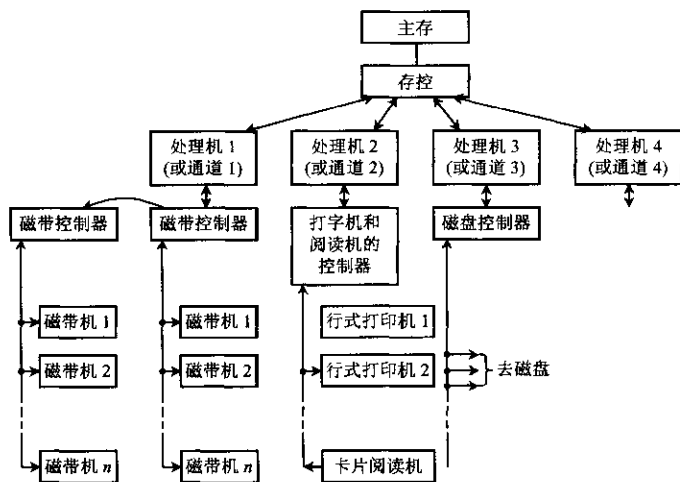


图 5.5 具有通道的 I/O 系统

一个通道可以和一个以上的设备控制器相连，一个设备控制器又可以控制若干台同一类型的设备。如 IBM/360 系统的一个通道可以连接 8 个设备控制器，一个设备控制器又与 8 台设备相连，因此，一个通道可以管理 64 台设备。如果一台机器有 6 个通道，便可带动 384 台设备。当然，实际上由于设备利用率和通道的频带影响，主机不可能带动这么多的设备。

5.1.3 I/O 设备与主机的联系方式

I/O 设备与主机交换信息和 CPU 与主存交换信息相比，有许多不同点。

例如, CPU 如何对 I/O 编址; 如何寻找 I/O 设备号; 信息传送是逐位串行还是多位并行; I/O 与主机以什么方式进行联络, 使它们之间彼此都知道双方处于何种状态; 以及 I/O 与主机是怎么连接的等等。这一系列问题统称为 I/O 与主机的联系方式。

1. I/O 编址方式

通常将 I/O 设备码视为地址码, 对 I/O 地址码的编址可采用两种方式: 统一编址或不统一编址。统一编址就是将 I/O 地址看作是存储器地址的一部分。如在 64K 地址的存储空间中, 划出 8K 地址作为 I/O 的地址, 凡是在这 8K 地址范围内的访问, 就是对 I/O 的访问, 所用的指令与访存指令相似。不统一编址就是指 I/O 地址和存储器地址是分开的, 所有对 I/O 的访问必须有专用的 I/O 指令。显然统一编址占用了存储空间, 减少了主存容量, 但无需专用的 I/O 指令。不统一编址由于不占用主存空间, 故不影响主存容量, 但需设 I/O 专用指令。因此, 设计机器时, 需根据实际情况权衡考虑选取何种编址方式。

当设备通过接口与主机相连时, CPU 可以通过接口地址来访问 I/O 设备。

2. 设备寻址

由于每台设备都赋予一个设备号, 因此, 当要启动某一设备时, 可由 I/O 指令的设备码字段直接指出该设备的设备号。通过接口电路中的设备选择电路, 便可选中要交换信息的设备。

3. 传送方式

在同一瞬间, n 位信息同时从 CPU 输送至 I/O 设备, 或由 I/O 设备输入到 CPU, 这种传送方式叫做并行传送。其特点是传送速度较快, 但要求数据线多, 如 16 位信息并行传送, 需 16 根数据线。

若在同一瞬间只传送一位信息, 在不同时刻连续逐位传送一串信息, 这种传送方式叫做串行传送。其特点是传送速度较慢, 但它只需一根数据线和一根地线。当 I/O 设备与主机距离很远时, 采用串行传送较为合理, 例如远距离数据通讯。

不同的传送方式需配置不同的接口电路, 如并行传送接口、串行传送接口或串并联用的传送接口等。用户可按需要选择合适的接口电路。

4. 联络方式

不论是串行传送还是并行传送, I/O 设备与主机之间必须互相了解彼此当时所处的状态, 如相互是否可以传送, 传送是否已结束等等。这就是 I/O 设备与主机之间的联络问题。按 I/O 设备工作速度的不同, 可分为三种联络方式。

(1) 立即响应方式

对于一些工作速度十分缓慢的 I/O 设备, 如指示灯的亮与灭; 开关的通与

断；A/D 转换器渐变信号的输入等等。当它们与 CPU 发生联系时，通常都已使其处于某种等待状态，因此，只要 CPU 的 I/O 指令一到，它们便立即响应，故这种设备无需特殊联络信号，称作立即响应方式。

(2) 异步工作采用应答信号联络

当 I/O 设备与主机工作速度不匹配时，通常采用异步工作方式。这种方式在交换信息前，I/O 与 CPU 各自完成自身的任务，一旦出现联络信号时，彼此才准备交换信息。图 5.6 示意了并行传送的异步联络方式。

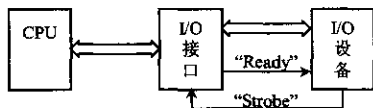


图 5.6 异步并行“应答”联络方式

图中示意当 CPU 将数据输出到 I/O 接口后，接口立即向 I/O 设备发出一个“Ready”（准备就绪）信号，告诉 I/O 设备可以从接口内取数据。I/O 设备收到“Ready”后，通常便立即从接口中取出数据，接着便向接口回发一个“Strobe”信号，并让接口转告 CPU，接口中的数据已被取走，CPU 还可继续向此接口送数。同理，倘若 I/O 设备需向 CPU 传送数据，则先由 I/O 向接口送数据，并向接口发“Strobe”信号，表明数据已送出。接口接到联络信号后便通知 CPU 可以来取数，一旦 CPU 取走时，接口便向 I/O 设备发“Ready”信号，告诉 I/O 设备，数据已被取走，尚可继续送数。这种一应一答的联络方式，称作异步联络。

图 5.7 示意了串行传送的异步联络方式。

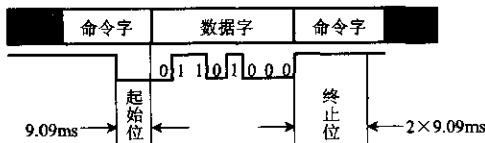


图 5.7 异步串行联络方式

I/O 设备与 CPU 双方设定一组特殊标记，用“起始”和“终了”来建立联系。图中 9.09ms 的低电平表示“起始”，又用 $2 \times 9.09\text{ms}$ 的高电平表示“终了”。

(3) 同步工作采用同步时标联络

同步工作要求 I/O 设备与 CPU 的工作速度完全同步，例如在数据采集过

程中,若外部数据以 2 400 位/秒速率传送至接口,则 CPU 也必须以 1/2 400 秒的速率接收每一位数。这种联络互相之间还得配有专用电路,用以产生同步时间来控制同步工作。

5. I/O 与主机的连接方式

I/O 设备与主机的连接方式通常有两种:辐射式和总线式。图 5.8 和图 5.2 分别示意了这两种方式。

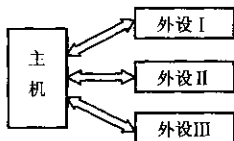


图 5.8 I/O 设备与主机的辐射式连接示意

采用辐射式连接方式时,要求每台 I/O 设备都有一套控制线路和一组信号线,因此所用的器件和连线较多,对 I/O 设备的增删都比较困难。这种连接方式大多出现在计算机发展的初期阶段。

图 5.2 所示的是总线连接方式,通过一组总线(包括地址线、数据线、控制线等),将所有的 I/O 设备与主机连接。这种连接方式是现代大多数计算机系统所采用的方式。

5.1.4 I/O 与主机信息传送的控制方式

I/O 设备与主机交换信息时,共有五种控制方式:程序查询方式、程序中中断方式、直接存储器存取方式(DMA)、I/O 通道方式、I/O 处理机方式。本节主要介绍前三种方式,后两种方式在 5.1.1 节已作了一般介绍,更详尽的内容将由《计算机体系结构》讲述。

1. 程序查询方式

程序查询方式是由 CPU 通过程序不断查询 I/O 设备是否已做好准备,从而控制 I/O 与主机交换信息。采用这种方式实现主机和 I/O 交换信息,要求 I/O 接口内设置一个能反映设备是否准备就绪的状态标记,CPU 通过对此标记的检测,可得知设备的准备情况。图 5.9 示意了 CPU 欲从某一外设读数据块(例如从磁带上读一记录块)至主存的查询方式流程。当现行程序需启动某设备工作时,即将此程序流程插入到运行的程序中。由图可知,CPU 启动 I/O 后便开始对 I/O 的状态进行查询。若查得 I/O 未准备就绪,就继续查询;若查得 I/O 准备就绪,就将数据从 I/O 接口送至 CPU,再由 CPU 送至主存。这样一个字一个字地传送,直至这个数据块的数据全部传送结束,CPU 又重新回到原现行程序。

由这个查询过程可见,只要 CPU 一启动 I/O 设备,CPU 便不断查询 I/O 的准备情况,从而终止了原程序的执行。CPU 在反复查询过程中,犹如就地

“踏步”。另一方面，I/O 准备就绪后，CPU 要一个字一个字地从 I/O 设备取出，经 CPU 送至主存，此刻 CPU 也不能执行原程序，可见这种方式使 CPU 和 I/O 处于串行工作状态，CPU 的工作效率不高。

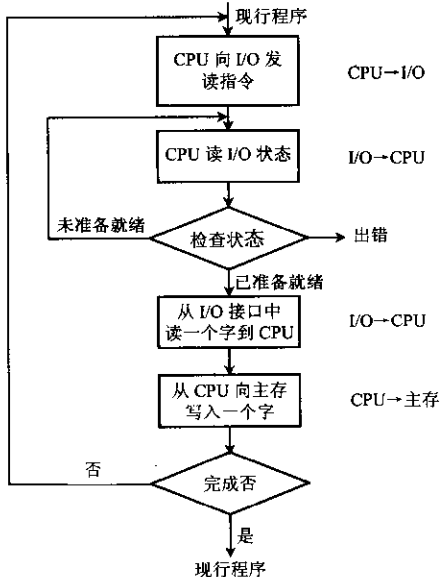


图 5.9 程序查询方式流程

2. 程序中中断方式

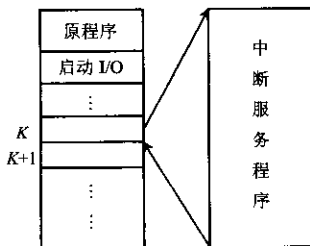


图 5.10 程序中中断方式示意图

倘若 CPU 在启动 I/O 设备后，对设备是否已准备就绪不加过问，继续执行自身程序，只是当 I/O 设备准备就绪并向 CPU 发出中断请求后才予以理睬，这将大大提高 CPU 的工作效率。图 5.10 示意了这种方式。

由图可见，CPU 启动 I/O 后仍继续执行原程序，在第 K 条指令执行结束后，CPU 响应了 I/O 的请求，中断了现行程序，转至中

断服务程序，待处理完后又返回到原程序断点处，继续从第 $K+1$ 条指令往下执行。由于这种方式使原程序中断了运行，故叫程序中中断方式。

图 5.11 示意了采用程序中中断方式从外设读数据块到主存的程序流程。

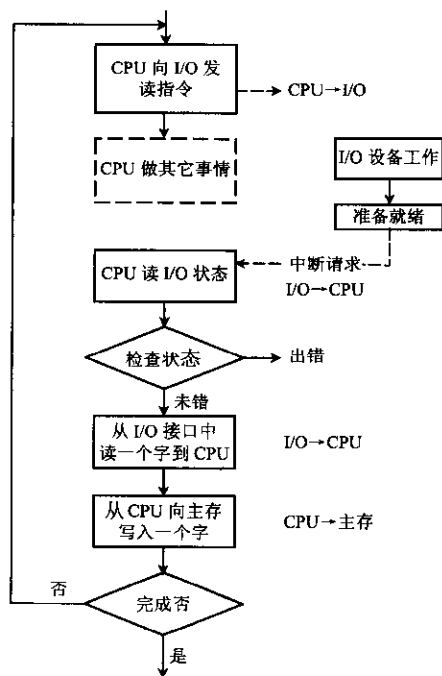


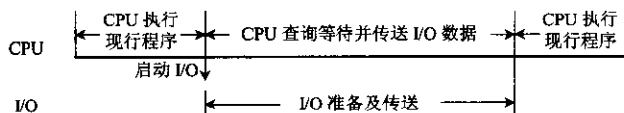
图 5.11 程序中中断方式流程

由图可见，CPU 向 I/O 设备发出读命令后，仍在处理其他一些事情（如继续在算题），当设备向 CPU 发出请求后，CPU 才从 I/O 接口读一个字经 CPU 送至主存（这是通过执行中断服务程序完成的）。如果 I/O 设备的一批数据（一个数据块的全部数据）尚未传送结束时，CPU 再次启动 I/O 设备，命令 I/O 设备再作准备，一旦又接收到 I/O 设备中断请求时，CPU 又重复上述中断服务过程，这样周而复始，直至一批数据传送完毕。

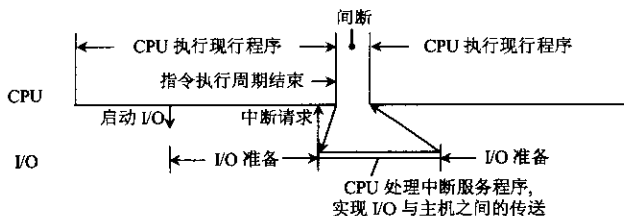
显然，程序中中断方式在 I/O 进行准备时，CPU 不必时刻查询 I/O 的准备情况，不出现“踏步”现象，即 CPU 执行程序与 I/O 设备作准备是同时进行的，

这种方式和 CPU 与 I/O 是串行工作的程序查询方式相比,其 CPU 的资源得到了充分的利用。图 5.12 (a)、(b) 分别示意了这两种方式 CPU 的工作效率。

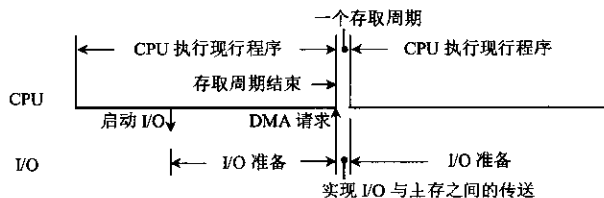
当然,采用程序中断方式, CPU 和 I/O 接口不仅在硬件方面需增加相应的电路,而且在软件方面还必须编制中断服务程序,这方面内容将在 5.3 和 5.5 节中详细讲述。



(a) 程序查询方式



(b) 程序中断方式



(c) DMA 方式

图 5.12 三种方式的 CPU 工作效率比较

3. DMA 方式

虽然程序中断方式消除了程序查询方式的“踏步”现象,提高了 CPU 资源的利用率,但是 CPU 在响应中断请求后,必须停止现行程序而转入中断服务程序,并且为了完成 I/O 与主存交换信息,还不得不占用 CPU 内部的一些寄存器,这同样是对 CPU 资源的消耗。如果 I/O 设备能直接与主存交换信息

而不占用 CPU，那么，CPU 的资源利用率显然又可进一步提高，这就出现了直接存储器存取的 DMA 方式。

在 DMA 方式中，主存与 I/O 设备之间有一条数据通路，主存与 I/O 设备交换信息时，无需处理中断服务程序。若出现 DMA 和 CPU 同时访问主存，CPU 总是将总线占有权让给 DMA，通常把 DMA 的这种占有叫做“窃取”或“挪用”。窃取的时间一般为一个存储周期，故又把 DMA 占用的存取周期叫做“窃取周期”或“挪用周期”。而且，在 DMA 窃取存取周期时，CPU 尚能继续作内部操作（如乘法运算）。可见，DMA 方式与程序查询和程序中断方式相比，又进一步提高了 CPU 的资源利用率。

图 5.12 (c) 示意了 DMA 方式的 CPU 效率。当然，采用 DMA 方式时，也需增加必要的 DMA 接口电路。有关 DMA 方式的详细内容，将在 5.6 节讲述。

5.2 外部设备

5.2.1 概述

中央处理器和主存构成了主机，除主机外的大部分硬件设备都可称作外部设备，或叫外围设备，简称外设。计算机系统没有输入输出设备，就如计算机系统没有软件一样，是毫无意义的。

随着计算机技术的发展，外部设备在计算机系统中的地位越来越重要，它在整个系统中所占的价格比也越来越大。早期的计算机系统主机结构简单、速度慢、应用范围窄，配置的外部设备种类有限，数量不多，外设价格仅占整个系统价格的几个百分点。现代的计算机系统外部设备向多样化、智能化方向发展，品种繁多，性能良好，其价格往往已占到系统总价的 80% 左右。

外部设备的组成通常可用图 5.13 中的虚线框内的结构来描述。

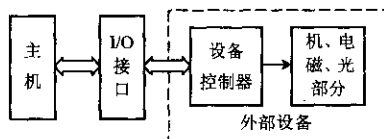


图 5.13 外部设备的结构框图

图中的设备控制器是用来控制设备具体动作的，不同的设备完成的控制功能也不同。机、电、光、磁部分与具体的设备有关，也即设备的具体结构大致

与机、电、光、磁的工作原理有关。本节主要介绍有关设备控制器的内容，要求读者能理解 I/O 设备的工作原理。现代的 I/O 设备，一般还通过接口与主机联系，至于接口的详细内容将在 5.3 节中讲述。

外部设备大致可分为三类：

(1) 人机交互设备

它是用来实现操作者与计算机之间互相交流信息的设备。它能将人体五官可识别的信息媒体转换成机器可识别的信息。如键盘、鼠标、手写板、图形扫描仪、摄像机、语言识别器等等。反之，另一类是将计算机的处理结果信息转换为人的可识别的信息媒体，如打印机、显示器、绘图仪、语音合成器等等。

(2) 计算机信息的驻留设备

它是作为大批信息的驻留设备。例如系统软件和各种计算机的有用信息，其信息量极大，需存储保留起来。这类设备多数可作为计算机系统的辅助存储器，如磁盘、光盘、磁带等等。

(3) 机—机通信设备

它是用来实现一台计算机与其他计算机或与别的系统之间完成通信任务的设备。例如，两台计算机之间可利用电话线进行通信，它们可以通过调制解调器(MODEM)完成。用计算机对各种工业控制实行即时操作，可通过 D/A、A/D 转换设备来完成。计算机与计算机及其他系统还可通过各种设备实现远距离的信息交换。

表 5.1 列出了现代常用的 I/O 设备名称及用途。

表 5.1 常用的 I/O 设备

输入设备	输入设备	键盘 图形输入设备（鼠标器、图形板、跟踪球、操纵杆、光笔） 图像输入设备（摄像机、扫描仪、传真机） 条形码 光学字符识别 语言与文字输入
	输出设备	显示器（字符、汉字、图形、图像） 打印设备（点阵式打印机、激光打印机、喷墨打印机） 绘图仪（平板式、滚筒式） 语音输出
	设备	终端设备（键盘+显示器） 汉字处理设备 A/D、D/A 转换 多媒体 脱机输入/输出设备（软磁盘数据站）

本节主要介绍人机交互设备，它可分为输入设备和输出设备两种，并且有

的设备既具有输入功能又具有输出功能。关于驻留设备已在第四章介绍，有关机—机通信设备将在计算机网络课程中讲述。

5.2.2 输入设备

输入设备完成输入程序、数据和操作命令等功能。当实现人工输入时，往往与显示器联用，以便检查和修正输入时的错误。也可以利用软盘、磁带等脱机录入的介质进行输入。预计到 21 世纪可以实现语音直接输入。

1. 键盘

键盘是应用最普遍的输入设备，它可通过键盘上的各个键，按某种规范向主机输入各种信息，如汉字、外文、数字等。

键盘由一组排列成阵列形式的按键开关组成，如图 5.14 所示。键盘上的按键分字符键和控制功能键两类。字符键包括字母、数字和一些特殊符号键；控制功能键是产生控制字符的键（由软件系统定义功能），还有控制光标移动的光标控制键，用于插入或删除字符的编辑键等。

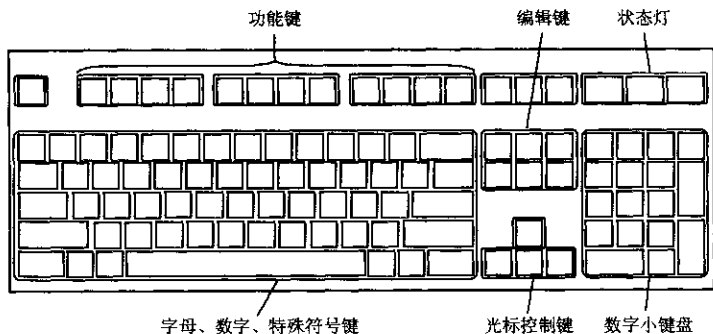


图 5.14 计算机键盘示意图

键盘输入信息分三个步骤：

- 按下一个键；
- 查出按下的是哪个键；
- 将此键翻译成 ASCII 码（见附录 5A.1），由计算机接收。

按键是由人工操作的，确认按下的是哪一个键，可由硬件或软件的办法来实现。

采用硬件确认哪个键被按下的方法叫作编码键盘法，它由硬件电路形成对应被按键的唯一的编码信息。图 5.15 示意了带只读存储器的编码键盘原理。

图中示意了 8×8 的键盘，由一个六位计数器经两个八选一的译码器对键

盘扫描，若键未按下，则扫描将随着计数器的循环计数而反复进行。一旦扫描发现某键被按下，则键盘通过一个单稳电路产生一个脉冲信号。该信号一方面使计数器停止计数，用以终止扫描，此刻计数器的值便与所按键的位置相对应，该值可作为只读存储器 ROM 的输入地址，而该地址中的内容即为所按键的 ASCII 码。可见只读存储器存储的信息便是对应各个键的 ASCII 码。另一方面此脉冲经中断请求触发器向 CPU 发中断请求，CPU 响应请求后便转入中断服务程序，在中断服务程序的执行过程中，CPU 通过执行读入指令，将计数器所对应的 ROM 地址中的内容，即所按键对应的 ASCII 码送入 CPU 中。CPU 的读入指令既可用作读出 ROM 内容的片选信号，而且经一段延迟后，又可用作清除中断请求触发器，并重新启动六位计数器开始新的扫描。

采用软件判断键是否按下的方法叫作非编码键盘法，它是利用简单的硬件和一套专用键盘编码程序来判断按键的位置，然后由 CPU 将位置码经查表程序转换成相应的编码信息。这种方法结构简单，但速度比较慢。

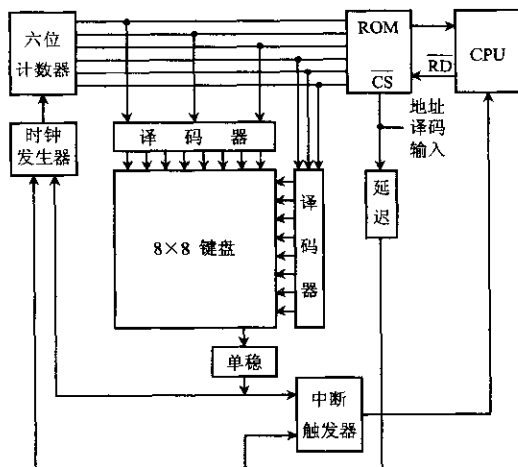


图 5.15 带只读存储器的编码键盘原理图

在按键时往往会出现键的机械抖动，容易造成误动。为了防止形成误判，在键盘控制电路中专门设有硬件消抖电路，或采取软件技术，便可有效地消除因键的抖动而出现的错误。

此外，为了提高传输的可靠性，可采用奇偶校验码（见附录 5A.3）。

随着大规模集成电路技术的发展,厂商已提供了许多种可编程键盘接口芯片,如 Intel 8279 就是可编程键盘/显示接口芯片,用户可以随意选择。近年来又出现了智能键盘,如 IBM PC 机的键盘内装有 Intel 8048 单片机,用它可完成键盘扫描、键盘监测、消除重键、自动重发、扫描码的缓冲以及与主机之间的通信等任务。

2. 鼠标器

鼠标器(Mouse)是一种手持式的坐标定位部件,由于它拖着一根长线与接口相连,外形有点像老鼠,故取名为鼠标器。鼠标器有两种:一种是机械式的,它的底座装有一个金属球,球在光滑表面上摩擦使球转动,球与四个方向的电位器接触,可测得上下左右四个方向的相对位移量,通过显示器便可确定欲寻求的方位。另一种是光电式鼠标器,它需与一块画满小方格的长方形金属板配合使用。安装在鼠标器底部的光电转换器可以确定坐标点的位置,同样由显示器显示所寻找的方位。光电式鼠标器比机械式鼠标器可靠性高,但需增加一块金属板。机械式鼠标器可以直接在光滑的桌面上摩擦,但往往因桌面上的灰尘随金属球滚动带入鼠标器内,致使金属球转动不灵。

3. 触摸屏

触摸屏是一种对物体的接触或靠近能产生反映的定位设备。按触摸原理的不同,大致可分为五类:电阻式、电容式、表面超声波式、扫描红外线式和压感式。

电阻式触摸屏是由显示屏上加一个两层高透明度的、并涂有导电物质的薄膜组成。在两层薄膜之间绝缘隔开,其间隙为 0.0001 英寸,如图 5.16 所示。

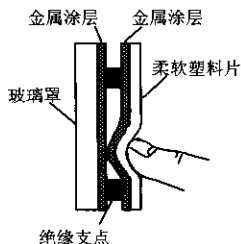


图 5.16 电阻式触摸屏原理

当用户触摸塑料薄膜片时,涂有金属导电物质的第一层塑料片与挨着玻璃罩上的第二层塑料片(也涂有金属导电物)接触,这样根据其接触电阻的大小求得触摸点所在的 x 和 y 坐标位置。

电容式触摸屏是在显示屏上加一个内部涂有金属层的玻璃罩。当用户触摸此罩表面时,与电场建立了电容耦合,在触摸点产生小电流到屏幕四个角,由四个电流大小计算出触摸点的位置。

表面超声波式触摸屏是由一个透明的玻璃罩组成。在罩的 x 和 y 轴方向都有一个发射和接收压电转换器和一组反射器条,触摸屏还有一个控制器发送 5MHz 的触发信号,给发射、接收转换器,让它转换成表面超声波,此超声波在屏幕表面传播。当用手指触摸屏幕时,在确定的位置上超声波被吸收,使接

收信号发生变化,经控制分析和数字转换为 x 和 y 的坐标值。

可见,任何一种触摸屏都是通过某种物理现象来测得人手触及屏幕上各点的位置,从而通过 CPU 对此作出反应,由显示屏再现你所需的位置。由于物理原理不同,体现出各类触摸屏的不同特点及其应用的合适环境。如电阻式能防尘、防潮,并可带手套触摸,适用于饭店、医院等。电容式触摸屏亮度高,清晰度好,也能防尘、防潮,但不可带手套触摸,并且易受温度、湿度变化的影响,因此,它适合于游戏机及供公共信息查询系统使用。表面超声波触摸屏透明、坚固、稳定、不受温度、湿度变化的影响,是一种抗恶劣环境的设备。

4. 其他输入设备

在此主要介绍图形、图像的输入设备,有关语音和文字的输入设备,不作介绍。

(1) 光笔

光笔(Light pen)的外形与钢笔相似,头部装有一个透镜系统,能把进入的光会聚成一个光点。光笔的后端用导线连到计算机输入电路上,光笔头部附有开关,当按下开关时,进行光检测,光笔便可拾取显示屏上的绝对坐标。光笔与屏幕的光标配合,可使光标跟踪光笔移动,在屏幕上画出图形或修改图形,类似人们用钢笔画图的过程。

(2) 画笔与图形板

画笔(Stylus)为笔状,但不是光笔。它不是用于 CRT 屏幕,而是用于图形板(Tablet)。当画笔接触到图形板上的某一位置时,画笔在图形板上的位置坐标就会自动传送到计算机中,随着画笔在板上的移动可以画出图形。图形板和画笔构成二维坐标的输入设备,主要用于输入工程图等。将图纸贴在图形板上,画笔沿着图纸上的图形移动,即可输入工程图。

图形板是一种二维的 A/D 变换器,又称作数字化板。坐标的测量方法有电阻式、电容式、电磁感应式和超声波式几种。

画笔与光笔都是输入绝对坐标,而鼠标器只能输入相对坐标。

(3) 图像输入设备

最直接的图像输入设备是摄像机(Camera),它能摄取任何地点、任何环境下的自然景物和各类物体,经数字量化后变成数字图像存入磁带或磁盘。

如果图像已记录在某种介质上,则可用读出装置来读出图像。例如记录在录像带上的图像可用录放机读出,再将视频信号经图像板量化输入到计算机中。记录在数字磁带上的遥感图像,可直接从磁带输入到计算机中。如果把纸上的图像输入到计算机内,则可用摄像机直接摄入,或用装有 CCD(电荷耦合器件)的图文扫描仪(Scanner)或图文传真机送入计算机,还有一种专用的光机扫描鼓,也可把纸上的图像直接转换成数字图像存入计算机。

5.2.3 输出设备

1. 显示设备

(1) 概述

以可见光的形式传递和处理信息的设备叫显示设备。它是应用最广的人-机通信设备。显示设备种类繁多,按显示器件划分,有阴极射线管(Cathode Ray Tube 简称 CRT)显示器、液晶显示器(Liquid Crystal Display 简称 LCD)、等离子显示器(PD)等等;按显示内容分有字符显示器、图形显示器和图像显示器;按显示器功能分有普通显示器和显示终端(终端是由显示器和键盘组成的一套独立完整的输入/输出设备,它可以通过标准接口接到远离主机的地方。终端的结构比显示器复杂得多)两类。在 CRT 显示器中,按扫描方式不同,可分为光栅扫描和随机扫描两种;按分辨率不同又可分为高分辨率和低分辨率的显示器。

CRT 是目前应用最广泛的显示器件,它既可作为字符显示器,又可作为图像、图形显示器。CRT 是一个漏斗形的电真空器件,它由电子枪、荧光屏及偏转装置组成,如图 5.17 所示。

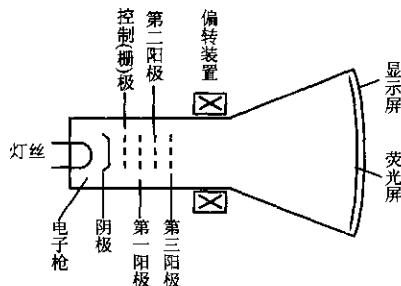


图 5.17 CRT 结构示意图

电子枪包括灯丝、阴极、控制(栅)极、第一阳极(加速阳极)、第二阳极(聚焦极)和第三阳极。当灯丝加热后,阴极受热而发射电子,电子的发射量和发射速度受控制极控制。电子经加速、聚焦而形成电子束,在第三阳极形成的均匀空间电位作用下,使电子束高速射到荧光屏上,荧光屏上的荧光粉受电子束的轰击产生亮点,其亮度取决于电子束的轰击速度、电子束电流强度和荧光粉的发光效率。电子束在偏转系统控制下,在荧光屏的不同位置产生光点,

由这些光点可以组成各种所需的字符、图形和图像。

彩色 CRT 与单色 CRT 其原理是相似的，只是对彩色 CRT 而言，通常用三个电子枪发射的电子束，经定色机构，分别触发红、绿、蓝三种颜色的荧光粉发光，按三基色迭加原理形成彩色图像。

CRT 荧光屏尺寸大小是按屏幕对角线长度表示，普通字符显示器的 CRT 有 12 英寸和 14 英寸两种，图形、图像显示器的 CRT 有 16 英寸和 19 英寸，目前还出现了 21 英寸大屏幕 CRT。

分辨率和灰度等级是 CRT 的两个重要技术指标。分辨率是指显示屏面能表示的像素点数，分辨率越高，图像越清晰。灰度等级是指显示像素点相对于亮暗的级差，在彩色显示器中它还表现为色彩的差别。

CRT 荧光屏发光是由电子束轰击荧光粉产生的，其发光亮度一般只能维持几十毫秒。为了使人眼能看到稳定的图像，电子束必须在图像变化前不断地进行整个屏幕的重复扫描，这个过程叫做刷新。每秒刷新的次数叫做刷新频率，一般刷新频率大于 30 次/秒时，人眼就不会感到闪烁。在显示设备中，通常都采用电视标准，每秒刷新 50 帧 (Frame) 图像。

为了不断地刷新，必须把瞬时图像保存在存储图像的存储器中，这种存储器叫做刷新存储器，又叫做“帧存储器”或“视频存储器”(VRAM)。刷新存储器的容量由图像分辨率和灰度等级决定。分辨率越高，灰度等级越多，刷新存储器容量就越大。例如分辨率为 512×512 ，灰度等级为 256 的图像，其刷新存储器的容量需达 $512 \times 512 \times 8\text{bits}$ ，即为 256KB。此外，刷新存储器的存取周期必须与刷新频率相匹配。

计算机的显示器大多采用光栅扫描方式。所谓光栅扫描是指电子束在荧光屏上按某种轨迹运动，光栅扫描是从上至下顺序扫描，可分为逐行扫描和隔行扫描两种。一般 CRT 都采用与电视相同的隔行扫描，即把一帧图像分为奇数场 (由 1、3、5 等奇数行组成) 和偶数场 (由 0、2、4、6 等偶数行组成)，一帧图像需扫描 625 行，则奇数场和偶数场各扫描 312.5 行。扫描顺序是先扫描偶数场，再扫描奇数场，交替进行，每秒显示 50 场。

(2) 字符显示器

字符显示器是计算机系统中最基本的输出设备，它通常由显示控制器和显示器 (CRT) 组成，图 5.18 示意了它的原理框图。

① 显示存储器 (刷新存储器) RAM

显示存储器存放欲显示字符的 ASCII 码，其容量与显示屏能显示的字符个数有关。如显示屏上能显示 80 列 $\times$ 25 行 = 2 000 个字符，则显示存储器 RAM 的容量应为 $2\,000 \times 8$ (字符编码 7 位，闪烁 1 位)，每个字符所在存储单元的地址与字符在荧光屏上的位置一一对应，即显示存储器单元的地址顺序与屏面

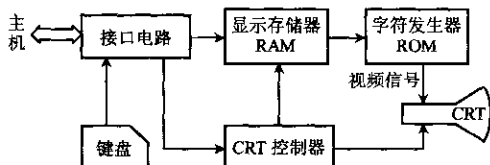
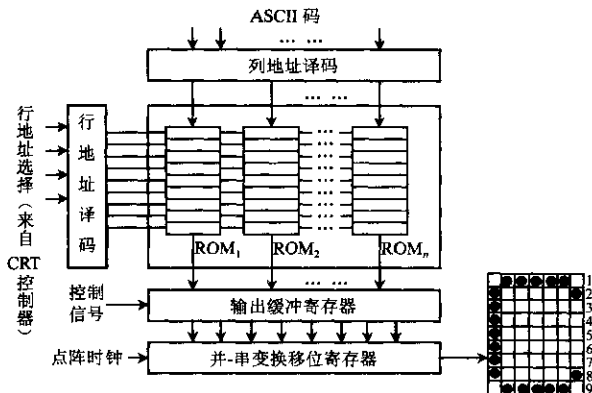


图 5.18 字符显示器原理框图

上每行从左到右，按行从上到下的显示器位置对应。

② 字符发生器

由于荧光屏上的字符是由光点组成，而显示 RAM 中存放的是 ASCII 码，因此，必须有一个部件能将每个 ASCII 字符码转变为一组 5×7 或 7×9 的光点矩阵信息。具有这种变换功能的部件叫做字符发生器，它实质是一个 ROM。图 5.19 是一个对应 7×9 光点矩阵的字符发生器原理框图。

图 5.19 对应 7×9 光点矩阵的字符发生器原理图

图中 ROM_i 的个数与显示器所能显示的字符种类有关，例如能显示 97 个字符，则 $i=1 \sim 97$ 。每个 ROM_i 共有 9 个单元（对应 9 行），每个单元中存放 7 位光点代码。如“C”的 9 个单元中，所存储的 9 组光点代码分别为 0111110、1000001、1000000、1000000、1000000、1000000、1000000、1000001、0111110（设“1”对应亮点，“0”对应暗点）。字符发生器工作时，由显示 RAM 输出的 ASCII 码作为 ROM 的高位地址（列地址），而 ROM 的低位地址（行地址）来自 CRT 控制器的光栅地址计数器。ROM 的输出并行加载到移位寄存器中，

然后在点阵时钟控制下, 移位输出形成视频信号, 作为 CRT 的亮度控制信号。显示器在水平同步、垂直同步 (来自 CRT 控制器) 和视频信号 (来自字符发生器) 的共同作用下, 连续不断地进行屏幕刷新, 就能显示稳定而不消失的字符图像。

③ CRT 控制器

CRT 控制器通常都做成专用芯片, 它可接收来自 CPU 的数据和控制信号, 并给出访问显示 RAM 的地址和访问字符发生器的光栅地址, 还能给出 CRT 所需的水平同步和垂直同步信号等。该芯片的定时控制电路要对显示每个字符的点 (光点) 数、每排 (字符行) 字 (7×9 点阵) 数、每排行 (光栅行) 数和每场排数计数。因此, 芯片中需配置点计数器、字计数器 (水平地址计数器)、行计数器 (光栅地址计数器) 和排计数器 (垂直地址计数器), 这些计数器用来控制显示器的逐点、逐行、逐排、逐幕的刷新显示, 还可以控制对显示 RAM 的访问和屏幕间扫描的同步。

点计数器记录每个字的横向光点, 因每个字符占 7 个光点, 字符间留一个光点作间隙, 共占 8 个光点, 故点计数器为模 8 计数器, 计满 8 个点向字计数器进位。字计数器用来记录屏幕上每排的字数, 若每排能显示 80 个字, 考虑到屏幕两边失真较大, 各空出 5 个字符位置, 再加上光栅回扫消隐时间 (此段时间屏幕不显示) 的需要, 占 20 个显示字符的时间, 总共 $80+10+20=110$, 则字计数器计满 110 就归零, 并向行计数器进位。行计数器用来记录每个字 (7×9 点阵) 的 9 行光栅地址, 外加每排字的 3 行间隔, 共为 $9+3=12$ 的行计数值, 即行计数器计满 12 归零, 并向排计数器进位。排计数器用来记录每屏字符的排数, 若能显示 25 排, 再考虑到屏幕上下失真空一排, 则共 26 排, 即排计数器计满 26 归零, 表示一场扫描结束。

字计数器反映了光栅扫描的水平方向, 排计数器反映了光栅扫描的垂直方向, 将这两个方向的同步信号输至 CRT 的 x 和 y 偏转线圈, 便可达到按指定位置进行显示的要求。

值得注意的是, CRT 的扫描方式不是一个字符一个字符的扫描, 而是每次对一排字符中所有字符的同一行进行扫描, 并显示亮点。例如某排字符为 WELCOME, 其显示次序是: 先从显示 RAM 中读出 “W” 字符, 送至字符发生器, 并从字符发生器中扫描选出 “W” 字符的第一行光点代码, 于是屏幕上显示出 “W” 字符第一行的七个光点代码; 再从显示 RAM 中读出 “E” 字符并送字符发生器, 又选出 “E” 字符的第一行七个光点代码……直到最后一个字符 “E” 的第一行七个光点代码显示完了。接着再进行每个字符点阵的第二行七个光点代码的扫描……直到该排每个字符的第九行光点代码扫描完毕, 则屏幕上完整地显示出 WELCOME 字符。

(3) 图形显示器

图形显示器是用点、线（直线和曲线）、面（平面和曲面）组合而成的平面或立体图形。并可作平移、比例变化、旋转、坐标变换、投影变换（把三维图形变为二维图形）、透视变换（由一个三维空间向另一个三维空间变换）、透视投影（把透视变换和投影变换结合在一起）、轴侧投影（三面图）、单点透视、两点或三点透视以及隐线处理（观察物体时把看不见的部分去掉）等操作。主要用于 CAD（计算机辅助设计）和 CAM（计算机辅助制造），如汽车、飞机、舰船、土建以及大规模集成电路板等的设计制造。

图形显示器经常配有键盘、光笔、鼠标器、CRT 显示器及绘图仪等。

利用 CRT 显示器产生图形有两种方法：一种是随机扫描法，另一种是光栅扫描法。

随机扫描法在随机扫描时，电子束产生图形的过程和用笔在纸上画图的过程相似，任何图形的线条都被认为是许多微小的首尾相接的线段来逼近的，这些微小的线段称为矢量，故这种方法又叫做矢量法。与此法相对应的显示器叫随机扫描图形显示器，其缺点是在显示复杂图形时，会出现闪烁现象。

与光栅扫描法对应的显示器叫做光栅扫描图形显示器。其特点是把对应于屏幕上的每个像素信息都存储在刷新存储器中。光栅扫描时，读出这些像素来调制 CRT 的灰度，以便控制屏幕上像素的亮度。同样也需不断地对屏幕进行刷新，使图形稳定显示。图 5.20 示意了光栅扫描图形显示器的硬件结构框图。

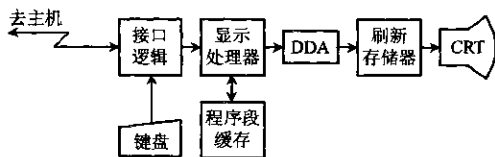


图 5.20 光栅扫描图形显示器的硬件结构框图

图中的程序段缓存用来存储计算机送来的显示文件和图形操作命令，如图形的局部放大、平移、旋转、比例变换以及图形的检索等。这些操作直接由显示处理器完成。刷新存储器存放一帧图形的形状信息，它与屏幕上的像素一一对应。例如屏幕的分辨率为 1024×1024 个像素，且像素的灰度为 256 级，则刷新存储器就有 1024×1024 个单元，每个单元的字长为 8 位。可见刷新存储器的容量与分辨率、灰度都有关。

图中的 DDA(Digital Difference Analyses)是数字差分分析器，它能将显示文件变换成图形形状，它是一种完成数据插补的部件，能够根据显示文件给出

的曲线类型和坐标值，生成直线、圆、抛物线甚至更复杂的曲线。插补后的数据存入刷新存储器用于显示。此外，对于数字化的图像数据也可直接输入刷新存储器，不经 DDA 等图形控制部分便可用来显示图像。

光栅扫描显示器的通用性强，灰度层次多，色调丰富，显示复杂图形时无闪烁，所形成的图形可以有消除隐藏面、阴影效应和涂色等功能。

(4) 图像显示器

图像显示器所显示的图形是由计算机用一定的算法形成的点、线、面、阴影等，它来自主观世界，故又称主观图像或叫做计算机图像。

图像显示器所显示的图像（如遥感图像、医学图像、自然景物、新闻照片等）通常来自客观世界，故又称为客观图像。图像显示器是把由计算机处理后的图像（称为数字图像），以点阵列的形式显示出来。通常采用光栅扫描方式，其分辨率在 256×256 个或 512×512 个像素，也可与图形显示器兼容，其分辨率可达 1024×1024 ，灰度等级可达 64 至 256 级。

图像显示器除了能存储从计算机输入的图像并在显示屏幕上显示外，还具有灰度变换、窗口技术、真彩色和伪彩色显示等图像增强技术功能。

- 灰度变换：可使原始图像的对比度增强或改变。
- 窗口技术：在图像存储器中，每个像素有 2048 级灰度值，而人的肉眼只能分辨到 40 级。如果从 2048 级中开一个小窗口，并把这窗口范围内的灰度级取出，使之变换为 64 级显示灰度，就可以使原来被掩盖的灰度细节充分显示出来。
- 真彩色和伪彩色：真彩色是指真实图像色彩显示，是属色还原技术，如彩色电视；伪彩色处理是一种图像增强技术。通常肉眼能分辨黑白色只有几十级灰度，但却能分辨出上千种色彩。利用伪彩色技术可以人为地对黑白图像进行染色，如把水的灰度染成蓝色，把植物的灰度染成绿色，把土地的灰度染成黄色等等。

此外，图像显示器还具有几何处理功能，如图像放大（按 2、4、8 倍放大）、图像分割或重叠、图像滚动等。

图 5.21 示意了一种简单的图像显示器原理框图。

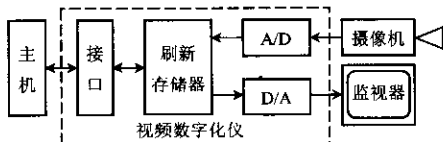


图 5.21 简单的图像显示器原理框图

简单的图像显示器只显示由计算机送来的数字图像，图像处理操作在主机中完成，显示器不做任何处理。其中 I/O 接口、刷新存储器、A/D、D/A 转换等组成单独的一部分，称作视频数字化仪（Video Digitizer）或叫图像输入控制板（简称图像板）。其功能是实现连续的视频信号与离散的数字量之间的转换。视频数字化仪接收摄像机的视频输入信号，经 A/D 变换为数字量存入刷新存储器用于显示，并可传送到主机进行图像处理操作。操作后的结果送回刷新存储器，又经 D/A 变为视频信号输出，由监视器显示。监视器只包括扫描、视频放大等有关的显示电路和显像管。也可接至电视机的视频输入端，用电视机代替监视器。一般通用计算机配置一块图像板和监视器便能组成一个图像处理系统。

（5）IBM PC 系列微型机的显示标准

IBM PC 系列微型机配套的显示系统有两大类。一类是基本显示系统，用于字符/图形显示；另一类是专用显示系统，用于高分辨率图形或图像显示。这里仅介绍几种显示标准。

① MDA (Monochrome Display Adapter) 标准

MDA 是单色字符显示标准，采用 9×14 点阵的字符窗口，满屏显示 80 列、25 行字符，对应分辨率为 720×350 像素。MDA 不能兼容图形显示。

② CGA (Color Graphics Adapter) 标准

CGA 是彩色图形/字符显示标准，它可兼容字符和图形两种显示方式。在字符方式下，字符窗口为 8×8 点阵，故字符质量不如 MDA，但字符的背景可以选择颜色。在图形方式下，可以显示 640×200 两种颜色或 320×200 四种颜色的图形。

③ EGA (Enhanced Graphics Adapter) 标准

EGA 标准集中了 MDA 和 CGA 两个显示标准的优点，并有所增强。其字符窗口为 8×14 点阵，字符显示质量优于 CGA 而接近 MDA。图形方式下分辨率为 640×350 ，有 16 种颜色，彩色图形的质量优于 CGA，且兼容原 CGA 和 MDA 的各种显示方式。

④ VGA (Video Graphics Array) 标准

VGA 标准在字符方式下，字符窗口为 9×16 点阵，在图形方式下分辨率为 640×480 、16 种颜色，或 320×200 、256 种颜色。改进型 VGA 显示控制板（如 VGA+、Super VGA 或 TVGA）的图形分辨率可达 800×600 、 960×720 和 1024×768 、256 种颜色。

习惯上将 MDA、CGA 称作 IBM PC 机的第一代显示标准，EGA 为第二代标准，VGA 为第三代标准。

2. 打印设备

打印设备可将计算机运行结果打印输出记录在纸上,并能长期保存,是一种硬拷贝设备。相比之下,显示器在屏幕上的信息是无法长期保存的,故它不属于硬拷贝设备。

(1) 打印设备的分类

打印设备的种类有很多种划分方法。

按印字原理划分,有击打式和非击打式两大类。击打式打印机是利用机械动作使印字机构与色带和纸相撞击而打印字符,其特点是设备成本低,印字质量较好,但噪音大,速度慢。它又分为活字打印机和点阵针式打印机两种。活字打印机是将字符刻在印字机构的表面上,印字机构的形状有圆柱形、球形、菊花瓣形、鼓轮形、链形等,现在用得越来越少。点阵打印机的字符是点阵结构,它利用钢针撞击的原理印字,目前仍用得较普遍。非击打式打印机是采用电、磁、光、喷墨等物理、化学方法来印刷字符。如激光打印机、静电打印机、喷墨打印机等,它们速度快,噪音低,印字质量比击打式好,但价格比较贵,有的设备需用专用纸张印刷。

按工作方式分,有串行打印机和行式打印机两种。前者是逐字打印,后者是逐行打印,故行式打印机比串行打印机速度快。

此外,按打印纸的宽度还可分宽行打印机和窄行打印机,还有能输出图的图形/图像打印机,具有色彩效果好的彩色打印机等等。

(2) 点阵针式打印机

点阵针式打印机结构简单、体积小、重量轻、价格低、字符种类不受限制、较易实现汉字打印,还可打印图形和图像,是目前应用最广泛的一种打印设备。一般在微型、小型计算机中都配有这类打印机。

点阵针式打印机的印字原理是由打印针(钢针)印出 $n \times m$ 个点阵来组成字符或图形。点越多越密,其字形质量越高。西文字符点阵通常采用 5×7 、 7×7 、 7×9 、 9×9 几种,汉字的点阵采用 16×16 、 24×24 、 32×32 和 48×48 多种。图 5.22 是 7×9 点阵字符的打印格式和打印头的示意。

打印头中的钢针数与打印机型号有关,有 7 针、9 针,也有双列 14 (2×7) 针或双列 24 (2×12) 针。打印头固定在托架上,托架可横向移动。图中为 7 根钢针,对应垂直方向的 7 点,由于受机械安装的限制,这 7 点之间有一定的间隙。水平方向各点的距离取决于打印头移动的位置,故可密集些,这对形成斜形或弧形笔划非常有利。字符的形成是按字符中各列所包含的点逐列形成的。如字符 E,先打印第 2 列的 1~7 个点,再打印第 4、6、8 列的第 1、4、7 三点,最后打第 10 列的 1、7 两个点。可见每根针可以单独驱动。打印一个字符后,空出 3 列(第 11、0、1 列)作为间隙。

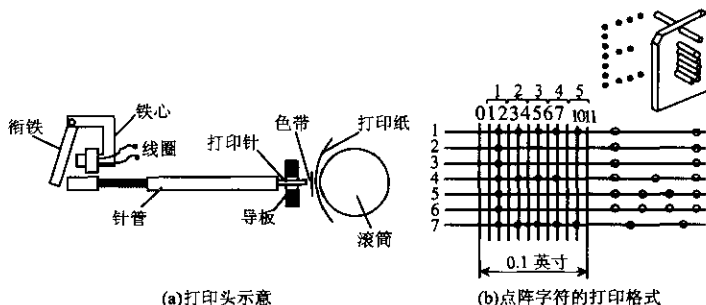
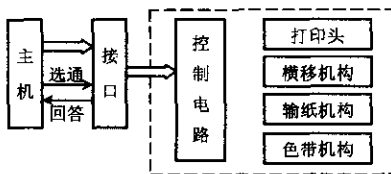
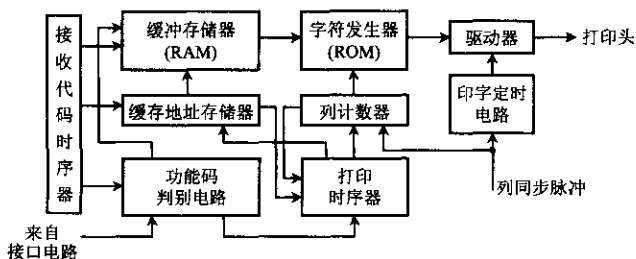


图 5.22 针式打印头和打印格式示意

针式打印机由打印头、横移机构、输纸机构、色带机构和相应的控制电路组成，如图 5.23 所示。其中 (b) 为 (a) 的控制电路细化图。



(a) 针式打印机结构框图



(b) 针式打印机控制电路框图

图 5.23 针式打印机的组成

打印机被 CPU 启动后，在接收代码时序器控制下，功能码判别电路开始

接收从主机送来的欲打印字符的字符代码 (ASCII 码)。首先判断该字符是打印字符码还是控制功能码 (如回车、换行、换页等), 若是打印字符码, 则送至缓冲存储器, 直到把缓存 RAM 装满为止; 若是控制功能码, 则打印控制器停止接收代码并转入打印状态。打印时首先启动打印时序器, 并在它控制下, 从缓存中逐个读出打印字符码, 再以该字符码作为字符发生器 ROM 的地址码, 从中选出对应的字符点阵信息 (字符发生器可将 ASCII 码转换成打印字符的点阵信息)。然后在列同步脉冲计数器控制下, 将一列列读出的字符点阵信息送至打印驱动电路, 驱动电磁铁带动相应的钢针进行打印。每打印一列, 固定钢针的托架就要横移一列距离, 直到打印完最后一列, 形成 $n \times m$ 点阵字符。当一行字符打印结束或换行打印、或缓存内容已全部打印完毕时, 托架就返回到起始位置, 并向主机报告, 请求打印新的数据。

图 5.23 (a) 中的输纸机构受步进电机驱动, 每打印完一行字符, 按给定要求走纸。色带的作用是供给色源, 如同复写纸的作用一样。图 5.22 (a) 所示, 钢针撞击在色带上, 就可将色印在纸上, 色带机构可使色带不断移动, 以改变受击打的位置, 避免色带的破损。

有的点阵针式打印机内部配有一个独立的微处理器, 用来产生各种控制信号, 完成复杂的打印任务。

上面介绍的针式打印机是串行点阵针式打印机, 打印速度每秒 100 个字符左右, 它被微型计算机系统中广泛采用。在中、大型通用计算机系统中, 为提高打印速度, 还配备行式点阵打印机, 它是将多根打印针沿横向排成一行, 安装在一块形似梳齿状的梳形板上, 每根针各由一个电磁铁驱动。打印时梳形板可向左右移动, 每移动一次印出一行印点。当梳形板改变移动方向时, 走纸机构使纸移动一个印点间距, 如此重复多次即可打印出一行字符。例如 44 针行式打印, 沿水平方向均匀排列 44 根打印针, 每根针负责打印 3 个字符, 打印行宽为 $44 \times 3 = 132$ 列字符。如果每根针负责打印两个字符, 则可采用 66 针结构。

(3) 激光打印机

激光打印机采用了激光技术和照相技术, 由于它的印字质量好, 在各种计算机系统中广泛被采用。图 5.24 示意了激光打印机的工作原理。

激光打印机由激光扫描系统、电子照相系统、字形发生器和接口控制器

几部分组成。接口控制器接收由计算机输出的二进制字符编码及其他控制信号; 字形发生器可将二进制字符编码转换成字符点阵脉冲信号; 激光扫描系统的光源是激光器, 该系统受字符点阵脉冲信号的控制, 能输出很细的激光束, 该激光束对作圆周运动的感光鼓进行轴向 (垂直于纸面) 扫描。感光鼓是电子照相系统的核心部件, 鼓面上涂有一层具有光敏特性的感光材料, 通常用硒,

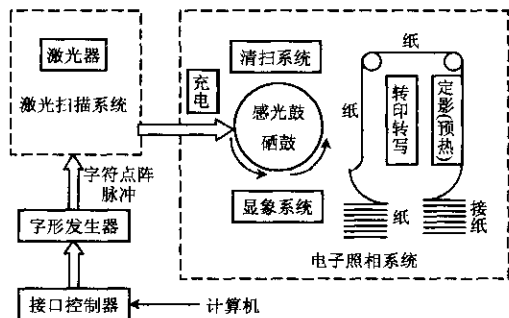


图 5.24 激光打印机原理框图

故又有晒鼓之称。感光鼓在未被激光扫描之前，先在黑暗中充电，使鼓表面均匀地沉积一层电荷，扫描时激光束对鼓表面有选择地曝光，被曝光的部分产生放电现象，未被曝光的部分仍保留充电时的电荷，这就形成了“潜像”。随着鼓的圆周运动，“潜像”部分通过装有碳粉盒的显像系统，使“潜像”部分（实际上是具有字符信息的区域）吸附上碳粉，达到“显影”的目的。当鼓上的字符信息区和打印纸接触时，由于纸的背面施以反向的静电电荷，则鼓面上的碳粉就会被吸附到纸面上，这就是“转印”或“转写”过程。最后经过定影系统就将碳粉永久性地粘在纸上。转印后的鼓面还留有残余的碳粉，故先要除去鼓面上的电荷，经清扫系统将残余碳粉全部清除，然后再重复上述充电、曝光、显影、转印、定影等一系列过程。

激光打印机可以使用普通纸张，输出速度快，一般可达 10 000 行/分（高速的可达 70 000 行/分），印字质量好，普通激光打印机的印字分辨率可达 300DPI（每英寸 300 个点）或 400DPI。字体字形可任意选择，还可打印图形、图像、表格、各种字母、数字和汉字等字符。

激光打印机是非击打式硬拷贝输出设备，是逐页输出的，故又有“页式输出设备”之称。普通击打式打印机是逐字或逐行输出的。页式输出设备的速度以每分钟输出的页数 PPM（Pages Per Minute）来描述。高速激光打印机的速度在 100PPM 以上，中速为 30~60PPM，它们主要用于大型计算机系统。低速激光打印机的速度为 10~20PPM 或 10PPM 以下，主要用于办公室自动化系统和文字编辑系统。

（4）喷墨打印机

喷墨打印机是串行非击打式打印机，印字原理是将墨水喷射到普通打印纸上。若采用红、绿、兰三色喷墨头，便可实现彩色打印。随着喷墨打印技术的

不断提高,使其输出效果接近于激光打印机,而价格又与点阵针式打印机相当,因此,在计算机系统中被广泛应用。

图 5.25 (a) 是一种电荷控制式喷墨打印机的原理框图。主要由喷头、充电电极、墨水供应、过滤回收系统及相应的控制电路组成。

喷墨头后部的压电陶瓷受振荡脉冲激励,使喷墨头喷出具有一定速度的一串不连续、不带电的墨水滴。墨水滴通过充电电极时被充上电荷,其电荷量的大小由字符发生器的输出控制。字符发生器可将字符编码转换成字符点阵信息。由于各点的位置不同,充电电极所加的电压也不同,电压越高,充电电荷越多,墨滴经偏转电极后偏移的距离也越大,最后墨滴落在印字纸上。图中只有一对垂直方向的偏转电极,因此墨滴只能在垂直方向偏移。若垂直线段上某处不需喷点(对应字符在此处无点阵信息),则相应墨滴不充电,在偏转电场中不发生偏转,而射入回收器中。横向没有偏转电极,靠喷头相对于记录纸作横向移动来完成横向偏转。图 5.25 (b) 示意了 H 字符由 5×7 点阵组成。墨滴的运动轨迹如图中所示的数字顺序移动,可见字符中的每个点都要一个个地进行控制,故字符发生器的输出必须是一个点一个点的信息。这与点阵针式打印机的字符发生器一次输出一列的上七个点信息,分 5 次打印一个字符是完全不同的(参见图 5.22)。

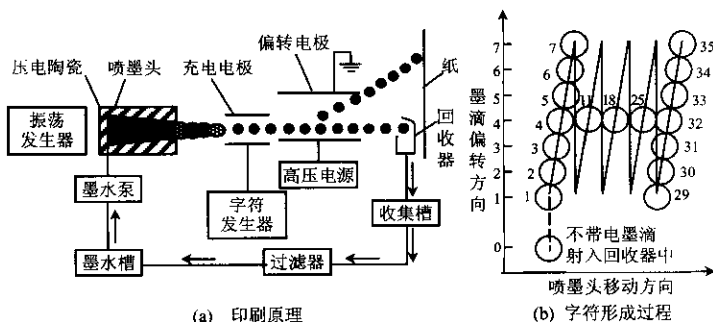


图 5.25 电荷控制式喷墨打印机原理框图

喷墨打印机还有很多种,如电场控制型连续式喷墨打印机、随机式喷墨打印机以及具有多个喷头的喷墨打印机(如日本 EPSON 公司的 TSQ-4800 喷墨打印机有 48 个喷嘴)等,在此不作详述。

(5) 几种打印机的比较

以上介绍的三种打印机都配有一个字符发生器,它们的共同点是都能将字

符编码信息变为点阵信息，不同的是这些点阵信息的控制对象不同。点阵针式打印机的字符点阵用于控制打印针的驱动电路；激光打印机的字符点阵脉冲信号用于控制激光束；喷墨打印机的字符点阵信息控制墨滴的运动轨迹。

此外，点阵针式打印机是属击打式的打印机，可以逐字打印也可以逐行打印，喷墨打印机只能逐字打印，激光打印机属页式输出设备。后两种都属非击打式打印机。

不同种类的打印机其性能和价格差别很大，用户可根据不同需要合理选用。要求印字质量高的场合可选用激光打印机；要求价格便宜的或只需具有文字处理功能的个人用计算机，可配置串行点阵针式打印机；要求处理的信息量很大，速度又要快，应该配行式打印机或高速激光打印机。

5.2.4 其他外部设备

计算机的外部设备中有一类属于既是输入设备，又是输出设备。如磁盘、终端、A/D、D/A 转换器，以及汉字处理设备等等。

1. 终端设备

终端是由显示器和键盘组成的一套独立完整的输入/输出设备，它可以通过标准接口接到远离主机的地方使用。终端与显示器是两个不同的概念，终端的结构比显示器复杂，它能完成显示控制与存储、键盘管理及通信控制等，还可完成简单的编辑操作。

2. A/D、D/A 转换器

当计算机用于过程控制时，其控制信号是模拟量，而计算机仅能处理数字量，这就要用 A/D、D/A 转换器来完成模拟量与数字量之间的相互转换任务。

A/D 转换器是模拟/数字转换器，它能将模拟量转换成数字量，是计算机的输入设备。A/D 转换器均已制成各种规格的芯片。

D/A 转换器是数字/模拟转换器，它能将计算机输出的数字量转换成控制所需的模拟量，以便控制被控对象或直接输出模拟信号，它是计算机的输出设备。D/A 转换器现在也均已制成规格化的各类芯片。

A/D、D/A 转换器均属于过程控制设备，往往还需要配置其他设备，如传感器、放大电路、执行机构以及开关量输入/输出设备等，与计算机共同完成对象的过程控制。

3. 汉字处理设备

计算机进行汉字信息处理时，必须将汉字代码化，即对汉字进行编码。汉字编码可分为输入码、内码和字形码三大类。输入码是解决汉字的输入和识别问题的；内码是由输入码转换而成的，只有内码才能在计算机内进行加工处理；字形码能显示或打印输出。汉字处理设备包括汉字输入、汉字存储和汉字输出

三个部分。

(1) 汉字的输入

采用西文标准键盘输入汉字时, 必须对汉字进行编码, 以便用字母、数字串替代汉字输入。

汉字编码方法主要有三类: 数字编码、拼音编码和字形编码。

- 数字编码就是用数字串代表一个汉字的输入, 常用的是国标区位码, 也有用电报码。使用区位码输入汉字时, 必须根据国标 GB3212《信息交换用汉字编码字符集——基本集》, 先查出汉字对应的代码, 然后才能输入。这种编码输入的优点是无重码, 而且输入码和内码的转换比较方便, 但每个汉字的编码都是一串等长的数字, 很难记忆。
- 拼音码是以汉语读音为基础的, 由于汉字同音字太多, 输入重码率很高, 因此按拼音输入后还必须进行同音字的选择, 影响了输入速度。
- 字形编码是以汉字形状确定的, 由于汉字都是由一笔一划构成的, 而笔划又是有限的, 而且汉字的结构(又称部件)也可以归结为几类, 因此, 把汉字的笔划和部件用字母和数字编码后, 再按笔划书写顺序依次输入, 就能表示出一个汉字。常用的有五笔字型编码。这种编码输入方法其效率目前是最高的。

上述介绍汉字输入方法均为“手动”操作, 主要用键盘敲入。为了提高输入速度, 又发展了词组输入、联想输入等输入方法。随着计算机技术的发展, 利用语音或图像识别技术, 直接将汉语或文本输入至计算机, 使计算机既能识别汉字, 又能听懂汉语, 并将其自动转换成机内代码。近年来有关语音识别、文字识别、自然语言理解及机器视觉等学科的研究都已有了不少好的成果, 读者可查阅有关资料进一步了解。

(2) 汉字的存储

汉字的存储包括汉字内码存储和字形码的存储。

汉字内码是汉字信息在机内存储、交换、检索等过程中所使用的机内代码, 通常用两个字节表示。使用汉字内码字符时, 应注意和英文字符区别开。英文字符的机内代码是七位 ASCII 码, 字节的最高位为“0”, 而汉字内码的两个字节最高位均为“1”。以汉字操作系统 CCDOS 中的汉字内码为例, 汉字国标码“兵”用十六进制表示为“3224H”, 每个字节最高位加“1”后, 便得汉字内码为“B2A4H”(参见附录 6A.1)。当使用编辑程序输入汉字时, 存储到磁盘上的文件就是用机内码表示汉字的。有些机器把字节的最高位用作奇偶校验位, 这时汉字内码需用三个字节表示。

汉字字形码是用点阵表示汉字字形的代码, 也称字模码, 它是汉字的输出形式。简易型的汉字为 16×16 点阵, 高精度的汉字用 24×24 点阵或 32×32

点阵表示。字模点阵的信息量很大，以 16×16 点阵为例，存放一个汉字就要占用 32 个字节。由国标给出的常用汉字 6763 个大约占 256K 字节，因此必须单设字模点阵库来存储每个汉字的点阵代码。当显示输出时，需检索字库，输出字模点阵，最后得到字形。图 5.26 是汉字“次”字形点阵及编码。

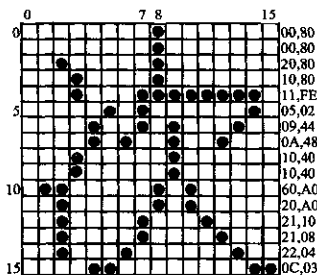


图 5.26 汉字字形点阵及编码

(3) 汉字的输出

汉字输出有打印输出和显示输出两种形式。针式汉字打印机有 24 针和 16 针两种，前者印字质量高。也可采用 9 针的西文打印机，当用 9 针打印机打印汉字时，需用软件控制把一行汉字分成两次打印，即每次打印 8 个点，第一次打印一行汉字的上半部，第二次打印一行汉字的下半部，拼在一起构成 16×16 的点阵汉字。

汉字显示可用通用显示器，在主机内由汉字显示控制板（简称汉卡）或通用的图形显示板形成点阵码，再将点阵码送至显示设备。只要设备具有输出点阵的能力，就可以输出汉字。此外，汉字显示终端除了显示汉字外，还可作为人一机通信设备。

5.2.5 多媒体技术

1. 什么是多媒体

多媒体是“Multimedia”的汉译。而“Multimedia”一词是由“Multi”和“Media”两个词构成的复合词，直译即为“多媒体”。

多媒体一词的核心词是媒体。所谓媒体是指信息传递和存储的最基本的技术和手段。日常生活中最常用的媒体包括音乐、语言、图片、文件、书籍、电视、广播、电话等。人们可以通过媒体获取他（她）们所需的信息，同时也可以利用这些媒体将有用信息传出去或保存起来。

然而到目前为止，提供给人类利用的媒体设施、工具和手段大多是单一功

能的。例如音响设备只能录音或放音；电视只能提供音频和视频信息；报纸只能提供文字和图像图表信息等等。由于都是单功能媒体，而且各自均独立分散，为此人们希望能有一个集多种功能的多媒体系统，这就是应用领域向计算机科学与技术 and 计算机工业提出的迫切要求。

此外，计算机本身的发展也提出了同样要求。回顾一下计算机的发展史，不难发现，计算机与某一信息形式结合便可以开拓一个新的应用领域。在 20 世纪 50 年代计算机局限于处理数字，应用领域也限制在求解复杂的数学问题。到了 60 年代计算机与字符处理、文本处理相结合，就出现了信息管理系统。后来计算机与图形结合，产生了 CAD。计算机与照相相结合，又产生了图像（静）处理等等。80 年代曾是人工智能研究领域的高潮时代，首先是日本提出了以研究具有高度智能的第五代计算机为目标的 FGCS 计划，给世界计算机技术形成了一次冲击，可是经过了十年含辛茹苦地探索，人们才发现研制人工智能第五代计算机的时代远未成熟，只有在计算机科学理论和信息处理技术的高度发展以及知识库体系自我完备的基础上，第五代人工智能计算机的研制才有可能成为现实。人们在认识世界和对某一事物作出判断时，绝不是或不仅仅是用某种单一媒体上的信息或孤立地利用某一时刻的信息。人脑首先是具有高度的信息融合能力，其次是具有历史和环境提供的启示信息，以减少推理搜索空间的能力。目前的计算机还远远不具备人脑的这种能力，因此，人工智能也很难取得突破性的进展。

研究多媒体计算机技术，就是要强调计算机与声音、活动图像和文字相结合。例如将录像内容输入到计算机盘上（如果需要可进行处理），在播放时，可与多种其他媒体信息（如文字、声音）混合在一起，形成一个多媒体的演示系统。又如，将计算机产生的图形或动画与摄像机摄得的图像叠加在一起等等。此外，采用人机对话方式，对计算机存储的各种信息进行查找、编辑以及实现同时播放，使多媒体系统成为一个交互式的系统。可见，多媒体计算机可作为研制高度智能计算机系统的一个平台。

2. 多媒体计算机的关键技术

（1）视频和音频数据的压缩和解压缩技术

多媒体计算机的关键问题是如何实时综合处理声、图和文字信息，需要将每幅图像从模拟量转换成数字量，然后进行图像处理，与图形、文字复合后存放在机器中。数字化图像和声音的信息量是非常大的。以一般彩色电视信号为例，设代表光强、色彩和色饱和度的 YIQ 色空间中各分量的带宽分别为 4.2MHz、1.5MHz 和 0.5MHz。根据采样原理，仅当采样频率 ≥ 2 倍的原始信号的频率时，才能保证采样后信号可被保真地恢复为原始信号。再设各分量均被数字化为 8 个比特，从而 1 秒钟的电视信号的数据总量应为：

$$(4.2+1.5+0.5) \times 2 \times 8 = 99.2 \text{ (Mbits)}$$

也就是说, 彩电信号的数据量约每秒为 100Mbits, 因而一个容量为 1GB 的 CD-ROM 仅能存放约一分钟的原始电视数据(每字节后面附有 2 位校验位), 很显然电视信号数字化后直接保存的方法是令人难以接受的。

对于语音的数据也一样, 一般人类语音的带宽为 4KHz, 同样依据采样定理, 并设数字化精度为 8 比特, 则一秒钟的数据量为: $4\text{K} \times 2 \times 8 = 64\text{Kbits}$, 因此在上述采样条件下, 讲一分钟话的数据量约为 480KB。

由此可见, 电视图像、彩色图像、彩色静图像、文件图像以及语音等数据量是相当大的。特别是电视图像的数据量, 在相同条件下要比语音数据量大 1 000 倍。再加上计算机总线的传输率也跟不上, 因此, 必须对信息进行压缩和解压缩。所谓图像压缩是指图像从像素存储的方式, 经过图像变换、量化和高效编码等处理, 转换成特殊形式的编码, 从而大大降低计算机所需存储和实时传送的数据量。例如 Intel 公司的交互式数字视频系统 DVI 能将动态图像数据压缩到 135KB/S 的传送速度。

信息编码方式很多, 应选用符合国际标准的, 并能用计算机或 VLSI 芯片快速实现的编码方法。

(2) 多媒体专用芯片

由于多媒体计算机承担大量与数据信号处理、图像处理、压缩与解压缩以及解决多媒体之间关系等有关的问题, 而且要求处理速度快, 因此需研制专用芯片。一般多媒体专用芯片有两种类型: 固定功能的和可编程的。

(3) 大容量存储器

多媒体计算机需要存储的信息量极大, 因此研制大容量的存储器仍是多媒体计算机系统的关键技术。

(4) 适用于多媒体技术的软件

图 5.27 示意了多媒体系统的层次结构。

最底层为计算机硬件, 还可配置电视机、录像机及音像设备等。其上层是多媒体实时压缩和解压缩层, 它将视频和音频信号压缩后存储在盘上, 播放时要解压缩, 而且要求处理速度快, 通常采用以专用芯片为基础的电路卡。

多媒体入/出控制及接口层与多媒体设备打交道, 驱动控制这些硬件设备, 并提供与高层软件的接口。

多媒体核心系统层是多媒体操作系统, Intel、IBM、Microsoft 和 Apple 等公司都开发了这层软件。

创作系统层是为方便用户开发应用系统而设置的, 具有编辑和播放等功能。

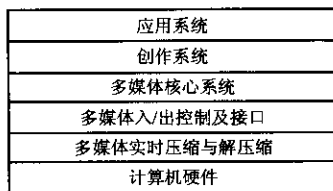


图 5.27 多媒体系统的层次结构

应用系统层包括厂家或用户开发的应用软件。

以上除最底层的硬件层外，其他层次都包含适用于多媒体技术的软件。

5.3 I/O 接口

5.3.1 概述

接口可以看作是两个系统或两个部件之间的交接部分，它既可以是两种硬设备之间的连接电路，也可以是两个软件之间的共同逻辑边界。I/O 接口通常是指主机与外部设备之间设置的一个硬件电路及其相应的软件控制。由图 5.13 可知，不同的设备都有其相应的设备控制器，而它们往往都是通过 I/O 接口与主机取得联系的。主机与外设之间设置接口的理由是：

(1) 一台机器通常配有多台外设，他们各自有其设备号（地址），通过接口可实现设备的选择。

(2) 外部设备种类繁多，速度不一，与 CPU 速度相差可能很大，通过接口可实现数据缓冲达到速度匹配。

(3) 有些外部设备可能串行传送数据，而 CPU 一般为并行传送，通过接口可实现数据串一并格式的转换。

(4) 外部设备的入/出电平可能与 CPU 的入/出电平不同，通过接口可实现电平转换。

(5) CPU 启动外部设备工作，要向外设发各种控制信号，通过接口可传送控制命令。

(6) 外部设备需将其工作状态（如“忙”、“就绪”、“错误”、“中断请求”等）及时向 CPU 报告，通过接口可监视设备的工作状态，并可保存状态信息，供 CPU 查询。

值得注意的是：接口（Interface）和端口（Port）是两个不同的概念。端口是指接口电路中的一些寄存器，这些寄存器分别用来存放数据信息，控制信息和状态信息，相应的就是数据端口、控制端口和状态端口。若干个端口加上

相应的控制逻辑才能组成接口。CPU 通过输入指令，从端口读入信息，通过输出指令，可将信息写入到端口中。

5.3.2 接口的功能和组成

1. 总线连接方式的 I/O 接口电路

图 5.28 示意了总线结构的计算机，每一台设备都是通过 I/O 接口挂到系统总线上的。图中的 I/O 总线包括数据线、设备选择线、命令线和状态线。

(1) 数据线

数据线是 I/O 与主机之间数据代码的传送线，其根数一般等于存储字长的位数或字符的位数，它通常是双向的，也可以是单向的。若采用单向数据总线，则必须用两组才能实现数据的输入和输出两种功能，而双向数据总线只需一组即可。

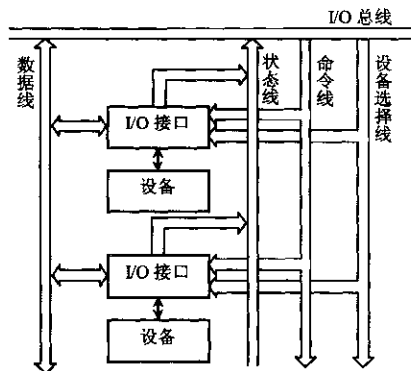


图 5.28 I/O 总线和接口部件

(2) 设备选择线

设备选择线是用来传送设备码的，它的根数取决于 I/O 指令中设备码的位数。如果把设备码看作是地址号，那么设备选择线又可称为地址线。设备选择线可以有一组也可以有两组，其中一组用于主机向 I/O 发送设备码，另一组用于 I/O 向主机回送设备码。当然设备选择线也可采用一组双向总线代替两组单向总线。

(3) 命令线

命令线主要用以传输 CPU 向设备发出的各种命令信号，如启动、清除、屏蔽、读、写等等。它是一组单向总线，其根数与命令信号多少有关。

(4) 状态线

状态线是将 I/O 设备的状态向主机报告的信号线，如设备是否准备就绪，是否向 CPU 发出中断请求等等。它也是一组单向总线。

现代计算机中大多采用三态逻辑电路来构成总线。

2. 接口的功能和组成

根据上述设置接口的理由，可归纳出接口通常应具有以下几个功能以及相应的硬件配置。

(1) 选址功能

由于 I/O 总线与所有设备的接口电路相连，但 CPU 究竟选择哪台 I/O，还得通过设备选择线上的设备码来确定。该设备码将送至所有设备的接口，因此，要求每个接口都必须具有选址功能，即当设备选择线上的设备码与本设备码相符时，应发出设备选中信号 SEL，这种功能可通过接口内的设备选择电路来实现。

图 5.29 示意了接口 1 和接口 2 的设备选择电路。这两个电路具体线路可以不同，它们分别能识别出自身的设备码，一旦某接口设备选择电路有输出时，它便可控制这个设备通过命令线、状态线和数据线与主机交换信息。

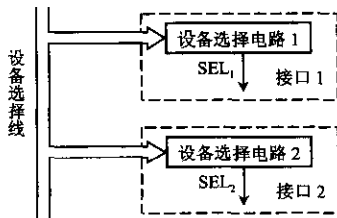


图 5.29 设备选择电路框图

(2) 传送命令的功能

当 CPU 向 I/O 发出命令时，要求 I/O 设备能作出响应，如果 I/O 接口不具备传送命令信息的功能，那么设备将无法响应，故通常在 I/O 接口中设有存放命令的命令寄存器以及命令译码器，如图 5.30 所示。

命令寄存器用来存放 I/O 指令中的命令码，它受设备选中信号控制。命令线和所有接口电路的命令寄存器相连，只有被选中的设备 SEL 信号有效，命令寄存器才可接受命令线上的命令码。

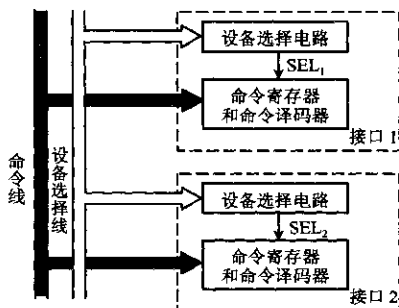


图 5.30 命令寄存器和命令译码器

(3) 传送数据的功能

既然接口处于主机与 I/O 设备之间，因此数据必须通过接口才能实现主机与 I/O 设备之间的传送。这就要求接口中具有数据通路，完成数据传送。这种数据通路还应具有缓冲能力，即将数据能暂存在接口内。接口中通常设有数据缓冲寄存器 DBR (Data Buffer Register)，它用来暂存 I/O 设备与主机准备交换的信息，它与 I/O 总线中的数据线是相连的。

每个接口中的数据缓冲寄存器的位数可以各不相同，它取决于各类 I/O 设备的不同需要。例如键盘接口其 DBR 定为 8 位，因为 ASCII 码为七位再加一位奇偶校验位（见附录 5A.2），故为 8 位。又如磁盘这类外设，其 DBR 的位数通常与存储字长的位数相等，而且还要求具有串—并转换能力，即可将从磁盘中串行读出的信息并行送至主存，又可将从主存中并行读出的信息串行输入至磁盘。

(4) 反映 I/O 设备工作状态的功能

为了使 CPU 能及时了解各 I/O 设备的工作状态，接口内必须设置一些反映设备工作状态的触发器。例如用完成触发器 D 和工作触发器 B 来标志设备所处的状态。

当 D=0, B=0 时，表示 I/O 设备处于暂停状态；

当 D=1, B=0 时，表示 I/O 设备已经准备就绪；

当 D=0, B=1 时，表示 I/O 设备正处于准备状态。

由于现代计算机系统中大多采用中断技术，因此接口电路中一般还设有中断请求触发器 INTR，当其为“1”时，表示该 I/O 设备向 CPU 发出中断请求。接口内还有屏蔽触发器 MASK，它与中断请求触发器配合使用，完成设备的屏蔽功能（有关内容将在 8.4 节讲述）。

所有的状态标记触发器都与 I/O 总线中的状态线相连。此外，不同的 I/O 设备其接口电路中还可根据需要增设一些其他状态标记触发器，如“出错”触发器、“数据迟到”触发器，或配置一些奇偶校验电路，循环码校验电路等等。随着大规模集成电路制作工艺的不断提高，目前大多数 I/O 设备所共用的电路都制作在一个芯片内，作为通用接口芯片。另一些 I/O 设备专用的电路，制作在 I/O 设备的设备控制器中。本节所讲述的接口功能及组成，均是指通用接口所具备的。图 5.31 示意了 I/O 接口的基本组成。

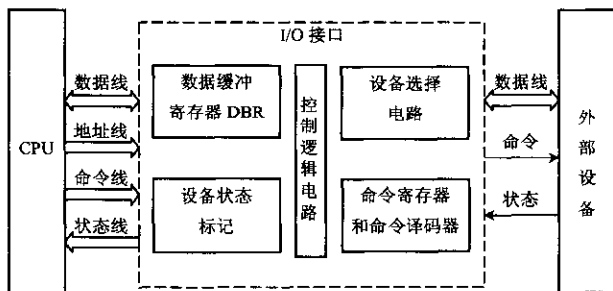


图 5.31 I/O 接口的基本组成

5.3.3 接口类型

I/O 接口按不同方式分类有以下几种。

(1) 按数据传送方式分类，有并行接口和串行接口两类。并行接口是将一个字节（或一个字）的所有位同时传送（如 Intel 8255）；串行接口是在设备与接口间一位一位传送（如 Intel 8251）。由于接口与主机之间是按字节或字并行传送，因此对串行接口而言，其内部还必须设有串—并转换装置。

(2) 按功能选择的灵活性分类，有可编程接口和不可编程接口两种。可编程接口的功能及操作方式，可用程序来改变或选择（如 Intel 8255、8251）；不可编程接口不能由程序来改变其功能，但可通过硬连线逻辑来实现不同的功能（如 Intel 8212）。

(3) 按通用性分类有通用接口和专用接口。通用接口可供多种外设使用，如 Intel 8255、8212；专用接口是为某类外设或某种用途专门设计的，如 Intel 8279 可编程键盘/显示器接口；Intel 8275 可编程 CRT 控制器接口等。

(4) 按数据传送的控制方式分类，有程序型式接口和 DMA 式接口。程序型式接口用于连接速度较慢的 I/O 设备，如显示终端、键盘、打印机等。现

代计算机一般都可采用程序中中断方式实现主机与 I/O 设备交换信息，故都配有这类接口，如 Intel 8259。DMA 型接口是用于连接高速 I/O 设备，如磁盘、磁带等，常用 Intel 8257。有关这两类接口，将在 5.5 和 5.6 中讲述它们的基本组成原理。

5.4 程序查询方式

5.4.1 程序查询流程

由 5.1.4 已知，程序查询方式的核心问题在于每时每刻需不断查询 I/O 设备是否准备就绪。图 5.32 是单个设备的查询流程示意。

当 I/O 设备较多时，CPU 需按各个 I/O 设备在系统中的优先级别进行逐级查询，其流程图如图 5.33 所示。图中设备的优先顺序按 1 至 N 降序排列。

为了正确完成这种查询，通常要执行如下三条指令：

(1) 测试指令，用来查询设备是否准备就绪；

(2) 传送指令，当设备已准备就绪时，执行传送指令；

(3) 转移指令，若设备未准备就绪，执行转移指令，转至测试指令，继续测试设备的状态。

图 5.34 是单个设备程序查询方式的程序流程图。

当 CPU 需启动外设前，必须做如下的准备工作：

(1) 由于这种方式传送数据时要占用 CPU 中的寄存器，故首先需将寄存器原内容保护起来（若该寄存器中存有有用信息）；

(2) 由于传送往往是一批数据，因此需先设置设备与主机交换数据的计数值；

(3) 设置欲传送数据在内存缓冲区的首地址。

然后 CPU 启动设备，接着以下步骤操作：

(4) 启动外部设备；

(5) 将 I/O 接口中的设备状态标记取至 CPU 并测试 I/O 是否准备就绪。如果未准备就绪，则踏步等待，直到准备就绪为止。当准备就绪时，接着可实现传送。对输入而言，准备就绪意味着接口电路中的数据缓冲寄存器已装满欲传送的数据，叫做“输入缓冲满”，CPU 即可取走数据；对输出而言，准备就

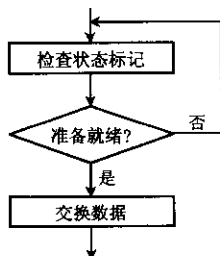


图 5.32 单个设备的查询流程示意

绪意味着接口电路中的数据已被设备取走，故叫“输出缓冲空”，这样 CPU 可再次将数据送到接口，设备可再次从接口接收数据；

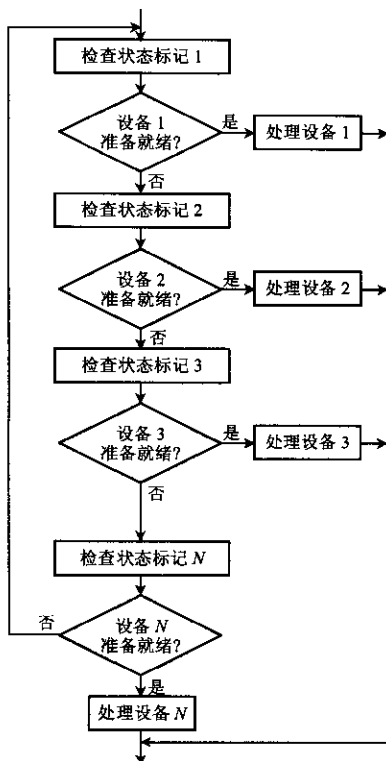


图 5.33 多个设备的查询流程示意

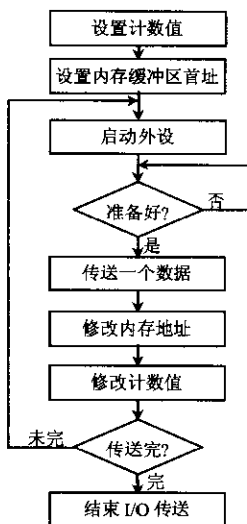


图 5.34 程序查询方式的程序流程

(6) CPU 执行 I/O 指令，或从 I/O 接口的数据缓冲寄存器中读出一个数据，或把一个数据写入到 I/O 接口中的数据缓冲寄存器内，同时把接口中的状态标记复位；

(7) 修改内存地址；

(8) 修改计数值，若原设置计数值为原码，则依次减 1；若原设置计数值为负数补码，则依次加 1（有关原码、补码的概念，可参阅 6.1）；

(9) 判断计数值。若计数值不为 0，表示一批数据尚未传送完，重新启动外设继续传送；若计数值为 0，则表示一批数据已传送完毕；

(10) 结束 I/O 传送，继续执行其他程序。

5.4.2 程序查询方式的接口电路

由上述的程序查询流程以及 5.3.2 所述的接口功能及组成，程序查询方式接口电路的基本组成如图 5.35 所示。

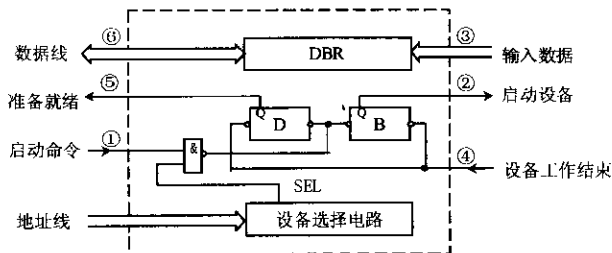


图 5.35 程序查询方式接口电路的基本组成

图中设备选择电路用以识别本设备地址，当地址线上的设备号与本设备号相符时，SEL 有效，可以接收命令。

DBR 是数据缓冲寄存器，用以存放欲传送的数据；

D、B 是反映设备工作状态的标记触发器，其功能如 5.3.2 所述。

以输入设备为例，本接口的工作过程如下：

当设备选中后，① 由 CPU 发出启动外设命令，将工作触发器 B 置“1”，完成触发器 D 置“0”；② 启动外设开始工作；③ 输入设备将数据送入 DBR；④ 外设工作完成，向接口发“设备工作结束”信号，将 D 置“1”，B 置“0”；⑤ D 触发器以“准备就绪”状态通知 CPU，表示“数据缓冲满”；⑥ CPU 执行输入指令，将输入数据送至 CPU 的通用寄存器，再存入主存相关单元。

5.5 程序中断方式

5.5.1 中断的概念

计算机在执行程序的过程中，当出现异常情况或特殊请求时，计算机停止现行程序的运行，转向对这些异常情况或特殊请求的处理，处理结束后再返回到现行程序的间断处，这就是“中断”（如图 5.10 所示）。中断是现代计算机能有效合理地发挥效能和提高效率的一个十分重要的功能。通常又把实现这种

功能所需的软硬件技术，统称为中断技术。

5.5.2 I/O 中断的产生

在 I/O 与主机交换信息时，由于设备本身机电特性的影响，其工作速度较低，与 CPU 无法匹配，因此，CPU 启动设备后，往往需要等待一段时间才能实现主机与设备交换信息。如果在设备准备的同时，CPU 不作无谓的等待，而继续执行现行程序，只有当 I/O 准备就绪向 CPU 提出请求后，再暂时中断 CPU 现行程序转入 I/O 服务程序，这便产生了 I/O 中断。

图 5.36 示意了由打印机引起的 I/O 中断时，CPU 与打印机的并行工作时间示意。

其实，计算机系统引入中断技术的原因不仅仅是为了适应 I/O 设备工作速度低的问题。例如，当计算机正在运行中，若出现突然掉电这种异常情况，将会导致 CPU 中的全部信息丢失。倘若能在突然掉电的瞬间，立即启动另一个备份电源，并迅速处理一些必要的事情，如将有用信息送至不受电源影响的存储系统内，待电源恢复后接着使用，这种处理技术也要用中断技术来实现。又如在实时控制领域中，要求 CPU 能即时响应外来信号的请求，并能完成各种相应的操作，也都要求采用中断技术。总之，为了提高计算机的整机效率，为了应付突发事件，为了实时控制的需要，在计算机技术中发展产生了“中断”技术。为了实现“中断”，计算机系统中必须配有相应的中断系统或中断机构。本节着重介绍 I/O 中断处理的相关内容，有关中断的其他内容将在第八章中讲述。

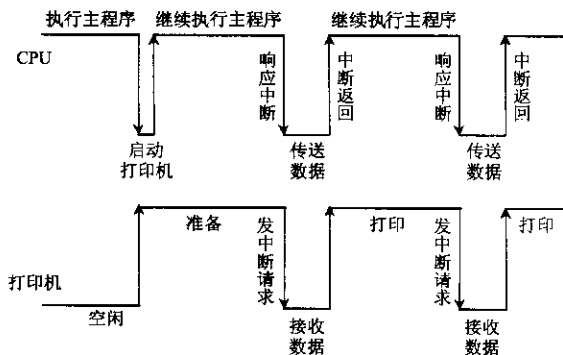


图 5.36 CPU 与打印机并行工作时间示意图

5.5.3 程序中断方式的接口电路

为处理 I/O 中断，在 I/O 接口电路中必须配置相关的硬件线路。

1. 中断请求触发器和中断屏蔽触发器

每台外部设备都必须配置一个中断请求触发器 INTR，当其为“1”时，表示该设备向 CPU 提出中断请求。但是设备欲提出中断请求时，其设备本身必须准备就绪，也即接口内的完成触发器 D 必为“1”状态。

由于计算机应用的范围越来越广泛，向 CPU 提出中断请求的原因也越来越多，除了各种 I/O 设备外，还有其他许多突发性事件都是引起中断的因素，为此，把凡能向 CPU 提出中断请求的各种因素统称为中断源。当多个中断源向 CPU 提出中断请求时，CPU 必须坚持一个原则，即在任何瞬间只能接受一个中断源的请求。所以，当多个中断源同时提出请求时，CPU 必须对各中断源的请求进行排队，且只能接受级别最高的中断源的请求，不允许级别低的中断源中断正在运行的中断服务程序。这样，在 I/O 接口中需设置一个屏蔽触发器 MASK，当其为 1 时，表示被屏蔽，即封锁其中断源的请求。可见中断请求触发器和中断屏蔽触发器在 I/O 接口中是成对出现的。有关屏蔽的详细内容将在 8.4.6 中讲述。

此外，CPU 总是在统一的时间，即执行每条指令的最后时刻，查询所有的设备是否有中断请求。

综合上述各因素，可得出接口电路中的完成触发器 D、中断请求触发器 INTR、中断屏蔽触发器 MASK 和中断查询信号的关系如图 5.37 所示。

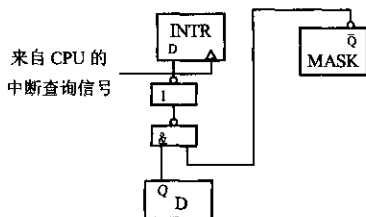


图 5.37 接口电路中 D、INTR、MASK 和中断查询信号的关系

2. 排队器

如上所述，当多个中断源同时向 CPU 提出请求时，CPU 只能按中断源的不同性质对其排队，给予不同等级的优先权，并按优先等级的高低予以响应。就 I/O 中断而言，速度越高的 I/O 设备其优先级越高，因为若 CPU 不及时响

应高速 I/O 的请求, 其信息可能立即会丢失。

设备优先权的处理可以采用硬件办法, 也可采用软件办法 (详见 8.4.2)。硬件排队器的实现方法很多, 既可在 CPU 内部设置一个统一的排队器, 对所有中断源进行排队 (详见图 8.22), 也可在接口电路内分别设置各个设备的排队器, 图 5.38 是设在各个接口电路中的排队器电路, 又叫链式排队电路。

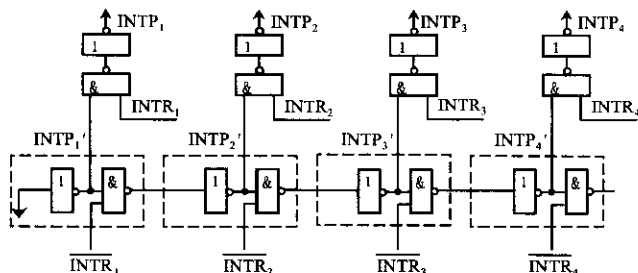


图 5.38 链式排队器

图中下面的一排门电路是链式排队器的核心。每个接口中有一个非门和一个与非门, 它们之间犹如链条一样串接在一起, 故有链式排队器之称。该电路中级别最高的中断源是 1 号, 其次是 2 号, 3 号, 4 号。不论是哪个中断源 (一个或多个) 提出中断请求, 排队器输出端 $\overline{INTP}_i$, 只有一个高电平。

当各中断源均无中断请求时, 各个 $\overline{INTR}_i$ 为高电平, 其 $\overline{INTP}_1'$ 、 $\overline{INTP}_2'$ 、 $\overline{INTP}_3'$ ……均为高电平。一旦某个中断源提出中断请求时, 就迫使比其优先级的中断源之 $\overline{INTP}_i'$ 变低电平, 封锁其发中断请求。如当 2 号和 3 号中断源同时有请求时 ($\overline{INTR}_2=0$, $\overline{INTR}_3=0$), 经分析可知 $\overline{INTP}_1'$ 和 $\overline{INTP}_2'$ 均为高电平, $\overline{INTP}_3'$ 及往后各级的 $\overline{INTP}_i'$ 均为低电平。各个 $\overline{INTP}_i'$ 再经图中上面一排两个输入头的与非门, 便可保证排队器只有 $\overline{INTP}_2$ 为高, 表示 2 号中断源排上队。

3. 中断向量地址形成部件 (设备编码器)

CPU 一旦响应了 I/O 中断, 就要暂停现行程序, 转去执行该设备的中断服务程序。不同的设备有不同的中断服务程序, 每个服务程序都有一个入口地址, CPU 必须找到这个入口地址。

I/O 的中断请求。因此，CPU 响应中断的时间一定是在每条指令执行阶段的结束时刻。

2. I/O 中断处理过程

下面以输入设备为例，结合图 5.41，说明 I/O 中断处理的全过程。当

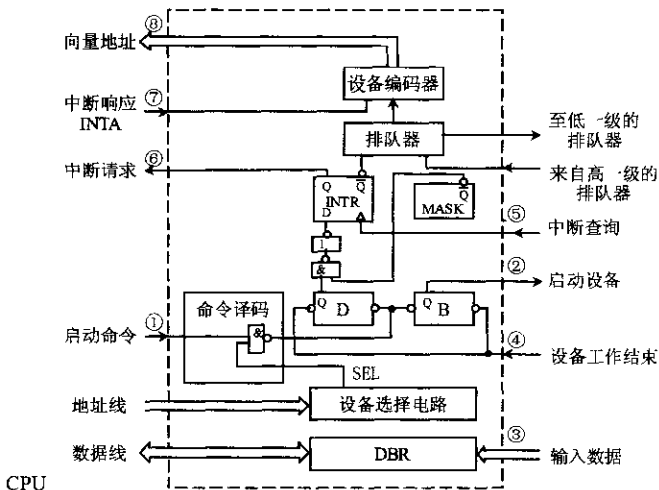


图 5.41 程序中断方式接口电路的基本组成

通过 I/O 指令的地址码选中某设备后，则

- ① 由 CPU 发启动外设命令，将接口中 B 置“1”，D 置“0”；
- ② 接口启动输入设备开始工作；
- ③ 输入设备将数据送入 DBR；
- ④ 输入设备向接口发“设备工作结束”信号，使 D 置“1”，B 置“0”，标志设备准备就绪；
- ⑤ 当设备准备就绪 (D=1)，且本设备未被屏蔽 (MASK=0) 时，在指令执行阶段的结束时刻，由 CPU 发中断查询信号；
- ⑥ 设备中断请求触发器 INTR 被置“1”，标志设备向 CPU 提出中断请求。与此同时，INTR 送至排队器，进行中断判优；
- ⑦ 若 CPU 允许中断 (EINT=1)，设备又被排队选中，即进入中断响应阶段，由中断响应信号 INTA 将排队器输出送至编码器形成向量地址；

⑧ 向量地址送至 PC，作为下一条指令的地址；

⑨ 由于向量地址中存放的是一条无条件转移指令（见图 5.40），故这条指令执行结束后，即无条件转至该设备的服务程序入口地址，开始执行中断服务程序，进入中断服务阶段；

⑩ 中断服务程序的最后一条指令执行结束，即中断返回至原程序的断点处。至此，一个完整的程序中断处理过程即告结束。

综上所述，可将一次中断处理过程简单地归纳为中断请求、中断判优、中断响应、中断服务和中断返回五个阶段。至于为什么能准确返回至原程序断点，CPU 在中断响应阶段除了将向量地址送至 PC 外，还做了什么其他操作等问题，将在 8.4 节讲述。

5.5.5 中断服务程序的流程

不同设备的服务程序是不相同的，可它们的程序流程又是类似的，一般中断服务程序的流程分四大部分：保护现场、中断服务、恢复现场和中断返回。

1. 保护现场

保护现场有两个含意，其一是保存程序的断点；其二是保存通用寄存器和状态寄存器的内容。前者由中断隐指令完成（详见 8.4.4 节），后者由中断服务程序完成。具体而言，可在中断服务程序的起始部分安排若干条存数指令，将寄存器的内容存至存储器中保存，或用进栈指令（PUSH）将各寄存器的内容推入堆栈保存，即将程序中中断时的“现场”保存起来。

2. 中断服务（设备服务）

这是中断服务程序的主体部分，不同的中断请求源其中断服务操作内容是不同的，如打印机要求 CPU 将需打印的一行字符代码，通过接口送入打印机的缓冲存储器中（见图 5.23）以供打印机打印；又如显示设备要求 CPU 将需显示的一屏字符代码，通过接口送入显示器的显示存储器中（见图 5.18）。

3. 恢复现场

这是中断服务程序的结尾部分，要求在退出服务程序前，将原程序中中断时的“现场”恢复到原来的寄存器中。通常可用取数指令或出栈指令（POP），将保存在存储器（或堆栈）中的信息送回到原来的寄存器中。

4. 中断返回

中断服务程序的最后一条指令通常是一条中断返回指令，使其返回到原程序的断点处，以便继续执行原程序。计算机在处理中断的过程中，有可能出现新的中断请求，此时如果 CPU 暂停现行的中断服务程序，转去处理新的中断请求，这种现象叫做中断嵌套，或叫做多重中断。倘若 CPU 在执行中断服务

程序时, 对新的中断请求不予理睬, 这种中断叫做单重中断。这两种处理方式的中断服务程序略有区别。图 5.42 (a) 和 (b) 分别为单重中断和多重中断服务程序流程。比较 (a) 和 (b) 发现其区别在于“开中断”的设置时间不同。

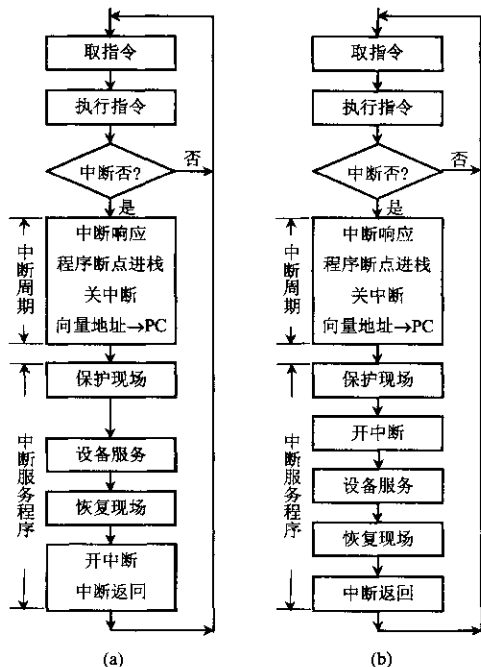


图 5.42 单重中断和多重中断服务程序流程

由于 CPU 一旦响应了某中断源的中断请求后, 便由硬件线路自动关中断, 即中断允许触发器 $EINT$ 被置“0” (详见图 8.27), 以确保该中断服务程序的顺利执行。因此如果不用“开中断”指令将 $EINT$ 置“1”, 则意味着 CPU 不能再响应其他任何一个中断源的中断请求。对于单重中断, 开中断指令设置在最后“中断返回”之前, 意味着在整个中断服务处理过程中, 不能再响应其他中断源的请求。对于多重中断, 开中断指令提前至“保护现场”之后, 意味着在保护现场后, 若有级别更高的中断源提出请求 (这是实现多重中断的必要条件), CPU 也可以响应, 即再次中断现行的服务程序, 转至新的中断服务程序, 这是单重中断与多重中断的主要区别。有关多重中断的详细内容参见 8.4.6 节。

从宏观上分析, 程序中中断方式克服了程序查询方式中的 CPU “踏步” 现象, 实现了 CPU 与 I/O 的并行工作, 提高了 CPU 的资源利用率。但从微观操作分析, 发现 CPU 在处理中断服务程序时, 仍需暂停原程序的正常运行。尤其是当高速 I/O 设备或辅助存储器需要频繁地、成批地与主存交换信息时, 不断地打断 CPU 执行主程序而执行中断服务程序。图 5.43 是主程序和服务程序抢占 CPU 的示意图。为此, 人们探索出使 CPU 效率更高的 DMA 控制方式。

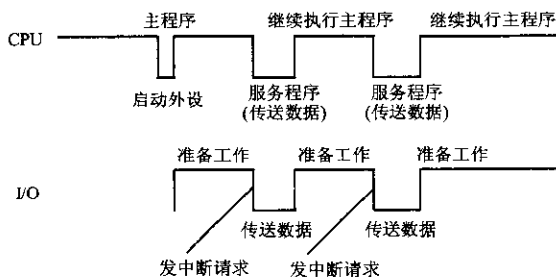


图 5.43 主程序和服务程序抢占 CPU 示意图

5.6 DMA 方式

5.6.1 DMA 方式的特点

图 5.44 示出了 DMA 方式与程序中中断方式两者数据通路的比较。

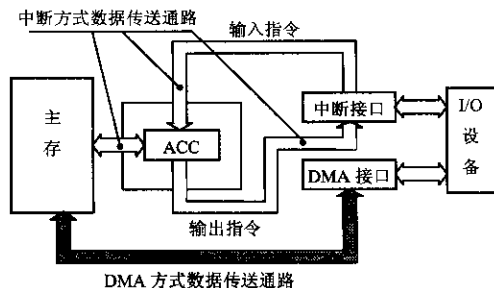


图 5.44 DMA 和程序中中断两种方式的数据通路

由图可见, 由于主存和 DMA 接口之间有一条数据通路, 因此主存和设备交换信息时, 不通过 CPU, 也不需要 CPU 暂停现行程序为设备服务, 省去了保护现场和恢复现场, 因此工作速度比程序中断方式高。这一特点特别适合于高速 I/O 或辅存与主存之间的信息交换。因为高速 I/O 设备若每次申请与主机交换信息时, 都要等待 CPU 作出中断响应后再交换, 很可能因此使数据丢失。

此外, 若出现高速 I/O (通过 DMA 接口) 和 CPU 同时访问主存, CPU 必须将总线 (如地址线、数据线) 占有权让给 DMA 接口使用, 即 DMA 采用周期窃取的方式占用一个存取周期。

在 DMA 方式中, 由于 DMA 接口与 CPU 共享主存, 这就有可能出现两者争用主存的冲突, 为了有效地分时使用主存, DMA 通常与主存交换数据时可采用如下三种方法。

(1) 停止 CPU 访问主存

当外设要求传送一批数据时, 由 DMA 接口向 CPU 发一个停止信号, 要求 CPU 放弃地址线、数据线和有关控制线的使用权。DMA 接口获得总线控制权后, 开始进行数据传送, 在数据传送结束后, DMA 接口通知 CPU 可以使用主存, 并把总线控制权交回给 CPU, 图 5.45 (a) 是该方式的时间示意图。

这种方式的优点是控制简单, 适用于数据传输率很高的 I/O 设备实现成组数据的传送。缺点是 DMA 接口在访存时, CPU 基本上处于不工作状态或保持原状态。而且即使 I/O 设备高速运行, 但其两个数据之间的准备间隔时间也总大于一个存取周期, 因此, CPU 对主存的利用率并没得到充分的发挥。如软盘读一个 8 位二进制数大约需要 $32\mu\text{s}$, 而半导体存储器的存取周期大大小于 $1\mu\text{s}$ 。为此在 DMA 接口中, 一般设有一个小容量存储器 (这种存储器是半导体芯片制作的), 使 I/O 设备首先与小容量存储器交换数据, 然后由小容量存储器与主存交换数据, 这便可减少 DMA 传送数据时占用总线的时间, 即可减少 CPU 的暂停工作时间。

(2) 周期挪用 (或周期窃取)

在这种方法中, 每当 I/O 设备发出 DMA 请求时, I/O 设备便挪用或窃取总线占用权一个或几个主存周期, 而 DMA 不请求时, CPU 仍继续访问主存。

I/O 设备要求 DMA 传送会遇到三种情况, 一种是此时 CPU 不需访问主存 (如 CPU 正在执行乘法指令, 由于乘法指令执行时间较长, 此时 CPU 不需访问主存), 故 I/O 设备访存与 CPU 不发生冲突。第二种情况是 I/O 设备要求 DMA 传送时, CPU 正在访存, 此时必须待存取周期结束时刻, CPU 才能将总线占有权让出。第三种情况是 I/O 设备要求访存时, CPU 也要求访存, 这就出现了访存冲突。此刻, I/O 访存优先于 CPU 访存, 因为 I/O 不立即访存就可能丢失数据, 这时 I/O 要窃取一、二个存取周期, 意味着 CPU 在执行访存指令过

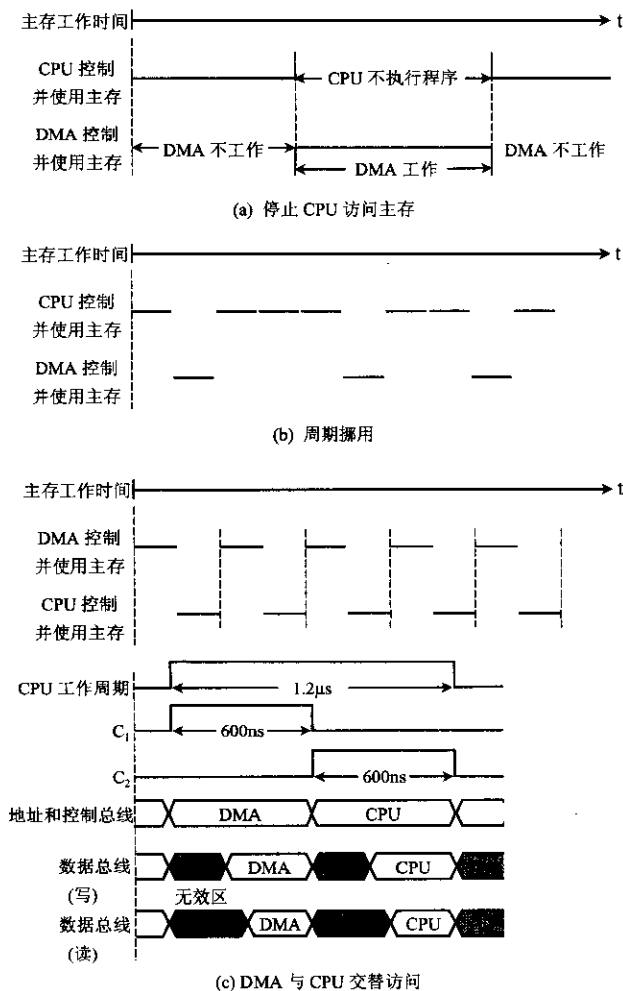


图 5.45 DMA 的三种传送方式

程中插入了 DMA 请求,并挪用了一、二个存取周期,使 CPU 延缓了一、二个存取周期再访存。图 5.45 (b) 示意了 DMA 周期挪用的时间对应关系。

与 CPU 暂停访存的方式相比,它既实现了 I/O 传送,又较好地发挥了主存与 CPU 的效率,是一种广泛采用的方法。

应该指出, I/O 设备每挪用一個主存周期都要申请总线控制权、建立总线控制权和归还总线控制权。因此,尽管传送一个字对主存而言只占用一个主存周期,但对 DMA 接口而言,实质上要占 2~5 个主存周期(由逻辑线路的延迟特性而定)。因此周期挪用的方法比较适合于 I/O 设备的读写周期大于主存周期的情况。

(3) DMA 与 CPU 交替访问

这种方法适合于 CPU 的工作周期比主存取周期长的情况。例如 CPU 的工作周期为 $1.2\mu\text{s}$,主存的存取周期小于 $0.6\mu\text{s}$,那么可将一个 CPU 周期分为 C_1 和 C_2 两个分周期,其中 C_1 专供 DMA 访存, C_2 专供 CPU 访存,如图 5.45 (c) 所示。

这种方式不需要总线使用权的申请建立和归还过程,总线使用权是通过 C_1 和 C_2 分别控制的。CPU 与 DMA 接口各自有独立访存地址寄存器、数据寄存器和读/写信号。实际上总线变成了在 C_1 和 C_2 控制下的多路转换器,其总线控制权的转移几乎不需要什么时间,具有很高的 DMA 传送效率。在这种工作方式下, CPU 既不停止主程序的运行也不进入等待状态,在 CPU 不知不觉中完成了 DMA 的数据传送,故又有“透明的 DMA”方式之称,当然其相应的硬件逻辑变得更为复杂。

5.6.2 DMA 接口的功能和组成

1. DMA 接口的功能

利用 DMA 方式传送数据时,数据的传输过程完全由 DMA 接口电路控制,故 DMA 接口又有 DMA 控制器之称。DMA 接口应具有如下几个功能:

- (1) 向 CPU 申请 DMA 传送;
- (2) 在 CPU 允许 DMA 工作时,处理总线控制权的转交,避免因进入 DMA 工作而影响 CPU 正常活动或引起总线竞争;
- (3) 在 DMA 期间管理系统总线,控制数据传送;
- (4) 确定数据传送的起始地址和数据长度,修正数据传送过程中的数据地址和数据长度;
- (5) 在数据块传送结束时,给出 DMA 操作完成的信号。

2. DMA 接口基本组成

最简单的 DMA 接口组成原理如图 5.46 所示,它由以下几个逻辑部件所

组成。

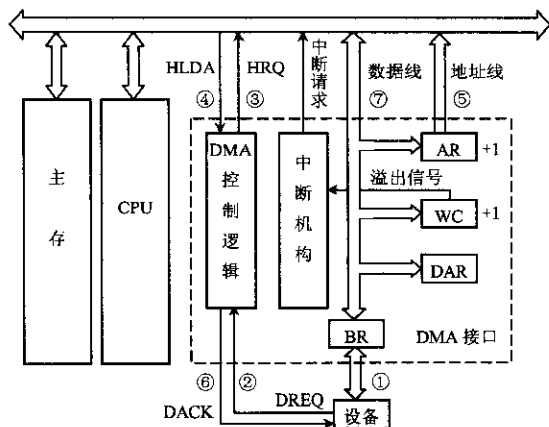


图 5.46 简单的 DMA 接口组成原理图

（1）主存地址寄存器 AR

AR 用于存放主存中需要交换数据的地址。在 DMA 传送前，须通过程序将数据在主存中的首地址送到主存地址寄存器。在 DMA 传送过程中，每交换一次数据，将地址寄存器内容加 1，直到一批数据传送完毕为止。

（2）字计数器 WC

WC 用于记录传送数据的总字数，通常以交换字数的补码值预置。在 DMA 传送过程中，每传送一个字，字计数器加 1，直到计数器为 0，即最高位产生进位时，表示该批数据传送完毕。于是 DMA 接口向 CPU 发中断请求信号。

（3）数据缓冲寄存器 BR

BR 用于暂存每次传送的数据。通常 DMA 接口与主存之间采用字传送，而 DMA 与设备之间可能是字节或位传送。因此 DMA 接口中还可能包括有装配或拆卸字信息的硬件逻辑，如数据移位缓冲寄存器、字节计数器等。

（4）DMA 控制逻辑

它用于负责管理 DMA 的传送过程，由控制电路、时序电路及命令状态控制寄存器等组成。每当设备准备好一个数据字（或一个字传送结束），就向 DMA 接口提出申请（DREQ），DMA 控制逻辑便向 CPU 请求 DMA 服务，发出总线使用权的请求信号（HRQ）。待收到 CPU 发出的响应信号 HLDA 后，DMA 控制逻辑便开始负责管理 DMA 传送的全过程，包括对主存地址寄存器和字计数器的修改、识别总线地址、指定传送类型（输入或输出）以及通知设备已经

被授予一个 DMA 周期 (DACK) 等。

(5) 中断机构

当字计数器溢出 (全“0”) 时, 表示一批数据交换完毕, 由“溢出信号”通过中断机构向 CPU 提出中断请求, 请求 CPU 作 DMA 操作的后处理。必须注意, 这里的中断与上一节介绍的 I/O 中断的技术相同, 但中断的目的不同, 前面是为了数据的输入或输出, 而这里是为了报告一批数据传送结束。它们是 I/O 系统中不同的中断事件。

(6) 设备地址寄存器 DAR

DAR 存放 I/O 设备的设备码或表示设备信息存储区的寻址信息, 如磁盘数据所在的区号、盘面号和柱面号。具体内容取决于设备的数据格式和地址的编址方式。

5.6.3 DMA 的工作过程

1. DMA 传送过程

DMA 的数据传送过程分预处理、数据传送和后处理三个阶段。

(1) 预处理

在 DMA 接口开始工作之前, CPU 必须给它预置如下信息:

- 给 DMA 控制逻辑指明数据传送方向是输入 (主存写) 还是输出 (主存读);
- 向 DMA 设备地址寄存器送入设备号, 并启动设备;
- 向 DMA 主存地址寄存器送入交换数据的主存起始地址;
- 对字计数器赋以交换数据的个数。

上述工作由 CPU 执行几条输入输出指令完成, 即程序的初始化阶段。这些工作完成后, CPU 继续执行原来的程序, 如图 5.47 (a) 所示。

当外部设备准备好发送的数据 (输入) 或上次接受的数据已经处理完毕 (输出) 时, 它便通过 DMA 接口向 CPU 提出占用总线的申请, 若有多个 DMA 同时申请, 则按轻重缓急由硬件排队判优先级决定优先等级。待设备得到主存总线的控制权后, 数据的传送便由该 DMA 接口进行管理。

(2) 数据传送

DMA 方式是以数据块为单位传送的, 结合图 5.46, 并以周期挪用的 DMA 方式为例, 其数据传送的流程可用图 5.47 (b) 示意。

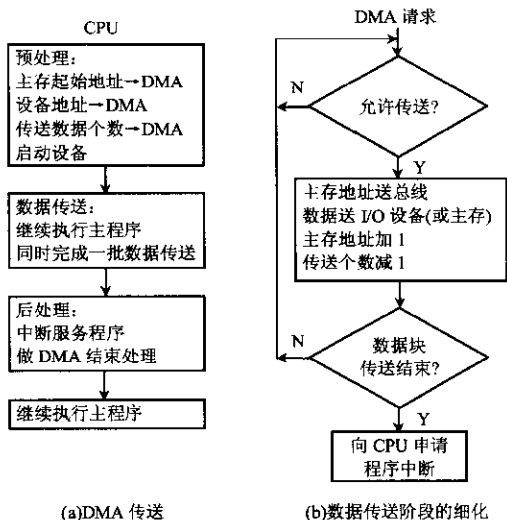


图 5.47 DMA 传送过程示意

以数据输入为例，具体操作如下：

- ① 从设备读入一个字到 DMA 的数据缓冲寄存器 BR 中，表示数据缓冲寄存器“满”（如果 I/O 设备是面向字符的，则一次读入一个字节，组装成一个字）；
 - ② 设备向 DMA 接口发请求（DREQ）；
 - ③ DMA 接口向 CPU 申请总线控制权（HRQ）；
 - ④ CPU 发回 HLDA 信号，表示允许将总线控制权交给 DMA 接口；
 - ⑤ 将 DMA 主存地址寄存器中的主存地址送地址总线；
 - ⑥ 通知设备已被授予一个 DMA 周期（DACK），并为交换下一个字做准备；
 - ⑦ 将 DMA 数据缓冲寄存器的内容送数据总线；
 - ⑧ 命令存储器作写操作；
 - ⑨ 修改主存地址和字计数值；
 - ⑩ 判断数据块是否传送结束，若未结束，则继续传送；若已结束，（字计数器溢出），则向 CPU 申请程序中断，标志数据块传送结束。
- 若为输出数据，则应完成以下操作：

① 当 DMA 数据缓冲寄存器已将输出数据送至 I/O 设备后, 表示数据缓冲寄存器已“空”;

② 设备向 DMA 接口发请求 (DREQ);

③ DMA 接口向 CPU 申请总线控制权 (HRQ);

④ CPU 发回 HLDA 信号, 表示允许将总线控制权交给 DMA 接口使用;

⑤ 将 DMA 主存地址寄存器中的主存地址送地址总线, 并命令存储器读;

⑥ 通知设备已被授予一个 DMA 周期 (DACK), 并为交换下一个字做准备;

⑦ 主存将相应地址单元的内容通过数据总线读入到 DMA 的数据缓冲寄存器中;

⑧ 将 DMA 数据缓冲寄存器的内容送到输出设备, 若为字符设备, 则需将其拆成字符输出;

⑨ 修改主存地址和字计数值;

⑩ 判断数据块是否已传送完毕, 若未完, 继续传送; 若已送完, 则向 CPU 申请程序中断。

(3) 后处理

当 DMA 的中断请求得到响应后, CPU 停止原程序的执行, 转去执行中断服务程序, 做一些 DMA 的结束工作, 如图 5.47 (a) 的后处理部分。它包括校验送入主存的数据是否正确; 决定是否继续用 DMA 传送其他数据块, 若继续传送, 则又要对 DMA 接口进行初始化, 若不需要传送, 则停止外设; 测试在传送过程中是否发生错误, 若出错, 则转错误诊断及处理错误程序。

2. DMA 接口与系统的连接方式

DMA 接口与系统的连接方式有两种, 如图 5.48 所示。

图 5.48 (a) 为具有公共请求线的 DMA 请求方式, 若干个 DMA 接口通过一条公用的 DMA 请求线向 CPU 申请总线控制权。CPU 发出响应信号用链式查询方式通过 DMA 接口, 首先选中的设备获得总线控制权, 即可占用总线与主存传送信息。

图 5.48 (b) 是独立的 DMA 请求方式, 每一个 DMA 接口各有一对独立的 DMA 请求线和 DMA 响应线, 它由 CPU 的优先级判别机构裁决首先响应哪个请求, 并在响应线上发出响应信号, 被获得响应信号的 DMA 接口便可控制总线与主存传送数据。

3. DMA 小结

与程序中断方式相比, DMA 方式有如下特点:

(1) 从数据传送看, 程序中断方式靠程序传送, DMA 方式靠硬件传送。

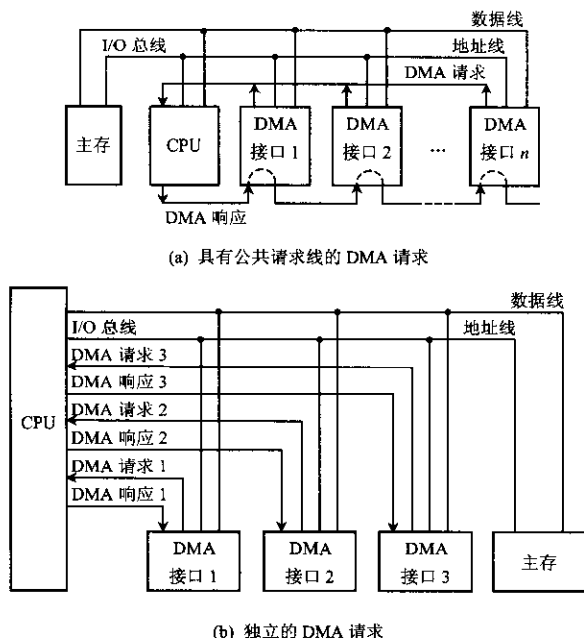


图 5.48 DMA 接口与系统的连接方式

(2) 从 CPU 响应时间看，程序中断方式是在一条指令执行结束时响应，而 DMA 方式可在指令周期内的任一存取周期结束时响应。

(3) 程序中断方式有处理异常事件的能力，DMA 方式没有这种能力，它主要用于大批数据的传送，如硬盘存取、图像处理、高速数据采集系统等，可提高数据吞吐量。

(4) 程序中断方式要中断现行程序，故需保护现场，DMA 方式不中断现行程序，无需保护现场。

(5) DMA 的优先级比程序中断高。

5.6.4 DMA 接口的类型

现代集成电路制造技术已将 DMA 接口制成芯片，通常有选择型和多路型两类。

1. 选择型 DMA 接口

这种类型的 DMA 接口基本组成如图 5.46 所示, 它的主要特点是在物理上可连接多个设备, 在逻辑上只允许连接一个设备。即在某一个时间内, DMA 接口只能为一个设备服务, 关键是在预处理时将所选设备的设备号送入设备地址寄存器。图 5.49 是选择型 DMA 接口的逻辑框图。选择型 DMA 接口特别适用于数据传输率很高的设备。

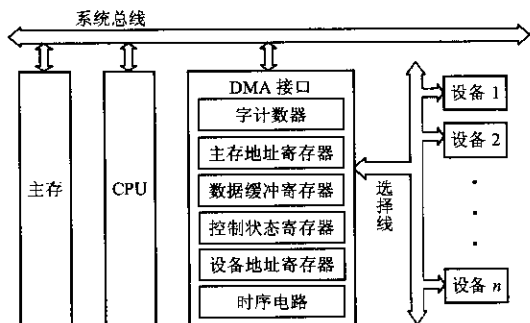


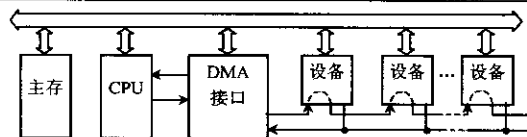
图 5.49 选择型 DMA 接口的逻辑框图

2. 多路型 DMA 接口

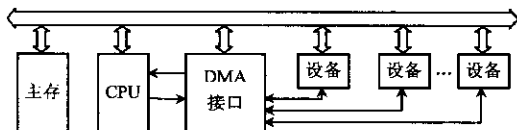
多路型 DMA 接口不仅在物理上可以连接多个设备, 而且在逻辑上也允许多个设备同时工作, 各个设备采用字节交叉的方式通过 DMA 接口进行数据传输。在多路型 DMA 接口中, 为每个与它连接的设备都设置了一套寄存器, 分别存放各自的传送参数。图 5.50 (a) 和 (b) 分别是链式多路型 DMA 接口和独立请求多路型 DMA 接口的逻辑框图。这类接口特别适合于同时为多个数据传输率不十分高的设备服务。

图 5.51 是多路型 DMA 接口工作原理示意图。图中磁盘、磁带、打印机同时工作。磁盘、磁带、打印机分别每隔 $30\mu\text{s}$ 、 $45\mu\text{s}$ 、 $150\mu\text{s}$ 向 DMA 接口发 DMA 请求, 磁盘的优先级高于磁带, 磁带的优先级高于打印机。

假设 DMA 接口完成一次 DMA 数据传送需 $5\mu\text{s}$ 。由图 5.51, 打印机首先发请求, 故 DMA 接口首先为打印机服务 (T_1), 接着磁盘、磁带同时又有 DMA 请求, DMA 接口按优先级别, 先响应磁盘请求 (T_2), 再响应磁带请求 (T_3), 每次 DMA 传送都是一个字节。这样, 在 90 多 μs 的时间里, DMA 接口为打印机服务一次 (T_1), 为磁盘服务四次 (T_2 、 T_4 、 T_6 、 T_7) 为磁带服务三次 (T_3 、 T_5 、 T_8)。可见 DMA 接口还有很多空闲时间, 可再容纳更多的外部设备。



(a) 链式多路型 DMA 接口



(b) 独立请求多路型 DMA 接口

图 5.50 多路型 DMA 接口的逻辑框图

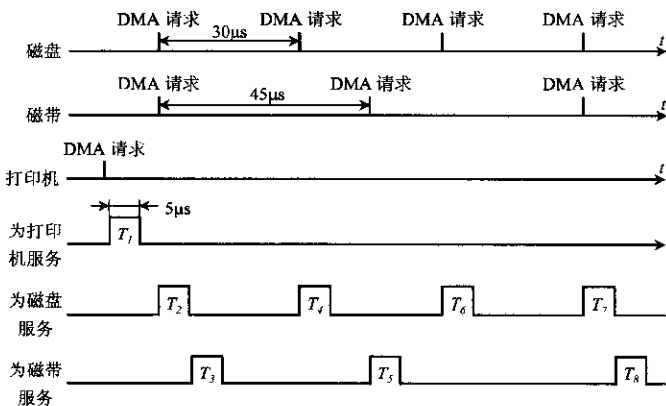


图 5.51 多路型 DMA 接口工作原理示意图

-
25. 假设某设备向 CPU 传送信息的最高频率是 40K 次/秒, 而相应的中断处理程序其执行时间为 $40\mu\text{s}$, 试问该外设是否可用程序中断方式与主机交换信息, 为什么?
26. 设磁盘存储器转速为 3 000 转/分, 分 8 个扇区, 每扇区存储 1K 字节, 主存与磁盘存储器数据传送的宽度为 16 位 (即每次传送 16 位)。假设一条指令最长执行时间是 $25\mu\text{s}$, 是否可采用一条指令执行结束时响应 DMA 请求的方案, 为什么? 若不行, 应采取什么方案?
27. 试从下面七个方面比较程序查询、程序中断和 DMA 三种方式的综合性能。
- (1) 数据传送依赖软件还是硬件;
 - (2) 传送数据的基本单位;
 - (3) 并行性;
 - (4) 主动性;
 - (5) 传输速度;
 - (6) 经济性;
 - (7) 应用对象。
28. 解释周期挪用, 分析周期挪用可能会出现的情况。
29. 试从五个方面比较程序中断和 DMA 方式有何区别。
30. 什么是多重中断? 实现多重中断的必要条件是什么?

附录 5A.1 ASCII 码

表 5.2 列出了 ASCII 码(American Standard Code for Information Interchange, 美国国家信息交换标准字符码)是美国信息交换标准委员会制定的 7 位二进制码, 共有 128 种元素, 其中包括 32 个通用控制字符, 10 个十进制数码, 52 个英文大写与小写字母, 34 个专用符号, 如 \$、%、+、=……。除了 32 个控制字符不打印外, 其余 96 个全部可以打印出字符。

ASCII 码由 $b_7b_6b_5b_4b_3b_2b_1$ 七个二进制位组成, 书写上可用两位十六进制数表示, 如“A”可用 41H 表示, “7”可用 37H 表示。为了提高信息传输的可靠性, 通常增加一位 b_8 做校验位, 这样一个字符就可用 8 位二进制代码表示。

表 5.2 ASCII 码 $b_7b_6b_5b_4b_3b_2b_1$

$b_7b_6b_5$ $b_4b_3b_2b_1$	000	001	010	011	100	101	110	111
0000	NUL	DLE	SP	0	@	P	`	~
0001	SOH	DC1	!	1	A	Q	a	q
0010	STX	DC2	"	2	B	R	b	r
0011	ETX	DC3	#	3	C	S	c	s
0100	EOT	DC4	\$	4	D	T	d	t
0101	ENQ	NAK	%	5	E	U	e	u
0110	ACK	SYN	&	6	F	V	f	v
0111	BEL	ETB	'	7	G	W	g	w
1000	BS	CAN	(	8	H	X	h	x
1001	HT	EM	)	9	I	Y	i	y
1010	LF	SUB	*	:	J	Z	j	z
1011	VT	ESC	+	;	K	[	k	{
1100	FF	FS	,	<	L	/	l	
1101	CR	GS	-	=	M	]	m	}
1110	SO	RS	.	>	N	^	n	~
1111	SI	VS	/	?	O	_	o	DEL

注:	LF	换行	NAK	否定回答	
NUL	空行	VT	纵向制表	SYN	同步空转
SOH	标题开始	FF	改换格式	ETB	信息组传送结束
STX	文件开始	CR	回车	CAN	作废
ETX	文件结束	SO	移出	EM	记录媒体结束
EOT	传送结束	SI	移入	SUB	代替
ENQ	询问	DEL	删除	ESC	脱离
ACK	回答	DC1	设备控制 1	FS	字段分隔
BEL	报警	DC2	设备控制 2	GS	字组分隔

用 ASCII 码可方便地表示十进制数串。十进制数串在计算机内主要有两种表示形式: 非压缩型和压缩型。

(1) 非压缩型

非压缩型的十进制数每一个字符占一个字节，又根据符号位的不同位置，将其分为前分隔式和后嵌入式两种。

前分隔式的符号位占一个字节，并且放在数字位之前。用 2B (即字符 “+” 的 ASCII 码) 表示正号，用 2D (即字符 “-” 的 ASCII 码) 表示负号。每个十进制数位均用对应的 ASCII 码表示，如：

+427 表示为 2B 34 32 37

-427 表示为 2D 34 32 37

后嵌入式的符号位不占一个字节，而是将符号嵌入到最低一位数字中，其规则如下：如果是负数，就将最低位十进制数的 ASCII 码加上 40H；如果是正数则不变。如：

+427 表示为 34 32 37

-427 表示为 34 32 77

可见最低一个字节既表示数值，又表示符号。

用非压缩型表示的十进制数进行算术运算很不方便，因为每个字节占 8 位，只有其低 4 位的值才表示数值，高 4 位值在算术运算时无数值意义，这种表示主要用于非数值计算的有关领域中。

(2) 压缩型

如果采用一个字节存放两个十进制的数位，就成了压缩型的十进制数。这种方式比非压缩型节省了存储空间，又便于完成十进制数的算术运算。压缩型十进制数的每个数位可用数字的 ASCII 码的低 4 位表示，或用 BCD 码表示。

附录 5A.2 BCD 码

BCD 码 (Binary Coded Decimal Code) 又叫二—十编码, 它用 4 位二进制代码表示一位十进制数。最常见的 BCD 码是 8421 码, 又叫 NBCD 码 (Natural Binary Coded Decimal Code)。由于 8421 码每位的权与二进制数完全相同, 而 4 位二进制代码共有 16 种组合, 因此 1010~1111 这六种代码是无效的。NBCD 码与十进制数的对应关系如表 5.3 所示。

表 5.3 8421 码与十进制数对照表

十进制数	8421 码	8421 奇校验码	8421 偶校验码
0	0000	10000	00000
1	0001	00001	10001
2	0010	00010	10010
3	0011	10011	00011
4	0100	00100	10100
5	0101	10101	00101
6	0110	10110	00110
7	0111	00111	10111
8	1000	01000	11000
9	1001	11001	01001

采用 BCD 码所表示的十进制数, 再用十六进制 C 表示 “+” 号, 用十六进制 D 表示 “-” 减号, 而且均放在数字串的最后, 就可表示有符号的十进制数。如:

+427 表示为 0100 0010 0111 1100

-427 表示为 0100 0010 0111 1101

当十进制数串为偶数时, 在第一个字节的高 4 位补 “0”, 即

+42 表示为 0000 0100 0010 1100

-42 表示为 0000 0100 0010 1101

附录 5A.3 奇偶校验码

为了提高编码的检测能力，在被传送的 n 位代码上增加一位检验位，并使其配置后的 $n+1$ 位代码中“1”的个数为奇数，则称其为奇校验；若配置后“1”的个数为偶数，则称其为偶校验。例如，在十进制数的 8421 码的前面加上一位检测位，组成 5 位代码，若五位二进制代码配置结果“1”的个数为奇数，就叫奇校验码；若配置结果“1”的个数为偶数，就叫偶校验码，如表 5.3 所示。对表中奇校验码而言，倘若传送过程中五位代码中“1”的个数不为奇数，则表明传送出错，可见奇校验码具有检错能力。同理，偶校验码也具有检错能力。

奇偶校验码通常用于外部设备，如键盘输入时使用 ASCII 码，再配一位校验位，组成 8 位的奇偶校验码，正好占一个字节。在传送过程中如果出现一位错，便能检测出来，但由于不知出错位的位置，故无法纠错。此外，一旦传送过程中出现两位错，奇偶性不变，也无法判断出错。

第三篇 中央处理器（CPU）

以上各章节基本上把 CPU 看作为一个“黑匣子”，并且分析了它通过总线与存储器和 I/O 部件之间的相互关系。本篇将剖析其内部结构，讲述 CPU 的功能，包括计算机的运算、指令系统、时序系统、中断系统及控制单元。除控制单元将在第四篇单独讲述外，其余部分均在此篇介绍。

第六章 计算机的运算方法

计算机的应用领域极其广泛，但不论其应用在什么地方，信息在机器内部的形式都是一致的，即均为 0 和 1 组成的各种编码。本章主要介绍参与运算的各类数据（包括无符号数和有符号数；定点数和浮点数等），以及它们在计算机中的算术运算方法。使读者进一步认识到计算机在自动解题过程中数据信息的加工处理流程，从而进一步加深对计算机硬件组成及整机工作原理的理解。有关逻辑运算以及计算机中采用的各种进位制均在前修课中介绍过，本章只在附录中给出了各种进位制及其相互转换的关系（可参阅附录 6A.1, 6A.2）。至于计算机中的字符编码以及校验码，读者可分别参阅本书附录 5A.1（ASCII 码）、附录 5A.2（BCD 码）、附录 5A.3（奇偶校验码）、4.2.6（海明码）和 4.4.6（循环冗余校验码）等章节。

6.1 无符号数和有符号数

在计算机中参与运算的数有两大类：无符号数和有符号数。

6.1.1 无符号数

计算机中的数均放在寄存器中，通常称寄存器的位数为机器字长。所谓无符号数即没有符号的数，在寄存器中的每一位均可用来存放数值。当存放有符号数时，则需留出位置存放“符号”。因此，在机器字长相同时，无符号数与有符号数所对应的数值范围是不同的。以机器字长为 16 位为例，无符号数的表示范围为 0~65535，而有符号数的表示范围为-32768~+32767（此数值对应原码表示，详见 6.1.2 节）。

6.1.2 有符号数

1. 机器数与真值

对有符号数而言，符号的“正”、“负”机器是无法识别的，但由于“正”、“负”恰好是两种截然不同的状态，如果用“0”表示“正”，用“1”表示“负”，这样符号也被数字化了，并且规定将它放在有效数字的前面，这样就组成了有符号数。

如有符号数（小数）：

+0.1011 在机器中表示为 

-0.1011 在机器中表示为 

又如有符号数 (整数):

+1100 在机器中表示为 

-1100 在机器中表示为 

把符号“数字化”的数叫做机器数, 而把带“+”或“-”符号的数叫做真值。一旦符号数字化后, 符号和数值就形成了一种新的编码。在运算过程中, 符号位能否和数值部分一起参加运算? 如果参加运算, 符号位又需作哪些处理? 这些问题都与符号位和数值位所构成的编码有关, 这些编码就是原码、补码、反码和移码。

2. 原码表示法

原码是机器数中最简单的一种表示形式, 其符号位为 0 表示正数, 符号位为 1 表示负数, 数值位即真值的绝对值, 故原码表示又称作带符号的绝对值表示。上面列举的四个真值所对应的机器数即为原码。为了书写方便以及区别整数和小数, 约定整数的符号位与数值位之间用逗号“,”隔开; 小数的符号位与数值位之间用小数点“.”隔开。如上面四个数的原码分别是 0.1011、1.1011、0,1100 和 1,1100。由此可得原码的定义。

整数原码的定义为

$$[x]_{\text{原}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^n - x & 0 \geq x > -2^n \end{cases}$$

式中 x 为真值, n 为整数的位数。

例如, 当 $x = +1110$ 时, $[x]_{\text{原}} = 0,1110$

当 $x = -1110$ 时, $[x]_{\text{原}} = 2^4 - (-1110) = 1,1110$

小数原码的定义为

$$[x]_{\text{原}} = \begin{cases} x & 1 > x \geq 0 \\ 1 - x & 0 \geq x > -1 \end{cases}$$

例如, 当 $x = 0.1101$ 时, $[x]_{\text{原}} = 0.1101$

当 $x = -0.1101$ 时, $[x]_{\text{原}} = 1 - (-0.1101) = 1.1101$

根据定义, 已知真值可求原码, 反之已知原码也可求真值。如:

当 $[x]_{\text{原}}=1.0011$ 时,

由定义得 $x=1-[x]_{\text{原}}=1-1.0011=-0.0011$

当 $[x]_{\text{原}}=1.1100$ 时,

由定义得 $x=2^n-[x]_{\text{原}}=2^4-1, 1100=10000-11100=-1100$

当 $[x]_{\text{原}}=0.1101$ 时, $x=0.1101$

当 $x=0$ 时

$$[+0.0000]_{\text{原}}=0.0000$$

$$[-0.0000]_{\text{原}}=1-(0.0000)=1.0000$$

可见 $[+0]_{\text{原}}$ 不等于 $[-0]_{\text{原}}$, 即原码中的“零”有两种表示形式。

原码表示简单明了, 并易于和真值转换。但用原码进行加减运算时, 却带来了许多麻烦。例如, 当两个操作数符号不同且要作加法运算时, 先要判断两数绝对值大小, 然后将绝对值大的数减去绝对值小的数, 结果的符号以绝对值大的数为准。运算步骤既复杂又费时, 而且本来是加法运算却要用减法器实现。那么能否在计算机中只设加法器, 只作加法操作呢? 如果能找到一个与负数等价的正数来代替该负数, 就可把减法操作用加法代替。而机器数采用补码时, 就能满足此要求。

3. 补码表示法

(1) 补数的概念

在日常生活中, 常会遇到“补数”的概念。如时钟指示 6 点, 欲使它指示 3 点, 既可按顺时针方向将分针转 9 圈, 又可按逆时针方向将分针转 3 圈, 结果是一致的。假设顺时针方向转为正, 逆时针方向转为负, 则有

$$\begin{array}{r} 6 \\ -3 \\ \hline 3 \end{array} \qquad \begin{array}{r} 6 \\ +9 \\ \hline 15 \end{array}$$

由于时钟的时针转一圈能指示 12 个小时, 这“12”在时钟里是不被显示而自动丢失的, 即 $15-12=3$, 故 15 点和 3 点均显示 3 点。这样 -3 和 $+9$ 对时钟而言其作用是一致的。在数学上称 12 为模, 写作 $\text{mod } 12$, 而称 $+9$ 是 3 以 12 为模的补数, 记作

$$-3 \equiv +9 \pmod{12}$$

或者说, 对模 12 而言, -3 和 $+9$ 是互为补数的。同理有

$$-4 \equiv +8 \pmod{12}$$

$$-5 \equiv +7 \pmod{12}$$

即对模 12 而言, $+8$ 和 $+7$ 分别是 -4 和 -5 的补数。可见, 只要确定了“模”, 就可找到一个与负数等价的正数 (该正数即为负数的补数) 来代替此负数, 这样就可把减法运算用加法实现。例如:

设 $A=9$, $B=5$

求: $A-B \pmod{12}$ 。

解: $A-B=9-5=4$ (作减法)

对模 12 而言, -5 可以用其补数 $+7$ 代替, 即

$$-5 \equiv +7 \pmod{12}$$

所以 $A-B=9+7=16$ (作加法)

对模 12 而言, 12 会自动丢失, 所以 16 等价于 4, 即 $4 \equiv 16 \pmod{12}$ 。

进一步分析发现, 3 点、15 点、27 点……在时钟上看见的都是 3 点, 即

$$3 \equiv 15 \equiv 27 \pmod{12}$$

也即 $3 \equiv 3+12 \equiv 3+24 \equiv 3 \pmod{12}$

这说明正数相对于“模”的补数就是正数本身。

上述补数的概念可以用到任意“模”上, 如

$$-3 \equiv +7 \pmod{10}$$

$$+7 \equiv +7 \pmod{10}$$

$$-3 \equiv +97 \pmod{10^2}$$

$$+97 \equiv +97 \pmod{10^2}$$

$$1011 \equiv +0101 \pmod{2^4}$$

$$+0101 \equiv +0101 \pmod{2^4}$$

$$0.1001 \equiv +1.0111 \pmod{2}$$

$$+0.1001 \equiv +0.1001 \pmod{2}$$

由此可得如下结论:

- 一个负数可用它的正补数来代替, 而这个正补数可以用模加上负数本身求得。
- 两个互为补数的数, 它们绝对值之和即为模数。
- 正数的补数即该正数本身。

将补数的概念用到计算机中, 便出现了补码这种机器数。

(2) 补码的定义

整数补码的定义为

$$[x]_{\text{补}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ 2^{n+1} + x & 0 > x \geq -2^n \pmod{2^{n+1}} \end{cases}$$

式中 x 为真值, n 为整数的位数。

例如, 当 $x = +1010$ 时,

$$[x]_{\text{补}} = 0.1010$$

↑

用逗号将符号位和数值部分隔开

当 $x = -1101$ 时,

$$[x]_{\text{补}} = 2^{n+1} + x = 100000 - 1101 = 1,0011$$

↑

用逗号将符号位和数值部分隔开

小数补码的定义为

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 2+x & 0 > x \geq -1 \end{cases} \quad (\text{mod } 2)$$

式中 x 为真值。

例如, 当 $x = 0.1001$ 时, $[x]_{\text{补}} = 0.1001$

当 $x = -0.0110$ 时,

$$[x]_{\text{补}} = 2 + x = 10.0000 - 0.0110 = 1.1010$$

当 $x = 0$ 时,

$$[+0.0000]_{\text{补}} = 0.0000$$

$$[-0.0000]_{\text{补}} = 2 + (-0.0000) = 10.0000 - 0.0000 = 0.0000$$

显然 $[+0]_{\text{补}} = [-0]_{\text{补}} = 0.0000$, 即补码中的“零”只有一种表示形式。

对于小数, 若 $x = -1$, 则根据小数补码定义, 有 $[x]_{\text{补}} = 2 + x = 10.0000 - 1.0000 = 1.0000$ 。可见, -1 本不属于小数范围, 但却有 $[-1]_{\text{补}}$ 存在 (其实在小数补码定义中已指明), 这是由于补码中的零只有一种表示形式, 故它比原码能多表示一个“1”。此外, 根据补码定义, 已知补码还可以求真值, 如

$$\text{若 } [x]_{\text{补}} = 1.0101$$

$$\text{则 } x = [x]_{\text{补}} - 2 = 1.0101 - 10.0000 = -0.1011$$

$$\text{若 } [x]_{\text{补}} = 1.1110$$

$$\text{则 } x = [x]_{\text{补}} - 2^{4+1} = 1.1110 - 100000 = -0010$$

$$\text{若 } [x]_{\text{补}} = 0.1101$$

$$\text{则 } x = [x]_{\text{补}} = 0.1101$$

同理, 当模数为 4 时, 形成了双符号位的补码。如 $x = -0.1001$, 对 $(\text{mod } 2^2)$ 而言,

$$[x]_{\text{补}} = 2^2 + x = 100.0000 - 0.1001 = 11.0111$$

这种双符号位的补码又叫做变形补码, 它在阶码运算和溢出判断中, 有其特殊作用, 后面有关章节中将详细介绍。

由上讨论可知, 引入补码的概念是为了消除减法运算, 但是根据补码的

定义, 在形成补码的过程中又出现了减法。如

$$x = -1011$$

$$[x]_{\text{补}} = 2^{4+1} + x = 100000 - 1011 = 1,0101 \quad (6.1)$$

若我们把模 2^{4+1} 改写成 $2^5 = 100000 = 11111 + 00001$ 时, 则式 (6.1) 可写成:

$$[x]_{\text{补}} = 2^5 + x = 11111 + 00001 + x \quad (6.2)$$

又因 x 是负数, 若 x 用 $-x_1x_2x_3x_4$ 表示, 其中 $x_i (i = 1, 2, 3, 4)$ 不为 0 则为 1, 于是式 (6.2) 可写成:

$$\begin{aligned} [x]_{\text{补}} &= 2^5 + x = 11111 + 00001 - x_1x_2x_3x_4 \\ &= 1\bar{x}_1\bar{x}_2\bar{x}_3\bar{x}_4 + 00001 \end{aligned} \quad (6.3)$$

因为任一位 “1” 减去 x_i 即为 $\bar{x}_i$, 所以式 (6.3) 成立。

由于负数 $-x_1x_2x_3x_4$ 的原码为 $1x_1x_2x_3x_4$, 因此对这个负数求补, 可以看作对它的原码除符号位外, 每位求反, 末位加 1, 简称 “求反加 1”。这样, 由真值通过原码求补码就可避免减法运算。同理, 对于小数也有同样结论, 读者可以自行证明。

“由原码除符号位外, 每位求反, 末位加 1 求补码” 这一规则, 同样适用于由 $[x]_{\text{补}}$ 求 $[x]_{\text{原}}$ 。而对于一个负数, 若对其原码除符号位外, 每位求反 (简称 “每位求反”), 或是对其补码减去末位的 1, 即得机器数的反码。

4. 反码表示法

反码通常用来作为由原码求补码或者由补码求原码的中间过渡。反码的定义如下:

整数反码的定义为:

$$[x]_{\text{反}} = \begin{cases} 0, x & 2^n > x \geq 0 \\ (2^{n+1} - 1) + x & 0 \geq x > -2^n \pmod{(2^{n+1} - 1)} \end{cases}$$

式中 x 为真值, n 为整数的位数。

例如, 当 $x = +1101$ 时,

$$[x]_{\text{反}} = 0,1101$$

↑

用逗号将符号位和数值部分隔开

当 $x = -1101$ 时,

$$[x]_{\text{反}} = (2^{4+1} - 1) + x = 11111 - 1101 = 1,0010$$

↑

用逗号将符号位和数值部分隔开

小数反码的定义为：

$$[x]_{\text{反}} = \begin{cases} x & 1 > x \geq 0 \\ (2-2^{-n})+x & 0 \geq x > -1 \pmod{(2-2^{-n})} \end{cases}$$

式中 x 为真值， n 为小数的位数。

例如，当 $x = +0.0110$ 时， $[x]_{\text{反}} = 0.0110$

当 $x = -0.0110$ 时，

$$[x]_{\text{反}} = (2-2^{-4})+x = 1.1111 - 0.0110 = 1.1001$$

当 $x = 0$ 时，

$$[+0.0000]_{\text{反}} = 0.0000$$

$$[-0.0000]_{\text{反}} = (10.0000 - 0.0001) - 0.0000 = 1.1111$$

可见 $[+0]_{\text{反}}$ 不等于 $[-0]_{\text{反}}$ ，即反码中的“零”也有两种表示形式。

实际上，反码也可看作是 $\text{mod}(2-2^{-n})$ （对于小数）或 $\text{mod}(2^{n+1}-1)$ （对于整数）的补码。与补码相比，仅在末位差 1，因此有些书上称小数的补码为 2 的补码，而称小数的反码为 1 的补码。

综上所述，三种机器数的特点可归纳如下：

- 三种机器数的最高位均为符号位。符号位和数值部分之间可用“.”（对于小数）或“，”（对于整数）隔开。
- 当真值为正时，原码、补码和反码的表示形式均相同，即符号位用“0”表示，数值部分与真值相同。
- 当真值为负时，原码、补码和反码的表示形式不同，但其符号位都用“1”表示，而数值部分有如下关系，即补码是原码的“求反加 1”，反码是原码的“每位求反”。

下面通过实例来进一步理解和掌握三种机器数的表示。

例 6.1 设机器数字长为 8 位（其中一位为符号位），对于整数，当其分别代表无符号数、原码、补码和反码时，对应的真值范围各为多少？

表 6.1 列出了八位寄存器中所有二进制代码组合与无符号数、原码、补码和反码所代表的真值的对应关系。

由此可得出一个结论：由于“零”在补码中只有一种表示形式，故补码比原码和反码可以多表示一个负数。

例 6.2 已知： $[y]_{\text{补}}$ 求： $[-y]_{\text{补}}$ 。

解：设 $[y]_{\text{补}} = y_0y_1y_2\cdots y_n$

第一种情况 $[y]_{\text{补}} = 0.y_1y_2\cdots y_n$ (6.4)

所以 $y = 0.y_1y_2\cdots y_n$

故 $-y = -0.y_1y_2\cdots y_n$

$$\text{则 } [-y]_{\text{补}} = 1.y_1y_2\cdots y_n+2^{-n} \quad (6.5)$$

比较式 (6.4) 和式 (6.5), 发现由 $[y]_{\text{补}}$ 连同符号位在内每位取反, 末位加 1 即可得 $[-y]_{\text{补}}$ 。

$$\text{第二种情况 } [y]_{\text{补}} = 1.y_1y_2\cdots y_n \quad (6.6)$$

$$\text{所以 } [y]_{\text{原}} = 1.\bar{y}_1\bar{y}_2\cdots\bar{y}_n + 2^{-n}$$

$$\text{得 } y = -(0.y_1y_2\cdots y_n+2^{-n})$$

$$\text{故 } -y = 0.y_1y_2\cdots y_n+2^{-n}$$

$$\text{则 } [-y]_{\text{补}} = 0.\bar{y}_1\bar{y}_2\cdots\bar{y}_n+2^{-n} \quad (6.7)$$

比较式 (6.6) 式 (6.7), 发现由 $[y]_{\text{补}}$ 连同符号位在内每位取反, 末位加 1 即可得 $[-y]_{\text{补}}$ 。

可见, 不论真值是正 (第一种情况) 或负 (第二种情况), 由 $[y]_{\text{补}}$ 求 $[-y]_{\text{补}}$ 都是采用“连同符号位在内, 每位取反, 末位加 1”的规则。这一结论在补码减法运算时将经常用到 (详见 6.3 节有关内容)。

有符号数在计算机中除了用原码、补码和反码表示外, 在一些通用计算机中还用另一种机器数——移码表示, 由于它的一些突出的优点, 目前已被广泛采用。

表 6.1 例 6.1 对应的真值范围

二进制代码	无符号数 对应的真值	原码对应 的真值	补码对应 的真值	反码对应 的真值
00000000	0	+0	+0	+0
00000001	1	+1	+1	+1
00000010	2	+2	+2	+2
⋮	⋮	⋮	⋮	⋮
01111110	126	+126	+126	+126
01111111	127	+127	+127	+127
10000000	128	-0	-128	-127
10000001	129	-1	-127	-126
10000010	130	-2	-126	-125
⋮	⋮	⋮	⋮	⋮
11111101	253	-125	-3	-2
11111110	254	-126	-2	-1
11111111	255	-127	-1	-0

5. 移码表示法

当真值用补码表示时, 由于符号位和数值部分一起编码, 与习惯上的表示法不同, 因此人们很难从补码的形式上直接判断其真值的大小, 如:

十进制数 $x=21$ ，对应的二进制数为 $+10101$ ，则 $[x]_{\text{补}} = 0,10101$ ；

十进制数 $x=-21$ ，对应的二进制数为 -10101 ，则 $[x]_{\text{补}} = 1,01011$ ；

十进制数 $x=31$ ，对应的二进制数为 $+11111$ ，则 $[x]_{\text{补}} = 0,11111$ ；

十进制数 $x=-31$ ，对应的二进制数为 -11111 ，则 $[x]_{\text{补}} = 1,00001$ ；

上述补码表示中“,”逗号在计算机内部是不存在的，因此，从代码形式看，符号位也是一位二进制数。按这六位二进制代码比较其大小的话，会得出 $101011 > 010101$ ， $100001 > 011111$ ，其实恰恰相反。

如果我们对每个真值加上一个 2^n (n 为整数的位数)，情况就发生了变化，如：

$x=10101$ 加上 2^5 可得 $10101 + 100000 = 110101$ ；

$x=-10101$ 加上 2^5 可得 $-10101 + 100000 = 001011$ ；

$x=11111$ 加上 2^5 可得 $11111 + 100000 = 111111$ ；

$x=-11111$ 加上 2^5 可得 $-11111 + 100000 = 000001$ ；

比较它们的结果可见， $110101 > 001011$ ， $111111 > 000001$ 。这样一来，从六位代码本身就可看出真值的实际大小。

由此可得移码的定义

$$[x]_{\text{移}} = 2^n + x \quad (2^n > x \geq -2^n)$$

式中 x 为真值， n 为整数的位数。

其实移码就是在真值上加一个常数 2^n 。在数轴上移码所表示的范围恰好对应与真值在数轴上的范围向轴的正方向移动 2^n 个单元。如图 6.1 所示，由此而得移码之称。

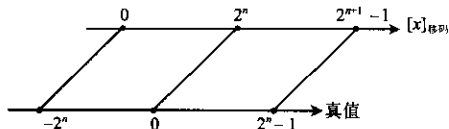


图 6.1 移码在数轴上的表示

例如， $x=10100$

$$[x]_{\text{移}} = 2^5 + 10100 = 1,10100$$

↑

用逗号将符号位和数值部分隔开

$$x = -10100$$

$$[x]_{\text{移}} = 2^5 - 10100 = 0,01100$$

↑

用逗号将符号位和数值部分隔开

当 $x=0$ 时

$$[+0]_{\text{移}} = 2^5 + 0 = 1,00000$$

$$[-0]_{\text{移}} = 2^5 - 0 = 1,00000$$

可见 $[+0]_{\text{移}}$ 等于 $[-0]_{\text{移}}$ ，即移码表示中零也是唯一的。

此外，由移码的定义可见，当 $n=5$ 时，其最小的真值为 $x=-2^5=-100000$ ，则 $[-100000]_{\text{移}} = 2^5 + x = 100000 - 100000 = 0,00000$ ，即最小真值的移码为全 0，这符合人们的习惯。利用移码的这一特点，当浮点数的阶码用移码表示时，就能很方便地判断阶码的大小（详见 6.2.4）。

进一步观察发现，同一个真值的移码和补码仅差一个符号位，若将补码的符号位由“0”改为“1”，或从“1”改为“0”，即可得该真值的移码。表 6.2 列出了真值、补码和移码的对应关系。

表 6.2 真值、补码和移码对照表

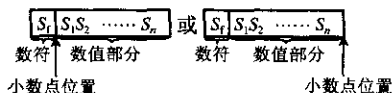
真值 x	$[x]_{\text{补}}$	$[x]_{\text{移}}$	$[x]_{\text{移}}$ 对应的十进制整数
-100000	100000	000000	0
-11111	100001	000001	1
-11110	100010	000010	2
⋮	⋮	⋮	⋮
-00001	111111	011111	31
±00000	000000	100000	32
+00001	000001	100001	33
+00010	000010	100010	34
⋮	⋮	⋮	⋮
+11110	011110	111110	62
+11111	011111	111111	63

6.2 数的定点表示和浮点表示

在计算机中，小数点不用专门的器件表示，而是按约定的方式标出。共有两种方法来表示小数点的存在，即定点表示和浮点表示。定点表示的数称为定点数，浮点表示的数称为浮点数。

6.2.1 定点表示

小数点固定在某一位置的数为定点数，有以下两种格式。



当小数点位于数符和第一数值位之间时, 机器内的数为纯小数; 当小数点位于数值位之后时, 机器内的数为纯整数。采用定点数的机器叫做定点机。数值部分的位数 n 决定了定点机中数的表示范围。若机器数采用原码, 小数定点机中数的表示范围是 $-(1-2^{-n}) \sim (1-2^{-n})$, 整数定点机中数的表示范围是 $-(2^n-1) \sim (2^n-1)$ 。

在定点机中, 由于小数点的位置固定不变, 故当机器处理的数不是纯小数或纯整数时, 必须乘上一个比例因子, 否则会产生“溢出”。

6.2.2 浮点表示

实际上计算机中处理的数不一定是纯小数或纯整数 (如圆周率 3.1416), 而且有些数据的数值范围相差很大 (如电子的质量 9×10^{-28} 克, 太阳的质量 2×10^{33} 克), 它们都不能直接用定点小数或定点整数表示, 但均可用浮点数表示。浮点数即小数点的位置可以浮动的数, 如

$$\begin{aligned} 352.47 &= 3.5247 \times 10^2 \\ &= 3524.7 \times 10^{-1} \\ &= 0.35247 \times 10^3 \end{aligned}$$

显然, 这里小数点的位置是变化的, 但因为分别乘上了不同的 10 的方幂, 故值不变。

通常, 浮点数被表示成

$$N = S \times r^j$$

式中 S 为尾数 (可正可负), j 为阶码 (可正可负), r 是基数 (或基值)。在计算机中, 基数可取 2、4、8 或 16 等。

以基数 $r=2$ 为例, 数 N 可写成下列不同形式:

$$\begin{aligned} N &= 11.0101 \\ &= 0.110101 \times 2^{10} \\ &= 1.10101 \times 2^1 \\ &= 1101.01 \times 2^{-10} \\ &= 0.00110101 \times 2^{100} \\ &\vdots \end{aligned}$$

为了提高数据精度以及便于浮点数的比较, 在计算机中规定浮点数的尾数用纯小数形式, 故上例中 0.110101×2^{10} 和 $0.00110101 \times 2^{100}$ 形式是可以采用的。此外, 将尾数最高位为 1 的浮点数称作规格化数, 即 $N = 0.110101 \times 2^{10}$ 为浮点数的规格化形式。浮点数表示成规格化形式后, 其精度最高。

1. 浮点数的表示形式

浮点数在机器中的形式如下所示。采用这种数据格式的机器叫做浮点机。



可见浮点数由阶码 j 和尾数 S 两部分组成。阶码是整数，阶符和阶码的位数 m 合起来反映浮点数的表示范围及小数点的实际位置；尾数是小数，其位数 n 反映了浮点数的精度；尾数的符号 S_1 代表浮点数的正负。

2. 浮点数的表示范围

以通式 $N = S \times r^j$ 为例，设浮点数阶码的数值位取 m 位，尾数的数值位取 n 位，当浮点数为非规格化数时，它在数轴上的表示范围如图 6.2 所示。

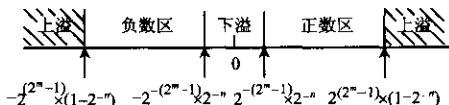


图 6.2 浮点数在数轴上的表示范围

由图可见，其最大正数为 $2^{(2^m-1)} \times (1-2^{-n})$ ；最小正数为 $2^{-(2^m-1)} \times 2^{-n}$ ；

最大负数为 $-2^{(2^m-1)} \times 2^{-n}$ ；最小负数为 $-2^{(2^m-1)} \times (1-2^{-n})$ 。当浮点数阶码大于最大阶码时，称为“上溢”，此时机器停止运算，进行中断溢出处理；当浮点数阶码小于最小阶码时，称为“下溢”，此时“溢出”的数绝对值很小，通常将尾数各位置为零，按机器零处理，此时机器可以继续运行。

一旦浮点数的位数确定后，合理分配阶码和尾数的位数，直接影响浮点数的表示范围和精度。通常对于短实数（总位数为 32 位），阶码取 8 位（含阶符一位），尾数取 24 位（含数符一位）；对于长实数（总位数为 64 位），阶码取 11 位（含阶符一位），尾数取 53 位（含数符一位）；对于临时实数（总位数为 80 位），阶码取 15 位（含阶符一位），尾数取 65 位（含数符一位）。

3. 浮点数的规格化

为了提高浮点数的精度，其尾数必须为规格化数。如果不是规格化数，就要通过修改阶码并同时左右移尾数的办法，使其变成规格化数。将非规格化数转换成规格化数的过程叫做规格化。对于基数不同的浮点数，因其规格化数的形式不同，规格化过程也不同。

当基数为 2 时, 尾数最高位为 1 的数为规格化数。规格化时, 尾数左移一位, 阶码减 1, (这种规格化叫做向左规格化, 简称左规); 尾数右移一位, 阶码加 1 (这种规格化叫做向右规格化, 简称右规)。图 6.2 所示的浮点数规格化后, 其最大正数为 $2^{(2^n-1)} \times (1-2^{-n})$; 最小正数为 $2^{(2^n-1)} \times 2^{-1}$; 最大负数为 $-2^{(2^n-1)} \times 2^{-1}$; 最小负数为 $-2^{(2^n-1)} \times (1-2^{-n})$ 。

当基数为 4 时, 尾数的最高两位不全为零的数为规格化数。规格化时, 尾数左移两位, 阶码减 1; 尾数右移两位, 阶码加 1。

当基数为 8 时, 尾数的最高三位不全为零的数为规格化数。规格化时, 尾数左移三位, 阶码减 1; 尾数右移三位, 阶码加 1。

同理类推, 不难得到基数为 16 或 2^m 时的规格化过程。

浮点机中一旦基数确定后就不再变了, 而且基数是隐含的, 故不同基数的浮点数其表示形式完全相同。但基数不同, 对数的表示范围和精度等都受影响。一般来说, 基数 r 越大, 可表示的浮点数范围越宽, 而且所表示的数其个数越多。但 r 越大, 浮点数的精度反而下降。如 $r=16$ 的浮点数, 因其规格化数的尾数最高三位可能出现零, 故与其尾数位数相同的 $r=2$ 的浮点数相比, 后者可能比前者多三位精度。

6.2.3 定点数和浮点数的比较

定点数和浮点数可从如下几个方面进行比较:

(1) 当浮点机和定点机中的数其位数相同时, 浮点数的表示范围比定点数大得多。

(2) 当浮点数为规格化数时, 其精度远比定点数高。

(3) 浮点数运算要分阶码部分和尾数部分, 而且运算结果都要求规格化, 故浮点运算步骤比定点运算步骤多, 运算速度比定点低, 运算线路比定点复杂。

(4) 在溢出的判断方法上, 浮点数是对规格化数的阶码进行判断, 而定点数是对数值本身进行判断。如小数定点机中的数其绝对值必须小于 1, 否则即“溢出”, 此时要求机器停止运算, 进行处理。为了防止溢出, 上机前必须选择比例因子, 这个工作比较麻烦, 给编程带来不便。而浮点数的表示范围远比定点数大, 仅当“上溢”时机器才停止运算, 故一般不必考虑比例因子的选择。

总之, 浮点数在数的表示范围、数的精度、溢出处理和程序编程方面 (不取比例因子) 均优于定点数。但在运算规则、运算速度及硬件成本方面又不如定点数。因此, 究竟选用定点数还是浮点数, 应根据具体应用综合考虑。一般来说, 通用的大型计算机大多采用浮点数, 或同时采用定、浮点数; 小型、微

型及某些专用机、控制机则大多采用定点数。当需要作浮点运算时，可通过软件实现，也可外加浮点扩展硬件（如协处理器）来实现。

6.2.4 举例

例 6.3 将十进制数 $+\frac{13}{128}$ 写成二进制定点数和浮点数（数值部分取 10 位，

阶码部分取 4 位，阶符和数符各取 1 位），分别写出它在定点机和浮点机中的机器数形式。

解：令 $x = +\frac{13}{128}$

其二进制形式： $x = 0.0001101000$

定点数表示： $x = 0.0001101000$

浮点数规格化表示： $x = 0.1101000000 \times 2^{-11}$

定点机中 $[x]_{原} = [x]_{补} = [x]_{反} = 0.0001101000$

浮点机中

$[x]_{原}$:

1	0011	0	1101000000
---	------	---	------------

 或写成 1,0011; 0.1101000000

$[x]_{补}$:

1	1101	0	1101000000
---	------	---	------------

 或写成 1,1101; 0.1101000000

例 6.4 将十进制数 -54 表示成二进制定点数和浮点数，并写出它在定点机和浮点机中的机器数形式（其他要求同上例）。

解：令 $x = -54$

其二进制形式： $x = -110110$

定点数表示： $x = -0000110110$

浮点数规格化表示 $x = -(0.1101100000) \times 2^{110}$

定点机中

$[x]_{原} = 1,0000110110$

$[x]_{补} = 1,1111001010$

$[x]_{反} = 1,1111001001$

浮点机中

$[x]_{原} = 0,0110; 1.1101100000$

$[x]_{补} = 0,0110; 1.0010100000$

$[x]_{反} = 0,0110; 1.0010011111$

例 6.5 写出对应图 6.2 所示的浮点数的补码形式。设图中 $n = 10$, $m = 4$, 阶符、数符各取 1 位。

解:	真值	补码
最大正数	$2^{15} \times (1-2^{-10})$	0,1111; 0.1111111111
最小正数	$2^{-15} \times 2^{-10}$	1,0001; 0.0000000001
最大负数	$-2^{-15} \times 2^{-10}$	1,0001; 1.1111111111
最小负数	$-2^{15} \times (1-2^{-10})$	0,1111; 1.0000000001

计算机中浮点数的阶码和尾数可以采用同一种机器数表示，也可采用不同的机器数表示。

例 6.6 设浮点数字长为 16 位，其中阶码为 5 位（含一位阶符），尾数为 11 位（含一位数符），写出 $-\frac{53}{512}$ 对应的浮点规格化数的原码、补码、反码和阶码用移码，尾数用补码的形式。

解： 设 $x = -\frac{53}{512} = -0.000110101 = 2^{-11} \times (-0.1101010000)$

$[x]_{\text{原}}: 1,0011; 1.1101010000$

$[x]_{\text{补}}: 1,1101; 1.0010110000$

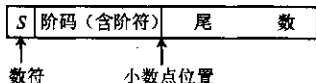
$[x]_{\text{反}}: 1,1100; 1.0010101111$

$[x]_{\text{阶移, 尾补}}: 0,1101; 1.0010110000$

值得注意的是，当一个浮点数尾数为 0 时，不论其阶码为何值；或阶码等于或小于它所能表示的最小数时，不管其尾数为何值，机器都把该浮点数当作零看待，并称之为“机器零”。如果浮点数的阶码用移码表示，尾数用补码表示，则当阶码为它所能表示的最小数 2^{-m} （式中 m 为阶码的位数）且尾数为 0 时，其阶码（移码）全为 0，尾数（补码）也全为 0，这样的机器零为 0000……0000 全零表示，有利于简化机器中判“0”电路。

6.2.5 IEEE 754 标准

现代计算机中，浮点数一般采用 IEEE 制定的国际标准，这种标准形式如下：



按 IEEE 标准，常用的浮点数有三种：

	符号位 S	阶码	尾数	总位数
短实数	1	8	23	32
长实数	1	11	52	64
临时实数	1	15	64	80

其中 S 为数符, 它表示浮点数的正负, 但与其有效位(尾数)是分开的。阶码用移码表示, 阶码的真值都被加上一个常数(偏移量), 如短实数、长实数和临时实数的偏移量分别用十六进制(见附录 6A.1)表示为 7FH、3FFH 和 3FFFFH。尾数部分通常都是规格化表示, 即非“0”的有效位最高位总是“1”, 但在 IEEE 标准中, 有效位呈如下形式:

1▲ffff.....fff

其中▲表示假想的二进制小数点。在实际表示中, 对短实数和长实数, 这个整数位的 1 省略, 称隐藏位; 对于临时实数不采用隐藏位方案, 表 6.3 列出了十进制数 178.125 的实数表示。

表 6.3 实数 178.125 的几种不同表示

实数表示	数 值		
原始十进制数	178.125		
二进制数	10110010.001		
二进制浮点表示	$1.0110010001 \times 2^{11}$		
短实数表示	符号	偏移的阶码	有效值
	0	00000111+01111111 =10000110	0110100010000000000000 ↑ 1▲ (隐含的)

6.3 定点运算

定点运算包括移位、加、减、乘、除几种。

6.3.1 移位运算

1. 移位的意义

移位运算在日常生活中常见。例如 15 米可写作 1 500 厘米, 单就数字而言, 1500 相当于数 15 相对于小数点左移了两位, 并在小数点前面添了两个 0; 同样 15 也相当于 1500 相对于小数点右移了两位, 并删去了小数点后面的两个 0。可见, 当某个十进制数相对于小数点左移 n 位时, 相当于该数乘以 10^n ; 右移 n 位时, 相当于该数除以 10^n 。

计算机中小数点的位置是事先约定的, 因此, 二进制表示的机器数在相对于小数点作 n 位左移或右移时, 其实质就使该数乘以或除以 $2^n (n=1, 2, \dots, n)$ 。

移位运算又叫移位操作, 对计算机来说, 有很大的实用价值。例如, 当某计算机没有乘(除)法运算线路时, 可以采用移位和加法相结合, 实现乘(除)运算。

计算机中机器数的字长往往是固定的, 当机器数左移 n 位或右移 n 位时,

必然会使其 n 位低位或 n 位高位出现空位。那么, 对空出的空位应该添补 0 还是 1 呢? 这与机器数采用有符号数还是无符号数有关。对有符号数的移位叫算术移位。

2. 算术移位规则

对于正数, 由于 $[x]_{\text{原}} = [x]_{\text{补}} = [x]_{\text{反}} = \text{真值}$, 故移位后出现的空位均以 0 添之。对于负数, 由于原码、补码和反码的表示形式不同, 故当机器数移位时, 对其空位的添补规则也不同。表 6.4 列出了三种不同码制的机器数 (整数或小数均可), 分别对应正数或负数, 移位后的添补规则。必须注意的是: 不论是正数还是负数, 移位后其符号位均不变, 这是算术移位的重要特点。

表 6.4 不同码制机器数移位后的空位添补规则

	码 制	添补代码
正数	原码、补码、反码	0
负数	原 码	0
	补 码	左移添 0
		右移添 1
	反 码	1

由表 6.4 可得出如下结论:

- (1) 机器数为正时, 不论左移或右移, 添补代码均为 0。
- (2) 由于负数的原码其数值部分与真值相同, 故在移位时只要使符号位不变, 其空位均添 0。
- (3) 由于负数的反码其各位除符号位外与负数的原码正好相反, 故移位后所添的代码应与原码相反, 即全部添 1。
- (4) 分析任意负数的补码可发现, 当对其由低位向高位找到第一个“1”时, 在此“1”左边的各位均与对应的反码相同, 而在此“1”右边的各位 (包括此“1”在内) 均与对应的原码相同。故负数的补码左移时, 因空位出现在低位, 则添补的代码与原码相同, 即添 0; 右移时因空位出现在高位, 则添补的代码应与反码相同, 即添 1。

例 6.7 设机器数字长为 8 位 (含一位符号位), 若 $A = \pm 26$, 写出三种机器数左、右移一位和两位后的表示形式及对应的真值, 并分析结果的正确性。

解: (1) $A = +26 = (+11010)_{\text{二}}$

则 $[A]_{\text{原}} = [A]_{\text{补}} = [A]_{\text{反}} = 0, 0011010$

移位结果示于表 6.5。

表 6.5 对 $A=+26$ 移位后的结果

移位操作	机 器 数	对应的真值
	$[A]_{原}=[A]_{补}=[A]_{反}$	
移位前	0,0011010	+26
左移一位	0,0110100	+52
左移两位	0,1101000	+104
右移一位	0,0001101	+13
右移两位	0,0000110	+6

可见，对于正数，三种机器数移位后符号位均不变，左移时最高数位丢 1，结果出错；右移时最低数位丢 1，影响精度。

(2) $A=-26=(-11010)_2$

三种机器数移位结果示于表 6.6。

表 6.6 对 $A=-26$ 移位后的结果

移位操作	机 器 数		对应的真值
移位前	原 码	1,0011010	-26
左移一位		1,0110100	-52
左移两位		1,1101000	-104
右移一位		1,0001101	-13
右移两位		1,0000110	-6
移位前	补 码	1,1100110	-26
左移一位		1,1001100	-52
左移两位		1,0011000	-104
右移一位		1,1110011	-13
右移两位		1,1111001	-7
移位前	反 码	1,1100101	-26
左移一位		1,1001011	-52
左移两位		1,0010111	-104
右移一位		1,1110010	-13
右移两位		1,1111001	-6

可见，对于负数，三种机器数移位后符号位均不变。负数的原码左移时，高位丢 1，结果出错；低位丢 1，影响精度。负数的补码左移时，高位丢 0，

结果出错；低位丢 1，影响精度。负数的反码左移时，高位丢 0，结果出错；低位丢 0，影响精度。

图 6.3 示意了机器中实现算术左移和右移操作的硬件框图。其中 (a) 真值为正的三种机器数的移位操作；(b) 负数原码的移位操作；(c) 负数补码的移位操作；(d) 负数反码的移位操作。

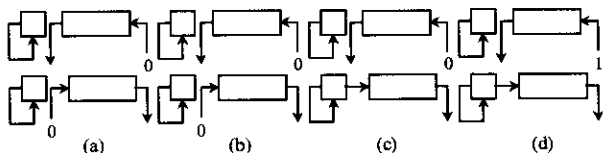


图 6.3 实现算术左移和右移操作的硬件示意图

3. 算术移位和逻辑移位的区别

有符号数的移位称为算术移位，无符号数的移位称为逻辑移位。逻辑移位的规则是：逻辑左移时，高位移出，低位添 0；逻辑右移时，低位移出，高位添 0。例如，寄存器内容为 01010011，逻辑左移为 10100110，算术左移为 00100110（最高数位“1”移丢）。又如寄存器内容为 10110010，逻辑右移为 01011001。若将其视为补码，算术右移为 11011001。显然，两种移位的结果是不同的。上例中为了避免算术左移时最高数位丢 1，可采用带进位 (C_y) 的移位，其示意图如图 6.4 所示。算术左移时，符号位移至 C_y ，最高数位就可避免移出。

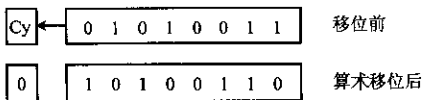


图 6.4 用带进位的移位实现算术左移

6.3.2 加法与减法运算

加减法运算是计算机中最基本的运算，因减法运算可看作被减数加上一个减数的负值，即 $A - B = A + (-B)$ ，故在此将机器中的减法运算和加法运算合在一起讨论。现代计算机中都采用补码作加减法运算。

1. 补码加减运算的基本公式

补码加法的基本公式为：

整数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A]_{\text{补}} + [B]_{\text{补}} = [A+B]_{\text{补}} \pmod{2}$

即补码表示的两个数在进行加法运算时，可以把符号位与数位同等处理，只要结果不超出机器能表示的数值范围，运算后的结果按 2^{n+1} 取模（对于整数）或按 2 取模（对于小数），就能得到本次加法的运算结果。

读者可根据补码定义，按两个操作数的四种正负组合情况加以证明。

对于减法因 $A-B=A+(-B)$

则 $[A-B]_{\text{补}} = [A+(-B)]_{\text{补}}$

由补码加法基本公式可得：

整数 $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2^{n+1}}$

小数 $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} \pmod{2}$

因此，若机器数采用补码，当求 $A-B$ 时，只需先求 $[-B]_{\text{补}}$ （称 $[-B]_{\text{补}}$ 为“求补”后的减数），就可按补码加法规则进行运算。而 $[-B]_{\text{补}}$ 由 $[B]_{\text{补}}$ 连同符号位在内，每位取反，末位加 1 而得。

例 6.8 已知： $A=0.1011$ ， $B=-0.0101$ 求： $[A+B]_{\text{补}}$

解：因为 $A=0.1011$ ， $B=-0.0101$

所以 $[A]_{\text{补}}=0.1011$ ， $[B]_{\text{补}}=1.1011$

则 $[A]_{\text{补}} + [B]_{\text{补}} = 0.1011$

$$\begin{array}{r} +1.1011 \\ \hline \boxed{1} \quad 0.0110 = [A+B]_{\text{补}} \\ \text{丢掉} \leftarrow \end{array}$$

按模 2 的意义，最左边的 1 丢掉，故 $[A+B]_{\text{补}}=0.0110$ 结果正确。

例 6.9 已知： $A=-1001$ ， $B=-0101$ 求： $[A+B]_{\text{补}}$

解：因为 $A=-1001$ ， $B=-0101$

所以 $[A]_{\text{补}}=1,0111$ ， $[B]_{\text{补}}=1,1011$

则 $[A]_{\text{补}} + [B]_{\text{补}} = 1,0111$

$$\begin{array}{r} +1,1011 \\ \hline \boxed{1} \quad 1,0010 = [A+B]_{\text{补}} \\ \text{丢掉} \leftarrow \end{array}$$

按模 2^{n+1} 的意义，最左边的 1 丢掉。

例 6.10 设机器数字长为 8 位（含一位符号位在内），若 $A=+15$ ， $B=+24$ ，求 $[A-B]_{\text{补}}$ 并还原成真值。

解：因为 $A=+15=+0001111$ ， $B=+24=+0011000$

所以 $[A]_{\text{补}}=0,0001111$ ， $[B]_{\text{补}}=0,0011000$ ， $[-B]_{\text{补}}=1,1101000$

则 $[A-B]_{\text{补}} = [A]_{\text{补}} + [-B]_{\text{补}} = 0,0001111$

$$\begin{array}{r} +1,1101000 \\ \hline 1,1101011 \end{array}$$

所以 $[A-B]_{\text{补}} = 1,1110111$

故 $A-B = -0001001 = -9$

可见,不论操作数是正还是负,在做补码加减法时,只需将符号位和数值部分一起参加运算,并且将符号位产生的进位自然丢掉即可。

例 6.11 设机器数字长为 8 位,其中一位为符号位,令 $A = -93$, $B = +45$, 求 $[A-B]_{\text{补}}$ 。

解: 由 $A = -93 = -1011101$, 得 $[A]_{\text{补}} = 1,0100011$

由 $B = +45 = +0101101$, 得 $[B]_{\text{补}} = 0,0101101$, $[-B]_{\text{补}} = 1,1010011$

$$\begin{array}{r}
 [A]_{\text{补}} = \quad 1,0100011 \\
 + [-B]_{\text{补}} = \quad 1,1010011 \\
 \hline
 [A]_{\text{补}} + [-B]_{\text{补}} = \boxed{1} 0,1110110 = [A-B]_{\text{补}} \\
 \text{丢掉} \leftarrow
 \end{array}$$

按模 2^{n+1} 的意义,最左边的“1”自然丢掉,故 $[A-B]_{\text{补}} = 0,1110110$,还原成真值得 $A-B=118$,结果出错,这是因为 $A-B=-138$ 超出了机器字长所能表示的范围。在计算机中,这种超出机器字长的现象,叫溢出。为此,在补码定点加减运算过程中,必须对结果是否溢出作出明确的判断。

2. 溢出判断

补码定点加减运算判断溢出有两种方法。

(1) 用一位符号位判断溢出

对于加法,只有在正数加正数和负数加负数两种情况下才可能出现溢出,符号不同的两个数相加是不会出现溢出的。

对于减法,只有在正数减负数或负数减正数两种情况下才可能出现溢出,符号相同的两个数相减是不会出现溢出的。

下面以机器字长为 4 位(含一位符号位)为例,说明机器是如何判断溢出的。

机器字长为 4 位的补码其所对应的真值范围为 -8 至 +7,运算结果一旦超过这个范围即为溢出。表 6.7 列出了四种溢出情况。

由于减法运算在机器中是用加法器实现的,因此可得出如下结论:不论是作加法还是减法,只要实际参加操作的两个数(减法时即为被减数和“求补”以后的减数)符号相同,结果又与原操作数的符号不同,即为溢出。

例 6.12 已知: $A = -\frac{11}{16}$, $B = -\frac{7}{16}$ 求: $[A+B]_{\text{补}}$

解: 由 $A = -\frac{11}{16} = -0.1011$, $B = -\frac{7}{16} = -0.011$

得 $[A]_{\text{补}} = 1.0101$, $[B]_{\text{补}} = 1.1001$

所以 $[A+B]_{\text{补}} = [A]_{\text{补}} = 1.0101$

$$\begin{array}{r} + [B]_{\text{补}} = 1.1001 \\ \hline 1 \quad 0.1110 \\ \text{丢掉} \end{array}$$

两操作数符号均为 1, 结果的符号为 0, 故为溢出。

表 6.7 补码定点运算溢出判断举例

真 值	补码运算
$A = 5$ $+ B = 4$ $A+B = 9 > 7$ 溢出	$[A]_{\text{补}} = 0,101$ $+ [B]_{\text{补}} = 0,100$ $[A+B]_{\text{补}} = 1,001$
$A = -5$ $+ B = -4$ $A+B = -9 < -8$ 溢出	$[A]_{\text{补}} = 1,001$ $+ [B]_{\text{补}} = 1,100$ $[A+B]_{\text{补}} = 10,111$
$A = 5$ $- B = -4$ $A-B = 9 > 7$ 溢出	$[A]_{\text{补}} = 0,101$ $+ [-B]_{\text{补}} = 0,100$ $[A-B]_{\text{补}} = 1,001$
$A = -5$ $- B = +4$ $A-B = -9 < -8$ 溢出	$[A]_{\text{补}} = 1,011$ $+ [-B]_{\text{补}} = 1,100$ $[A-B]_{\text{补}} = 10,111$

例 6.13 已知: $A = -0.1000$, $B = -0.1000$ 求: $[A+B]_{\text{补}}$

解: 由 $A = -0.1000$, $B = -0.1000$

得 $[A]_{\text{补}} = 1.1000$, $[B]_{\text{补}} = 1.1000$

所以 $[A+B]_{\text{补}} = [A]_{\text{补}} = 1.1000$

$$\begin{array}{r} + [B]_{\text{补}} = 1.1000 \\ \hline 1 \quad 1.0000 \\ \text{丢掉} \end{array}$$

结果的符号同原操作数符号, 故未溢出。

由 $[A+B]_{\text{补}} = 1.0000$, 得 $A+B = -1$, 由此可见, 用补码表示定点小数时, 它能表示 -1 的值。

计算机中采用一位符号位判断时, 为了节省时间, 通常用符号位产生的进位与最高有效位产生的进位异或操作后, 按其结果进行判断。若异或结果为 1, 即为溢出; 异或结果为 0, 则无溢出。上例 6.12 中符号位有进位, 最高有效位无进位, 即 $1 \oplus 0 = 1$, 故溢出。例 6.13 中符号位有进位, 最高有效位也有

进位, 即 $1 \oplus 1 = 0$, 故无溢出。

(2) 用两位符号位判断溢出

在 6.1.2 中已提到过两位符号位的补码, 即变形补码, 它是以 4 为模的, 其定义为

$$[x]_{\text{补}} = \begin{cases} x & 1 > x \geq 0 \\ 4 + x & 0 > x \geq -1 \pmod{4} \end{cases}$$

在用变形补码作加法时, 两位符号位要连同数值部分一起参加运算, 而且高位符号位产生的进位自动丢失, 便可得正确结果。即

$$[x]_{\text{补}} + [y]_{\text{补}} = [x + y]_{\text{补}} \pmod{4}$$

变形补码判断溢出的原则是: 当两位符号位不同时, 表示溢出, 否则, 无溢出。不论是否发生溢出, 高位(第一位)符号位永远代表真正的符号。

例 6.14 设 $x = +\frac{11}{16}$, $y = +\frac{3}{16}$, 试用变形补码计算 $x+y$ 。

解: 因为 $x = +\frac{11}{16} = 0.1011$, $y = +\frac{3}{16} = 0.0011$

所以 $[x]_{\text{补}} = 00.1011$, $[y]_{\text{补}} = 00.0011$

则 $[x]_{\text{补}} + [y]_{\text{补}} = 00.1011$

$$\begin{array}{r} + 00.0011 \\ 00.1011 \end{array}$$

故 $[x+y]_{\text{补}} = 00.1110$

$$x+y = 0.1110$$

例 6.15 设 $x = +\frac{11}{16}$, $y = +\frac{7}{16}$, 试用变形补码计算 $x+y$ 。

解: 因为 $x = +\frac{11}{16} = 0.1011$, $y = +\frac{7}{16} = 0.0111$

所以 $[x]_{\text{补}} = 00.1011$, $[y]_{\text{补}} = 00.0111$

则 $[x]_{\text{补}} + [y]_{\text{补}} = 00.1011$

$$\begin{array}{r} + 00.0111 \\ 00.1011 \\ \hline 01.0010 \end{array}$$

溢出

此时, 符号位为“01”, 表示溢出, 又因第一位符号位为“0”, 表示结果的真正符号, 故“01”表示正溢出。

例 6.16 设 $x = -\frac{11}{16}$, $y = -\frac{7}{16}$, 用变形补码计算 $x+y$ 。

解: 因为 $x = -\frac{11}{16} = -0.1011$, $y = -\frac{7}{16} = -0.0111$

所以 $[x]_{\text{补}} = 11.0101$, $[y]_{\text{补}} = 11.1001$

则 $[x]_{\text{补}} + [y]_{\text{补}} = 11.0101$

$$\begin{array}{r} + 11.1001 \\ \hline 10.1110 \end{array}$$

丢掉 ←

符号位为“10”，表示溢出。由于第一符号位为1，则表示负溢出。

上述结论对于整数也同样适用。在浮点机中，当阶码用两位符号位表示时，判断溢出的原则与小数完全相同。

这里需要说明一点，采用双符号位方案时，寄存器或主存中的操作数只需保存一位符号位即可。因为任何正确的数其两个符号位的值总是相同的。而双符号位在加法器中又是必要的，故在相加时，寄存器中一位符号的值要同时送到加法器的两位符号位的输入端。

3. 补码定点加减法所需的硬件配置

图 6.5 示出了实现补码定点加减法的基本硬件配置。

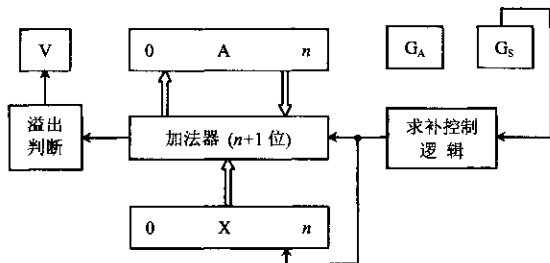


图 6.5 补码定点加减法硬件配置

图中寄存器 A、X、加法器的位数相等，其中 A 存放被加数（或被减数）的补码，X 存放加数（或减数）的补码。当作减法时，由“求补控制逻辑”将 X 送至加法器，并使加法器的最末位外来进位为 1，以达到对减数求补的目的。运算结果溢出时，通过溢出判断电路置“1”溢出标记 V。 G_A 为加法标记， G_S 为减法标记。

4. 补码加减运算控制流程

图 6.6 示出了补码加减运算控制流程。

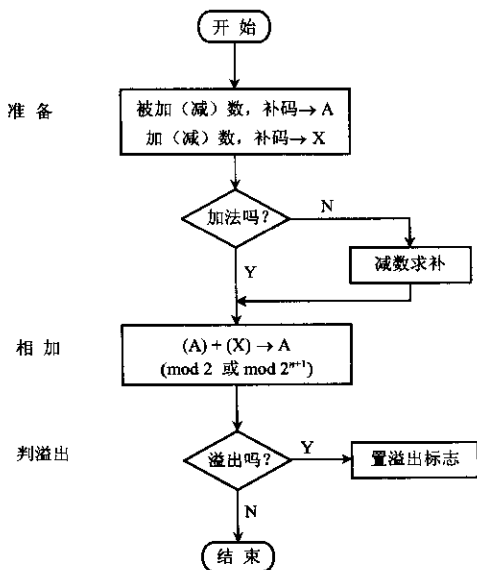


图 6.6 补码加减运算控制流程

由图可见，加（减）法运算前，被加（减）数的补码在 A 中，加（减）数的补码在 X 中。若是加法直接完成 $(A) + (X) \rightarrow A \pmod{2 \text{ 或 } \pmod{2^{n+1}}}$ 的运算；若是减法则需对减数求补，再和 A 寄存器的内容相加，结果送 A。最后完成溢出判断。

6.3.3 乘法运算

在计算机中，乘法运算是一种很重要的运算，有的机器由硬件乘法器直接完成乘法运算，有的机器内没有乘法器，但可以按机器作乘法运算的方法，用软件编程实现。因此，学习乘法运算方法不仅有助于乘法器的设计，也有助于乘法编程。

下面从分析笔算乘法入手，介绍机器中用到的几种乘法运算方法。

1. 分析笔算乘法

设 $A=0.1101$, $B=0.1011$, 求 $A \times B$ 。

笔算乘法时，乘积的符号由两数符号心算而得：正正得正；其数值部分的运算如下：

$$\begin{array}{r}
 0.1101 \\
 \times 0.1011 \\
 \hline
 1101 \cdots \cdots A \times 2^0 \quad A \text{ 不移位} \\
 1101 \cdots \cdots A \times 2^1 \quad A \text{ 左移 1 位} \\
 0000 \cdots \cdots 0 \times 2^2 \quad 0 \text{ 左移 2 位} \\
 1101 \cdots \cdots A \times 2^3 \quad A \text{ 左移 3 位} \\
 \hline
 0.10001111
 \end{array}$$

所以 $A \times B = +0.10001111$

可见，这里包含着被乘数 A 的多次左移，以及四个位积的相加运算。

若计算机完全模仿笔算乘法步骤，将会有两大困难：其一，将四个位积一次相加，机器难以实现；其二，乘积位数增长了一倍，这将造成器材的浪费和运算时间的增加。为此，对笔算乘法做些改进。

2. 笔算乘法的改进

将 $A \cdot B = A \cdot 0.1011$

$$\begin{aligned}
 &= 0.1A + 0.001A + 0.0001A \\
 &= 0.1A + 0.00A + 0.001(A + 0.1A) \\
 &= 0.1A + 0.01[0A + 0.1(A + 0.1A)] \\
 &= 0.1\{A + 0.1[0A + 0.1(A + 0.1A)]\} \\
 &= 2^{-1}\{A + 2^{-1}[0A + 2^{-1}(A + 2^{-1}A)]\} \\
 &= 2^{-1}\{A + 2^{-1}[0A + 2^{-1}(A + 2^{-1}(A + 0))] \} \quad (6.8)
 \end{aligned}$$

由式 (6.8) 可见，两数相乘的过程，可视为加法和移位（乘 2^{-1} 相当于做一位右移）两种运算，这对计算机来说是非常容易实现的。

从初始值为 0 开始，对式 (6.8) 作分步运算，则

$$\begin{aligned}
 \text{第一步：被乘数加零} \quad & A + 0 = 0.1101 + 0.0000 = 0.1101 \\
 \text{第二步：右移一位，得新的部分积} \quad & 2^{-1}(A + 0) = 0.01101 \\
 \text{第三步：被乘数加部分积} \quad & A + 2^{-1}(A + 0) = 0.1101 + 0.01101 = 1.00111 \\
 \text{第四步：右移一位，得新的部分积} \quad & 2^{-1}[A + 2^{-1}(A + 0)] = 0.100111 \\
 \text{第五步：} \quad & 0A + 2^{-1}[A + 2^{-1}(A + 0)] = 0.100111 \\
 \text{第六步：} \quad & 2^{-1}\{0A + 2^{-1}[A + 2^{-1}(A + 0)]\} = 0.0100111 \\
 \text{第七步：} \quad & A + 2^{-1}\{0A + 2^{-1}[A + 2^{-1}(A + 0)]\} = 1.0001111 \\
 \text{第八步：} \quad & 2^{-1}\{A + 2^{-1}[0A + 2^{-1}(A + 2^{-1}(A + 0))]\} = 0.10001111
 \end{aligned}$$

表 6.8 列出了式 (6.8) 的全部运算过程。

上述运算过程可归纳为：

① 乘法运算可用移位和加法来实现，当两个四位数相乘，总共需做四次加法和四次移位。

② 由乘数的末位值确定被乘数是否与原部分积相加，然后右移一位，形成新的部分积；同时，乘数也右移一位，由次低位作新的末位，空出最高位放

表 6.8 式 (6.8) 的运算过程

部分积	乘数	说 明
$\begin{array}{r} 0.0000 \\ + 0.1101 \end{array}$	1011	初始条件, 部分积为 0 乘数为 1, 加被乘数
$\begin{array}{r} 0.1101 \\ 0.0110 \\ + 0.1101 \end{array}$	1101	$\rightarrow 1$ 位, 形成新的部分积; 乘数同时 $\rightarrow 1$ 位 乘数为 1, 加被乘数
$\begin{array}{r} 1.0011 \\ 0.1001 \\ + 0.0000 \end{array}$	1110	$\rightarrow 1$ 位, 形成新的部分积; 乘数同时 $\rightarrow 1$ 位 乘数为 0, 加上 0
$\begin{array}{r} 0.1001 \\ 0.0100 \\ + 0.1101 \end{array}$	1111	$\rightarrow 1$ 位, 形成新的部分积; 乘数同时 $\rightarrow 1$ 位 乘数为 1, 加被乘数
$\begin{array}{r} 1.0001 \\ 0.1000 \end{array}$	1111	$\rightarrow 1$ 位, 形成最终结果

部分积的最低位。

③ 每次做加法时, 被乘数仅仅与原部分积的高位相加, 其低位被移至乘数所空出的高位位置。

计算机很容易实现这种运算规则。用一个寄存器存放被乘数, 一个寄存器存放乘积的高位, 另一个寄存器存放乘数及乘积的低位, 再配上加法器及其他相应电路, 就可组成乘法器。又因加法只在部分积的高位进行, 故不但节省了器材, 而且还缩短了运算时间。

3. 原码乘法

由于原码表示与真值极为相似, 只差一个符号, 而乘积的符号又可通过两数符号的逻辑异或求得, 因此, 上述讨论的结果可以直接用于原码一位乘, 只需加上符号位处理即可。

(1) 原码一位乘运算规则

以小数为例

$$\text{设 } [x]_{\text{原}} = x_0.x_1x_2\cdots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2\cdots y_n$$

$$\text{则 } [x]_{\text{原}} [y]_{\text{原}} = x_0 \oplus y_0 (0.x_1x_2\cdots x_n)(0.y_1y_2\cdots y_n)$$

式中 $0.x_1x_2\cdots x_n$ 为 x 的绝对值, 记作 x^*

$0.y_1y_2\cdots y_n$ 为 y 的绝对值, 记作 y^*

原码一位乘的运算规则为:

- ① 乘积的符号位由两原码符号位异或运算结果决定。
- ② 乘积的数值部分由两数绝对值相乘, 其通式为:

$$\left. \begin{aligned}
 x^* \cdot y^* &= x^*(0.y_1y_2\cdots y_n) \\
 &= x^*(y_12^{-1} + y_22^{-2} + \cdots + y_n2^{-n}) \\
 &= 2^{-1}(y_1x^* + 2^{-1}(y_2x^* + 2^{-1}(\cdots + 2^{-1}(y_{n-1}x^* + 2^{-1}(y_nx^* + 0)\cdots)))
 \end{aligned} \right\} \quad (6.9)$$

$\underbrace{\hspace{10em}}_{z_0}$
 $\underbrace{\hspace{5em}}_{z_1}$
 $\underbrace{\hspace{5em}}_{z_2}$
 $\cdots$
 $\underbrace{\hspace{10em}}_{z_{n-1}}$
 $\underbrace{\hspace{10em}}_{z_n}$

再令 z_i 表示第 i 次部分积，式(6.9)可写成递推公式：

$$\left. \begin{aligned}
 z_0 &= 0 \\
 z_1 &= 2^{-1}(y_1 \cdot x^* + z_0) \\
 z_2 &= 2^{-1}(y_2 \cdot x^* + z_1) \\
 &\vdots \\
 z_i &= 2^{-1}(y_{i+1} \cdot x^* + z_{i-1}) \\
 &\vdots \\
 z_n &= 2^{-1}(y_1 \cdot x^* + z_{n-1})
 \end{aligned} \right\} \quad (6.10)$$

例 6.17 已知： $x = -0.1110$ ， $y = -0.1101$ 求： $[x \cdot y]_{\text{原}}$

解：因为 $x = -0.1110$

所以 $[x]_{\text{原}} = 1.1110$ ， $x^* = 0.1110$ (为绝对值)， $x_0 = 1$

又因为 $y = -0.1101$

所以 $[y]_{\text{原}} = 1.1101$ ， $y^* = 0.1101$ (为绝对值)， $y_0 = 1$

按原码一位乘运算规则， $[x \cdot y]_{\text{原}}$ 的数值部分计算如表 6.9 所示。

表 6.9 例 6.17 数值部分的计算

部分积	乘数	说 明
$\begin{array}{r} 0.0000 \\ + 0.1110 \end{array}$	1101	开始部分积 $z_0=0$ 乘数为 1，加上 x^*
$\begin{array}{r} 0.1110 \\ 0.0111 \\ + 0.0000 \end{array}$	0110	→1 位得 z_1 ，乘数同时→1 位 乘数为 0，加上 0
$\begin{array}{r} 0.0111 \\ 0.0011 \\ + 0.1110 \end{array}$	1011	→1 位得 z_2 ，乘数同时→1 位 乘数为 1，加上 x^*
$\begin{array}{r} 1.0001 \\ 0.1000 \\ + 0.1110 \end{array}$	101	→1 位得 z_3 ，乘数同时→1 位 乘数为 1，加上 x^*
$\begin{array}{r} 1.0110 \\ 0.1011 \end{array}$	110 0110	→1 位得 z_4 ，乘数已全部移出

即 $x^* \cdot y^* = 0.10110110$

乘积的符号位为 $x_0 \oplus y_0 = 1 \oplus 1 = 0$

故 $[x \cdot y]_{\text{原}} = 0.10110110$

值得注意的是：这里部分积取 $n+1$ 位，以便存放乘法过程中绝对值大于或等于 1 的值。此外，由于乘积的数值部分是两数绝对值相乘的结果，故原码一位乘法运算过程中的右移操作均为逻辑右移。

(2) 原码一位乘所需的硬件配置

图 6.7 是实现原码一位乘运算的基本硬件配置框图。

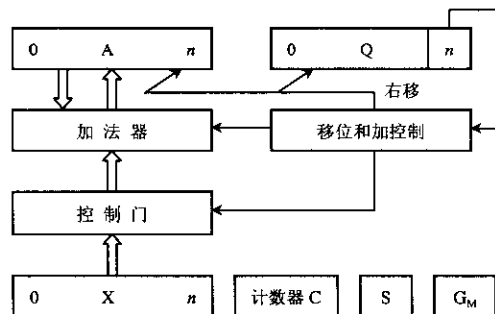


图 6.7 原码一位乘运算基本配置

图中 A、X、Q 均为 $n+1$ 位的寄存器，其中 X 存放被乘数的原码，Q 存放乘数的原码。移位和加控制电路受末位乘数 Q_n 的控制（当 $Q_n=1$ 时，A 和 X 内容相加后，A、Q 右移一位；当 $Q_n=0$ 时，只作 A、Q 右移一位的操作）。计数器 C 用于控制逐位相乘的次数。S 存放乘积的符号。 G_M 为乘法标记。

(3) 原码一位乘控制流程

原码一位乘控制流程如图 6.8 所示。

乘法运算前，A 寄存器被清零，作为初始部分积，被乘数原码在 X 中，乘数原码在 Q 中，计数器 C 中存放乘数的位数 n 。乘法开始后，首先通过异或运算，求出乘积的符号并存于 S，接着将被乘数和乘数从原码形式变为绝对值。然后根据 Q_n 的状态决定部分积是否加上被乘数，再逻辑右移一位，重复 n 次，即得运算结果。

上述讨论的运算规则同样可用于整数原码。为了区别于小数乘法，书写上可将表 6.9 中的“.”改为“,”。

为了提高乘法速度，可采用原码两位乘。

(4) 原码两位乘

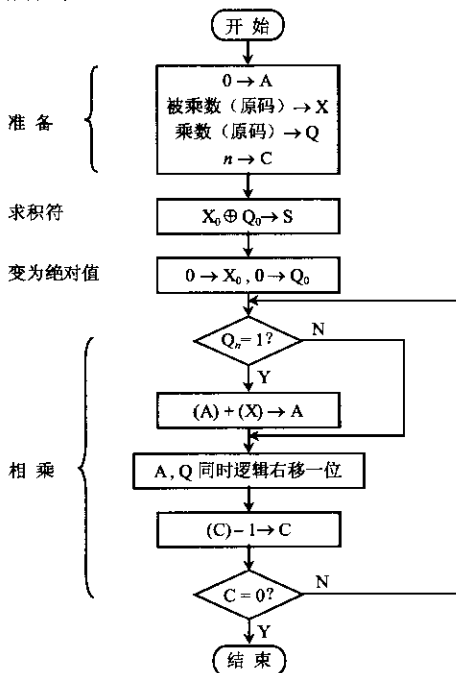


图 6.8 原码一位乘控制流程

原码两位乘与原码一位乘一样，符号位的运算和数值部分是分开进行的，但原码两位乘是用两位乘数的状态来决定新的部分积如何形成，因此可提高运算速度。

两位乘数共有四种状态，对应这四种状态可得表 6.10。

表 6.10 两位乘数所对应的新的部分积

乘数 $y_n y_{n-1}$	新的部分积
00	新部分积等于原部分积右移两位
01	新部分积等于原部分积加被乘数后右移两位
10	新部分积等于原部分积加 2 倍被乘数后右移两位
11	新部分积等于原部分积加 3 倍被乘数后右移两位

表中 2 倍被乘数可通过将被乘数左移一位实现, 但 3 倍被乘数的获得较难, 此刻可将 3 视为 4 减去 1, 即

表 6.12 例 6.18 原码两位乘数值部分的运算过程

部 分 积	乘数 y'	C_j	说 明
000.000000 000.111111	001110 <u>0</u> 1	0	开始, 部分积为 0, $C_j=0$ 根据 $y_n y_{n-1} C_j=010$, 加 x' , 保持 $C_j=0$
000.111111 000.001111 001.111110	110011 <u>1</u> 0	0	$\rightarrow 2$ 位, 得新的部分积, 乘数同时 $\rightarrow 2$ 位 根据 “100” 加 $2x'$, 保持 $C_j=0$
010.001101 000.100011 111.000001	11 011100 <u>1</u> 1	0	$\rightarrow 2$ 位, 得新的部分积, 乘数同时 $\rightarrow 2$ 位 根据 “110” 减 x' (即加 $[-x']_{补}$), 置 “1” C_j
111.100100 111.111001 000.111111	0111 000111 <u>0</u> 0	1	$\rightarrow 2$ 位, 得新的部分积, 乘数同时 $\rightarrow 2$ 位 根据 “001” 加 x' , 置 “0” C_j
000.111000	000111		形成最终结果

数为奇数时, 乘数高位前可只增加一个“0”, 此时需作 $\frac{n}{2}+1$ 次加法, $\frac{n}{2}-1$ 次移位 (最后一步移一位)。

虽然两位乘法可提高乘法速度, 但它仍基于重复相加和移位的思想, 而且随着乘数位数的增加, 重复次数增多, 仍然影响乘法速度的进一步提高。采用并行阵列乘法器可大大提高乘法速度。有关阵列乘法器的内容可参见附录 6B。

原码乘法实现比较容易, 但由于机器都采用补码作加减运算, 倘若做乘法前再将补码转换成原码, 相乘之后又要将负积的原码变为补码形式, 这样增添了许多操作步骤, 反而使运算复杂。为此, 有不少机器直接用补码相乘, 机器里配置实现补码乘法的乘法器, 避免了码制的转换, 提高了机器效率。

4. 补码乘法

(1) 补码一位乘运算规则

设被乘数 $[x]_{补} = x_0 x_1 x_2 \cdots x_n$

乘数 $[y]_{补} = y_0 y_1 y_2 \cdots y_n$

① 当被乘数 x 符号任意, 乘数 y 符号为正时。

$$[x]_{补} = x_0 x_1 x_2 \cdots x_n = 2 + x \pmod{2}$$

$$[y]_{补} = 0 y_1 y_2 \cdots y_n = y$$

$$\text{则 } [x]_{补} \cdot [y]_{补} = [x]_{补} \cdot y = (2+x) \cdot y = 2y + xy$$

由于 $y = 0 y_1 y_2 \cdots y_n = \sum_{i=1}^n y_i 2^i$ 是一个大于或等于 1 的正整数, 根据模运算的性质, 有 $2y = 2 \pmod{2}$, 故

$$[x]_{\text{补}} \cdot [y]_{\text{补}} = 2y + xy = 2 + xy = [x \cdot y]_{\text{补}} \pmod{2}$$

$$\text{即 } [x \cdot y]_{\text{补}} = [x]_{\text{补}} \cdot [y]_{\text{补}} = [x]_{\text{补}} \cdot y$$

对照原码乘法式 (6.9) 和式 (6.10) 可见, 当乘数 y 为正数时, 不管被乘数 x 符号如何, 都可按原码乘法的规则运算, 即

$$\left. \begin{aligned} [z_0]_{\text{补}} &= 0 \\ [z_1]_{\text{补}} &= 2^{-1}(y_n[x]_{\text{补}} + [z_0]_{\text{补}}) \\ [z_2]_{\text{补}} &= 2^{-1}(y_{n-1}[x]_{\text{补}} + [z_1]_{\text{补}}) \\ &\vdots \\ [z_i]_{\text{补}} &= 2^{-1}(y_{n-i+1}[x]_{\text{补}} + [z_{i-1}]_{\text{补}}) \\ &\vdots \\ [x \cdot y]_{\text{补}} &= [z_n]_{\text{补}} = 2^{-1}(y_1[x]_{\text{补}} + [z_{n-1}]_{\text{补}}) \end{aligned} \right\} \quad (6.11)$$

当然这里的加和移位都必须按补码规则运算。

② 当被乘数 x 符号任意, 乘数 y 符号为负时。

$$[x]_{\text{补}} = x_0 \cdot x_1 x_2 \cdots x_n$$

$$[y]_{\text{补}} = 1 \cdot y_1 y_2 \cdots y_n = 2 + y \pmod{2}$$

$$\text{则 } y = [y]_{\text{补}} - 2 = 1 \cdot y_1 y_2 \cdots y_n - 2 = 0 \cdot y_1 y_2 \cdots y_n - 1$$

$$x \cdot y = x(0 \cdot y_1 y_2 \cdots y_n - 1)$$

$$= x(0 \cdot y_1 y_2 \cdots y_n) - x$$

$$\text{故 } [x \cdot y]_{\text{补}} = [x(0 \cdot y_1 y_2 \cdots y_n)]_{\text{补}} + [-x]_{\text{补}}$$

将上式 $0 \cdot y_1 y_2 \cdots y_n$ 视为一个正数, 正好与上述情况相同

$$\text{则 } [x(0 \cdot y_1 y_2 \cdots y_n)]_{\text{补}} = [x]_{\text{补}}(0 \cdot y_1 y_2 \cdots y_n)$$

$$\text{所以 } [x \cdot y]_{\text{补}} = [x]_{\text{补}}(0 \cdot y_1 y_2 \cdots y_n) + [-x]_{\text{补}}$$

(6.12)

由此可得, 当乘数为负时是把乘数的补码 $[y]_{\text{补}}$ 去掉符号位, 当成一个正数与 $[x]_{\text{补}}$ 相乘, 然后加上 $[-x]_{\text{补}}$ 进行校正, 也称校正法, 用递推公式表示时:

$$\left. \begin{aligned} [z_0]_{\text{补}} &= 0 \\ [z_1]_{\text{补}} &= 2^{-1}(y_n[x]_{\text{补}} + [z_0]_{\text{补}}) \\ [z_2]_{\text{补}} &= 2^{-1}(y_{n-1}[x]_{\text{补}} + [z_1]_{\text{补}}) \\ &\vdots \\ [z_i]_{\text{补}} &= 2^{-1}(y_{n-i+1}[x]_{\text{补}} + [z_{i-1}]_{\text{补}}) \\ &\vdots \\ [z_n]_{\text{补}} &= 2^{-1}(y_1[x]_{\text{补}} + [z_{n-1}]_{\text{补}}) \\ [x \cdot y]_{\text{补}} &= [z_n]_{\text{补}} + [-x]_{\text{补}} \end{aligned} \right\} \quad (6.13)$$

比较 (6.13) 与 (6.11) 可见, 乘数为负的补码乘法与乘数为正时完全类似, 只需最后加上一项校正项 $[-x]$ 补即可。

例 6.19 已知: $[x]_{\text{补}} = 1.0101$, $[y]_{\text{补}} = 0.1101$ 求: $[xy]_{\text{补}}$

解: 因为乘数 $y > 0$, 所以按原码一位乘的算法运算, 只是在相加和移位时按补码规则进行, 如表 6.13 所示。考虑到运算时可能出现绝对值大于 1 的情况 (但此刻并不是溢出), 故部分积和被乘数取双符号位。

表 6.13 例 6.19 的运算过程

部分积	乘数	说 明
$\begin{array}{r} 00.0000 \\ + 11.0101 \\ \hline \end{array}$	$\begin{array}{r} 110 \\ 1 \\ \hline \end{array}$	初值 $[z_0]_{\text{补}} = 0$ $y_4 = 1$, $+ [x]_{\text{补}}$
$\begin{array}{r} 11.0101 \\ 11.1010 \\ 11.1101 \\ + 11.0101 \\ \hline \end{array}$	$\begin{array}{r} 111 \\ 0 \\ 011 \\ 1 \\ \hline \end{array}$	$\rightarrow 1$ 位, 得 $[z_1]_{\text{补}}$, 乘数同时 $\rightarrow 1$ 位 $y_3 = 0$, 右移一位, 得 $[z_2]_{\text{补}}$, 乘数同时 $\rightarrow 1$ 位 $y_2 = 1$, $+ [x]_{\text{补}}$
$\begin{array}{r} 11.0010 \\ 11.1001 \\ + 11.0101 \\ \hline \end{array}$	$\begin{array}{r} 01 \\ 001 \\ 1 \\ \hline \end{array}$	$\rightarrow 1$ 位, 得 $[z_3]_{\text{补}}$, 乘数同时 $\rightarrow 1$ 位 $y_1 = 1$, $+ [x]_{\text{补}}$
$\begin{array}{r} 10.1110 \\ 11.0111 \\ \hline \end{array}$	$\begin{array}{r} 001 \\ 000 \\ 1 \\ \hline \end{array}$	$\rightarrow 1$ 位, 得 $[z_4]_{\text{补}}$

故乘积 $[xy]_{\text{补}} = 1.01110001$

例 6.20 已知: $[x]_{\text{补}} = 0.1101$, $[y]_{\text{补}} = 1.0101$ 求: $[xy]_{\text{补}}$

解: 因为乘数 $y < 0$, 故先不考虑符号位, 按原码相乘, 最后再加上 $[-x]_{\text{补}}$, 如表 6.14 所示。

故乘积 $[x \cdot y]_{\text{补}} = 1.01110001$

由上两例可见, 乘积的符号位在运算过程中自然形成, 这是补码乘法和原码乘法的重要区别。

上述校正法与乘数的符号有关, 虽然可将乘数和被乘数互换, 使乘数保持正, 不必校正, 但当两数均为负时必须校正。总之, 实现校正法的控制线路比较复杂。若不考虑操作数符号, 用统一的规则进行运算, 就可采用比较法。

③ 当被乘数 x 和乘数 y 符号均为任意时。

比较法是 Booth 夫妇首先提出来的, 故又叫 Booth 算法。它的运算规则可

表 6.14 例 6.20 的运算过程

部分积	乘数	说 明
$\begin{array}{r} 00.0000 \\ + 00.1101 \end{array}$	0101	初值 $[z_0]_{\text{补}}=0$ $y_4=1, +[x]_{\text{补}}$
$\begin{array}{r} 00.1101 \\ 00.0110 \\ 00.0011 \\ + 00.1101 \end{array}$	$\begin{array}{r} 1010 \\ 0101 \end{array}$	$\rightarrow 1$ 位, 得 $[z_1]_{\text{补}}$, 乘数同时 $\rightarrow 1$ 位 $y_3=0, \rightarrow 1$ 位, 得 $[z_2]_{\text{补}}$ $y_2=1, +[x]_{\text{补}}$
$\begin{array}{r} 01.0000 \\ 00.1000 \\ 00.0100 \\ 11.0011 \end{array}$	$\begin{array}{r} 0010 \\ 0001 \end{array}$	$\rightarrow 1$ 位, 得 $[z_3]_{\text{补}}$, 乘数同时 $\rightarrow 1$ 位 $y_1=0, \rightarrow 1$ 位, 得 $[z_4]_{\text{补}}$ $+[-x]_{\text{补}}$ 进行校正
11.0111	0001	得最后结果 $[xy]_{\text{补}}$

由校正法导出。

$$\text{设 } [x]_{\text{补}} = x_0.x_1x_2\cdots x_n$$

$$[y]_{\text{补}} = y_0.y_1y_2\cdots y_n$$

按补码乘法校正法规则, 其基本算法可用一个统一的公式表示为:

$$[xy]_{\text{补}} = [x]_{\text{补}}(0.y_1y_2\cdots y_n) - [x]_{\text{补}}y_0 \quad (6.14)$$

当 $y_0 = 0$ 时, 表示乘数 y 为正, 无需校正, 即

$$[xy]_{\text{补}} = [x]_{\text{补}}(0.y_1y_2\cdots y_n) \quad (6.15)$$

当 $y_0 = 1$ 时, 表示乘数 y 为负, 则

$$[xy]_{\text{补}} = [x]_{\text{补}}(0.y_1y_2\cdots y_n) - [x]_{\text{补}} \quad (6.16)$$

比较式 (6.12) 和式 (6.16), 在 $\text{mod } 2$ 的前提下, $[-x]_{\text{补}} = -[x]_{\text{补}}$ 成立^①, 所以 (6.15) 和 (6.16) 两式表达的算法与校正法的结论完全相同, 故式 (6.14) 可以改写为:

① 证明: $[-x]_{\text{补}} = -[x]_{\text{补}} \pmod{2}$

(1) 若 $[x]_{\text{补}} = 0.x_1x_2\cdots x_n$

则 $x = 0.x_1x_2\cdots x_n$

所以 $-x = -0.x_1x_2\cdots x_n$

故 $[-x]_{\text{补}} = 1.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n}$

$\pmod{2} \cdots (a)$

又因为 $[x]_{\text{补}} = 0.x_1x_2\cdots x_n$

所以 $-[x]_{\text{补}} = -0.x_1x_2\cdots x_n$

$\equiv 2 - 0.x_1x_2\cdots x_n \pmod{2}$

$= 1.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n} \cdots (b)$

比较(a)、(b)两式可得

$[-x]_{\text{补}} = -[x]_{\text{补}} \pmod{2}$

证毕

(2) 若 $[x]_{\text{补}} = 1.x_1x_2\cdots x_n$

则 $x = (0.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n})$

所以 $-x = 0.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n}$

故 $[-x]_{\text{补}} = 0.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n}$

$\pmod{2} \cdots (c)$

又因为 $[x]_{\text{补}} = 1.x_1x_2\cdots x_n$

$\equiv -(0.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n}) \pmod{2}$

所以 $-[x]_{\text{补}} = 0.\bar{x}_1\bar{x}_2\cdots\bar{x}_n + 2^{-n} \cdots (d)$

比较(c)、(d)两式可得

$[-x]_{\text{补}} = -[x]_{\text{补}} \pmod{2}$

$$\begin{aligned}
[x \cdot y]_{\text{补}} &= [x]_{\text{补}} (y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) - [x]_{\text{补}} y_0 \\
&= [x]_{\text{补}} (-y_0 + y_1 2^{-1} + y_2 2^{-2} + \cdots + y_n 2^{-n}) \\
&= [x]_{\text{补}} [-y_0 + (y_1 - y_1 2^{-1}) + (y_2 2^{-1} - y_2 2^{-2}) + \cdots + (y_n 2^{-(n-1)} - y_n 2^{-n})] \\
&= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_n - y_{n-1}) 2^{-(n-1)} + (0 - y_n) 2^{-n}] \\
&= [x]_{\text{补}} [(y_1 - y_0) + (y_2 - y_1) 2^{-1} + \cdots + (y_{n+1} - y_n) 2^{-n}]
\end{aligned} \quad (6.17)$$

其中 $y_{n+1} = 0$

这样, 可得递推公式:

$$\left. \begin{aligned}
[z_0]_{\text{补}} &= 0 \\
[z_1]_{\text{补}} &= 2^{-1} \{ [z_0]_{\text{补}} + (y_{n+1} - y_n) [x]_{\text{补}} \} \\
[z_2]_{\text{补}} &= 2^{-1} \{ [z_1]_{\text{补}} + (y_n - y_{n-1}) [x]_{\text{补}} \} \\
&\vdots \\
[z_i]_{\text{补}} &= 2^{-1} \{ [z_{i-1}]_{\text{补}} + (y_{n-i+2} - y_{n-i+1}) [x]_{\text{补}} \} \\
&\vdots \\
[z_n]_{\text{补}} &= 2^{-1} \{ [z_{n-1}]_{\text{补}} + (y_2 - y_1) [x]_{\text{补}} \} \\
[x \cdot y]_{\text{补}} &= [z_{n+1}]_{\text{补}} = [z_n]_{\text{补}} + (y_1 - y_0) [x]_{\text{补}}
\end{aligned} \right\} \quad (6.18)$$

由此可见, 开始时 $y_{n+1} = 0$, 部分积初值 $[z_0]_{\text{补}}$ 为 0, 每一步乘法由 $(y_{i+1} - y_i)$ ($i=1, 2, \dots, n$) 决定原部分积加 $[x]_{\text{补}}$ 或加 $[-x]_{\text{补}}$ 或加 0, 再右移一位得新的部分积, 以此重复 n 步。第 $n+1$ 步由 $(y_1 - y_0)$ 决定原部分积加 $[x]_{\text{补}}$ (或 $[-x]_{\text{补}}$ 或 0), 但不移位, 即得 $[x \cdot y]_{\text{补}}$ 。

这里的 $(y_{i+1} - y_i)$ 之差值, 恰恰与乘数末两位 y_i 及 y_{i+1} 的状态对应, 如表 6.15 所示, 只是当运算至最后一步时, 乘积不再右移。这样的运算规则计算机很容易实现。

应该注意的是, 按比较法进行补码乘法时, 像补码加、减法一样, 符号位也一起参加运算。

表 6.15 $y_i y_{i+1}$ 的状态对操作的影响

$y_i y_{i+1}$	$y_{i+1} - y_i$	操 作
0 0	0	部分积右移一位
0 1	1	部分积加 $[x]_{\text{补}}$, 再右移一位
1 0	-1	部分积加 $[-x]_{\text{补}}$, 再右移一位
1 1	0	部分积右移一位

例 6.21 已知: $[x]_{\text{补}} = 0.1101$, $[y]_{\text{补}} = 0.1011$ 求: $[x \cdot y]_{\text{补}}$

解: 表 6.16 列出了例 6.21 的求解过程。

故 $[x \cdot y]_{\text{补}} = 0.10001111$

由表 6.16 可见, 与校正法 (见表 6.13 和表 6.14) 相比, Booth 算法的部分积仍取双符号位, 乘数因符号位参加运算, 故多取一位。

例 6.22 已知: $[x]_{\text{补}} = 1.0101$, $[y]_{\text{补}} = 1.0011$ 求: $[x \cdot y]_{\text{补}}$

解: 表 6.17 列出了例 6.22 的求解过程。

故 $[x \cdot y]_{\text{补}} = 0.10001111$

由于比较法的补码乘法运算规则不受乘数符号的约束, 因此, 控制线路比较简明, 在计算机中普遍采用。

表 6.16 例 6.21 求 $[x \cdot y]_{\text{补}}$ 的过程

部分积	乘数 y_n	附加位 y_{n+1}	说 明
00.0000 $+ 11.0011$	01011	0	初值 $[z_0]_{\text{补}} = 0$ $y_n y_{n+1} = 10$, 部分积加 $[-x]_{\text{补}}$
11.0011 11.1001 11.1100 $+ 00.1101$	10101 11010	1 1	$\rightarrow 1$ 位, 得 $[z_1]_{\text{补}}$ $y_n y_{n+1} = 11$, 部分积 $\rightarrow 1$ 位得 $[z_2]_{\text{补}}$ $y_n y_{n+1} = 01$, 部分积加 $[x]_{\text{补}}$
00.1001 00.0100 $+ 11.0011$	11 11101	0	$\rightarrow 1$ 位, 得 $[z_3]_{\text{补}}$ $y_n y_{n+1} = 10$, 部分积加 $[-x]_{\text{补}}$
11.0111 11.1011 $+ 00.1101$	11110	1	$\rightarrow 1$ 位, 得 $[z_4]_{\text{补}}$ $y_n y_{n+1} = 01$, 部分积加 $[x]_{\text{补}}$
00.1000	1111		最后一步不移位, 得 $[xy]_{\text{补}}$

表 6.17 例 6.22 求 $[x \cdot y]_{\text{补}}$ 的过程

部分积	乘数 y_n	附加位 y_{n+1}	说 明
00.0000 $+ 00.1011$	10011	0	$y_n y_{n+1} = 10$, 部分积加 $[-x]_{\text{补}}$
00.1011 00.0101 00.0010 $+ 11.0101$	11001 11100	1 1	$\rightarrow 1$ 位, 得 $[z_1]_{\text{补}}$ $y_n y_{n+1} = 11$, 部分积 $\rightarrow 1$ 位, 得 $[z_2]_{\text{补}}$ $y_n y_{n+1} = 01$, 部分积加 $[x]_{\text{补}}$
11.0111 11.1011 11.1101 00.1011	11 11110 11111	0 0	$\rightarrow 1$ 位, 得 $[z_3]_{\text{补}}$ $y_n y_{n+1} = 00$, 部分 $\rightarrow 1$ 位, 得 $[z_4]_{\text{补}}$ $y_n y_{n+1} = 10$, 部分积加 $[-x]_{\text{补}}$
00.1000	1111		最后一步不移位, 得 $[xy]_{\text{补}}$

(2) 补码比较法 (Booth 算法) 所需的硬件配置

图 6.9 是实现补码一位乘比较法乘法运算的基本硬件配置框图。

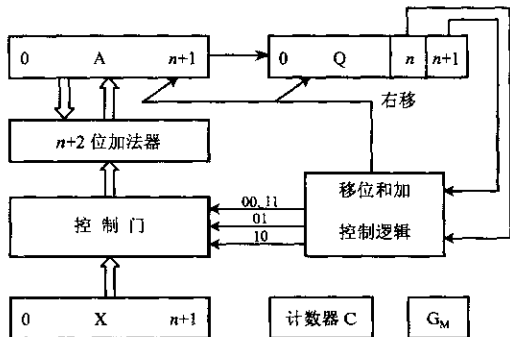


图 6.9 补码比较法运算基本硬件配置

图中 A、X、Q 均为 $n+2$ 位寄存器，其中 X 存放被乘数的补码（含两位符号位），Q 存放乘数的补码（含最高一位符号位和最末一位附加位），移位和加控制逻辑受 Q 寄存器末 2 位乘数控制。当其为 01 时，A、X 内容相加后 A、Q 右移一位；当其为 10 时，A、X 内容相减后 A、Q 右移一位。计数器 C 用于控制逐位相乘的次数， G_M 为乘法标记。

(3) 补码比较法 (Booth 算法) 控制流程

补码一位乘比较法的控制流程图如图 6.10 所示。乘法运算前 A 寄存器被清 0，作为初始部分积。Q 寄存器末位清 0，作为附加位的初态。被乘数的补码存在 X 中（双符号位），乘数的补码在 Q 高 $n+1$ 位中，计数器 C 存放乘数的位数 n 。乘法开始后，根据 Q 寄存器末两位 Q_n, Q_{n+1} 的状态决定部分积与被乘数相加还是相减，或是不加也不减，然后按补码规则进行算术移位，这样重复 n 次。最后再根据 Q 的末两位状态决定部分积是否与被乘数相加（或相减），或是不加也不减，但不必移位，这样便可得到最后结果。补码乘法乘积的符号位在运算中自然形成。

需要说明的是图中 $(A) - (X) \rightarrow A$ 实际是用加法器实现的，即 $(A) + (\bar{X} + 1) \rightarrow A$ 。同理，Booth 运算规则也适用于整数补码。

为了提高乘法的运算速度，可采用补码两位乘。

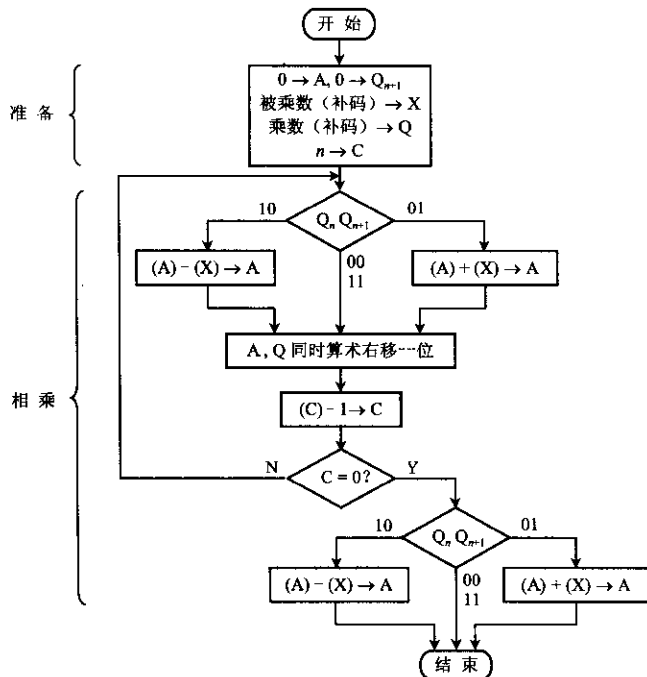


图 6.10 补码一位乘比较法控制流程图

(4) 补码两位乘

补码两位乘运算规则是根据补码一位乘的规则,把比较 $y_n y_{n+1}$ 的状态应执行的操作和比较 $y_{n-1} y_n$ 的状态应执行的操作合并成一步,便可得出补码两位乘的运算方法。

例如, $y_{n-1} y_n y_{n+1}$ 为 011, 则第一步由 $y_n y_{n+1} = 11$ 得出只作右移, 即 $2^{-1}[z_i]_{补}$, 第二步由 $y_{n-1} y_n = 01$ 得出需作 $2^{-1}\{2^{-1}[z_i]_{补} + [x]_{补}\}$ 的操作, 它可改写为 $2^{-2}\{[z_i]_{补} + 2[x]_{补}\}$, 即最后结论为当 $y_{n-1} y_n y_{n+1}$ 为 011 时, 完成 $2^{-2}\{[z_i]_{补} + 2[x]_{补}\}$ 操作, 同理可分析其余七种情况。表 6.18 列出了补码两位乘的运算规则。

表 6.18 补码两位乘法运算规则

判断位 y_n, y_n, y_{n+1}	操作内容
000	$\{z_{i+1}\}_\text{补} = 2^{-2}\{z_i\}_\text{补}$
001	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + [x]_\text{补}\}$
010	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + [x]_\text{补}\}$
011	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + 2[x]_\text{补}\}$
100	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + 2[-x]_\text{补}\}$
101	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + [-x]_\text{补}\}$
110	$\{z_{i+1}\}_\text{补} = 2^{-2}\{\{z_i\}_\text{补} + [-x]_\text{补}\}$
111	$\{z_{i+1}\}_\text{补} = 2^{-2}\{z_i\}_\text{补}$

由表 6.18 可见, 操作中出現加 $2[x]_\text{补}$ 和加 $2[-x]_\text{补}$, 故除右移两位的操作外, 还有被乘数左移一位的操作; 而加 $2[x]_\text{补}$ 和加 $2[-x]_\text{补}$, 都可能因溢出而侵占双符号位, 故部分积和被乘数采用三位符号位。

例 6.23 已知: $[x]_\text{补} = 0.0101$, $[y]_\text{补} = 1.0101$ 求: $[x \cdot y]_\text{补}$

解: 表 6.19 列出了此例的求解过程。其中乘数取两位符号位即 11.0101, $[-x]_\text{补} = 1.1011$ 取三位符号位为 111.1011。

表 6.19 例 6.23 补码两位乘求 $[x \cdot y]_\text{补}$ 的过程

部分积	乘数	说明
000.0000 + 000.0101	1101 <u>010</u>	判断位为 010, 加 $[x]_\text{补}$
000.0101 000.0001 + 000.0101	0111 <u>010</u>	→2 位 判断位为 010, 加 $[x]_\text{补}$
000.0110 000.0001 + 111.1011	01 1001 <u>110</u>	→2 位 判断位为 110, 加 $[-x]_\text{补}$
111.1100	1001	最后一步不移位, 得 $[x \cdot y]_\text{补}$

故 $[x \cdot y]_\text{补} = 1.11001001$

由表 6.19 可见, 与补码一位乘相比 (见表 6.16 和表 6.17), 补码两位乘的部分积多取一位符号位 (共 3 位), 乘数也多取一位符号位 (共 2 位), 这是由于乘数每次右移 2 位, 且用 3 位判断, 故其采用双符号位更便于硬件实现。

可见,当乘数数值位为偶数时,乘数取 2 位符号位,共需作 $\frac{n}{2}$ 次移位,最多作 $\frac{n}{2}+1$ 次加法,最后一步不移位;当 n 为奇数时,可补 0 变为偶数位,以简化逻辑操作。也可对乘数取 1 位符号位,此时共作 $\frac{n}{2}+1$ 次加法和 $\frac{n}{2}+1$ 次移位(最后一步移一位)。

对于整数补码乘法,其过程与小数补码乘法完全相同。为了区别于小数乘法,在书写上可将符号位和数值位中间的“.”改为“,”即可。

6.3.4 除法运算

1. 分析笔算除法

以小数为例,设 $x=-0.1011$, $y=0.1101$, 求 x/y 。

笔算除法时,商的符号心算而得:负正得负;其数值部分的运算如下面竖式。

所以 商 $x/y=-0.1101$, 余数 $=-0.00000111$

其特点可归纳如下:

- ① 每次上商都是由心算来比较余数(被除数)和除数的大小,确定商为“1”还是“0”。

$$\begin{array}{r}
 0.1101 \overline{) 0.10110} \\
 \underline{0.01101} 2^{-1} \cdot y \\
 0.010010 \\
 \underline{0.001101} 2^{-2} \cdot y \\
 0.00010100 \\
 \underline{0.00001101} 2^{-4} \cdot y \\
 0.00000111
 \end{array}$$

- ② 每做一次减法,总是保持余数不动,低位补 0,再减去右移后的除数。

- ③ 商符单独处理。

如果将上述规则完全照搬到计算机内,实现起来有一定困难,主要问题是:

- ① 机器不能“心算”上商,必须通过比较被除数(或余数)和除数绝对值的大小来确定商值,即 $|x| - |y|$,若差为正(够减)上商 1,差为负(不够减)上商 0。

- ② 按照每次减法总是保持余数不动低位补 0,再减去右移后的除数这一

规则,则要求加法器的位数必须为除数的两倍。仔细分析发现,右移除数可以用左移余数的办法代替,其运算结果是一样的,但对线路结构更有利。不过此刻所得到的余数不是真正的余数,只有将它乘上 2^{-n} 才是真正的余数。

③ 笔算求商时是从高位向低位逐位求的,而要求机器把每位商直接写到寄存器的不同位也是不可取的。计算机可将每一位商直接写到寄存器的最低位,并把原来的部分商左移一位。

综上所述便可得原码除法运算规则。

2. 原码除法

原码除法和原码乘法一样,符号位是单独处理的。以小数为例

$$\text{设 } [x]_{\text{原}} = x_0.x_1x_2\cdots x_n$$

$$[y]_{\text{原}} = y_0.y_1y_2\cdots y_n$$

$$\left[\frac{x}{y}\right]_{\text{原}} = (x_0 \oplus y_0) \cdot \frac{0.x_1x_2\cdots x_n}{0.y_1y_2\cdots y_n}$$

式中 $0.x_1x_2\cdots x_n$ 为 x 的绝对值,记作 x^* ;

$0.y_1y_2\cdots y_n$ 为 y 的绝对值,记作 y^* 。

即商符由两数符号位“异或”运算求得,商值由两数绝对值相除(x^*/y^*)求得。

小数定点除法对被除数和除数有一定的约束,即必须满足下列条件:

$$0 < |\text{被除数}| \leq |\text{除数}|$$

实现除法运算时,还应避免除数为 0 或被除数为 0。前者结果为无限大,不能用机器的有限位数表示;后者结果总是 0,这个除法操作等于白做,浪费了机器时间。至于商的位数一般与操作数的位数相同。

原码除法中由于对余数的处理不同,又可分为恢复余数法和不恢复余数法(加减交替法)两种。

(1) 恢复余数法

恢复余数法的特点是:当余数为负时,需加上除数,将其恢复成原来的余数。

由上所述,商值的确定是通过比较被除数和除数的绝对值大小,即 $x^* - y^*$ 实现的,而计算机内只设加法器,故需将 $x^* - y^*$ 操作变为 $[x^*]_{\text{补}} + [-y^*]_{\text{补}}$ 的操作。

例 6.24 已知: $x = -0.1011$, $y = -0.1101$ 求: $\left[\frac{x}{y}\right]_{\text{原}}$

解: 由 $x = -0.1011$, $y = -0.1101$

$$\text{得 } x^* = 0.1011, [x]_{\text{原}} = 1.1011$$

$$y^* = 0.1101, [-y^*]_{\text{补}} = 1.0011, [y]_{\text{原}} = 1.1101$$

表 6.20 列出了例 6.24 商值的求解过程。

表 6.20 例 6.24 恢复余数法求解过程

被除数 (余数)	商	说 明
0.1011 + 1.0011	0.0000	$+[-y']_{补}$ (减去除数)
1.1110 + 0.1101	0	余数为负, 上商 0 恢复余数 $+ [y']_{补}$
0.1011 1.0110 + 1.0011	0	被恢复的被除数 $\leftarrow 1$ 位 $+ [-y']_{补}$ (减去除数)
0.1001 1.0010 + 1.0011	01 01	余数为正, 上商 1 $\leftarrow 1$ 位 $+ [-y']_{补}$ (减去除数)
0.0101 0.1010 + 1.0011	011 011	余数为正, 上商 1 $\leftarrow 1$ 位 $+ [-y']_{补}$ (减去除数)
1.1101 - 0.1101	0110	余数为负, 上商 0 恢复余数 $+ [y']_{补}$
0.1010 1.0100 + 1.0011	0110	被恢复的余数 $\leftarrow 1$ 位 $+ [-y']_{补}$ (减去除数)
0.0111	01101	余数为正, 上商 1

故商值为 0.1101

商的符号位为 $x_0 \oplus y_0 = 1 \oplus 1 = 0$

所以 $[\frac{x}{y}]_{原} = 0.1101$

由此例可见, 共上商 5 次, 第一次上的商在商的整数位上, 这对小数除法而言, 可用它作溢出判断。即当该位为“1”时, 表示此除法为溢出, 不能进行, 应由程序进行处理; 当该位为“0”时, 说明除法合法, 可以进行。

在恢复余数法中, 每当余数为负时, 都需恢复余数, 这就延长了机器除法的时间, 操作也很不规则, 对线路结构不利。加减交替法可克服这些缺点。

(2) 加减交替法

加减交替法又称不恢复余数法, 可以认为它是恢复余数法的一种改进算法。

分析原码恢复余数法得知:

当余数 $R_i > 0$ 时, 可上商“1”, 再对 R_i 左移一位后减除数, 即 $2R_i - y'$ 。

当余数 $R_i < 0$ 时, 可上商“0”, 然后先做 $R_i + y'$, 即完成恢复余数的运算, 再做 $2(R_i + y') - y'$, 也即 $2R_i + y'$ 。

可见, 原码恢复余数法可归纳为:

当 $R_i > 0$, 商上“1”, 做 $2R_i - y^*$ 的运算;

当 $R_i < 0$, 商上“0”, 做 $2R_i + y^*$ 的运算。

这里已经看不出余数的恢复问题了, 而只是做加 y^* 或减 y^* , 因此, 一般把它叫做加减交替法或不恢复余数法。

例 6.25 已知: $x = -0.1011$, $y = 0.1101$ 求: $[\frac{x}{y}]_{\text{原}}$

解: 由 $x = -0.1011$, $y = 0.1101$

得 $[x]_{\text{原}} = 1.1011$, $x^* = 0.1011$

$[y]_{\text{原}} = 0.1101$, $y^* = 0.1101$, $[-y^*]_{\text{补}} = 1.0011$

表 6.21 列出了此例商值的求解过程。

商的符号位为 $x_0 \oplus y_0 = 1 \oplus 0 = 1$

所以 $[\frac{x}{y}]_{\text{原}} = 1.1101$

分析此例可见, n 位小数的除法共上商 $n+1$ 次, 第一次商用来判断是否溢出。倘若比例因子选择恰当, 除数结果不溢出, 则第一次商肯定是 0。如果省去这位商, 只需上商 n 次即可, 此时除法运算一开始应将被除数左移一位减去除数, 然后再根据余数上商。读者可以自己练习。

表 6.21 例 6.25 加减交替法运算过程

被除数 (余数)	商	说 明
0.1011 + 1.0011	0.0000	$+[-y^*]_{\text{补}}$ (减除数)
1.1110 1.1100 + 0.1101	0 0	余数为负, 上商 0 $\leftarrow 1$ 位 $+ [y^*]_{\text{补}}$ (加除数)
0.1001 1.0010 + 1.0011	01 01	余数为正, 上商 1 $\leftarrow 1$ 位 $+ [-y^*]_{\text{补}}$ (减除数)
0.0101 0.1010 + 1.0011	011 011	余数为正, 上商 1 $\leftarrow 1$ 位 $+ [-y^*]_{\text{补}}$ (减除数)
1.1101 1.1010 + 0.1101	0110 0110	余数为负, 上商 0 $\leftarrow 1$ 位 $+ [y^*]_{\text{补}}$ (加除数)
0.0111	01101	余数为正, 上商 1

(3) 原码加减交替法所需的硬件配置

图 6.11 是实现原码加减交替除法运算的基本硬件配置框图。

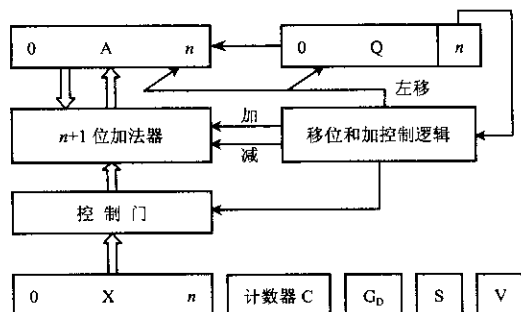


图 6.11 原码加减交替除法运算的基本硬件配置

图中 A、X、Q 均为 $n+1$ 位寄存器，其中 A 存放被除数的原码，X 存放除数的原码。移位和加控制逻辑受 Q 的末位 Q_n 控制。（ $Q_n = 1$ 作减法， $Q_n = 0$ 作加法），计数器 C 用于控制逐位相除的次数 n ， G_D 为除法标记，V 为溢出标记，S 为商符。

（4）原码加减交替除法控制流程

图 6.12 为原码加减交替除法控制流程图。

除法开始前，Q 寄存器被清 0，准备接收商，被除数的原码放在 A 中，除数的原码放在 X 中，计数器 C 中存放除数的位数 n 。除法开始后，首先通过异或运算求出商符，并存于 S。接着将被除数和除数变为绝对值，然后开始用第一次上商判断是否溢出。若溢出，则置溢出标记 V 为 1，停止运算，进行中断处理，重新选择比例因子；若无溢出，则先上商，接着 A、Q 同时左移一位，然后再根据上一次商值的状态，决定是加还是减除数，这样重复 n 次后，再上最后一次商（共上商 $n+1$ 次），即得运算结果。

对于整数除法，要求满足以下条件：

$$0 < |\text{除数}| \leq |\text{被除数}|$$

因为这样才能得到整数商。通常在做整数除法前，先要对这个条件进行判断，若不满足上述条件，机器发出出错信号，程序要重新设定比例因子。

上述讨论的小数除法完全适用于整数除法，只是整数除法的被除数位数可以是除数的两倍，且要求被除数的高 n 位要比除数（ n 位）小，否则即为溢出。如果被除数和除数的位数都是单字长，则要在被除数前面加上一个字的 0，从而扩展成双倍字长再进行运算。

为了提高除法速度，可采用阵列除法器，有关内容参见附录 6B。

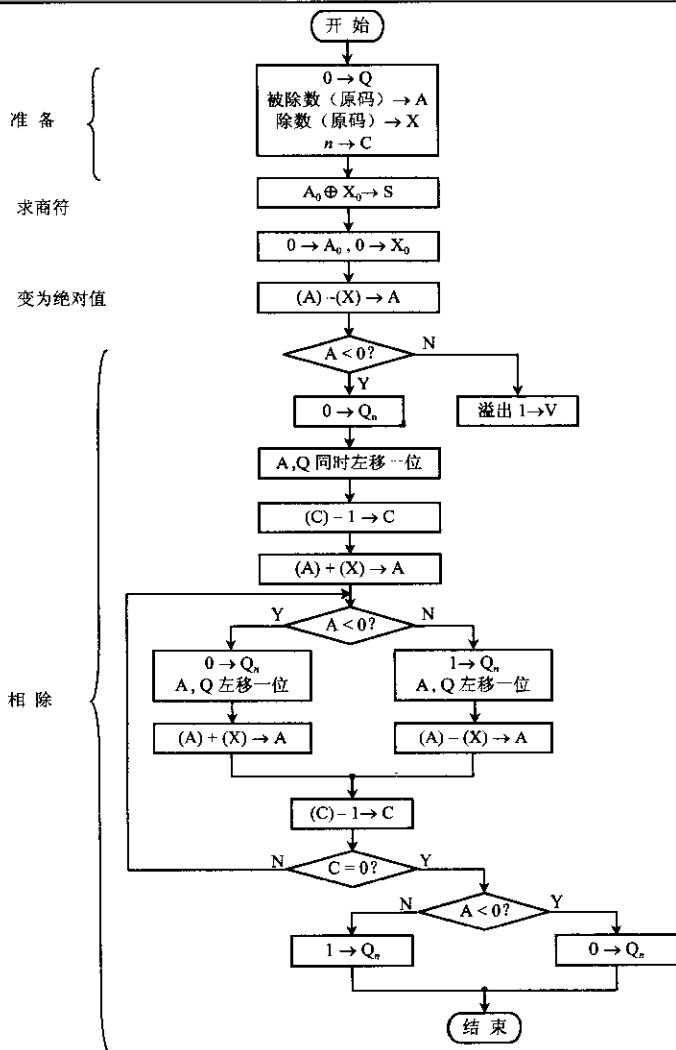


图 6.12 原码加减交替除法控制流程图

3. 补码除法

与补码乘法类似，也可以用补码完成除法操作。补码除法也分恢复余数法和加减交替法，后者用得较多，在此只讨论加减交替法。

(1) 补码加减交替法运算规则

补码除法其符号位和数值部分是一起参加运算的，因此在算法上不像原码除法那样直观，主要需解决三个问题：① 如何确定商值；② 如何形成商符；③ 如何获得新的余数。

① 商值的确定。

欲确定商值，必须先比较被除数和除数的大小，然后才能求得商值。

• 比较被除数（余数）和除数的大小

补码除法的操作数均为补码，其符号又是任意的，因此要比较被除数 $[x]_{补}$ 和除数 $[y]_{补}$ 的大小就不能简单的用 $[x]_{补}$ 减去 $[y]_{补}$ 。实质上比较 $[x]_{补}$ 和 $[y]_{补}$ 的大小就是比较它们所对应的绝对值的大小。同样在求商的过程中，比较余数 $[R_i]_{补}$ 与除数 $[y]_{补}$ 的大小，也是比较它们所对应的绝对值。这种比较的算法可归纳为以下两点：

第一，当被除数与除数同号时，做减法，若得到的余数与除数同号，表示“够减”，否则表示“不够减”。

第二，当被除数与除数异号时，做加法，若得到的余数与除数异号，表示“够减”，否则表示“不够减”。

此算法如表 6.22 所示。

表 6.22 比较算法表

比较 $[x]_{补}$ 与 $[y]_{补}$ 的符号	求余数	比较 $[R_i]_{补}$ 与 $[y]_{补}$ 的符号
同号	$[x]_{补} - [y]_{补}$	同号，表示“够减”
异号	$[x]_{补} + [y]_{补}$	异号，表示“够减”

• 商值的确定

补码除法的商也是用补码表示的，如果我们约定商的末位用“恒置 1”的舍入规则，那么除末位商外，其余各位的商值对正商和负商而言，上商规则是不同的。因为在负商的情况下，除末位商以外，其余任何一位的商与真值都正好相反。因此，上商的算法可归纳为以下两点：

第一，如果 $[x]_{补}$ 与 $[y]_{补}$ 同号，商为正，则“够减”时上商“1”，“不够减”时上商“0”（按原码规则上商）。

第二，如果 $[x]_{补}$ 与 $[y]_{补}$ 异号，商为负，则“够减”时上商“0”，“不够减”时上商“1”（按反码规则上商）。

结合比较规则与上商规则，便可得商值的确定办法，如表 6.23 所示。

表 6.23 商的确定

$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$	商	$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同号	正	同号, 表示“够减”	1
		异号, 表示“不够减”	0
异号	负	异号, 表示“够减”	0
		同号, 表示“不够减”	1

进一步简化, 商值可直接由表 6.24 确定。

表 6.24 简化的商值确定

$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$	商值
同号	1
异号	0

② 商符的形成。

在补码除法中, 商符是在求商的过程中自动形成的。

在小数定点除法中, 被除数的绝对值必须小于除数的绝对值, 否则商大于 1 而溢出。因此, 当 $[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号时, $[x]_{\text{补}} - [y]_{\text{补}}$ 所得的余数 $[R_0]_{\text{补}}$ 必与 $[y]_{\text{补}}$ 异号, 商上“0”, 恰好与商的符号(正)一致; 当 $[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号时, $[x]_{\text{补}} + [y]_{\text{补}}$ 所得的余数 $[R_0]_{\text{补}}$ 必与 $[y]_{\text{补}}$ 同号, 商上“1”, 这也与商的符号(负)一致。可见, 商符是在求商值过程中自动形成的。

此外, 商的符号还可用来判断商是否溢出。例如, 当 $[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号时, 若 $[R_0]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号, 上商“1”, 即溢出。当 $[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号时, 若 $[R_0]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号, 上商“0”, 即溢出。

当然, 对于小数补码运算, 商等于“-1”应该是允许的, 但这需要特殊处理, 为简化问题, 这里不予考虑。

③ 新余数 $[R_{i+1}]_{\text{补}}$ 的获得。

新余数 $[R_{i+1}]_{\text{补}}$ 的获得方法与原码加减交替法极相似, 其算法规则为:

当 $[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号时, 商上“1”, 新余数

$$[R_{i+1}]_{\text{补}} = 2[R_i]_{\text{补}} - [y]_{\text{补}} = 2[R_i]_{\text{补}} + [-y]_{\text{补}}$$

当 $[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号时, 商上“0”, 新余数

$$[R_{i+1}]_{\text{补}} = 2[R_i]_{\text{补}} + [y]_{\text{补}}$$

将此算法列于表 6.25。

表 6.25 新余数的算法

$[R_i]_{\text{补}}$ 与 $[y]_{\text{补}}$	商	新余数 $[R_{i+1}]_{\text{补}}$
同号	1	$[R_{i+1}]_{\text{补}} = 2[R_i]_{\text{补}} + [-y]_{\text{补}}$
异号	0	$[R_{i+1}]_{\text{补}} = 2[R_i]_{\text{补}} + [y]_{\text{补}}$

如果对商的精度没有特殊要求,一般可采用“末位恒置1”法,这种方法操作简单,易于实现,而且最大误差仅为 2^{-n} 。

例 6.26 已知: $x=0.1001, y=0.1101$ 求: $[\frac{x}{y}]_{\text{补}}$

解: 由 $x=0.1001, y=0.1101$

得 $[x]_{\text{补}}=0.1001, [y]_{\text{补}}=0.1101, [-y]_{\text{补}}=1.0011$

其运算过程如表 6.26 所示。

所以 $[\frac{x}{y}]_{\text{补}}=0.1011$

例 6.27 已知: $x=-0.1001, y=+0.1101$ 求: $[\frac{x}{y}]_{\text{补}}$

解: 由 $x=-0.1001, y=+0.1101$

得 $[x]_{\text{补}}=1.0111, [y]_{\text{补}}=0.1101, [-y]_{\text{补}}=1.0011$

其运算过程如表 6.27 所示。

所以 $[\frac{x}{y}]_{\text{补}}=1.0101$

表 6.26 例 6.26 的运算过程

被除数 (余数)	商 上商	说 明
0.1001 + 1.0011	0.0000	$[x]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号, $+[-y]_{\text{补}}$
1.1100 1.1000 + 0.1101	0 0	$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号, 上商 0 ←1 位 $+ [y]_{\text{补}}$
0.0101 0.1010 + 1.0011	01 01	$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号, 上商 1 ←1 位 $+ [-y]_{\text{补}}$
1.1101 1.1010 + 0.1101	010 010	$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$ 异号, 上商 0 ←1 位 $+ [y]_{\text{补}}$
0.0111 0.1110	0101 0101	$[R]_{\text{补}}$ 与 $[y]_{\text{补}}$ 同号, 上商 1 ←1 位, 末位商恒置“1”

表 6.27 例 6.27 的运算过程

被除数 (余数)	商 上商	说 明
1.0111 + 0.1101	0.0000	$[x]_H$ 与 $[y]_H$ 异号, $+[y]_H$
0.0100 0.1000 + 1.0011	1 1	$[R]_H$ 与 $[y]_H$ 同号, 上商 1 ←1 位 $+[y]_H$
1.1011 1.0110 + 0.1101	10 10	$[R]_H$ 与 $[y]_H$ 异号, 上商 0 ←1 位 $+[y]_H$
0.0011 0.0110 + 1.0011	101 101	$[R]_H$ 与 $[y]_H$ 同号, 上商 1 ←1 位 $+[y]_H$
1.1001 1.0010	1010 10101	$[R]_H$ 与 $[y]_H$ 异号, 上商 0 ←1 位, 末位商恒置“1”

(2) 补码加减交替法所需的硬件配置

补码加减交替法所需的硬件配置基本上与图 6.11 相似, 只是图 6.11 中的 S 触发器可以省掉, 因为补码除法的商符在运算中自动形成。此外, 在寄存器中存放的均为补码。

如果机器数采用补码, 实现乘法和除法均用补码运算, 那么, 为了与补码取得相同的寄存器位数, 表 6.26 和表 6.27 中的被除数 (余数) 可取双符号位, 整个运算过程与取一位符号位完全相同 (见例 6.31 表 6.31)。

(3) 补码加减交替法的控制流程

图 6.13 示出了补码加减交替除法的控制流程。

除法开始前, Q 寄存器被清 0, 准备接收商, 被除数的补码在 A 中, 除数的补码在 X 中, 计数器 C 中存放除数的位数 n 。除法开始后, 首先根据两操作数的符号确定是作加法还是减法, 加 (或减) 操作后, 即上第一次商 (商符), 然后 A、Q 同时左移一位, 再根据商值的状态决定加或减除数, 这样重复 n 次后, 再上一次末位商 “1” (恒置 “1” 法), 即得运算结果。

补充说明几点:

① 图中未画出补码除法溢出判断的内容; ② 按流程图所示, 多作一次加 (或减) 法, 其实末位恒置 “1” 前, 只须移位不必作加 (或减) 法; ③ 与原码除一样, 图中均未指出对 0 进行检测, 实际上在除法运算前, 先检测被除数和除数是否为 0, 若被除数为 0, 结果即为 0; 若除数为 0, 结果为无穷大, 这两种情况都无需继续作除法运算。④ 为了节省时间, 上商和移位操作可以同时进行。

以上介绍了计算机定点四则运算方法, 根据这些运算规则, 可以设计乘

法器和除法器。有些机器的乘、除法可用编程来实现。分析上述运算方法对理解机器内部的操作过程和编制乘、除法运算的标准程序都是很有用的。

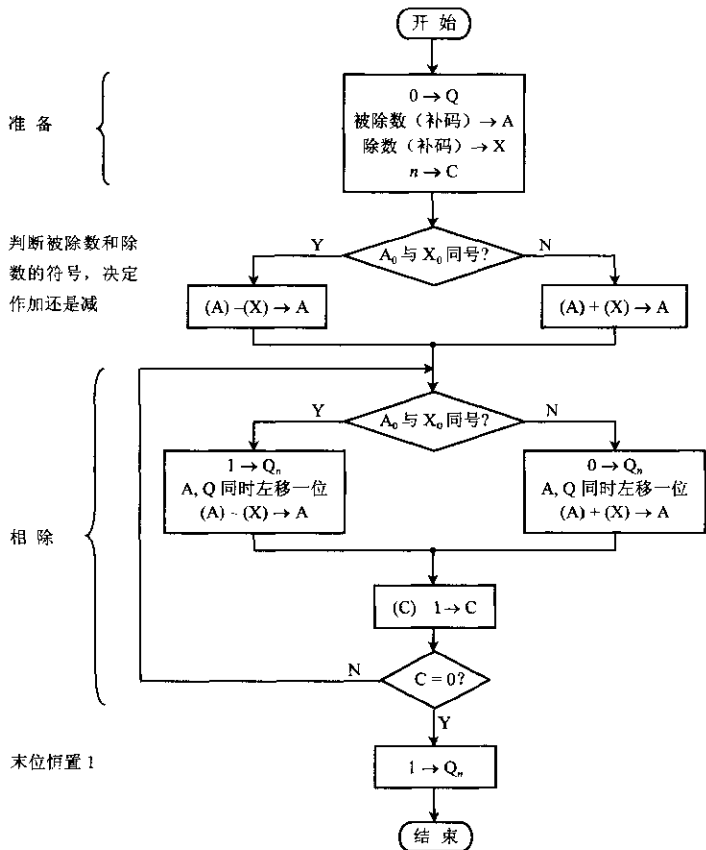


图 6.13 补码加减交替除法控制流程

6.4 浮点四则运算

从 6.2 浮点数的讨论可知，机器中任何一个浮点数可写成

$$x = S_x \cdot r^{j_x}$$

的形式，其中 S_x 为浮点数的尾数，一般为绝对值小于 1 的规格化数（补码表示时允许为 -1），机器中可用原码或补码表示； j_x 为浮点数的阶码，一般为整数，机器中大多用补码或移码表示。 r 为浮点数的基数，常用 2、4、8 或 16 表示。以下以基数为 2 进行讨论。

6.4.1 浮点加减运算

设两个浮点数

$$x = S_x \cdot r^{j_x}$$

$$y = S_y \cdot r^{j_y}$$

由于浮点数尾数的小数点均固定在第一数值位前，所以尾数的加减运算规则与定点数完全相同。但由于其阶码的大小又直接反映尾数有效值的小数点位置，因此当两浮点数阶码不等时，因两尾数小数点的实际位置不一样，尾数部分无法直接进行加减运算。因此，浮点数加减运算必须按以下几步进行：

- ① 对阶，使两数的小数点位置对齐。
- ② 尾数求和，将对阶后的两尾数按定点加减运算规则求和（差）。
- ③ 规格化，为增加有效数字的位数，提高运算精度，必须将求和（差）后的尾数规格化。
- ④ 舍入，为提高精度，要考虑尾数右移时丢失的数值位。
- ⑤ 判断结果，即判断结果是否溢出。

1. 对阶

对阶的目的是使两操作数的小数点位置对齐，即使两数的阶码相等。为此，首先要求出阶差，再按小阶向大阶看齐的原则，使阶小的尾数向右移位，每右移一位，阶码加 1，直到两数的阶码相等为止。右移的次数正好等于阶差。尾数右移时可能会发生数码丢失，影响精度。

例如，两浮点数 $x = 0.1101 \times 2^{01}$ ， $y = (0.1010) \times 2^{11}$ ，求 $x + y$ 。

首先写出 x ， y 在计算机中的补码表示。

$[x]_{\text{补}} = 00,01; 00.1101$ ， $[y]_{\text{补}} = 00,11; 11.0110$

在进行加法前，必须先对阶，故先求阶差：

$$[\Delta_j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 00,01 + 11,01 = 11,10$$

即 $\Delta_j = -2$ ，表示 x 的阶码比 y 的阶码小，再按小阶向大阶看齐的原则，将 x 的尾数右移两位，其阶码加 2。

$$\text{得 } [x]_{\text{补}} = 00,11; 00.0011$$

此时， $\Delta_j = 0$ ，表示对阶完毕。

2. 尾数求和

将对阶后的两个尾数按定点加（减）运算规则进行运算。

如上例中的两数对阶后得：

$$[x]_{\text{补}} = 00,11; 00.0011$$

$$[y]_{\text{补}} = 00,11; 11.0110$$

则求 $[S_x + S_y]_{\text{补}}$ 为：

$$\begin{array}{r} 00.0011 \\ + 11.0110 \\ \hline 11.1001 \end{array} \quad \begin{array}{l} [S_x]_{\text{补}} \\ [S_y]_{\text{补}} \\ [S_x + S_y]_{\text{补}} \end{array}$$

$$\text{即 } [x+y]_{\text{补}} = 00,11; 11.1001$$

3. 规格化

由第二章可知，尾数 S 的规格化形式为

$$\frac{1}{2} \leq |S| < 1 \quad (6.19)$$

如果采用双符号位的补码，则

当 $S > 0$ 时，其补码规格化形式为

$$[S]_{\text{补}} = 00.1 \times \cdots \times \quad (6.20)$$

当 $S < 0$ 时，其补码规格化形式为

$$[S]_{\text{补}} = 11.0 \times \cdots \times \quad (6.21)$$

可见，当尾数的最高数值位与符号位不同时，即为规格化形式，但对 $S < 0$ 时，有两种情况需特殊处理。

① $S = -\frac{1}{2}$ ，则 $[S]_{\text{补}} = 11.100 \cdots 0$ 。此时对于真值 $-\frac{1}{2}$ 而言，它满足式 (6.19)，对于补码 $[S]_{\text{补}}$ 而言，它不满足于式 (6.21)。为了便于硬件判断，特规定 $-\frac{1}{2}$ 不是规格化的数（对补码而言）。

② $S = -1$, 则 $[S]_{\text{补}} = 11.00 \cdots 0$, 因小数补码允许表示 -1, 故 -1 视为规格化的数。

当尾数求和(差)结果不满足式(6.20)或式(6.21)时, 则需规格化。规格化又分左规和右规两种。

(1) 左规

当尾数出现 $00.0 \times \cdots \times$ 或 $11.1 \times \cdots \times$ 时, 需左规。左规时尾数左移一位, 阶码减 1, 直到符合式(6.20)或(6.21)为止。

如上例求和结果为

$$[x+y]_{\text{补}} = 00, 11; 11.1001$$

尾数的第一数值位与符号位相同, 需左规, 即将其左移一位, 同时阶码减 1, 得 $[x+y]_{\text{补}} = 00, 10; 11.0010$ 。

(2) 右规

当尾数出现 $01. \times \cdots \times$ 或 $10. \times \cdots \times$ 时, 表示尾数溢出, 这在定点加减运算中是不允许的, 但在浮点运算中这不算溢出, 可通过右规处理。右规时尾数右移一位, 阶码加 1。

例 6.28 已知: 两浮点数 $x = 0.1101 \times 2^{10}$, $y = 0.1011 \times 2^{01}$ 求: $x+y$

解: x 、 y 在机器中以补码表示为

$$[x]_{\text{补}} = 00, 10; 00.1101$$

$$[y]_{\text{补}} = 00, 01; 00.1011$$

① 对阶

$$\begin{aligned} [\Delta_j]_{\text{补}} &= [j_x]_{\text{补}} - [j_y]_{\text{补}} \\ &= 00, 10 + 11, 11 = 00, 01 \end{aligned}$$

即 $\Delta_j = 1$, 表示 y 的阶码比 x 的阶码小 1, 因此将 y 的尾数向右移 1 位, 阶码相应加 1, 即

$$[y]_{\text{补}} = 00, 10; 00.0101$$

这时 $[y]_{\text{补}}$ 的阶码与 $[x]_{\text{补}}$ 的阶码相等, 阶差为 0, 表示对阶完毕。

② 求和

$$\begin{array}{rcl} 00.1101 & [S_x]_{\text{补}} & \\ + 11.0101 & [S_y]_{\text{补}} & \\ \hline 01.0010 & [S_x + S_y]_{\text{补}} & \end{array}$$

即 $[x+y]_{\text{补}} = 00, 10; 01.0010$

③ 右规

运算结果两符号位不等,表示尾数之和绝对值大于1,需右规,即将尾数之和向右移1位,阶码加1,故得

$$[x+y]_{\text{补}} = 00,11; 00.1001$$

则 $x+y=0.1001 \times 2^{11}$

4. 舍入

在对阶和右规的过程中,可能会将尾数的低位丢失,引起误差,影响了精度。为此可用舍入法来提高尾数的精度。常用的舍入方法有两种。

(1) “0舍1入”法

“0舍1入”法类似于十进制运算中的“四舍五入”法,即在尾数右移时,被移去的最高数值位为0,则舍去;被移去的最高数值位为1,则在尾数的末位加1。这样做可能使尾数又溢出,此时需再做一次右规。

(2) “恒置1”法

尾数右移时,不论丢掉的最高数值位是“1”或“0”,都使右移后的尾数末位恒置“1”。这种方法同样有使尾数变大和变小的两种可能。

综上所述,浮点加减运算经过对阶、尾数求和、规格化和舍入等步骤。与定点加减运算相比,显然要复杂得多。

例 6.29 设 $x=2^{-101} \times (-0.101000)$, $y=2^{-100} \times (+0.111011)$, 并假设阶符取2位,阶码取3位,数符取2位,尾数取6位,求 $x-y$ 。

解: 由 $x=2^{-101} \times (-0.101000)$, $y=2^{-100} \times (+0.111011)$

$$\text{得 } [x]_{\text{补}} = 11,011; 11.011000, [y]_{\text{补}} = 11,100; 00.111011$$

① 对阶

$$[\Delta_j]_{\text{补}} = [j_x]_{\text{补}} - [j_y]_{\text{补}} = 11,011 + 00,100 = 11,111$$

即 $\Delta_j = -1$, 则 x 的尾数向右移一位,阶码相应加1,即

$$[x]_{\text{补}} = 11,100; 11.011000$$

② 求和

$$\begin{aligned} [S_x]_{\text{补}} - [S_y]_{\text{补}} &= [S_x]_{\text{补}} + [-S_y]_{\text{补}} \\ &= 11.101100 + 11.000101 \\ &= 10.110001 \end{aligned}$$

即 $[x-y]_{\text{补}} = 11,100; 10.110001$

尾数符号位出现“10”,需右规。

③ 规格化

右规后得 $[x-y]_{\text{补}} = 11,101; 11.011000$ 1

④ 舍入处理

采用0舍1入法,其尾数右规时末位丢1,则有

$$\begin{array}{r} 11.011000 \\ + \quad \quad 1 \\ \hline 11.011001 \end{array}$$

所以 $[x-y]_{\text{补}} = 11,101; 11.011001$

5. 溢出判断

与定点加减法一样，浮点加减运算最后一步也需判溢出。在浮点规格化中已指出，当尾数之和（差）出现 $01.\times\times\times$ 或 $10.\times\times\times$ 时，并不表示溢出，只有将此数右规后，再根据阶码来判断浮点运算结果是否溢出。

若机器数为补码，尾数为规格化形式，并假设阶符取 2 位，阶码取 7 位，数符取 2 位，尾数取 n 位，则它们能表示的补码在数轴上的表示范围如图 6.14。

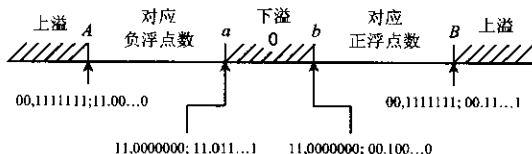


图 6.14 补码在数轴上的表示

图中 A 、 B 、 a 、 b 的坐标均为补码表示，分别对应最小负数、最大正数、最大负数和最小正数。它们所对应的真值分别是：

- A 最小负数 $2^{+127} \times (-1)$
- B 最大正数 $2^{+127} \times (1-2^{-n})$
- a 最大负数 $2^{-128} \times (-2^{-1}-2^{-n})$
- b 最小正数 $2^{-128} \times 2^{-1}$

请读者注意，由于图 6.14 所示的 A 、 B 、 a 、 b 均为补码规格化的形式，故其对应的真值与图 6.2 所示的结果有所不同。

图 6.14 a 、 b 之间的阴影部分，对应阶码小于 -128 的情况，叫做浮点数的下溢。下溢时，浮点数值趋于零，故机器不做溢出处理，仅把它作为机器零。

图 6.14 的 A 、 B 两侧阴影部分，对应阶码大于 +127 的情况，叫做浮点数的上溢。此刻，浮点数真正溢出，机器需停止运算，作溢出中断处理。一般说浮点溢出，均是指上溢。

可见，浮点机的溢出与否可由阶码的符号决定。即

阶码 $[\text{阶}]_{\text{补}} = 01, \times \times \times$ 为上溢

阶码 $[\text{阶}]_{\text{补}} = 10, \times \times \times$ 为下溢，按机器零处理

当阶符为“01”时，需做溢出处理。

例 6.29 经舍入处理后得 $[x-y]_{\text{补}} = 11,101; 11.011001$ ，阶符为“11”，不溢

出, 故最终结果为

$$x-y=2^{-011}\times(-0.100111)$$

当计算机中阶码用移码表示时, 移码运算规则参见浮点乘除运算。
最后可得浮点加减运算的流程。

6. 浮点加减运算流程

图 6.15 为浮点补码加减运算的流程图。

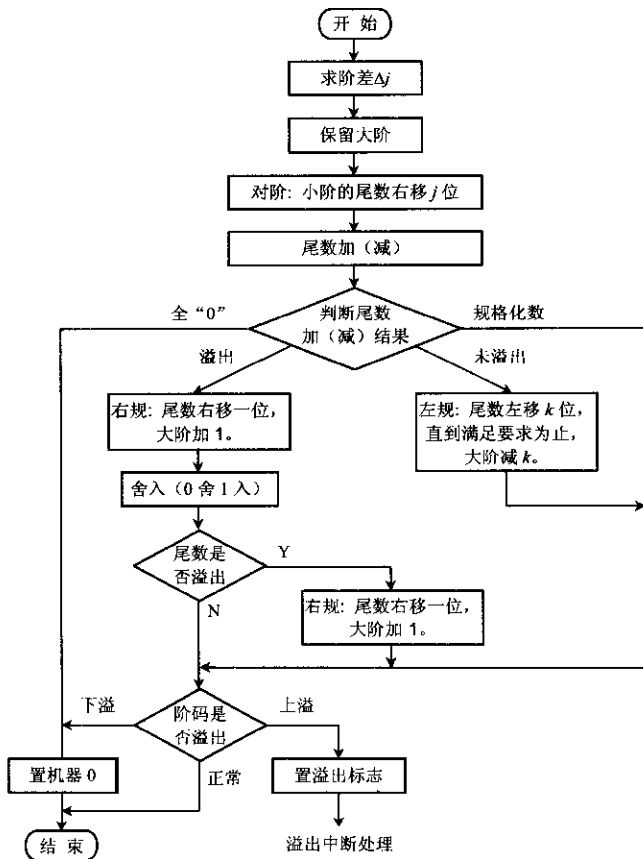


图 6.15 浮点补码加减运算流程

6.4.2 浮点乘除法运算

两个浮点数相乘，其乘积的阶码应为相乘两数的阶码之和，其乘积的尾数应为相乘两数的尾数之积。两个浮点数相除，商的阶码为被除数的阶码减去除数的阶码，其尾数为被除数的尾数除以除数的尾数所得的商。可用下式描述。

$$\text{设两浮点数} \quad x = S_x \cdot r^{j_x}$$

$$y = S_y \cdot r^{j_y}$$

$$\text{则} \quad x \cdot y = (S_x \cdot S_y) \times r^{j_x + j_y}$$

$$\frac{x}{y} = \frac{S_x}{S_y} \cdot r^{j_x - j_y}$$

在运算中也要考虑规格化和舍入问题。

1. 阶码运算

若阶码用补码运算，乘积的阶码为 $[j_x]_{\text{补}} + [j_y]_{\text{补}}$ ，商的阶码为 $[j_x]_{\text{补}} - [j_y]_{\text{补}}$ ，两个同号的阶码相加或异号的阶码相减可能产生溢出，此时应作溢出判断。

若阶码用移码运算，则

$$\text{因为} \quad [j_x]_{\text{移}} = 2^n + j_x \quad -2^n \leq j_x < 2^n \quad (n \text{ 为整数的位数})$$

$$[j_y]_{\text{移}} = 2^n + j_y \quad -2^n \leq j_y < 2^n \quad (n \text{ 为整数的位数})$$

$$\begin{aligned} \text{所以} \quad [j_x]_{\text{移}} + [j_y]_{\text{移}} &= 2^n + j_x + 2^n + j_y \\ &= 2^n + [2^n + (j_x + j_y)] \\ &= 2^n + [j_x + j_y]_{\text{移}} \end{aligned}$$

可见，直接用移码求阶码和时，其最高位多了一个 2^n ，要得到移码形式的结果，必须减去 2^n 。

由于同一个真值的移码和补码其数值部分完全相同，而符号位正好相反，即

$$[j_y]_{\text{补}} = 2^{n+1} + j_y \quad (\text{mod } 2^{n+1})$$

因此如果求阶码和可用下式完成：

$$\begin{aligned} [j_x]_{\text{移}} + [j_y]_{\text{补}} &= 2^n + j_x + 2^{n+1} + j_y \\ &= 2^{n+1} + [2^n + (j_x + j_y)] \\ &= [j_x + j_y]_{\text{移}} \quad (\text{mod } 2^{n+1}) \end{aligned}$$

则直接可得移码形式。

同理，当作除法运算时，商的阶码可用下式完成：

$$[i_x]_{移} + [-j_y]_{补} = [i_x - j_y]_{移}$$

可见进行移码加减运算时，只需将移码表示的加数或减数的符号位取反（即变为补码），然后进行运算，就可得阶和（或阶差）的移码。

阶码采用移码表示后又如何判断溢出呢？如果在原有移码符号位的前面（即高位）再增加一位符号位，并规定该位恒用“0”表示，便能方便地进行溢出判断。溢出的条件是运算结果移码的最高符号位为1。此时若低位符号位为0，表示上溢；低位符号位为1，表示下溢。如果运算结果移码的最高符号位为0，即表明没溢出。此时若低位符号位为1，表明结果为正；低位符号位为0，表示结果为负。

例如，若阶码取3位（不含符号位），则其对应的真值范围是-8~+7。

当 $j_x = +101$, $j_y = +100$ 时，则有

$$[i_x]_{移} = 01, 101; [j_y]_{补} = 00, 100$$

故 $[i_x + j_y]_{移} = [i_x]_{移} + [j_y]_{补} = 01, 101 + 00, 100 = 10, 001$ 结果上溢

$$[i_x - j_y]_{移} = [i_x]_{移} + [-j_y]_{补} = 01, 101 + 11, 100 = 01, 001 \quad \text{结果为} +1$$

当 $j_x = -101$, $j_y = -100$ 时，则有

$$[i_x]_{移} = 00, 011, [j_y]_{补} = 11, 100$$

故 $[i_x + j_y]_{移} = [i_x]_{移} + [j_y]_{补} = 00, 011 + 11, 100 = 11, 111$ 结果下溢

$$[i_x - j_y]_{移} = [i_x]_{移} + [-j_y]_{补} = 00, 011 + 00, 100 = 00, 111 \quad \text{结果为} -1$$

2. 尾数运算

(1) 浮点乘法尾数运算

两个浮点数的尾数相乘，可按下列步骤进行：

① 检测两个尾数中是否有一个为0，若有一个为0，乘积必为0，不再作其他操作；如果两尾数均不为0，则可进行乘法运算。

② 两个浮点数的尾数相乘可以采用定点小数的任何一种乘法运算来完成。相乘结果可能要左规，左规时调整阶码后如果发生阶下溢，则作机器零处理；如果发生阶上溢，则作溢出处理。此外，尾数相乘会得到一个双倍字长的结果，若限定只取1倍字长，则乘积的若干低位将会丢失。如何处理丢失的各位值，通常有两种办法。

其一，无条件的丢掉正常尾数最低位之后的全部数值，这种办法被称为截断处理，其优点是处理简单，但影响精度。

其二，按浮点加减运算讨论的两种舍入原则进行舍入处理。对于原码，采用0舍1入法时，不论其值是正数或负数，“舍”使数的绝对值变小，“入”使数的绝对值变大。对于补码，采用0舍1入法时，若丢失的位不是全0，对正数来说，“舍”、“入”的结果与原码分析正好相同；对负数来说，“舍”、“入”

的结果与原码分析正好相反,即“舍”使绝对值变大,“入”使绝对值变小。为了使原码、补码舍入处理后的结果相同,对负数的补码可采用如下规则进行舍入处理。

- ① 当丢失的各位均为 0 时,不必舍入;
- ② 当丢失的各位数中的最高位为 0 时,且以下各位不全为 0;或丢失的各位数中的最高位为 1,且以下各位均为 0 时,则舍去被丢失的各位;
- ③ 当丢失的各位数中的最高位为 1,且以下各位又不全为 0 时,则在保留尾数的最末位加 1 修正。

例如对下列 4 个补码进行只保留小数点后 4 位有效数字的舍入操作如表 6.28 所示。

表 6.28 补码舍入操作实例

$[x]_{\text{补}}$ 舍入前	舍入后	对应真值 x
1.011110000	1.0111 (不舍不入)	-0.1001
1.01111000	1.0111 (舍)	-0.1001
1.01110101	1.0111 (舍)	-0.1001
1.01111100	1.1000 (入)	-0.1000

如果将上述 4 个补码变成原码后再舍入,其结果列于表 6.29。

表 6.29 原码舍入操作实例

$[x]_{\text{原}}$ 舍入前	舍入后	对应真值 x
1.10010000	1.1001 (不舍不入)	-0.1001
1.10001000	1.1001 (入)	-0.1001
1.10001011	1.1001 (入)	-0.1001
1.10000100	1.1000 (舍)	-0.1000

比较表 6.28 和表 6.29 可见,按照上述的约定对负数的补码进行舍入处理,与对其原码进行舍入处理后的真值是一样的。

下面举例说明浮点乘法运算的全过程。

设机器数阶码取 3 位(不含阶符),尾数取 7 位(不含数符),要求阶码用移码运算,尾数用补码运算,最后结果保留 1 倍字长。

例 6.30 已知: $x=2^{-101} \times 0.0110011$, $y=2^{011} \times (-0.1110010)$ 求: xy

解: 由 $x=2^{-101} \times 0.0110011$, $y=2^{011} \times (-0.1110010)$

得 $[x]_{\text{补}} = 11,011; 00.0110011$

$[y]_{\text{补}} = 00,011; 11.0001110$

① 阶码运算

$[j_x]_{\text{移}} = 00,011$, $[j_y]_{\text{补}} = 00,011$

$[j_x + j_y]_{\text{移}} = [j_x]_{\text{移}} + [j_y]_{\text{补}}$

$$=00,011+00,011$$

$$=00,110 \quad \text{对应真值}-2$$

② 尾数相乘 (采用 Booth 算法)

其过程如表 6.30 所示。

表 6.30 例 6.30 尾数相乘过程

部分积	乘 数	y_{n+1}	说 明
00.00000000 00.00000000 + 11.1001101	1.0001110 0 1000111	0 0	$\rightarrow 1$ 位 $+[-S_x]_H$
11.1001101 11.1100110 11.1110011 11.1111001 + 00.0110011	0 10100011 01010001 10101000	0 1 1 1	$\rightarrow 1$ 位 $\rightarrow 1$ 位 $\rightarrow 1$ 位 $+ [S_x]_H$
00.0101100 00.0010110 00.0001011 00.0000101 + 11.1001101	1010 01010100 00101010 10010101	0 0 0 0	$\rightarrow 1$ 位 $\rightarrow 1$ 位 $\rightarrow 1$ 位 $+ [-S_x]_H$
11.1010010	1001010		

③ 规格化

尾数相乘结果为 $[S_x \cdot S_y]_H = 11.10100101001010$, 需左规, 即

$$[x \cdot y]_H = 11,110; 11.10100101001010$$

左规后 $[x \cdot y]_H = 11,101; 11.01001010010100$

④ 舍入处理

尾数为负, 按负数补码的舍入规则, 取 1 倍字长, 丢失的 7 位为 0010100, 应“舍”, 故最终结果为

$$[x \cdot y]_H = 11,101; 11.0100101$$

$$xy = 2^{-011} \times (-0.1011011)$$

(2) 浮点除法尾数运算

两个浮点数的尾数相除, 可按下列步骤进行:

① 检测被除数是否为 0, 若为 0, 则商为 0; 再检测除数是否为 0, 若为 0, 则商为无穷大, 另作处理。若两数均不为 0, 则可进行除法运算。

② 两浮点数尾数相除同样可采取定点小数的任何一种除法运算来完成。对已规格化的尾数, 为了防止除法结果溢出, 可先比较被除数和除数的绝对值, 如果被除数的绝对值大于除数的绝对值, 则先将被除数右移一位, 其阶码加 1, 再作尾数相除。此时所得结果必然是规格化的定点小数。

例 6.31 按补码浮点运算步骤, 计算 $[2^5 \times (+\frac{9}{16})] \div [2^3 \times (-\frac{13}{16})]$

解: 令 $x = [2^5 \times (+\frac{9}{16})] = 2^{10} \times (0.1001)$

$$y = [2^3 \times (-\frac{13}{16})] = 2^{011} \times (-0.1101)$$

所以 $[x]_{\text{补}} = 00,101; 00.1001$

$[y]_{\text{补}} = 00,011; 11.0011$, $[-S_y]_{\text{补}} = 00.1101$

① 阶码相减

$$[j_x]_{\text{补}} - [j_y]_{\text{补}} = 00,101 - 00,011 = 00,101 + 11,101 = 00,010$$

② 尾数相除(采用补码除法)

其过程如表 6.31 所示。表中被除数(余数)采用双符号位, 与采用一位符号位结果一致。

所以 $[\frac{S_x}{S_y}]_{\text{补}} = 1.0101$

③ 规格化

尾数相除结果已为规格化数。

所以 $[\frac{x}{y}]_{\text{补}} = 00,010; 11.0101$

$$\text{则} [\frac{x}{y}] = 2^{010} \times (-0.1011) = [2^2 \times (-\frac{11}{16})]$$

表 6.31 例 6.31 尾数相除过程

被除数(余数)	商	说 明
$\begin{array}{r} 00.1001 \\ + 11.0011 \end{array}$		$[S_x]_{\text{补}}$ 与 $[S_y]_{\text{补}}$ 异号, $+ [S_y]_{\text{补}}$
$\begin{array}{r} 11.1100 \\ 11.1000 \\ + 00.1101 \end{array}$	$\begin{array}{r} 1 \\ 1 \end{array}$	$[R]_{\text{补}}$ 与 $[S_y]_{\text{补}}$ 同号, 上商 1 ←1 位 $+ [-S_y]_{\text{补}}$
$\begin{array}{r} 00.0101 \\ 00.1010 \\ + 11.0011 \end{array}$	$\begin{array}{r} 10 \\ 10 \end{array}$	$[R]_{\text{补}}$ 与 $[S_y]_{\text{补}}$ 异号, 上商 0 ←1 位 $+ [S_y]_{\text{补}}$
$\begin{array}{r} 11.1101 \\ 11.1010 \\ + 00.1101 \end{array}$	$\begin{array}{r} 101 \\ 101 \end{array}$	$[R]_{\text{补}}$ 与 $[S_y]_{\text{补}}$ 同号, 上商 1 ←1 位 $+ [-S_y]_{\text{补}}$
$\begin{array}{r} 00.0111 \\ + 00.1110 \end{array}$	$\begin{array}{r} 1010 \\ 10101 \end{array}$	$[R]_{\text{补}}$ 与 $[S_y]_{\text{补}}$ 异号, 上商 0 ←1 位, 末位商恒置 1

6.4.3 浮点运算所需的硬件配置

由于浮点运算分阶码和尾数两部分，因此浮点运算器的硬件配置比定点运算器复杂。分析浮点四则运算发现，对于阶码只有加减运算，对于尾数则有加、减、乘、除四种运算。可见浮点运算器主要由两个定点运算部件组成，一个是阶码运算部件，用来完成阶码加、减，以及控制对阶时小阶的尾数右移次数和规格化时对阶码的调整；另一个是尾数运算部件，用来完成尾数的四则运算以及判断尾数是否已规格化，此外，还需有判断运算结果是否溢出的电路等。

现代计算机可把浮点运算部件做成独立的选件，或叫协处理器，用户可根据需要选择，不用选件的机器，也可用编程的办法来完成浮点运算，不过这将会影响机器的运算速度。

例如，Intel 80287 是浮点协处理器，它可与 Intel 80286 或 80386 微处理器配合处理浮点数的算术运算和多种函数计算。

6.5 算术逻辑单元

针对每一种算术运算，都必须有一个相对应的基本硬件配置，其核心部件是加法器和寄存器。当需完成逻辑运算时，势必需要配置相应的逻辑电路，而 ALU 电路是既能完成算术运算又能完成逻辑运算的部件。

6.5.1 ALU 电路

图 6.16 是 ALU 框图。图中 A_i 和 B_i 为输入变量； k_i 为控制信号， k_i 的不同取值可决定该电路作哪一种算术运算或哪一种逻辑运算； F_i 是输出函数。

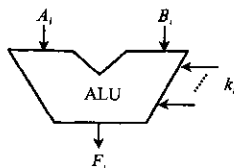


图 6.16 ALU 框图

现在 ALU 电路已制成集成电路芯片，如 74181 是能完成四位二进制代码的算逻运算部件，其外特性如图 6.17 所示。

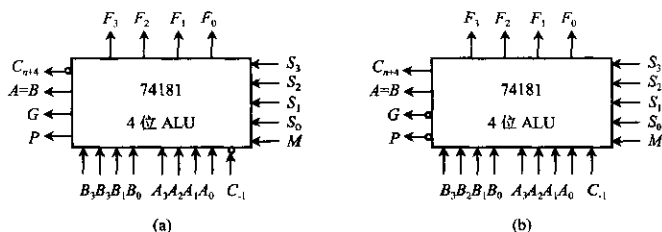


图 6.17 74181 外特性示意图

74181 有两种工作方式：正逻辑和负逻辑，图 6.17(a)、(b)分别示意了这两种方式。表 6.32 列了其算术/逻辑运算功能，逻辑电路参见附录 6C 的图 6.30。

表 6.32 74181ALU 的算术/逻辑运算功能表

工作方式选择 输入 $S_3S_2S_1S_0$	负逻辑输入或输出		正逻辑输入或输出	
	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=0$)	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=1$)
0000	$\overline{A}$	A 减 1	$\overline{A}$	A
0001	$\overline{AB}$	AB 减 1	$\overline{A+B}$	$A+B$
0010	$\overline{A+B}$	$A\overline{B}$ 减 1	$\overline{AB}$	$A+\overline{B}$
0011	逻辑 1	减 1	逻辑 0	减 1
0100	$\overline{A+B}$	A 加 $(A+\overline{B})$	$\overline{AB}$	A 加 $\overline{AB}$
0101	$\overline{B}$	AB 加 $(A+\overline{B})$	$\overline{B}$	$(A+B)$ 加 $\overline{AB}$
0110	$\overline{A\oplus B}$	A 减 B 减 1	$A\oplus B$	A 减 B 减 1
0111	$A+\overline{B}$	$A+\overline{B}$	$\overline{AB}$	$A\overline{B}$ 减 1
1000	$\overline{AB}$	A 加 $(A+B)$	$\overline{A+B}$	A 加 AB
1001	$A\oplus B$	A 加 B	$\overline{A\oplus B}$	A 加 B
1010	B	$A\overline{B}$ 加 $(A+B)$	B	$(A+\overline{B})$ 加 AB
1011	$A+B$	$A+B$	AB	AB 减 1
1100	逻辑 0	A 加 A^*	逻辑 1	A 加 A^*
1101	$A\overline{B}$	AB 加 A	$A+\overline{B}$	$(A+B)$ 加 A
1110	AB	$A\overline{B}$ 加 A	$A+B$	$(A+\overline{B})$ 加 A
1111	A	A	A	A 减 1

(1) 1=高电平；0=低电平；(2) *表示每一位均移到下一个更高位，即 $A^*=2A$

以正逻辑为例， $B_3\sim B_0$ 和 $A_3\sim A_0$ 是两个操作数， $F_3\sim F_0$ 为输出结果。 C_{-1} 表示最低位的外来进位， C_{n+4} 是 74181 向高位的进位； P 、 G 可供先行进位使用（有关 P 、 G 的具体含义参见 6.5.2）。 M 用于区别算术运算还是逻辑运算； $S_3\sim S_0$ 的不同取值可实现不同的运算。例如，当 $M=1$ ， $S_3\sim S_0=0110$ 时，74181 作逻辑运算 $A\oplus B$ ；当 $M=0$ ， $S_3\sim S_0=0110$ 时，74181 作算术运算。由表 6.32 可见，在正逻辑条件下， $M=0$ ， $S_3\sim S_0=0110$ ，且 $C_{-1}=1$ 时，完成 A 减 B 减 1 的

操作。若想完成 A 减 B 运算, 可使 $C_{i-1}=0$ 。请读者注意, 74181 算术运算是用补码实现的, 其中减数的反码是由内部电路形成的, 而末位加“1”, 则通过 $C_{i-1}=0$ 来体现 (图 6.17(a) C_{i-1} 输入端处有一个小圈, 意味着 $C_{i-1}=0$ 反相后为 1)。尤其要注意的是, ALU 为组合逻辑电路, 因此实际应用 ALU 时, 其输入端口 A 和 B 必须与锁存器相连, 而且在运算的过程中锁存器的内容是不变的。其输出也必须送至寄存器中保存。现在有的芯片将寄存器和 ALU 电路集成在一个芯片内, 如 29C101, 其框图如图 6.18 所示 (图中 ALU 的控制端 $I_8 \sim I_0$ 未画出)。

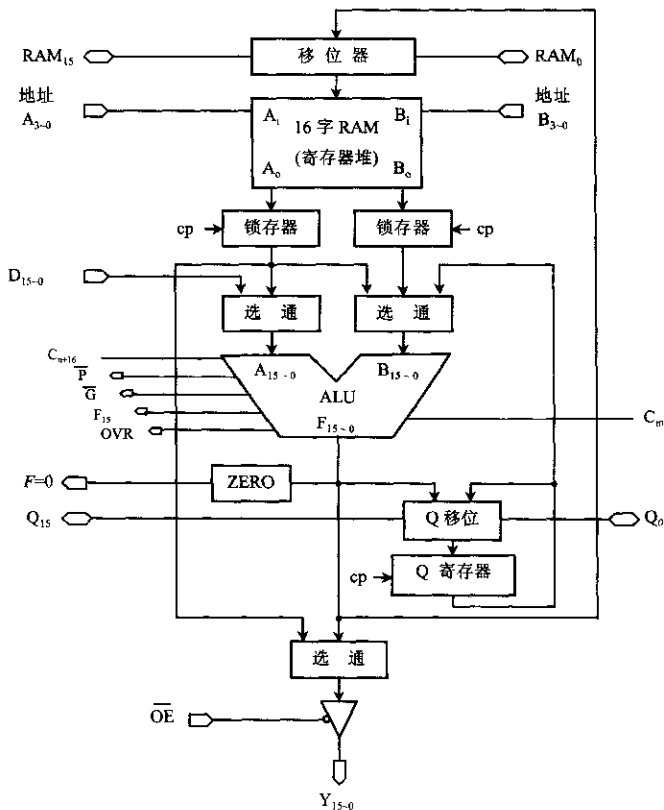


图 6.18 29C101 框图

该芯片的核心部件是一个容量为 16 字的双端口 RAM 和一个高速 ALU 电路。

RAM 可视为由 16 个寄存器组成的寄存器堆。只要给出 A_i 口或 B_i 口的四位地址, 就可以从 A_0 出口或 B_0 出口读出对应于口地址的存储单元内容。写入时, 只能写入到由 B_i 口指定的那个单元内。参与操作的两个数分别由 RAM 的 A_0 、 B_0 出口输出至两个锁存器中。

ALU 受 $I_8 \sim I_0$ 控制, I_1 、 I_0 控制 ALU 的数据源; I_5 、 I_4 、 I_3 控制 ALU 所能完成的 3 种算术运算和 5 种逻辑运算; $I_8 \sim I_6$ 用来控制 RAM 和 Q 移位器, 决定是否移位以及 Y 口输出是来自 RAM 的 A 出口还是 ALU 的 F 出口。

ALU 的 C_m 为低位来的外来进位, C_{n+16} 为向高位的进位, 可供 29C101 级联时用。ALU 结果为 0 时, $F=0$ 可直接输出, OVR 为溢出标记。而 $\bar{P}$ 、 $\bar{G}$ 与 74181 的 P、G 含义相同, 它们可供先行进位方式时使用。ALU 的输出可直接通过移位器存入 RAM, 也可通过选通门在 $\overline{OE}$ 有效时, 从 Y_{15-0} 输出。Q 寄存器主要为乘法和除法服务, $D_{15} \sim D_0$ 为 16 位立即数的输入口。

6.5.2 快速进位链

随着操作数位数的增加, 电路中进位的速度对运算时间的影响也越大, 为了提高运算速度, 本节将通过分析进位过程的设计快速进位链。

1. 并行加法器

并行加法器由若干个全加器组成, 如图 6.19 所示。 $n+1$ 个全加器级联, 就组成了一个 $n+1$ 位的并行加法器。

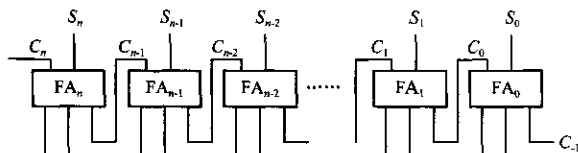


图 6.19 并行加法器示意图

由于每位全加器的进位输出是高位一位全加器的进位输入, 因此当全加器有进位时, 这种一级一级传递进位的过程, 将会大大影响运算速度。

由全加器的逻辑表达式可知:

$$\text{和 } S_i = \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1}$$

$$\text{进位 } C_i = \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i \bar{C}_{i-1} + A_i B_i C_{i-1}$$

$$= A_i B_i + (A_i + B_i) C_{i-1}$$

可见, C_i 进位由两部分组成: 本地进位 $A_i B_i$, 可记作 d_i , 与低位无关; 传递进位 $(A_i + B_i) C_{i-1}$, 与低位有关, 可称 $A_i + B_i$ 为传递条件, 记作 t_i 则:

$$C_i = d_i + t_i C_{i-1}$$

由 C_i 的组成可以将逐级传递进位的结构, 转换为以进位链的方式实现快速进位。目前进位链通常采用串行和并行两种。

2. 串行进位链

串行进位链是指并行加法器中的进位信号采用串行传递, 如图 6.19 就是一个典型的串行进位加法器。

以四位并行加法器为例, 每一位的进位表达式可示为:

$$\left. \begin{aligned} C_0 &= d_0 + t_0 C_{-1} \\ C_1 &= d_1 + t_1 C_0 \\ C_2 &= d_2 + t_2 C_1 \\ C_3 &= d_3 + t_3 C_2 \end{aligned} \right\} \quad (6.22)$$

由 (6.22) 式可见, 采用与非逻辑电路可方便地实现进位传递, 如图 6.20 所示。

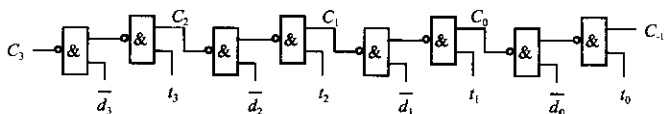


图 6.20 四位串行进位链

若设与非门的级延迟时间为 t_y , 那么当 d_i 、 t_i 形成后, 共需 $8t_y$ 便可产生最高位的进位。实际上每增加一位全加器, 进位时间就会增加 $2t_y$ 。 n 位全加器的最长进位时间为 $2nt_y$ 。

3. 并行进位链

并行进位链是指并行加法器中的进位信号是同时产生的, 又称先行进位、跳跃进位等。理想的并行进位链是 n 位全加器的 n 位进位同时产生, 但实际实现有困难。通常并行进位链有单重分组和双重分组两种实现方案。

(1) 单重分组跳跃进位

单重分组跳跃进位就是将 n 位全加器分成若干小组, 小组内的进位同时产生, 小组与小组之间采用串行进位, 这种进位又有组内并行、组间串行之称。

以四位并行加法器为例, 对 6.22 式稍作变换, 便可获得并行进位表达式:

$$\left. \begin{aligned} C_0 &= d_0 + t_0 C_{-1} \\ C_1 &= d_1 + t_1 C_0 = d_1 + t_1 d_0 + t_1 t_0 C_{-1} \\ C_2 &= d_2 + t_2 C_1 = d_2 + t_2 d_1 + t_2 t_1 d_0 + t_2 t_1 t_0 C_{-1} \\ C_3 &= d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_{-1} \end{aligned} \right\} \quad (6.23)$$

按 (6.23) 式可得与其对应的逻辑图, 如图 6.21 所示。

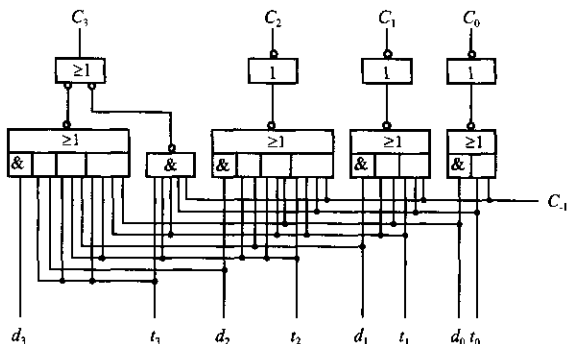


图 6.21 四位一组并行进位链

设与或非门的级延迟时间为 $1.5t_p$, 与非门的级延迟时间仍为 $1t_p$, 则 d_i 、 t_i 形成后, 只需 $2.5t_p$ 就可产生全部进位。

如果将 16 位的全加器按四位一组分组, 便可得单重分组跳跃进位链框图, 如图 6.22 所示。

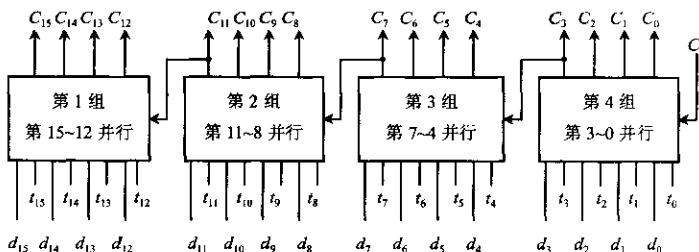


图 6.22 单重分组跳跃进位链框图

不难理解在 d_i 、 t_i 形成后, 经 $2.5t_p$ 可产生 C_3 、 C_2 、 C_1 、 C_0 四个进位信息, 经 $10t_p$ 就可产生全部进位, 而 $n=16$ 的串行进位链的全部进位时间为 $32t_p$, 可见单重分组方案进位时间仅为串行进位链的三分之一。

但随着 n 的增大, 其优势便很快减弱, 如当 $n=64$ 时, 按四位分组, 共为 16 组, 组间有 16 位串行进位, 在 d_i 、 t_i 形成后, 还需经 $40t_p$ 才能产生全部进位, 显然进位时间太长。如果能使组间进位也同时产生, 必然会更大提高进

位速度，这就是组内、组间均为并行进位的方案。

(2) 双重分组跳跃进位

双重分组跳跃进位就是将 n 位全加器分成几个大组，每个大组又包含几个小组，而每个大组内所包含的各个小组的最高位进位是同时形成的，大组与大组间采用串行进位。因各小组最高位进位是同时形成的，小组内的其他进位也是同时形成的（注意：小组内的其他进位与小组的最高位进位并不是同时产生的），故又有组（小组）内并行、组（小组）间并行之称。图 6.23 是一个 32 位并行加法器双重分组跳跃进位链的框图。

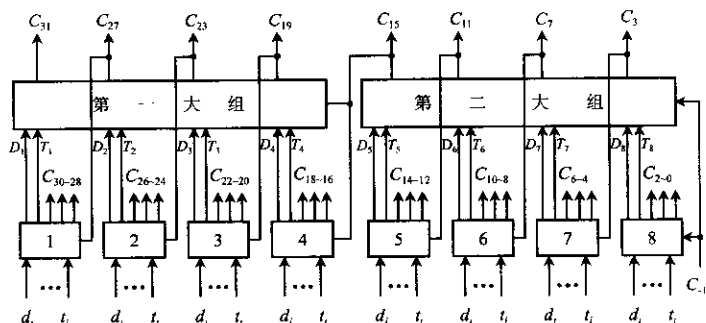


图 6.23 32 位并行加法器双重分组跳跃进位链框图

图中共分两大组，每个大组内包含四个小组，第一大组内的四个小组的最高位进位 C_{31} 、 C_{27} 、 C_{23} 、 C_{19} 是同时产生的；第二大组内四个小组的最高位进位 C_{15} 、 C_{11} 、 C_7 、 C_3 也是同时产生的，而第二大组向第一大组的进位 C_{15} 采用串行进位方式。

以第二大组为例，分析各进位的逻辑关系。

按 (6.23) 式，可写出第八小组的最高位进位表达式：

$$\begin{aligned} C_3 &= d_3 + t_3 C_2 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0 + t_3 t_2 t_1 t_0 C_1 \\ &= D_8 + T_8 C_1 \end{aligned}$$

式中 $D_8 = d_3 + t_3 d_2 + t_3 t_2 d_1 + t_3 t_2 t_1 d_0$ 仅与本小组内的 d_i 、 t_i 有关，不依赖外来进位 C_1 ，故称 D_8 为第八小组的本地进位； $T_8 = t_3 t_2 t_1 t_0$ 是将低位进位 C_1 传向高位小组的条件，故称 T_8 为第八小组的传送条件。

同理可写出第五、六、七小组的最高位进位表达式：

$$\begin{aligned}
 \text{第七小组 } C_7 &= d_7 + t_7 d_6 + t_7 t_6 d_5 + t_7 t_6 t_5 d_4 + t_7 t_6 t_5 t_4 C_3 \\
 &= D_7 + T_7 C_3 \\
 \text{第六小组 } C_{11} &= d_{11} + t_{11} d_{10} + t_{11} t_{10} d_9 + t_{11} t_{10} t_9 d_8 + t_{11} t_{10} t_9 t_8 C_7 \\
 &= D_6 + T_6 C_7 \\
 \text{第五小组 } C_{15} &= d_{15} + t_{15} d_{14} + t_{15} t_{14} d_{13} + t_{15} t_{14} t_{13} d_{12} + t_{15} t_{14} t_{13} t_{12} C_{11} \\
 &= D_5 + T_5 C_{11}
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} C_7 \\ C_{11} \\ C_{15} \end{aligned}} \right\} (6.24)$$

进一步展开又得

$$\begin{aligned}
 C_3 &= D_8 + T_8 C_{-1} \\
 C_7 &= D_7 + T_7 C_3 = D_7 + T_7 D_8 + T_7 T_8 C_{-1} \\
 C_{11} &= D_6 + T_6 C_7 = D_6 + T_6 D_7 + T_6 T_7 D_8 + T_6 T_7 T_8 C_{-1} \\
 C_{15} &= D_5 + T_5 C_{11} = D_5 + T_5 D_6 + T_5 T_6 D_7 + T_5 T_6 T_7 D_8 + T_5 T_6 T_7 T_8 C_{-1}
 \end{aligned}
 \quad \left. \vphantom{\begin{aligned} C_3 \\ C_7 \\ C_{11} \\ C_{15} \end{aligned}} \right\} (6.25)$$

可见，式 (6.25) 和式 (6.23) 极为相似，因此，只需将图 6.21 中的 d_0 、 d_1 、 d_2 、 d_3 改为 D_8 、 D_7 、 D_6 、 D_5 ，又将 t_0 、 t_1 、 t_2 、 t_3 改为 T_8 、 T_7 、 T_6 、 T_5 便可构成第二重跳跃进位链，即大组跳跃进位链，如图 6.24 所示。

由图可见，当 D_i 、 T_i ($i=5\sim 8$) 及外来进位 C_{-1} 形成后，再经过 $2.5t_i$ 便可同时产生 C_{15} 、 C_{11} 、 C_7 、 C_3 。至于 D_i 和 T_i 可由式 (6.24) 求得，它们都是由小组产生的，按其逻辑表达式可画出相应的电路。实际上只需对图 6.21 略作修改便得双重分组进位链中的小组进位链线路，该线路能产生 D_i 和 T_i ，如图 6.25 所示。

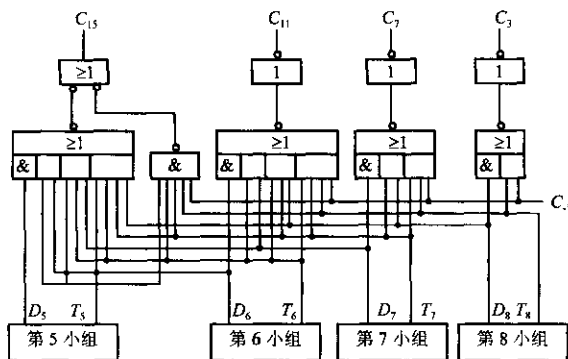


图 6.24 双重分组跳跃进位链的大组进位线路

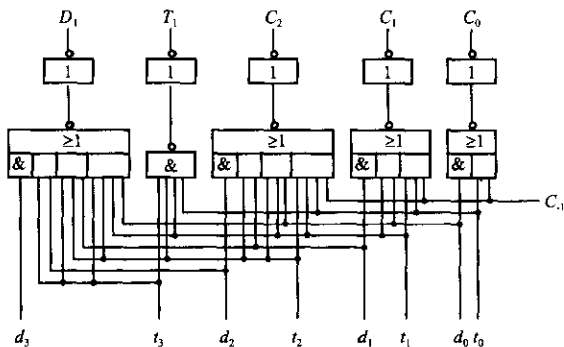


图 6.25 双重分组跳跃进位链的小组进位线路

可见，每小组可产生本小组的本地进位 D_i 和进位条件 T_i 以及组内的各低位进位，但不能产生组内最高位进位，即

第五组形成 D_5 、 T_5 、 C_{14} 、 C_{13} 、 C_{12} ，不产生 C_{15} ；

第六组形成 D_6 、 T_6 、 C_{10} 、 C_9 、 C_8 ，不产生 C_{11} ；

第七组形成 D_7 、 T_7 、 C_6 、 C_5 、 C_4 ，不产生 C_7 ；

第八组形成 D_8 、 T_8 、 C_2 、 C_1 、 C_0 ，不产生 C_3 。

图 6.24 和图 6.25 两种类型的线路可构成 16 位加法器的双重分组跳跃进位链框图，如图 6.26 所示。

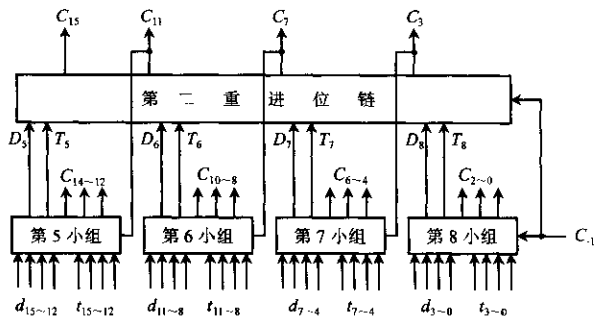


图 6.26 16 位并行加法器的双重分组跳跃进位链框图

由图 6.24、图 6.25 和图 6.26 可计算出从 d_i 、 t_i 及 C_1 （外来进位）形成后

开始, 经 $2.5t_p$ 形成 C_2 、 C_1 、 C_0 和全部 D_i 、 T_i ; 再经 $2.5t_p$ 形成大组内的四个进位 C_{15} 、 C_{11} 、 C_7 、 C_3 ; 再经过 $2.5t_p$ 形成第五、六、七小组的其余进位 C_{14} 、 C_{13} 、 C_{12} 、 C_{10} 、 C_9 、 C_8 、 C_6 、 C_5 、 C_4 , 可见, 按双重分组设计 $n=16$ 的进位链, 最长进位时间为 $7.5t_p$, 比单重分组进位链又省了 $2.5t_p$ 。

对应图 6.23 所示 32 位加法器的双重分组进位链, 不难理解从 d_i 、 t_i 、 C_1 形成后算起, 经 $2.5t_p$ 产生 C_2 、 C_1 、 C_0 及 $D_1 \sim D_8$ 、 $T_1 \sim T_8$; 再经 $2.5t_p$ 后产生 C_{15} 、 C_{11} 、 C_7 、 C_3 ; 再经 $2.5t_p$ 后产生 $C_{18} \sim C_{16}$ 、 $C_{14} \sim C_{12}$ 、 $C_{10} \sim C_8$ 、 $C_6 \sim C_4$ 及 C_{31} 、 C_{27} 、 C_{23} 、 C_{19} ; 最后再经 $2.5t_p$ 产生 $C_{30} \sim C_{28}$ 、 $C_{26} \sim C_{24}$ 、 $C_{22} \sim C_{20}$, 可见产生全部进位的最长时间为 $10t_p$ 。若采用单重分组进位链, 仍以四位一组分组, 则产生全部进位时间为 $20t_p$, 比双重分组多一倍。显然, 随着 n 的增大, 双重分组的优越性显得格外突出。

机器究竟采用哪种方案, 每个小组内应包含几位, 应根据运算速度指标及所选元件等诸方面因素综合考虑。

由上述分析可知, D_i 和 T_i 均是由小组进位链产生的, 它们与低位进位无关。而 D_i 和 T_i 又是大组进位链的输入, 因此, 引入 D_i 和 T_i 可采用双重分组进位链, 大大提高了运算速度。

6.5.1 介绍的 74181 芯片是四位 ALU 电路, 其四位进位是同时产生的, 多片 74181 级联就犹如本节介绍的单重分组跳跃进位, 即组内 (74181 片内) 并行, 组间 (74181 片间) 串行。74181 芯片的 G 、 P 输出就如本节介绍的 D 、 T 。当需要进一步提高进位速度时, 将 74181 与 74182 芯片配合, 就可组成双重分组跳跃进位链, 如图 6.27 所示。

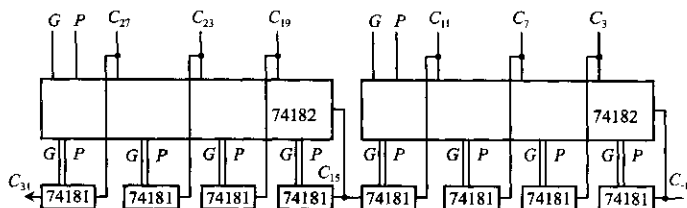


图 6.27 由 74181 和 74182 组成双重分组跳跃进位链

图中 74182 为先行进位部件, 两片 74182 和 8 片 74181 组成 32 位 ALU 电路, 该电路采用双重分组先行进位方案, 其原理与图 6.23 类似, 其不同点是 74182 还提供了大组的本地进位 G 和大组的传送条件 P 。

思考题与习题

1. 最少用几位二进制数即可表示任一五位长的十进制正整数?
 2. 已知 $X=0.a_1a_2a_3a_4a_5a_6$ (a_i 为 0 或 1), 讨论下列几种情况时 a_i 各取何值。

$$(1) X > \frac{1}{2}$$

$$(2) X \geq \frac{1}{8}$$

$$(3) \frac{1}{4} \geq X > \frac{1}{16}$$

3. 设 x 为整数, $[x]_H = 1x_1x_2x_3x_4x_5$, 若要求 $x < -16$, 试问 $x_1 \sim x_5$ 应取何值?
 4. 设机器数字长为 8 位 (含 1 位符号位在内), 写出对应下列各真值的原码、补码和反码。

$$-\frac{13}{64}, \quad \frac{29}{128}, \quad 100, \quad -87$$

5. 已知 $[x]_H$, 求 $[x]_B$ 和 x 。

$$[x]_H = 1.1100; [x]_H = 1.1001; [x]_H = 0.1110; [x]_H = 1.0000;$$

$$[x]_H = 1.0101; [x]_H = 1.1100; [x]_H = 0.0111; [x]_H = 1.0000;$$

6. 设机器数字长为 8 位 (含 1 位符号位在内), 分整数和小数两种情况讨论真值 x 为何值时, $[x]_H = [x]_B$ 成立。
 7. 设 x 为真值, x^* 为绝对值, 说明 $[-x^*]_H = [-x]_B$ 能否成立。
 8. 讨论若 $[x]_H > [y]_H$, 是否有 $x > y$?
 9. 当十六进制数 9B 和 FF 分别表示为原码、补码、反码、移码和无符号数时, 所对应的十进制数各为多少 (设机器数采用一位符号位)。
 10. 在整数定点机中, 设机器数采用一位符号位, 写出 ± 0 的原码、补码、反码和移码, 得出什么结论?
 11. 已知机器数字长为 4 位 (其中 1 位为符号位), 写出整数定点机和小数定点机中原码、补码和反码的全部形式, 并注明其对应的十进制真值。
 12. 设浮点数格式为: 阶符 1 位、阶码 4 位、数符 1 位、尾数 10 位。写出 $\frac{51}{128}$ 、 $-\frac{27}{1024}$ 、

7.375、-86.5 所对应的机器数。要求

- (1) 阶码和尾数均为原码;
 (2) 阶码和尾数均为补码;
 (3) 阶码为移码, 尾数为补码。

13. 浮点数格式同上题, 当阶码基值分别取 2 和 16 时,
- (1) 说明 2 和 16 在浮点数中如何表示。
 - (2) 基值不同对浮点数有什么影响?
 - (3) 当阶码和尾数均用补码表示, 且尾数采用规格化形式, 给出两种情况下所能表示的最大正数和非零最小正数真值。
14. 设浮点数字长为 32 位, 欲表示 ± 6 万间的十进制数, 在保证数的最大精度条件下, 除阶符、数符各取 1 位外, 阶码和尾数各取几位? 按这样分配, 该浮点数溢出的条件是什么?
15. 什么是机器零? 若要求全 0 表示机器零, 浮点数的阶码和尾数应采用什么机器数形式?
16. 设机器数数字长为 16 位, 写出下列各种情况下它能表示的数的范围。设机器数采用一位符号位, 答案均用十进制表示。
- (1) 无符号数;
 - (2) 原码表示的定点小数;
 - (3) 补码表示的定点小数;
 - (4) 补码表示的定点整数;
 - (5) 原码表示的定点整数;
 - (6) 浮点数的格式为: 阶符 1 位、阶码 5 位、数符 1 位、尾数 9 位 (共 16 位)。分别写出其正数和负数的表示范围;
 - (7) 浮点数格式同(6), 机器数采用补码规格化形式, 分别写出其对应的正数和负数的真值范围。
17. 设机器数数字长为 8 位 (包括一位符号位), 对下列各机器数进行算术左移一位、两位, 算术右移一位、两位, 讨论结果是否正确。
- $[x]_{\text{原}}=0.0011010$; $[x]_{\text{补}}=0.1010100$; $[x]_{\text{原}}=1.0101111$;
 $[x]_{\text{原}}=1.1101000$; $[x]_{\text{补}}=1.1101000$; $[x]_{\text{原}}=1.1101000$;
 $[x]_{\text{原}}=1.0011001$; $[x]_{\text{补}}=1.0011001$; $[x]_{\text{原}}=1.0011001$ 。
18. 试比较逻辑移位和算术移位。
19. 设机器数数字长为 8 位 (含 1 位符号位), 用补码运算规则计算下列各题。
- (1) $A=\frac{9}{64}$, $B=-\frac{13}{32}$, 求 $A+B$;
 - (2) $A=\frac{19}{32}$, $B=-\frac{17}{128}$, 求 $A-B$;
 - (3) $A=-\frac{3}{16}$, $B=\frac{9}{32}$, 求 $A+B$;
 - (4) $A=-87$, $B=53$, 求 $A-B$;
 - (5) $A=115$, $B=-24$, 求 $A+B$ 。

20. 用原码一位乘、两位乘和补码一位乘 (Booth 算法)、两位乘计算 $x \cdot y$ 。

(1) $x=0.110111, y=-0.101110$;

(2) $x=-0.010111, y=-0.010101$;

(3) $x=19, y=35$;

(4) $x=0.11011, y=-0.11101$ 。

21. 用原码加减交替法和补码加减交替法计算 $x \div y$ 。

(1) $x=0.100111, y=0.101011$;

(2) $x=-0.10101, y=0.11011$;

(3) $x=0.10100, y=-0.10001$;

(4) $x=\frac{13}{32}, y=-\frac{27}{32}$ 。

22. 设机器数字长为 16 位 (含 1 位符号位), 若一次移位需 $1\mu s$, 一次加法需 $1\mu s$, 试问原码一位乘、补码一位乘、原码加减交替除和补码加减交替除法各最多需多少时间?

23. 对于尾数为 40 位的浮点数 (不包括符号位在内), 若采用不同的机器数表示, 试问当尾数左规或右规时, 最多移位次数各为多少?

24. 按机器补码浮点运算步骤, 计算 $[x \pm y]_H$ 。

(1) $x=2^{+11} \times 0.101100, y=2^{-010} \times (-0.011100)$;

(2) $x=2^{+11} \times (-0.100010), y=2^{-010} \times (-0.011111)$;

(3) $x=2^{+10} \times (-0.100101), y=2^{+100} \times (-0.001111)$ 。

25. 假设阶码取 3 位, 尾数取 6 位 (均不包括符号位), 计算下列各题。

(1) $[2^4 \times \frac{11}{16}] + [2^{-4} \times (-\frac{9}{16})]$

(2) $[2^{-3} \times \frac{13}{16}] - [2^{-4} \times (-\frac{5}{8})]$

(3) $[2^3 \times \frac{13}{16}] \times [2^4 \times (-\frac{9}{16})]$

(4) $[2^6 \times (-\frac{11}{16})] \div [2^3 \times (-\frac{15}{16})]$

(5) $[2^3 \times (-1)] \times [2^{-2} \times \frac{57}{64}]$

(6) $[2^{-6} \times (-1)] \div [2^7 \times (-\frac{1}{2})]$

(7) $3.3125 + 6.125$

(8) $14.75 - 2.4375$

26. 如何判断定点和浮点补码加减运算结果是否溢出, 如何判断原码和补码定点除法运算

附录 6A.1 各种进位制的对应关系

表 6.33 是十进制、二进制、十六进制对照表。书写时，可在十六进制数后面加上“H”如 17DBH 或 $(17DB)_{16}$ ，若在数的后面加上“B”，如 10101100B，即表示此数为二进制，或写成 $(10101100)_2$ 。

表 6.33 十进制、二进制、十六进制对照表

十进制	二进制	十六进制	十进制	二进制	十六进制
0	00000	0	16	10000	10
1	00001	1	17	10001	11
2	00010	2	18	10010	12
3	00011	3	19	10011	13
4	00100	4	20	10100	14
5	00101	5	21	10101	15
6	00110	6	22	10110	16
7	00111	7	23	10111	17
8	01000	8	24	11000	18
9	01001	9	25	11001	19
10	01010	A	26	11010	1A
11	01011	B	27	11011	1B
12	01100	C	28	11100	1C
13	01101	D	29	11101	1D
14	01110	E	30	11110	1E
15	01111	F	31	11111	1F

附录 6A.2 各种进位制的转换

任意一个数 N 可用下式表示

$$\begin{aligned} N &= (d_{n-1}d_{n-2}\cdots d_1d_0.d_{-1}d_{-2}\cdots d_{-m})_r \\ &= d_{n-1}r^{n-1} + d_{n-2}r^{n-2} + \cdots + d_1r^1 + d_0r^0 + d_{-1}r^{-1} + \cdots + d_{-m}r^{-m} \\ &= \sum_{i=-m}^{n-1} d_i r^i \end{aligned}$$

其中, r 为基值;

n 、 m 为正整数, 分别代表整数位和小数位的位数;

d_i 为系数, 代表第 i 位的一个数码, 可以是 0 到 $(r-1)$ 数码中的任意一个;

r^i 为第 i 位的权数。

1. 二进制数转换成十进制数

(1) 按“权”展开法

$$\begin{aligned} \text{例 } (11011.1)_2 &= 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} \\ &= 27.5 \end{aligned}$$

(2) 按基值重复相乘(除)法

• 整数部分采用基值重复相乘法

$$\text{例 } (101001)_2 = (((((1 \times 2 + 0) \times 2 + 1) \times 2 + 0) \times 2 + 0) \times 2 + 1) \times 2 + 1) \times 2 + 1 = 41$$

• 小数部分采用基值重复相除

$$\text{例 } (0.1011)_2 = (((((1 \div 2) + 1) \div 2 + 0) \div 2 + 1) \div 2 + 0) \div 2 + 0 = 0.6875$$

2. 十进制数转换成二进制数

(1) 重复相除(乘)法

这种方法的规则是整数部分除以 2 取余数, 直到商为 0 止; 小数部分乘以 2 取整数, 直到小数部分为 0 止(或按精度要求确定位数)。

例 将十进制数 123.6875 转换成二进制数。

解: 整数部分

重复除以 2	得商	取余数	
$123 \div 2$	61	1	最低位
$61 \div 2$	30	1	
$30 \div 2$	15	0	

$15 \div 2$	7	1	
$7 \div 2$	3	1	
$3 \div 2$	1	1	
$1 \div 2$	0	1	最高位

故整数部分 $(123)_{+} = (1111011)_{-}$ 。

小数部分

重复乘以 2	得小数部分	取整数	
0.6875×2	0.3750	1	最高位
0.3750×2	0.7500	0	
0.7500×2	0.5000	1	
0.5000×2	1.0000	1	最低位

故小数部分 $(0.6875)_{+} = (0.1011)_{-}$ 。

所以 $(123.6875)_{+} = (1111011.1011)_{-}$ 。

(2) 减权定位法

当十进制数较大时, 采用减权定位法可减少重复除法(或乘法)的次数。

例 将十进制数 5148 转换成二进制数, 可按下列步骤进行。

十进制数	位权	转换后的结果
5148		$2^{12} \ 2^{11} \ 2^{10} \ 2^9 \ 2^8 \ 2^7 \ 2^6 \ 2^5 \ 2^4 \ 2^3 \ 2^2 \ 2^1 \ 2^0$
-4096	$2^{12} \rightarrow$	1 0 1 0 0 0 0 0 0 1 1 1 0 0
1052		
-1024	2^{10}	
28		
-16	2^4	
12		
-8	2^3	
4		
-4	2^2	
0		

即 $(5148)_{+} = (1010000011100)_{-}$ 。

可见, 只要从 5148 中减去所含的最大的 2 的方幂 4096, 根据 $2^{12} = 4096$, 确定该权值对应的二进制数的位置(即数位), 然后再从减得的 1052 中减去所含的最大的 2 的方幂 1024, 又得该权值对应的数位, 这样依次继续, 直到差为 0。最后凡是在有权值的对应数位上添 1, 无权值的对应数位上添 0, 使得转换结果。

3. 二进制数与八、十六进制数之间的转换

由于 $2^3 = 8$, $2^4 = 16$, 故三位二进制数正好对应一位八进制数, 四位二进制数正好对应一位十六进制数, 因此可按下述方法将二进制数转换成八、十六进制数。

例如:

1111000010.01101

↑
分组起点

高位补 0, 凑足三位;

低位补 0, 凑足三位

↑					↑
001	111	000	010	011	010
1	7	0	2	3	2

故对应的八进制数为 $(1702.32)_8 = (1111000010.01101)_2$ 。

高位补 0, 凑足四位;

低位补 0, 凑足四位

↑				↑
0011	1100	0010	0110	1000
3	C	2	6	8

故对应的十六进制数为 $(3C2.68)_{16} = (1111000010.01101)_2$ 。

反之, 将一位八进制数用三位二进制数表示, 或一位十六进制数用四位二进制数表示, 便可将八进制或十六进制数转换成二进制数。

例如:

$(247.63)_8 = (10100111.110011)_2$

$(F5B.48)_{16} = (111101011011.01001)_2$

附录 6B 阵列乘法器和阵列除法器

图 6.28 是一个完成 $X(X=X_1X_2X_3X_4) \times Y(Y=Y_1Y_2Y_3Y_4)$ 绝对值相乘的阵列乘法器原理图。图中方框的排列形式与笔算乘法的位积排列相似。阵列的每一行由乘数 Y 的每一位数位控制，而各行错开形成的每一斜列则由被乘数 X 的每一位数位控制。图中方框内的电路由一个与门和一个一位全加器组成。由于采用阵列结构，加法器数量很多，不靠“重复加和移位”的步骤运算，因此可大大提高乘法速度。该方案虽然加法器数量多，但内部结构规则，标准化程度高，适于用超大规模集成电路实现。

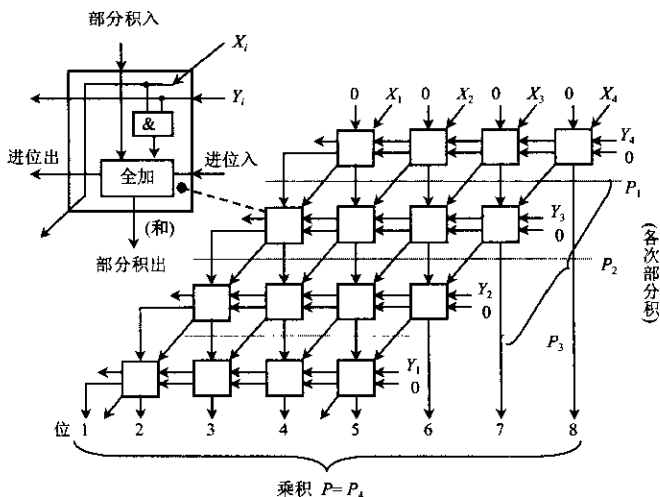
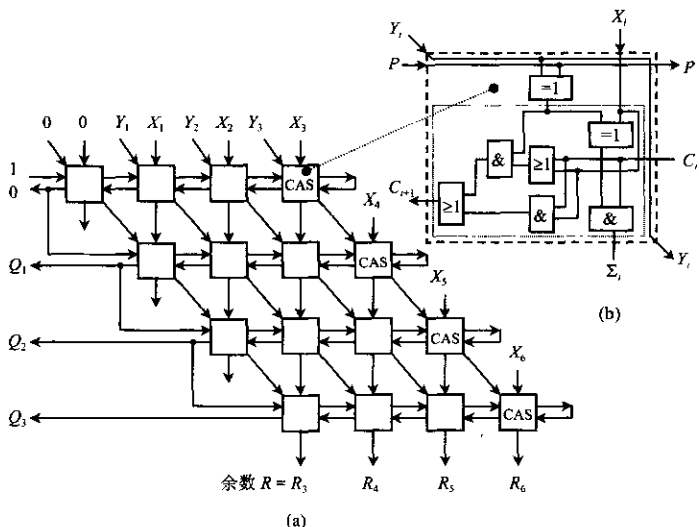


图 6.28 绝对值相乘的阵列乘法器

图 6.29 是一个完成 $X(X=X_1X_2X_3X_4X_5X_6) \div Y(Y=Y_1Y_2Y_3Y_4Y_5Y_6)$ 绝对值相除的不恢复余数除法器原理图。图中的每一方框为一个可控加法/减法 (CAS) 单元，当输入控制 $P=0$ 时，CAS 作加法运算；当 $P=1$ 时，CAS 作减法运算。被除数 $X_1 \sim X_6$ 由顶部一行和最右边对角线 (斜列) 上各 CAS 的垂直输入端提供；除数 $Y_1 \sim Y_3$ 则沿对角线方向进入阵列，其作用是使余数固定 (不左移) 而除数右移，类似笔算除法；商 $Q_1Q_2Q_3$ 由阵列每一行最左边的 CAS 的进位输出 C_{i+1} 产生；余数 $R_3 \sim R_6$ 在阵列的最下行产生。由于绝对值除和 6.3.4 所述的原码除

中数值部分的运算完全相同, 故运算过程中需作 $X+Y$ 和 $X-Y$ 操作, 而减法均用 $[X]_{\text{补}} + [-Y]_{\text{补}}$ 实现, 因此阵列除法器中必有一些 CAS 单元用于对应符号位的运算, 如图 6.29 中每行最左边的小方框 (共四个 CAS)。

以绝对值整数除法为例, 第一步检查是否溢出, 由第一行完成 $X_1X_2X_3Y_1Y_2Y_3$ 操作, 故控制电位 $P=1$ 。减法用 $[X]_{\text{补}} + [-Y]_{\text{补}}$ 实现, 正好用 $P=1$ 作为第一行末位 CAS 的进位输入。由于 $X < Y$, 所以相减后符号位的进位输出为 0, 即商为 0, 表示未溢出, 除法可继续进行。此商接到第二行的 P 端, 决定第二行作加法。同理每个当前商反馈到下一行, 决定下一行是作加法还是减法, 满足“上商 1 作减法, 上商 0 作加法”的运算规则。



被除数 $X = 0.X_1X_2X_3X_4X_5X_6$

除数 $Y = 0.Y_1Y_2Y_3$

商 $Q = 0.Q_1Q_2Q_3$

余数 $R = 0.00R_3R_4R_5R_6$

(a) 4 位不恢复余数阵列除法器

(b) 可控加法/减法(CAS)单元

图 6.29 绝对值相除的阵列除法器

例 设 $x=101001$, $y=111$, 用阵列除法器计算 $x \div y$ 。

解: $[x^*]_{\text{补}}=0,101001$; $[y^*]_{\text{补}}=0,111$; $[-y^*]_{\text{补}}=1,001$

被除数 x	0,101001	商	控制端
减除数 y	+ 1,001		$P=1$
余数为负	1,110001	$Q_0=0$ 未溢出	$P=0$
余数左移	1,10001		
加除数	+ 0,111		
余数为正	0,01101	$Q_1=1$	$P=1$
余数左移	0,1101		
减除数	+ 1,001		
余数为负	1,111	$Q_2=0$	$P=0$
余数左移	1,11		
加除数	+ 0,111		
	0,110	$Q_3=1$	

故商为 $Q_1Q_2Q_3=101$; 余数为 $R_4R_5R_6=110$ 。

附录 6C 74181 逻辑电路

图 6.30 是 74181 ALU 的负逻辑电路，其逻辑关系如表 6.32 所示。

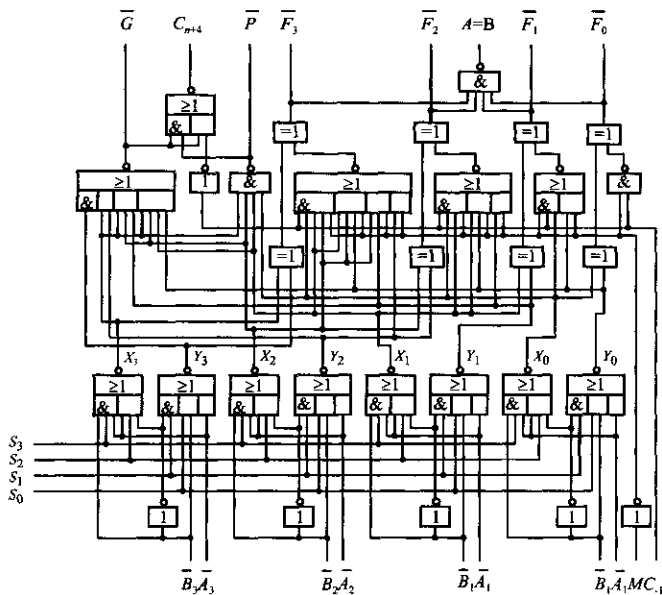


图 6.30 负逻辑操作数表示的 74181 ALU 电路

第七章 指令系统

本章主要介绍机器指令系统的分类、常见的寻址方式、指令格式以及设计指令系统时应考虑的各种因素。此外对 RISC 技术也作了简要的介绍。希望读者进一步体会指令系统与机器主要功能以及与硬件结构之间存在的密切关系。

7.1 机器指令

由第一章所知,计算机能解题是由于机器本身存在一种语言,它既能理解人的意图,又能被机器自身识别。机器语言是由一条条语句构成的,每一条语句又能准确表达某种语义。例如它可命令机器做某种操作,指出参与操作的数或其他信息在什么地方等等。计算机就是连续执行每一条机器语句而实现全自动工作的。人们习惯把每一条机器语言的语句叫机器指令,而又将全部机器指令的集合叫做机器的指令系统。因此机器的指令系统集中反映了机器的功能。

计算机设计者主要研究如何确定机器的指令系统,如何用硬件电路、芯片、设备来实现机器指令系统的功能。计算机的使用者则是依据机器提供的指令系统,使用汇编语言来编制各种程序。计算机使用者根据机器指令系统所描述的机器功能,能很清楚地了解计算机内部寄存器-存储器的结构,以及计算机能直接支持的各种数据类型。

7.1.1 指令的一般格式

指令是由操作码和地址码两部分组成的,其基本格式如图 7.1 所示。

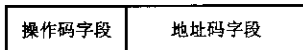


图 7.1 指令的一般格式

1. 操作码

操作码是用来指明该指令所要完成的操作,如加法、减法、传送、移位、转移等等。通常,其位数反映了机器的操作种类,也即机器允许的指令条数,如操作码占 7 位,则该机器最多包含 $2^7=128$ 条指令。

操作码的长度可以是固定的,也可以是变化的。前者将操作码集中放在指令字的一个字段内,如图 7.1 所示。这种格式便于硬件设计,指令译码时间短,广泛用于字长较长的、大中型计算机和超级小型计算机以及 RISC (Reduced

Instruction Set Computer) 中。如 IBM370 和 VAX-11 系列机, 操作码长度均为 8 位。

操作码长度不固定的指令, 其操作码分散在指令字的不同字段中。这种格式可有效地压缩操作码的平均长度, 在字长较短的微机中被广泛采用。如 PDP-11, Intel8086/80386 等, 操作码的长度是可变的。

操作码长度不固定会增加指令译码和分析的难度, 使控制器的设计复杂。通常采用扩展操作码技术, 使操作码的长度随地址数的减少而增加, 不同地址数的指令可以具有不同长度的操作码, 从而在满足需要的前提下, 有效地缩短指令字长。图 7.2 是一种扩展操作码的安排示意。

	OP	A ₁	A ₂	A ₃	
四位操作码	0000	A ₁	A ₂	A ₃	15 条三地址指令
	0001	A ₁	A ₂	A ₃	
	:	:	:	:	
	1110	A ₁	A ₂	A ₃	
八位操作码	1111	0000	A ₂	A ₃	15 条二地址指令
	1111	0001	A ₂	A ₃	
	:	:	:	:	
	1111	1110	A ₂	A ₃	
十二位操作码	1111	1111	0000	A ₃	15 条一地址指令
	1111	1111	0001	A ₃	
	:	:	:	:	
	1111	1111	1110	A ₃	
十六位操作码	1111	1111	1111	0000	16 条零地址指令
	1111	1111	1111	0001	
	:	:	:	:	
	1111	1111	1111	1111	

图 7.2 一种扩展操作码的安排示意

图中指令字长为 16 位, 其中 4 位为基本操作码字段 OP, 另有三个 4 位长的地址字段为 A₁、A₂、A₃。4 位基本操作码若全部用于三地址指令, 则有 16 条。图中所示是三地址指令 15 条, 二地址指令 15 条, 一地址指令 15 条, 零

地址指令 16 条，共 61 条。

除了这种安排以外，还有其他多种扩展方法，如形成 15 条三地址指令，12 条二地址指令，31 条一地址指令和 16 条零地址指令，共 74 条指令，读者可自行安排。

在设计操作码不固定的指令系统时，应尽量考虑安排指令使用频度（即指令在程序中出现的概率）高的指令占用短的操作码，对使用频度低的指令可占用较长的操作码，这样可以缩短经常使用的指令的译码时间。当然，考虑操作码长度时也应考虑地址码的要求。

2. 地址码

地址码用来指出该指令的源操作数的地址（一个或两个）、结果的地址以及下一条指令的地址。这里的“地址”可以是主存的地址，也可以是寄存器的地址，甚至可以是 I/O 设备的地址。

下面以主存地址为例，分析指令的地址码字段。

（1）四地址指令

这种指令的地址字段有四个，其格式为：

OP	A ₁	A ₂	A ₃	A ₄
----	----------------	----------------	----------------	----------------

其中，OP 为操作码；

A₁ 为第一操作数地址；

A₂ 为第二操作数地址；

A₃ 为结果地址；

A₄ 为下一条指令的地址。

该指令完成 (A₁) OP (A₂) → A₃ 的操作。这种指令直观易懂，后续指令地址可以任意填写，可直接寻址的地址范围与地址字段的位数有关。如果指令字长为 32 位，操作码占 8 位，4 个地址字段各占 6 位，则指令的直接寻址范围为 $2^6 = 64$ 。如果地址字段均指示主存的地址，则完成一条四地址指令，共需访问四次存储器（取指令一次，取两个操作数两次，存放结果一次）。

因为程序中大多数指令是按顺序执行的，而程序计数器 PC 既能存放当前欲执行指令的地址，又有计数功能，因此它能自动形成下一条指令的地址。这样，指令字中的第四地址字段 A₄ 便可省去，即得三地址指令格式。

（2）三地址指令

三地址指令中只有三个地址，其格式为：

OP	A ₁	A ₂	A ₃
----	----------------	----------------	----------------

它可完成 (A₁) OP (A₂) → A₃ 的操作，后续指令的地址隐含在程序计数器

PC 之中。如果指令字长不变, 设 OP 仍为 8 位, 则三个地址字段各占 8 位, 故三地址指令直接寻址范围可达 $2^8 = 256$ 。同理, 若地址字段均为主存地址, 则完成一条三地址指令也需访问四次存储器。

其实, 机器在运行过程中, 没有必要将每次运算结果都存入主存, 中间结果可以暂时存放在 CPU 的寄存器 (如 ACC) 中, 这样又可省去一个地址字段 A_3 , 从而得出二地址指令。

(3) 二地址指令

二地址指令中只含两个地址字段, 其格式为:

OP	A_1	A_2
----	-------	-------

它可完成 $(A_1) OP (A_2) \rightarrow A_1$ 的操作, 即 A_1 字段既代表源操作数的地址, 又代表存放本次运算结果的地址。有的机器也可以表示 $(A_1) OP (A_2) \rightarrow A_2$ 的操作, 此时 A_2 除了代表操作数源地址外, 还代表中间结果的存放地址。这两种情况完成一条指令仍需访问四次存储器。如果使其完成 $(A_1) OP (A_2) \rightarrow ACC$, 此时, 它完成一条指令只需三次访存, 它的含义是中间结果暂存于累加器 ACC 中。在不改变指令字长和操作码的位数前提下, 二地址指令可直接寻访的主存地址数为 $2^{12} = 4K$ 。

如果将一个操作数的地址隐含在运算器的 ACC 中, 则指令字中只需给出一个地址码, 构成了一地址指令。

(4) 一地址指令

一地址指令的地址码字段只有一个, 其格式为:

OP	A_1
----	-------

它可完成 $(ACC) OP (A_1) \rightarrow ACC$ 的操作, ACC 既存放参与运算的操作数, 又存放运算的中间结果, 这样, 完成一条一地址指令只需两次访存。在指令字长仍为 32 位、操作码位数仍固定位 8 位时, 一地址指令可直接寻址的范围达 2^{24} , 即 16M。

在指令系统中, 还有一种指令可以不设地址字段, 即所谓零地址指令。

(5) 零地址指令

零地址指令在指令字中无地址码, 例如进栈 (PUSH)、出栈 (POP) 这类指令, 其操作数的地址隐含在堆栈指针 SP 中 (有关这类指令的内容详见 7.4)。

通过上述介绍可见, 用一些硬件资源如 PC、ACC 承担指令字中需指明的地址码, 可在不改变指令字长的前提下, 扩大指令直接寻址的范围。此外, 用 PC、ACC 等硬件代替指令字中的某些地址字段, 还可缩短指令字长, 并可减少访存次数。因此, 究竟采用什么样的地址格式, 必须从机器性能出发综合考

虑。

以上讨论的地址格式均以主存地址为例，实际上地址字段也可用来表示寄存器。当 CPU 中含有多个通用寄存器时，对每一个寄存器赋以一个编号，便可指明源操作数和结果存放在哪个寄存器中。地址字段表示寄存器时，也可有三地址、二地址、一地址之分。它们的共同点是，在指令的执行阶段都不必访问存储器，直接访问寄存器，使机器运行速度得到提高（因为寄存器类型的指令只需在取指阶段访问一次存储器）。

7.1.2 指令字长

指令字长取决于操作码的长度、操作数地址的长度和操作数地址的个数。不同机器的指令字长是不相同的。

早期的计算机指令字长、机器字长和存储字长均相等，因此访问某个存储单元，便可取出一条完整的指令或一个完整的数据。这种机器的指令字长是固定的，控制方式比较简单。

随着计算机的发展，存储容量的增大，要求处理的数据类型增多，计算机的指令字长也发生了很大的变化。一台机器的指令系统可以采用位数不相同的指令，即指令字长是可变的，如单字长指令、多字长指令。控制这类指令的电路比较复杂，而且多字长指令要多次访问存储器才能取出一条完整的指令，因此使 CPU 速度下降。为了提高指令的运行速度和节省存储空间，通常尽可能把常用的指令（如数据传送指令、算术运算指令等）设计成单字长或短字长格式的指令。

例如，PDP-8 指令字长固定取 7 位；NOVA 指令字长固定取 16 位；IBM370 指令字长可变，可以是 16 位（半个字）、32 位（一个字）、48 位（一字半）；Intel8086 的指令字长可以为 8、16、24、32、40 和 48 位六种。通常指令字长取 8 的整数倍。

7.2 操作数类型和操作类型

7.2.1 操作数类型

机器中常见的操作数类型有：地址、数字、字符、逻辑数据等。

（1）地址

地址实际上也可看作是一种数据，在许多情况下要计算操作数的地址。这时，地址可被认为是一个无符号的整数，有关地址的计算问题将在 7.4 节讨论。

（2）数字

计算机中常见的数字有：定点数、浮点数和十进制数。前两种数字在第六章中已作了介绍，十进制数在第五章附录中作了说明，读者可自行复习。

(3) 字符

在应用计算机时,文本或者字符串也是一种常见的数据类型。由于计算机在处理信息过程中不能以简单的字符形式存储和传送,因此普遍采用 ASCII 码(见表 5.2),它是很重要的一种字符编码。当然还有其他一些字符编码,如 8 位 EBCDIC 码(Extended Binary Coded Decimal Interchange Code),又称扩展 BCD 交换码,在此不作详述。

(4) 逻辑数据

计算机除了作算术运算外,有时还需作逻辑运算,此时 n 个 0 和 1 的组合不是被看作算术数字,而是看作逻辑数。例如,在 ASCII 码中的 0110101,它表示十进制数 5,若要将它转换为 NBCD 短十进制码,只需通过它与逻辑数 0001111 完成逻辑与运算,抽取低四位,即可获得 0101。此外,有时希望存储一个布尔类型的数据,它们的每一位都代表着真(1)和假(0),这时 n 个 0 和 1 组合的数就都被看作逻辑数。

例如,奔腾处理器的数据类型有逻辑数、有符号数(补码)、无符号数、压缩和未压缩的 BCD 码、地址指针、位串、字符串以及浮点数(符合 IEEE754 标准)等等。

7.2.2 数据在存储器中的存放方式

通常计算机中的数据存放在存储器或寄存器中,而寄存器的位数便可反映机器字长。一般机器字长可取字节的 1、2、4、8 倍,这样便于字符处理。在大、中型机器中字长为 32 位和 64 位,在微型机中字长从 4 位、8 位逐渐发展到目前的 16 位和 32 位。

由于不同的机器数据字长不同,每台机器处理的数据字长也不统一,例如奔腾处理器可处理 8(字节)、16(字)、32(双字)、64(四字);PowerPC 可处理 8(字节)、16(半字)、32(字)、64(双字)。因此,为了便于硬件实现,通常要求多字节的数据在存储器的存放方式能满足“边界对准”的要求,如图 7.3 所示。

图中所示的存储器其存储字长为 32 位,可按字节、半字、字、双字访问。在对准边界的 32 位字长的计算机中(如图 7.3(a)所示),半字地址是 2 的整数倍,字地址是 4 的整数倍,双字地址是 8 的整数倍。当所存数据不能满足此要求时,可填充一个至多个空白字节。而字节的次序有两种,如图 7.4 所示,其中(a)表示低字节为低地址,(b)表示高字节为低地址。

在数据不对准边界的计算机中,数据(例如一个字)可能在两个存储单元中,此时需要访问两次存储器,并对高低字节的位置进行调整后,才能取得一个字,图 7.3(b)的阴影部分即属于这种情况。

存储器				地址(十进制)
字(地址 0)				0
字(地址 4)				4
字节(地址 11)	字节(地址 10)	字节(地址 9)	字节(地址 8)	8
字节(地址 15)	字节(地址 14)	字节(地址 13)	字节(地址 12)	12
半字(地址 18)		半字(地址 16)		16
半字(地址 22)		半字(地址 20)		20
双字(地址 24)				24
双字				28
双字(地址 32)				32
双字				36

(a)

存储器		地址(十进制)
字(地址 2)	半字(地址 0)	0
字节(地址 7)	字节(地址 6)	4
半字(地址 10)	半字(地址 8)	8

(b)

图 7.3 存储器中数据的存放

字地址					字节地址				
0	3	2	1	0	0	0	1	2	3
4	7	6	5	4	4	4	5	6	7

(a)

(b)

图 7.4 两种字节次序

7.2.3 操作类型

不同的机器操作类型也是不同的,但几乎所有的机器都有以下几类通用的操作。

1. 数据传送

数据传送包括寄存器与寄存器、寄存器与存储单元、存储单元与存储单元之间的传送。如从源到目的之间的传送、对存储器读 (LOAD) 和写 (STORE)、交换源和目的的内容、置 1、清 0、进栈、出栈等。

2. 算术逻辑操作

这类操作可实现算术运算（加、减、乘、除、增 1、减 1、取负数即求补）和逻辑运算（与、或、非、异或）。对于低档机而言，一般算术运算只支持最基本的二进制加减、比较、求补等，高档机还能支持浮点运算和十进制运算。

有些机器还具有位操作功能，如位测试（测试指定位的值）、位清除（清除指定位）、位求反（对指定位求反）等。

3. 移位

移位可分为算术移位、逻辑移位和循环移位三种。算术移位和逻辑移位分别可实现对有符号数和无符号数乘以 2^n （左移）或整除以 2^n （右移）的运算。并且移位操作所需时间远比乘除操作执行时间短，因此，移位操作经常被用来代替简单的乘法和除法运算。

4. 转移

在多数情况下，计算机是按顺序执行程序每条指令的，但有时需要改变这种顺序，此刻可采用转移类指令来完成。转移指令按其转移特征又可分为无条件转移、条件转移、跳转、过程调用与返回、陷阱（Trap）等几种。

（1）无条件转移

无条件转移不受任何条件约束，可直接把程序转移到下一条需执行指令的地址。如 `JMP X` 其功能是将指令地址无条件转至 X。

（2）条件转移

条件转移是根据当前指令的执行结果，来决定是否需转移。若条件满足，则转移；若条件不满足，则继续按顺序执行。一般机器都能提供一些条件码，这些条件码是某些操作的结果。如零标志位（Z），结果为 0， $Z=1$ ；负标志位（N），结果为负， $N=1$ ；溢出标志位（V），结果有溢出， $V=1$ ；进位标志位（C），最高位有进位， $C=1$ ；奇偶标志位（P），结果呈偶数， $P=1$ 等等。

例如，指令 `BRO X`，表示若结果（有符号数）溢出（ $V=1$ ），则指令跳转至 X。如，指令 `BRC Y`，表示若最高位有进位（ $C=1$ ），则指令跳转至 Y。

还有一种条件转移指令，`SKP (Skip)`，它暗示其下一条指令将被跳过，从而隐含了转移地址是 `SKP` 后的第二条指令。如：

```
200
  :
205 SKP DZ
206
207
```

这里 `SKP DZ` 表示若设备的完成触发器 D 为零，则执行完 205 条指令后，立即跳至第 207 条指令，再顺序执行。

(3) 调用与返回

在编写程序时,有些具有特定功能的程序段会被反复使用。为避免重复编写,可将这些程序段设定为独立子程序,当需要执行某子程序时,只需用子程序调用指令即可。此外,计算机系统还提供了通用子程序,如申请资源、读写文件、控制外设等等。需要时均可由用户直接调用,不必重新编写。

通常调用指令包括过程调用、系统调用和子程序调用。它可实现从一个程序转移到另一个程序的操作。

调用指令(CALL)一般与返回指令(RETURN)配合使用。CALL用于从当前的程序位置转至子程序的入口;RETURN用于子程序执行完后重新返回到原程序的断点。图7.5示意了调用(CALL)和返回(RETURN)指令在程序执行中的流程。

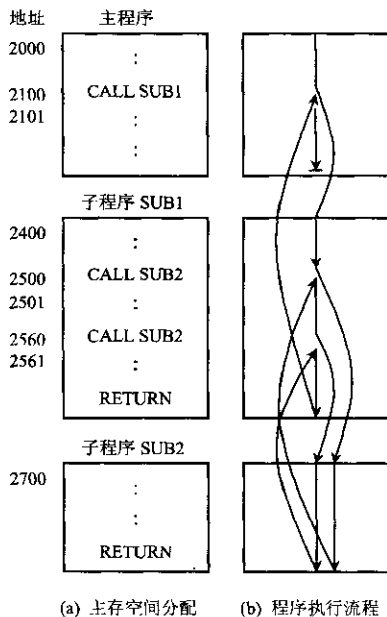


图 7.5 调用和返回指令示意

图 7.5 (a) 示意了主程序和子程序在主存所占空间。主程序从 2000 地址

单元开始，并在 2100 处有一调用指令，当执行到 2100 处指令时，CPU 停止下一条顺序号为 2101 的指令，而转至 2400 执行 SUB1 子程序。在 SUB1 中又有两次（2500 和 2560 处）调用子程序 SUB2。每一次都将 SUB1 挂起，而执行 SUB2。子程序末尾的 RETURN 的指令可使 CPU 返回调用点。

图 7.5 (b) 示意了主程序→SUB1→SUB2→SUB1→SUB2→ SUB1→主程序的执行流程。

需注意几点：

- 子程序可在多处被调用；
- 子程序调用可出现在子程序中，即允许子程序嵌套；
- 每个 CALL 指令都对应该一条 RETURN 指令。

由于可以在许多处调用子程序，因此，CPU 必须记住返回地址，使子程序能准确返回。返回地址可存放在三处：

- 寄存器内。机器内设有专用寄存器，专门用于存放返回地址；
- 子程序的入口地址内；
- 栈顶内。现代计算机都设有堆栈，执行 RETURN 指令后，便可自动从栈顶内取出应返回的地址。

(4) 陷阱 (Trap) 与陷阱指令

陷阱其实是一种意外事故的中断。如机器在运行中，可能会出现电源电压不稳定、存储器校验出差错、输入输出设备出现了故障、用户使用未被定义的指令、除数出现为 0、运算结果溢出以及特权指令等种种意外事件，致使计算机不能正常工作。此刻必须及时采取措施，否则将影响整个系统正常运行。因此，一旦出现意外故障，计算机就发出陷阱信号，暂停当前程序的执行，转入故障处理程序进行相应的故障处理。

计算机的陷阱指令一般不提供给用户直接使用，而作为隐指令（即指令系统中不提供的指令），在出现意外故障时，由 CPU 自动产生并执行。也有的机器设置供用户使用的陷阱指令或“访管”指令，利用它完成系统调用和程序请求。如 IBM PC (Intel8086) 的软中断 INT TYPE (TYPE 是 8 位常数，表示中断类型)，其实就是直接提供给用户使用的陷阱指令，用来完成系统调用。

5. 输入输出

对于 I/O 单独编址的计算机而言，通常设有输入输出指令，它完成从外设中的寄存器读入一个数据到 CPU 的寄存器内，或将数据从 CPU 的寄存器输出至某外设的寄存器中。

6. 其他

其他包括等待指令、停机指令、空操作指令、开中断指令、关中断指令、置条件码指令等等。

为了适应计算机的信息管理、数据处理及办公自动化等领域的应用,有的计算机还设有非数值处理指令。如字符串传送、字符串比较、字符串查询及字符串转换等。

在多用户、多任务的计算机系统中,还设有特权指令,这类指令只能用于操作系统或其他系统软件,用户是不能使用的。

在有些大型或巨型机中,还设有向量指令,可对整个向量或矩阵进行求和、求积运算。在多处理器系统中还配有专门的多处理机指令。

7.3 寻址方式

寻址方式是指确定本条指令的数据地址,以及下一条将要执行的指令地址的方法,它与硬件结构紧密相关,而且也直接影响指令格式和指令功能。

寻址方式分为指令寻址和数据寻址两大类。

7.3.1 指令寻址

指令寻址比较简单,它分为顺序寻址和跳跃寻址两种。

顺序寻址可通过程序计数器 PC 加 1,自动形成下一条指令的地址;跳跃寻址则通过转移类指令实现。图 7.6 示意了指令寻址过程。

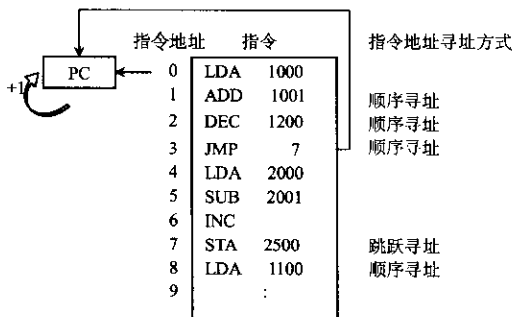


图 7.6 指令的寻址方式示意

如果程序的首地址为 0,只要先将 0 送至程序计数器 PC 中,启动机器运行后,程序便按 0、1、2、3、7、8、9……顺序执行。其中第 1、2、3 号指令地址均由 PC 自动形成。因第 3 号地址指令为 JMP 7,故执行完第 3 号指令后,便无条件将 7 送至 PC,因此,此刻指令地址跳过 4、5、6 三条,直接执行第 7 条指令,接着又顺序执行第 8 条、第 9 条等等指令。

关于跳跃寻址的转移地址形成方式，将在 7.3.2 的直接寻址和相对寻址中作介绍。

7.3.2 数据寻址

数据寻址方式种类较多，在指令字中必须设一字段来指明属哪一种寻址方式。指令的地址码字段，通常都不代表操作数的真实地址，把它称作形式地址，记作 A。操作数的真实地址叫做有效地址，记作 EA，它是由寻址方式和形式地址共同来确定的。由此可得指令的格式应如图 7.7 所示。

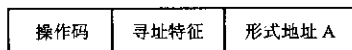


图 7.7 一种一地址指令的格式

为了便于分析研究各类寻址方式，假设指令字长、存储字长、机器字长均相同。

1. 立即寻址

立即寻址的特点是操作数本身设在指令字内，即形式地址 A 不是操作数的地址，而是操作数本身，又称之为立即数。数据是采用补码形式存放的，如图 7.8 所示，图中#表示立即寻址特征标记。

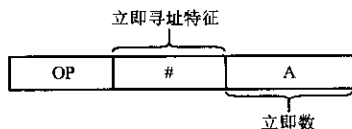


图 7.8 立即寻址示意

可见，它的优点在于只要取出指令，便可立即获得操作数，这样指令在执行阶段不必再访问存储器。显然 A 的位数限制了这类指令所能表述的立即数的范围。

2. 直接寻址

直接寻址的特点是，指令字中的形式地址 A 就是操作数的真实地址 EA，即

$$EA = A$$

图 7.9 示意了直接寻址。

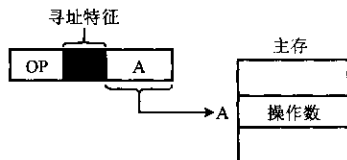


图 7.9 直接寻址示意

它的优点是寻找操作数比较简单，也不需要专门计算操作数的地址，在指令执行阶段对主存只访问一次。它的缺点在于 A 的位数限制了指令的寻址范围，而且必须修改 A 的值，才能修改操作数的地址。

3. 隐含寻址

隐含寻址是指指令字中不明显地给出操作数的地址，其操作数的地址隐含在操作码或某个寄存器中。例如，一地址格式的加法指令只给出一个操作数的地址，另一个操作数隐含在累加器 ACC 中，这样累加器 ACC 成了另一个数的地址。图 7.10 示意了隐含寻址。

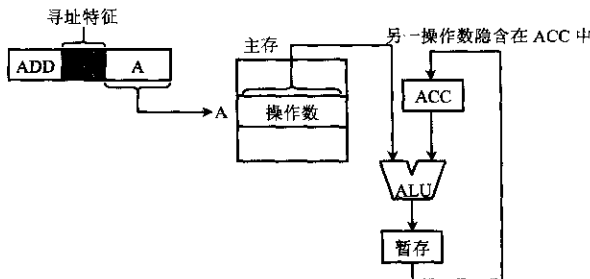


图 7.10 隐含寻址示意

又如 IBM PC (Intel 8086) 中的乘法指令，被乘数隐含在寄存器 AX (16 位) 或寄存器 AL (8 位) 中，可见 AX (或 AL) 就是被乘数的地址。又如字符串传送指令 MOVS，其源操作数的地址隐含在 SI 寄存器中 (即操作数在 SI 指明的存储单元中)，目的操作数的地址隐含在 DI 寄存器中。

由于隐含寻址在指令字中少了一个地址，因此，这种寻址方式的指令有利于缩短指令字长。

4. 间接寻址

倘若指令字中的形式地址不直接指出操作数的地址，而是指出操作数有效地址所在的存储单元地址，也就是说，有效地址是由形式地址间接提供的，故为间接寻址。即

$$EA = (A)$$

如图 7.11 所示。

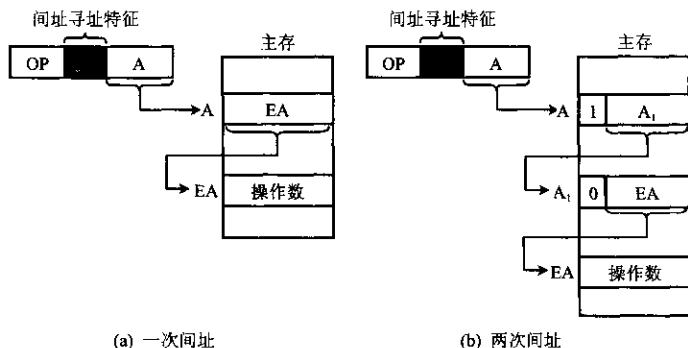


图 7.11 间接寻址示意

图中 (a) 为一次间址，即 A 地址单元的内容 EA 是操作数的有效地址；(b) 为两次间址，即 A 地址单元的内容 A_1 还不是有效地址，而由 A_1 所指单元的内容 EA 才是有效地址。

这种寻址方式与直接寻址相比，它扩大了操作数的寻址范围，因为 A 的位数通常小于指令字长，而存储字长可与指令字长相等。若设指令字长和存储字长均为 16 位，A 为 8 位，显然直接寻址范围为 2^8 ，一次间址的寻址范围可达 2^{16} 。当多次间址时，可用存储字的首位来标志间接寻址是否结束。如图 7.11 (b) 中，当存储字首位为“1”时，标明还需继续访存寻址；当存储字首位为“0”时，标明该存储字即为 EA。由此可见，存储字首位不能作为 EA 的组成部分，因此，它的寻址范围为 2^{15} 。

间接寻址的第二个优点在于它便于编制程序。例如，用间接寻址可以很方便地完成子程序返回，图 7.12 示意了用于子程序返回的间址过程。

图中表示两次调用子程序，只要在调用前先将返回地址存入子程序最末条指令的形式地址 A 的存储单元内，便可准确返回到原程序断点。如第一次调用前，使 $[A] = 81$ ，第二次调用前，使 $[A] = 202$ 。这样，当第一次子程序执行到最末条指令 $JMP @A$ ($@$ 为间址特征位)，便可无条件转至 81 号单元。同理，第二次执行完子程序后，便可返回到 202 号单元。

间接寻址的缺点在于指令的执行阶段需要访存两次(一次间址)或多次(多次间址),致使指令执行时间延长。

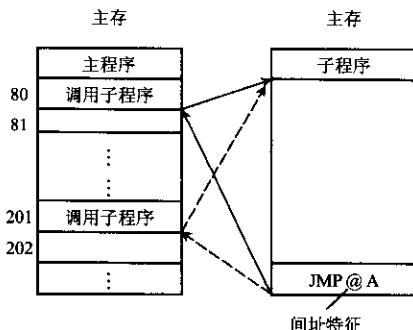


图 7.12 用于子程序返回的间接寻址

5. 寄存器寻址

在寄存器寻址的指令字中,地址码字段直接指出了寄存器的编号,即 $EA=R_i$,如图 7.13 所示。其操作数在由 R_i 所指的寄存器内。由于操作数不在主存中,故寄存器寻址在指令执行阶段无须访存,减少了执行时间。由于地址字段只需指明寄存器编号(计算机中寄存器数有限),故指令字较短,节省了存储空间,因此寄存器寻址在计算机中得到广泛应用。

6. 寄存器间接寻址

图 7.14 示意了寄存器间接寻址过程。

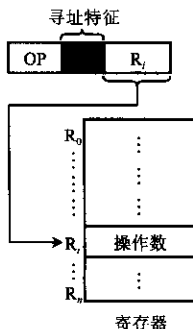


图 7.13 寄存器寻址

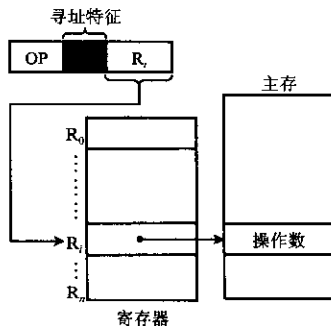


图 7.14 寄存器间接寻址

图中 R_i 中的内容不是操作数, 而是操作数所在主存单元的地址号, 即有效地址 $EA = (R_i)$ 。与寄存器寻址相比, 指令的执行阶段还需访问主存。与图 7.11 (a) 相比, 因有效地址不是存放在存储单元中, 而是存放在寄存器中, 故称其为寄存器间接寻址, 它比间接寻址少一次访存。

7. 基址寻址

基址寻址需设有基址寄存器 **BR**, 其操作数的有效地址 EA 等于指令字中的形式地址与基址寄存器中的内容 (称作基地址) 相加。即

$$EA = A + (BR)$$

图 7.15 示意了基址寻址过程。

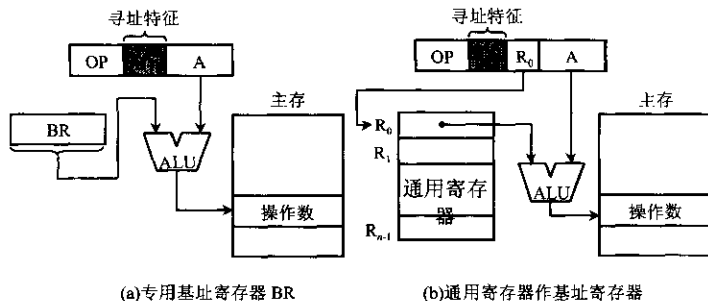


图 7.15 基址寻址示意

基址寄存器可采用隐式的和显式的两种。所谓隐式是在计算机内专门设有一个基址寄存器 **BR**, 使用时用户不必明显指出该基址寄存器, 只需由指令的寻址特征位反映出基址寻址即可。显式是在一组通用寄存器里, 由用户明确指出哪个寄存器用作基址寄存器, 存放基地址。如 IBM 370 计算机中设有 16 个通用寄存器, 用户可任意选中某个寄存器作为基址寄存器。对应图 7.15 (a) 为隐式基址寻址, (b) 为显式基址寻址。

基址寻址可以扩大指令对主存的寻址范围, 因基址寄存器的位数可以大于形式地址 A 的位数。当内存容量较大时, 若采用直接寻址, 因受 A 的位数限制, 无法对内存所有单元进行访问, 但采用基址寻址便可实现全空间寻访。例如, 将内存空间分为若干段, 每段首地址存于基址寄存器中, 段内的位移量由指令字中形式地址 A 指出, 这样操作数的有效地址就等于基址寄存器内容与段内位移量之和, 只要对基址寄存器的内容作修改, 便可访问主存的任一单元。

基址寻址在多道程序和浮动程序编制时极为有用。用户可不必考虑自己的程序存于主存的哪一空间区域, 完全可由操作系统或管理程序根据主存的使用

状况, 赋予基址寄存器内一个初始值 (即基地址), 便可将用户程序的逻辑地址转化为主存的物理地址 (实际地址), 使用户程序安置于主存的某一空间区域。例如, 对于一个有多个寄存器的机器来说, 用户只需指出哪一个寄存器作为基址寄存器即可, 至于这个基址寄存器应赋予何值, 完全由操作系统或管理程序根据内存空间状况来确定。在程序执行过程中, 用户不知道自己的程序在主存的哪个空间, 用户也不可修改基址寄存器的内容, 以确保系统安全可靠地运行。

8. 变址寻址

变址寻址与基址寻址极为相似。其有效地址 EA 等于指令字中的形式地址 A 与变址寄存器 IX 的内容相加之和。即

$$EA = A + (IX)$$

显然只要变址寄存器位数足够, 也可扩大指令的寻址范围, 其寻址过程如图 7.16 所示。

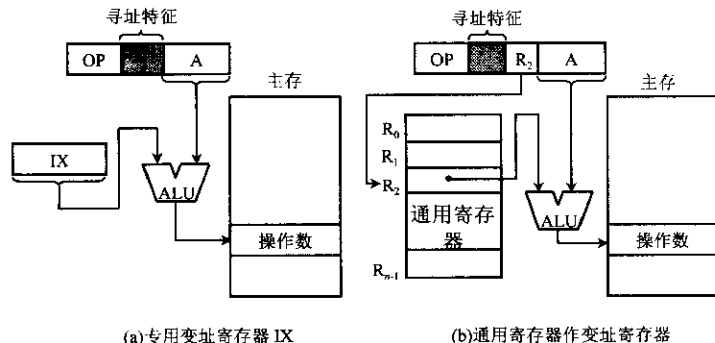


图 7.16 变址寻址示意

图 7.16 (a)、(b) 与图 7.15 (a)、(b) 相比, 显见变址寻址与基址寻址的有效地址形成过程极为相似。由于两者的应用场合不同, 因此从本质来认识, 它们还是有较大的区别。基址寻址主要用于为程序或数据分配存储空间, 故基址寄存器的内容通常由操作系统或管理程序确定, 而指令字中的 A 是可变的。在变址寻址中, 变址寄存器的内容是由用户设定的, 在程序执行过程中其值可变, 而指令字中的 A 是不可变的。变址寻址主要用于处理数组问题, 在数组处理过程中, 可设定 A 为数组的首地址, 不断改变变址寄存器 IX 的内容, 便可很容易形成数组中任一数据的地址, 特别适合编制循环程序。例如某数组有 N 个数存放在以 D 为首地址的主存一段空间内。如果求 N 个数的平均值, 则

用直接寻址方式很容易完成程序的编制。表 7.1 列出了用直接寻址求 N 个数平均值的程序。

表 7.1 直接寻址求 N 个数的平均值程序

程 序	说 明
LDA D	$[D] \rightarrow \text{ACC}$
ADD D+1	$[\text{ACC}] + [D+1] \rightarrow \text{ACC}$
ADD D+2	$[\text{ACC}] + [D+2] \rightarrow \text{ACC}$
⋮	⋮
ADD D+(N-1)	$[\text{ACC}] + [D+(N-1)] \rightarrow \text{ACC}$
DIV #N	$[\text{ACC}] \div N \rightarrow \text{ACC}$
STA ANS	$[\text{ACC}] \rightarrow \text{ANS 单元 (ANS 为内存某单元地址)}$

显然，当 $N=100$ 时，该程序用了 102 条指令，除数据外，共占用 102 个单元存放指令。

若用变址寻址，则只要改变变址寄存器的内容，而保持指令 ADD X,D (X 为变址寄存器) 不变，便可依次完成 N 个数相加。用变址寻址编制的程序如表 7.2 所示。

表 7.2 变址寻址求 N 个数的平均值程序

程 序	说 明
LDA #0	$0 \rightarrow \text{ACC}$
LDX #0	$0 \rightarrow X$ (X 为变址寄存器)
M ADD X,D	$[\text{ACC}] + [D+(X)] \rightarrow \text{ACC}$ (D 为形式地址，X 为变址寄存器)
INX	$[X] + 1 \rightarrow X$
CPX #N	$[X] - N$ ，并建立 Z 的状态，结果为“0” Z=1；结果非“0” Z=0
BNE M	当 Z=1 时，按顺序执行；当 Z=0 时，转至 M
DIV #N	$[\text{ACC}] \div N \rightarrow \text{ACC}$
STA ANS	$[\text{ACC}] \rightarrow \text{ANS (ANS 为内存某单元地址)}$

这里仅用了八条指令编完了解题程序，指令所占的存储单元大大减少。

有的机器（如 Intel 8086、VAX-11）的变址寻址具有自动变址的功能，即每存取一个数据，根据数据长度（即所占字节数），变址寄存器能自动增量或减量，以便形成下一个数据的地址。

变址寻址还可以与其他寻址方式结合使用。例如变址寻址可与基址寻址合用，此时有效地址 EA 等于指令字中的形式地址 A 和变址寄存器 IX 的内容 (IX) 及基址寄存器 BR 中的内容 (BR) 相加之和。即

$$EA = A + (IX) + (BR)$$

变址寻址还可与间址合用，形成先变址后间址或先间址再变址等寻址方式，读者在使用各类机器时可注意分析。

9. 相对寻址

相对寻址的有效地址是将程序计数器 PC 的内容（即当前指令的地址）与指令字中的形式地址 A 相加而成。即

$$EA = (PC) + A$$

图 7.17 示意了相对寻址的过程，由图可见，操作数的位置与当前指令的位置有一段距离 A。

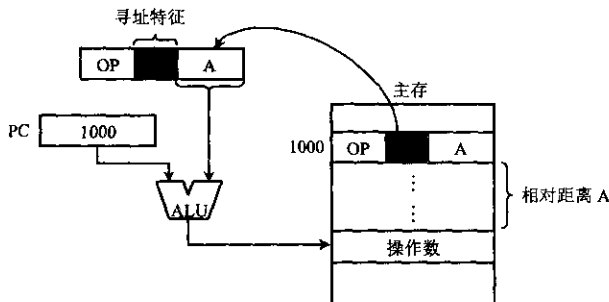


图 7.17 相对寻址示意

相对寻址常被用于转移类指令，转移后的目标地址与当前指令有一段距离，叫做相对位移量，它由指令字的形式地址 A 给出，故 A 又可称位移量。位移量 A 可正可负，通常用补码表示。倘若位移量为 8 位，则指令的寻址范围在 $(PC)+127$ 至 $(PC)-128$ 之间。

相对寻址的最大特点是转移地址不固定，它可随 PC 值的变化而变，因此，无论程序在主存的哪段区域，都可正确运行，对于编写浮动程序特别有利。例如表 7.2 中有一条转移指令 BNE M，它存于 M+3 单元内，也即

	⋮
→	M ADD X,D
	M+1 INX
	M+2 CPX #N
└─	M+3 BNE M

显然，随程序首地址改变，M 也改变。如果采用相对寻址，将 BNE M 改写为 BNE *-3 (*为相对寻址特征)，就可使该程序浮动至任一地址空间都能正常运行。因为从第 M+3 条指令转至第 M 条指令，其相对位移量为 -3，故当执行第 M+3 条指令 BNE *-3 时，其有效地址为

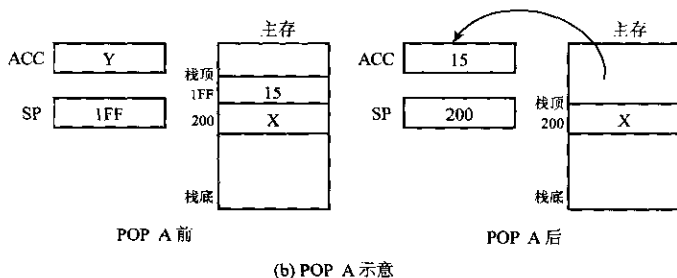
$$EA = (PC) + (-3) = M + 3 - 3 = M$$

直接指向了转移后的目标地址。

相对寻址也可与间址寻址配合使用。

10. 堆栈寻址

在堆栈寻址的指令字中没有形式地址码字段，它是一种零地址指令。堆栈寻址要求计算机中设有堆栈。堆栈既可用寄存器组（称为硬堆栈）来实现，也可利用主存的一部分空间作堆栈（称为软堆栈）。堆栈的运行方式为先进后出或先进先出两种，先进后出型堆栈的操作数只能从一个口进行读或写。以软堆栈为例，可用堆栈指针 SP（Stack Point）指出栈顶地址，也可用 CPU 中一个或两个寄存器作为 SP。操作数只能从栈顶地址指示的存储单元存取。可见堆栈寻址也可视为一种隐含寻址，其操作数的地址总被隐含在 SP 中。堆栈寻址就其本质也可视为寄存器间址，因 SP 可视为寄存器，它存放着操作数的有效地址。图 7.18 示意了堆栈寻址过程。



(b) POP A 示意

图 7.18 堆栈寻址示意

图中 (a)、(b) 分别表示进栈 PUSH A 和出栈 POP A 的过程。

由于 SP 始终指示着栈顶地址，因此不论是执行进栈 (PUSH)，还是出栈 (POP)，SP 的内容都需发生变化。若栈底地址大于栈顶地址，则每次进栈 $(SP) - \Delta \rightarrow SP$ ；每次出栈 $(SP) + \Delta \rightarrow SP$ 。 Δ 取值与内存编址方式有关。若按字编址，则 Δ 取 1（如图 7.18 所示）；若按字节编址，则需根据存储字长是几个字节构成才能确定 Δ ，例如字长为 16 位，则 $\Delta=2$ ，字长为 32 位， $\Delta=4$ 。

由于当前计算机种类繁多，各类机器的寻址方式均有各自特点，还有些机器的寻址方式可能本教材并未提到，故读者在使用时需自行分析，以利编程。

从高级语言角度考虑问题，机器指令的寻址方式对用户无关紧要，但一旦采用汇编语言编程，用户只有了解掌握该机的寻址方式，才能正确编程，否则程序将无法正常运行。如果读者参与机器的指令系统设计，则了解寻址方式对确定机器指令格式是不可缺少的。从另一角度来看，倘若透彻了解了机器指令

的寻址方式，将会使读者进一步加深对机器内信息流程及整机工作概念的理解。

7.4 指令格式举例

指令格式不仅体现了指令系统的各种功能，而且也突出地反映了机器的硬件结构特点。设计指令格式时必须从诸多方面综合考虑，并经一段模拟运行后，最后确定。

7.4.1 设计指令格式应考虑的各种因素

指令系统集中反映了机器的性能，又是程序员编程的依据。用户在编程时既希望指令系统很丰富，便于用户选择，同时还要求机器执行程序时速度快、占用主存空间少，实现高效运行。此外，为了继承已有的软件，必须考虑新机器的指令系统与同一系列机器指令系统的兼容性，即高档机必须能兼容低档机的程序运行，称之为“向上兼容”。

指令格式集中体现了指令系统的功能，为此，在确定指令格式时，必须从以下几个方面综合考虑。

- 操作类型：包括指令数及操作的难易程度；
- 数据类型：确定哪些数据类型可以参与操作；
- 指令格式：包括指令字长、操作码位数、地址码位数、地址个数、寻址方式以及指令字长和操作码位数是否可变等等；
- 寻址方式；
- 寄存器个数：寄存器的多少直接影响指令的执行时间。

7.4.2 指令格式举例

不同机器其指令格式可以有很大的差别，本教材不可能将各种机器的指令格式都作介绍，只能列举几种较为典型的格式供读者学习。

1. PDP-8

PDP-8 的指令字长统一为 12 位，CPU 内只设一个通用寄存器，即累加器 ACC，其主存被划分为若干个容量相等的存储空间（每个相同的空间被称为一页）。该机的指令格式可分为三大类，如图 7.19 所示。

相同的一系列计算机。IBM 370 对 IBM 360 是完全向上兼容的。所以 IBM 370 可看作 IBM 360 的扩展或延伸或改进。

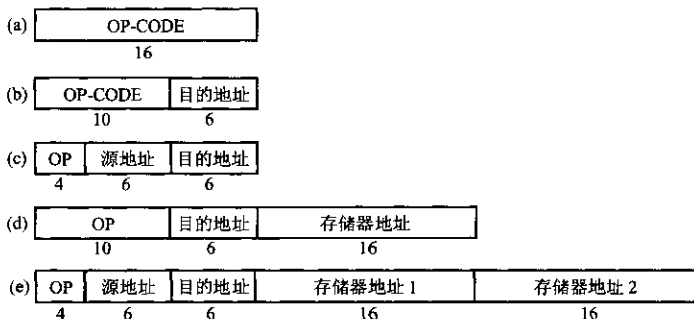


图 7.20 PDP-11 五种指令格式

IBM 360 是 32 位机器，按字节寻址，并可支持多种数据类型，如字节、半字、字、双字（双精度实数）、压缩十进制数、字符串等等。在 CPU 中有 16 个 32 位通用寄存器（用户可选定任一个寄存器作为基址寄存器 BR 或变址寄存器 IX），4 个双精度（64 位）浮点寄存器。指令字长有 16 位、32 位、48 位三种，如图 7.21 所示。

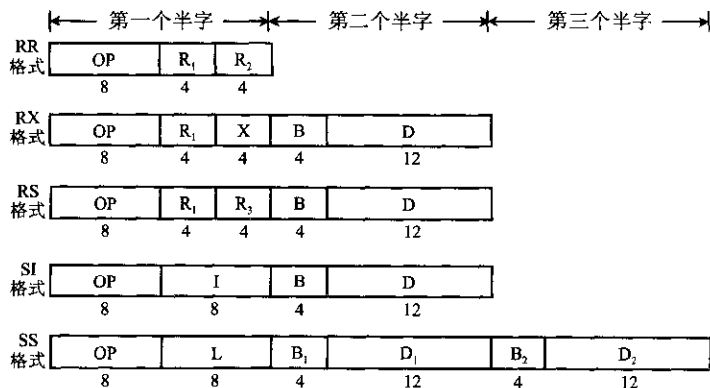


图 7.21 IBM 360/370 指令格式

图中共画出了五种指令格式，它们的操作码位数均为 8 位。RR 格式是寄

寄存器—寄存器格式，两个操作数均在寄存器中，完成 $(R_1) OP (R_2) \rightarrow R_1$ 的操作。RX 是二地址格式的寄存器—存储器型指令，一个操作数在寄存器中，另一个操作数在存储器中，其有效地址由变址 (X) 和基址 (B) 寻址方式求得，可以完成 $(R) OP M[(X) + (B) + (D)] \rightarrow R$ 的操作。RS 格式是三地址格式的寄存器—存储器型指令，完成 $(R_3) OP M[(B) + D] \rightarrow R_1$ 操作。SI 格式中的 I 为立即数，它完成立即数 $\rightarrow M[(B) + D]$ 的操作。SS 格式是存储器—存储器型指令，两个操作数均在存储器，这类指令用于十进制运算和字符串处理，数据长度字段 L 可定义一个长度 (1~256 个字符) 或两个长度 (每一个为 1~16 个十进制数)，它完成 $M[(B_1) + D_1] OP M[(B_2) + D_2] \rightarrow M[(B_1) + D_1]$ 的操作。

4. Intel 8086/80486 系列机

Intel 8086/80486 系列微机的指令字长为 1~6 个字节，即不定长。如零地址格式的空操作指令 NOP 只占一个字节；一地址格式的 CALL 指令可以是 3 字节 (段内调用) 或 5 字节 (段间调用)；二地址格式指令中的两个操作数，既可以是寄存器—寄存器型、寄存器—存储器型，也可以是寄存器—立即数型或存储器—立即数型，它们所占的字节数分别为 2、2~4、2~3、3~6 个字节。有关该系列机的指令格式请读者查阅有关资料自行分析。

7.5 RISC 技术

RISC 是精简指令系统计算机的英文缩写，即 Reduced Instruction Set Computer，与其对应的是 CISC，即复杂指令系统计算机 (Complex Instruction Set Computer)。

7.5.1 RISC 的产生和发展

计算机发展至今，机器的功能越来越强，硬件结构越来越复杂。尤其是随着集成电路技术的发展及计算机应用领域的不断扩大，计算机系统的软件价格相对而言在不断提高。为了节省开销，人们希望已开发的软件能被继承、兼容，这就希望新机种的指令系统和寻址方式一定能包含旧机种所有的指令和寻址方式。通过向上兼容不仅可降低新机种的开发周期和代价，还可吸引更多的新、老用户，于是出现了同类型的系列机。在系列机的发展过程中，致使同一系列计算机指令系统变得越来越复杂，某些机器的指令系统竟可包含几百条指令。例如 DEC 公司的 VAX-11/780 有 16 种寻址方式，9 种数据格式，303 条指令。又如 32 位的 68020 微机指令种数比 6800 多两倍，寻址方式多 11 种，达 18 种之多，指令长度从一个字 (16 位) 发展到 16 个字。这类机器被叫做复杂指令系统计算机，简称 CISC。

通常对指令系统的改进都是围绕着缩小与高级语言语义的差异和有利于操

作系统的优化而进行的。由于编写编译器的人们其任务是为每一条高级语言的语句编制一系列的机器指令，而如果机器指令能类似于高级语言的语句，显然编写编译器的任务就变得十分简单了。于是人们产生了用增加复杂指令的办法来缩短与语义的差距。后来又发现，倘若编译器过多依赖复杂指令，同样会出现新的矛盾。例如对减少机器代码、降低指令执行数以及为提高流水性能而优化生成代码等都是非常不利的。尤其当指令过于复杂时，机器的设计周期会很长、资金耗费会更大。如 Intel 80386 32 位机器耗资达 1.5 亿美元，开发时间长达三年多，结果正确性还很难保证，维护也很困难。最值得一提的例子是，1975 年 IBM 公司投资 10 亿美元研制的高速机器 FS 机，最终以“复杂结构不宜构成高速计算机”的结论，宣告研制失败。

为了解决这些问题，20 世纪 70 年代中期，人们开始进一步分析研究 CISC，发现一个 80-20 规律，即典型程序中 80% 的语句仅仅使用处理机中 20% 的指令，而且这些指令都是属于简单指令，如取数、加、转移等等。这一点告诫人们，付出再大的代价增添复杂指令，也仅有 20% 的使用概率，而且当执行频度高的简单指令时，因复杂指令的存在，致使执行速度也无法提高。表 7.3 是 HP 公司对 IBM 370 高级语言中指令使用频度的分析结果。Marathe 在 1978 年对 PDP-11 机在五种不同应用领域中的指令混合测试，也得出了类似的结论。

表 7.3 IBM370 机指令的使用频度 (%)

指令类型	转移	逻辑	数据 存取	存-存 传送	整数 运算	浮点 运算	十进制 运算	其他
COBOL	24.6	14.6	40.2	12.4	6.4	0.0	1.6	0.6
FORTRAN	18.0	8.1	48.7	2.1	11.0	11.9	0.0	0.2
PASCAL	18.4	9.9	54.0	4.8	7.0	6.8	0.0	0.1

人们从 80-20 规律中得到启示：能否仅仅用最常用的 20% 的简单指令，重新组合不常用的 80% 的指令功能呢？这便引发出 RISC 技术。

1975 年 IBM 公司 John Cocke 提出了精简指令系统的设想，1982 年美国加州伯克莱大学的研究人员，专门研究了如何有效利用 VLSIC（超大规模集成电路）的有效空间。RISC 由于设计的指令条数有限，相对而言，它只需用较小的芯片空间便可制作逻辑控制电路，更多的芯片空间可用来增强处理机的性能或使其功能多样化。他们用大部分芯片空间作成寄存器，并且用它们作暂时数据存储的快速存储区，从而有效地降低了 RISC 机器在调用子程序时所需付出的时间。他们研制的 RISC I（后来又出现 RISC II），采用 VLSI CPU 芯片上的晶体管数量达 44000 个，线宽为 3 μ m，字长为 32 位，其中有 128 个寄存器（而用户只能见到 32 个），仅有 31 条指令，两种寻址方式，访存指令只

有两条,即取数 (LOAD) 和存数 (STORE)。显然其指令系统极为简单,但它们的功能已超过 VAX-11/780 和 M68000,其速度比 VAX-11/780 快了一倍。

与此同时,美国斯坦福大学 RISC 研究的课题是 MIPS (Micro Processor Without Interlocking Pipeline Stages),即“消除流水线各段互锁的微处理器”。他们把 IBM 公司对优化编译程序的研究与伯克莱大学对 VLSI 有效空间利用的思想结合在一起,最终的研究成果后来转化为 MIPS 公司 RX000 的系列产品。IBM 公司又继其 IBM801 型机、IBM RT/PC 后,于 1990 年推出了著名的 IBM RS/6000 系列产品。伯克莱大学的研究成果,最后发展成 Sun 微系统公司的 RISC 芯片,称为 SPARC (Scalable Processor ARChitecture)。

到目前为止,RISC 体系结构的芯片可以说已经历了三代:第一代以 32 位数据通路为代表,支持 Cache,软件支持较少,性能与 CISC 体系结构的产品相当,如 RISC I、MIPS、IBM801 等等。第二代产品提高了集成度,增加了对多处理机系统的支持,提高了时钟频率,建立了完善的存储管理体系,软件支持系统也逐渐完善。它们已具有单指令流水线,可同时执行多条指令,每个时钟周期发出一条指令(有关流水线的概念详见 8.3)。如 MIPS 公司的 R3000 处理器,时钟频率为 25MHz 和 33MHz,集成度达 11.5 万个晶体管,字长为 32 位。第三代 RISC 产品为 64 位微处理器,采用了巨型计算机或大型计算机的设计技术——超级流水线 (Superpipelining) 技术和超标量 (Superscalar) 技术,提高了指令级的并行处理能力,每个时钟周期发出两条或三条指令,使 RISC 处理器的整体性能更好。如 MIPS 的 R4000 处理器采用 50MHz 和 75MHz 的外部时钟频率,内部流水时钟达 100 MHz 和 150 MHz,芯片集成度高达 110 万个晶体管,字长 64 位,并有 16KB 的片内 Cache。它有 R4000PC、R4000SC 和 R4000MC 三种版本,对应不同的时钟频率,分别提供给台式系统、高性能服务器和多处理器环境下使用。表 7.4 列出了 MIPS 公司 R 系列 RISC 处理器的几项指标。

自 1983 年开始出现商品化的 RISC 机以来,比较著名的 RISC 机有 IBM 公司的 IBM RT 系列,HP 公司的精密结构计算机 (HPPA)、MIPS R3000、Motorola M88000、Intel 80960、INMOS Transputer、AMD AM29000、Fairchild Clipper 等。其中 Clipper 兼顾了 RISC 和 CISC 两方面的特点,又称为类 RISC 机。在计算机工作站方面,Sun microsystems 公司于 1987 年推出 SPARC,速度达 7~10MIPS。1988 年 Apollo 公司推出 Series 10000 个人超级计算机,称为并行精简指令系统多处理机 PRISM (Parallel Reduced Instruction Set Multiprocessor),单机系统速度达 15~25MIPS,四处理机则可达 60~100MIPS,后来 HP 合并了 Apollo 公司,继续发展工作站。

较为著名的第三代 RISC 机的有关性能指标,如表 7.5 所示。

表 7.4 MIPS R 系列 RISC 处理器比较

机 种	R2000	R3000	R4000
宣布时间	1986	1988	1991
时钟频率	16.67MHz	25/33 MHz	50/75 MHz
芯片规模	10 万晶体管	11.5 万晶体管	110 万晶体管
结构形式	流水线	流水线	超级流水线
寄存器集	32×32 位	32×32 位	32×64 位, 16×64 位
片上 Cache	—	—	16KB
片外 Cache	最大 128KB	最大 512KB	128KB~4MB
工艺	2μm CMOS	1.2μm CMOS	0.8μm CMOS
功耗	3W	3.5W	
SPEC 分	11.2	17.6(25MHz)	63(50MHz)

表 7.5 第三代 RISC 处理器的性能比较

机 种	R4000	Alpha	Motorola 88110	Super SPARC	RS/6000	i860	C400
公司名称	MIPS	DEC	Motorola	Sun/TI	IBM	Intel	Intergraph
时钟频率 (MHz)	50/75	150/200	50	50/100	33	25/40/50	50
集成度 (万晶体管)	110	168	130	310	120	255	30
结构形式	超流水线	超标量	超标量	超标量	超标量	超长指令	超标量
寄存器集	32×64 16×64 浮	32×64 32×64 浮	32×64 32×64 (32×80)	32×32	32×64 32×64	32×32 16×64	32×32 16×64
片上 Cache	16KB	16KB	16KB	36KB	8KB	32KB	
片外 Cache	128KB ~1MB	最大可达 8MB	256KB ~1MB	2MB			128KB
工艺	0.8μm CMOS	0.75μm CMOS	1μm CMOS	0.8μm CMOS			
功耗		23W		8W	4W		7W
SPEC 分	63 (50MHz)	100(估计)	63.7(估计)	75(估计)	25.9	42	42

注：表中空项待查。

7.5.2 RISC 的主要特征

由上分析可知, RISC 技术是用 20% 的简单指令的组合来实现不常用的 80% 的那些指令功能, 但这并不意味着 RISC 技术就是简单地精简其指令集。在提高性能方面, RISC 技术还采取了许多有效措施, 最有效的办法就是减少指令的执行周期数。

计算机执行程序所需的时间 P 可用下式表述:

$$P = I \times C \times T$$

其中 I 是高级语言程序编译后在机器上运行的机器指令数, C 为执行每条机器指令所需的平均机器周期, T 是每个机器周期的执行时间。

表 7.6 列出了第二代 RISC 机与 CISC 机的 I 、 C 、 T 统计, 其中 I 、 T 为比值, C 为实际周期数。

表 7.6 RISC/CISC 的 I 、 C 、 T 统计比较

	I	C	T
RISC	1.2~1.4	1.3~1.7	<1
CISC	1	4~10	1

由于 RISC 指令比较简单, 用这些简单指令编制出的子程序来代替 CISC 机中比较复杂的指令, 因此 RISC 中的 I 比 CISC 多 20%~40%。但 RISC 的大多数指令仅用一个机器周期完成, C 的值比 CISC 小得多。而且 RISC 结构简单, 完成一个操作所经过的数据通路较短, 使 T 值也大大下降。因此总折算结果, RISC 的性能仍优于 CISC 2~5 倍。

由于计算机的硬件和软件在逻辑上的等效性, 使得指令系统的精简成为可能。曾有人在 1956 就证明, 只要用一条“把主存中指定地址的内容同累加器中的内容求差, 把结果留在累加器中并存入主存原来地址中”的指令, 就可以编出通用程序。

又有人提出, 只要用一条“条件传送 (CMOVE)”指令就可以做出一台计算机。并在 1982 年某大学做出了一台 8 位的 CMOVE 系统结构样机, 叫做 SIC (单指令计算机)。而且, 指令系统所精简的部分可以通过其他部件以及软件 (编译程序) 的功能来替代, 因此, 实现 RISC 技术是完全可能的。

1. RISC 的主要特点

通过对 RISC 各种产品的分析, 可归纳出 RISC 机应具有如下一些特点。

- 选取使用频率较高的一些简单指令以及一些很有用但又不复杂的指令, 让复杂指令的功能由频度高的简单指令的组合来实现。

- 指令长度固定, 指令格式种类少, 寻址方式种类少。
- 只有取数/存数 (LOAD/STORE) 指令访问存储器, 其余指令的操作都在寄存器内完成。
- 采用流水线技术, 大部分指令在一个时钟周期内完成。采用超标量和超流水线技术, 可使每条指令的平均执行时间小于一个时钟周期。
- 控制器采用组合逻辑控制, 不用微程序控制。
- CPU 中有多个通用寄存器。
- 采用优化的编译程序。

值得注意的是, 商品化的 RISC 机通常不会是纯 RISC 机, 故上述这些特点不是所有 RISC 机全部具备的。

表 7.7 列出了一些 RISC 机指令系统的指令条数。

表 7.7 一些 RISC 机的指令条数

机器名	指令数	机器名	指令数
RISC II	39	ACORN	44
MIPS	31	INMOS	111
IBM801	120	IBM RT	118
MIRIS	64	HPPA	140
PYRAMID	128	CLIPPER	101
RIDGE	128	SPARC	89

下面以 RISC II 为例, 着重分析其指令种类和指令格式。

2. RISC II 指令系统举例

(1) 指令种类

RISC II 共有 39 条指令, 分为四类:

- 寄存器—寄存器操作: 移位、逻辑、算术 (整数) 运算等 12 条;
- 取/存数指令: 取存字节、半字、字等 16 条;
- 控制转移指令: 条件转移、调用/返回等 6 条;
- 其他: 存取程序状态字 PSW 和程序计数器等 5 条。

在 RISC II 机中, 有一些常用指令未被选中, 但用上述这些指令并在硬件系统的辅助下, 足以实现其他一些指令的功能。例如该机约定 R_0 寄存器内容恒为 0, 这样加法指令可替代寄存器间的传送指令, 即

$$(R_s) + (R_0) \rightarrow R_d, \text{ 替代了 } R_s \rightarrow R_d$$

加法指令还可替代清除寄存器指令, 即

$$(R_0) + (R_0) \rightarrow R_d, \text{ 替代了 } 0 \rightarrow R_d$$

减法指令可替代取负数指令, 即

$(R_0) - (R_s) \rightarrow R_d$, 替代了 R_d 寄存器内容取负

此外, 该机可用立即数作为一个操作数, 这样当立即数取 1 时, 再用加法 (或减法) 指令就可替代寄存器内容增 1 (减 1) 指令, 即

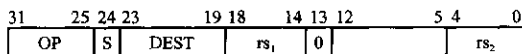
$(R_s) + 1 \rightarrow R_d$

当立即数取 -1 时, 异或指令可替代求反码指令, 即

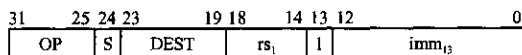
$R_s \oplus (1) \rightarrow R_d$ 替代 $\bar{R}_s \rightarrow R_d$

(2) 指令格式

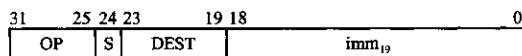
RISC 机的指令格式比较简单, 寻址方式也比较少, 如 RISC II 其指令格式有两种: 短立即数格式和长立即数格式。指令字长固定为 32 位, 指令字中每个字段都有固定位置, 如图 7.22 所示。



(a) 第二源操作数在寄存器中的短立即数格式



(b) 第二源操作数为 imm₁₃ 的短立即数格式



(c) 长立即数格式

图 7.22 RISC II 的指令格式

短立即数格式指令主要用于算逻运算, 其中第 31 位~25 位为操作码; 两个操作数一个在 rs₁ 中, 另一个操作数的来源由指令的第 13 位决定。当其为 0 时 (图 7.22 (a) 所示), 第二个操作数在寄存器中 (只用第 0~4 位); 当其为 1 时 (图 7.22 (b) 所示), 第二个操作数为 13 位的立即数 imm₁₃。运算结果存放在 DEST 所指示的寄存器中 (共 32 个)。指令字中的第 24 位 S 用来表示是否需要根据运算结果置状态位, S = 1 表示置状态位。RISC II 机有 4 个状态位, 即零标志位 Z, 负标志位 N, 溢出标志位 V, 进位标志位 C。

指令中的 DEST 字段在条件转移指令中, 用第 22~第 19 位作为转移条件, 第 23 位无用。对于图 7.22 (b) 所示的短立即数指令格式, 其 imm₁₃ 即为相对转移位移量。

长立即数指令格式主要用于相对转移指令, 此时 19 位的立即数 imm_{19} 指出转移指令的相对位移量, 与 13 位相比, 可扩大相对于 PC 的转移距离。

对于 LOAD 指令, 可根据计算所得的有效地址, 从存储器中读取数据并送入 DEST 字段中指示的目的寄存器中。如短立即数指令有效地址为

$$(\text{rs}_1) + (\text{rs}_2), \text{ 或为 } (\text{rs}_1) + \text{imm}_{13}$$

对于 STORE 指令, 是将 DEST 字段指示的源寄存器中的数, 取出并存入存储器中, 有效地址的计算与 LOAD 指令相同。

3. RISC 指令系统的扩充

从实用角度出发, 商品化的 RISC 机, 因用途不同还可扩充一些指令, 如:

- 浮点指令, 用于科学计算的 RISC 机, 为提高机器速度, 增设浮点指令。
- 特权指令, 为便于操作系统管理机器, 为防止用户破坏机器的运行环境, 特设置特权指令。
- 读后置数指令, 完成读—修改—写, 用于寄存器与存储单元交换数据等。
- 一些简单的专用指令。如某些指令用得较多, 实现起来又比较复杂, 若用子程序来实现, 占用较多的时间, 则可考虑设置一条指令来缩短子程序执行时间。有些机器用乘法步指令来加快乘法运算的执行速度。

7.5.3 RISC 和 CISC 的比较

与 CISC 机相比, RISC 机的主要优点可归纳如下:

1. 充分利用 VLSI 芯片的面积

CISC 机的控制器大多采用微程序控制(详见第十章), 其控制存储器在 CPU 芯片内所占的面积为 50% 以上(如 Motorola 公司的 MC68020 占 68%)。而 RISC 机控制器采用组合逻辑控制(详见第十章), 其硬布线逻辑只占 CPU 芯片面积的 10% 左右。可见它可将空出的面积供其他功能部件用, 例如用于增加大量的通用寄存器(如 Sun 微系统公司的 SPARC 有 100 多个通用寄存器), 或将存储管理部件也集成到 CPU 芯片内(如 MIPS 公司的 R2000/R3000)。以上两种芯片的集成度分别小于 10 万个和 20 万个晶体管。

随着半导体工艺技术的提高, 集成度可达 100 万~几百万个晶体管, 此时无论是 CISC 还是 RISC 都将多个功能部件集成在一个芯片内。但此时 RISC 已占领了市场, 尤其在工作站领域占有明显的优势。

2. 提高计算机运算速度

RISC 机能提高运算速度, 主要反映在以下五个方面。

- RISC 机的指令数、寻址方式和指令格式种类较少, 而且指令的编码很有规律, 因此 RISC 的指令译码比 CISC 快。

- RISC 机内通用寄存器多, 减少了访存次数, 可加快运行速度。
- RISC 机采用寄存器窗口重叠技术, 程序嵌套时不必将寄存器内容保存到存储器中, 故又提高了执行速度。
- RISC 机采用组合逻辑控制, 比采用微程序控制的 CISC 机的延迟小, 缩短了 CPU 的周期。
- RISC 机选用精简指令系统, 适合于流水线工作, 大多数指令在一个时钟周期内完成。

3. 便于设计, 可降低成本, 提高可靠性

RISC 机指令系统简单, 故机器设计周期短, 如美国加州伯克莱大学的 RISC I 机从设计到芯片试制成功只用了十几个月, 而 Intel 80386 处理器 (CISC) 的开发花了三年半时间。

RISC 机逻辑简单, 设计出错可能性小, 有错时也容易发现, 可靠性高。

4. 有效支持高级语言程序

RISC 机靠优化编译来更有效地支持高级语言程序。由于 RISC 指令少, 寻址方式少, 使编译程序容易选择更有效的指令和寻址方式。而且由于 RISC 机的通用寄存器多, 可尽量安排寄存器的操作, 使编译程序的代码优化效率提高。如 IBM 的研究人员发现, IBM 801 (RISC 机) 产生的代码大小是 IBM S/370 (CISC 机) 的 90%。

有些 RISC 机 (如 Sun 公司的 SPARC) 采用寄存器窗口重叠技术, 使过程间的参数传送加快, 且不必保存与恢复现场, 能直接支持调用子程序和过程的高级语言程序。表 7.8 列出了一些 CISC 与 RISC 微处理器的特征。

表 7.8 一些 CISC 与 RISC 微处理器的特征

特 征	CISC			RISC	
	IBM370/168	VAX11/780	Intel 80486	Motorola 88000	MIPS R4000
开发年份	1973	1978	1989	1988	1991
指令数	208	303	235	51	94
指令字长(bytes)	2~6	2~57	1~11	4	32
寻址方式	4	22	11	3	1
通用寄存器数	16	16	8	32	32
控制存储器容量(Kbits)	420	480	246	—	—
Cache 容量(Kbits)	64	64	8	16	128

此外, 从指令系统兼容性看, CISC 大多能实现软件兼容, 即高档机包含

了低档机的全部指令，并可加以扩充。但 RISC 机简化了指令系统，指令数量少，格式也不同于老机器，因此大多数 RISC 机不能与老机器兼容。

PowerPC 是 IBM、Apple、Motorola 三家公司于 1991 年联合，用 Motorola 的芯片制造经验、Apple 的微机软件支持、IBM 的体系结构及其世界计算机市场霸主的地位，向长期被 Intel 占据的微处理器市场挑战而开发的 RISC 产品。

Power PC 中的“PC”意为“Powerful Chip”，其中“Power”基于 20 世纪 80 年代后期，IBM 在其 801 小型机的基础上开发的工作站和服务器中的 Power 体系，意为“Performance Optimization With Enhanced RISC（性能优化的增强型 RISC）”。PowerPC 具有高超的性能、价廉、易仿真 CISC 指令集、可运行大量的现代 CISC 计算机应用软件，即集工作站的卓越性能、PC 机的低成本及运行众多的软件等优点于一身。此外，Power PC 扩展性强，可覆盖 PDA（个人数字助理）到多处理、超并行的中大型机，用单芯片提供整个解决方案。

多年来计算机体系结构和组织发展的趋势是增加 CPU 的复杂性，即使用更多的寻址方式及更加专门的寄存器等。RISC 的出现，象征着与这种趋势根本决裂，自然地引起了 RISC 与 CISC 的争端。随着技术不断发展，RISC 与 CISC 还不能说是截然不同的两大体系，很难对它们做出明确的评价。最近几年，RISC 与 CISC 的争端已减少了很多。原因在于这两种技术已逐渐融合。特别是芯片集成度和硬件速度的增大，RISC 系统也越来越复杂。与此同时，在努力挖掘最大性能的过程中，CISC 的设计已集中到和 RISC 相关联的主题上来，例如增加通用寄存器数以及更加强调指令流水线设计，所以更难去评价它们的优越性了。

RISC 技术发展很快，有关 RISC 体系结构、RISC 流水、RISC 编译系统、RISC 和 CISC 和 VLIW（Very Long Instruction Word，超长指令字）技术的融合等方面的资料不少。读者若想深入了解，请查阅有关文献。

思考题与习题

1. 什么叫机器指令？什么叫指令系统？为什么说指令系统与机器的主要功能以及与硬件结构之间存在着密切关系？
2. 什么叫寻址方式？为什么要学习寻址方式？
3. 什么是指令字长、机器字长和存储字长？
4. 零地址指令的操作数来自哪里？一地址指令中，另一个操作数的地址通常可采用什么寻址方式获得？各举一例说明。
5. 对于二地址指令而言，操作数的物理地址可安排在什么地方？举例说明。
6. 试比较间接寻址和寄存器寻址。

7. 试比较基址寻址和变址寻址。
8. 画出先变址再间址及先间址再变址的寻址过程示意图。
9. 画出 SUB @R1 指令对操作数的寻址及减法过程的流程图。设被减数和结果存于 ACC 中, @表示间接寻址, R1 寄存器的内容为 2074H。
10. 画出执行 ADD * -5 指令 (*为相对寻址特征)的信息流程图。设另一个操作数和结果存于 ACC 中, 并假设 (PC) = 4000H。
11. 设相对寻址的转移指令占两个字节, 第一个字节是操作码, 第二个字节是相对位移量, 用补码表示。假设当前转移指令第一字节所在的地址为 2000H, 且 CPU 每取出一个字节便自动完成 (PC) + 1 → PC 的操作。试问当执行 JMP * +8 和 JMP * -9 指令时, 转移指令第二字节的内容各为多少?
12. 某机主存容量为 4M×16 位, 且存储字长等于指令字长, 若该机指令系统可完成 108 种操作, 操作码位数固定, 且具有直接、间接、变址、基址、相对、立即等六种寻址方式, 试回答:
 - (1) 画出一地址指令格式并指出各字段的作用;
 - (2) 该指令直接寻址的最大范围;
 - (3) 一次间址和多次间址的寻址范围;
 - (4) 立即数的范围 (十进制表示);
 - (5) 相对寻址的位移量 (十进制表示);
 - (6) 上述六种寻址方式的指令哪一种执行时间最短? 哪一种最长? 为什么? 哪一种便于程序浮动? 哪一种最适合处理数组问题?
 - (7) 如何修改指令格式, 使指令的寻址范围可扩大到 4M?
 - (8) 为使一条转移指令能转移到主存的任一位置, 可采取什么措施? 简要说明之。
13. 举例说明哪几种寻址方式在指令的执行阶段不访问存储器? 哪几种寻址方式在指令的执行阶段只需访问一次存储器? 完成什么样的指令, 包括取指令在内共访问四次存储器?
14. 某机器共能完成 78 种操作, 若指令字长为 16 位, 试问一地址格式的指令地址码可取几位? 若想使指令寻址范围扩大到 2^{16} , 可采用什么办法? 举出三种不同例子加以说明。
15. 某 CPU 内有 32 个 32 位的通用寄存器, 设计一种能容纳 64 种操作的指令系统。假设指令字长等于机器字长, 试回答:
 - (1) 如果主存可直接或间接寻址, 采用“寄存器—存储器”型指令, 能直接寻址的最大存储空间是多少? 画出指令格式并说明各字段的含义。
 - (2) 如果采用通用寄存器作基址寄存器, 则上述“寄存器—存储器”型指令的指令格式有何特点? 画出指令格式并指出这类指令可访问多大的存储空间?
16. 某机字长 16 位, 存储器直接寻址空间为 128 字, 变址时的位移量为 -64 ~ +63, 16 个通用寄存器均可作为变址寄存器。采用扩展操作码技术, 设计一套指令系统格式, 满足下列寻址类型的要求:

- (1) 直接寻址的二地址指令 3 条;
- (2) 变址寻址的一地址指令 6 条;
- (3) 寄存器寻址的二地址指令 8 条;
- (4) 直接寻址的一地址指令 12 条;
- (5) 零地址指令 32 条。

试问还有多少种代码未用? 若安排寄存器寻址的一地址指令, 还能容纳多少条?

17. 某机指令字长 16 位, 每个操作数的地址码为 6 位, 设操作码长度固定, 指令分为零地址、一地址和二地址三种格式。若零地址指令有 M 种, 一地址指令有 N 种, 则二地址指令最多有几种? 若操作码位数可变, 则二地址指令最多允许有几种?
18. 什么是 RISC? 简述它的主要特点。
19. 试比较 RISC 和 CISC。
20. RISC 机中指令简单, 有些常用的指令未被选用, 它用什么方式来实现这些常用指令的功能, 举例说明。

第八章 CPU 的结构和功能

本章从分析 CPU 的功能和内部结构入手，详细讨论了机器完成一条指令的全过程，以及为了进一步提高数据的处理能力、开发系统的并行性所采取的流水技术。此外，本章还进一步概括了中断技术在提高整机系统效能方面的作用。通过本章的学习，希望读者对 CPU 在计算机中的地位和作用，以及对中断概念的理解比前面章节更加深入。

8.1 CPU 的结构

8.1.1 CPU 的功能

由第一章可知，CPU 实质包括运算器和控制器两大部分，第六章讨论了计算机内各种运算及相应的硬件配置，这里重点介绍控制器的功能。

对于冯·诺依曼结构的计算机而言，一旦程序进入存储器后，就可由计算机自动完成取指令和执行指令的任务，控制器就是专用于完成此项工作的，它负责协调并控制计算机各部件执行程序的指令序列，其基本功能是取指令、分析指令和执行指令。

1. 取指令

控制器必须具备能自动地从存储器中取出指令的功能。为此，要求控制器能自动形成指令的地址，并能发出取指令的命令，将对应此地址的指令取到控制器中。第一条指令的地址可以人为指定，也可由系统设定。

2. 分析指令

分析指令包括两部分内容，其一，分析此指令要完成什么操作，即控制器需发出什么操作命令；其二，分析参与这次操作的操作数地址，即操作数的有效地址。

3. 执行指令

执行指令就是根据分析指令产生的“操作命令”和“操作数地址”的要求，形成操作控制信号序列（不同的指令有不同的操作控制信号序列），通过对运算器、存储器以及 I/O 设备的操作，执行每条指令。

此外，控制器还必须能控制程序的输入和运算结果的输出（即控制主机与 I/O 交换信息）以及对总线的管理，甚至能处理机器运行过程中出现的异常情况（如掉电）和特殊请求（如打印机请求打印一行字符），即处理中断的能力。

总之，CPU 必须具有控制程序的顺序执行（称指令控制）、产生完成每条指令所需的控制命令（称操作控制）、对各种操作实施时间上的控制（称时间控制）、对数据进行算术运算和逻辑运算（数据加工）和处理中断等功能。

8.1.2 CPU 结构框图

根据 CPU 的功能不难设想，要取指令，必须有一个寄存器专用于存放当前指令的地址；要分析指令，必须有存放当前指令的寄存器和对指令操作码进行译码的部件；要执行指令，必须有一个能发出各种操作命令序列的控制部件 CU；要完成算术运算和逻辑运算，必须有存放操作数的寄存器和实现算逻辑运算的部件 ALU；为了处理异常情况和特殊请求，还必须中断系统。可见，CPU 可由四大部分组成，如图 8.1 所示。将图 8.1 细化，又可得图 8.2。图中 ALU 部件实际上只对 CPU 内部寄存器的数据进行操作，有关 ALU 的内容已在第六章中有所介绍。

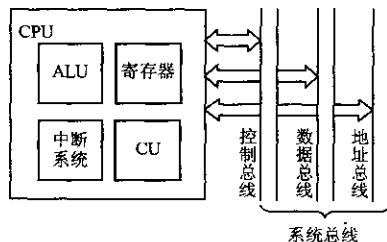


图 8.1 使用系统总线的 CPU

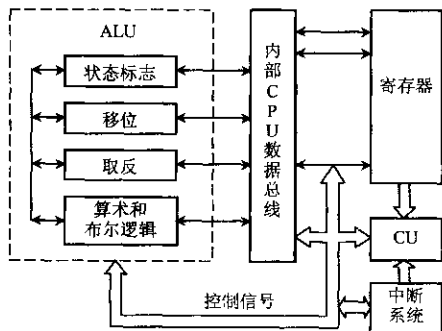


图 8.2 CPU 的内部结构

8.1.3 CPU 的寄存器

第四章图 4.1 示出了存储器的分层结构，最上层的寄存器速度最快，容量最小，每位价格最贵，它们通常放在 CPU 内部。CPU 中的寄存器大致可分两类：一类属用户可见寄存器，用户可对这类寄存器编程，以及通过优化使 CPU 因使用这类寄存器而减少对主存的访问次数；另一类属控制和状态寄存器，用户不可对这类寄存器编程，它们被控制部件使用，以控制 CPU 的操作，也可被带有特权的操作系统程序使用，从而控制程序的执行。

1. 用户可见寄存器

通常 CPU 执行机器语言访问的寄存器为用户可见寄存器，按其特征又可分为以下几类：

(1) 通用寄存器

通用寄存器可由程序设计者指定许多功能，可用于存放操作数，也可作为满足某种寻址方式所需的寄存器。如基址寻址所需的基址寄存器、变址寻址所需的变址寄存器和堆栈寻址所需的栈指针，都可用通用寄存器代替。寄存器间接寻址时还可用通用寄存器存放有效地址的地址。

当然，也有一些机器用专用寄存器作为基址寄存器、变址寄存器或栈指针，这样，在设计指令格式时只需将这类专用寄存器隐含在操作码中，而不必占用指令字中的位 (bits)。如图 7.15 (a) 就是用专用寄存器作基址寄存器，而 (b) 是用通用寄存器作基址寄存器，所以指令字中必须有“R”字段指出寄存器编号。又如图 7.21 所示的 IBM 360/370 指令格式中，由于用通用寄存器作变址寄存器和基址寄存器，故在指令字中设有 X 和 B 字段，分别指出作为变址寄存器和基址寄存器的通用寄存器编号。

(2) 数据寄存器

数据寄存器用于存放操作数，其位数应满足多数数据类型的数值范围，有些机器允许使用两个连读的寄存器存放双倍字长的值。还有些机器的数据寄存器只能用于保存数据，不能用于操作数地址的计算。

(3) 地址寄存器

地址寄存器用于存放地址，其本身可以具有通用性，也可用于特殊的寻址方式，如用于基址寻址的段指针（存放基地址）、用于变址寻址的变址寄存器和用于堆栈寻址的栈指针。地址寄存器的位数必须足够长，以满足最大的地址范围。

(4) 条件代码寄存器

这类寄存器中存放条件码，它们对用户来说是部分透明的。条件码是 CPU

根据运算结果由硬件设置的位 (bits), 如算术运算会产生正、负、零或溢出等结果。条件码可被测试, 作为分支运算的依据。此外, 有些条件码也可被设置, 如最高位进位标志 C, 可用指令对它置位和复位。将条件码放到一个或多个寄存器中, 就构成了条件码寄存器。

在调用子程序前, 必须将所有的用户可见寄存器的内容保存起来, 这种保存可由 CPU 自动保存, 也可由程序员编程保存, 视不同机器不同处理。

2. 控制和状态寄存器

CPU 中还有一类寄存器用于控制 CPU 的操作或运算。在一些机器里, 大部分这类寄存器对用户是透明的。如以下四种寄存器在指令执行过程中起重要作用。

- MAR 存储器地址寄存器, 用于存放将被访问的存储单元的地址;
- MDR 存储器数据寄存器, 用于存放欲存入存储器中的数据或最近从存储器中读出的数据;
- PC 程序计数器, 存放现行指令的地址, 通常具有计数功能。当遇到转移类指令时, PC 的值可被修改;
- IR 指令寄存器, 存放当前欲执行的指令。

通过这四个寄存器, CPU 和主存可交换信息。例如, 将现行指令地址从 PC 送至 MAR, 启动存储器作读操作, 存储器就可将指定地址单元内的指令读至 MDR, 再由 MDR 送至 IR。

在 CPU 内部必须给 ALU 提供数据, 因此 ALU 必须可直接访问 MDR 和用户可见寄存器, ALU 的外围还可以有另一些寄存器, 这些寄存器用于 ALU 的输入/输出以及用于和 MDR 及用户可见寄存器交换数据 (如图 9.4 中的 Y 和 Z 寄存器)。

在 CPU 的控制和状态寄存器中, 还有用来存放程序状态字 PSW 的寄存器, 该寄存器用来存放条件码和其他状态信息。在具有中断系统的机器中还有中断标记寄存器。

3. 举例

不同机器的 CPU 中, 寄存器组织是不一样的, 图 8.3 画出了 Z8000、8086 和 MC68000 三种机器的寄存器组织。

Zilog Z8000 有 16 个 16 位的通用寄存器, 这些寄存器可存放地址、数据, 也可作为变址寄存器, 其中有两个寄存器被用作栈指针, 寄存器可被用作 8 位和 32 位的运算。Z8000 中有 5 个与程序状态有关的寄存器, 一个用于存放状态标记, 两个用于程序计数器, 两个用于存放偏移量。确定一个地址需要两个寄存器。

Intel 8086 采用不同的寄存器组织, 尽管某些寄存器可以通用, 但它的每

个寄存器大多是专用的。它有 4 个 16 位的数据寄存器，即 AX（累加器），BX（基址寄存器），CX（计数寄存器）和 DX（数据寄存器），也可兼作八个 8 位的寄存器（AH、AL、BH、BL、CH、CL、DH、DL）。另外还有两个 16 位的指针（栈指针 SP 和基址指针 BP）和两个变址寄存器（源变址寄存器 SI 和目的变址寄存器 DI）。在一些指令中，寄存器是隐式使用的，如乘法指令总是用累加器。8086 还有四个段地址寄存器（代码段 CS，数据段 DS，堆栈段 SS 和附加段 ES）以及指令指针 IP（相当于 PC）和状态标志寄存器 F。



图 8.3 三种微处理器的寄存器组织

Motorola MC68000 的寄存器组织介于 Zilog 和 Intel 微处理器之间，它将 32 位寄存器分为 8 个数据寄存器(D0~D7)和 9 个地址寄存器(A0~A7')。数据寄存器主要用于数据运算，当需要变址时，也可作变址寄存器用。寄存器允许 8 位、16 位和 32 位的数据运算，这由操作码确定。地址寄存器存放 32 位地址（没有段），其中两个（A7 和 A7'）也可用作堆栈指针，分别供用户和

操作系统使用。针对当前执行的模式，这两个寄存器在某个时刻只能用一个。此外，MC68000 还有一个 32 位的程序计数器 PC 和一个 16 位的状态寄存器。

与 Zilog 的设计者类似，Motorola 设计的寄存器组织也不含专用寄存器。至于到底什么形式的寄存器组织最好，目前尚无一致的观点，主要由设计者自行决定。

计算机的设计者们为了给在早期机器上编写的程序提供向上的兼容性，在新机器的设计上经常保留原设计的寄存器组织形式。图 8.4 就是 Zilog 8000 和 Intel 80386 的用户可见寄存器组织，它们分别是 Z8000 和 8086 的扩展，它们都采用 32 位寄存器，但又分别保留了原先的一些特点。由于受这种限制，因此 32 位处理器在寄存器组织的设计上只有有限的灵活性。

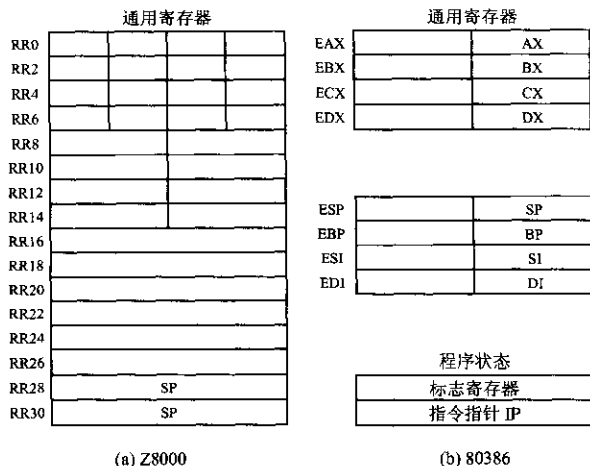


图 8.4 两种 32 位微处理器寄存器组织

8.1.4 控制单元 CU 和中断系统

控制单元 CU 是提供完成机器全部指令操作的微操作命令序列部件。现代计算机中微操作命令序列的形成方法有两种，一种是组合逻辑设计方法，为使连线逻辑；另一种是微程序设计方法，为存储逻辑，具体内容详见第四篇。

中断系统主要用于处理计算机的各种中断，详细内容在本章第四节介绍。

8.2 指令周期

8.2.1 指令周期的基本概念

CPU 每取出并执行一条指令所需的全部时间叫指令周期，也即 CPU 完成一条指令的时间叫指令周期，如图 8.5 所示。图中的取指阶段完成取指令和分析指令的操作，又叫取指周期；执行阶段完成执行指令的操作，又叫执行周期。在大多数情况下，CPU 就是按取指—执行一再取指一再执行……的顺序自动工作的。

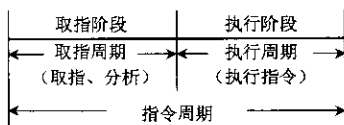


图 8.5 指令周期定义示意

由于各种指令操作功能不同，因此各种指令的指令周期是不相同的。例如无条件转移指令 **JMP X**，在执行阶段不需访问主存，而且操作简单，完全可以在取指阶段的后期将转移地址 **X** 送至 **PC**，以达到转移的目的。这样，**JMP X** 指令的指令周期就是取指周期。又如一地址格式的加法指令 **ADD X**，在执行阶段首先要从 **X** 所指示的存储单元中取出操作数，然后和 **ACC** 的内容相加，结果存于 **ACC**，故这种指令的指令周期在取指和执行阶段各访问一次存储器，其指令周期就包括两个存取周期。再如乘法指令，其执行阶段所要完成的操作比加法指令多得多，故它的执行周期超过了加法指令，如图 8.6 所示。



图 8.6 各种指令周期的比较

此外，当遇到间接寻址的指令时，由于指令字中只给出操作数有效地址的

地址, 因此, 为了取出操作数, 需先访问一次存储器, 取出有效地址, 然后再访问存储器, 取出操作数, 如图 7.11 (a) 所示。这样, 间接寻址的指令周期就包括取指周期、间址周期和执行周期三个阶段, 其中间址周期用于取操作数的有效地址, 因此间址周期介于取指周期和执行周期之间, 如图 8.7 所示。

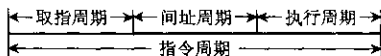


图 8.7 具有间址周期的指令周期

由第五章可知, 当 CPU 采用中断方式实现主机与 I/O 交换信息时, CPU 在每条指令执行阶段结束前, 都要发中断查询信号, 以检测是否有某个 I/O 提出中断请求。如果有请求, CPU 则要进入中断响应阶段, 又称中断周期。在这阶段, CPU 必须将程序断点保存到存储器中。这样, 一个完整的指令周期应包括取指、间址、执行和中断四个子周期, 如图 8.8 所示。由于间址周期和中断周期不一定包含在每个指令周期内, 故图中用菱形框判断。

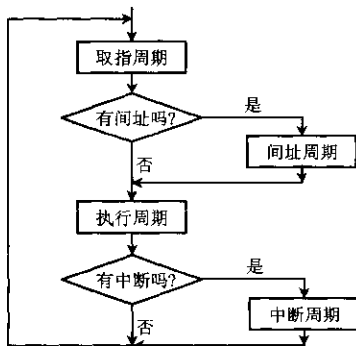


图 8.8 指令周期流程

总之, 上述四个周期都有 CPU 访存操作, 只是访存的目的不同。取指周期是为了取指令, 间址周期是为了取有效地址, 执行周期是为了取操作数 (当指令为访存指令时), 中断周期是为了保存程序断点。这四个周期又可叫 CPU 的工作周期, 为了区别它们, 在 CPU 内可设置四个标志触发器, 如图 8.9 所示。

图 8.9 所示的 FE、IND、EX 和 INT 分别对应取指、间址、执行和中断四

个周期，并以“1”状态表示有效，它们分别由 $1 \rightarrow \text{FE}$ 、 $1 \rightarrow \text{IND}$ 、 $1 \rightarrow \text{EX}$ 和 $1 \rightarrow \text{INT}$ 四个信号控制。

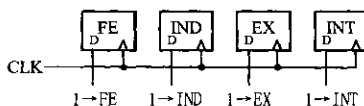


图 8.9 CPU 工作周期的标志

设置 CPU 工作周期标志触发器对设计控制单元十分有利。例如，在取指阶段，只要设置取指周期标志触发器 FE 为 1，由它控制取指阶段的各个操作，便获得对任何一条指令的取指命令序列。又如在间接寻址时，间址次数可由间址周期标志触发器 IND 确定，当它为“0”状态时，表示间址结束。再如对于一些执行周期不访存的指令（如转移指令，寄存器类型指令），同样可以用它们的操作码与取指周期标志触发器的状态相“与”，作为相应微操作的控制条件。这些特点读者在控制单元的设计中可进一步体会。

8.2.2 指令周期的数据流

为了便于分析指令周期中的数据流，假设 CPU 中有存储器地址寄存器 MAR、存储器数据寄存器 MDR、程序计数器 PC 和指令寄存器 IR。

1. 取指周期的数据流

图 8.10 是取指周期的数据流。PC 中存放现行指令的地址，该地址送到 MAR 并送至地址总线，然后由控制部件 CU 向存储器发出读命令，使对应 MAR 所指单元的内容（指令）经数据总线送至 MDR，再送至 IR，与此同时 CU 控制 PC 内容加 1，形成下一条指令的地址。

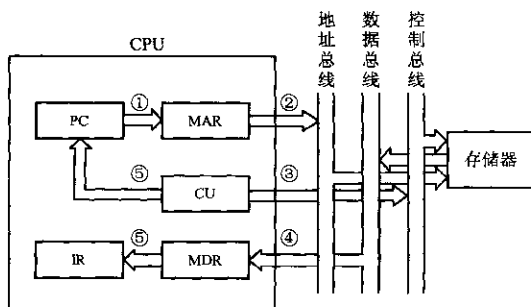


图 8.10 取指周期数据流

2. 间址周期的数据流

间址周期的数据流如图 8.11 所示。一旦取指周期结束，CU 便检查 IR 中的内容，以确定其是否有间址操作，如果需间址操作，则 MDR 中指示形式地址的右 N 位（记作 $Ad(MDR)$ ）将被送到 MAR，又送至地址总线，此后 CU 向存储器发读命令，以获取有效地址并存至 MDR。

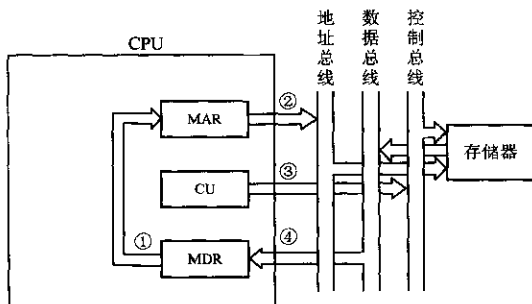


图 8.11 间址周期数据流

3. 执行周期的数据流

由于不同的指令在执行周期的操作不同，因此执行周期的数据流是多种多样的，可能涉及到 CPU 内部寄存器间的数据传送、或对存储器（或 I/O）进行读/写操作、或对 ALU 的操作，因此，无法用统一的数据流图表示。

4. 中断周期的数据流

CPU 进入中断周期要完成一系列操作（详见 9.1），其中 PC 当前的内容必须保存起来，以待执行完中断服务程序后可准确返回到该程序的间断处，这一操作的数据流如图 8.12 所示。

图中由 CU 把用于保存程序断点的存储器特殊地址（如堆栈指针的内容）送往 MAR，并送到地址总线上，同时将 PC 的内容（程序断点）送到 MDR，并命令存储器写，最终使程序断点经数据总线存入存储器。此外，还需将中断服务程序的入口地址送至 PC，为下一个指令周期的取指周期作好准备。

8.3 指令流水

由第四章讨论可知，为了提高访存速度，一方面要提高存储芯片的性能，另一方面从体系结构上，如采用多体、Cache 等分级存储措施来提高存储器的性能/价格比。为了进一步提高整机的处理能力，通常可从两方面入手。

(1) 提高器件的性能。

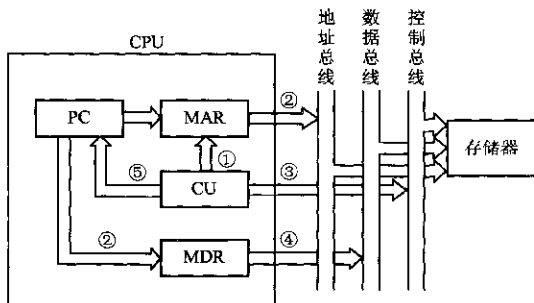


图 8.12 中断周期数据流

提高器件的性能一直是提高整机性能的重要途径，计算机的发展史就是按器件把计算机分为电子管、晶体管、集成电路和大规模集成电路这四代的。器件的每一次更新换代都使计算机的软硬件技术和计算机性能获得突破性进展。特别是大规模集成电路的发展，由于其集成度高、体积小、功耗低、可靠性高、价格便宜等特点，使人们可采用更复杂的系统结构造出性能更高、工作更可靠、价格更低的计算机。但是由于半导体器件的集成度越来越接近物理极限，机器速度的提高越来越慢。

(2) 改进系统的结构，开发系统的并行性。

所谓并行包含同时性和并发性两个方面。前者是指两个或多个事件在同一时刻发生，后者是指两个或多个事件在同一时间段发生。也就是说，在同一时刻或同一时间段内完成两种或两种以上性质相同或不同的功能，只要在时间上互相重叠，就存在并行性。

并行性体现在不同等级上。通常分为四个级别：作业级或程序级、任务级或进程级、指令之间级和指令内部级。前两级为粗粒度，又叫过程级，后两级为细粒度，又叫指令级。粗粒度并行性 (coarse-grained parallelism) 一般用算法 (软件) 实现，细粒度并行性 (fine-grained parallelism) 一般用硬件实现。从计算机体系上看，粗粒度并行性是在多个处理机上分别运行多个进程，由多台处理机合作完成一个程序；细粒度并行性是指在处理机的操作级和指令级的并行性，其中指令的流水作业就是一项重要技术。这里只讨论有关指令流水的一些主要问题，其他有关粗粒度并行和粗粒度并行技术将在《计算机体系结构》中讲述。

8.3.1 指令流水原理

指令流水类似于工厂的装配线，装配线利用了产品在装配的不同阶段其装配过程不同这一特点，使不同产品处在不同的装配段上，即每个装配段同时对不同产品进行加工，这样可大大提高装配效率。将这种装配生产线的思想用到指令的执行上，就引出了指令流水的概念。

从上面分析可知，完成一条指令实际上也可分为许多阶段。为简单起见，把指令的处理过程分为取指令和执行指令两个阶段，在不采用流水技术的计算机里，取指令和执行指令是周而复始地重复出现，各条指令是按顺序串行执行的，如图 8.13 所示。

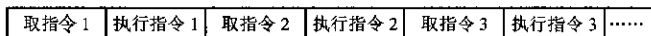


图 8.13 指令的串行执行

图中取指令的操作可由指令部件完成，执行指令的操作可由执行部件完成。进一步分析发现，这种顺序执行虽然控制简单，但执行中各部件的利用率不高，如指令部件工作时，执行部件基本空闲，而执行部件工作时，指令部件基本空闲。如果指令执行阶段不访问主存，则完全可以利用这段时间取下一条指令，这样就使取下一条指令的操作和执行当前指令的操作同时进行，如图 8.14 所示，这就是两条指令的重叠，也即指令的二级流水。

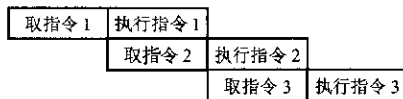


图 8.14 指令的二级流水

由指令部件取出一条指令，并将它暂存起来，如果执行部件空闲，就将暂存的指令传给执行部件执行。与此同时，指令部件又可取下一条指令并暂存起来，这就叫指令预取。显然，这种工作方式能加速指令的执行。如果取指令和执行阶段在时间上完全重叠，相当于指令周期将减半。然而进一步分析流水线，就会发现存在两个原因使得执行效率加倍是不可能的。

① 指令的执行时间一般大于取指时间，因此，取指阶段可能要等待一段时间，也即存放在指令部件缓冲区的指令还不能立即传给执行部件，缓冲区不能空出。

② 当遇到条件转移指令时，下一条指令是不可知的，因为必须等到执行阶段结束后，才能获知条件是否成立，从而决定下条指令的地址，造成时间损失。

通常为了减少时间损失，采用猜测法，即当条件转移指令从取指阶段进入执行阶段时，指令部件仍按顺序预取下一条指令，这样，如果条件不成立，转移没发生，则没有时间损失；若条件成立，转移发生，则所取的指令必须丢掉，并再取新的指令。

尽管这些因素降低了两级流水线的潜在效率，但还是可以获得一定程度的加速。为了进一步提高处理速度，需将指令的处理过程分解为更细的几个阶段。

- 取指 (FI)：从存储器取出一条指令并暂时存入指令部件的缓冲区。
- 指令译码 (DI)：确定操作性质和操作数地址的形成方式。
- 计算操作数地址 (CO)：计算操作数的有效地址，涉及到寄存器间址、间址、变址、基址、相对寻址等各种地址计算方式。
- 取操作数 (FO)：从存储器中取操作数（若操作数在寄存器中，则无需此阶段）。
- 执行指令 (EI)：执行指令所需的操作，并将结果存于目的位置（寄存器中）。
- 写操作数 (WO)：将结果存入存储器。

为了说明方便起见，假设上述各段的时间都是相等的，于是可得图 8.15 所示的指令六级流水时序。在这个流水线中，处理器有六个操作部件，同时对六条指令进行加工，加快了程序的执行速度。

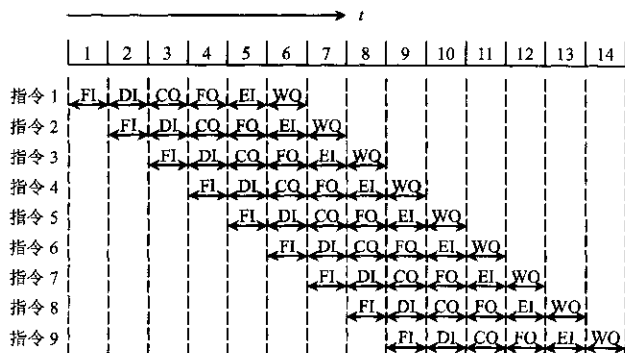


图 8.15 指令六级流水时序

图中 9 条指令若不采用流水线技术, 最终出结果需 54 个时间单元, 采用六级流水只需 14 个时间单元就可出最后结果, 大大提高了处理器速度。当然, 图中假设每条指令都经过流水线的六个阶段, 但事实并不总是这样。如取数指令并不需要 WO 阶段。此外, 这里还假设不存在存储器访问冲突, 所有阶段均并行执行。如 FI、FO 和 WO 阶段都涉及存储器访问, 如果出现冲突就无法并行执行, 该图示意所有这些访问都可以同时进行。但多数存储系统做不到这点, 从而影响了流水线的性能。

还有一些其他因素也会影响流水线性能, 如六个阶段时间不等或遇到转移指令, 都会出现讨论二级流水时出现的问题。

8.3.2 影响流水线性能的因素

影响流水线性能的因素主要反映在两方面: 访存冲突和相关问题。

1. 访存冲突

由上分析可知, 取指令、取操作数和存结果都要访问存储器。如在图 8.15 的第 5 个时间单元, 取第 5 条指令与取第二条指令的操作数发生了访存冲突。又如在第 6 个时间单元, 取第 6 条指令与取第 3 条指令的操作数和存第 1 条指令的结果二者之间又出现了访存冲突。为了避免冲突, 可采用如下一些方法:

① 设置两个独立的存储器分别存放操作数和指令, 以免取指令和取操作数同时进行互相冲突, 使取某条指令和取另一条指令的操作数实现时间上的重叠。

② 采用指令预取技术, 如在 CPU (8086) 中设置指令队列, 将指令预先取到指令队列中排队。指令预取技术的实现基于访存周期很短的情况, 如在执行指令阶段, 取数时间很短, 因此在执行指令时, 主存会有空闲, 此时, 只要指令队列空出, 就可取下一条指令, 并放至空出的指令队列中, 从而保证在执行第 K 条指令的同时对第 $K+1$ 条指令进行译码, 实现“执行 K ”与“分析 $K+1$ ”的重叠。

2. 相关问题

所谓相关问题是指程序的相近指令之间出现某种关联, 使指令流水线出现停顿, 影响了指令流水线的效率。例如, 当下一条指令需用到前面一条 (或几条) 指令的结果时, 必须待前面的指令流出流水线后才能执行。尤其当流水级数增多时, 由于同时解释多条指令, 所以相关的情况更为复杂。指令间的相关大体可分控制相关和数据相关两类。

(1) 控制相关

如果一条指令要等前一条 (或几条) 指令作出转移方向的决定后, 才能进入流水线, 便发生了控制相关, 最典型的情况就是条件转移指令。图 8.16 示

意了条件转移的效果，这里使用了和图 8.15 相同的程序，并假设指令 3 是一条条件转移指令，即指令 3 必须待指令 2 的结果出现后（第 7 个时间单元）才能决定下一条指令是 4（条件不满足）还是 15（条件满足）。由于结果无法预测，此流水线继续预取指令 4，并向前推进。当最后结果满足条件时，发现对第 4、5、6、7 条指令所做的操作全部报废。在第 8 个时间单元，指令 15 进入流水线。在时间单元 9 到 12 间没有指令完成，这就是由于不能预测转移条件而带来的性能损失。而图 8.15 中因转移条件不成立，未发生转移，得到了较好的流水线性能。

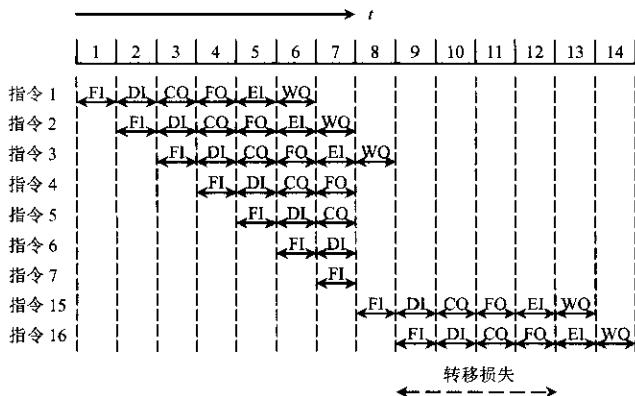


图 8.16 条件转移对指令流水操作的影响

为了解决控制相关，有各种措施，读者可查阅有关资料进一步了解。

(2) 数据相关

数据相关是发生在几条相近的指令间共用同一个存储单元或寄存器时发生的。例如某条指令为了计算操作数的地址需用到某寄存器的内容。但是产生这“内容”的指令还未执行结束，也即还没有产生结果，这时流水线也只能暂停等待。为了解决此问题，可采用旁路技术，即在执行部件与指令部件之间设置直接传送数据的通路，在执行部件产生结果向寄存器送数的同时，把此数直接送至指令部件，计算操作数的有效地址。

数据相关又分读—写相关、写—读相关、写—写相关。它们的发生与流水线的控制方式（顺序流动流水线和非顺序流动流水线）有关。有关这方面的内容在此不详述，读者可进一步查阅资料了解。

8.3.3 流水线中的多发技术

流水线技术使计算机系统结构产生重大革新, 为了进一步发展, 除了采用好的指令调度算法、重新组织指令执行顺序、降低相关带来的干扰以及优化编译外, 还可开发流水线中的多发技术, 设法在一个时钟周期(机器主频的倒数)内, 产生更多条指令的结果。常见的多发技术有超标量技术、超流水线技术和超长指令字技术。假设处理一条指令分四个阶段: 取指 (IF)、译码 (ID)、执行 (EX) 和回写 (WR)。图 8.17 是三种多发技术与普通四级流水线的比较, 其中 (a) 为普通四级流水线, 一个时钟周期出一个结果。

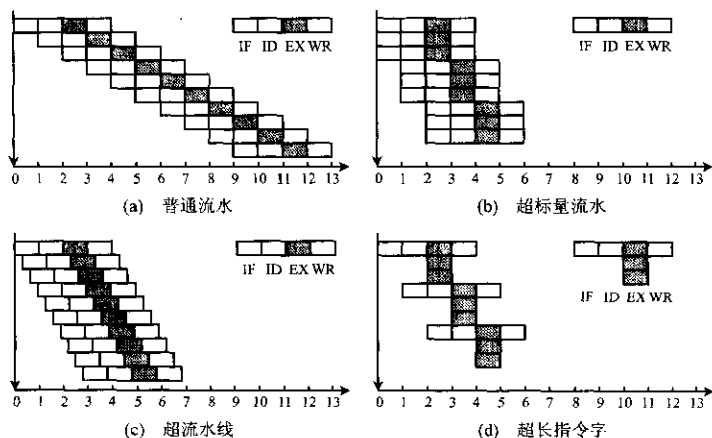


图 8.17 四种流水技术的比较

1. 超标量技术

超标量 (Super scalar) 技术如图 8.17 (b) 所示。它是指在每个时钟周期内可同时并发多条独立指令, 即以并行操作方式将两条或两条以上 (图中所示为三条) 指令编译并执行。

要实现超标量技术, 要求处理机中配置多个功能部件和指令译码电路, 以及多个寄存器端口和总线, 以便能实现同时执行多个操作, 此外还要编译程序决定哪几条相邻指令可并行执行。

例如下面两个程序段:

程序段 1	程序段 2
MOV BL, 8	INC AX
ADD AX, 1756H	ADD AX, BX
ADD CL, 4EH	MOV DS, AX

左边程序段中的 3 条指令是互相独立的, 不存在数据相关, 可实现指令级并行。右边程序段中的三条指令存在数据相关, 不能并行执行。超标量计算机不能重新安排指令的执行顺序, 但可以通过编译优化技术, 在高级语言翻译成机器语言时, 精心安排, 把能并行执行的指令搭配起来, 挖掘更多的指令并行性。

2. 超流水线技术

超流水线 (Super pipe lining) 技术是将一些流水线寄存器插入到流水线段中, 好比将流水线再分道, 如图 8.17 (c) 所示。图中将原来的一个时钟周期又分成三段, 使超级流水线的处理器周期比一般流水线的处理器周期 (如图 8.17 (a) 所示) 短, 这样, 在原来的时钟周期内, 功能部件被使用三次, 使流水线以 3 倍于原来时钟频率的速度运行。与超标量计算机一样, 硬件不能调整指令的执行顺序, 靠编译程序解决优化问题。

3. 超长指令字技术

超长指令字 (VLIW) 技术和超标量技术都是采用多条指令在多个处理部件中并行处理的体系结构, 在一个时钟周期内能流出多条指令。但超标量的指令来自同一标准的指令流, VLIW 则是由编译程序在编译时挖掘出指令间潜在的并行性后, 把多条能并行操作的指令组合成一条具有多个操作码字段的超长指令 (指令字长可达几百位), 由这条超长指令控制 VLIW 机中多个独立工作的功能部件, 由每一个操作码字段控制一个功能部件, 相当于同时执行多条指令, 如图 8.17 (d) 所示。VLIW 较超标量具有更高的并行处理能力, 但对优化编译器的要求更高, 对 Cache 的容量要求更大。

8.3.4 流水线结构

1. 指令流水线结构

指令流水线是将指令的整个执行过程用流水线进行分段处理, 典型的指令执行过程分为取指令—指令译码—形成地址—取操作数—执行指令—回写结果—修改指令指针这几阶段, 与此相对应的指令流水线结构由图 8.18 所示的几个部件组成。

指令流水线对机器性能的改善程度取决于把处理过程分解成多少个相等的时间段数。如上述共分 7 段, 若每一段需一个时钟周期, 则当不采用流水技术时, 需 7 个时钟周期出一个结果。采用流水线后, 假设流水线不出现断流 (如

遇到转移指令), 则除第一条指令需 7 个时钟周期出结果外, 以后所有的指令都是一个时钟周期出一个结果。因此, 在理想的情况下 (流水线不断流) 该流水线的速度约提高到 7 倍。

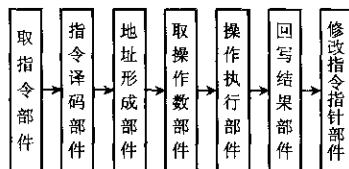


图 8.18 指令流水线结构框图

2. 运算流水线

上述讨论的指令流水线是指令级的流水技术, 实际上流水技术还可用于部件级。如浮点加法运算, 可以分成“对阶”“尾数加”及“结果规格化”三段, 每一段都有一个专门的逻辑电路完成操作, 并将其结果保存在锁存器中, 作为下一段的输入。如图 8.19 所示, 当对阶完成后, 将结果存入锁存器, 便可又可进入下一条指令的对阶运算。

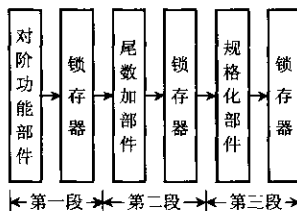


图 8.19 浮点加运算操作流水线

若执行浮点乘运算也按浮点加运算那样分段, 即分成阶码运算、尾数乘和结果规格化三级流水线, 就不够合理。因为尾数乘所需的时间比阶码运算和规格化操作长得多, 而且尾数乘可以和阶码运算同时进行, 因此, 尾数乘本身就可以用流水线。

由图 8.19 可见, 流水线相邻两段在执行不同的操作, 因此在相邻两段之间必须设置锁存器或寄存器, 以保证在一个周期内流水线的输入信号不变。这一指导思想也适用于指令流水。此外, 只有当流水线各段工作饱满时, 才能发挥最大作用。上例中如果浮点运算没有足够的数据来源, 那么流水线中的某些

段、甚至全部段都处于空闲状态，使流水线的作用没有充分发挥。因此具体是否采用流水线技术，以及在计算机的哪一部分采用流水线技术需根据情况而定。

8.4 中断系统

第五章已介绍了有关中断的一些概念，特别对 I/O 中断作了较详细的讨论。实际上 I/O 中断只是 CPU 众多中断中的一种，引起中断的因素很多，为了处理各种中断，CPU 内通常设有处理中断的机构——中断系统，以解决各种中断的共性问题。本节进一步分析中断系统的功能，从而更深入地了解中断系统在 CPU 中的作用和地位。

8.4.1 概述

从前面分析可知，采用中断方式实现主机与 I/O 交换信息可使 CPU 和 I/O 并行工作，提高 CPU 的效率。其实，计算机在运行过程中，除了会遇到 I/O 中断外，还有许多意外事件发生，如电源突然掉电，机器硬件突然出现故障，人们在机器运行过程中想随机抽查计算的中间结果，实现人机联系等。此外，在实时处理系统中，必须及时处理某个事件或现象，如在过程控制系统中，当突然出现温度过高、电压过大等情况时，必须及时将这些信息送至计算机，由计算机暂时中断现行程序，转去执行中断服务程序，以解决这种异常情况。再如计算机实现多道程序运行时，可以通过分配每道程序一个固定时间片，利用时钟定时发中断进行程序切换。在多处理机系统中，各处理器之间的信息交流和任务切换，也可通过中断来实现。总之，为了提高计算机的效率，为了处理一些异常情况以及实时控制、多道程序和多处理机的需要，提出了中断的概念。

1. 引起中断的各种因素

引起中断的因素很多，大致可分为以下几类：

(1) 人为设置的中断

这种中断一般叫做自愿中断，因为它是在程序中人为设置的，故一旦机器执行这种人为中断时，便自愿停止现行程序而转入中断处理，如图 8.20 所示。

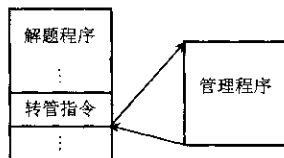


图 8.20 自愿中断示意

图中的“转管”指令可能是转至从 I/O 设备调入一批信息到主存的管理程序；也可能是转至将一批数据送往打印机打印的管理程序。显然，当解题程序执行了“转管”指令后，便中断解题程序，转入了管理程序，这种转移完全是自愿的。

IBM PC (Intel 8086) 的 INT TYPE 指令类似于这种自愿中断，它完成系统调用。TYPE 决定了系统调用的类型。

(2) 程序性事故

如定点溢出、浮点溢出、操作码不能识别、除法中出现“非法”等，这些都属于由程序设计不周而引起的中断。

(3) 硬件故障

硬件故障类型很多，如插件接触不良、通风不良、磁表面损坏、电源掉电等都属硬设备故障。

(4) I/O 设备

I/O 设备被启动以后，一旦其准备就绪，便向 CPU 发出中断请求。每个 I/O 设备都能发中断请求，因此这种中断与计算机所配置的 I/O 设备多少有关。

(5) 外部事件

用户通过键盘来中断现行程序属于外部事件中断。

上述各种中断因素除自愿中断是人为的外，大多都是随机的。通常将能引起中断的各个因素称作中断源，中断源可分两大类，一类为不可屏蔽中断，这类中断 CPU 不能禁止响应，如电源掉电；另一类属可屏蔽中断，对可屏蔽中断源的请求，CPU 可根据该中断源是否被屏蔽来确定是否给予响应，若未屏蔽则能响应；若已被屏蔽，则 CPU 不能响应（有关内容详见 8.4.6 中断屏蔽技术）。

2. 中断系统需解决的问题

- (1) 各中断源如何向 CPU 提出中断请求；
- (2) 当多个中断源同时提出中断请求时，中断系统如何确定优先响应哪个中断源的请求；
- (3) CPU 在什么条件、什么时候、以什么方式来响应中断；
- (4) CPU 响应中断后如何保护现场；
- (5) CPU 响应中断后，如何停止原程序的执行而转入中断服务程序的入口地址；
- (6) 中断处理结束后，CPU 如何恢复现场，如何返回到原程序的间断处；
- (7) 在中断处理过程中又出现了新的中断请求，CPU 该如何处理。

要解决上述七个问题，只有在中断系统中配置相应的硬件和软件，才能完成中断处理任务。

8.4.2 中断请求标记和中断判优逻辑

1. 中断请求标记

为了判断是哪个中断源提出请求，在中断系统中必须设置中断请求标记触发器，简称中断请求触发器，记作 INTR，当其状态为“1”时，表示中断源有请求。这种触发器可集中设在 CPU 内，组成一个中断请求标记寄存器，如图 8.21 所示。

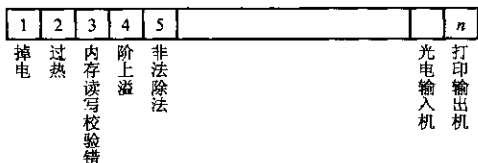


图 8.21 中断请求标记寄存器

图中 1、2、3、4……分别对应掉电、过热、内存读写校验错、阶上溢、非法除法……打印机输出等中断源的中断请求触发器，其中任一个触发器为 1，即表明对应的中断源提出了中断请求。显然，中断请求触发器越多，说明机器处理中断的能力越强。

有一点需要说明，尽管中断请求标记寄存器是由各中断请求触发器组成，但这些触发器既可以集中在 CPU 的中断系统内，也可以分散到各个中断源中。在图 5.41 所示的程序中断方式接口电路中，INTR 就是分散在各个接口电路内的中断请求触发器。

2. 中断判优逻辑

任何一个中断系统，在任一时刻，只能响应一个中断源的请求。但许多中断源提出请求都是随机的，当某一时刻有多个中断源提出中断请求时，中断系统必须按其优先顺序予以响应，这就叫中断判优。各中断源的优先顺序是根据该中断源若得不到及时响应，致使机器工作出错的严重程度而定的。例如，电源掉电对机器工作影响程度最大，优先等级为最高。又如“定点溢出”对机器正常工作影响也很大，若不及时响应，将使机器一切运行均无效，故它的优先等级也较高。对于 I/O 设备，则可按其速度高低安排优先等级，速度高的设备其优先级比速度低的设备高。

中断判优可用硬件实现，也可用软件实现。

(1) 硬件排队

硬件排队又分两种，一种为链式排队器，它对应中断请求触发器分散在各个接口电路中的情况，如图 5.38 所示，每一个接口电路中都设有一个非门和一个与非门，它们犹如链条一样串接起来。另一种排队器设在 CPU 内，如图 8.22 所示，图中假设其优先顺序按 1、2、3、4 由高向低排列。这样，当最高优先级的中断源有请求时 $INTR_1=1$ ，就可封住比它级别低的中断源的请求。

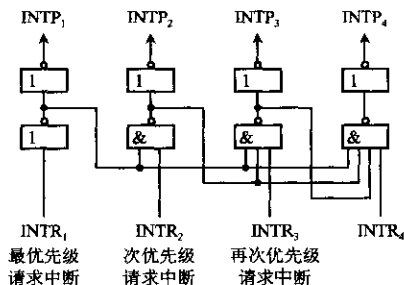


图 8.22 集中在 CPU 的排队器

(2) 软件排队

软件排队是通过编写查询程序实现的，其程序框图如图 8.23 所示。程序按中断源的优先等级，从高至低逐级查询各中断源是否有中断请求，这样就可保证 CPU 首先响应级别高的中断源的请求。

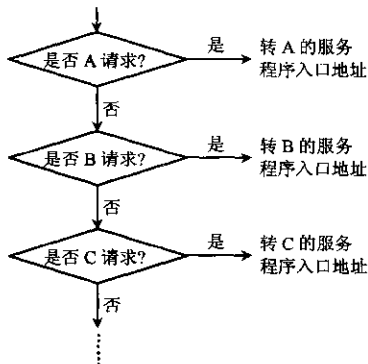


图 8.23 按 A>B>C……优先级别的软件排队

8.4.3 中断服务程序入口地址的寻找

由于不同的中断源对应不同的中断服务程序，故准确找到服务程序的入口地址是中断处理的核心问题。通常有两种方法寻找入口地址：硬件向量法和软件查询法。

1. 硬件向量法

硬件向量法就是利用硬件产生向量地址，再由向量地址找到中断服务程序的入口地址。向量地址由中断向量地址形成部件产生，这个电路可分散设置在各个接口电路中（如图 5.41 中的设备编码器），也可设置在 CPU 内，如图 8.24 所示。

由向量地址寻找中断服务程序的入口地址通常采用两种办法。一种如图 5.40 所示，在向量地址内存放一条无条件转移指令，CPU 响应中断时，只要将向量地址（如 12H）送至 PC，执行这条指令，便可无条件转向打印机服务程序的入口地址 200。另一种是设置向量地址表，如图 8.25 所示。该表设在存储器内，存储单元的地址为向量地址，存储单元的内容为入口地址，如图中 12H、13H、14H 为向量地址，200、300、400 为入口地址，只要访问向量地址所指示的存储单元，便可获得入口地址。

硬件向量法寻找入口地址速度快，在现代计算机中被普遍采用。

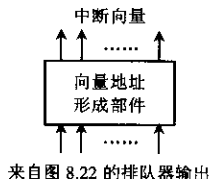


图 8.24 集中在 CPU 内的向量地址形成部件

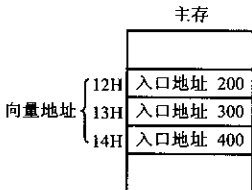


图 8.25 中断向量地址表

2. 软件查询法

用软件寻找中断服务程序入口地址的方法叫软件查询法，其框图同图 8.23。由图可见，当查到某一中断源有中断请求时，接着安排一条转移指令，直接指向此中断源的中断服务程序入口地址，机器便能自动进入中断处理。至于各中断源对应的入口地址，则由程序员（或系统）事先确定。这种方法不涉及硬设备，但查询时间较长。计算机可具备软、硬件两种方法寻找入口地址，

使用户使用更方便、灵活。

8.4.4 中断响应

1. 响应中断的条件

由第五章已知，CPU 响应 I/O 中断的条件是允许中断触发器必须为“1”，这一结论同样适合于其他中断源。在中断系统中有一个允许中断触发器 EINT，它可被开中断指令置“1”，也可被关中断指令置“0”。当允许中断触发器为“1”时，意味着 CPU 允许响应中断源请求；当其为“0”时，意味着 CPU 禁止响应中断。故当 $EINT=1$ ，且有中断请求（即中断请求标记触发器 $INTR=1$ ）时，CPU 可以响应中断。

2. 响应中断的时间

与响应 I/O 中断一样，CPU 总是在指令执行周期结束后，响应任何中断源的请求，如图 8.8 所示。在指令执行周期结束后，若有中断，CPU 则进入中断周期；若无中断，则进入下一条指令的取指周期。

至所以 CPU 在指令的执行周期后进入中断周期，是因为 CPU 在执行周期的结束时刻统一向所有中断源发中断查询信号，只有此时，CPU 才能获知哪个中断源有请求。如图 8.26 所示，图中 $INTR_i$ ($i=1,2,\dots$) 是各个中断源的中断请求触发器，触发器的数据端来自各中断源，当它们有请求时，数据端为“1”，而且只有当 CPU 查询信号到触发器的时钟端时，才能置“1” $INTR_i$ 。

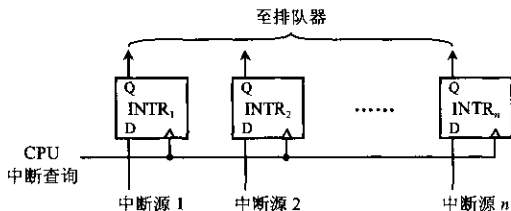


图 8.26 CPU 在统一时间发中断查询信号

在某些计算机中有些指令执行时间很长，若 CPU 的查询信号一律安排在执行周期结束时刻，有可能因 CPU 发现中断请求过迟而出差错。为此，可在指令执行过程中设置若干个查询断点，CPU 在每个“查询断点”时刻均发中断查询信号，以便发现有中断请求便可及时响应。

3. 中断隐指令

CPU 响应中断后，即进入中断周期。在中断周期内，CPU 要自动完成一

系列操作，如：

(1) 保护程序断点

保护程序断点就是要将当前程序计数器 PC 的内容（程序断点）保存到存储器中。它可以存在存储器的特定单元（如 0 号地址）内，也可以存入堆栈。

(2) 寻找中断服务程序的入口地址

由于中断周期结束后进入下一条指令（即中断服务程序的第一条指令）的取指周期，因此在中断周期内必须设法找到中断服务程序的入口地址。由于入口地址有两种方法获得，因此在中断周期内也有两种方法寻找入口地址：

其一，在中断周期内，将向量地址送至 PC（对应硬件向量法），使 CPU 下一条执行无条件转移指令，转至中断服务程序的入口地址。

其二，在中断周期内，将如图 8.23 所示的软件查询入口地址的程序（又叫中断识别程序）其首地址送至 PC，使 CPU 执行中断识别程序，找到入口地址（对应软件查询法）。

(3) 关中断

CPU 进入中断周期，意味着 CPU 响应了某个中断源请求，为了确保 CPU 响应后所需作的一系列操作不至于又受到新的中断请求的干扰，在中断周期内必须自动关中断，以禁止 CPU 再次响应新的中断请求。图 8.27 是 CPU 自动关中断的示意图。图中允许中断触发器 EINT 和中断标记触发器 INT 可选用标准的 R-S 触发器。当进入中断周期时，INT 为“1”状态，触发器原端输出有一个正跳变，经反相后产生一个负跳变，使 EINT 置 0，即关中断。

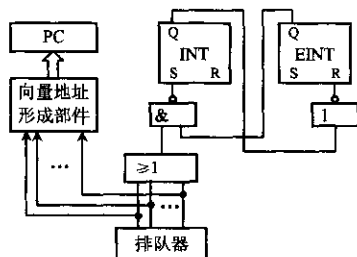


图 8.27 硬件关中断示意

上述保护断点、寻找入口地址和关中断这些操作都是在中断周期内由一条中断隐指令完成的。所谓中断隐指令即在机器指令系统中没有的指令，它是 CPU 在中断周期内由硬件自动完成的一条指令。

8.4.5 保护现场和恢复现场

保护现场应该包括保护程序断点和保护 CPU 内部各寄存器内容的现场两个方面。程序断点的现场由中断隐指令完成，各寄存器的内容可在中断服务程序中由用户（或系统）用机器指令编程实现，参见 5.5.5 及图 5.42。

恢复现场是指在中断返回前，必须将寄存器的内容恢复到中断处理前的状态，这部分工作也由中断服务程序完成，如图 5.42 所示。

8.4.6 中断屏蔽技术

中断屏蔽技术主要用于多重中断。

1. 多重中断的概念

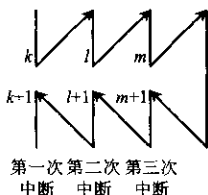


图 8.28 多重中断示意

当 CPU 正在执行某个中断服务程序时，另一个中断源又提出了新的中断请求，而 CPU 又响应了这新的请求，暂时停止了正在运行的服务程序，转去执行新的中断服务程序，这就叫多重中断，或叫中断嵌套，如图 8.28 所示。如果 CPU 对新的请求不予响应，待执行完当前的服务程序后再响应，即为单重中断。中断系统若要具有处理多重中断的功能，必须具备各项条件。

2. 实现多重中断的条件

① 提前设置“开中断”指令。

由上述分析可知，CPU 进入中断周期后，由中断隐指令自动置“0”EINT，即关中断，这就意味着 CPU 在执行中断服务程序中禁止响应新的中断请求。CPU 若想再次响应中断请求，必须开中断，这一任务通常由中断服务程序中的开中断指令实现。由于开中断指令设置的位置不同，决定了 CPU 能否实现多重中断。由图 5.42 可见，多重中断“开中断”指令的位置前于单重中断，从而保证了多重中断允许出现中断嵌套。

② 优先级别高的中断源有权中断优先级别低的中断源。

在满足①的前提下，只有优先级别更高的中断源请求，才可以中断比其级别低的中断服务程序，反之则不然。例如有 A、B、C、D 四个中断源，其优先级按 A、B、C、D 由高向低次序排列。在 CPU 执行主程序期间，同时出现了 B 和 C 的中断请求，由于 B 优先于 C，故首先执行 B 的服务程序。当 B 级

中断服务程序执行完返回主程序后，由于 C 请求未撤消，故 CPU 又再去执行 C 级的中断服务程序。若此时又出现了 D 请求，因为 D 级别低于 C，故 CPU 不予理采，当 C 级中断服务程序执行完返回主程序后再去执行 D 级的服务程序。若此时又出现了 A 请求，因 A 级别高于 D，故 CPU 暂停对 D 级中断服务程序的执行，转去执行 A 级中断服务程序，等 A 级中断服务程序执行完后，再去执行 D 级中断服务程序。上述的中断处理示意图如图 8.29 所示。

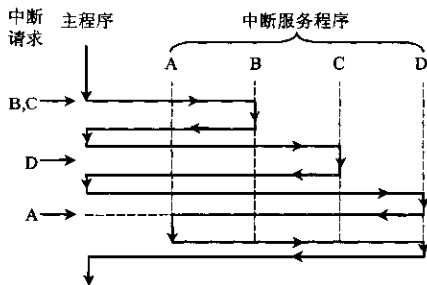


图 8.29 多重中断处理示意

为了保证级别低的中断源不干扰比其级别高的中断源的中断处理过程，保证上述②的实施，可采用屏蔽技术。

3. 屏蔽技术

(1) 屏蔽触发器与屏蔽字

图 5.37 示出了程序中中断接口电路中完成触发器 D、中断请求触发器 INTR 和屏蔽触发器 MASK 三者之间的关系。当该中断源被屏蔽时（MASK=1），此时即使 D=1，中断查询信号到来时刻将 INTR 置 0，CPU 接收不到该中断源的中断请求，即它被屏蔽。若该中断源未被屏蔽（MASK=0），则当设备工作已完成时（D=1），中断查询信号将 INTR 置 1，表示该中断源向 CPU 发出中断请求，该信号送至排队器进行优先级判断。

如果排队器集中设在 CPU 内，加上屏蔽条件，就可组成具有屏蔽功能的排队器，如图 8.30 所示。

显然，对应每个中断请求触发器就有一个屏蔽触发器，将所有屏蔽触发器组合在一起，便构成一个屏蔽寄存器，屏蔽寄存器的内容称作屏蔽字。屏蔽字与中断源的优先级别是一一对应的，如表 8.1 所示。

表 8.1 是对应 16 个中断源的屏蔽字，每个屏蔽字由左向右排序为第 1、2、

3……共 16 位。不难发现, 每个中断源对应的屏蔽字是不同的。1 级中断源的屏蔽字是 16 个 1; 2 级中断源的屏蔽字是从第 2 位开始共 15 个 1; 3 级中断源的屏蔽字是从第 3 位开始共 14 个 1; ……第 16 级中断源的屏蔽字只有第 16 位为 1, 其余各位为 0。

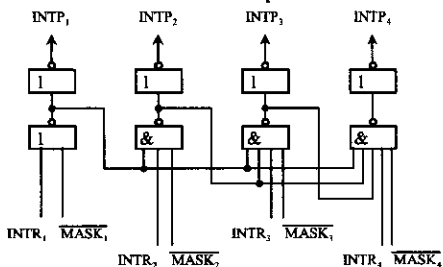


图 8.30 具有屏蔽功能的排队器

表 8.1 中断优先级与屏蔽字的关系

优先级	屏蔽字
1	1111111111111111
2	0111111111111111
3	0011111111111111
4	0001111111111111
5	0000111111111111
6	0000011111111111
⋮	⋮
15	0000000000000011
16	0000000000000001

在中断服务程序中设置适当的屏蔽字, 能起到对优先级不同的中断源的屏蔽作用。例如, 1 级中断源的请求已被 CPU 响应, 若在其中断服务程序中 (通常在开中断指令前) 设置一个全“1”的屏蔽字, 便可保证在执行 1 级中断服务程序过程中, CPU 不再响应任何一个中断源 (包括本级在内) 的中断请求, 即此刻不能实现多重中断。如果在 4 级中断源的服务程序中设置一个屏蔽字 0001111111111111, 由于第 1~3 位为 0, 意味着第 1~3 级的中断源未被屏蔽, 因此在开中断指令后, 比第 4 级中断源级别更高的 1、2、3 级中断源可以中断 4 级中断源的中断服务程序, 实现多重中断。

(2) 屏蔽技术可改变优先等级

利用屏蔽技术可以任意改变中断源的优先等级。例如 5 级中断源级别高于 6 级中断源，但若在中断服务程序中先设置一个屏蔽字 0000101111111111，这样，当 5、6 级中断源同时有请求时，由于 5 级被屏蔽，6 级未被屏蔽，因此 CPU 优先响应 6 级中断源的请求。只有当处理完 6 级中断源的请求后，再设一屏蔽字 0000011111111111，CPU 才能响应 5 级中断源的请求。

(3) 屏蔽技术的其他作用

屏蔽技术还能给程序控制带来更大的灵活性。例如，在浮点运算中，当程序员估计到执行某段程序时可能出现“阶上溢”，但又不希望因“阶上溢”而使机器停机，为此可设一屏蔽字，使对应“阶上溢”的屏蔽位为“1”，这样，即使出现“阶上溢”，机器也不停机。

4. 多重中断的断点保护

多重中断时，每次中断出现的断点都必须保存起来，如图 8.28 中共出现三次中断，有三个断点 $k+1$ 、 $l+1$ 、 $m+1$ 需保存。中断系统对断点的保存都是在中断周期内由中断隐指令实现的，对用户是透明的。

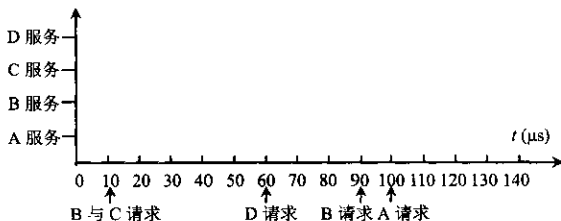
断点可以保存在堆栈中，由于堆栈先进后出的特点，因此图 8.28 中的 $k+1$ 先进栈，接着是 $l+1$ 进栈，最后是 $m+1$ 进栈。出栈时，按相反顺序便可准确返回到程序间断处。

断点也可保存在特定的存储单元内，例如约定一律将程序断点存至主存的 0 号地址单元内。由于保存断点是由中断隐指令自动完成的，因此三次中断的断点都将存入 0 地址单元，这势必造成前两次存入的断点 $k+1$ 和 $l+1$ 被冲掉。为此，在中断服务程序中的开中断指令之前，必须先将 0 地址单元的内容转存至别的地址单元中，才能真正保存每一个断点。读者可自行练习，画出将程序断点保存到 0 号地址单元的多重中断服务程序流程。

思考题与习题

1. CPU 有哪些功能，画出其结构框图并简要说明每个部件的作用。
2. 什么是指令周期？指令周期是否有一个固定值？为什么？
3. 画出指令周期的流程图，分别说明图中每个子周期的作用。
4. 设 CPU 内有下列部件：PC、IR、SP、AC、MAR、MDR 和 CU，要求：
 - (1) 画出完成间接寻址的取数指令 LDA @X（将主存某地址单元 X 的内容取至 AC 中）的数据流（从取指令开始）。
 - (2) 画出中断周期的数据流。
5. 中断周期前是什么阶段？中断周期后又是什么阶段？在中断周期 CPU 应完成什么操作？

6. 什么叫系统的并行性？粗粒度并行和细粒度并行有何区别？
7. 什么是指令流水？画出指令二级流水和四级流水的示意图，它们中哪一个更能提高处理器速度，为什么？
8. 当遇到什么情况时流水线将受阻？举例说明。
9. 为什么说超长指令字比超标量更能提高并行处理能力？
10. 指令流水线和运算流水线在结构上有何共同之处？
11. 什么是中断？设计中断系统需考虑哪些主要问题？
12. 计算机为了管理中中断，在硬件上通常有哪些设置？各有何作用？对指令系统有何考虑？
13. 在中断系统中 INTR、INT、EINT 三个触发器各有何作用？
14. 什么是中断隐指令，它有哪些功能？
15. 中断系统中采用屏蔽技术有何作用？
16. 为实现多重中断，需有哪些硬件支持？
17. CPU 在处理中断过程中，有几种方法找到中断服务程序的入口地址？举例说明。
18. 中断处理过程中为什么要中断判优？有几种实现方法？若想改变原定的优先顺序，可采取什么措施？
19. 中断处理过程中“保护现场”需完成哪些任务？如何实现？
20. 现有 A、B、C、D 四个中断源，其优先级由高向低按 A、B、C、D 顺序排列。若中断服务程序的执行时间为 $20\mu\text{s}$ ，请根据下图所示时间轴给出的中断源请求中断的时刻，画出 CPU 执行程序的轨迹。



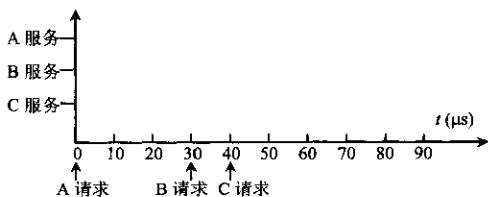
21. 某机有五个中断源 L_0 、 L_1 、 L_2 、 L_3 、 L_4 ，按中断响应的优先次序由高向低排序为 $L_0 \rightarrow L_1 \rightarrow L_2 \rightarrow L_3 \rightarrow L_4$ ，现要求中断处理次序改为 $L_1 \rightarrow L_3 \rightarrow L_4 \rightarrow L_0 \rightarrow L_2$ ，根据下式格式，写出各中断源的屏蔽字。

中断源	屏蔽字				
	1	2	3	4	5
L_0					
L_1					
L_2					
L_3					
L_4					

22. 设某机配有 A、B、C 三台设备，其优先顺序是 $A > B > C$ ，为改变中断处理次序，将他们的中断屏蔽字分别设为：

设备	屏蔽字
A	1 1 1
B	0 1 0
C	0 1 1

请按下图所示时间轴给出的设备请求中断的时刻，画出 CPU 执行程序的轨迹。设 A、B、C 中断服务程序的执行时间均为 $20\mu\text{s}$ 。



第四篇 控制单元 CU

计算机能自动协调地工作是在控制单元 CU 的统一指挥下进行的，本篇详细分析 CU 的功能及其设计思想。

第九章 控制单元的功能

本章结合指令周期的四个阶段，着重分析了控制单元为完成不同指令所发出的各种操作命令，这些命令（又称控制信号）控制着计算机的所有部件有次序地完成相应的操作，以达到执行程序的目的。旨在使读者进一步理解指令周期、机器周期、时钟周期（节拍）和控制信号的关系，进一步体会控制单元在机器运行中所起到的核心作用，为下一章设计控制单元打好基础。

9.1 微操作命令的分析

控制单元具有发出各种微操作命令（即控制信号）序列的功能。

概括地说，计算机的功能就是执行程序。在执行程序的过程中，控制单元要发出各种微操作命令，而且不同的指令对应不同的命令。进一步分析发现，完成不同指令的过程中，有些操作是相同或相似的，如取指令、取操作数地址（当间接寻址时）以及进入中断周期由中断隐指令完成的一系列操作。为更清晰起见，下面按指令周期的四个阶段进一步分析其对应的微操作命令。

9.1.1 取指周期

为了便于讨论，假设 CPU 内有四个寄存器，如图 8.10 所示。MAR 与地址总线相连，存放欲访问的存储单元地址；MDR 与数据总线相连，存放欲写入存储器的信息或最近从存储器中读出的信息；PC 存放现行指令的地址，有计数功能；IR 存放现行指令。取指令的过程可归纳为以下几个操作：

- （1）现行指令地址送至存储器地址寄存器，记作 $PC \rightarrow MAR$ ；
- （2）向主存发读命令，启动主存作读操作，记作 $1 \rightarrow R$ ；
- （3）将 MAR（通过地址总线）所指的主存单元中的内容（指令）经数据总线读至 MDR 内，记作 $M(MAR) \rightarrow MDR$ ；
- （4）将 MDR 的内容送至 IR，记作 $MDR \rightarrow IR$ ；
- （5）形成下一条指令的地址，记作 $(PC) + 1 \rightarrow PC$ 。

9.1.2 间址周期

间址周期完成取操作数有效地址的任务，具体操作如下：

- （1）将指令的地址码部分（形式地址）送至存储器地址寄存器，记作 $Ad(IR) \rightarrow MAR$ ；
- （2）向主存发读命令，启动主存作读操作，记作 $1 \rightarrow R$ ；

(3) 将 MAR (通过地址总线) 所指的主存单元中的内容 (有效地址) 经数据总线读至 MDR 内, 记作 $M(MAR) \rightarrow MDR$;

(4) 将有效地址送至指令寄存器的地址字段, 记作 $MDR \rightarrow Ad(IR)$ 。此操作在有些机器中可省略。

9.1.3 执行周期

不同指令执行周期的微操作是不同的, 下面分别讨论非访存指令、访存指令和转移类指令的微操作。

1. 非访存指令

这类指令在执行周期不访问存储器。

(1) 清除累加器指令 CLA

该指令在执行阶段只完成清除累加器操作, 记作 $0 \rightarrow ACC$ 。

(2) 累加器取反指令 COM

该指令在执行阶段只完成累加器内容取反, 结果送累加器的操作, 记作

$$\overline{ACC} \rightarrow ACC$$

(3) 算术右移一位指令 SHR

该指令在执行阶段只完成累加器内容算术右移一位的操作, 记作

$$L(ACC) \rightarrow R(ACC), ACC_0 \rightarrow ACC_0 \text{ (ACC 的符号位不变)}$$

(4) 循环左移一位指令 CSL

该指令在执行阶段只完成累加器内容循环左移一位的操作, 记作

$$R(ACC) \rightarrow L(ACC), ACC_0 \rightarrow ACC_n \text{ (或 } p^{-1} \text{ (ACC))}$$

(5) 停机指令 STP

计算机中有一运行标志触发器 G, 当 $G=1$ 时, 表示机器运行; 当 $G=0$ 时, 表示停机。STP 指令在执行阶段只需将运行标志触发器置“0”, 记作 $0 \rightarrow G$ 。

2. 访存指令

这类指令在执行阶段都需访问存储器。为简单起见, 这里只考虑直接寻址的情况, 不考虑其他寻址方式。

(1) 加法指令 ADD X

该指令在执行阶段需完成累加器内容与对应于主存 X 地址单元的内容相加, 结果送累加器的操作, 具体为:

① 将指令的地址码部分送至存储器地址寄存器, 记作 $Ad(IR) \rightarrow MAR$;

② 向主存发读命令, 启动主存作读操作, 记作 $1 \rightarrow R$;

③ 将 MAR (通过地址总线) 所指的主存单元中的内容 (操作数) 经数据总线读至 MDR 内, 记作 $M(MAR) \rightarrow MDR$;

④ 给 ALU 发加命令, 将 ACC 的内容和 MDR 的内容相加, 结果存于 ACC, 记作 $(ACC) + (MDR) \rightarrow ACC$ 。

当然, 也有的加指令指定两个寄存器的内容相加, 如 $ADD\ AX, BX$, 该指令在执行阶段无需访存, 只需完成 $(AX) + (BX) \rightarrow AX$ 的操作。

(2) 存数指令 $STA\ X$

该指令在执行阶段需将累加器 ACC 的内容存于主存的 X 地址单元中, 具体操作如下:

① 将指令的地址码部分送至存储器地址寄存器, 记作 $Ad(IR) \rightarrow MAR$;

② 向主存发写命令, 启动主存作写操作, 记作 $1 \rightarrow W$;

③ 将累加器内容送至 MDR, 记作 $ACC \rightarrow MDR$;

④ 将 MDR 的内容 (通过数据总线) 写入到 MAR (通过地址总线) 所指的主存单元中, 记作 $MDR \rightarrow M(MAR)$ 。

(3) 取数指令 $LDA\ X$

该指令在执行阶段需将主存 X 地址单元的内容取至累加器 ACC 中, 具体操作如下:

① 将指令的地址码部分送至存储器地址寄存器, 记作 $Ad(IR) \rightarrow MAR$;

② 向主存发读命令, 启动主存作读操作, 记作 $1 \rightarrow R$;

③ 将 MAR (通过地址总线) 所指的主存单元中的内容 (操作数) 经数据总线读至 MDR 内, 记作 $M(MAR) \rightarrow MDR$;

④ 将 MDR 的内容送至 ACC, 记作 $MDR \rightarrow ACC$ 。

3. 转移类指令

这类指令在执行阶段也不访问存储器。

(1) 无条件转移指令 $JMP\ X$

该指令在执行阶段完成将指令的地址码部分 X 送至 PC 的操作, 记作 $Ad(IR) \rightarrow PC$ 。

(2) 条件转移 (负则转) 指令 $BAN\ X$

该指令根据上一条指令运行的结果决定下一条指令的地址, 若结果为负 (累加器最高位为 1, 即 $A_0=1$), 则指令的地址码送至 PC, 否则程序按原顺序执行。由于在取指阶段已完成了 $(PC) + 1 \rightarrow PC$, 所以当累加器结果不为负

(即 $A_0=0$) 时, 就按取指阶段形成的 PC 执行, 记作 $A_0 Ad(IR) + \bar{A}_0 (PC) \rightarrow PC$ 。

由此可见, 不同指令在执行阶段所完成的操作是不同的。如果将访存指令分为直接访存和间接访存两种, 则上述三类指令的指令周期如图 9.1 所示。

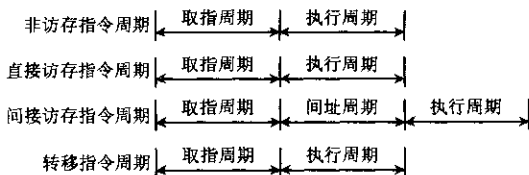


图 9.1 三类指令的指令周期

9.1.4 中断周期

在执行周期结束时刻，CPU 要查询是否有允许中断的中断事件发生，如果有则进入中断周期。由 8.4.4 可知，在中断周期，由中断隐指令自动完成保护断点、寻找中断服务程序入口地址以及硬件关中断的操作。假设程序断点存至主存的 0 地址单元，且采用硬件向量法寻找入口地址，则在中断周期需完成如下操作：

- (1) 将特定地址“0”送至存储器地址寄存器，记作 $0 \rightarrow \text{MAR}$ ；
- (2) 向主存发写命令，启动存储器作写操作，记作 $1 \rightarrow W$ ；
- (3) 将 PC 的内容（程序断点）送至 MDR，记作 $\text{PC} \rightarrow \text{MDR}$ ；
- (4) 将 MDR 的内容（程序断点）通过数据总线写入到 MAR（通过地址总线）所指示的主存单元（0 地址单元）中，记作 $\text{MDR} \rightarrow \text{M}(\text{MAR})$ ；
- (5) 向量地址形成部件的输出送至 PC，记作 向量地址 $\rightarrow \text{PC}$ ，为下一条指令周期作准备；
- (6) 关中断，将允许中断触发器清 0，记作 $0 \rightarrow \text{EINT}$ （该操作可直接由硬件线路完成，参见图 8.27）。

如果程序断点存入堆栈，只须将上述（1）改为堆栈指针 $\text{SP} \rightarrow \text{MAR}$ 。

上述所有操作都是在控制单元发出的控制信号（即微操作命令）控制下完成的。

9.2 控制单元的功能

9.2.1 控制单元的外特性

图 9.2 是反映控制单元外特性的框图。

1. 输入信号

(1) 时钟

上述各种操作有两点应特别注意：

- 完成每个操作都需占用一定时间；

- 各个操作是有先后顺序的。例如存储器读操作要用到 MAR 中的地址，故 $PC \rightarrow MAR$ 应先于 $M(MAR) \rightarrow MDR$ 。

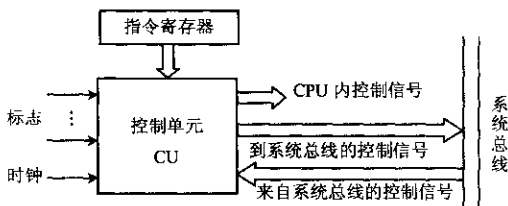


图 9.2 控制单元外特性

为了使控制单元按一定的先后顺序、按一定的节奏发出各个控制信号，CU 必须受时钟控制，即每一个时钟脉冲使控制单元发一个操作命令，或发一组需同时执行的操作命令。

（2）指令寄存器

现行指令的操作码决定了不同指令在执行周期所需完成的不同操作，故指令的操作码字段是 CU 的输入信号，它与时钟配合可产生不同的控制信号。

（3）标志

控制单元有时需依赖 CPU 当前所处的状态（如 ALU 操作的结果）产生控制信号，如 BAN 指令，控制单元要根据上条指令的结果是否为负而产生不同的控制信号。因此“标志”也是 CU 的输入信号。

（4）来自控制总线的控制信号，如中断请求、DMA 请求。

2. 输出信号

（1）CPU 内的控制信号，主要用于 CPU 内的寄存器之间的传送和控制 ALU 实现不同的操作。

（2）送至控制总线的信号，如命令主存或 I/O 读/写，中断响应等。

9.2.2 控制信号举例

控制单元的主要功能就是能发出各种不同的控制信号。下面以间接寻址的加法指令 $ADD @X$ 为例，进一步理解控制信号在完成一条指令的过程中所起的作用。

1. 不采用 CPU 内部总线的方式

图 9.3 示意了未采用 CPU 内部总线方式的数据通路和控制信号的关系。图中未画出每个寄存器的输入或输出控制门，但标出了控制这些门电路的控制

信号 C_i ，考虑到从存储器取出的指令或有效地址都先送至 MDR 再送至 IR，故这里省去了 IR 送至 MAR 的数据通路，凡是需从 IR 送至 MAR 的操作均可由 MDR 送至 MAR 代替。

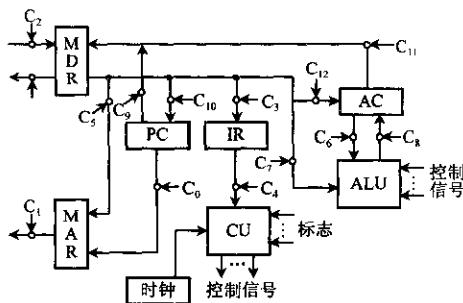


图 9.3 未采用 CPU 内部总线方式的数据通路和控制信号

(1) 取指周期

- ① 控制信号 C_0 有效，打开 PC 送往 MAR 的控制门；
- ② 控制信号 C_1 有效，打开 MAR 送往地址总线的输出门；
- ③ 通过控制总线向主存发读命令；
- ④ C_2 有效，打开数据总线送至 MDR 的输入门；
- ⑤ C_3 有效，打开 MDR 和 IR 之间的控制门，至此指令送至 IR；
- ⑥ C_4 有效，打开指令操作码送至 CU 的输出门。CU 在操作码和时钟的控制下，可产生各种控制信号；
- ⑦ 使 PC 内容加 1 (图中未标出)。

(2) 间址周期

- ① C_5 有效，打开 MDR 和 MAR 之间的控制门，将指令的形式地址送至 MAR；
- ② C_1 有效，打开 MAR 送往地址总线的输出门；
- ③ 通过控制总线向主存发读命令；
- ④ C_2 有效，打开数据总线送至 MDR 的输入门，至此，有效地址存入 MDR；
- ⑤ C_3 有效，打开 MDR 和 IR 之间的控制门，将有效地址送至 IR 的地址码字段。

(3) 执行周期

- ① C_5 有效，打开 MDR 和 MAR 之间的控制门，将有效地址送至 MAR；

- ② C_1 有效, 打开 MAR 送往地址总线的输出门;
 - ③ 通过控制总线向主存发读命令;
 - ④ C_2 有效, 打开数据总线送至 MDR 的输入门, 至此, 操作数存入 MDR;
 - ⑤ C_6 、 C_7 同时有效, 打开 AC 和 MDR 通往 ALU 的控制门;
 - ⑥ 通过 CPU 内部控制总线对 ALU 发“ADD”加控制信号, 完成 AC 的内容和 MDR 的内容相加;
 - ⑦ C_8 有效, 打开 ALU 通往 AC 的控制门, 至此将求和结果存入 AC。
- 图中 C_9 和 C_{10} 分别是控制 PC 的输出和输入的控制信号, C_{11} 和 C_{12} 分别是控制 AC 的输出和输入的控制信号。

2. 采用 CPU 内部总线的方式

图 9.4 示意了采用 CPU 内部总线方式的数据通路和控制信号的关系, 图中每一个小圈处都有一个控制信号, 它控制寄存器到总线或总线到寄存器之间的传送。如 IR_i 表示控制从内部总线到指令寄存器的输入控制门; PC_o 表示控制从程序计数器到内部总线的输出控制门。下标为 i 表示输入控制, 下标为 o 表示输出控制, 以此类推。与图 9.3 相比, 图 9.4 多了两个寄存器 Y 和 Z, 这是由于 ALU 是一个组合逻辑电路, 在其运算过程中必须保持两个输入端不变, 其中一个输入可以从 Y 寄存器中获得, 另一个输入可以从内部总线上获得。当 CPU 内有多个通用寄存器时, 由于设置了寄存器 Y, 可实现任意两个寄存器之间的算逻运算。此外, ALU 的输出不能直接与内部总线相连, 因为其输出又会通过总线反馈到 ALU 的输入, 影响运算的正确性, 故用寄存器 Z 暂存运算结果, 再根据需要送至指定的目标。

下面仍以完成间接寻址的加法指令 ADD @X 为例, 分析控制单元发出的控制信号。

(1) 取指周期

- ① PC_o 和 MAR_i 有效, 完成 PC 经内部总线送至 MAR 的操作, 即 $PC \rightarrow MAR$;
- ② 通过控制总线 (图中未画出) 向主存发读命令, 即 $1 \rightarrow R$;
- ③ 存储器通过数据总线将 MAR 所指单元的内容 (指令) 送至 MDR;
- ④ MDR_o 和 IR_i 有效, 将 MDR 的内容送至 IR, 即 $MDR \rightarrow IR$, 至此, 指令送至 IR, 其操作码字段开始控制 CU;
- ⑤ 使 PC 内容加 1 (图中未标出)。

(2) 间址周期

- ① MDR_o 和 MAR_i 有效, 将指令的形式地址经内部总线送至 MAR, 即 $MDR \rightarrow MAR$;
- ② 通过控制总线向主存发读命令, 即 $1 \rightarrow R$;

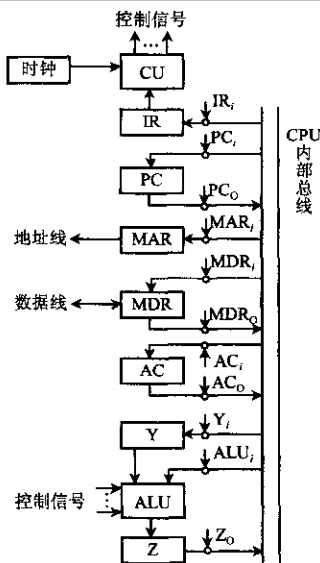


图 9.4 采用 CPU 内部总线方式的数据通路和控制信号

- ③ 存储器通过数据总线将 MAR 所指单元的内容（有效地址）送至 MDR；
 ④ MDR_o 和 IR_i 有效，将 MDR 中的有效地址送至 IR 的地址码字段，即 MDR→Ad(IR)。

（3）执行周期

- ① MDR_o 和 MAR_i 有效，将有效地址经内部总线送至 MAR，即 MDR→MAR；
 ② 通过控制总线向主存发读命令，即 1→R；
 ③ 存储器通过数据总线将 MAR 所指单元的内容（操作数）送至 MDR；
 ④ MDR_o 和 Y_i 有效，将操作数送至 Y，即 MDR→Y；
 ⑤ AC_o 和 ALU_i 有效，同时 CU 向 ALU 发“ADD”加控制信号，使 AC 的内容和 Y 的内容相加，结果送寄存器 Z，即 (AC)+(Y)→Z；
 ⑥ Z_o 和 AC_i 有效，将运算结果存入 AC，即 Z→AC。

现代计算机的 CPU 都集成在一个硅片内，在芯片内采用内部总线的方式可大大节省芯片内部寄存器之间的连线，使芯片内各部件布局更合理。

9.2.3 多级时序系统

1. 机器周期

机器周期可看作是所有指令执行过程中的一个基准时间，机器周期取决于指令的功能及器件的速度。确定机器周期时，通常要分析机器指令的执行步骤及每一步骤所需的时间。例如，取数、存数指令能反映存储器的速度及其与 CPU 的配合情况；加法指令能反映 ALU 的速度；条件转移指令因为要根据上一条指令的执行结果，经测试后才能决定是否转移，所需的时间较长。总之，通过对机器指令执行步骤的分析，会找到一个基准时间，在这个基准时间内，所有指令的操作都能结束。若以这个基准时间定为机器周期，显然不是最合理的。因为只有以完成复杂指令功能所需的时间（最长时间）作为基准，才能保证所有指令在这时间内完成全部操作，这对简单指令来说，显然是一种浪费。

进一步分析发现，机器内的各种操作大致可归属为对 CPU 内部的操作和对主存的操作两大类，由于 CPU 内部的操作速度较快，CPU 访存的操作时间较长，因此通常以访问一次存储器的时间定为基准时间较为合理，这基准时间就是机器周期。又由于不论执行什么指令，都需访问存储器取出指令，因此在存储字长等于指令字长的前提下，取指周期也可看作机器周期。

2. 时钟周期（节拍、状态）

在一个机器周期里可完成若干个微操作，每个微操作都需一定的时间，可用时钟信号来控制产生每一个微操作命令（如图 9.3 和图 9.4 中的 C_i ）。这样，一个机器周期内就包含了若干个时钟周期，又称节拍或状态。每个节拍的宽度正好对应一个时钟周期。在每个节拍内机器可完成一个或几个需同时执行的操作。

3. 多级时序系统

图 9.5 反映了指令周期、机器周期、节拍（状态）和时钟周期的关系。由图可见，一个指令周期包含若干个机器周期，一个机器周期又包含若干个时钟周期（节拍），每个指令周期内的机器周期数可以不等，每个机器周期内的节拍数也可以不等。其中（a）为定长的机器周期，每个机器周期包含 4 个节拍（4 个 T_i ）；（b）为不定长的机器周期，每个机器周期包含的节拍数可以为 4 个也可以为 3 个，这种情况适合于操作比较简单的指令，它可跳过某些时钟周期（如 T_3 ），从而缩短指令周期。

机器周期、节拍（状态）组成了多级时序系统。

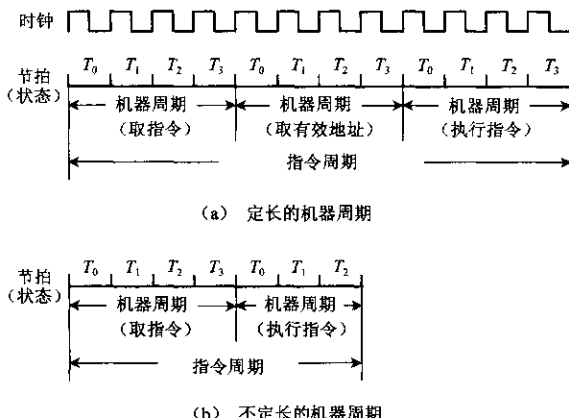


图 9.5 指令周期、机器周期、节拍和时钟周期的关系

9.2.4 控制方式

控制单元控制一条指令执行的过程，实质上是依次执行一个确定的微操作序列的过程。由于不同指令所对应的微操作数及其复杂程度不同，因此每条指令和每个微操作所需的执行时间也不同。通常将如何形成控制不同微操作序列所采用的时序控制方式称作 CU 的控制方式。常见的控制方式有同步控制、异步控制、联合控制和人工控制四种。

1. 同步控制方式

任何一条指令或指令中的任何一个微操作的执行，都由事先确定且有统一基准时标的时序信号所控制的方式，叫做同步控制方式。

图 9.5 (a) 就是一种典型的同步控制方式，每个机器周期都包含 4 个节拍。如果机器内的存储器其存取周期不统一，那么只有把最长的存取周期作为机器周期，才能采用同步控制，否则取指令和取数时间不同，无法用统一的基准。又比如有些访存的指令，执行周期的微操作较少，无须 4 个节拍。因此，为了提高 CPU 的效率，在同步控制中又有三种方案。

(1) 采用完全统一的机器周期和节拍

这种方案的特点是：不论指令所对应的微操作序列有多长，也不管微操作的简繁，一律以最长的微操作序列和最繁的微操作作为标准，采取完全统一的、具有相同时间间隔和相同数目的节拍作为机器周期来运行各种不同的指令。显然，这种方案对于微操作序列较短的指令来说，会造成时间上的浪费。

(2) 采用不同节拍的机器周期

这种方案每个机器周期内的节拍数可以不等,图 9.5 (b) 就是其中一例。这种控制方式可解决微操作执行时间不统一的问题。通常把大多数微操作安排在一个较短的机器周期内完成,而对某些复杂的微操作,采用延长机器周期或增加节拍的办法来解决,如图 9.6 所示。

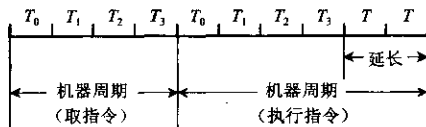


图 9.6 延长机器周期示意

(3) 采用中央控制和局部控制相结合的方法

这种方案将机器的大部分指令安排在统一的、较短的机器周期内完成,称为中央控制,而将少数操作复杂的指令中的某些操作采用局部控制方式来完成,如乘除法和浮点运算等。图 9.7 示意了中央控制和局部控制的时序关系。

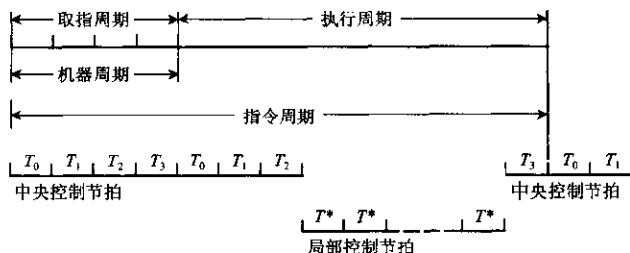


图 9.7 中央控制和局部控制的时序关系

在设计局部控制线路时需注意两点:其一,使局部控制的每一个节拍 T^* 的宽度与中央控制的节拍宽度相同;其二,将局部控制节拍作为中央控制中机器节拍的延续,插入到中央控制的执行周期内,使机器以同样的节奏工作,保证了局部控制和中央控制的同步。 T^* 的多少可根据情况而定,对于乘法,当操作数位数固定后, T^* 的个数也就确定了。而对于浮点运算的对阶操作,由于移位次数不是一个固定值,因此 T^* 的个数不能事先确定。

以乘法指令为例,第一个机器周期采用中央控制的节拍控制取指令操作,接着仍用中央控制的 T_0 、 T_1 、 T_2 节拍去完成将操作数从存储器中取出并送

至寄存器的操作，然后转局部控制，用局部控制节拍 T^* 完成重复加和移位的操作。

2. 异步控制方式

异步控制方式不存在基准时钟信号，没有固定的周期节拍和严格的时钟同步，执行每条指令和每个操作需要多少时间就占用多少时间。这种方式微操作的时序由专门的应答线路控制，即当 CU 发出执行某一微操作的控制信号后，等待执行部件完成了该操作后发回“回答”（或“结束”）信号，再开始新的微操作，使 CPU 没有空闲状态，但因需要采用各种应答电路，故其结构比同步控制方式复杂。

3. 联合控制方式

同步控制和异步控制相结合就是联合控制方式。这种方式对各种不同指令的微操作实行大部分统一、小部分区别对待的办法。例如，对每条指令都有的取指令操作，采用同步方式控制；对那些时间难以确定的微操作，如 I/O 操作，则采用异步控制，以执行部件送回的“回答”信号作为本次微操作的结束。

4. 人工控制方式

人工控制是为了调机和软件开发的需要，在机器面板或内部设置一些开关或按键，来达到人工控制的目的。

（1）Reset（复位）键

按下 Reset 键，使计算机处于初始状态。当机器出现死锁状态或无法继续运行时，可按此键。若在机器运行时按此键，将会破坏机器内某些状态而引起错误，因此要慎用。有些微机未设此键，当机器死锁时，可采用停电后再加电的办法重新启动计算机。

（2）连续或单条执行转换开关

由于调机的需要，有时需观察执行完一条指令后的机器状态，有时又需要观察连续运行程序后的结果，设置连续或单条执行转换开关，能为用户提供这两种选择。

（3）符合停机开关

有些计算机还配有符合停机开关，这组开关指示存储器的位置，当程序运行到与开关指示的地址相符时，机器便停止运行，称为符合停机。

9.2.5 多级时序系统实例分析

为了加深对本章内容的理解，下面以 Intel 8085 为例，通过对一条 I/O 写操作指令运行过程的分析，使读者进一步认识多级时序系统与控制单元发出的控制信号的关系。

1. Intel 8085 的组成

图 9.8 是 Intel 8085 的组成框图。其内部有三个 16 位寄存器：SP、PC 和增减地址寄存器 IDAL；11 个 8 位寄存器：B、C、D、E、H、L、IR、AC、暂存器 TR 以及地址缓冲寄存器 ABR 和地址数据缓冲寄存器 ADBR；一个 5 位的状态标志寄存器 FR。ALU 能实现 8 位算术运算和逻辑运算。控制单元的具体组成将在第十章讲述，图中的定时和控制能对外发出各种控制信号。8085 内还有中断控制和 I/O 控制，内部数据总线为 8 位。图中未标出 8085 片内的控制信号。

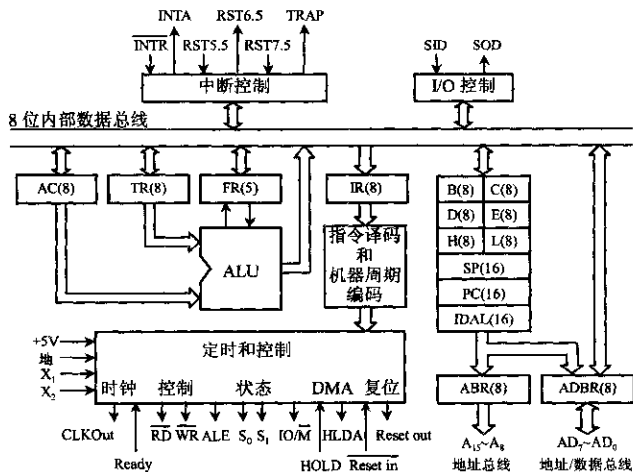


图 9.8 Intel 8085 的组成框图

2. Intel 8085 的外部信号

8085 芯片引脚图如图 9.9 所示，共 40 个引出头。外部信号分以下几类。

(1) 地址和数据信号

- $A_{15} \sim A_8$ (出)，16 位地址的高 8 位；
- $AD_7 \sim AD_0$ (入/出)，16 位地址的低 8 位或 8 位数据，它们共用相同的管脚；
- SID (入)，串行输入；
- SOD (出)，串行输出。

- INTA (出), 中断响应信号。

(5) CPU 初始化

- $\overline{\text{Reset in}}$ (入), 清“0”PC, 假设 CPU 从 0 地址开始执行;
- Reset out (出), 对 CPU 的置“0”作出响应, 该信号能用于重置系统的剩余部分。

(6) 电源和地

- V_{cc} , +5V 电源;
- V_{ss} , 地。

3. 机器周期和节拍(状态)与控制信号的关系

8085 一条指令可分成 1~5 个机器周期, 每个机器周期内又包含 3~5 个节拍, 每个节拍持续一个时钟周期。在每个节拍内, CPU 根据控制信号执行一个或一组同步的微操作。下面分析一条输出指令, 其功能是将 AC 的内容写入到所选择的设备中, 执行该指令的时序图如图 9.10 所示。

由图可见, 该指令的指令周期包含 3 个机器周期 M_1 、 M_2 和 M_3 , 每个机器周期内所包含的节拍如下:

M_1 : 节拍 1 为取指令, 节拍 2 为对 PC 加 1, 节拍 3 为对 PC 加 1 后的操作。
 M_2 : 节拍 1 为对 PC 加 1, 节拍 2 为对 PC 加 1 后的操作, 节拍 3 为对 PC 加 1 后的操作。
 M_3 : 节拍 1 为对 PC 加 1, 节拍 2 为对 PC 加 1 后的操作, 节拍 3 为对 PC 加 1 后的操作。

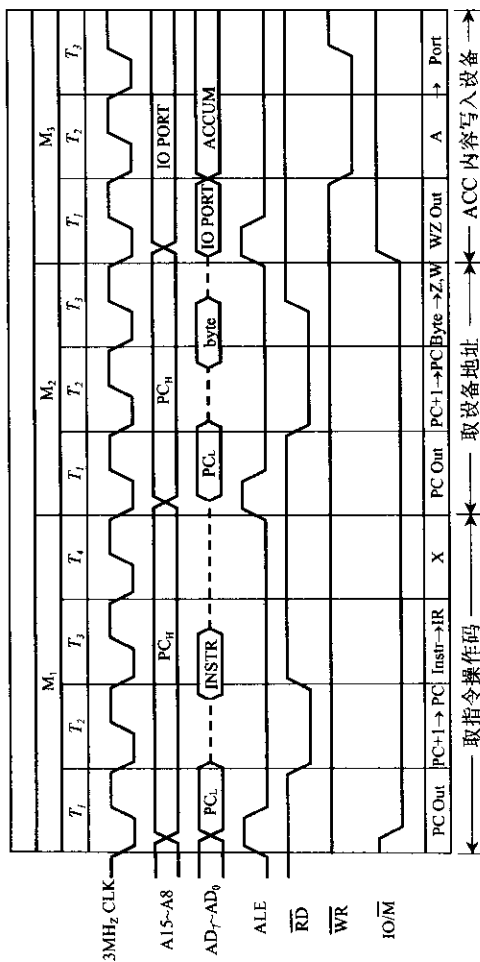


图 9.10 8085 输出指令时序图

(3) 第三个机器周期 M_3 , I/O 写。

① T_1 状态, $\overline{IO/\overline{M}}$ 高电平, 表示 I/O 操作, CU 将 I/O 口地址送至 $A_{15} \sim A_8$ 和 $AD_7 \sim D_0$, 并由 ALE 下降沿激活 I/O 保存地址。

② T_2 状态, $\overline{WR}$ (低) 有效, 表示 I/O 写操作, AC 的内容通过 $AD_7 \sim AD_0$ 数据总线送至被选中的设备中。

可见, 控制单元的每一个控制信号都是在指定机器周期内的指定 T 时刻发出的, 反映了多级时序系统与控制信号间的关系。

思考题与习题

1. 设 CPU 内有下列部件: PC、IR、MAR、MDR、AC、CU。

(1) 写出取指周期的全部微操作;

(2) 写出加法指令 ADD X, 取数指令 LDA X, 存数指令 STA X 在执行阶段所需的全部微操作;

(3) 当上述指令为间接寻址时, 写出运行这些指令所需的全部微操作;

(4) 写出无条件转移指令 JMP Y 和结果为零则转指令 BAZ Y 在执行阶段所需的全部微操作。

2. 控制单元的功能是什么? 其输入受什么控制?

3. 什么是指令周期、机器周期和时钟周期? 三者有何关系?

4. 能不能说机器的主频越快, 机器的速度就越快, 为什么?

5. 设机器 A 的主频为 8MHz, 机器周期含 4 个时钟周期, 且该机的平均指令执行速度是 0.4MIPS, 试求该机的平均指令周期和机器周期, 每个指令周期中含几个机器周期? 如果机器 B 的主频为 12MHz, 且机器周期也含 4 个时钟周期, 试问 B 机的平均指令执行速度为多少 MIPS?

6. 设某机主频为 8MHz, 每个机器周期平均含 2 个时钟周期, 每条指令平均有 2.5 个机器周期, 试问该机的平均指令执行速度为多 MIPS? 若机器主频不变, 但每个机器周期平均含 4 个时钟周期, 每条指令平均有 5 个机器周期, 则该机的平均指令执行速度又是多 MIPS? 由此可得出什么结论?

7. 某 CPU 的主频为 8MHz, 若已知每个机器周期平均包含 4 个时钟周期, 该机的平均指令执行速度为 0.8MIPS, 试求该机的平均指令周期及每个指令周期含几个机器周期? 若改用时钟周期为 $0.4\mu s$ 的 CPU 芯片, 则计算机的平均指令执行速度为多少 MIPS? 若要得到平均每秒 40 万次的指令执行速度, 则应采用主频为多少的 CPU 芯片?

8. 某计算机的主频为 4MHz, 各类指令的平均执行时间和使用频度如下表所示, 试计算该机的速度 (单位用 MIPS 表示), 若上述 CPU 芯片升档为 6MHz, 则该机的速度又为多少?

指令类别	存取	加、减、比较、转移	乘除	其他
平均指令执行时间	0.6 μ s	0.8 μ s	10 μ s	1.4 μ s
使用频度	35%	50%	5%	10%

9. 试比较同步控制、异步控制和联合控制的区别。

10. 什么是典型的同步控制？为了提高 CPU 的效率，在同步控制方式中又有哪些方式？以 8085 的输出指令为例，说明它属哪种控制方式？

11. 设 CPU 内部结构如图 9.4 所示，此外还设有 B、C、D、E、H、L 六个寄存器，它们各自的输入和输出端都与内部总线相通，并分别受控制信号控制（如 B_i 为寄存器 B 的输入控制； B_o 为寄存器 B 的输出控制）。要求从取指令开始，写出完成下列指令所需的控制信号。

(1) ADD B,C $((B) + (C) \rightarrow B)$

(2) SUB A,H $((AC) - (H) \rightarrow AC)$

12. CPU 结构同上题，写出完成下列指令所需的控制信号（包括取指令）。

(1) 寄存器间址的无条件转移指令 JMP @B;

(2) 间接寻址的存数指令 STA @X。

第十章 控制单元的设计

本章以十条机器指令为例，介绍控制单元的两种设计方法，旨在使读者初步掌握设计控制单元的思路，为今后设计计算机打下初步基础。

10.1 组合逻辑设计

10.1.1 组合逻辑控制单元框图

图 9.2 示出了控制单元的外特性，其中指令的操作码是决定控制单元发出不同控制信号的关键。为了简化控制单元的逻辑，将存放在 IR 的 n 位操作码经过一个译码电路产生 2^n 个输出，这样，每对应一种操作码便有一个输出送至 CU。当然，若指令的操作码长度可变，指令译码线路将更复杂。

控制单元的时钟输入实际上是一个脉冲序列，其频率即为机器的主频，它使 CU 能按一定的节拍（ T ）发出各种控制信号。节拍的宽度应满足数据信息通过数据总线从源到目的所需的时间。以时钟为计数脉冲，通过一个计数器，又称节拍发生器，便可产生一个与时钟周期等宽的节拍序列。如果将指令译码和节拍发生器从 CU 中分离出来，便可得简化的控制单元框图，如图 10.1 所示。

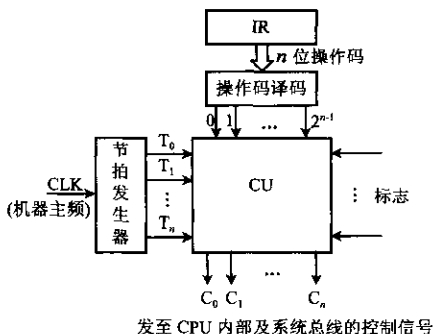


图 10.1 带译码和节拍输入的控制单元框图

10.1.2 微操作的节拍安排

假设机器采用同步控制，每个机器周期包含 3 个节拍，而且 CPU 内部结构如图 9.3 所示，其中 MAR 和 MDR 分别直接和地址总线与数据总线相连，并假设 IR 的地址码部分与 MAR 之间有通路。

安排微操作节拍时应注意三点：

第一，有些微操作的次序是不容改变的，故安排微操作节拍时必须注意微操作的先后顺序。

第二，凡是被控制对象不同的微操作，若能在一个节拍内执行，应尽可能安排在同一个节拍内，以节省时间。

第三，如果有些微操作所占的时间不长，应该将它们安排在一个节拍内完成，并且允许这些微操作有先后次序。

按上述三条原则，以 9.1 所分析的 10 条指令为例，其微操作的节拍安排如下：

1. 取指周期微操作的节拍安排

- ① 根据原则二， T_0 节拍可安排两个微操作： $PC \rightarrow MAR$ ， $1 \rightarrow R$ ；
- ② 根据原则二， T_1 节拍可安排 $M(MAR) \rightarrow MDR$ 和 $(PC) + 1 \rightarrow PC$ 两个微操作。
- ③ T_2 节拍可安排 $MDR \rightarrow IR$ ，考虑到指令译码时间较短，根据原则三，可将指令译码 $OP(IR) \rightarrow ID$ 也安排 T_2 节拍内；

实际上 $(PC) + 1 \rightarrow PC$ 操作也可安排在 T_2 节拍内，因一旦 $PC \rightarrow MAR$ 后，PC 的内容就可修改。

2. 间址周期微操作的节拍安排

T_0 $Ad(IR) \rightarrow MAR$ ， $1 \rightarrow R$

T_1 $M(MAR) \rightarrow MDR$

T_2 $MDR \rightarrow Ad(IR)$

3. 执行周期微操作的节拍安排

• 非访存指令

- ① 清除累加器指令 CLA。

该指令在执行周期只有一个微操作，按同步控制的原则，此操作可安排在 $T_0 \sim T_2$ 的任一节拍内，其余节拍空，如

T_0

T_1

T_2 $0 \rightarrow AC$

② 累加器取反指令 COM。

同理，累加器取反操作可安排在 $T_0 \sim T_2$ 的任一节拍中，即

$$T_0$$

$$T_1$$

$$T_2 \quad \overline{AC} \rightarrow AC$$

③ 算术右移一位指令 SHR。

$$T_0$$

$$T_1$$

$$T_2 \quad L(AC) \rightarrow R(AC), \quad AC_0 \rightarrow AC_0$$

④ 循环左移一位指令 CSL。

$$T_0$$

$$T_1$$

$$T_2 \quad R(AC) \rightarrow L(AC), \quad AC_0 \rightarrow AC_n \text{ (即 } p^{-1}(AC) \text{)}$$

⑤ 停机指令 STP。

$$T_0$$

$$T_1$$

$$T_2 \quad 0 \rightarrow G$$

• 访存指令

⑥ 加法指令 ADD X。

$$T_0 \quad Ad(IR) \rightarrow MAR, \quad 1 \rightarrow R$$

$$T_1 \quad M(MAR) \rightarrow MDR$$

$$T_2 \quad (AC) + (MDR) \rightarrow AC \text{ (该操作实际包括 } (AC) \rightarrow ALU, \\ (MDR) \rightarrow ALU, ALU \rightarrow AC)$$

⑦ 存数指令 STA X。

$$T_0 \quad Ad(IR) \rightarrow MAR, \quad 1 \rightarrow W$$

$$T_1 \quad AC \rightarrow MDR$$

$$T_2 \quad MDR \rightarrow M(MAR)$$

⑧ 取数指令 LDA X。

$$T_0 \quad Ad(IR) \rightarrow MAR, \quad 1 \rightarrow R$$

$$T_1 \quad M(MAR) \rightarrow MDR$$

$$T_2 \quad MDR \rightarrow AC$$

• 转移类指令

⑨ 无条件转移指令 JMP X。

$$T_0$$

$$T_1$$

$$T_2 \text{ Ad(IR)} \rightarrow \text{PC}$$

⑩ 有条件转移（负则转）指令 BAN X。

$$T_0$$

$$T_1$$

$$T_2 \text{ A}_0 \cdot \text{Ad(IR)} + \bar{\text{A}}_0 \cdot (\text{PC}) \rightarrow \text{PC}$$

4. 中断周期微操作的节拍安排

在执行周期的最后时刻，CPU 要向所有中断源发中断查询信号，若检测到某个中断源有请求，并且未被屏蔽又被排队选中，则在允许中断的条件下，CPU 进入中断周期，此时 CPU 由中断隐指令完成下列操作（假设程序断点存入主存 0 号地址单元内）：

$$T_0 \text{ 0} \rightarrow \text{MAR}, \text{1} \rightarrow \text{W}$$

$$T_1 \text{ PC} \rightarrow \text{MDR}$$

$$T_2 \text{ MDR} \rightarrow \text{M(MAR)}, \text{向量地址} \rightarrow \text{PC}$$

此外，由图 8.27 可知，CPU 进入中断周期，由硬件置“0”允许中断触发器 EINT，即关中断。

10.1.3 组合逻辑设计步骤

组合逻辑设计控制单元时，首先根据上述微操作的节拍安排，列出微操作命令的操作时间表，然后写出每一个微操作命令（控制信号）的逻辑表达式，最后根据逻辑表达式画出相应的组合逻辑电路图。

1. 列出微操作命令的操作时间表

表 10.1 列出了上述 10 条机器指令微操作命令的操作时间表。表中 FE、IND 和 EX 为 CPU 工作周期标志（参见图 8.9）， $T_0 \sim T_2$ 为节拍，I 为间址标志，在取指周期的 T_2 时刻，若测得 $I=1$ ，则置“1”IND 触发器，标志进入间址周期；若 $I=0$ ，则置“1”EX 触发器，标志进入执行周期。同理，在间址周期的 T_2 时刻，若测得 $\text{IND}=0$ （表示一次间址），则置“1”EX，进入执行周期；若测得 $\text{IND}=1$ （表示多次间址），则继续间接寻址。在执行周期的 T_2 时刻，CPU 要向所有中断源发中断查询信号，若检测到有中断请求并且满足响应条件，则置“1”INT 触发器，标志进入中断周期，表中未列出中断周期的微操作。表中第一行对应 10 条指令的操作码，代表不同的指令。若某指令有表中所列的微操作命令，其对应的空格内为 1。

2. 写出微操作命令的最简逻辑表达式

纵览表 10.1 便可列出每一个微操作命令的初始逻辑表达式，经化简、整理便可获得能用现成电路实现的微操作命令逻辑表达式。

表 10.1 操作时间表

[illegible]

例如, 根据表可写出 $M(MAR) \rightarrow MDR$ 微命令的逻辑表达式:

$$\begin{aligned} M(MAR) \rightarrow MDR &= FE \cdot T_1 + IND \cdot T_1(ADD+STA+LDA+JMP+BAN)+EX \cdot T_1(ADD+LDA) \\ &= T_1\{FE+IND(ADD+STA+LDA+JMP+BAN)+EX(ADD+LDA)\} \end{aligned}$$

式中 ADD、STA、LDA、JMP、BAN 均来自操作码译码器的输出。

3. 画出微操作命令的逻辑图

对应每一个微操作命令的逻辑表达式都可画出一个逻辑图。如 $M(MAR) \rightarrow MDR$ 的逻辑表达式所对应的逻辑图如图 10.2 所示, 图中未考虑门的扇入系数。

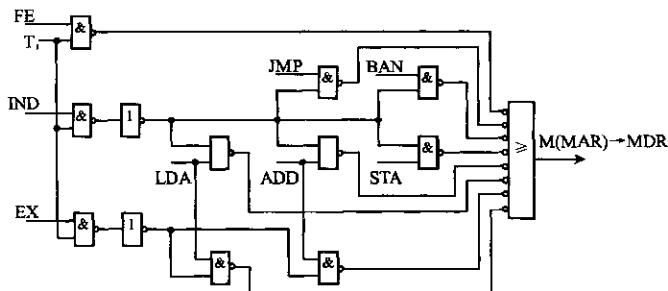


图 10.2 产生 $M(MAR) \rightarrow MDR$ 命令的逻辑图

当然, 在设计逻辑图时要考虑门的扇入系数和逻辑级数。如果采用现成芯片, 还需选择芯片型号。

采用组合逻辑设计方法设计控制单元, 思路清晰, 简单明了, 但因为每一个微操作命令都对应一个逻辑电路, 因此一旦设计完毕便会发现, 这种控制单元的线路结构十分庞杂, 也不规范, 尤如一棵大树, 到处都是不规整的枝叉。而且指令系统功能越全, 微操作命令就越多, 线路也越复杂, 调试就更困难了。为了克服这些缺点, 可采用微程序设计方案。但是, 正如 7.5 章节所述, 随着 RISC 的出现, 组合逻辑设计仍然是设计计算机的一种重要方法。

10.2 微程序设计

10.2.1 微程序设计思想的产生

微程序设计思想是英国剑桥大学教授 M.V.Wilkes 在 1951 年首先提出的。为了克服组合逻辑控制单元线路庞杂的缺点, 他大胆设想采用与存储程序相关

似的办法,来解决微操作命令序列的形成。Wilkes 提出,将一条机器指令编写成一个微程序,每一个微程序包含若干条微指令,每一条微指令对应一个或几个微操作命令。然后把这些微程序存到一个控制存储器中,用寻找用户程序机器指令的办法来寻找每个微程序中的微指令。由于这些微指令是以二进制代码形式表示的,每位代表一个控制信号(若该位为 1,表示该控制信号有效;若该位为 0,表示此控制信号无效),因此,逐条执行每一条微指令,也就相应地完成了一条机器指令的全部操作。可见,微程序控制单元的核心部件是一个控制存储器,简称控存。由于执行一条机器指令必须多次访问控制存储器,以取出多条微指令来控制执行各个微操作,因此要求控存的速度较高。可惜在 Wilkes 那个年代电子器件生产水平有限,因此微程序设计思想并未实现。直到 60 年代出现了半导体存储器,才使这个设计思想成为现实。1964 年 4 月,世界上第一台微程序设计的机器 IBM 360 研制成功。

微程序设计省去了组合逻辑设计过程中对逻辑表达式的化简步骤,无需考虑逻辑门级数和门的扇入系数,使设计更简便。而且由于控制信号是以二进制代码的形式出现的,因此只要修改微指令的代码,就可改变操作内容,便于调试、修改,甚至增删机器指令,有利于计算机仿真。

10.2.2 微程序控制单元框图及工作原理

1. 机器指令对应的微程序

微程序设计控制单元的过程就是编写每一条机器指令的微程序,它是按执行每条机器指令所需的微操作命令的先后顺序而编写的,因此,一条机器指令对应一个微程序,如图 10.3 所示。图中每一条机器指令都与一个以操作性质命名的微程序对应。

由于任何一条机器指令的取指令操作是相同的,因此将取指令操作的命令统一编成一个微程序,这个微程序只负责将指令从主存单元中取出送至指令寄存器中,如图 10.3 所示的取指周期微程序。此外,如果指令是间接寻址,其操作也是可以预测的,也可先编出对应间址周期的微程序。当出现中断时,中断隐指令所需完成的操作可由一个对应中断周期的微程序控制完成。这样,控制存储器中的微程序个数应为机器指令数再加上对应取指、间址和中断周期的三个微程序。

2. 微程序控制单元的基本框图

图 10.4 示意了微程序控制单元的基本组成。

图中虚线框内为微程序控制单元,与图 9.2 相比,它们都有相同的输入,如指令寄存器、各种标志和时钟,输出也是输至 CPU 内部或系统总线的控制信号。

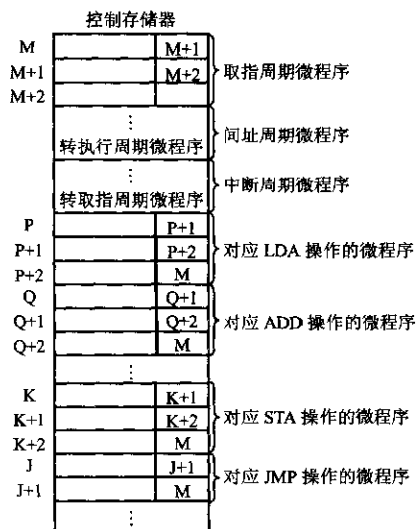


图 10.3 不同机器指令所对应的微程序

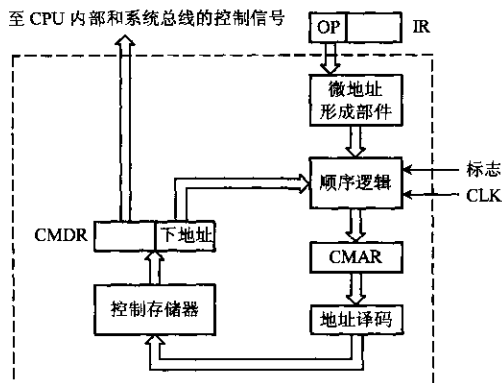


图 10.4 微程序控制单元的基本组成

虚框内的控制存储器（简称控存）是微程序控制单元的核心部件，用来存放全部微程序；CMAR 是控存地址寄存器，用来存放欲读出的微指令地址；

CMDR 是控存数据寄存器，用来存放从控存读出的微指令；顺序逻辑是用来控制微指令序列的，具体就是控制形成下一条微指令（即后继微指令）的地址，其输入与微地址形成部件（与指令寄存器相连）、微指令的下地址字段以及外来的标志有关。有关微指令序列地址的形成将在 10.2.4 章节中介绍。

微指令的基本格式如图 10.5 所示，共分两个字段，一个为操作控制字段，该字段发出各种控制信号；另一个为顺序控制字段，它可指出下条微指令的地址（简称下地址），以控制微指令序列的执行顺序。

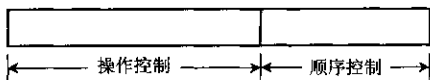


图 10.5 微指令的基本格式

3. 工作原理

假设有一个用户程序如下所示，它存于以 2000H 为首地址的主存空间内。

```
LDA X
ADD Y
STA Z
STP
```

下面结合图 10.3 和图 10.4，分析运行上述程序时，微程序控制单元的工作原理。

首先将用户程序的首地址送至 PC，然后进入取指阶段。

（1）取指阶段

- ① 将取指周期微程序首地址 $M \rightarrow CMAR$ ；
- ② 取微指令，将对应控存 M 地址单元中的第一条微指令读到控存数据寄存器中，记作 $CM(CMAR) \rightarrow CMDR$ ；

③ 产生微操作命令；

第一条微指令的操作控制字段中为“1”的各位发出控制信号，如 $PC \rightarrow MAR$ ， $I \rightarrow R$ ，命令主存接受程序首地址并进行读操作。

④ 形成下一条微指令的地址；

此微指令的顺序控制字段指出了下一条微指令的地址为 $M+1$ ，将 $M+1$ 送至 $CMAR$ 。

⑤ 取下一条微指令；

将对应控存 $M+1$ 地址单元中的第二条微指令读到 $CMDR$ 中，即

$CM(CMAR) \rightarrow CMDR$

⑥ 产生微操作指令；

由第二条微指令的操作控制字段中对应“1”的各位发出控制信号，如 $M(MAR) \rightarrow MDR$ 使对应主存 2000H 地址单元中的第一条机器指令从主存中读出送至 MDR 中。

⑦ 形成下一条微指令的地址；

将第二条微指令下地址字段指出的地址 $M+2$ 送至 CMAR，即

$Ad(CMDR) \rightarrow CMAR$

⋮
⋮

以此类推，直到取出取指周期最后一条微指令，并发出微命令为止，此时第一条机器指令 LDA X 已存至指令寄存器 IR 中。

(2) 执行阶段

① 取数指令微程序首地址的形成；

当取数指令存入 IR 后，其操作码 $OP(IR)$ 直接送到微地址形成部件，该部件的输出即为取数指令微程序的首地址 P，且将 P 送至 CMAR，记作

$OP(IR) \rightarrow CMAR$

② 取微指令；

将对应控存 P 地址单元中的微指令读到 CMDR 中，即

$CM(CMAR) \rightarrow CMDR$

③ 产生微操作命令；

由微指令操作控制字段中对应“1”的各位发出控制信号，如 $Ad(IR) \rightarrow MAR$ ， $I \rightarrow R$ ，命令主存读操作数。

④ 形成下一条微指令的地址；

将此条微指令下地址字段指出的 $P+1$ 送至 CMAR，即

$Ad(CMDR) \rightarrow CMAR$

⑤ 取微指令，即 $CM(CMAR) \rightarrow CMDR$ ；

⑥ 产生微操作命令；

⋮
⋮

以此类推，直到取出取数指令微程序的最后一条微指令 $P+2$ ，并发出微命令，至此即完成了将主存 X 地址单元中的操作数取至累加器 AC 的操作。这条微指令的顺序控制字段为 M，即表明 CPU 又开始进入下一条机器指令的取指周期，控存又要依次读出取指周期微程序的逐条微指令，发出微命令，完成将第二条机器指令 ADD Y 从主存取至指令寄存器 IR 中……微程序控制单元就是这样，通过逐条取出微指令，发出各种微命令，从而实现从主存逐条取出、分析并执行机器指令，以达到运行程序的目的。

由此可见,对微程序控制单元的控存而言,内部信息一旦按所设计的微程序被灌注后,在机器运行过程中,只须具有读出的性能即可,故可采用 ROM。此外,在微程序的执行过程中,关键问题是如何由微指令的操作控制字段形成微操作命令,以及如何形成下一条微指令的地址。这是微程序设计必须解决的问题,它们与微指令的编码方式和微地址的形成方式有关。

10.2.3 微指令的编码方式

微指令的编码方式又叫微指令的控制方式,它是指如何对微指令的控制字段进行编码,以形成控制信号。

1. 直接编码(直接控制)方式

在微指令的操作控制字段中,每一位代表一个微命令,这种编码方式即为直接编码方式。上面所述的用控制字段中的某位为“1”表示控制信号有效(如打开某个控制门),以及某位为“0”表示控制信号无效(如不打开某个控制门)就是直接控制方式,如图 10.6 所示。这种方式含义清晰,但由于机器中微命令甚多,可能使微指令操作控制字段达几百位,造成控存容量极大。

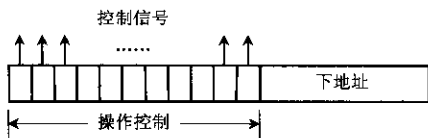


图 10.6 直接编码方式

2. 字段直接编码方式

这种方式就是将微指令的操作控制字段分成若干段,将一组互斥的微命令放在一个字段内,通过对这个字段译码,便可对应每一个微命令,如图 10.7 所示。这种方式因靠字段直接译码发出微命令,故又有显式编码之称。

采用字段直接编码方法可用较少的二进制信息表示较多的微命令信号,例如 3 位二进制代码译码后可表示 7 个互斥的微命令,留出一种状态表示不发微命令,与直接编码用 7 位表示 7 个微命令相比,减少了 4 位,缩短了微指令的长度。但由于增加了译码电路,使微程序的执行速度稍微减慢。

至于操作控制字段应分几段,与需要并行发出的微命令个数有关,若需并行发出 8 个微命令,就可分 8 段。每段的长度可以不等,与具体要求互斥的微命令个数有关,如某类操作要求互斥的微命令仅有 6 个,则字段只需安排 3 位即可。

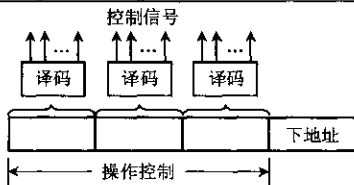


图 10.7 字段直接编码方式

3. 字段间接编码方式

这种方式一个字段的某些微命令还需由另一个字段中的某些微命令来解释，如图 10.8 所示。图中字段 1 译码的某些输出受字段 2 译码输出的控制，由于不是靠字段直接译码发出微命令，故称为字段间接编码，又称隐式编码。

这种方法虽然可以进一步缩短微指令字长，但因削弱了微指令的并行控制能力，因此通常用作字段直接编码法的一种辅助手段。

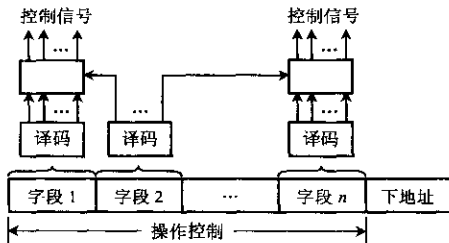


图 10.8 字段间接编码方式

4. 混合编码

这种方法是把直接编码和字段编码（直接或间接）混合使用，以便能综合考虑微指令的字长、灵活性和执行微程序的速度等方面的要求。

5. 其他

微指令中还可设置常数字段，用来提供常数、计数器初值等。常数字段还可以和某些解释位配合，如解释位为 0，表示该字段提供常数；解释位为 1，表示该字段提供某种命令，使微指令更灵活。

此外，微指令还可类似机器指令操作码的方式编码，有关内容参见 10.2.5 微指令的格式。

10.2.4 微指令序列地址的形成

由图 10.4 可见, 后继微指令的地址大致有两种方式形成:

1. 直接由微指令的下地址字段指出

图 10.3 中大部分微指令的下地址字段都直接指出了后继微指令的地址。

2. 根据机器指令的操作码形成

当机器指令取至指令寄存器后, 微指令的地址由操作码经微地址形成部件形成。微地址形成部件实际是一个编码器, 其输入为指令操作码, 输出就是对应该机器指令微程序的首地址。它可采用 PROM 实现, 以指令的操作码作为 PROM 的地址, 而相应的存储单元内容就是对应该指令微程序的首地址。

实际上微指令序列地址的形成方式还有:

3. 增量计数器法

仔细分析发现, 在很多情况下, 后继微指令的地址是连续的, 因此对于顺序地址, 微指令可采用增量计数方法, 即 $(CMAR)+1 \rightarrow CMAR$ 来形成后继微指令的地址。

4. 分支转移

当遇到条件转移指令时, 微指令出现了分支, 必须根据各种标志来决定下一条微指令的地址。微指令的格式为:

操作控制字段	转移方式	转移地址
--------	------	------

其中转移方式是指明判别条件, 转移地址是指明转移成功后的去向, 若不成功则顺序执行。也有的转移微指令中设两个转移地址, 条件满足时选择其中一个转移地址; 条件不满足时选择另一个转移地址。

5. 通过测试网络形成

微指令的地址还可通过测试网络形成, 如图 10.9 所示。图中微指令的地址分两部分, 高段 h 为非测试地址, 由微指令的 H 段地址码直接形成; 低段 l 为测试地址, 由微指令的 L 段地址码通过测试网络形成。

6. 微程序入口地址

当电源加电后, 第一条微指令的地址可由专门的硬件电路产生, 也可由外部直接向 $CMAR$ 输入微指令的地址, 这个地址即为取指周期微程序的入口地址。

当有中断请求时, 若条件满足, CPU 响应中断进入中断周期, 此时需中断现行程序, 转至对应中断周期的微程序。由于设计控制单元时已安排好中断

周期微程序的入口地址（参见图 10.3），故响应中断时，可由硬件产生中断周期微程序的入口地址。

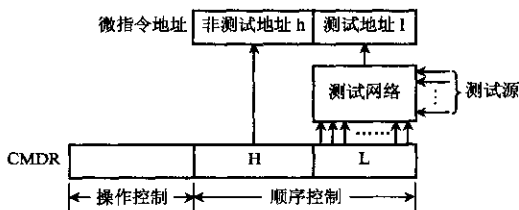


图 10.9 通过测试网络形成微指令地址

同理当出现间接寻址时，也可由硬件产生间址周期微程序的入口地址。

综合上述各种方法，可得出形成后继微指令地址的原理图如图 10.10 所示。

图中多路选择器可选择四路地址：

- $(CMAR) + 1 \rightarrow CMAR$;
- 微指令的下地址字段;
- 指令寄存 (通过微地址形成部件);
- 微程序入口地址。

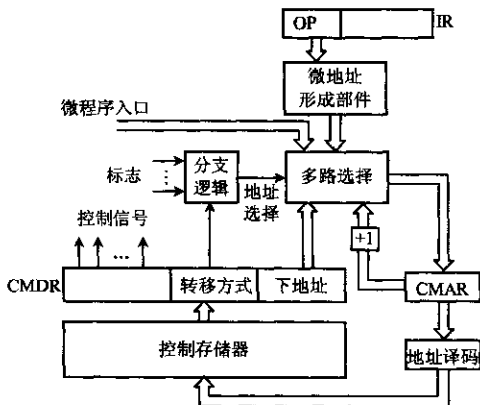


图 10.10 后继微指令地址形成方式的原理图

10.2.5 微指令格式

微指令格式与微指令的编码方式有关，通常分为水平型微指令和垂直型微

指令两种。

1. 水平型微指令

水平型微指令的特点是一次能定义并执行多个并行操作的微命令。图 10.6 就是典型的水平型微指令。从编码方式看,直接编码、字段直接编码、字段间接编码以及直接和字段混合编码都属水平型微指令。其中直接编码速度最快,字段编码要经过译码,故速度受影响。

2. 垂直型微指令

垂直型微指令的特点是采用类似机器指令操作码的方式,在微指令字中,设置微操作码字段,由微操作码规定微指令的功能。通常一条微指令有 1~2 个微命令,控制 1~2 种操作。这种微指令不强调其并行控制功能。

表 10.2 列出了一种垂直型微指令的格式,其中微操作码 3 位,共分六类操作;地址码字段共 10 位,对不同的操作有不同的含义;其他字段 3 位,可协助本条微指令完成其他控制功能。

表 10.2 垂直型微指令示例

微操作码	地址码		其他		微指令类别及功能
012	3~7	8~12	13	15	
000	源寄存器	目的寄存器	其他控制		传送型微指令
001	ALU 左输入	ALU 右输入	ALU		运算控制型微指令。按 ALU 字段所规定的功能执行, 其结果送暂存器
010	寄存器	移位次数	移位方式		移位控制型微指令。按移位方式对寄存器中的数据移位
011	寄存器	存储器	读写	其他	访存微指令。完成存储器和寄存器之间的传送
100	D			S	无条件转移微指令。D 为微指令的目的地址
101	D		测试条件		条件转移微指令。最低 4 位为测试条件
110 111					可定义 I/O 或其他操作。第 3~15 位可根据需要定义各种微命令。

3. 两种微指令格式的比较

- (1) 水平型微指令比垂直型微指令并行操作能力强,效率高,灵活性强。
- (2) 水平型微指令执行一条机器指令所需的微指令数目少,因此速度比垂直型微指令快。
- (3) 水平型微指令用较短的微程序结构换取较长的微指令结构,垂直型

微指令正相反，它以较长的微程序结构换取较短的微指令结构。

(4) 水平型微指令与机器指令差别较大，垂直型微指令与机器指令相似。

10.2.6 静态微程序设计和动态微程序设计

通常指令系统是固定的，对应每一条机器指令的微程序是计算机设计者事先编好的，因此一般微程序无需改变，这种微程序设计技术即称为静态微程序设计，其控存采用 ROM。前面讲述的内容基本上属于这一类。

如果采用 EPROM 作为控制存储器，人们可以通过改变微指令和微程序来改变机器的指令系统，这种微程序设计技术称为动态微程序设计。动态微程序设计由于可以根据需要改变微指令和微程序，因此可以在一台机器上实现不同类型的指令系统，有利于仿真。但是这种设计对用户的要求很高，目前难以推广。

10.2.7 毫微程序设计

微程序可看作是解释机器指令的，毫微程序可看作是解释微程序的，而组成毫微程序的毫微指令则是用来解释微指令的。采用毫微程序设计计算机的优点是用少量的控存空间来达到高度的并行。

毫微程序设计采用两级微程序的设计方法。第一级微程序为垂直型微指令，并行功能不强，但有严格的顺序结构，由它确定后继微指令的地址，当需要时可调用第二级。第二级微程序为水平型微指令，具有很强的并行操作能力，但不包含后继微指令的地址。第二级微程序执行完后又返回到第一级微程序。两级微程序分别放在两级控制存储器内。图 10.11 示意了毫微程序控制存储器的基本组成。

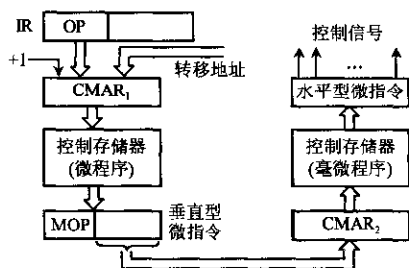


图 10.11 毫微程序控制存储器的基本组成

图中 $CMAR_1$ 为第一级控存地址寄存器， $CMAR_2$ 存放从第一级控存中读

出的微指令, 如果该微指令只产生一些简单的控制信号, 则可以通过译码, 直接形成微操作命令, 不必调用第二级。如果需调用第二级控存时, 则将毫微程序的地址送至 $CMAR_2$, 然后由从第二级控存中读出的微指令去直接控制硬件。值得注意的是垂直型微指令不是和水平型微指令一条一条地对应, 而是由水平型微指令 (称作毫微指令) 组成的毫微程序去执行垂直型微指令的操作。毫微指令与微指令的关系就好比微指令与机器指令的关系一样。

二级控存虽然能减少控存的容量, 但因有时一条微指令要访问两次控存, 影响了速度。

10.2.8 串行微程序控制和并行微程序控制

与机器指令一样, 完成一条微指令也分两个阶段: 取微指令和执行微指令。如果这两个阶段按图 10.12 (a) 所示的方式运行, 则为串行微程序控制。由于取微指令和执行微指令的操作是在两个完全不同的部件中完成的, 因此可将这两部分操作并行进行, 以缩短微指令周期, 这就是并行微程序控制, 如图 10.12 (b) 所示, 与指令二级流水相似。

当采用并行微程序控制时, 为了不影响本条微指令的正确执行, 需增加一个微指令寄存器来暂存下一条微指令。由于执行本条微指令与取下一条微指令是同时进行的, 因此当遇到需要根据本条微指令的处理结果来决定下条微指令的地址时, 就不能并行操作, 此时可延迟一个微指令周期再取微指令。



(a) 串行操作



(b) 并行操作

图 10.12 串行微程序和并行微程序控制方式

10.2.9 微程序设计举例

微程序设计控制单元的主要任务是编写对应各条机器指令的微程序, 具体步骤是首先写出对应机器指令的全部微操作安排, 然后确定微指令格式, 最后编写出每条微指令的二进制代码 (称作微指令码点)。

1. 写出对应机器指令的微操作及节拍安排

为了便于与组合逻辑设计比较,仍以十条机器指令为例,而且 CPU 结构同组合逻辑设计假设相同。此外,为了简化起见,不考虑间接寻址和中断的情况。下面分别按取指阶段和执行阶段列出其微操作序列。

(1) 取指阶段的微操作及节拍安排

取指阶段的微操作基本与组合逻辑控制相同,不同的是指令取至 IR 后,微程序控制需由操作码形成执行阶段微程序的入口地址。即

$$T_0 \quad PC \rightarrow MAR, 1 \rightarrow R$$

$$T_1 \quad M(MAR) \rightarrow MDR, (PC)+1 \rightarrow PC$$

$$T_2 \quad MDR \rightarrow IR, OP(IR) \rightarrow \text{微地址形成部件 (编码器)}$$

如果把一个 T 内的微操作安排在一条微指令中完成,上述微操作对应 3 条微指令。

值得注意的是,由于微程序控制的所有控制信号都来自微指令,而微指令又存于控存中,因此欲完成上述这些微操作,必须先将微指令从控存中读出,也即必须先给出这些微指令的地址。由图 10.3 可见,在取指微程序中,除第一条微指令外,其余微指令的地址均由上一条微指令的下地址字段直接给出,因此上述每一条微指令都需增加一个将微指令下地址字段送至 CMAR 的微操作,记作 $Ad(CMDR) \rightarrow CMAR$,而这一操作只能由下一个时钟周期 T 的上升沿将地址打入到 CMAR 内。至于取指微程序的最后一条微指令,其后继微指令的地址是由微地址形成部件形成的,而且也只能由下一个 T 的上升沿将该地址打入到 CMAR 中,即微地址形成部件 $\rightarrow CMAR$,为了反映该地址与操作码有关,故记作 $OP(IR) \rightarrow CMAR$ 。

综上所述,考虑到需要形成后继微指令的地址,上述分析的取指操作共需六条微指令完成。即:

$$T_0 \quad PC \rightarrow MAR, 1 \rightarrow R$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR$$

$$T_2 \quad M(MAR) \rightarrow MDR, (PC)+1 \rightarrow PC$$

$$T_3 \quad Ad(CMDR) \rightarrow CMAR,$$

$$T_4 \quad MDR \rightarrow IR, OP(IR) \rightarrow \text{微地址形成部件 (编码器)}$$

$$T_5 \quad OP(IR) \rightarrow CMAR$$

所有微指令均由 T 的上升沿打入到 CMDR 中。

(2) 执行阶段的微操作及节拍安排

执行阶段的微操作由操作码性质而定,同时也需考虑后继微指令地址的形成问题。

• 非访存指令

① CLA 指令

与组合逻辑控制一样,该指令在执行阶段只有一个微操作 $0 \rightarrow AC$, 只需一个时钟周期 T , 故对应一条微指令。该微指令的下地址字段应直接给出取指微程序的入口地址, 而且由下一个 T 的上升沿将地址打入到 CMAR 内。这样, 对应 CLA 指令执行阶段的微指令有两条:

$$T_0 \quad 0 \rightarrow AC$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

同理可得其余 4 条非访存指令对应的微操作。

② COM 指令

$$T_0 \quad \overline{AC} \rightarrow AC$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

③ SHR 指令

$$T_0 \quad L(AC) \rightarrow R(AC), AC_0 \rightarrow AC_6$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

④ CSL 指令

$$T_0 \quad R(AC) \rightarrow L(AC), AC_0 \rightarrow AC_n \quad (\text{即 } p^i(AC))$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

⑤ STP 指令

$$T_0 \quad 0 \rightarrow G$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

这里由于安排了 $Ad(CMDR) \rightarrow CMAR$, 使再次启动机器时, 可直接用已存入 CMAR 中的取指微程序的入口地址。

• 访存指令

⑥ ADD 指令

$$T_0 \quad Ad(IR) \rightarrow MAR, 1 \rightarrow R$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR$$

$$T_2 \quad M(MAR) \rightarrow MDR$$

$$T_3 \quad Ad(CMDR) \rightarrow CMAR$$

$$T_4 \quad (AC) + (MDR) \rightarrow AC$$

$$T_5 \quad Ad(CMDR) \rightarrow CMAR, \text{取指微程序入口地址} \rightarrow CMAR$$

⑦ STA 指令

$$T_0 \quad Ad(IR) \rightarrow MAR, 1 \rightarrow W$$

$$T_1 \quad Ad(CMDR) \rightarrow CMAR$$

$$T_2 \quad AC \rightarrow MDR$$

T_3 Ad(CMDR)→CMAR

T_4 MDR→M(MAR)

T_5 Ad(CMDR)→CMAR, 取指微程序入口地址→CMAR

⑧ LDA 指令

T_0 Ad(IR)→MAR, 1→R

T_1 Ad(CMDR)→CMAR

T_2 M(MAR)→MDR

T_3 Ad(CMDR)→CMAR

T_4 MDR→AC

T_5 Ad(CMDR)→CMAR, 取指微程序入口地址→CMAR

• 转移类指令

⑨ JMP 指令

T_0 Ad(IR)→PC

T_1 Ad(CMDR)→CMAR, 取指微程序入口地址→CMAR

⑩ BAN 指令

T_0 $A_0 \cdot \text{Ad(IR)} + \bar{A}_0 \cdot (\text{PC}) \rightarrow \text{PC}$

T_1 Ad(CMDR)→CMAR, 取指微程序入口地址→CMAR

上述全部微操作共 20 个, 微指令共 38 条。

2. 确定微指令格式

微指令的格式包括微指令的编码方式、后继微指令的地址形成方式和微指令字长等三个方面。

(1) 微指令的编码方式

上述微操作数不多, 可采用直接编码方式, 由微指令控制字段的某一位直接控制一个微操作。

(2) 后继微指令的地址形成方式

根据上述分析, 可采用由指令的操作码和微指令的下地址字段两种方式形成后继微指令的地址。

(3) 微指令字长

微指令由操作控制字段和下地址字段两部分组成。根据直接编码方式, 20 个微操作对应 20 位操作控制字段; 根据 38 条微指令, 对应 6 位下地址字段。这样, 微指令字长至少 26 位。

仔细分析发现, 在 38 条微指令中有 19 条微指令是为了控制将后继微指令的地址打入到 CMAR 的操作(其中 18 条微指令地址字段 Ad(CMDR)→CMAR 和 1 条指令操作码 OP(IR)→CMAR), 因此实际上是每两个时钟周期才能取出并执行一条微指令。如果能做到每一个时钟周期取出并执行一条微指令, 将大

大提高微程序控制的速度。

事实上如果将 CMDR 的下地址字段 Ad(CMDR)直接接到控存的地址线上,并由下一个时钟周期的上升沿将该地址单元的内容(微指令)读到 CMDR 中,便能做到在一个时钟周期内读出并执行一条微指令。这就好比将 Ad(CMDR)当作 CMAR 使用。同理也可将指令寄存器的操作码字段 OP(IR)经微地址形成部件形成后继微指令的地址,直接送到控存的地址线上。这两路地址可通过一个多路选择器,根据需要任选一路,如图 10.13 所示。

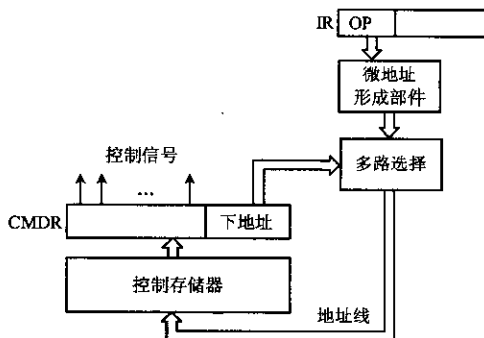


图 10.13 省去了 CMAR 的控制存储器

综上所述,在省去了 19 条微指令的同时也省去了两个微操作(微指令地址字段 Ad(CMDR)→CMAR 和指令操作码 OP(IR)→CMAR),这样,10 条机器指令共对应 18 个微操作和 19 条微指令。为了便于扩充,操作控制字段取 24 位,下地址字段取 6 位,其微指令格式如图 10.14 所示。

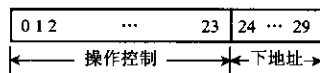


图 10.14 对应 10 条机器指令的微指令格式

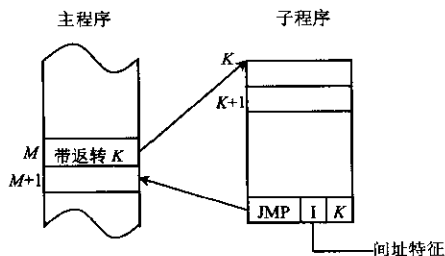
- | | |
|-------------|------------|
| 其中第 0 位表示控制 | PC→MAR 微操作 |
| 第 1 位表示控制 | I→R 微操作 |
| 第 2 位表示控制 | M(MAR)→MDR |
| 第 3 位表示控制 | (PC)+1→PC |
| 第 4 位表示控制 | MDR→IR |

思考题与习题

- 假设响应中断时,要求将程序断点存在堆栈内,并且采用软件办法寻找中断服务程序的入口地址,试写出中断隐指令的微操作及节拍安排。
- 写出完成下列指令的微操作及节拍安排(包括取指操作)。
 - 指令 $\text{ADD } R_i, X$ 完成将 R_i 寄存器的内容和主存 X 单元的内容相加结果存于 R_i 的操作。
 - 指令 $\text{ISZ } X$ 完成将主存 X 单元的内容增 1,并根据其结果若为 0,则跳过下一条指令执行。
- 按序写出下列程序所需的全部微操作命令及节拍安排。

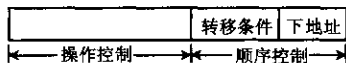
指令地址	指令
200	LDA 206
201	ADD 207
202	BAN 204
203	STA 205
204	STP

- 已知带返转指令的含义如下图所示,写出机器在完成带返转指令时,取指阶段和执行阶段所需的全部微操作及节拍安排。



- 画出组合逻辑控制单元的组成框图,根据指令处理过程,结合有关部件说明其工作原理。
- 画出微程序控制单元的组成框图,根据指令处理过程,结合有关部件说明其工作原理。
- 试比较组合逻辑设计和微程序的设计步骤和硬件组成,说明哪一种控制速度更快,为什么?
- 微指令的操作控制有几种编码方式,各有何特点?哪一种控制速度最快?
- 什么是垂直型微指令?什么是水平型微指令?各有何特点?

10. 能否说水平型微指令就是直接编码的微指令,为什么?
11. 微指令的地址有几种形成方式?各有何特点?
12. 微指令操作控制字段采用直接编码或显式编码时,其微指令字长如何确定?
13. 设控制存储器的容量为 512×48 位,微程序可在整个控存空间实现转移,而控制微程序转移的条件共有 4 个(采用直接控制),微指令格式如下:



试问微指令中的三个字段分别为多少位?

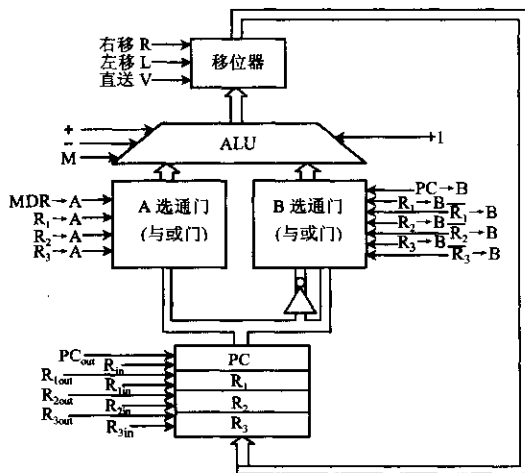
14. 试比较静态微程序设计和动态微程序设计。
15. 解释机器指令、微指令、微程序、毫微指令和毫微程序,它们之间有什么对应关系?
16. 毫微程序设计的特点是什么?与微程序设计相比,其硬件组成有何不同?
17. 假设机器的主要部件有:程序计数器 PC,指令寄存器 IR,通用寄存器 R_0 、 R_1 、 R_2 、 R_3 ,寄存器 C、D, ALU, 移位器,存储器地址寄存器 MAR, 存储器数据寄存器 MDR 及存储矩阵 M。
- (1) 要求采用单总线结构画出包含上述部件的硬件框图,并注明数据流动方向。
- (2) 画出 $\text{ADD } (R_1), (R_2)$ 指令在取指阶段和执行阶段的信息流程图。 R_1 寄存器存放原操作数地址, R_2 寄存器存放目的操作数的地址。
- (3) 写出对应该流程图所需的全部微操作命令。
18. 假设机器的主要部件同 17 题,外加一个控制门 G。
- (1) 要求采用双总线结构(每组总线的的数据流动方向是单向的)画出包含上述部件的硬件框图,并注明数据流动方向。
- (2) 画出 $\text{SUB } R_1, R_3$ 完成 $(R_1) - (R_3) \rightarrow R_1$ 操作的指令周期信息流程图(假设指令地址已放在 PC 中),并列出的微操作控制信号序列。
19. 下表给出 8 条微指令 $I_1 \sim I_8$ 及所包含的微命令控制信号,设计微指令操作控制字段格式,要求所使用的控制位最少,而且保持微指令本身内在的并行性。

微指令	所含的微命令
I_1	a b c d e
I_2	a d f g
I_3	b h
I_4	c
I_5	c e g I
I_6	a h j
I_7	c d h
I_8	a b h

20. 设有一运算器通路如下图所示,假设操作数 a 和 b (均为补码) 分别放在通用寄存器 R_1 和 R_2 中, ALU 有 +、-、M (传送) 三种操作功能,移位器可实现左移、右移和直送功

能。试回答：

- (1) 指出相容性微操作和相斥性微操作；
- (2) 采用字段直接编码方式设计适合于此运算器的微指令格式；
- (3) 画出计算 $1/2(a-b) - R_2$ 的微程序流程图，试问执行周期需用几条微指令？
- (4) 按设计的微指令格式，写出 (3) 要求的微代码。



参 考 文 献

- 1 Stallings W. Computer organization and architecture: designing for performance, 4th ed. 北京: 清华大学出版社, 1997
- 2 唐朔飞. 电子数字计算机原理. 第二版. 哈尔滨: 哈尔滨工业大学出版社, 1990
- 3 袁开榜. 计算机组成原理. 北京: 高等教育出版社, 1994
- 4 王爱英. 计算机组成与结构. 第二版. 北京: 清华大学出版社, 1994
- 5 张基温. 计算机组成原理教程. 北京: 清华大学出版社, 1998
- 6 白中英, 韩兆轩. 计算机组成原理教程. 北京: 科学出版社, 1988
- 7 毕庶本. 64 位和 32 位高档微机系统设计. 济南: 山东科学技术出版社, 1994
- 8 唐朔飞. 计算机组成原理习题集. 北京: 电子工业出版社, 1995
- 9 白中英. 计算机组成原理题库及其实现. 北京: 科学出版社, 1991
- 10 苏东庄. 计算机系统结构. 西安: 西北电讯工程学院出版社, 1984
- 11 柳青, 耿可立. 微型机系统与应用基础教程. 北京: 高等教育出版社, 1998
- 12 高文. 多媒体数据压缩技术. 北京: 电子工业出版社, 1994
- 13 曾建超, 俞志和. 虚拟现实的技术及其应用. 北京: 清华大学出版社, 1996
- 14 张海藩. 软件工程导论 (修订版). 北京: 清华大学出版社, 1992